Statistical Learning

STAT3006 / STAT7305 - Theory, Worked Examples and Reproducible R

A study edition reconstructed from the supplied course materials

2026-10-13

Table of contents

How to use this book **1**
Two pictures . . . 1
A running example: Fisher's iris data . . . 1
Reading code and figures . . . 2
Notation and assumptions . . . 2
Sources and scope . . . 3

I Multivariate statistics **4**

1 Multivariate distributions and the multivariate normal **5**
1.1 Random vectors and joint distributions . . . 5
1.2 Marginal and conditional distributions . . . 6
1.3 Moments . . . 7
1.4 The multivariate normal distribution . . . 9
1.5 Linear transformations, marginals, and conditionals . . . 11
1.6 R laboratory: one species or three? . . . 16
1.7 Further exercises . . . 17

2 Estimation and inference for the multivariate normal **19**
2.1 The likelihood of a normal sample . . . 19
2.2 The Wishart distribution . . . 22
2.3 Sampling from a multivariate normal distribution . . . 23
2.4 Central limit theorems . . . 24
2.5 Hotelling's T^2 test . . . 25
2.6 The two-sample test . . . 28
2.7 R laboratory: a joint mean test for setosa . . . 31
2.8 Further exercises . . . 31

3 Principal component analysis **34**
3.1 Aim and terminology . . . 34
3.2 Derivation by Lagrange multipliers . . . 35
3.3 Choosing and using components . . . 36
3.4 PCA and the singular value decomposition . . . 39
3.5 R laboratory: PCA of iris, with and without scaling . . . 40
3.6 Further exercises . . . 41

II Clustering and density estimation **43**

4 Clustering and K-means **44**

4.1 The K-means idea . . . 44
4.2 Within-cluster scatter . . . 45
4.3 The algorithm . . . 46
4.4 Initialisation and restarts . . . 47
4.5 Choosing K: the gap statistic . . . 48
4.6 Voronoi geometry . . . 49
4.7 R laboratory: do three clusters recover three species? . . . 50
4.8 Further exercises . . . 51

5 Mixture models and the EM algorithm 53
5.1 Finite mixture models . . . 53
5.2 The likelihood of a normal mixture . . . 54
5.3 The idea behind EM . . . 55
5.4 The complete-data likelihood . . . 55
5.5 The E and M steps . . . 56
5.6 Pathologies and practical advice . . . 60
5.7 Clustering with a fitted mixture . . . 61
5.8 Uncertainty about the parameters . . . 62
5.9 R laboratory: uncertainty about component membership . . . 63
5.10 Further exercises . . . 63

6 Model selection for stochastic models 65
6.1 Bayes factors and posterior odds . . . 65
6.2 The Bayesian information criterion . . . 66
6.3 Cross-validation of the likelihood . . . 68
6.4 The Akaike information criterion . . . 69
6.5 R laboratory: make the BIC convention explicit . . . 71
6.6 Further exercises . . . 71

7 Hierarchical clustering and kernel density estimation 73
7.1 Hierarchical clustering . . . 73
7.2 Kernel density estimation . . . 76
7.3 R laboratory: linkage and bandwidth are part of the analysis . . . 77
7.4 Further exercises . . . 78

III Classification 80

8 Statistical classifiers and discriminant analysis 81
8.1 The classification problem . . . 81
8.2 Posterior probabilities and the Bayes rule . . . 82
8.3 Error rates and optimal classifiers . . . 83
8.4 Linear discriminant analysis . . . 85
8.5 Quadratic discriminant analysis . . . 87
8.6 Fisher's linear discriminant analysis . . . 89
8.7 Mixture discriminant analysis . . . 90
8.8 Kernel density discriminant analysis . . . 91
8.9 R laboratory: LDA and QDA on held-out iris flowers . . . 91
8.10 Further exercises . . . 91

9 Estimating error rates 93
9.1 What is being estimated . . . 93
9.2 The apparent error rate . . . 94

9.3 The test-set method . . . 94
9.4 Cross-validation . . . 95
9.5 The bootstrap . . . 98
9.6 The jackknife . . . 99
9.7 Choosing between estimators . . . 100
9.8 R laboratory: tune neighbours without leaking the test set . . . 100
9.9 Further exercises . . . 101

10 Logistic regression 102
10.1 From a linear model to class probabilities . . . 102
10.2 Likelihood, score, and fitting . . . 103
10.3 Interpretation and decision boundary . . . 105
10.4 More than two classes . . . 106
10.5 Inference, deviance, and model comparison . . . 107
10.6 R laboratory: odds and probabilities for two iris species . . . 108
10.7 Further exercises . . . 108

11 Nearest-neighbour classifiers 110
11.1 The distance metric and scale . . . 110
11.2 Ties and the choice of k . . . 112
11.3 Theoretical properties . . . 112
11.4 R laboratory: see what a distance metric changes . . . 114
11.5 Further exercises . . . 114

12 Classification trees 116
12.1 Building a tree . . . 116
12.2 Node impurity . . . 117
12.3 Choosing a split . . . 117
12.4 Controlling the size of a tree . . . 120
12.5 Interpretation and instability . . . 121
12.6 R laboratory: prune using training cross-validation . . . 122
12.7 Further exercises . . . 122

13 Ensembles, random forests, and boosting 124
13.1 Why averaging helps . . . 124
13.2 Decision forests, bagging, and random forests . . . 125
13.3 Boosting . . . 128
13.4 R laboratory: a forest and its out-of-bag observations . . . 134
13.5 Further exercises . . . 135

IV Neural networks and support vector machines 136

14 Artificial neurons and networks 137
14.1 Biological background and a little history . . . 137
14.2 The unit . . . 138
14.3 What can a single unit do? . . . 140
14.4 Learning for a single threshold unit: the perceptron . . . 140
14.5 The XOR problem . . . 141
14.6 Multi-layer networks . . . 143
14.7 Universal approximation . . . 145
14.8 What needs to be chosen . . . 145
14.9 Notation for the rest of Part IV . . . 146

14.10 R laboratory: verify XOR before training anything . . . 146
14.11 Further exercises . . . 147

15 Learning in neural networks: gradient descent and backpropagation 149
15.1 The error function . . . 149
15.2 Gradient descent . . . 150
15.3 A single layer of sigmoid units . . . 151
15.4 Backpropagation . . . 152
15.5 How many weights? . . . 154
15.6 Behaviour on XOR . . . 155
15.7 R laboratory: a numerical gradient check . . . 157
15.8 Further exercises . . . 157

16 Neural networks in practice 159
16.1 Parameter values and other issues . . . 159
16.2 Weight initialisation . . . 160
16.3 Batch and online learning . . . 160
16.4 Preparing the data . . . 161
16.5 Output coding, softmax, and cross-entropy . . . 161
16.6 Constructing and training a network . . . 162
16.7 Evaluating the network . . . 164
16.8 Applications . . . 164
16.9 Recent innovations . . . 165
16.10 R laboratory: a small softmax network for iris . . . 165
16.11 Further exercises . . . 166

17 Support vector machines 167
17.1 Linear separators and the margin . . . 167
17.2 The hard-margin problem . . . 169
17.3 Soft-margin classification . . . 171
17.4 Non-linear SVMs and the kernel trick . . . 172
17.5 Practice, applications, and extensions . . . 176
17.6 R laboratory: tune an RBF SVM inside training data . . . 177
17.7 Further exercises . . . 178

V High-dimensional analysis and graphical models 180

18 High-dimensional analysis 181
18.1 Why classical methods fail when $p > n$ . . . 181
18.2 Principal component analysis for high-dimensional data . . . 183
18.3 Single-variable analysis . . . 184
18.4 Variable selection and the lasso . . . 187
18.5 Example application: the Alon colon-cancer microarray data . . . 192
18.6 R laboratory: a sparse simulation where the truth is known . . . 193
18.7 Further exercises . . . 194

19 Graphical models 196
19.1 Markov graphs . . . 196
19.2 Gaussian graphical models . . . 199
19.3 Model selection in Gaussian graphical models . . . 201
19.4 Regression-linked estimation for a known graph . . . 202
19.5 The graphical lasso . . . 205

19.6 An application: the top fifteen genes from the colon-cancer FDR analysis 206
19.7 R laboratory: a chain with dense marginal correlations 207
19.8 Further exercises 208

Source map and editorial notes **210**
Source coverage 210
Lecture transcripts 210
What was added, and what was left out 211
Editorial checks 211
Corrections in the guided revision 212
Verification record of the earlier edition 213
Research and exposition revision, 13 October 2026 213
Limitations 213
Changes in the executable R edition 214

Further problems **215**
Multivariate statistics 215
Clustering 216
Classification 1: the Bayes classifier 217
Classification 2: iris and discriminant analysis 218
Classification 3: nearest neighbours and validation 219
Graphical models and high-dimensional analysis 220

VI Worked solutions **222**

Solutions for Chapter 1 **223**

Solutions for Chapter 2 **226**

Solutions for Chapter 3 **229**

Solutions for Chapter 4 **231**

Solutions for Chapter 5 **234**

Solutions for Chapter 6 **236**

Solutions for Chapter 7 **238**

Solutions for Chapter 8 **240**

Solutions for Chapter 9 **242**

Solutions for Chapter 10 **244**

Solutions for Chapter 11 **246**

Solutions for Chapter 12 **249**

Solutions for Chapter 13 **251**

Solutions for Chapter 14 **253**

Solutions for Chapter 15 **255**

Solutions for Chapter 16 **257**

Solutions for Chapter 17 **259**

Solutions for Chapter 18 **262**

Solutions for Chapter 19 **265**

References **268**

Appendices 274

A Mathematical background **274**
A.1 Maximum likelihood estimation 274
A.2 Spectral decomposition 275
A.3 Mahalanobis distances and transformation 275
A.4 Convex hulls 276
A.5 The ANOVA identity 276

B R code from the notes **278**
B.1 Nearest-neighbour example (Chapter 11) 278
B.2 Analysis of the Alon colon-cancer data (Chapter 18) 279

C Reproducibility and computational checks **281**
C.1 What is checked 281
C.2 Rebuild instructions 281

How to use this book

This book is for a first serious encounter with statistical learning. We will begin with a problem, work out what information and assumptions are available, and build a method that uses them. The aim is to make the equations intelligible enough that you can reconstruct an argument instead of remembering which formula happened to be on the slide.

Keep asking what is random, what is being estimated, and what would have to be true for the conclusion to follow. A derivation establishes a consequence of assumptions. A numerical check can catch a sign error or a broken implementation. Neither tells us, by itself, that the assumptions describe the data. Those distinctions are useful throughout the course, especially when a fitted picture looks persuasive.

Read with a pencil and run the R laboratories as you go. Before opening a solution, make your own attempt and identify the exact step where you get stuck. The worked solutions explain a route through the calculation and retain detailed algebra. The 24 further tutorial problems ask for longer arguments and analyses; they remain unsolved here.

Two pictures

Start with two versions of the same picture. One is an unlabelled cloud of points; the other attaches a species label to each point. The first invites questions about variation, density and grouping. The second also permits supervised prediction. Removing the labels changes the information available to the method, even though the measurement matrix has not changed.

A conditional model predicts a response from supplied predictors. A joint model describes the variables together. This difference in scope does not force their fitted prediction lines to differ. On the same complete bivariate sample, the conditional-mean line from an unconstrained Gaussian fit is exactly the ordinary least-squares line: the covariance divisor cancels in the slope. A principal axis answers a geometric question and generally gives another line. Reverse regression answers another conditional question. We will derive these distinctions rather than relying on a sketch to settle them.

Choose a method for the question it answers, the loss it minimises and the structure it can represent. A covariance matrix, a nearest-neighbour rule and a neural network each discard and retain different information. Their failure cases are often as instructive as their successful fits: singular matrices reveal missing information, XOR reveals a linear boundary's limitation, and reused validation data reveal how selection can flatter a result.

A running example: Fisher's iris data

R's `datasets::iris` contains 150 flowers, 50 from each of three species, with four measurements in centimetres: sepal length, sepal width, petal length and petal width. The data were collected by Edgar Anderson and used by Fisher in his 1936 discriminant-analysis paper. The provenance

and measurement definitions follow the official dataset documentation (R Core Team, 2026d); Fisher's paper supplies the historical discriminant-analysis context (Fisher, 1936).

Iris ties together covariance, PCA, clustering, density estimation and supervised classification. It also exposes a useful distinction: species labels may be observed, withheld during unsupervised learning, or predicted during classification. Those are different tasks even when the measurement matrix is identical.

For supervised laboratories, a fixed stratified split uses 35 flowers per species for training and 15 for testing. Cross-validation for tuning is confined to training data, and preprocessing is refitted inside each fold. Descriptive figures that use all 150 flowers are identified as descriptive. Because the same small test set is revisited for teaching, the book does **not** use its scores to declare a winning algorithm.

Iris is deliberately not used where it would conceal the mechanism: XOR explains nonlinear features, controlled simulations expose overfitting, sparse high-dimensional simulations separate prediction from variable selection, and a known Gaussian chain explains conditional independence.

Reading code and figures

Every numbered figure in the original reconstructed book has an R reproduction, including its three embedded diagrams. The new figures can use different data or layouts; their captions describe the actual R calculation. Additional iris figures are new to this edition. No original figure PDF is loaded by the new build.

Each chapter ends with an **R laboratory** that executes when the book is rendered. Some inherited short snippets are retained as **R templates** because they illustrate an interface or require historical data not supplied with the book. They are displayed as ordinary code, not passed off as executed analyses. The longer historical source programs in the appendix are likewise preserved as templates. Statements reporting source-only analyses remain attributed to those sources.

Hand-worked examples remain beside the arguments they illustrate. Where the processed historical data are absent, reported results are labelled as such. A seeded R illustration is its own experiment: its output should be compared with its own code and caption, not forced to reproduce an unrelated historical simulation.

Notation and assumptions

Scalars use ordinary mathematical type; vectors are bold; matrix symbols are bold blue. There are n observations and p predictors unless a chapter explicitly defines another indexing convention. The sample covariance uses divisor $n - 1$; likelihood calculations can use divisor n, and the graphical-model chapter states its convention.

Quantiles in the inherited derivations follow the notes' upper-tail convention. For example, $t_{\alpha/2;n-1}$ has probability $\alpha/2$ to its right. In R the corresponding command is `qt(1 - alpha/2, df = n - 1)`. Confusing tail probability with cumulative probability reverses a critical value.

Normality, independence, equal covariances, class prevalence and loss functions are modelling assumptions, not properties guaranteed by successful software execution. Diagnostics and sensitivity checks inform their plausibility; they do not convert an observational association into a causal effect.

Sources and scope

The course notes and slide decks determine the scope and chapter order. The source map records their coverage. This revision also checks important claims against academic papers, authoritative texts and software documentation, with author–date citations beside the relevant statements. The accompanying claim audit records the arguments consulted, corrections made and historical results that could not be independently reproduced.

To rebuild, open the `quarto-book` folder and run `quarto render`. The editable sources are the chapter Qmd files; mathematical displays retain LaTeX syntax so that detailed algebra and cross-references survive intact in the PDF.

Part I

Multivariate statistics

Chapter 1

Multivariate distributions and the multivariate normal

Take one iris flower and record its four measurements. You now have four numbers, but they belong to the same flower. If its petals are unusually long, does that change what you expect its petal width to be? Four separate histograms cannot answer that question. We need a description of how the measurements vary together.

We'll build that description in three stages. First, collect the measurements into a random vector. Then use a covariance matrix to describe their scales and linear associations. Finally, introduce the multivariate normal as a complete probability model. The distinction matters: a covariance matrix summarises a distribution; it does not usually identify it. Within the Gaussian family, however, the mean and covariance do identify the distribution. That additional assumption is what makes the conditional calculations later in this chapter possible.

Keep one question in view as we work: which conclusions follow from covariance algebra alone, and which need joint normality? Zero covariance gives independence for jointly Gaussian blocks. Outside that family, it can coexist with a very strong nonlinear relationship. We will also compare pooled iris measurements with measurements within a species. Changing the population changes the distribution we are trying to describe.

Source. Lecture notes, Chapter 1, Sections 1.1–1.5 (pp. 4–16); the notes draw heavily on Morrison (2005), and Härdle and Simar (2019) covers much of the same ground.

1.1 Random vectors and joint distributions

Write $\boldsymbol{X}_i$ for the random measurement vector on unit i, and $\boldsymbol{x}_i$ for its observed value. The index $i = 1, \ldots, n$ identifies the unit; $j = 1, \ldots, p$ identifies a coordinate within that unit. These indices do different jobs. Our usual sampling assumption is that the vectors $\boldsymbol{X}_1, \ldots, \boldsymbol{X}_n$ are independent and identically distributed. It does **not** say that the coordinates within a vector are independent. Unless stated otherwise, we consider continuous coordinates and write $\boldsymbol{X} = (X_1, \ldots, X_p)^\top$ for a generic observation.

So one row of the data matrix is one observational unit. The four iris measurements are four coordinates, not four independent flowers. Keeping that distinction straight now saves considerable trouble when we count sample sizes and degrees of freedom later.

Definition 1.1 (Joint distribution function and density)**.** The *joint distribution function* of $\boldsymbol{X}$ is

$$F(\boldsymbol{x}) = \mathbb{P}(\boldsymbol{X} \leq \boldsymbol{x}) = \mathbb{P}(X_1 \leq x_1, \ldots, X_p \leq x_p), \qquad \boldsymbol{x} \in \mathbb{R}^p.$$

If F is absolutely continuous, $\boldsymbol{X}$ has a *joint density*

$$f(\boldsymbol{x}) = \frac{\partial^p F(\boldsymbol{x})}{\partial x_1 \cdots \partial x_p} \quad \text{(almost everywhere)}, \qquad F(\boldsymbol{x}) = \int_{-\infty}^{x_p} \cdots \int_{-\infty}^{x_1} f(u_1, \ldots, u_p)\, \mathrm{d}u_1 \cdots \mathrm{d}u_p.$$

If the elements of $\boldsymbol{X}$ are independent, then the density and the distribution function both factorise,

$$f(\boldsymbol{x}) = \prod_{j=1}^{p} f_j(x_j), \qquad F(\boldsymbol{x}) = \prod_{j=1}^{p} F_j(x_j),$$

where f_j and F_j belong to X_j alone. Conversely, a factorisation of this form implies independence. In practice the elements of $\boldsymbol{X}$ are usually dependent, and the whole point of multivariate analysis is to describe that dependence.

A density can be easy to evaluate even when its integral has no convenient elementary expression. The correlated multivariate normal is an example: probabilities over rectangles generally require numerical integration. This is one reason we often calculate with densities. It does not mean a continuous observation has positive probability at an exact point; probability still comes from integrating over a region.

1.2 Marginal and conditional distributions

There are two different operations we can perform on a joint distribution. If we stop recording one coordinate, we need a **marginal** distribution: integrate that coordinate out. If we know its value, we need a **conditional** distribution: restrict attention to that value and renormalise. Dropping information and using information are different operations, even though both leave us with fewer variables.

To obtain the joint density of a subset of the variables, say the first $q < p$, integrate out the others:

$$g(x_1, \ldots, x_q) = \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} f(x_1, \ldots, x_p)\, \mathrm{d}x_{q+1} \cdots \mathrm{d}x_p. \tag{1.1}$$

The corresponding marginal distribution function is obtained by sending the remaining arguments to infinity,

$$G(x_1, \ldots, x_q) = \mathbb{P}(X_1 \le x_1, \ldots, X_q \le x_q) = F(x_1, \ldots, x_q, \infty, \ldots, \infty). \tag{1.2}$$

For conditioning, the elementary ratio $\mathbb{P}(A \mid B) = \mathbb{P}(A \cap B)/\mathbb{P}(B)$ needs care. An exact value of a continuous variable ordinarily has probability zero. We therefore use a ratio of densities, defined for conditioning values where the marginal density is positive. In the display below, A and B are shorthand for the two coordinate blocks evaluated at their stated values, not positive-probability point events.

$$\begin{aligned} f_{A|B}(A) &= \frac{f_{AB}(A,B)}{f_B(B)}, \qquad f_B(B) > 0, \\ &= \frac{f(x_1, \ldots, x_q, x_{q+1}, \ldots, x_p)}{\int \cdots \int f(x_1, \ldots, x_q, x_{q+1}, \ldots, x_p)\, \mathrm{d}x_1 \cdots \mathrm{d}x_q} = \frac{f(x_1, \ldots, x_p)}{g(x_{q+1}, \ldots, x_p)}. \end{aligned} \tag{1.3}$$

Here $g(x_{q+1}, \ldots, x_p)$ denotes the marginal density of the *last* $p - q$ variables, evaluated at the conditioning values. It is not the same function as the marginal density in (1.1); the notation follows the argument.

Because $f_{AB}(A,B) = f_{A|B}(A)f_B(B) = f_{B|A}(B)f_A(A)$ whenever $f_A(A) > 0$ and $f_B(B) > 0$, we obtain Bayes' rule for densities,

$$f_{A|B}(A) = \frac{f_{B|A}(B)\, f_A(A)}{f_B(B)}. \tag{1.4}$$

1.3 Moments

A full joint density can be a substantial object to estimate. Let's first ask for a smaller description: where is each coordinate centred, and which deviations tend to occur together? Expectations provide that description when the required moments exist. Throughout this section, assume finite second moments; otherwise the covariance matrix may not exist.

Definition 1.2 (Mean vector, covariance, covariance matrix)**.** The expected value of $\boldsymbol{X}$ is the vector of the expected values of its elements,

$$\mathbb{E}(\boldsymbol{X}) = \boldsymbol{\mu} = \big(\mathbb{E}(X_1), \ldots, \mathbb{E}(X_p)\big)^\top. \tag{1.5}$$

The covariance between elements j and k is

$$\sigma_{jk} = \mathrm{Cov}(X_j, X_k) = \mathbb{E}\big[(X_j - \mathbb{E}X_j)(X_k - \mathbb{E}X_k)\big] = \mathbb{E}(X_jX_k) - \mathbb{E}(X_j)\mathbb{E}(X_k), \tag{1.6}$$

which can be written as a double integral against the joint density f_{jk} of (X_j, X_k) minus the product of two single integrals against the marginal densities f_j, f_k. The *covariance matrix* is

$$\boldsymbol{\Sigma} = \mathrm{Var}(\boldsymbol{X}) = \mathrm{Cov}(\boldsymbol{X}, \boldsymbol{X}^\top) = \mathbb{E}\big[(\boldsymbol{X} - \boldsymbol{\mu})(\boldsymbol{X} - \boldsymbol{\mu})^\top\big] = \begin{pmatrix} \sigma_{11} & \cdots & \sigma_{1p} \\ \vdots & \ddots & \vdots \\ \sigma_{1p} & \cdots & \sigma_{pp} \end{pmatrix}. \tag{1.7}$$

The diagonal entry $\sigma_{jj} = \sigma_j^2$ is the variance of X_j and σ_j its standard deviation.

1.3.1 Positive semi-definiteness

The matrix $\boldsymbol{\Sigma}$ is real and symmetric, and it is positive semi-definite. To see the latter, take any $\boldsymbol{z} \in \mathbb{R}^p$. The scalar $\boldsymbol{z}^\top\boldsymbol{X}$ has variance

$$\mathrm{Var}(\boldsymbol{z}^\top\boldsymbol{X}) = \mathbb{E}\big[\boldsymbol{z}^\top(\boldsymbol{X} - \boldsymbol{\mu})(\boldsymbol{X} - \boldsymbol{\mu})^\top\boldsymbol{z}\big] = \boldsymbol{z}^\top\boldsymbol{\Sigma}\boldsymbol{z},$$

and a variance cannot be negative, so $\boldsymbol{z}^\top\boldsymbol{\Sigma}\boldsymbol{z} \geq 0$ for every $\boldsymbol{z}$. For a symmetric real matrix this is equivalent to all eigenvalues being non-negative (Appendix A.2). Positive semi-definiteness is exactly what guarantees $|\sigma_{jk}| \leq \sqrt{\sigma_{jj}\sigma_{kk}}$: apply the condition to the 2×2 principal submatrix with entries $\sigma_{jj}, \sigma_{jk}, \sigma_{kk}$, whose determinant must be non-negative.

This gives a useful check on a proposed covariance matrix. Symmetry is necessary, but it is not enough: every linear combination must have nonnegative variance. Sylvester's criterion turns this into determinant conditions. Strict positive definiteness requires all **leading** principal minors to be positive; positive semidefiniteness requires **all** principal minors to be nonnegative. Those are different tests. A matrix can pass the leading-minor semidefinite check and still fail elsewhere.

1.3.2 Linear transformations of moments

Translations do not change covariances: for constants $c, d \in \mathbb{R}$,

$$\operatorname{Cov}(X_j + c,\, X_k + d) = \operatorname{Cov}(X_j, X_k). \tag{1.8}$$

Scale changes pass through the covariance,

$$\operatorname{Cov}(cX_j,\, dX_k) = cd\, \operatorname{Cov}(X_j, X_k). \tag{1.9}$$

For a fixed vector $\boldsymbol{a} = (a_1, \ldots, a_p)^\top$,

$$\operatorname{Var}(\boldsymbol{a}^\top \boldsymbol{X}) = \sum_{j=1}^{p} \sum_{k=1}^{p} a_j a_k \sigma_{jk} = \boldsymbol{a}^\top \boldsymbol{\Sigma} \boldsymbol{a}, \tag{1.10}$$

and for a second fixed vector $\boldsymbol{b}$,

$$\operatorname{Cov}(\boldsymbol{a}^\top \boldsymbol{X},\, \boldsymbol{b}^\top \boldsymbol{X}) = \sum_{j=1}^{p} \sum_{k=1}^{p} a_j b_k \sigma_{jk} = \boldsymbol{a}^\top \boldsymbol{\Sigma} \boldsymbol{b}. \tag{1.11}$$

More generally, let $\boldsymbol{A}$ be $r \times p$ and $\boldsymbol{B}$ be $s \times p$ constant matrices and set $\boldsymbol{Y} = \boldsymbol{A}\boldsymbol{X}$, $\boldsymbol{Z} = \boldsymbol{B}\boldsymbol{X}$. Then $\mathbb{E}\boldsymbol{Y} = \boldsymbol{A}\boldsymbol{\mu}$, and

$$\operatorname{Cov}(\boldsymbol{Y}, \boldsymbol{Y}^\top) = \mathbb{E}[\boldsymbol{A}(\boldsymbol{X} - \boldsymbol{\mu})(\boldsymbol{X} - \boldsymbol{\mu})^\top \boldsymbol{A}^\top] = \boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^\top, \tag{1.12}$$

$$\operatorname{Cov}(\boldsymbol{Z}, \boldsymbol{Z}^\top) = \boldsymbol{B}\boldsymbol{\Sigma}\boldsymbol{B}^\top, \tag{1.13}$$

$$\operatorname{Cov}(\boldsymbol{Y}, \boldsymbol{Z}^\top) = \mathbb{E}[\boldsymbol{A}(\boldsymbol{X} - \boldsymbol{\mu})(\boldsymbol{X} - \boldsymbol{\mu})^\top \boldsymbol{B}^\top] = \boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{B}^\top. \tag{1.14}$$

The reason the matrices can move outside the expectation is that they are fixed. Their order matters: $\boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{B}^\top$ has size $r \times s$, exactly the size required for the cross-covariances of an r-vector and an s-vector. Checking those dimensions is often quicker than trying to remember where the transpose belongs. The scalar formulas above follow by taking one-row matrices.

1.3.3 Correlation

To report dependence on a scale-free footing, covariances are standardised into the Pearson correlation,

$$\rho_{jk} = \operatorname{corr}(X_j, X_k) = \frac{\operatorname{Cov}(X_j, X_k)}{\sqrt{\operatorname{Var}(X_j)\operatorname{Var}(X_k)}} = \frac{\sigma_{jk}}{\sigma_j \sigma_k} \in [-1, 1]. \tag{1.15}$$

If X_j and X_k are independent then $\rho_{jk} = 0$, but the converse is false (Exercise 1.3). If $X_k = cX_j + d$ then $\rho_{jk} = 1$ for $c > 0$ and $\rho_{jk} = -1$ for $c < 0$.

Correlation becomes easier to interpret after standardising both variables. With positive, finite variances, the slope of the best linear predictor of Y from X is $\rho\sigma_Y/\sigma_X$, and its residual variance is $\sigma_Y^2(1 - \rho^2)$. Thus ρ is the slope in standard-deviation units, while $1 - \rho^2$ is the fraction of response variance left by this linear prediction. On the original axes, slopes also depend on the units. A perfect positive correlation means an exact increasing affine relationship, not necessarily a slope of one. For a symmetric example such as $Y = X^2$ with $X \sim \mathrm{N}(0, 1)$, correlation is zero

although the response is completely determined by the predictor. If a variance is zero, correlation is undefined.

The correlation matrix is $\boldsymbol{P} = (\rho_{jk})$, with ones on the diagonal. Let $\boldsymbol{D} = \operatorname{diag}(\sigma_1, \ldots, \sigma_p)$, whose inverse is $\operatorname{diag}(1/\sigma_1, \ldots, 1/\sigma_p)$. Entry (j, k) of $\boldsymbol{D}^{-1}\boldsymbol{\Sigma}\boldsymbol{D}^{-1}$ is $\sigma_{jk}/(\sigma_j\sigma_k) = \rho_{jk}$, so

$$\boldsymbol{P} = \boldsymbol{D}^{-1}\boldsymbol{\Sigma}\boldsymbol{D}^{-1}, \qquad \boldsymbol{\Sigma} = \boldsymbol{D}\boldsymbol{P}\boldsymbol{D}. \tag{1.16}$$

1.4 The multivariate normal distribution

The univariate normal density is

$$\phi(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\Big\{ -\frac{(x-\mu)^2}{2\sigma^2} \Big\}, \qquad x \in \mathbb{R}. \tag{1.17}$$

If the elements of $\boldsymbol{X} = (X_1, \ldots, X_p)^\top$ are independent normals, $X_j \sim \mathrm{N}(\mu_j, \sigma_j^2)$, then the mean vector is $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_p)^\top$ and the covariance matrix is diagonal, $\boldsymbol{\Sigma} = \operatorname{diag}(\sigma_1^2, \ldots, \sigma_p^2)$. The joint density is the product of the marginals,

$$\begin{aligned} f(\boldsymbol{x}) &= \prod_{j=1}^{p} \frac{1}{\sqrt{2\pi}\,\sigma_j} \exp\Big\{ -\frac{(x_j-\mu_j)^2}{2\sigma_j^2} \Big\} \\ &= \frac{1}{(2\pi)^{p/2} \prod_{j=1}^{p} \sigma_j} \exp\Big\{ -\frac{1}{2} \sum_{j=1}^{p} \Big(\frac{x_j - \mu_j}{\sigma_j} \Big)^2 \Big\}. \end{aligned} \tag{1.18}$$

For this diagonal $\boldsymbol{\Sigma}$ we have $|\boldsymbol{\Sigma}|^{1/2} = \prod_j \sigma_j$ and $(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{x}-\boldsymbol{\mu}) = \sum_j (x_j - \mu_j)^2/\sigma_j^2$, so (1.18) can be written without reference to the diagonal structure. That rewriting defines the general case.

Definition 1.3 (Multivariate normal distribution)**.** Let $\boldsymbol{\mu} \in \mathbb{R}^p$ and let $\boldsymbol{\Sigma}$ be a symmetric positive-definite $p \times p$ matrix. A random vector $\boldsymbol{X}$ has the p-variate normal distribution $\mathrm{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ if it has density

$$f(\boldsymbol{x}) = \frac{1}{(2\pi)^{p/2} |\boldsymbol{\Sigma}|^{1/2}} \exp\Big\{ -\frac{1}{2} (\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{x}-\boldsymbol{\mu}) \Big\}, \qquad \boldsymbol{x} \in \mathbb{R}^p. \tag{1.19}$$

When $\boldsymbol{\Sigma}$ is positive definite, $f(\boldsymbol{x}) > 0$ for every $\boldsymbol{x}$ and $\int f = 1$ for every $\boldsymbol{\mu}$, so f is a valid density. The mean vector is $\boldsymbol{\mu}$ and the covariance matrix is $\boldsymbol{\Sigma}$. (The subscript p is dropped when the dimension is clear, and the distribution is then written $\mathrm{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.)

1.4.1 The bivariate case

Start with two dimensions. The surface height is a density; probability is the volume above a region of the horizontal plane. A normal model assigns some probability to every open region, including physically impossible negative lengths. Whether that matters depends on how much probability falls there and what question we ask of the approximation. A familiar distribution is still an assumption.

The bivariate normal is the simplest multivariate case, and it is worth studying before higher dimensions. It has five parameters: the two means, the two variances, and the correlation ρ. In matrix form

$$\boldsymbol{\mu} = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \qquad \boldsymbol{\Sigma} = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}. \tag{1.20}$$

Since $|\boldsymbol{\Sigma}| = \sigma_1^2\sigma_2^2(1-\rho^2)$ and

$$\boldsymbol{\Sigma}^{-1} = \frac{1}{\sigma_1^2\sigma_2^2(1-\rho^2)} \begin{pmatrix} \sigma_2^2 & -\rho\sigma_1\sigma_2 \\ -\rho\sigma_1\sigma_2 & \sigma_1^2 \end{pmatrix},$$

the density (1.19) becomes

$$f(x_1, x_2) = \frac{1}{2\pi\sigma_1\sigma_2\sqrt{1-\rho^2}} \exp\Big\{ -\frac{1}{2(1-\rho^2)} \Big[z_1^2 - 2\rho\, z_1 z_2 + z_2^2 \Big] \Big\}, \qquad z_j = \frac{x_j - \mu_j}{\sigma_j}. \tag{1.21}$$

1.4.2 Level sets are ellipsoids

The ellipse through the data cloud describes the spread of observations. A confidence region for the mean describes uncertainty about the centre. The latter typically contracts as we collect more independent observations; the population cloud does not. For the moment we are studying contours of the density, so hold the model parameters fixed and find points with equal density.

Fix a density value $c > 0$ and consider the level set $\{\boldsymbol{x} : f(\boldsymbol{x}) = c\}$. For c larger than the maximum of f this is empty. Otherwise, divide both sides of $f(\boldsymbol{x}) = c$ by the leading constant and take logarithms:

$$-\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}) = \log\left(c\,(2\pi)^{p/2}|\boldsymbol{\Sigma}|^{1/2}\right),$$

that is,

$$(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}) = C \tag{1.22}$$

for a constant C determined by c. Provided $C > 0$ this describes an ellipsoid centred at $\boldsymbol{\mu}$. In the bivariate case the level sets are ellipses, and the axes follow the longest and shortest lines through the centre that touch the contour twice.

To find the axes, write the spectral decomposition $\boldsymbol{\Sigma} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^\top$ (Appendix A.2) with orthogonal $\boldsymbol{\Gamma} = (\boldsymbol{\gamma}_1, \ldots, \boldsymbol{\gamma}_p)$ and $\boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_p)$. Then $\boldsymbol{\Sigma}^{-1} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}^{-1}\boldsymbol{\Gamma}^\top$ and, with the rotated coordinates $\boldsymbol{u} = \boldsymbol{\Gamma}^\top(\boldsymbol{x}-\boldsymbol{\mu})$,

$$(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}) = \boldsymbol{u}^\top \boldsymbol{\Lambda}^{-1}\boldsymbol{u} = \sum_{j=1}^{p} \frac{u_j^2}{\lambda_j} = C.$$

This is the equation of an ellipsoid whose jth semi-axis points along the eigenvector $\boldsymbol{\gamma}_j$ and has length $\sqrt{C\lambda_j}$. The axes of a multivariate normal ellipsoid estimated from data are the principal component directions of Chapter 3.

Remark 1.4 (Squared Mahalanobis distance). The quantity $(\boldsymbol{X}-\boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\boldsymbol{X}-\boldsymbol{\mu})$ is the squared Mahalanobis distance of $\boldsymbol{X}$ from its mean (Appendix A.3). If $\boldsymbol{X} \sim \mathrm{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, set $\boldsymbol{Z} = \boldsymbol{\Sigma}^{-1/2}(\boldsymbol{X}-\boldsymbol{\mu})$. By (1.12), $\operatorname{Var}(\boldsymbol{Z}) = \boldsymbol{\Sigma}^{-1/2}\boldsymbol{\Sigma}\boldsymbol{\Sigma}^{-1/2} = \boldsymbol{I}$, and $\boldsymbol{Z}$ is normal by Proposition 1.5 below, so its elements are independent standard normals. Hence the squared Mahalanobis distance equals $\boldsymbol{Z}^\top\boldsymbol{Z} = \sum_j Z_j^2 \sim \chi^2_p$. A density contour with $C = \chi^2_{p;\,\alpha}$ (the upper α quantile) therefore encloses probability $1-\alpha$.

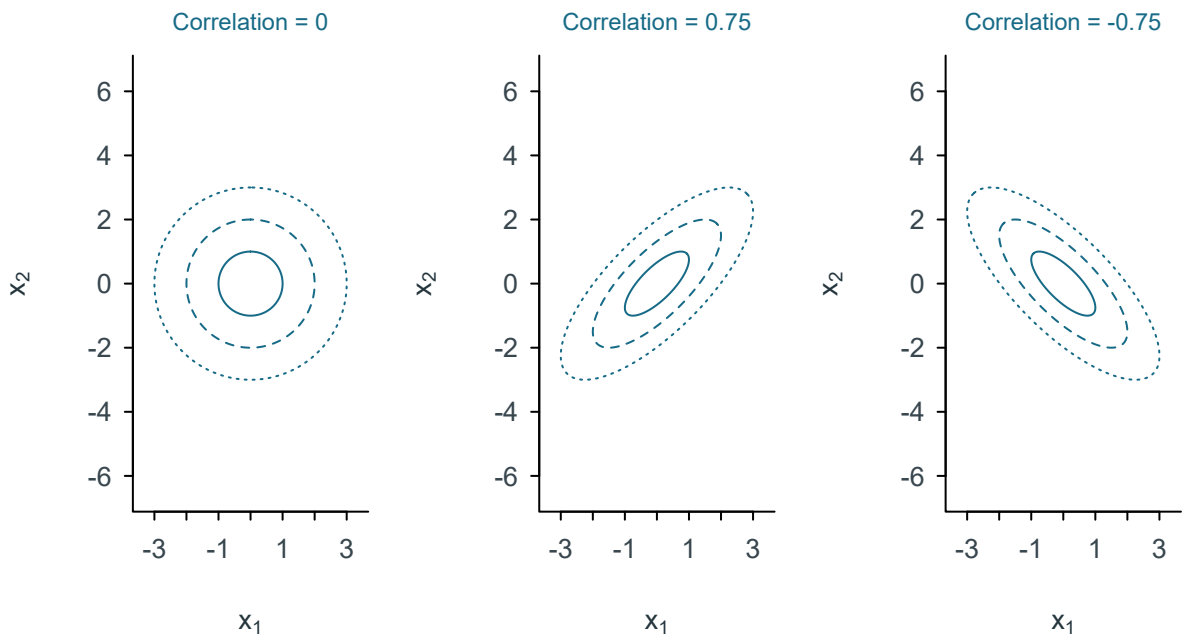


Figure 1.1: Equal-Mahalanobis-distance contours for three bivariate normal covariance matrices. Correlation rotates and stretches the contours.

1.5 Linear transformations, marginals, and conditionals

Proposition 1.5 (Linear transformations of a normal vector)**.** Let $\boldsymbol{X} \sim \mathrm{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with $\boldsymbol{\Sigma}$ of rank p, and let $\boldsymbol{A}$ be an $m \times p$ matrix of rank $m \leq p$. Then $\boldsymbol{Y} = \boldsymbol{A}\boldsymbol{X}$ is an m-vector with

$$\boldsymbol{Y} \sim \mathrm{N}_m(\boldsymbol{A}\boldsymbol{\mu},\ \boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^\top).$$

The mean and covariance are the general results (1.5) and (1.12); that the distribution is again normal is the substantive claim, proved in Anderson (2003) and Graybill (2001).

1.5.1 Marginals

A binary matrix of zeros and ones extracts a marginal from a trivariate normal by the linear-transformation property. With $A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, the product $Y = AX$ equals $(X_1, X_2)^\top$, and the marginal covariance is the corresponding 2×2 block of $\boldsymbol{\Sigma}$.

Some first courses write the standard deviation in the second slot of the normal; this book writes the variance, or the covariance matrix, there.

Taking $\boldsymbol{A}$ to be a binary selection matrix, with a single 1 in each row picking out a chosen element of $\boldsymbol{X}$, shows that *any subset of the elements of a multivariate normal vector is multivariate normal*, with the corresponding sub-vector of $\boldsymbol{\mu}$ and sub-matrix of $\boldsymbol{\Sigma}$. For example, $\boldsymbol{A} = (\boldsymbol{I}_q \ \ \boldsymbol{0})$ selects the first q elements.

The converse fails: having normal marginals does not imply that the joint distribution is multivariate normal (Exercise 1.5).

1.5.2 Partitioned vectors

Split $\boldsymbol{X}$ into $\boldsymbol{X}_A$, containing the first q elements ($0 < q < p$), and $\boldsymbol{X}_B$, containing the last $p - q$, and partition the parameters conformably:

$$\boldsymbol{X} = \begin{pmatrix} \boldsymbol{X}_A \\ \boldsymbol{X}_B \end{pmatrix}, \qquad \boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_A \\ \boldsymbol{\mu}_B \end{pmatrix}, \qquad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{AA} & \boldsymbol{\Sigma}_{AB} \\ \boldsymbol{\Sigma}_{AB}^\top & \boldsymbol{\Sigma}_{BB} \end{pmatrix}. \tag{1.23}$$

By the previous paragraph $\boldsymbol{X}_A \sim \mathrm{N}_q(\boldsymbol{\mu}_A, \boldsymbol{\Sigma}_{AA})$ and $\boldsymbol{X}_B \sim \mathrm{N}_{p-q}(\boldsymbol{\mu}_B, \boldsymbol{\Sigma}_{BB})$. The blocks have dimensions $q \times q$, $q \times (p-q)$, and $(p-q) \times (p-q)$. Moreover $\boldsymbol{X}_A$ and $\boldsymbol{X}_B$ are independent if and only if $\boldsymbol{\Sigma}_{AB} = \boldsymbol{0}$. This is a special property of the normal family: in general, zero covariance does not give independence.

1.5.3 The conditional distribution

We already know how to extract a marginal: select the relevant mean entries and covariance block. Conditioning needs a correction because the observed coordinates carry information about the unobserved ones. We will obtain the correction by dividing the joint density by the marginal density and completing a square. The covariance left over is the Schur complement. Since the full $\boldsymbol{\Sigma}$ is positive definite here, this complement is positive definite too; a singular complement would require a degenerate version of the model. See Boyd and Vandenberghe (2004, Appendix A.5.5) for the block-matrix identity.

We want the density of $\boldsymbol{X}_A$ given $\boldsymbol{X}_B = \boldsymbol{x}_B$ in terms of the blocks of $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$. By (1.3),

$$f_{\boldsymbol{X}_A \mid \boldsymbol{X}_B} = \frac{f_{\boldsymbol{X}_A \boldsymbol{X}_B}}{f_{\boldsymbol{X}_B}}, \tag{1.24}$$

where the denominator is the normal density

$$f_{\boldsymbol{X}_B}(\boldsymbol{x}_B) = \frac{1}{(2\pi)^{(p-q)/2} |\boldsymbol{\Sigma}_{BB}|^{1/2}} \exp\Big\{ -\tfrac{1}{2} (\boldsymbol{x}_B - \boldsymbol{\mu}_B)^\top \boldsymbol{\Sigma}_{BB}^{-1} (\boldsymbol{x}_B - \boldsymbol{\mu}_B) \Big\}.$$

The numerator needs $\boldsymbol{\Sigma}^{-1}$ and $|\boldsymbol{\Sigma}|$ in terms of the blocks. Both come from the *Schur complement* of $\boldsymbol{\Sigma}_{BB}$ in $\boldsymbol{\Sigma}$ (Boyd and Vandenberghe, 2004; Zhang, 2005),

$$\boldsymbol{\Sigma}_{A|B} = \boldsymbol{\Sigma}_{AA} - \boldsymbol{\Sigma}_{AB} \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{\Sigma}_{AB}^\top. \tag{1.25}$$

Step 1: block inverse and determinant.

The block-inverse identity gives

$$\boldsymbol{\Sigma}^{-1} = \begin{pmatrix} \boldsymbol{\Sigma}_{A|B}^{-1} & -\boldsymbol{\Sigma}_{A|B}^{-1} \boldsymbol{\Sigma}_{AB} \boldsymbol{\Sigma}_{BB}^{-1} \\ -\boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{\Sigma}_{AB}^\top \boldsymbol{\Sigma}_{A|B}^{-1} & \boldsymbol{\Sigma}_{BB}^{-1} + \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{\Sigma}_{AB}^\top \boldsymbol{\Sigma}_{A|B}^{-1} \boldsymbol{\Sigma}_{AB} \boldsymbol{\Sigma}_{BB}^{-1} \end{pmatrix}, \tag{1.26}$$

and the determinant factorises as

$$|\boldsymbol{\Sigma}| = |\boldsymbol{\Sigma}_{BB}| \, |\boldsymbol{\Sigma}_{A|B}|. \tag{1.27}$$

Step 2: normalising constants.

Using (1.27), the ratio of the constants in numerator and denominator of (1.24) is

$$\frac{(2\pi)^{(p-q)/2} |\boldsymbol{\Sigma}_{BB}|^{1/2}}{(2\pi)^{p/2} |\boldsymbol{\Sigma}_{BB}|^{1/2} |\boldsymbol{\Sigma}_{A|B}|^{1/2}} = \frac{1}{(2\pi)^{q/2} |\boldsymbol{\Sigma}_{A|B}|^{1/2}}. \tag{1.28}$$

Step 3: the exponent.

Write $\boldsymbol{a} = \boldsymbol{x}_A - \boldsymbol{\mu}_A$ and $\boldsymbol{b} = \boldsymbol{x}_B - \boldsymbol{\mu}_B$. The exponent of the ratio is $-\frac{1}{2}E$ with

$$\begin{aligned} E &= \begin{pmatrix} \boldsymbol{a} \\ \boldsymbol{b} \end{pmatrix}^\top \boldsymbol{\Sigma}^{-1} \begin{pmatrix} \boldsymbol{a} \\ \boldsymbol{b} \end{pmatrix} - \boldsymbol{b}^\top \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{b} \\ &= \boldsymbol{a}^\top \boldsymbol{\Sigma}_{A|B}^{-1} \boldsymbol{a} - \boldsymbol{a}^\top \boldsymbol{\Sigma}_{A|B}^{-1} \boldsymbol{\Sigma}_{AB} \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{b} - \boldsymbol{b}^\top \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{\Sigma}_{AB}^\top \boldsymbol{\Sigma}_{A|B}^{-1} \boldsymbol{a} \\ &\quad + \boldsymbol{b}^\top (\boldsymbol{\Sigma}_{BB}^{-1} + \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{\Sigma}_{AB}^\top \boldsymbol{\Sigma}_{A|B}^{-1} \boldsymbol{\Sigma}_{AB} \boldsymbol{\Sigma}_{BB}^{-1}) \boldsymbol{b} - \boldsymbol{b}^\top \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{b}. \end{aligned}$$

In the last line the two terms $\pm \boldsymbol{b}^\top \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{b}$ cancel. The two cross terms are transposes of each other and both are scalars, so they are equal. Hence

$$E = \boldsymbol{a}^\top \boldsymbol{\Sigma}_{A|B}^{-1} \boldsymbol{a} - 2\, \boldsymbol{a}^\top \boldsymbol{\Sigma}_{A|B}^{-1} \boldsymbol{\Sigma}_{AB} \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{b} + \boldsymbol{b}^\top \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{\Sigma}_{AB}^\top \boldsymbol{\Sigma}_{A|B}^{-1} \boldsymbol{\Sigma}_{AB} \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{b}. \tag{1.29}$$

Step 4: completing the square.

We want a density for $\boldsymbol{x}_A$, so everything not involving $\boldsymbol{x}_A$ is constant, and the density must integrate to one. This is the multivariate version of completing the square. Define

$$\boldsymbol{\mu}_{A|B}(\boldsymbol{x}_B) = \boldsymbol{\mu}_A + \boldsymbol{\Sigma}_{AB} \boldsymbol{\Sigma}_{BB}^{-1} (\boldsymbol{x}_B - \boldsymbol{\mu}_B). \tag{1.30}$$

Then $\boldsymbol{a} - \boldsymbol{\Sigma}_{AB} \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{b} = \boldsymbol{x}_A - \boldsymbol{\mu}_{A|B}(\boldsymbol{x}_B)$, and expanding the quadratic

$$(\boldsymbol{a} - \boldsymbol{\Sigma}_{AB} \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{b})^\top \boldsymbol{\Sigma}_{A|B}^{-1} (\boldsymbol{a} - \boldsymbol{\Sigma}_{AB} \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{b})$$

reproduces exactly the three terms in (1.29). Therefore $E = [\boldsymbol{x}_A - \boldsymbol{\mu}_{A|B}(\boldsymbol{x}_B)]^\top \boldsymbol{\Sigma}_{A|B}^{-1} [\boldsymbol{x}_A - \boldsymbol{\mu}_{A|B}(\boldsymbol{x}_B)]$, and combining with (1.28),

$$f_{\boldsymbol{X}_A | \boldsymbol{X}_B = \boldsymbol{x}_B}(\boldsymbol{x}_A) = \frac{1}{(2\pi)^{q/2} |\boldsymbol{\Sigma}_{A|B}|^{1/2}} \exp\Big\{ -\tfrac{1}{2} [\boldsymbol{x}_A - \boldsymbol{\mu}_{A|B}(\boldsymbol{x}_B)]^\top \boldsymbol{\Sigma}_{A|B}^{-1} [\boldsymbol{x}_A - \boldsymbol{\mu}_{A|B}(\boldsymbol{x}_B)] \Big\}. \tag{1.31}$$

Theorem 1.6 (Conditional distribution of a multivariate normal)**.** If $\boldsymbol{X} = (\boldsymbol{X}_A, \boldsymbol{X}_B) \sim \mathrm{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ is partitioned as in (1.23), then

$$\boxed{\boldsymbol{X}_A \mid (\boldsymbol{X}_B = \boldsymbol{x}_B) \;\sim\; \mathrm{N}_q(\boldsymbol{\mu}_{A|B}(\boldsymbol{x}_B),\; \boldsymbol{\Sigma}_{A|B}),} \tag{1.32}$$

with $\boldsymbol{\mu}_{A|B}(\boldsymbol{x}_B)$ as in (1.30) and $\boldsymbol{\Sigma}_{A|B}$ as in (1.25).

Now read the formula before using it. The conditional mean is **affine** in $\boldsymbol{x}_B$: it starts at $\boldsymbol{\mu}_A$ and adjusts according to the observed deviation $\boldsymbol{x}_B - \boldsymbol{\mu}_B$. The conditional covariance is constant in those observed values. Also, $\boldsymbol{\Sigma}_{AB} \boldsymbol{\Sigma}_{BB}^{-1} \boldsymbol{\Sigma}_{AB}^\top$ is positive semidefinite, so conditioning reduces covariance in the matrix sense $\boldsymbol{\Sigma}_{A|B} \preceq \boldsymbol{\Sigma}_{AA}$. This conclusion concerns the Gaussian model with its parameters fixed. The narrowing ends of an ellipse do not imply a smaller conditional variance at extreme predictor values: a contour and a conditional density are different objects.

Key idea.
Conditioning a multivariate normal on some of its coordinates gives another multivariate normal, with mean linear in the conditioning values and covariance equal to

the Schur complement. The conditional covariance does not depend on the observed values.

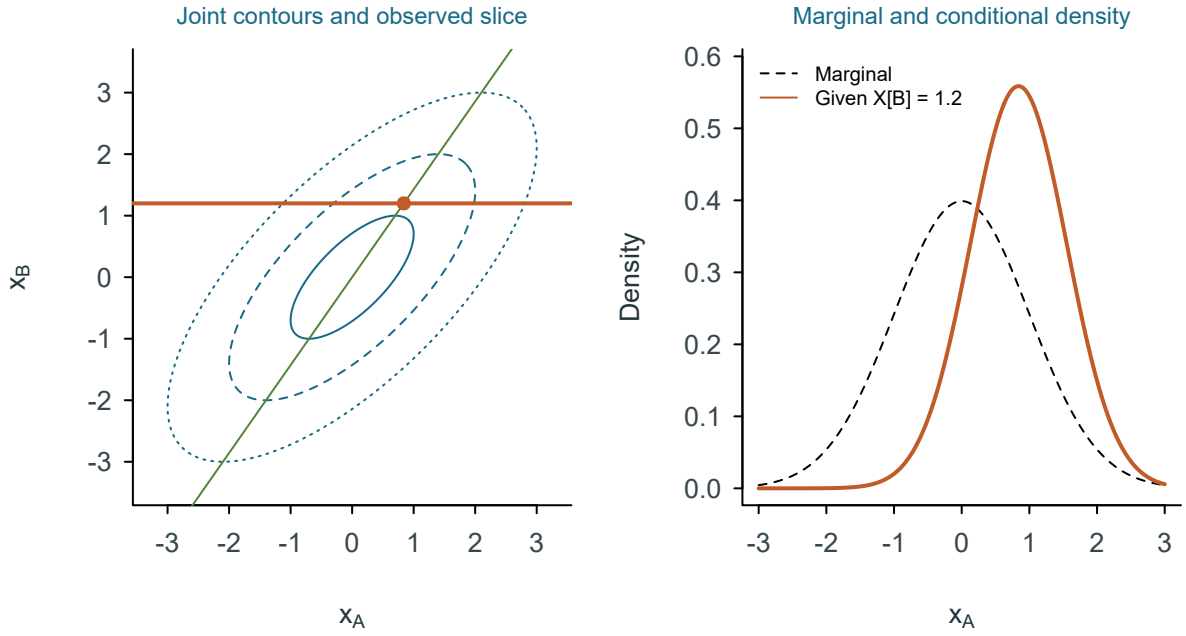


Figure 1.2: A bivariate normal slice and the corresponding conditional density. Conditioning changes the mean and reduces the variance.

The conditional mean is the best predictor under squared-error loss whenever it exists, including outside the Gaussian family. What joint normality supplies is this particular affine formula and the accompanying constant conditional covariance. Replacing a missing value by its conditional mean also suppresses residual variability; it does not turn the missing measurement into an observed one.

1.5.4 Residuals and a shorter derivation

Given the theorem, one can define a vector of *residuals* per observation,

$$\boldsymbol{X}_{A\cdot B} = \boldsymbol{X}_A - \boldsymbol{\mu}_{A|B}(\boldsymbol{X}_B), \tag{1.33}$$

This is the part of $\boldsymbol{X}_A$ left after its population conditional mean has been removed. Under joint normality it is independent of $\boldsymbol{X}_B$. Do not transfer that conclusion automatically to residuals from a model fitted to the same sample. OLS gives sample orthogonality, $\boldsymbol{X}^\top \widehat{\boldsymbol{e}} = 0$, but orthogonality is not independence. For a random design and homoscedastic Gaussian errors, $\widehat{\boldsymbol{e}} \mid \boldsymbol{X}$ has covariance $\sigma^2(\boldsymbol{I} - \boldsymbol{H}_X)$, which generally depends on the design through its projection matrix. That dependence is precisely why a population-residual statement and a fitted-residual statement need separate arguments.

Ordinary least squares of a response Y on a predictor X minimises $\sum_i (y_i - a - bx_i)^2$ and treats the observed x_i as fixed. The unconstrained bivariate normal, fitted to the same complete pairs, estimates $(\widehat{\mu}_X, \widehat{\mu}_Y)$ and a covariance with a single divisor in every entry, whether that divisor is n or $n-1$. Its conditional mean of Y given $X = x$ is

$$\widehat{\mu}_Y + \frac{s_{YX}}{s_{XX}}(x - \widehat{\mu}_X).$$

The slope s_{YX}/s_{XX} and the intercept $\widehat{\mu}_Y - (s_{YX}/s_{XX})\widehat{\mu}_X$ are exactly the least-squares slope and intercept. The Gaussian fit and the least-squares fit draw the same conditional-mean line. They do not estimate the same object. Least squares does not model the marginal distribution of X, and it does not by itself supply the conditional variance $s_{YY} - s_{YX}^2/s_{XX}$. The major axis of the fitted ellipse is a different line again, as is the line obtained by regressing X on Y and solving for Y. The reverse-regression line, when it can be solved for Y, agrees with the forward regression only under perfect correlation. A principal-axis line can coincide in special cases, such as an uncorrelated cloud whose larger variance is horizontal; there is no universal rule that all three lines differ. The laboratory below checks the agreement on setosa.

The residuals have two useful properties, both direct from (1.12)–(1.14). Write $\boldsymbol{X}_{A\cdot B} - \mathbb{E}(\boldsymbol{X}_{A\cdot B}) = \boldsymbol{X}_A - \boldsymbol{\mu}_A - \boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1}(\boldsymbol{X}_B - \boldsymbol{\mu}_B) = \boldsymbol{M}(\boldsymbol{X} - \boldsymbol{\mu})$ with $\boldsymbol{M} = (\boldsymbol{I}_q,\ -\boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1})$. Then

$$\begin{aligned}\operatorname{Cov}(\boldsymbol{X}_{A\cdot B}, \boldsymbol{X}_{A\cdot B}^\top) &= \boldsymbol{M}\boldsymbol{\Sigma}\boldsymbol{M}^\top = \boldsymbol{\Sigma}_{AA} - \boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1}\boldsymbol{\Sigma}_{AB}^\top - \boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1}\boldsymbol{\Sigma}_{AB}^\top + \boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1}\boldsymbol{\Sigma}_{AB}^\top \\ &= \boldsymbol{\Sigma}_{A|B}, \\ \operatorname{Cov}(\boldsymbol{X}_{A\cdot B}, \boldsymbol{X}_B^\top) &= \boldsymbol{\Sigma}_{AB} - \boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1}\boldsymbol{\Sigma}_{BB} = \boldsymbol{0}.\end{aligned}$$

Since $(\boldsymbol{X}_{A\cdot B}, \boldsymbol{X}_B)$ is a linear image of a normal vector and the cross-covariance is zero, the residuals are independent of the conditioning variables. This gives a second proof of Theorem 1.6: $\boldsymbol{X}_A = \boldsymbol{\mu}_{A|B}(\boldsymbol{X}_B) + \boldsymbol{X}_{A\cdot B}$ with $\boldsymbol{X}_{A\cdot B} \sim \mathrm{N}_q(\boldsymbol{0}, \boldsymbol{\Sigma}_{A|B})$ independent of $\boldsymbol{X}_B$, so given $\boldsymbol{X}_B = \boldsymbol{x}_B$ the first term is the constant $\boldsymbol{\mu}_{A|B}(\boldsymbol{x}_B)$ and the second keeps its distribution.

1.5.5 Partial variances and correlations

When the variables $X_{q+1}, \ldots, X_p$ are held constant, the entries of $\boldsymbol{\Sigma}_{A|B}$ are called *partial variances and covariances*. Write $\sigma_{jk|q+1,\ldots,p}$ for the (j,k) entry $(1 \le j,k \le q)$. The (j,k) *partial correlation* is

$$\rho_{jk|q+1,\ldots,p} = \frac{\sigma_{jk|q+1,\ldots,p}}{\sqrt{\sigma_{jj|q+1,\ldots,p}\,\sigma_{kk|q+1,\ldots,p}}}. \tag{1.34}$$

If $\boldsymbol{D}_{A|B} = \operatorname{diag}(\boldsymbol{\Sigma}_{A|B})$, the matrix of partial correlations is

$$\boldsymbol{P}_{A|B} = \boldsymbol{D}_{A|B}^{-1/2}\,\boldsymbol{\Sigma}_{A|B}\,\boldsymbol{D}_{A|B}^{-1/2}, \tag{1.35}$$

where the power $-1/2$ of a diagonal matrix takes the reciprocal square root of each diagonal entry. Partial correlations reappear in Chapter 19, where they are read directly from the inverse covariance matrix.

Within this Gaussian model, partial correlation describes the association between the remaining coordinates after conditioning on the other coordinates. Equivalently, it is the correlation of their population linear-regression residuals. Outside the Gaussian family, residual partial correlation remains well defined when the moments and regressions exist, but it need not equal a conditional correlation at every conditioning value. In R, `cov2cor` standardises a covariance matrix; it cannot establish the assumptions behind a conditional interpretation.

Example 1.7 (A trivariate conditional)**.** Let $\boldsymbol{X} = (X_1, X_2, X_3)^\top \sim \mathrm{N}_3(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with

$$\boldsymbol{\mu} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \qquad \boldsymbol{\Sigma} = \begin{pmatrix} 4 & 2 & 1 \\ 2 & 3 & 0.5 \\ 1 & 0.5 & 2 \end{pmatrix}.$$

Question. Find the distribution of (X_1, X_2) given $X_3 = x_3$, and the partial correlation of X_1 and X_2 given X_3.

Solution. Take $A = \{1, 2\}$ and $B = \{3\}$. Then $\boldsymbol{\Sigma}_{AB} = (1,\, 0.5)^\top$ and $\boldsymbol{\Sigma}_{BB} = 2$, so $\boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1} = (0.5,\, 0.25)^\top$ and

$$\boldsymbol{\mu}_{A|B}(x_3) = \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0.5 \\ 0.25 \end{pmatrix}(x_3 - 3) = \begin{pmatrix} 1 + 0.5(x_3 - 3) \\ 2 + 0.25(x_3 - 3) \end{pmatrix}.$$

The Schur complement is

$$\boldsymbol{\Sigma}_{A|B} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix} - \begin{pmatrix} 0.5 \\ 0.25 \end{pmatrix}\begin{pmatrix} 1 & 0.5 \end{pmatrix} = \begin{pmatrix} 4 - 0.5 & 2 - 0.25 \\ 2 - 0.25 & 3 - 0.125 \end{pmatrix} = \begin{pmatrix} 3.5 & 1.75 \\ 1.75 & 2.875 \end{pmatrix}.$$

The partial correlation is $\rho_{12|3} = 1.75/\sqrt{3.5 \times 2.875} = 1.75/\sqrt{10.0625} = 0.5517$, slightly below the marginal correlation $\rho_{12} = 2/\sqrt{12} = 0.5774$: the association remaining after conditioning on X_3 is weaker in this model. This is a statistical adjustment, not evidence that X_3 causes either measurement.

1.6 R laboratory: one species or three?

The four iris measurements share centimetre units, but pooling species changes the question. Compare the covariance of all flowers with the covariance within setosa. A single Gaussian approximation to the pooled data ignores the visible species structure. The same setosa flowers also check the claim above: the slope s_{YX}/s_{XX} from the sample covariance must match the slope from `lm`.

```
round(cov(iris_x), 3)
#>              Sepal.Length Sepal.Width Petal.Length Petal.Width
#> Sepal.Length        0.686      -0.042        1.274       0.516
#> Sepal.Width        -0.042       0.190       -0.330      -0.122
#> Petal.Length        1.274      -0.330        3.116       1.296
#> Petal.Width         0.516      -0.122        1.296       0.581
round(cov(subset(iris, Species == "setosa")[, 1:4]), 3)
#>              Sepal.Length Sepal.Width Petal.Length Petal.Width
#> Sepal.Length        0.124       0.099        0.016       0.010
#> Sepal.Width         0.099       0.144        0.012       0.009
#> Petal.Length        0.016       0.012        0.030       0.006
#> Petal.Width         0.010       0.009        0.006       0.011
aggregate(. ~ Species, data = iris, FUN = mean)
#>      Species Sepal.Length Sepal.Width Petal.Length Petal.Width
#> 1     setosa        5.006       3.428        1.462       0.246
#> 2 versicolor        5.936       2.770        4.260       1.326
#> 3  virginica        6.588       2.974        5.552       2.026

setosa <- subset(iris, Species == "setosa")
x <- setosa$Sepal.Length
y <- setosa$Sepal.Width
ols <- coef(lm(y ~ x))
S <- cov(cbind(x, y))
slope <- S[2, 1] / S[1, 1]
gauss <- c(intercept = mean(y) - slope * mean(x), slope = slope)
rbind(ols = unname(ols), gaussian = unname(gauss))
```

```
#>              [,1]    [,2]
#> ols      -0.5694 0.7985
#> gaussian -0.5694 0.7985
stopifnot(max(abs(unname(ols) - unname(gauss))) < 1e-10)
```

```
pairs(iris[, 1:4], col = palette_book[as.integer(iris_y)],
      pch = 19, cex = 0.55, lower.panel = NULL)
```

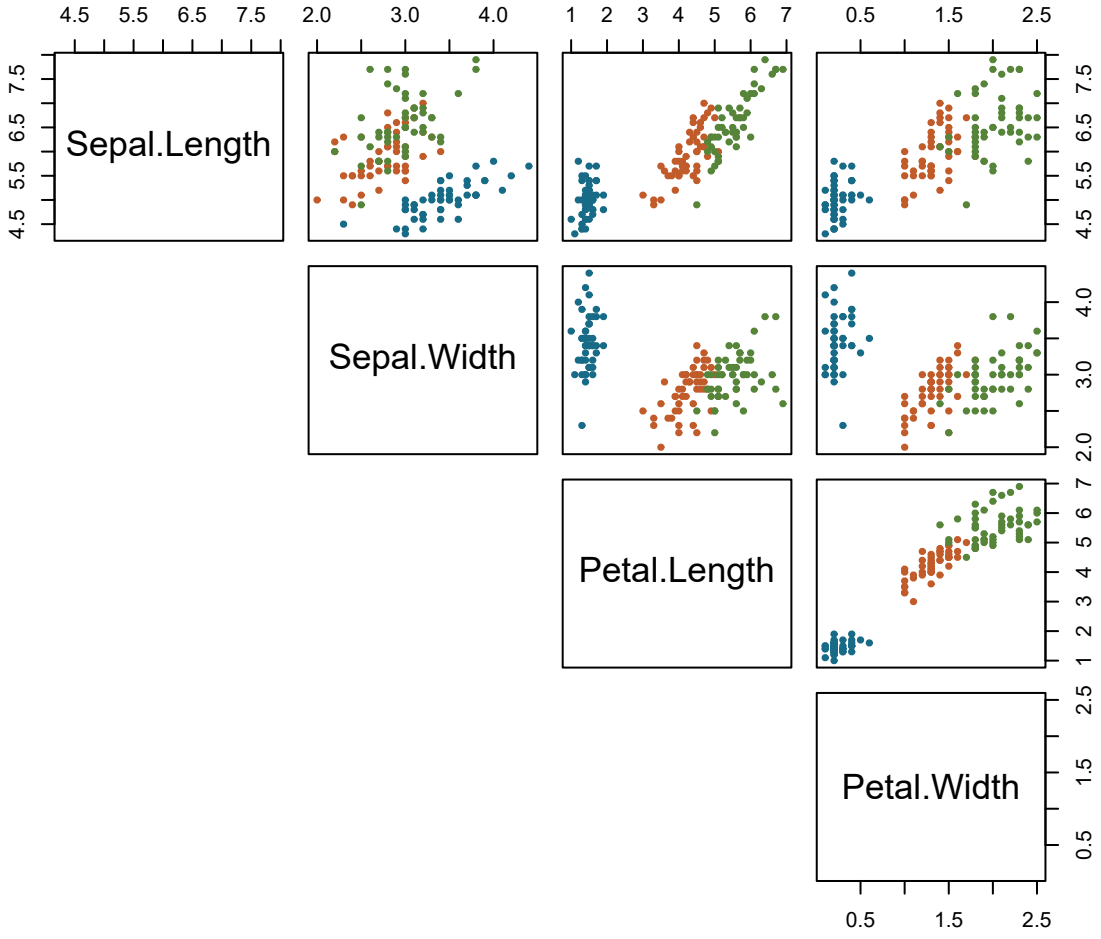


Figure 1.3: Iris measurements coloured by species. The colours are observed labels; no model has been fitted.

The pooled covariance includes both within-species variation and variation between species means. The law of total covariance makes this explicit: $\mathrm{Cov}(X) = E\{\mathrm{Cov}(X \mid G)\} + \mathrm{Cov}\{E(X \mid G)\}$. A strong pooled correlation can therefore reflect differences between groups.

1.7 Further exercises

Exercise 1.1 (Why covariance matrices are positive semi-definite)**.** Show that for any random vector with finite second moments, $\boldsymbol{z}^\top \boldsymbol{\Sigma} \boldsymbol{z} = \mathrm{Var}(\boldsymbol{z}^\top \boldsymbol{X}) \geq 0$. Deduce $|\sigma_{jk}| \leq \sqrt{\sigma_{jj}\sigma_{kk}}$ from the

2×2 principal minor. Use Sylvester's criterion to decide whether

$$\begin{pmatrix} 1 & 0.9 & 0.9 \\ 0.9 & 1 & -0.9 \\ 0.9 & -0.9 & 1 \end{pmatrix}$$

could be a correlation matrix.

Solution

Exercise 1.2 (Linear combinations)**.** Let $\boldsymbol{X}$ have covariance matrix $\boldsymbol{\Sigma} = \begin{pmatrix} 4 & 2 & 1 \\ 2 & 3 & 0.5 \\ 1 & 0.5 & 2 \end{pmatrix}$. Compute $\operatorname{Var}(X_1 - 2X_2 + X_3)$ and $\operatorname{Cov}(X_1 + X_2,\ X_2 - X_3)$ using (1.10) and (1.11).

Solution

Exercise 1.3 (Uncorrelated but dependent)**.** Let $X \sim \mathrm{N}(0,1)$ and $Y = X^2$. Show $\operatorname{Cov}(X, Y) = 0$ although Y is a function of X.

Solution

Exercise 1.4 (Axes of a contour)**.** For $\boldsymbol{\Sigma} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ and $\boldsymbol{\mu} = \mathbf{0}$, find the eigenvalues and eigenvectors of $\boldsymbol{\Sigma}$, the directions and lengths of the semi-axes of the contour $\boldsymbol{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{x} = C$, and the value of C for which the ellipse contains probability 0.95.

Solution

Exercise 1.5 (Normal marginals without joint normality)**.** Let $X \sim \mathrm{N}(0,1)$ and let W be independent of X with $\mathbb{P}(W = 1) = \mathbb{P}(W = -1) = \tfrac{1}{2}$. Put $Y = WX$. Show that $Y \sim \mathrm{N}(0,1)$ and $\operatorname{Cov}(X, Y) = 0$, but that (X, Y) is not bivariate normal. (Hint: consider $X + Y$.)

Solution

Exercise 1.6 (The bivariate conditional)**.** Specialise Theorem 1.6 to $p = 2$. Show that

$$X_1 \mid X_2 = x_2 \ \sim\ \mathrm{N}\Big(\mu_1 + \rho \frac{\sigma_1}{\sigma_2}(x_2 - \mu_2),\ \sigma_1^2(1 - \rho^2)\Big),$$

and explain why the regression line of X_1 on X_2 is not the same as the line obtained by solving the regression of X_2 on X_1 for x_2.

Solution

Exercise 1.7 (Schur-complement check)**.** For the matrix of Example 1.7, verify the determinant identity (1.27) with $A = \{1, 2\}$, $B = \{3\}$ by computing $|\boldsymbol{\Sigma}|$ directly and comparing it with $|\boldsymbol{\Sigma}_{BB}|\,|\boldsymbol{\Sigma}_{A|B}|$.

Solution

Exercise 1.8 (Conditional normal prediction)**.** For a jointly normal vector $(X_1, X_2)^T$, the mean is $(2, -1)^T$ and the covariance matrix is

$$\Sigma = \begin{pmatrix} 9 & 3 \\ 3 & 4 \end{pmatrix}.$$

You may use $X_A \mid X_B = x_B \sim N(\mu_A + \Sigma_{AB}\Sigma_{BB}^{-1}(x_B - \mu_B), \Sigma_{AA} - \Sigma_{AB}\Sigma_{BB}^{-1}\Sigma_{BA})$.

(a) Determine the conditional distribution of X_1 given $X_2 = 3$. Show the substitution, not just the final parameters.

(b) Let $R = X_1 - E(X_1 \mid X_2)$. State its distribution and explain whether it is independent of X_2.

(c) Would the same independence conclusion follow from the covariance matrix alone without joint normality? Explain.

Solution

Chapter 2

Estimation and inference for the multivariate normal

In the previous chapter we could calculate a conditional distribution because the population mean and covariance were given. With real data, they usually aren't. We have a sample, and we need to estimate them. We also need to describe how much those estimates would change if we repeated the sampling. A point estimate and its uncertainty answer different questions.

We'll follow one argument from beginning to end: write the Gaussian likelihood, estimate its parameters, derive a test of a proposed mean vector, and invert that test to obtain a confidence region. Watch the covariance carefully. Individual observations have covariance $\boldsymbol{\Sigma}$; the average of n independent observations has covariance $\boldsymbol{\Sigma}/n$. More data locate the centre more accurately without making individual observations less variable. The exact test requires independent Gaussian observations and $n > p$; the same estimated means and pooled covariance will later appear in discriminant analysis.

Source. Lecture notes, Sections 1.6–1.7 (pp. 16–29), with Appendix A on maximum likelihood estimation (pp. 158–160). The Hotelling construction follows Anderson (2003).

2.1 The likelihood of a normal sample

A likelihood reads a familiar density in a different direction. Hold the observed sample fixed and vary the parameters. Values giving greater density to this sample have greater likelihood, although that statement alone assigns no probability distribution to the parameters. For continuous data the likelihood is built from density values, which may exceed one. We will use *estimator* for the rule applied to a random sample and *estimate* for its realised numerical value. Sampling uncertainty belongs to the former.

Model the sample as n independent draws $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$ from $\mathrm{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, and stack the realised vectors as the rows of a data matrix,

$$\boldsymbol{X}_{\text{all}} = \begin{pmatrix} x_{11} & \cdots & x_{1p} \\ \vdots & & \vdots \\ x_{n1} & \cdots & x_{np} \end{pmatrix} = \begin{pmatrix} \boldsymbol{x}_1^\top \\ \vdots \\ \boldsymbol{x}_n^\top \end{pmatrix}. \tag{2.1}$$

Independence multiplies the n normal densities (1.19). Hold the rows fixed and regard the product as a function of $(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. That function is the likelihood,

$$L(\boldsymbol{\mu}, \boldsymbol{\Sigma}; \boldsymbol{X}_{\text{all}}) = \frac{1}{(2\pi)^{np/2}|\boldsymbol{\Sigma}|^{n/2}} \exp\Big\{ -\frac{1}{2}\sum_{i=1}^{n}(\boldsymbol{x}_i - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_i - \boldsymbol{\mu})\Big\}. \tag{2.2}$$

Appendix A.1 records the general definition of the maximum likelihood estimator. Because log is strictly increasing, L and $\ell = \log L$ have the same maximiser, and the derivative is easier to take on the logarithm:

$$\ell(\boldsymbol{\mu}, \boldsymbol{\Sigma}; \boldsymbol{X}_{\text{all}}) = -\tfrac{1}{2}np\ln(2\pi) - \tfrac{1}{2}n\ln|\boldsymbol{\Sigma}| - \tfrac{1}{2}\sum_{i=1}^{n}(\boldsymbol{x}_i - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_i - \boldsymbol{\mu}). \tag{2.3}$$

Taking logarithms changes the arithmetic without changing the maximiser. Products become sums, which are easier to differentiate and usually safer to compute. Terms independent of the parameter can be dropped while finding its maximiser. Keep them when an absolute density value or a comparison requiring the full likelihood is needed.

Two summaries of the sample carry the maximiser. The *sample mean vector* is the candidate for $\boldsymbol{\mu}$,

$$\overline{\boldsymbol{x}} = \frac{1}{n}\sum_{i=1}^{n}\boldsymbol{x}_i, \tag{2.4}$$

and the *matrix of corrected sums of squares and cross-products* is the raw material for $\boldsymbol{\Sigma}$,

$$\boldsymbol{A} = \sum_{i=1}^{n}(\boldsymbol{x}_i - \overline{\boldsymbol{x}})(\boldsymbol{x}_i - \overline{\boldsymbol{x}})^\top = \Big(\sum_{i=1}^{n}\boldsymbol{x}_i\boldsymbol{x}_i^\top\Big) - n\,\overline{\boldsymbol{x}}\,\overline{\boldsymbol{x}}^\top. \tag{2.5}$$

The second expression is the first one expanded. The cross terms cancel because $\sum_i \boldsymbol{x}_i = n\overline{\boldsymbol{x}}$, which is the definition of the sample mean. In particular $\boldsymbol{A}$ does not depend on $(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.

2.1.1 The mean

The log-likelihood depends on $\boldsymbol{\mu}$ only through the sum of quadratic forms, and $\boldsymbol{\Sigma}$ is still an arbitrary positive definite matrix. Split each residual into a deviation from the sample mean and a deviation of the sample mean from $\boldsymbol{\mu}$:

$$\begin{aligned}\sum_{i=1}^{n}(\boldsymbol{x}_i - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_i - \boldsymbol{\mu}) &= \sum_{i=1}^{n}\big[(\boldsymbol{x}_i - \overline{\boldsymbol{x}}) + (\overline{\boldsymbol{x}} - \boldsymbol{\mu})\big]^\top \boldsymbol{\Sigma}^{-1}\big[(\boldsymbol{x}_i - \overline{\boldsymbol{x}}) + (\overline{\boldsymbol{x}} - \boldsymbol{\mu})\big] \\ &= \sum_{i=1}^{n}(\boldsymbol{x}_i - \overline{\boldsymbol{x}})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_i - \overline{\boldsymbol{x}}) + n(\overline{\boldsymbol{x}} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\overline{\boldsymbol{x}} - \boldsymbol{\mu}),\end{aligned}$$

The cross terms contain $\sum_i(\boldsymbol{x}_i - \overline{\boldsymbol{x}}) = \boldsymbol{0}$, so they vanish. That sum is zero by the definition of the sample mean. The first sum does not involve $\boldsymbol{\mu}$. The second is n times a quadratic form in the positive definite matrix $\boldsymbol{\Sigma}^{-1}$, so it is nonnegative and equals zero only at $\boldsymbol{\mu} = \overline{\boldsymbol{x}}$. The maximiser in $\boldsymbol{\mu}$ is therefore

$$\widehat{\boldsymbol{\mu}} = \overline{\boldsymbol{x}}, \tag{2.6}$$

for every positive definite $\boldsymbol{\Sigma}$. The covariance does not have to be known before the mean is estimated. The next step substitutes this maximiser and maximises in $\boldsymbol{\Sigma}$ alone.

2.1.2 The covariance matrix and the invariance lemma

The inverse covariance is a more convenient optimisation variable. Set $\boldsymbol{\Theta} = \boldsymbol{\Sigma}^{-1}$. The quadratic part of the log-likelihood is linear in $\boldsymbol{\Theta}$, while its determinant term becomes $\log|\boldsymbol{\Theta}|$. Together they form a concave function on the positive-definite matrices. We can find its maximiser and invert it. The invariance lemma explains why this gives the covariance MLE (Zehna, 1966; Mood et al., 1974).

Lemma 2.1 (Invariance of the MLE)**.** Let the distribution of the data depend on parameters $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_k)$ with maximum likelihood estimator $\widehat{\boldsymbol{\theta}}$, and let $\boldsymbol{\tau}(\boldsymbol{\theta}) = (\tau_1(\boldsymbol{\theta}), \ldots, \tau_r(\boldsymbol{\theta}))$, $1 \le r \le k$, be a transformation of the parameter space Θ. Then the maximum likelihood estimator of $\tau_l(\boldsymbol{\theta})$ is $\widehat{\tau}_l = \tau_l(\widehat{\boldsymbol{\theta}})$.

Proof. Let $M(\boldsymbol{\tau}; \boldsymbol{x}) = \sup_{\boldsymbol{\theta}:\, \boldsymbol{\tau}(\boldsymbol{\theta})=\boldsymbol{\tau}} L(\boldsymbol{\theta}; \boldsymbol{x})$ be the likelihood induced on the transformed parameter. For any value $\boldsymbol{\tau}$ in the range of the transformation,

$$\sup_{\boldsymbol{\theta}:\, \boldsymbol{\tau}(\boldsymbol{\theta})=\boldsymbol{\tau}} L(\boldsymbol{\theta}; \boldsymbol{x}) \le \sup_{\boldsymbol{\theta}\in\Theta} L(\boldsymbol{\theta}; \boldsymbol{x}) = L(\widehat{\boldsymbol{\theta}}; \boldsymbol{x}) = \sup_{\boldsymbol{\theta}:\, \boldsymbol{\tau}(\boldsymbol{\theta})=\boldsymbol{\tau}(\widehat{\boldsymbol{\theta}})} L(\boldsymbol{\theta}; \boldsymbol{x}) = M(\boldsymbol{\tau}(\widehat{\boldsymbol{\theta}}); \boldsymbol{x}).$$

The last equality holds because $\widehat{\boldsymbol{\theta}}$ itself satisfies the constraint, so the supremum over that constraint set is attained at $\widehat{\boldsymbol{\theta}}$. Hence $M(\boldsymbol{\tau}(\widehat{\boldsymbol{\theta}})) \ge M(\boldsymbol{\tau})$ for all $\boldsymbol{\tau}$, so $\boldsymbol{\tau}(\widehat{\boldsymbol{\theta}})$ maximises M.

□

Apply the lemma with $\boldsymbol{\theta} = \boldsymbol{\Sigma}^{-1} =: \boldsymbol{\Theta}$. The free coordinates θ_l are the $k = p(p+1)/2$ distinct entries of the precision, and the transformed coordinates τ_l are the distinct entries of $\boldsymbol{\Sigma} = \boldsymbol{\Theta}^{-1}$. Substitute $\boldsymbol{\Sigma}^{-1} = \boldsymbol{\Theta}$ and $\ln|\boldsymbol{\Sigma}| = -\ln|\boldsymbol{\Theta}|$ into (2.3):

$$\ell(\boldsymbol{\mu}, \boldsymbol{\Theta}) = -\tfrac{1}{2} np \ln(2\pi) + \tfrac{1}{2} n \ln|\boldsymbol{\Theta}| - \tfrac{1}{2} \sum_{i=1}^{n} (\boldsymbol{x}_i - \boldsymbol{\mu})^\top \boldsymbol{\Theta} (\boldsymbol{x}_i - \boldsymbol{\mu}).$$

The derivative with respect to a symmetric matrix uses $\partial \ln|\boldsymbol{\Theta}| / \partial \boldsymbol{\Theta} = \boldsymbol{\Theta}^{-1}$ and $\partial(\boldsymbol{v}^\top \boldsymbol{\Theta} \boldsymbol{v}) / \partial \boldsymbol{\Theta} = \boldsymbol{v}\boldsymbol{v}^\top$ (Petersen and Pedersen, 2012, identities (57) and (70)). Those two identities give

$$\frac{\partial \ell}{\partial \boldsymbol{\Theta}} = \frac{n}{2} \boldsymbol{\Theta}^{-1} - \frac{1}{2} \sum_{i=1}^{n} (\boldsymbol{x}_i - \boldsymbol{\mu})(\boldsymbol{x}_i - \boldsymbol{\mu})^\top.$$

The mean has already been maximised. Set $\boldsymbol{\mu} = \widehat{\boldsymbol{\mu}} = \overline{\boldsymbol{x}}$ from (2.6) and set the derivative equal to zero:

$$\frac{n}{2} \widehat{\boldsymbol{\Theta}}^{-1} - \frac{1}{2} \sum_{i=1}^{n} (\boldsymbol{x}_i - \overline{\boldsymbol{x}})(\boldsymbol{x}_i - \overline{\boldsymbol{x}})^\top = \boldsymbol{0} \quad \Longrightarrow \quad n \widehat{\boldsymbol{\Theta}}^{-1} - \boldsymbol{A} = \boldsymbol{0} \quad \Longrightarrow \quad \widehat{\boldsymbol{\Theta}}^{-1} = \frac{1}{n} \boldsymbol{A}.$$

Lemma 2.1, applied to $\boldsymbol{\Theta} \mapsto \boldsymbol{\Theta}^{-1}$, says that the maximum likelihood estimator of $\boldsymbol{\Sigma}$ is

$$\widehat{\boldsymbol{\Sigma}} = \frac{1}{n} \boldsymbol{A}. \tag{2.7}$$

There is an important condition behind that answer: $\boldsymbol{A}$ must be positive definite. Then strict concavity in the precision makes the stationary point the unique maximum. If $\boldsymbol{A}$ is singular, the unrestricted positive-definite Gaussian covariance has no finite interior MLE; shrinking the variance in an unobserved direction increases the likelihood without bound. An algebraic expression such as $\boldsymbol{A}/n$ is not automatically an admissible parameter. When the MLE does exist, its divisor is n. We now ask a different question: is that estimator unbiased?

2.1.3 Bias and the sample covariance matrix

Whether $\widehat{\boldsymbol{\Sigma}}$ is unbiased is not settled by the likelihood equation. Expand the cross-product matrix about the true mean, $\boldsymbol{A} = \sum_i (\boldsymbol{X}_i - \boldsymbol{\mu})(\boldsymbol{X}_i - \boldsymbol{\mu})^\top - n(\overline{\boldsymbol{X}} - \boldsymbol{\mu})(\overline{\boldsymbol{X}} - \boldsymbol{\mu})^\top$. If the observations are uncorrelated, with common mean $\boldsymbol{\mu}$ and common covariance $\boldsymbol{\Sigma}$, each outer product $(\boldsymbol{X}_i - \boldsymbol{\mu})(\boldsymbol{X}_i - \boldsymbol{\mu})^\top$ has expectation $\boldsymbol{\Sigma}$, so the first sum has expectation $n\boldsymbol{\Sigma}$. The same uncorrelatedness gives $\mathrm{Var}(\overline{\boldsymbol{X}}) = \boldsymbol{\Sigma}/n$, and the subtracted term then has expectation $\boldsymbol{\Sigma}$. Therefore

$$\mathbb{E}(\boldsymbol{A}) = (n-1)\boldsymbol{\Sigma}, \qquad \mathbb{E}(\widehat{\boldsymbol{\Sigma}}) = \frac{n-1}{n}\,\boldsymbol{\Sigma}. \tag{2.8}$$

Dividing $\boldsymbol{A}$ by n estimates $\boldsymbol{\Sigma}$ by $(n-1)/n$ times $\boldsymbol{\Sigma}$ on average, so the maximum likelihood estimator is biased low. The unbiased matrix in common use divides by the degrees of freedom that remain after estimating the mean. It is the *sample covariance matrix*

$$\boldsymbol{S} = \frac{1}{n-1}\boldsymbol{A}. \tag{2.9}$$

The two estimates use the same measured scatter and differ only in their divisor. Dividing by n solves the likelihood problem; dividing by $n-1$ corrects the expectation. Neither calculation contradicts the other. We will use $\boldsymbol{S} = \boldsymbol{A}/(n-1)$ in Hotelling's statistic because the stated F calibration is derived for that convention. Substituting a different divisor without adjusting the calibration changes the test.

Several groups.

A single mean is the wrong centre when the observations fall into G groups that may have different means and are assumed to share one covariance $\boldsymbol{\Sigma}$. Linear discriminant analysis uses that structure. Let group g have n_g observations $\boldsymbol{x}_{ig}$, $i = 1, \ldots, n_g$, with $n = \sum_g n_g$. Independence across groups lets the argument for (2.6) run inside each group, so the maximum likelihood estimator of $\boldsymbol{\mu}_g$ is the group mean $\overline{\boldsymbol{x}}_g$. The unbiased estimator of the common $\boldsymbol{\Sigma}$ pools the corrected cross-product matrices. It is the *pooled* sample covariance matrix

$$\boldsymbol{S} = \frac{1}{n-G}\sum_{g=1}^{G}\sum_{i=1}^{n_g}(\boldsymbol{x}_{ig} - \overline{\boldsymbol{x}}_g)(\boldsymbol{x}_{ig} - \overline{\boldsymbol{x}}_g)^\top = \frac{1}{n-G}\sum_{g=1}^{G}\boldsymbol{A}_g. \tag{2.10}$$

Each estimated group mean consumes one degree of freedom, which is why the divisor is $n-G$ rather than $n-1$. The one-sample matrix (2.9) is the case $G = 1$.

2.2 The Wishart distribution

Knowing $\mathbb{E}(\boldsymbol{A})$ does not tell us how much $\boldsymbol{A}$ fluctuates. For an exact test we need its sampling distribution. Under Gaussian sampling, the centred cross-products have a Wishart distribution with $n-1$ degrees of freedom (Wishart, 1928). Think of a chi-squared variable first: it is a sum of squares of independent standard normals. Wishart replaces each square by a vector outer product.

Definition 2.2 (Wishart distribution)**.** Let $\boldsymbol{U}_1, \ldots, \boldsymbol{U}_\nu$ be independent $\mathrm{N}_p(\boldsymbol{0}, \boldsymbol{\Sigma})$ vectors with $\nu \geq p$ and $\boldsymbol{\Sigma}$ positive definite. The matrix $\boldsymbol{W} = \sum_{m=1}^{\nu} \boldsymbol{U}_m \boldsymbol{U}_m^\top$ has the Wishart distribution $\boldsymbol{W} \sim \mathrm{W}_p(\boldsymbol{\Sigma}, \nu)$ with density

$$w(\boldsymbol{W}; \boldsymbol{\Sigma}, \nu) = \frac{|\boldsymbol{W}|^{(\nu-p-1)/2}\exp\{-\frac{1}{2}\operatorname{tr}(\boldsymbol{W}\boldsymbol{\Sigma}^{-1})\}}{2^{\nu p/2}\,\pi^{p(p-1)/4}\,|\boldsymbol{\Sigma}|^{\nu/2}\prod_{j=1}^{p}\Gamma\!\left(\frac{\nu+1-j}{2}\right)} \tag{2.11}$$

for positive-definite $\boldsymbol{W}$, and 0 otherwise.

For the corrected cross-product matrix of a normal sample the degrees of freedom are $\nu = n - 1$, and the density is

$$w(\boldsymbol{A}; \boldsymbol{\Sigma}, n-1) = \frac{|\boldsymbol{A}|^{(n-p-2)/2} \exp\{-\frac{1}{2}\operatorname{tr}(\boldsymbol{A}\boldsymbol{\Sigma}^{-1})\}}{2^{(n-1)p/2}\pi^{p(p-1)/4}|\boldsymbol{\Sigma}|^{(n-1)/2}\prod_{j=1}^{p}\Gamma(\frac{n-j}{2})}.$$

When $p = 1$ and $\boldsymbol{\Sigma} = 1$, $\boldsymbol{W}$ reduces to a sum of squares of standard normals, which is χ^2_ν. The Wishart family is that statement with outer products in place of squares. The expectation does not use the density. From the definition $\boldsymbol{W} = \sum_m \boldsymbol{U}_m\boldsymbol{U}_m^\top$ and $\mathbb{E}(\boldsymbol{U}_m\boldsymbol{U}_m^\top) = \boldsymbol{\Sigma}$, linearity gives $\mathbb{E}(\boldsymbol{W}) = \nu\boldsymbol{\Sigma}$. Setting $\nu = n - 1$ recovers (2.8). Normality is not required for this expectation. It is required for the density, and for the claim that $\boldsymbol{A}$ is Wishart.

Under the same normal model the sample mean and $\boldsymbol{A}$ are independent, with marginal distributions

$$\overline{\boldsymbol{X}} \sim \mathrm{N}_p(\boldsymbol{\mu}, \tfrac{1}{n}\boldsymbol{\Sigma}), \qquad \boldsymbol{A} \sim \mathrm{W}_p(\boldsymbol{\Sigma}, n-1), \qquad \overline{\boldsymbol{X}} \perp \boldsymbol{A}. \tag{2.12}$$

The normal law for $\overline{\boldsymbol{X}}$ is Proposition 1.5 applied to the stacked sample. Stack the n observations as one vector $\boldsymbol{Z}$ of length np. Independence puts $\boldsymbol{\Sigma}$ on the diagonal blocks of $\operatorname{Var}(\boldsymbol{Z})$ and zeros off those blocks. The average is $\overline{\boldsymbol{X}} = \frac{1}{n}(\boldsymbol{I} \;\cdots\; \boldsymbol{I})\,\boldsymbol{Z}$, so the covariance formula (1.12) counts n copies of $\boldsymbol{\Sigma}$ and the factor $1/n^2$ leaves $\boldsymbol{\Sigma}/n$. Independence of $\overline{\boldsymbol{X}}$ and $\boldsymbol{A}$, and the Wishart law for $\boldsymbol{A}$, are the multivariate forms of two univariate facts: the sample mean of a normal sample is independent of the sample variance, and $(n-1)S^2/\sigma^2 \sim \chi^2_{n-1}$. Those two facts are used below without proof; see (Anderson, 2003). They are the part of (2.12) that uses normality beyond the mean and covariance of $\overline{\boldsymbol{X}}$.

2.3 Sampling from a multivariate normal distribution

The distributions just stated are also the distributions a simulation should reproduce. A generator usually supplies independent uniforms, or independent standard normals, and the step from those draws to $\mathrm{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ is the linear transformation in Proposition 1.5. Start from uniforms by the inverse transform. Let $U \sim U[0,1]$ and $X = \Phi^{-1}(U)$, with Φ the standard normal distribution function. Because Φ is increasing,

$$\mathbb{P}(X \le x) = \mathbb{P}(\Phi^{-1}(U) \le x) = \mathbb{P}(U \le \Phi(x)) = \Phi(x),$$

so X is standard normal. Draw p independent copies and stack them as $\boldsymbol{Z} \sim \mathrm{N}_p(\boldsymbol{0}, \boldsymbol{I}_p)$. Any matrix $\boldsymbol{B}$ with $\boldsymbol{B}\boldsymbol{B}^\top = \boldsymbol{\Sigma}$ then has the covariance required by Proposition 1.5. The spectral factor $\boldsymbol{B} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}^{1/2}$ from Appendix A.2 is one such square root, and the Cholesky factor is another. Set

$$\boldsymbol{X} = \boldsymbol{\mu} + \boldsymbol{B}\boldsymbol{Z}. \tag{2.13}$$

The construction works because the mean of $\boldsymbol{B}\boldsymbol{Z} + \boldsymbol{\mu}$ is $\boldsymbol{\mu}$ and its covariance is $\boldsymbol{B}\boldsymbol{B}^\top = \boldsymbol{\Sigma}$. Gaussianity follows from the linear-transformation theorem. In software, `rnorm` supplies pseudorandom draws rather than a literal source of independent real-valued random variables. Set a seed to reproduce the computation and use a documented generator. A sample covariance close to its target checks this implementation; it is not a proof that every property of the generator is correct (L'Ecuyer and Simard, 2007).

The spectral and Cholesky factors both satisfy $\boldsymbol{B}\boldsymbol{B}^\top = \boldsymbol{\Sigma}$. Such a covariance factor need not satisfy $\boldsymbol{B}^2 = \boldsymbol{\Sigma}$; reserve *symmetric square root* for $\boldsymbol{\Gamma}\boldsymbol{\Lambda}^{1/2}\boldsymbol{\Gamma}^\top$. A third construction never forms $\boldsymbol{B}$. Draw the first coordinate from its univariate marginal normal, then each later coordinate from

the univariate conditional normal given the coordinates already drawn, using Section 1.5. The constructions target the same $\mathrm{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. They factor it differently.

A goodness-of-fit test for normality does not decide whether the exact F calibration below applies. With small n the test has little power, and a rejection does not name a replacement model. The exact results in this chapter assume normality in the statements that use it. The next section records what remains for the sample mean when that assumption is dropped and only a finite covariance is kept.

The fragment below is the Cholesky route just derived. The laboratory at the end of the chapter evaluates the one-sample test on data, rather than checking a simulated covariance.

```
Sigma <- matrix(c(4, 2, 2, 3), 2, 2); mu <- c(1, 2)
B <- t(chol(Sigma))                  # lower-triangular factor, B %*% t(B) =
↪  Sigma
Z <- matrix(rnorm(2 * 1000), 2)      # columns are independent N(0, I) vectors
X <- t(mu + B %*% Z)                 # rows are N(mu, Sigma) draws
colMeans(X); cov(X)                  # close to mu and Sigma
```

2.4 Central limit theorems

Hotelling's exact F law, derived in the next section, uses normality of the observations. The sample mean still has a normal approximation without that assumption, once a mean and a covariance exist. The classical theorem is the univariate case.

Theorem 2.3 (Central limit theorem)**.** Let $X_1, \ldots, X_n$ be iid with finite mean μ and variance $\sigma^2 > 0$, and $\overline{X}_n = (X_1 + \cdots + X_n)/n$. If $Z \sim \mathrm{N}(0, 1)$, then for every $z \in \mathbb{R}$

$$\lim_{n \to \infty} \mathbb{P}\Big(\frac{\overline{X}_n - \mu}{\sigma / \sqrt{n}} \le z \Big) = \mathbb{P}(Z \le z) = \Phi(z).$$

The theorem is a limiting statement, not a sample-size threshold. Heavy tails or strong skewness can make its approximation poor at a sample size that works well for another distribution. Finite variance is enough for the iid theorem, but it does not guarantee that a moment-generating function exists; characteristic functions provide a more general proof (Keener, 2010). For products of positive observations, a CLT for their logarithms additionally requires the appropriate moments of those logarithms. Taking a logarithm does not supply missing assumptions.

Theorem 2.4 (Multivariate central limit theorem)**.** Let $\boldsymbol{X}_1, \ldots, \boldsymbol{X}_n$ be iid p-dimensional random vectors with finite mean $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$, and $\overline{\boldsymbol{X}}_n = \frac{1}{n}(\boldsymbol{X}_1 + \cdots + \boldsymbol{X}_n)$. Then

$$\sqrt{n}\,(\overline{\boldsymbol{X}}_n - \boldsymbol{\mu}) \xrightarrow{D} \mathrm{N}_p(\mathbf{0}, \boldsymbol{\Sigma}) \qquad \text{as } n \to \infty.$$

Convergence in distribution means that the distribution function of $\sqrt{n}(\overline{\boldsymbol{X}}_n - \boldsymbol{\mu})$ converges at every continuity point to the $\mathrm{N}_p(\mathbf{0}, \boldsymbol{\Sigma})$ distribution function. The usual proof is again by characteristic functions (Anderson, 2003). The exact null distribution of T^2 in the next section needs the normal model. The remark records the consequence of this theorem that does not: if $\boldsymbol{S}$ is consistent for $\boldsymbol{\Sigma}$, the Hotelling quadratic form is asymptotically χ^2_p without normality.

Remark 2.5 (Why this matters for T^2). Section 2.5 derives its exact null distribution under multivariate normality. Under the null, for fixed p and positive-definite $\boldsymbol{\Sigma}$, the multivariate CLT shows what survives without that assumption: $n(\overline{\boldsymbol{X}} - \boldsymbol{\mu}_0)^\top \boldsymbol{S}^{-1} (\overline{\boldsymbol{X}} - \boldsymbol{\mu}_0)$ is asymptotically χ^2_p, since $\boldsymbol{S} \to \boldsymbol{\Sigma}$ in probability and $\sqrt{n}\boldsymbol{\Sigma}^{-1/2}(\overline{\boldsymbol{X}} - \boldsymbol{\mu}_0)$ is asymptotically $\mathrm{N}_p(\mathbf{0}, \boldsymbol{I})$. This is a standard consequence of the multivariate central limit theorem.

2.5 Hotelling's T^2 test

A sample mean, however accurately computed, is not a test. A test needs a null value, an alternative, a statistic, and the distribution of that statistic when the null is true. The statistic derived here is the likelihood ratio for a normal mean vector. Its null distribution will be read from the normal-and-Wishart pair (2.12).

2.5.1 The univariate t-test as a template

The shape of the result is already visible when $p = 1$. For iid $\mathrm{N}(\mu, \sigma^2)$ data, the test of $H_0 : \mu = \mu_0$ against $H_a : \mu \neq \mu_0$ uses

$$t = \frac{\overline{x} - \mu_0}{s/\sqrt{n}} \sim t_{n-1} \quad \text{under } H_0.$$

Under H_0 the distribution of t does not depend on the unknown (μ, σ^2). A quantity with that property is a pivot, so the critical value can be taken from the t_{n-1} table alone. At level α the two-sided rule rejects H_0 when $|t| > t_{\alpha/2;\, n-1}$. In this book the first subscript of a critical value is the upper-tail probability, and any remaining subscripts are degrees of freedom. The algebra below is arranged so that the multivariate statistic reduces to the square of this t when $p = 1$.

2.5.2 Hypotheses and the likelihood-ratio criterion

Take n independent observations from $\mathrm{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, with both parameters unknown, and test

$$H_0 : \boldsymbol{\mu} = \boldsymbol{\mu}_0 \qquad \text{against} \qquad H_1 : \boldsymbol{\mu} \neq \boldsymbol{\mu}_0. \tag{2.14}$$

The unrestricted fit uses $\overline{\boldsymbol{x}}$ and $\boldsymbol{A}/n$. Under the null, the mean is fixed and only the covariance can adapt. Comparing those two best fits gives the likelihood-ratio test developed by Anderson (2003). Its likelihood construction does not by itself establish uniform optimality against every multivariate alternative. The Neyman–Pearson and monotone-likelihood-ratio results have specific hypotheses and cannot be invoked as a blanket guarantee here (Casella and Berger, 2002).

Compare the maximised likelihood with $\boldsymbol{\mu}$ fixed at $\boldsymbol{\mu}_0$ to the maximised likelihood with $\boldsymbol{\mu}$ free:

$$\lambda = \frac{\max_{\boldsymbol{\Sigma}} L(\boldsymbol{\mu}_0, \boldsymbol{\Sigma})}{\max_{\boldsymbol{\mu}, \boldsymbol{\Sigma}} L(\boldsymbol{\mu}, \boldsymbol{\Sigma})} \in [0, 1], \tag{2.15}$$

The numerator maximises over a subset of the denominator's parameter space, so the ratio cannot exceed one. If it is small, imposing the proposed mean costs a great deal of fit. Our next task is computational: simplify the ratio until we can recognise a statistic with a known null distribution.

Maximisation under H_0.

Under H_0 only $\boldsymbol{\Sigma}$ is free. The derivative that produced (2.7) measures residuals from the centring point in use. With residuals measured from $\boldsymbol{\mu}_0$ rather than from a free mean, the same derivative gives $\widehat{\boldsymbol{\Sigma}}_\omega = \frac{1}{n}\sum_i (\boldsymbol{x}_i - \boldsymbol{\mu}_0)(\boldsymbol{x}_i - \boldsymbol{\mu}_0)^\top$. Split those residuals at $\overline{\boldsymbol{x}}$, as in the expansion that gave (2.6):

$$\begin{aligned} \widehat{\boldsymbol{\Sigma}}_\omega &= \frac{1}{n}\sum_{i=1}^{n} \left[(\boldsymbol{x}_i - \overline{\boldsymbol{x}}) + (\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0)\right]\left[(\boldsymbol{x}_i - \overline{\boldsymbol{x}}) + (\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0)\right]^\top \\ &= \frac{1}{n}\Big[\boldsymbol{A} + n(\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0)(\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0)^\top\Big], \end{aligned} \tag{2.16}$$

The cross terms vanish for the same reason as in that expansion: $\sum_i (\boldsymbol{x}_i - \overline{\boldsymbol{x}}) = \boldsymbol{0}$. The constrained covariance estimate is therefore $\boldsymbol{A}/n$ plus the outer product of $\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0$. The added term is nonnegative, and it is zero only when the sample mean equals the null mean.

Maximisation without restriction.

With both parameters free, (2.6) and (2.7) give $\widehat{\boldsymbol{\mu}}_\Omega = \overline{\boldsymbol{x}}$ and $\widehat{\boldsymbol{\Sigma}}_\Omega = \boldsymbol{A}/n$. The denominator of λ uses those two estimators, and no new maximisation is required.

The ratio.

Substitute the two maximisers into (2.15). The powers of 2π agree. The exponents also agree, for a reason that separates from the determinant algebra below: at each maximiser the quadratic form in the exponent equals np. The ratio begins as

$$\lambda = \frac{(2\pi)^{-np/2}|\widehat{\boldsymbol{\Sigma}}_\omega|^{-n/2}\exp\{-\tfrac{1}{2}\sum_i(\boldsymbol{x}_i-\boldsymbol{\mu}_0)^\top\widehat{\boldsymbol{\Sigma}}_\omega^{-1}(\boldsymbol{x}_i-\boldsymbol{\mu}_0)\}}{(2\pi)^{-np/2}|\widehat{\boldsymbol{\Sigma}}_\Omega|^{-n/2}\exp\{-\tfrac{1}{2}\sum_i(\boldsymbol{x}_i-\overline{\boldsymbol{x}})^\top\widehat{\boldsymbol{\Sigma}}_\Omega^{-1}(\boldsymbol{x}_i-\overline{\boldsymbol{x}})\}}.$$

A scalar equals its own trace, and the trace is cyclic, so $\boldsymbol{v}^\top\boldsymbol{M}^{-1}\boldsymbol{v} = \operatorname{tr}(\boldsymbol{M}^{-1}\boldsymbol{v}\boldsymbol{v}^\top)$. Apply that identity to the constrained exponent:

$$\begin{aligned}\sum_{i=1}^n(\boldsymbol{x}_i-\boldsymbol{\mu}_0)^\top\widehat{\boldsymbol{\Sigma}}_\omega^{-1}(\boldsymbol{x}_i-\boldsymbol{\mu}_0) &= \operatorname{tr}\Big(\sum_{i=1}^n(\boldsymbol{x}_i-\boldsymbol{\mu}_0)^\top\widehat{\boldsymbol{\Sigma}}_\omega^{-1}(\boldsymbol{x}_i-\boldsymbol{\mu}_0)\Big)\\ &= \operatorname{tr}\Big(\widehat{\boldsymbol{\Sigma}}_\omega^{-1}\sum_{i=1}^n(\boldsymbol{x}_i-\boldsymbol{\mu}_0)(\boldsymbol{x}_i-\boldsymbol{\mu}_0)^\top\Big)\\ &= \operatorname{tr}\big(\widehat{\boldsymbol{\Sigma}}_\omega^{-1}\cdot n\widehat{\boldsymbol{\Sigma}}_\omega\big) = n\operatorname{tr}(\boldsymbol{I}_p) = np.\end{aligned}$$

At the constrained maximiser, $n\widehat{\boldsymbol{\Sigma}}_\omega$ equals $\sum_i(\boldsymbol{x}_i-\boldsymbol{\mu}_0)(\boldsymbol{x}_i-\boldsymbol{\mu}_0)^\top$ by the formula for $\widehat{\boldsymbol{\Sigma}}_\omega$, so the trace collapses to $n\operatorname{tr}(\boldsymbol{I}_p) = np$. The same steps with $\widehat{\boldsymbol{\Sigma}}_\Omega$ and residuals from $\overline{\boldsymbol{x}}$ also give np, because $n\widehat{\boldsymbol{\Sigma}}_\Omega = \boldsymbol{A}$. Both exponential factors equal $\mathrm{e}^{-np/2}$ and cancel. Normality was used to write the likelihood. The cancellation itself is this trace identity at the two critical points. The likelihood ratio is a ratio of determinants,

$$\lambda = \frac{|\widehat{\boldsymbol{\Sigma}}_\Omega|^{n/2}}{|\widehat{\boldsymbol{\Sigma}}_\omega|^{n/2}} = \frac{|\boldsymbol{A}|^{n/2}}{|\boldsymbol{A}+n(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)^\top|^{n/2}}. \tag{2.17}$$

A determinant identity.

The null mean and the sample mean are still inside one of those determinants. A block-determinant identity pulls them into the Mahalanobis quadratic form. For conformable blocks with $\boldsymbol{B}$ invertible,

$$\begin{vmatrix}\boldsymbol{B} & \boldsymbol{C}\\ \boldsymbol{D} & \boldsymbol{E}\end{vmatrix} = \begin{vmatrix}\boldsymbol{B} & \boldsymbol{C}\\ \boldsymbol{0} & \boldsymbol{E}-\boldsymbol{D}\boldsymbol{B}^{-1}\boldsymbol{C}\end{vmatrix} = |\boldsymbol{B}|\,|\boldsymbol{E}-\boldsymbol{D}\boldsymbol{B}^{-1}\boldsymbol{C}|. \tag{2.18}$$

The first equality subtracts $\boldsymbol{D}\boldsymbol{B}^{-1}$ times the first block row from the second, which does not change the determinant. Apply the identity with $\boldsymbol{B} = \boldsymbol{A}$, $\boldsymbol{C} = -\sqrt{n}(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)$, $\boldsymbol{D} = \sqrt{n}(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)^\top$ and $\boldsymbol{E} = 1$:

$$\begin{vmatrix}\boldsymbol{A} & -\sqrt{n}(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)\\ \sqrt{n}(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)^\top & 1\end{vmatrix} = |\boldsymbol{A}|\Big(1+n(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)^\top\boldsymbol{A}^{-1}(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)\Big).$$

The same determinant, eliminated on the lower-right block instead, has the companion form $|\boldsymbol{B}|\,|\boldsymbol{E}-\boldsymbol{D}\boldsymbol{B}^{-1}\boldsymbol{C}| = |\boldsymbol{E}|\,|\boldsymbol{B}-\boldsymbol{C}\boldsymbol{E}^{-1}\boldsymbol{D}|$. With $\boldsymbol{E} = 1$ the right-hand side is $|\boldsymbol{A}+n(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)^\top|$, since $-\boldsymbol{C}\boldsymbol{E}^{-1}\boldsymbol{D} = +n(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)^\top$. The two evaluations therefore give

$$|\boldsymbol{A}+n(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)^\top| = |\boldsymbol{A}|\Big(1+n(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)^\top\boldsymbol{A}^{-1}(\overline{\boldsymbol{x}}-\boldsymbol{\mu}_0)\Big).$$

This comparison is algebraic. Raise λ in (2.17) to the power $2/n$,

$$\lambda^{2/n} = \frac{1}{1 + n(\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0)^\top \boldsymbol{A}^{-1}(\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0)} = \frac{1}{1 + t^2/(n-1)}, \tag{2.19}$$

where $\boldsymbol{A} = (n-1)\boldsymbol{S}$ has been used to name the quadratic form

$$t^2 = n(\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0)^\top \boldsymbol{S}^{-1}(\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0) = (n-1)\, n(\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0)^\top \boldsymbol{A}^{-1}(\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0). \tag{2.20}$$

Thus λ is a strictly decreasing function of t^2. Rejecting for small λ is the same test as rejecting for large t^2. The block determinant is what converts the ratio of covariance determinants into that quadratic form, and the conversion does not use normality. Normality enters when the threshold is given a probability. The statistic t^2 is n times the squared Mahalanobis distance from $\overline{\boldsymbol{x}}$ to $\boldsymbol{\mu}_0$, computed with $\boldsymbol{S}$. When $p = 1$, $\boldsymbol{S}$ reduces to s^2 and t^2 is the square of the univariate t statistic from the template above.

2.5.3 The null distribution

Write T^2 for the random version of (2.20), so that $\overline{\boldsymbol{X}}$, $\boldsymbol{S}$, and $\boldsymbol{A}$ are random. The event $\lambda < \lambda_0$ is the event $T^2 > t_0^2$, where $t_0^2 = (n-1)(\lambda_0^{-2/n} - 1)$. The threshold is chosen so that the rejection probability equals α when H_0 is true. Under H_0, (2.12) supplies the two ingredients of the quadratic form: $\sqrt{n}(\overline{\boldsymbol{X}} - \boldsymbol{\mu}_0) \sim \mathrm{N}_p(\boldsymbol{0}, \boldsymbol{\Sigma})$ is independent of $(n-1)\boldsymbol{S} = \boldsymbol{A} \sim \mathrm{W}_p(\boldsymbol{\Sigma}, n-1)$. For that pair,

$$F = \frac{n-p}{p(n-1)}\, T^2 \;\sim\; F_{p,\,n-p}. \tag{2.21}$$

Anderson (2003, p. 174) derives the F law from this normal-and-Wishart pair. It is the same relationship that, when $p = 1$, turns a normal numerator and an independent chi-squared denominator into a t statistic, because the square of a t_{n-1} variable is $F_{1,n-1}$. Reject H_0 at level α when $F > F_{\alpha;\,p,\,n-p}$. The denominator degrees of freedom are $n - p$, so the classical statistic needs $n > p$. If $n \le p$ then $\boldsymbol{A}$ is singular and T^2, written with $\boldsymbol{S}^{-1}$, is undefined.

Key idea.
Hotelling's $T^2 = n(\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0)^\top \boldsymbol{S}^{-1}(\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0)$ is the squared Mahalanobis distance of the sample mean from the hypothesised mean, in units of the sample covariance. It is the likelihood-ratio statistic, and a rescaled version has an exact $F_{p,n-p}$ distribution under H_0. When $p = 1$ it is the square of the ordinary t-statistic.

2.5.4 Confidence region for the mean vector

A confidence region can now be built without inventing another statistic. Run the level-α test for every proposed mean and retain the values it does not reject. Under the Gaussian model, the resulting random region contains the fixed true mean in a proportion $1 - \alpha$ of repeated samples. Once the data are observed, this frequentist statement is not a posterior probability that the parameter lies in this particular region.

$$n(\overline{\boldsymbol{x}} - \boldsymbol{\mu})^\top \boldsymbol{S}^{-1}(\overline{\boldsymbol{x}} - \boldsymbol{\mu}) \le \frac{(n-1)p}{n-p}\, F_{\alpha;\,p,\,n-p}. \tag{2.22}$$

Section 1.4 showed that a level set of $(\boldsymbol{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x} - \boldsymbol{\mu})$ is an ellipsoid whose axes are the eigenvectors of $\boldsymbol{\Sigma}$. The same algebra, with $\boldsymbol{S}$ in place of $\boldsymbol{\Sigma}$ and with centre $\overline{\boldsymbol{x}}$, shows that (2.22)

is an ellipsoid centred at the sample mean, with axes along the eigenvectors of $\boldsymbol{S}$. The quadratic form carries a factor n, so the ellipsoid describes uncertainty about the mean and shrinks as n grows. The scatter of the individual observations is what $\boldsymbol{S}$ itself measures. The numbers in Example 2.6 are a case in which this ellipsoid excludes the origin while each separate t-interval still contains it. The two measurements move together, and the sample mean leaves the origin along a direction in which the sample hardly varies.

Example 2.6 (Two paired measurements on twelve subjects)**. Question.** Twelve subjects each provide the change in two correlated measurements, (x_1, x_2):

Subject	1	2	3	4	5	6	7	8	9	10	11	12
x_1	-1.8	6.2	-1.5	-0.9	0.3	1.4	2.6	1.6	-2.3	0.5	3.4	0.9
x_2	-2.1	3.8	-2.2	-1.4	-1.3	-1.9	0.8	0.8	-2.5	-0.2	0.3	-1.0

Test $H_0 : \boldsymbol{\mu} = \mathbf{0}$ at the 5% level, and compare with two separate t-tests.

Solution. The sums are $\sum x_1 = 10.4$ and $\sum x_2 = -6.9$, so $\overline{\boldsymbol{x}} = (0.8667, -0.5750)^\top$. The corrected cross-product matrix and sample covariance are

$$\boldsymbol{A} = \begin{pmatrix} 65.0067 & 43.9600 \\ 43.9600 & 35.6425 \end{pmatrix}, \qquad \boldsymbol{S} = \frac{\boldsymbol{A}}{11} = \begin{pmatrix} 5.9097 & 3.9964 \\ 3.9964 & 3.2402 \end{pmatrix},$$

with sample correlation $r = 3.9964/\sqrt{5.9097 \times 3.2402} = 0.913$. Then $|\boldsymbol{S}| = 5.9097 \times 3.2402 - 3.9964^2 = 3.1778$ and

$$\boldsymbol{S}^{-1} = \frac{1}{3.1778} \begin{pmatrix} 3.2402 & -3.9964 \\ -3.9964 & 5.9097 \end{pmatrix} = \begin{pmatrix} 1.0196 & -1.2576 \\ -1.2576 & 1.8597 \end{pmatrix}.$$

Hence

$$\begin{aligned} T^2 &= 12\, \overline{\boldsymbol{x}}^\top \boldsymbol{S}^{-1} \overline{\boldsymbol{x}} \\ &= 12\big[1.0196(0.8667)^2 - 2(1.2576)(0.8667)(-0.5750) + 1.8597(0.5750)^2\big] = 31.61, \end{aligned}$$

and by (2.21), $F = \frac{12-2}{2\times 11} \times 31.61 = 14.37$ on $(2, 10)$ degrees of freedom. The critical value is $F_{0.05;2,10} = 4.103$ and the p-value is 0.0011, so H_0 is rejected decisively.

The separate t-tests give $t_1 = 0.8667/\sqrt{5.9097/12} = 1.235$ $(p = 0.243)$ and $t_2 = -0.5750/\sqrt{3.2402/12} = -1.107$ $(p = 0.292)$: neither coordinate is individually significant. The two coordinates are strongly positively correlated $(r = 0.913)$ while the sample mean departs from the origin in opposite directions, $(+, -)$. Such a departure is very unusual in the direction of the smaller principal axis of the data cloud, where there is little variability. The T^2 test takes the correlation into account; the marginal tests cannot. Figure 2.1 shows the confidence region (2.22), which excludes the origin, next to the rectangle of Bonferroni-adjusted t-intervals, which includes it.

2.6 The two-sample test

The one-sample test compares a mean with a fully specified vector. The more common question is whether two samples share the same mean. Let $\boldsymbol{X}_i^{(1)} \sim \mathrm{N}_p(\boldsymbol{\mu}_1, \boldsymbol{\Sigma})$ for $i = 1, \ldots, n_1$ and $\boldsymbol{X}_i^{(2)} \sim \mathrm{N}_p(\boldsymbol{\mu}_2, \boldsymbol{\Sigma})$ for $i = 1, \ldots, n_2$, with the two samples independent of each other and the observations independent inside each sample, and with one unknown covariance $\boldsymbol{\Sigma}$. Test $H_0 : \boldsymbol{\mu}_1 - \boldsymbol{\mu}_2 = \boldsymbol{\delta}_0$ against $H_1 : \boldsymbol{\mu}_1 - \boldsymbol{\mu}_2 \neq \boldsymbol{\delta}_0$, usually with $\boldsymbol{\delta}_0 = \mathbf{0}$. Every observation's group is known before the statistic is computed. Predicting those labels would be a classification problem. This section estimates a difference of means.

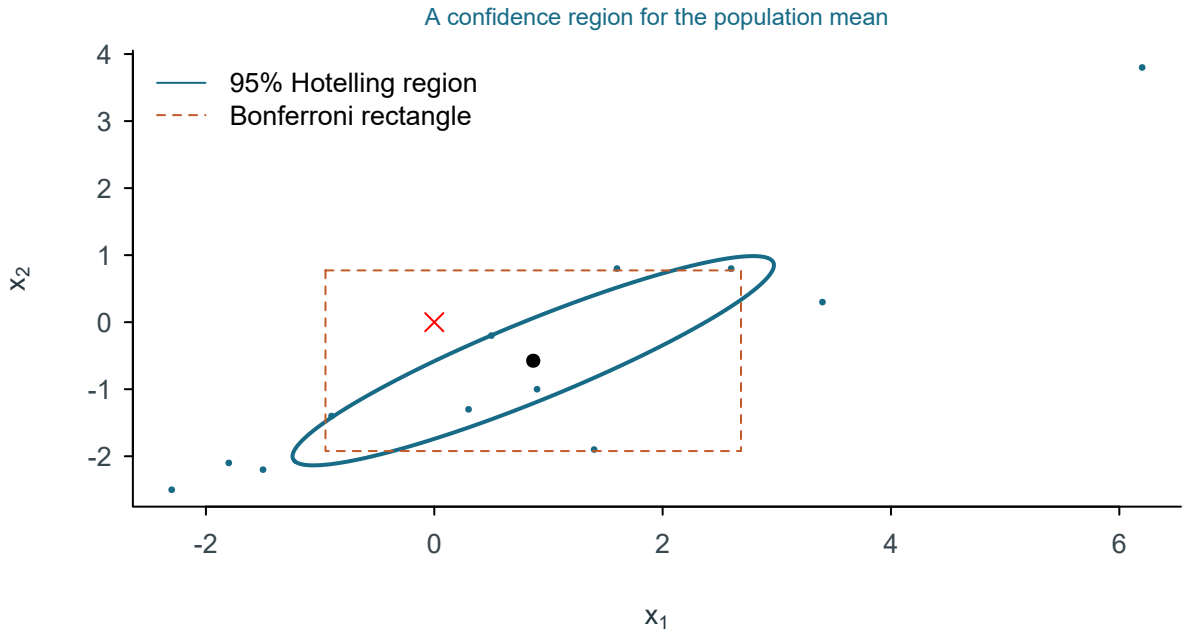


Figure 2.1: The twelve-subject data, a 95% Hotelling confidence region for the mean, and Bonferroni simultaneous intervals. The cross marks the null mean.

Distribution of the mean difference.

The difference of the sample means is a linear image of the stacked data, so Proposition 1.5 applies inside each sample and the covariances add. Stack the n_l observations of sample l as a vector $\boldsymbol{Z}$ of length $n_l p$. Independence makes $\mathrm{Var}(\boldsymbol{Z})$ block diagonal, with $\boldsymbol{\Sigma}$ repeated n_l times. With $\boldsymbol{B} = (\boldsymbol{I}_p \ \cdots \ \boldsymbol{I}_p)$ (n_l copies), $\boldsymbol{Y} = \boldsymbol{B}\boldsymbol{Z} = \sum_i \boldsymbol{X}_i^{(l)}$ has covariance $\boldsymbol{B}\,\mathrm{Var}(\boldsymbol{Z})\boldsymbol{B}^\top = n_l\boldsymbol{\Sigma}$, and therefore $\overline{\boldsymbol{X}}^{(l)} = \boldsymbol{Y}/n_l \sim \mathrm{N}_p(\boldsymbol{\mu}_l, \boldsymbol{\Sigma}/n_l)$. Under H_0 with $\boldsymbol{\delta}_0 = \boldsymbol{0}$, independence of the two samples adds the covariances:

$$\overline{\boldsymbol{X}}^{(1)} - \overline{\boldsymbol{X}}^{(2)} \sim \mathrm{N}_p\Big(\boldsymbol{0}, \Big(\frac{1}{n_1} + \frac{1}{n_2}\Big)\boldsymbol{\Sigma}\Big) = \mathrm{N}_p\Big(\boldsymbol{0}, \frac{n_1 + n_2}{n_1 n_2}\boldsymbol{\Sigma}\Big). \tag{2.23}$$

The factor $1/n_1 + 1/n_2$ is $(n_1 + n_2)/(n_1 n_2)$. It is the sum of the sampling-variance coefficients of two independent sample means. A single sample mean contributes the coefficient $1/n$.

The pooled covariance.

The common $\boldsymbol{\Sigma}$ is estimated from both samples. Every observation informs that covariance, but the two means are not assumed equal under the alternative, so each sample is centred at its own mean. Estimating two means removes two degrees of freedom:

$$\boldsymbol{S} = \frac{1}{n_1 + n_2 - 2}\Big\{\sum_{i=1}^{n_1}(\boldsymbol{x}_i^{(1)} - \overline{\boldsymbol{x}}^{(1)})(\boldsymbol{x}_i^{(1)} - \overline{\boldsymbol{x}}^{(1)})^\top + \sum_{i=1}^{n_2}(\boldsymbol{x}_i^{(2)} - \overline{\boldsymbol{x}}^{(2)})(\boldsymbol{x}_i^{(2)} - \overline{\boldsymbol{x}}^{(2)})^\top\Big\}. \tag{2.24}$$

This is (2.10) with $G = 2$. The Wishart statement in Section 2.2, applied to the pooled residuals, gives $(n_1 + n_2 - 2)\boldsymbol{S} \sim \mathrm{W}_p(\boldsymbol{\Sigma}, n_1 + n_2 - 2)$, independent of both sample means and therefore of their difference. The one-sample passage from a normal vector and an independent Wishart matrix to an F statistic used $\sqrt{n}(\overline{\boldsymbol{X}} - \boldsymbol{\mu}_0)$ and $n - 1$ degrees of freedom. Replace those ingredients by (2.23) and by $n_1 + n_2 - 2$ degrees of freedom. The resulting statistic is

$$T^2 = \frac{n_1 n_2}{n_1 + n_2} (\overline{\boldsymbol{X}}^{(1)} - \overline{\boldsymbol{X}}^{(2)})^\top \boldsymbol{S}^{-1} (\overline{\boldsymbol{X}}^{(1)} - \overline{\boldsymbol{X}}^{(2)}) \tag{2.25}$$

and it follows the T^2 law on n_1+n_2-2 degrees of freedom. The same rescaling that produced (2.21) produces

$$\frac{n_1 + n_2 - p - 1}{(n_1 + n_2 - 2)\, p} T^2 \;\sim\; F_{p,\, n_1+n_2-p-1}. \tag{2.26}$$

Reject H_0 at level α when

$$\frac{n_1 + n_2 - p - 1}{(n_1 + n_2 - 2)\, p} T^2 > F_{\alpha;\, p,\, n_1+n_2-p-1}. \tag{2.27}$$

The second degree of freedom is $n_1 + n_2 - p - 1$. It is the Wishart degrees of freedom $n_1 + n_2 - 2$, reduced by the dimension in the same way that $n - 1$ became $n - p$ in the one-sample F. The calibration uses the common covariance and the normal model.

Confidence statements.

Invert the test, as with the one-sample region (2.22). The values of $\boldsymbol{\delta} = \boldsymbol{\mu}_1 - \boldsymbol{\mu}_2$ that (2.27) would not reject form the confidence set

$$(\overline{\boldsymbol{x}}^{(1)} - \overline{\boldsymbol{x}}^{(2)} - \boldsymbol{\delta})^\top \boldsymbol{S}^{-1} (\overline{\boldsymbol{x}}^{(1)} - \overline{\boldsymbol{x}}^{(2)} - \boldsymbol{\delta}) \le \frac{n_1 + n_2}{n_1 n_2} T^2_{\alpha;\, p,\, n_1+n_2-p-1}, \tag{2.28}$$

where

$$T^2_{\alpha;\, p,\, n_1+n_2-p-1} = \frac{(n_1 + n_2 - 2)\, p}{n_1 + n_2 - p - 1} F_{\alpha;\, p,\, n_1+n_2-p-1}. \tag{2.29}$$

The same critical value yields simultaneous intervals for every linear functional $\boldsymbol{a}^\top \boldsymbol{\delta}$, $\boldsymbol{a} \in \mathbb{R}^p$, and not only for the p coordinates:

$$\boldsymbol{a}^\top (\overline{\boldsymbol{x}}^{(1)} - \overline{\boldsymbol{x}}^{(2)}) \pm \sqrt{\boldsymbol{a}^\top \boldsymbol{S} \boldsymbol{a}\, \frac{n_1 + n_2}{n_1 n_2} T^2_{\alpha;\, p,\, n_1+n_2-p-1}}. \tag{2.30}$$

A coordinate interval is the case in which $\boldsymbol{a}$ is a unit vector along one axis. The half-width uses the T^2 critical value rather than a single two-sample t critical value, because the intervals are required to cover $\boldsymbol{a}^\top \boldsymbol{\delta}$ simultaneously for every $\boldsymbol{a}$, including the direction in which the two sample means differ most. Separate t-intervals each control one coordinate and do not control a search over directions, so the simultaneous intervals are wider.

2.6.1 Unequal covariance matrices

Pooling is justified by the common-covariance model. Equal group sizes do not make that assumption true and do not restore the exact finite-sample F distribution under arbitrary heteroscedasticity. Ito and Schull (1964) studies large-sample robustness, which is a more limited conclusion. With unequal covariances we face the multivariate Behrens–Fisher problem even when the group sizes happen to match. One approach is the approximation of Krishnamoorthy and Yu (2004). It estimates the covariance of the mean difference directly: define $\widetilde{\boldsymbol{S}}_1 = \boldsymbol{S}_1/n_1$, $\widetilde{\boldsymbol{S}}_2 = \boldsymbol{S}_2/n_2$, and $\widetilde{\boldsymbol{S}} = \widetilde{\boldsymbol{S}}_1 + \widetilde{\boldsymbol{S}}_2$. The resulting statistic is

$$T^2 = (\overline{\boldsymbol{X}}^{(1)} - \overline{\boldsymbol{X}}^{(2)})^\top \widetilde{\boldsymbol{S}}^{-1} (\overline{\boldsymbol{X}}^{(1)} - \overline{\boldsymbol{X}}^{(2)}). \tag{2.31}$$

For independent normal samples with $n_l > p$ and positive-definite population covariances, an F approximation is

$$\frac{\nu - p + 1}{\nu p} T^2 \sim F_{p,\, \nu - p + 1}, \tag{2.32}$$

with denominator degrees of freedom $\nu - p + 1$, where

$$\nu = \frac{p + p^2}{\displaystyle\sum_{l=1}^{2} \frac{\operatorname{tr}\left[(\widetilde{\boldsymbol{S}}_l \widetilde{\boldsymbol{S}}^{-1})^2\right] + \left[\operatorname{tr}(\widetilde{\boldsymbol{S}}_l \widetilde{\boldsymbol{S}}^{-1})\right]^2}{n_l - 1}}. \tag{2.33}$$

The traces compare each sample's share of the total covariance estimate. The divisors are $n_l - 1$, the covariance degrees of freedom: the paper calls these n_l and calls sample sizes N_l (Krishnamoorthy and Yu, 2004, Section 2.3, equation (7)). For $p = 1$, put $v_l = s_l^2 / n_l$. The formula reduces to $\nu = (v_1 + v_2)^2 / \{v_1^2/(n_1 - 1) + v_2^2/(n_2 - 1)\}$, exactly Welch's degrees of freedom. The reference law remains approximate. Comparisons of several mean vectors continue as multivariate analysis of variance; see Morrison (2005, Chapter 3).

2.7 R laboratory: a joint mean test for setosa

The one-sample test is applied here to the two sepal measurements of *Iris setosa*, with illustrative null mean $(5.0, 3.4)$ cm. That vector is a teaching choice. It is not a biological standard. The probability from (2.21) is exact under independent bivariate normal sampling, with $n > p$ and nonsingular $\boldsymbol{S}$.

```
x <- as.matrix(subset(iris, Species == "setosa")[, 1:2])
n <- nrow(x); p <- ncol(x)
d <- colMeans(x) - c(5.0, 3.4)
T2 <- n * drop(t(d) %*% solve(cov(x), d))
Fstat <- (n - p) * T2 / (p * (n - 1))
c(T_squared = T2, F = Fstat,
  df1 = p, df2 = n - p,
  p_value = pf(Fstat, p, n - p, lower.tail = FALSE))
#> T_squared         F       df1       df2   p_value
#>    0.4323    0.2117    2.0000   48.0000    0.8099
```

The computed p-value is 0.81. At the 5% level we do not reject this joint null. A failure to reject does not establish that the true mean equals $(5.0, 3.4)$. The reference distribution $F_{p,\, n-p}$ is the exact null law only under the independent normal sampling used to derive (2.21). The p-value does not check that sampling model.

2.8 Further exercises

Exercise 2.1 (Where the $n - 1$ comes from)**.** Show that $\sum_{i=1}^{n} (\boldsymbol{X}_i - \boldsymbol{\mu})(\boldsymbol{X}_i - \boldsymbol{\mu})^\top = \boldsymbol{A} + n(\overline{\boldsymbol{X}} - \boldsymbol{\mu})(\overline{\boldsymbol{X}} - \boldsymbol{\mu})^\top$, and use it to prove $\mathbb{E}(\boldsymbol{A}) = (n-1)\boldsymbol{\Sigma}$. Which assumption about the $\boldsymbol{X}_i$ does the proof use beyond a common mean and covariance?

Solution

Exercise 2.2 (Pooling)**.** Group 1 has $n_1 = 10$ observations with mean $(3, 2)^\top$ and $\boldsymbol{A}_1 = \begin{pmatrix} 18 & 6 \\ 6 & 12 \end{pmatrix}$; Group 2 has $n_2 = 15$ observations with mean $(1, 1)^\top$ and $\boldsymbol{A}_2 = \begin{pmatrix} 30 & -9 \\ -9 & 24 \end{pmatrix}$. Compute the pooled covariance matrix (2.10). Then compute the sample covariance matrix of the combined sample of 25 observations about its grand mean, using $\boldsymbol{A}_{\text{total}} = \boldsymbol{A}_1 + \boldsymbol{A}_2 + \frac{n_1 n_2}{n}(\overline{\boldsymbol{x}}_1 - \overline{\boldsymbol{x}}_2)(\overline{\boldsymbol{x}}_1 - \overline{\boldsymbol{x}}_2)^\top$ and divisor $n - 1$, and explain why the two differ.

Solution

Exercise 2.3 (Simulating correlated normals)**.** Take $\boldsymbol{\Sigma} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$. Find a lower-triangular $\boldsymbol{L}$ with $\boldsymbol{L}\boldsymbol{L}^\top = \boldsymbol{\Sigma}$ (Cholesky), and also the matrix $\boldsymbol{B} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}^{1/2}$ from the spectral decomposition. Show that both give $\operatorname{Var}(\boldsymbol{B}\boldsymbol{Z}) = \boldsymbol{\Sigma}$ for $\boldsymbol{Z} \sim \mathrm{N}_2(\boldsymbol{0}, \boldsymbol{I})$, although $\boldsymbol{L} \neq \boldsymbol{B}$.

Solution

Exercise 2.4 (Likelihood-ratio identity)**.** Verify numerically that for $\boldsymbol{A} = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}$ and $\boldsymbol{d} = \sqrt{n}(\overline{\boldsymbol{x}} - \boldsymbol{\mu}_0) = (1, 2)^\top$, $|\boldsymbol{A} + \boldsymbol{d}\boldsymbol{d}^\top| = |\boldsymbol{A}|\,(1 + \boldsymbol{d}^\top \boldsymbol{A}^{-1}\boldsymbol{d})$.

Solution

Exercise 2.5 (A small T^2 test)**.** Six subjects give $\overline{\boldsymbol{x}} = (2,\ 1)^\top$ and $\boldsymbol{S} = \begin{pmatrix} 4 & 1 \\ 1 & 1 \end{pmatrix}$ for the changes in two scores. Test $H_0 : \boldsymbol{\mu} = \boldsymbol{0}$ at the 5% level using T^2, given $F_{0.05;2,4} = 6.944$. State the exact null distribution you use.

Solution

Exercise 2.6 (Two-sample statistic)**.** Two groups of sizes $n_1 = n_2 = 10$ in $p = 2$ dimensions have mean difference $\overline{\boldsymbol{x}}^{(1)} - \overline{\boldsymbol{x}}^{(2)} = (1.5,\ 1.0)^\top$ and pooled covariance matrix $\boldsymbol{S} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. Compute T^2 from (2.25), convert it to an F-statistic, and decide at the 5% level given $F_{0.05;2,17} = 3.592$.

Solution

Exercise 2.7 (Whitening and its limits)**.** A dataset has $n = 80$ observations in $p = 4$ dimensions, sample mean $\bar{x}$ and positive-definite sample covariance $S = U\Lambda U^T$. Define $z_i = S^{-1/2}(x_i - \bar{x})$.

(a) Explain how $S^{-1/2}$ is obtained from the spectral decomposition.

(b) Determine the sample mean and covariance of the z_i, showing the matrix calculation.

(c) Explain why this operation need not produce independent normally distributed coordinates, and what fails if $p = 100$ with the same n.

Solution

Exercise 2.8 (Normal conditioning and whitening)**.** You may use the conditional-normal formula

$$X_A \mid X_B = x_B \sim N\Big(\mu_A + \Sigma_{AB}\Sigma_{BB}^{-1}(x_B - \mu_B),\, \Sigma_{AA} - \Sigma_{AB}\Sigma_{BB}^{-1}\Sigma_{BA}\Big).$$

(a) Let $(X_1, X_2)^T \sim N_2((1, 2)^T, \left(\begin{smallmatrix} 4 & 2 \\ 2 & 9 \end{smallmatrix}\right))$. Derive the conditional distribution of X_1 given $X_2 = 5$.

(b) For this model, explain the distribution of $X_1 - E(X_1 \mid X_2)$ and whether changing the observed value of X_2 changes its variance.

(c) A different dataset has positive-definite sample covariance S. Describe the sample mean and covariance after applying $z_i = S^{-1/2}(x_i - \bar{x})$.

Solution

Exercise 2.9 (A transformation is not a distributional assumption)**.** Let $X \sim N_2(\mu, \Sigma)$ with $\mu = (0, 2)^T$ and $\Sigma = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 3 \end{smallmatrix}\right)$.

(a) Derive the mean and variance of $Y = X_1 - 2X_2$ and state its distribution.

(b) Explain how the covariance matrix determines the orientation and relative axis lengths of equal-density contours.

(c) A different, nonnormal dataset is whitened to have sample covariance I. Does this establish independent standard normal observations? Explain.

Solution

Chapter 3

Principal component analysis

An iris flower has four measured coordinates. Suppose we want a two-dimensional display that keeps as much of the data's variation as possible. Which two directions should we choose? Picking two original measurements is one option, but a combination of measurements may describe the cloud better. Principal component analysis chooses those combinations by solving a precise optimisation problem.

We'll derive PCA in two ways: maximise the variance retained by a projection, and minimise the squared error of reconstructing the centred observations. The two criteria agree. Neither asks which direction predicts a response or represents a causal mechanism. If PCA is used before a predictor, its centring, scaling and directions must be estimated within the training sample, just like the predictor itself.

Source. Lecture notes, Section 1.8 (pp. 29–33) and Appendix B (spectral decomposition, p. 160). The notes follow the terminology of Hastie et al. (2009).

3.1 Aim and terminology

A direction needs both an orientation and a scale convention. Without the scale convention we could make every projected score more variable just by multiplying its coefficients. We therefore compare unit vectors. After finding the first direction, we require the next one to be orthogonal to it; otherwise the same solution would be selected again. In all score formulas below, the observations are centred unless a mean is explicitly shown.

Definition 3.1 (Principal component directions and components)**.** The *first principal component direction* is the unit vector $\boldsymbol{a}_1 \in \mathbb{R}^p$ along which the sample variance of the data is largest. The *first principal components* of the data (the *scores*) are the projections of the observations onto $\boldsymbol{a}_1$, i.e. the dot products $\boldsymbol{a}_1^\top \boldsymbol{x}_i$. The *second principal component direction* $\boldsymbol{a}_2$ is the unit vector orthogonal to $\boldsymbol{a}_1$ along which the sample variance is largest subject to that constraint; the third is orthogonal to the first two; and so on until p directions, and the p corresponding sets of components, are defined.

Keep the two objects separate. A **direction** is a vector of p coefficients; a **score** is the number obtained by projecting one observation onto that direction. A matrix of directions transforms a data matrix into a matrix of scores. Terminology varies across texts, so check what an author means by *component* or *loading*. R's `prcomp` calls its coefficient matrix `rotation` and its score matrix `x` (R Core Team, 2026a).

In practice the scores are used for dimension reduction. The p original coordinates are replaced by the first few scores when those scores carry enough variance for the question in hand. Whether they carry the scientifically important information is a separate question. A coordinate with little

variation is often dropped on the hope that it has little to contribute. That hope is not implied by the definition. Exercise 3.6 is a case in which the largest-variance direction of the explanatory variables carries none of the response.

3.2 Derivation by Lagrange multipliers

Let $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ have sample mean $\overline{\boldsymbol{x}}$ and sample covariance $\boldsymbol{S}$. The first direction is the coefficient vector of a linear combination whose sample variance is as large as possible. The combination is

$$Y_1 = a_{11}X_1 + \cdots + a_{p1}X_p = \boldsymbol{a}_1^\top \boldsymbol{X} \tag{3.1}$$

The sample version of (1.10), with $\boldsymbol{S}$ in place of $\boldsymbol{\Sigma}$, writes that variance as a quadratic form,

$$s_{Y_1}^2 = \sum_{j=1}^{p}\sum_{k=1}^{p} a_{j1}a_{k1}s_{jk} = \boldsymbol{a}_1^\top \boldsymbol{S}\boldsymbol{a}_1. \tag{3.2}$$

If there is a direction with positive variance, the unconstrained optimisation has no finite answer. Replace $\boldsymbol{a}_1$ by $c\boldsymbol{a}_1$ and the variance becomes c^2 times as large. That is a change of ruler, not a better summary. Fix $\boldsymbol{a}_1^\top \boldsymbol{a}_1 = 1$ so we can compare orientations fairly. A Lagrange multiplier lets us optimise while enforcing this constraint:

$$u_1 = \boldsymbol{a}_1^\top \boldsymbol{S}\boldsymbol{a}_1 - \lambda(\boldsymbol{a}_1^\top \boldsymbol{a}_1 - 1). \tag{3.3}$$

Differentiate in the direction. The gradient is $\partial u_1/\partial \boldsymbol{a}_1 = 2\boldsymbol{S}\boldsymbol{a}_1 - 2\lambda\boldsymbol{a}_1$. Set it to zero:

$$\boldsymbol{S}\boldsymbol{a}_1 = \lambda\boldsymbol{a}_1. \tag{3.4}$$

The stationary points are the eigenvectors of $\boldsymbol{S}$, and the multipliers are the eigenvalues. Substitute a stationary point back into the objective:

$$\boldsymbol{a}_1^\top \boldsymbol{S}\boldsymbol{a}_1 = \lambda\, \boldsymbol{a}_1^\top \boldsymbol{a}_1 = \lambda, \tag{3.5}$$

At a unit eigenvector, the objective equals its eigenvalue. The objective is continuous and the unit sphere is compact, so a maximum exists. The largest eigenvalue gives it; the weighted-average argument below makes this global claim explicit. If that eigenvalue is repeated, several directions are equally good. Even with a unique largest eigenvalue, $\boldsymbol{a}_1$ and $-\boldsymbol{a}_1$ describe the same axis.

A second direction that is not constrained will select the same eigenvector again. Later directions are required to be orthogonal to those already chosen. One construction removes the first component from the centred data and repeats the maximisation on what remains. The theorem reaches the same directions in one induction. Any unit vector is written in the eigenbasis of $\boldsymbol{S}$, so its sample variance becomes a weighted average of the eigenvalues. Orthogonality to the first $k-1$ eigenvectors sets the first $k-1$ weights to zero, and the largest remaining eigenvalue is then the maximum.

Theorem 3.2 (Principal component directions)**.** Let $\boldsymbol{S} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^\top$ with $\lambda_1 \geq \cdots \geq \lambda_p \geq 0$ and $\boldsymbol{\Gamma} = (\boldsymbol{\gamma}_1, \ldots, \boldsymbol{\gamma}_p)$ orthogonal. The jth principal component direction is $\boldsymbol{a}_j = \boldsymbol{\gamma}_j$, the jth principal component $Y_j = \boldsymbol{\gamma}_j^\top \boldsymbol{X}$ has sample variance λ_j, distinct components are uncorrelated, and the total variance is preserved,

$$\sum_{j=1}^{p} \mathrm{Var}(Y_j) = \sum_{j=1}^{p} \lambda_j = \mathrm{tr}(\boldsymbol{S}) = \sum_{j=1}^{p} s_{jj}. \tag{3.6}$$

Proof. Any unit vector can be written $\boldsymbol{a} = \boldsymbol{\Gamma}\boldsymbol{b}$ with $\boldsymbol{b} = \boldsymbol{\Gamma}^\top \boldsymbol{a}$ and $\boldsymbol{b}^\top \boldsymbol{b} = 1$. Then

$$\boldsymbol{a}^\top \boldsymbol{S} \boldsymbol{a} = \boldsymbol{b}^\top \boldsymbol{\Gamma}^\top \boldsymbol{\Gamma} \boldsymbol{\Lambda} \boldsymbol{\Gamma}^\top \boldsymbol{\Gamma} \boldsymbol{b} = \sum_{j=1}^{p} \lambda_j b_j^2.$$

For $j = 1$ the right side is a weighted average of the λ_j with weights b_j^2 that sum to one, so it is at most λ_1, with equality for $\boldsymbol{b} = \boldsymbol{e}_1$, i.e. $\boldsymbol{a} = \boldsymbol{\gamma}_1$. Suppose $\boldsymbol{a}_1 = \boldsymbol{\gamma}_1, \ldots, \boldsymbol{a}_{k-1} = \boldsymbol{\gamma}_{k-1}$. Orthogonality of $\boldsymbol{a}$ to these means $b_1 = \cdots = b_{k-1} = 0$, so $\boldsymbol{a}^\top \boldsymbol{S} \boldsymbol{a} = \sum_{j \geq k} \lambda_j b_j^2 \leq \lambda_k$, again attained at $\boldsymbol{b} = \boldsymbol{e}_k$. By induction $\boldsymbol{a}_j = \boldsymbol{\gamma}_j$ for all j. The variance of Y_j is $\boldsymbol{\gamma}_j^\top \boldsymbol{S} \boldsymbol{\gamma}_j = \lambda_j$. For $j \neq k$, $\mathrm{Cov}(Y_j, Y_k) = \boldsymbol{\gamma}_j^\top \boldsymbol{S} \boldsymbol{\gamma}_k = \lambda_k \boldsymbol{\gamma}_j^\top \boldsymbol{\gamma}_k = 0$. Finally $\sum_j \lambda_j = \mathrm{tr}(\boldsymbol{\Lambda}) = \mathrm{tr}(\boldsymbol{\Gamma}^\top \boldsymbol{S} \boldsymbol{\Gamma}) = \mathrm{tr}(\boldsymbol{S} \boldsymbol{\Gamma} \boldsymbol{\Gamma}^\top) = \mathrm{tr}(\boldsymbol{S})$. □

The calculation has diagonalised the **sample** covariance. It guarantees zero sample covariance between different score columns, not independence and not zero covariance for arbitrary future observations. In the theorem and proof, Var and Cov refer to the empirical score variances and covariances computed with divisor $n - 1$. For population PCA, repeat the argument with $\boldsymbol{\Sigma}$ in place of $\boldsymbol{S}$. Under an additional Gaussian population model, the population principal components are independent. Without it, uncorrelated is the conclusion we have earned.

Key idea.
The principal component directions are the eigenvectors of the sample covariance matrix, ordered by eigenvalue. The jth eigenvalue is the variance of the jth component, so the proportion of variance explained by the first k components is $\sum_{j \leq k} \lambda_j / \sum_{j \leq p} \lambda_j$.

3.2.1 Reconstruction view

Equation (3.6) says that the sum of all component variances equals $\mathrm{tr}(\boldsymbol{S})$, which does not depend on the basis. Maximising the variance kept by the first k components is therefore the same problem as minimising the variance left out. Let $\boldsymbol{\Gamma}_k = (\boldsymbol{\gamma}_1, \ldots, \boldsymbol{\gamma}_k)$. Replacing each centred observation by its projection $\boldsymbol{\Gamma}_k \boldsymbol{\Gamma}_k^\top (\boldsymbol{x}_i - \overline{\boldsymbol{x}})$ leaves residuals whose total squared length, divided by $n - 1$, equals $\sum_{j=k+1}^{p} \lambda_j$. That value is the smallest residual attainable by any rank-k linear approximation. The leading components are at once the highest-variance coordinates and the best linear summary of the cloud in squared error. The second description is the one that autoencoder networks generalise (Remark 3.4).

3.3 Choosing and using components

Theorem 3.2 already identifies λ_j as the variance of component j, so the proportion of variance explained by that component is $\lambda_j / \sum_k \lambda_k$, and the proportion explained by the first k components is the corresponding partial sum. A common rule keeps the smallest k that reaches a stated fraction of the total, for example 90%, 95%, or 99%. The fraction is a choice about residual squared error. The rule does not consult a response.

A component can explain little predictor variance and still be highly predictive of the response. Exercise 3.6 constructs exactly that situation. Variance explained is therefore a reconstruction criterion, not a general measure of predictive usefulness. If prediction is the goal, choose the

number of components using validation inside the modelling pipeline. Estimate all preprocessing on the training folds and leave the final test set for assessment (Cawley and Talbot, 2010, Section 5).

Remark 3.3 (Scaling). Changing metres to millimetres multiplies a coordinate by 1000 and its variance by 10^6. Covariance PCA can therefore change drastically even though the physical observations have not changed. Standardising each nonconstant coordinate to unit variance gives correlation PCA. This chooses equal weight per standardised coordinate, which may or may not suit the scientific question. It is a modelling choice, not an automatic improvement. Record the choice and reuse the training means and scales for new observations.

Remark 3.4 (Other representations). There are many other ways to reduce dimension or re-represent data via latent variables, with fewer or different constraints: factor analysis, non-negative matrix factorisation, and neural networks. If a hidden layer of a feedforward network is smaller than the input layer, the output of the hidden layer is a reduced-dimension representation of the input. This is the idea of *autoencoder* networks, which try to reproduce the input at the output while passing through a lower-dimensional hidden layer.

Example 2.6 rejected a zero mean for twelve paired measurements even though each marginal t-test did not. The eigenvalues of the same $\boldsymbol{S}$ show how unequal the two axis variances are. That inequality is the geometric reason the joint statistic had power along the short axis.

Example 3.5 (PCA of the twelve-subject data)**.** Return to the twelve pairs (x_1, x_2) of Example 2.6, with

$$\boldsymbol{S} = \begin{pmatrix} 5.9097 & 3.9964 \\ 3.9964 & 3.2402 \end{pmatrix}.$$

The characteristic equation $\lambda^2 - \operatorname{tr}(\boldsymbol{S})\lambda + |\boldsymbol{S}| = \lambda^2 - 9.1499\lambda + 3.1778 = 0$ has roots $\lambda_1 = 8.788$ and $\lambda_2 = 0.362$. For λ_1, $(\boldsymbol{S} - \lambda_1 \boldsymbol{I})\boldsymbol{a} = \boldsymbol{0}$ gives $(5.9097 - 8.788)a_1 + 3.9964a_2 = 0$, so $a_2 = 0.7202a_1$ and the unit eigenvector is

$$\boldsymbol{a}_1 = (0.8114,\ 0.5845)^\top, \qquad \boldsymbol{a}_2 = (-0.5845,\ 0.8114)^\top,$$

the second being orthogonal to the first. The first component is $Y_1 = 0.8114X_1 + 0.5845X_2$: roughly a weighted average of the two measurements. Its variance is 8.788, which is $8.788/9.150 = 96.05\%$ of the total variance $\operatorname{tr}(\boldsymbol{S}) = 9.150$, while $Y_2 = -0.5845X_1 + 0.8114X_2$ contributes only 3.95%. Direct computation of the scores confirms that the sample variances of Y_1, Y_2 equal $8.788, 0.362$ and that their sample covariance is zero (up to rounding). Figure 3.1 shows the data with the two directions. The sample mean $\overline{\boldsymbol{x}} = (0.87, -0.58)^\top$ points mostly along the second direction ($|\cos| = 0.94$), where the data hardly vary; this is why the T^2 test rejects in Example 2.6 although neither marginal test does.

If the variables are standardised first, the covariance matrix becomes the correlation matrix $\boldsymbol{R} = \begin{pmatrix} 1 & 0.9133 \\ 0.9133 & 1 \end{pmatrix}$ with eigenvalues $1 + r = 1.913$ and $1 - r = 0.087$ and directions exactly $(1, 1)^\top/\sqrt{2}$ and $(1, -1)^\top/\sqrt{2}$. The first component now explains 95.7%. The directions are different from the unscaled analysis because the two variables have different variances (5.91 and 3.24).

How many components?

The right-hand panel of Figure 3.1 is an eight-dimensional correlation matrix with no response attached. The question is only how fast the cumulative proportion in Theorem 3.2 grows. The eigenvalues are

```
lambda <- eigen(0.6^abs(outer(1:8, 1:8, "-")),
                symmetric = TRUE)$values
knitr::kable(data.frame(Component = 1:8,
```

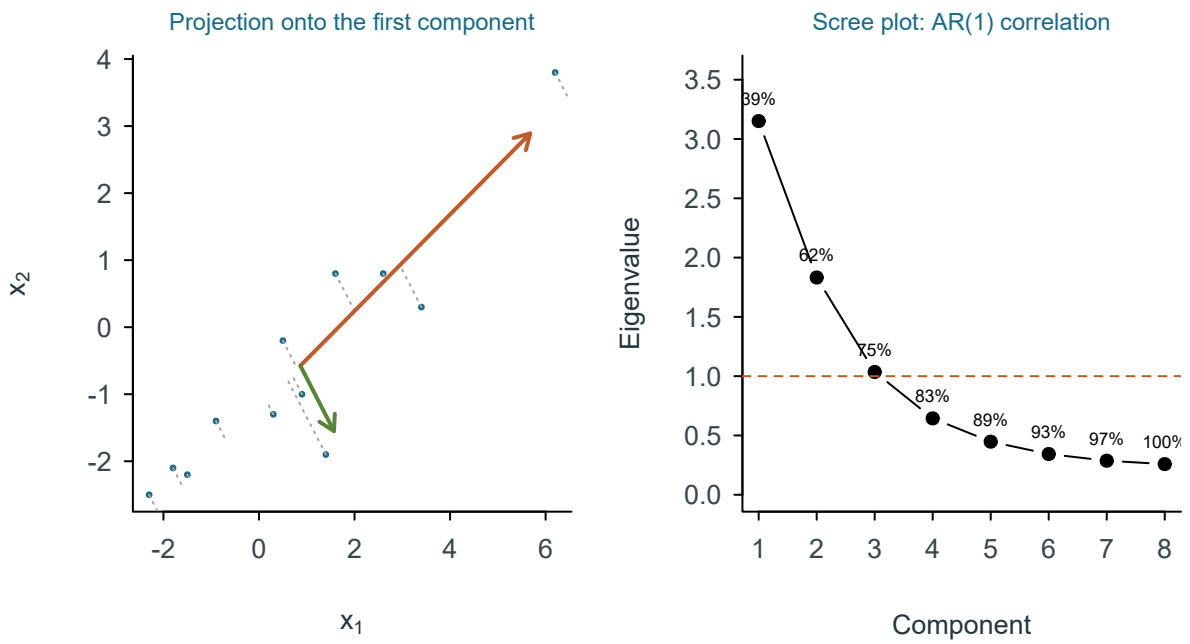


Figure 3.1: Projection onto the first principal direction for the twelve-subject data, and an AR(1) scree plot. Percentage labels give cumulative explained variance.

```
Eigenvalue = round(lambda, 3),
Cumulative_percent = round(100 * cumsum(lambda) / sum(lambda), 1)),
col.names = c("Component", "Eigenvalue", "Cumulative percent"))
```

Component	Eigenvalue	Cumulative percent
1	3.151	39.4
2	1.832	62.3
3	1.035	75.2
4	0.644	83.3
5	0.448	88.9
6	0.344	93.2
7	0.287	96.8
8	0.259	100.0

A rule that keeps 90% of the variance keeps six components. A rule that keeps 75% keeps three. Keeping every eigenvalue above the average, here 1, also keeps three. None of these cutoffs is best for every purpose, and none of them sees a response. Each of them answers the reconstruction problem, with a chosen fraction of $\mathrm{tr}(\boldsymbol{S})$.

A scree plot offers a visual heuristic: look for a point after which additional eigenvalues contribute relatively little. A bend can be ambiguous or absent, and it does not validate a predictive model. For prediction, select the number of components using a validation set or inner cross-validation. Keep the final test set untouched. Calling a repeatedly consulted dataset a test set does not preserve its independence.

3.4 PCA and the singular value decomposition

Forming $\boldsymbol{S} = \boldsymbol{X}^\top \boldsymbol{X}/(n-1)$ and then computing its eigenvectors repeats information that the singular value decomposition of the centred data matrix already contains. The decomposition is also the stable computation, including when $n < p$. Centre each column and write $\boldsymbol{X}$ for the centred $n \times p$ matrix, so that $\boldsymbol{S} = \frac{1}{n-1}\boldsymbol{X}^\top \boldsymbol{X}$ agrees with the sample covariance already used. The singular value decomposition writes

$$\boldsymbol{X} = \boldsymbol{U}\boldsymbol{E}\boldsymbol{V}^\top, \tag{3.7}$$

Use a compact SVD with $r = \operatorname{rank}(\boldsymbol{X})$: $\boldsymbol{U}$ is $n \times r$, $\boldsymbol{E}$ is $r \times r$, and $\boldsymbol{V}$ is $p \times r$. Both $\boldsymbol{U}^\top \boldsymbol{U}$ and $\boldsymbol{V}^\top \boldsymbol{V}$ equal $\boldsymbol{I}_r$. This works whether n exceeds p or not. Substituting gives

$$\boldsymbol{S} = \frac{1}{n-1}\boldsymbol{V}\boldsymbol{E}\boldsymbol{U}^\top \boldsymbol{U}\boldsymbol{E}\boldsymbol{V}^\top = \frac{1}{n-1}\boldsymbol{V}\boldsymbol{E}^2\boldsymbol{V}^\top.$$

Thus the nonzero eigenvalues are $e_j^2/(n-1)$ and their directions are the columns of $\boldsymbol{V}$. If $r < p$, complete these columns to an orthonormal basis for the zero-variance directions when a full p-dimensional basis is wanted. The nonzero score columns are

$$\boldsymbol{X}\boldsymbol{V} = \boldsymbol{U}\boldsymbol{E}\boldsymbol{V}^\top \boldsymbol{V} = \boldsymbol{U}\boldsymbol{E}. \tag{3.8}$$

This is also the computation used by `prcomp`: it applies SVD directly to the centred and optionally scaled data, and uses divisor $n-1$ for reported variances (R Core Team, 2026a, Details and Value). The signs of directions may differ between implementations. Compare the axes or reconstructions, rather than treating a sign reversal as a disagreement.

Whitening takes one more step: divide each nonzero principal-component score by its standard deviation. For positive-definite $\boldsymbol{S}$ this gives identity sample covariance. If $\boldsymbol{S}$ is singular, only the r positive-variance directions can be whitened this way; division by a zero eigenvalue is undefined. Identity covariance makes Gaussian contours spherical, but it does not make an arbitrary distribution Gaussian or spherically symmetric.

A principal direction and a prediction of a response answer different questions. If the goal is to predict a response, the object to report is that prediction. An axis of the explanatory cloud summarises the explanatory variables, and Theorem 3.2 does not consult the response.

Notation check.
In a full SVD the square matrices $\boldsymbol{U}$ and $\boldsymbol{V}$ are orthogonal. In a compact SVD their columns are orthonormal, but $\boldsymbol{U}\boldsymbol{U}^\top$ and $\boldsymbol{V}\boldsymbol{V}^\top$ are projections onto their column spaces. Their dimensions are not interchangeable. This distinction becomes unavoidable when there are more variables than observations.

The fragment below reads the singular-value route from a `prcomp` object: squared singular values scaled by $n-1$, directions in the rotation matrix, and scores in the coordinates. The laboratory is the executed comparison of covariance PCA with correlation PCA.

```
pc <- prcomp(x, center = TRUE, scale. = FALSE)  # SVD of the centred data
pc$sdev^2                                        # eigenvalues lambda_j =
 ↪  e_j^2/(n-1)
pc$rotation                                      # columns are the directions
 ↪  a_j
head(pc$x)                                       # the components (scores) X V =
 ↪  U E
summary(pc)$importance["Proportion of Variance", ]
```

3.5 R laboratory: PCA of iris, with and without scaling

The fit uses every iris flower, and it is descriptive. Species labels colour the plot after the fit. They do not enter $\boldsymbol{S}$. Correlation PCA standardises each measurement to unit variance before the eigendecomposition. Covariance PCA keeps the original variances, so a measurement with larger sample variance takes a larger share of the first direction, which is the unit dependence in Remark 3.3.

```
pc_cov <- prcomp(iris_x, center = TRUE, scale. = FALSE)
pc_cor <- prcomp(iris_x, center = TRUE, scale. = TRUE)
round(rbind(Covariance = pc_cov$sdev^2 / sum(pc_cov$sdev^2),
            Correlation = pc_cor$sdev^2 / sum(pc_cor$sdev^2)), 3)
#>              [,1]  [,2]  [,3]  [,4]
#> Covariance  0.925 0.053 0.017 0.005
#> Correlation 0.730 0.229 0.037 0.005
round(pc_cor$rotation[, 1:2], 3)
#>                PC1    PC2
#> Sepal.Length  0.521 -0.377
#> Sepal.Width  -0.269 -0.923
#> Petal.Length  0.580 -0.024
#> Petal.Width   0.565 -0.067
stopifnot(max(abs(cov(pc_cor$x) - diag(pc_cor$sdev^2))) < 1e-10)
```

```
plot(pc_cor$x[, 1:2], col = palette_book[as.integer(iris_y)],
     pch = 19, xlab = "PC1 score", ylab = "PC2 score")
legend("topright", levels(iris_y), col = palette_book[1:3],
       pch = 19, bty = "n", cex = 0.8)
```

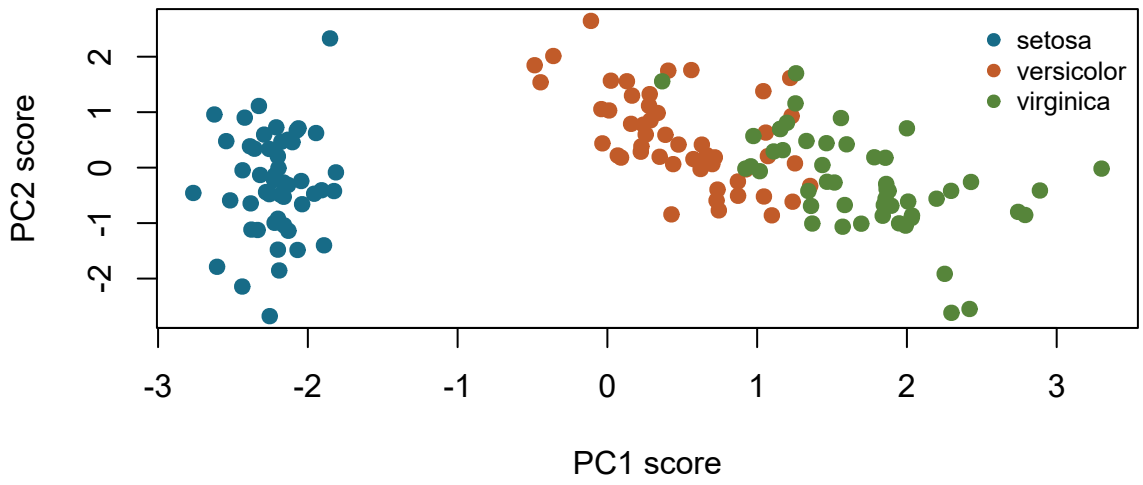


Figure 3.2: Correlation-PCA scores for iris. Species labels are added after fitting.

The first two standardised components explain 95.8% of the total standardised variance: the

partial sum of eigenvalues from Theorem 3.2, divided by the trace. The sign of a loading may be flipped without changing the line it spans or the variance it carries. Separation of species in the plot describes this sample. A prediction experiment has to estimate the centring, the scaling, and the directions inside each training fold, and apply that transformation unchanged to the validation rows.

3.6 Further exercises

Exercise 3.1 (A two-variable PCA)**.** For $\boldsymbol{S} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, find the principal component directions, the variances of the two components, and the proportion of variance explained by the first.

Solution

Exercise 3.2 (Properties of the components)**.** Using $\boldsymbol{S} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^\top$, show that the covariance matrix of the full vector of components $\boldsymbol{Y} = \boldsymbol{\Gamma}^\top(\boldsymbol{X} - \overline{\boldsymbol{x}})$ is $\boldsymbol{\Lambda}$, and that the generalised variance $|\boldsymbol{S}|$ equals $\prod_j \lambda_j$.

Solution

Exercise 3.3 (Effect of correlation)**.** For $p = 2$ and correlation matrix $\boldsymbol{R} = \begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}$ with $r > 0$, show that the eigenvalues are $1 \pm r$ for every r, and that the eigenvectors do not depend on r. What happens to the proportion of variance explained by the first component as $r \to 0$ and as $r \to 1$?

Solution

Exercise 3.4 (PCA via the SVD)**.** The centred data matrix $\boldsymbol{X} = \begin{pmatrix} 2 & 1 \\ -2 & -1 \\ 1 & -1 \\ -1 & 1 \end{pmatrix}$ has $\boldsymbol{X}^\top\boldsymbol{X} = \begin{pmatrix} 10 & 2 \\ 2 & 4 \end{pmatrix}$. Find the eigenvalues of $\boldsymbol{X}^\top\boldsymbol{X}$, the singular values e_1, e_2 of $\boldsymbol{X}$, and the eigenvalues $\lambda_i = e_i^2/(n-1)$ of the sample covariance matrix. Check that $\sum_i \lambda_i = \operatorname{tr}(\boldsymbol{S})$.

Solution

Exercise 3.5 (How many components?)**.** A sample covariance matrix has eigenvalues 5.87, 1.81, and 1.32. Find the proportion of variance explained by each component and the cumulative proportions. How many components are needed to reach 80%?

Solution

Exercise 3.6 (A tempting wrong turn)**.** Let $X_1 \sim \mathrm{N}(0, 100)$ and $X_2 \sim \mathrm{N}(0, 1)$ be independent and let the response be $Y = X_2$ exactly. An analyst reduces the explanatory variables to their first principal component and regresses Y on it. What is the first principal direction (population version), and what proportion of the variance of Y can the regression explain? What does this show about using PCA before regression?

Solution

Exercise 3.7 (PCA as a model-building step)**.** A student wants to classify iris species from four measurements. They propose retaining the first two principal components of the standardised measurements.

(a) Explain how the principal directions and component scores are obtained. Distinguish a direction from a score.

(b) Explain what changes if centimetres are converted to millimetres for just one variable before covariance-based PCA, and why correlation-based PCA avoids this issue.

(c) Explain why retaining the two components with largest variance is not guaranteed to give the best classifier. Describe where PCA must be fitted during cross-validation.

Solution

Part II

Clustering and density estimation

Chapter 4

Clustering and K-means

Suppose we hide the species column of iris and keep the four measurements. Can we still find useful groups? Clustering makes that question precise by choosing a measure of similarity and looking for a partition that scores well under it. A group found this way is a property of the measurements and the criterion. Calling it a biological population requires further evidence.

Density estimation, taken up in Chapters 5–7, has a different target: an approximation to the distribution that generated the observations.

A class label and a cluster label answer different questions. If observations that share a class form separated clouds, a clustering can agree with the classes. If the classes are mixed through the same clouds, or if they split along a direction the measurements do not record, the clusters say little about the classification. The algorithm never sees the class labels, so nothing in the criterion forces the two partitions to coincide.

Before choosing an algorithm, settle what a useful group would mean for this problem. Which differences between observations should count? How much detail should the partition retain? Are we describing a sample or proposing a population model? These choices determine the task. Here we begin with a geometric description: nearby observations should share a centre.

K-means is the first procedure. It asks the user to choose K, and it minimises a within-cluster sum of squared Euclidean distances. The labels it returns are names for the groups in that partition. They are not measured species identities. The choice of K, and the choice of scale, are part of the specification. Several random starts reduce the chance of reporting a poor local minimum. They do not establish that the groups are biological populations.

Source. Lecture notes, Chapter 2 introduction and Section 2.1 (pp. 34–39), with the Voronoi remarks of p. 39.

4.1 The K-means idea

Give each of K centres a position, then assign each observation to its nearest centre. Now move each centre to the mean of its assigned observations. Those two simple operations are Lloyd's algorithm. Squared Euclidean distance makes the mean the right update; the identity below explains why. The resulting cells are convex polyhedra, not necessarily spheres. The method nevertheless tends to suit compact groups of comparable spread better than elongated groups or nested rings.

Distance quietly decides how much each coordinate matters. Multiplying one variable by ten multiplies its contribution to squared distance by one hundred; subtracting a constant changes nothing. Standardising to variance one gives each variable the same marginal scale, but that is a modelling choice, not a mathematical requirement. Even variables measured in the same

units can deserve different weights. For iris, compare the scientific meaning of absolute centimetre differences with differences measured in standard deviations before choosing the metric.

A different dissimilarity defines a different optimisation problem and may require a different centre update. Absolute deviations, for example, lead to coordinatewise medians rather than means. The calculations here use squared Euclidean distance,

$$d(\boldsymbol{x}_i, \boldsymbol{x}_l) = \sum_{j=1}^{p} (x_{ij} - x_{lj})^2 = \|\boldsymbol{x}_i - \boldsymbol{x}_l\|^2. \tag{4.1}$$

4.2 Within-cluster scatter

A clustering is a map C from the observation indices $i = 1, \ldots, n$ to the labels $1, \ldots, K$. To compare maps one needs a single number. The number used here is the total squared distance between observations that share a label, the *within-cluster point scatter*,

$$W(C) = \frac{1}{2} \sum_{k=1}^{K} \sum_{C(i)=k} \sum_{C(l)=k} \|\boldsymbol{x}_i - \boldsymbol{x}_l\|^2, \tag{4.2}$$

The factor $\frac{1}{2}$ stops each unordered pair from being counted twice. It does not change which clustering is best. It does make the identity with distances to means an equality, rather than an equality up to a factor 2.

4.2.1 From pairs to cluster means

The pairwise sum couples every pair inside a cluster, which is an awkward object to minimise directly. The equivalent form sums squared distances from each point to its cluster mean. Let $N_k = \sum_{i=1}^{n} \mathbf{1}\{C(i) = k\}$ be the size of cluster k and $\overline{\boldsymbol{x}}_k$ the mean of its points. Inside one cluster, add and subtract that mean. The cross term is the step that has to be checked. It vanishes for the same reason as in the normal-sample expansion of Chapter 2: the sum of deviations from the mean of those points is zero.

$$\begin{aligned}
\sum_{C(i)=k} \sum_{C(l)=k} \|\boldsymbol{x}_i - \boldsymbol{x}_l\|^2 &= \sum_{C(i)=k} \sum_{C(l)=k} \|(\boldsymbol{x}_i - \overline{\boldsymbol{x}}_k) - (\boldsymbol{x}_l - \overline{\boldsymbol{x}}_k)\|^2 \\
&= \sum_{C(i)=k} \sum_{C(l)=k} \left[\|\boldsymbol{x}_i - \overline{\boldsymbol{x}}_k\|^2 + \|\boldsymbol{x}_l - \overline{\boldsymbol{x}}_k\|^2 - 2(\boldsymbol{x}_i - \overline{\boldsymbol{x}}_k)^\top (\boldsymbol{x}_l - \overline{\boldsymbol{x}}_k) \right] \\
&= N_k \sum_{C(i)=k} \|\boldsymbol{x}_i - \overline{\boldsymbol{x}}_k\|^2 + N_k \sum_{C(l)=k} \|\boldsymbol{x}_l - \overline{\boldsymbol{x}}_k\|^2 - 0 \\
&= 2N_k \sum_{C(i)=k} \|\boldsymbol{x}_i - \overline{\boldsymbol{x}}_k\|^2.
\end{aligned}$$

The cross term vanishes because $\sum_{C(i)=k} (\boldsymbol{x}_i - \overline{\boldsymbol{x}}_k) = \mathbf{0}$. This is the ANOVA decomposition of Appendix A.5, applied inside each cluster. Therefore

$$W(C) = \sum_{k=1}^{K} N_k \sum_{C(i)=k} \|\boldsymbol{x}_i - \overline{\boldsymbol{x}}_k\|^2, \tag{4.3}$$

Equation (4.3) writes the pairwise criterion as a size-weighted sum of squared distances to the cluster means. Each point is weighted by N_k. The algorithm of Section 4.3 does not descend on

that weighted sum. It descends on the unweighted sum of squares in the following note. The two criteria come from the same pairwise distances after different normalisations, and the reassignment step decreases only the unweighted one.

Weighting check.
Equation (4.3) follows exactly from (4.2), so the two statements in the notes agree. But the K-means algorithm of Section 4.3, which alternates "assign to the closest mean" and "recompute the means", is guaranteed to decrease the *unweighted* within-cluster sum of squares

$$\widetilde{W}(C, \boldsymbol{m}_1, \ldots, \boldsymbol{m}_K) = \sum_{k=1}^{K} \sum_{C(i)=k} \|\boldsymbol{x}_i - \boldsymbol{m}_k\|^2, \tag{4.4}$$

which is $\frac{1}{2}\sum_k N_k^{-1} \sum_{C(i)=C(l)=k} \|\boldsymbol{x}_i - \boldsymbol{x}_l\|^2$ at $\boldsymbol{m}_k = \overline{\boldsymbol{x}}_k$. This is the criterion most software reports (`tot.withinss` in R). With the weights N_k of (4.3) the reassignment step need not decrease the criterion, because moving a point changes two cluster sizes as well as two sums. The worked example below reports both quantities.

4.2.2 Why not search all partitions?

Every partition has a value of (4.2), so the minimum could in principle be found by enumeration. The number of ways to partition n observations into K non-empty unlabelled clusters is the Stirling number of the second kind (see Jain and Dubes, 1988),

$$S(n, K) = \frac{1}{K!} \sum_{k=1}^{K} (-1)^{K-k} \binom{K}{k} k^n. \tag{4.5}$$

For a very small sample, enumeration is possible. It rapidly becomes expensive: $S(10, 3) = 9330$, whereas $S(25, 5) = 2\,436\,684\,974\,110\,751$ and $S(100, 5) \approx 6.6 \times 10^{67}$. The limitation is computational size. Lloyd's algorithm trades an exhaustive search for inexpensive alternating improvements, so we must distinguish reaching a stable answer from finding the best possible answer.

4.3 The algorithm

Algorithm 4.1 (Lloyd's K-means algorithm)**.** Choose K initial p-dimensional "means" $\boldsymbol{m}_1, \ldots, \boldsymbol{m}_K$. They need not be the mean of any subset of the observations, but they will be after one iteration. Then repeat:

1. *Assign* each observation to the closest cluster mean, $C(i) = \arg\min_k \|\boldsymbol{x}_i - \boldsymbol{m}_k\|^2$.
2. *Update* the means, $\boldsymbol{m}_k \leftarrow \overline{\boldsymbol{x}}_k$, the mean of the observations assigned to k.

until the assignments of observations to clusters do not change.

We need two conventions to make this algorithm precise. Retain an observation's current cluster if it is tied for nearest; break initial ties deterministically. Assume for the finite-termination argument that no cluster becomes empty. An empty cluster has no sample mean, so software must handle it explicitly, for example by restarting. These details matter because a zero-cost reassignment is not a strict improvement.

Write the objective as a function of two arguments: assignments and centres. With centres fixed, optimise the assignments; with assignments fixed, optimise the centres. Each step is simple even

though their joint optimisation is difficult. The following result concerns the unweighted $\widetilde{W}$, not the size-weighted W.

Proposition 4.2 (Descent)**.** Each step of Algorithm 4.1 does not increase $\widetilde{W}$. With the tie convention above and nonempty clusters throughout, the algorithm stops after finitely many assignment changes at a Lloyd fixed point. This is an optimum with respect to each of the two alternating steps separately; it need not be a global optimum.

Proof. For fixed means, minimise each observation's squared distance separately. For fixed assignments, add and subtract the cluster mean to obtain $\sum_{C(i)=k} \|\boldsymbol{x}_i - \boldsymbol{m}\|^2 = \sum_{C(i)=k} \|\boldsymbol{x}_i - \overline{\boldsymbol{x}}_k\|^2 + N_k\|\overline{\boldsymbol{x}}_k - \boldsymbol{m}\|^2$. The second term is minimised at the mean. After the first update, each partition determines its centres and cost. Any later assignment change strictly reduces cost because ties retain the current assignment. A partition therefore cannot repeat. There are finitely many partitions, so eventually neither step changes the answer.

□

Notice exactly what the proof permits us to say. Neither a nearest-centre reassignment with centres held fixed nor a mean update with assignments held fixed improves the final fit. A coordinated move of several centres and assignments may still do better. More iterations at a fixed point will not discover that move; a different start may.

Once the start and tie handling are fixed, this version of Lloyd's algorithm is deterministic. Different starts can lead to different partitions. Compare their objective values rather than assuming that the first stable fit is the right one.

Example 4.3 (Six points, two clusters)**. Question.** Run the algorithm on the points $A = (1, 1)$, $B = (1.5, 2)$, $C = (3, 4)$, $D = (5, 7)$, $E = (3.5, 5)$, $F = (4.5, 5)$ with $K = 2$ and initial means $\boldsymbol{m}_1 = (3, 4)$, $\boldsymbol{m}_2 = (5, 7)$.

Solution. Squared distances to the two means, with the assignment (the smaller entry):

	A	B	C	D	E	F
$\|\boldsymbol{x} - \boldsymbol{m}_1\|^2$, $\boldsymbol{m}_1 = (3, 4)$	**13**	**6.25**	**0**	13	**1.25**	**3.25**
$\|\boldsymbol{x} - \boldsymbol{m}_2\|^2$, $\boldsymbol{m}_2 = (5, 7)$	52	37.25	13	**0**	6.25	4.25

Only D is closer to $\boldsymbol{m}_2$. Cluster 1 is $\{A, B, C, E, F\}$, cluster 2 is $\{D\}$, and the update gives $\boldsymbol{m}_1 = (2.7, 3.4)$ and $\boldsymbol{m}_2 = (5, 7)$. Repeating:

Pass	Means before assignment	Assignment	$\widetilde{W}$	W from (4.3)
1	$(3, 4)$, $(5, 7)$	$\{A, B, C, E, F\}, \{D\}$	21.50	107.5
2	$(2.7, 3.4)$, $(5, 7)$	$\{A, B, C, E\}, \{D, F\}$	16.375	61.25
3	$(2.25, 3)$, $(4.75, 6)$	$\{A, B, C\}, \{D, E, F\}$	10.667	32.0
4	$(1.83, 2.33)$, $(4.33, 5.67)$	$\{A, B, C\}, \{D, E, F\}$	10.667	32.0

In pass 2, F is closer to $(5, 7)$ (squared distance 4.25) than to $(2.7, 3.4)$ (5.8) and moves; in pass 3, E moves. In pass 4 no assignment changes, so the algorithm stops. Both criteria decreased at every pass here. Figure 4.1 shows the trajectory.

4.4 Initialisation and restarts

There is no single accepted start, and many authors draw it at random. The initial means may be drawn uniformly in a box that contains the data, or K of the observations may be used as the initial means, or the observations may be split at random into K groups whose means are the start. Many other variants exist. A run is usually short, so repeat it and keep the clustering with the smallest $\widetilde{W}$. The kept value has not been shown to be the global minimum.

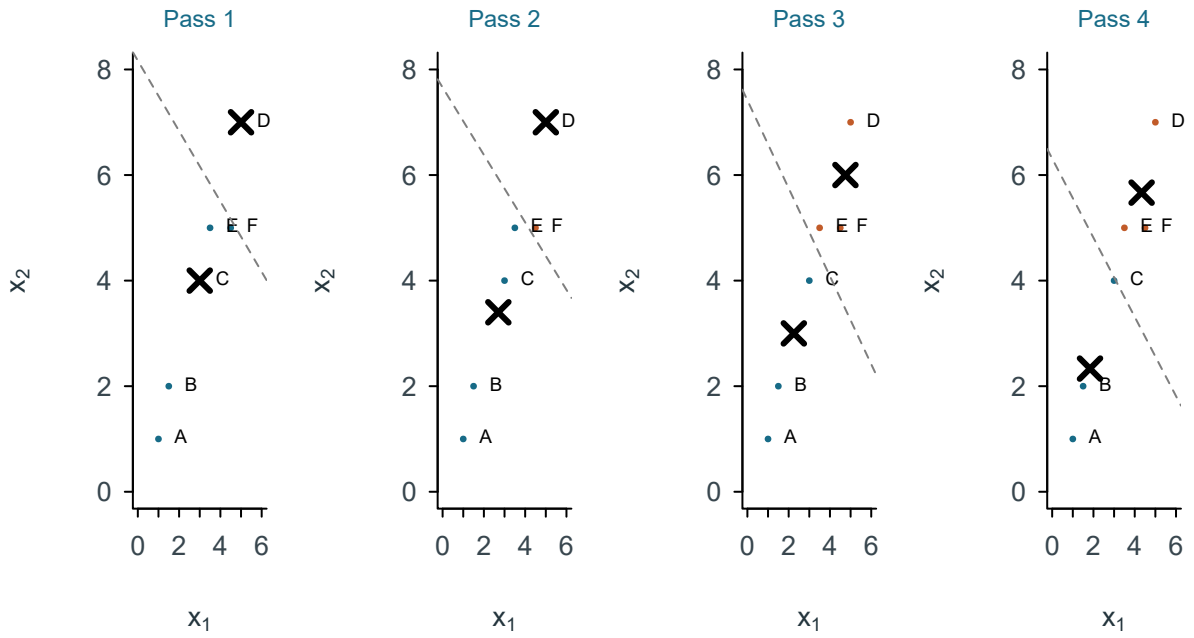


Figure 4.1: The six-point worked example, reproduced by four Lloyd passes. Crosses mark current means, dashed lines the Voronoi boundary, and colours the assignments.

The objective is non-convex, and different regions of the initialisation space lead to different local minima. Descent from a fixed start is deterministic, so two starts in one region finish at the same partition, and a restart that remains in that region repeats the answer. A different local minimum requires a start in another region. Allocating every point independently at random to a cluster is a poor way to spread those starts. The K means then all lie near the centre of the sample and close to one another.

The fragment standardises the variables, tries 25 starts and retains the smallest reported within-cluster sum of squares. R's default `kmeans()` uses the Hartigan–Wong algorithm, which targets the same sum of squares but uses different updates. Specify `algorithm = "Lloyd"` when reproducing the alternating steps above; see (R Core Team, 2026b). The iris laboratory uses the default and checks the reported objective directly.

```
set.seed(3006)
xs <- scale(x)                              # each column to mean 0, variance 1
fit <- kmeans(xs, centers = 3, nstart = 25)   # 25 random starts, best kept
fit$tot.withinss                            # the unweighted criterion (W-tilde)
table(fit$cluster)
```

4.5 Choosing K: the gap statistic

The criterion $\widetilde{W}$ falls as K increases, and it equals zero at $K = n$, with one cluster for each point. Minimising it over K therefore selects the largest K on the list. The objective that justifies the algorithm cannot choose K. Choosing K is a model-selection problem: which member of a family is appropriate, once each member can be fitted. A Bayesian treatment would put a prior on K and report a posterior. This section seeks one K. Chapter 6 returns to the question for mixture models. For K-means, one standard frequentist comparison is the gap statistic of Tibshirani et al. (2001). It asks how much smaller $\log W$ is for the data than for reference samples that have the same coordinate-wise range and no clusters.

Construction.

Record the coordinate-wise sample minima and maxima. Run K-means for each $k = 1, \dots, K_{\max}$ on the data and store the within-cluster scatter W^k_{data}. Draw B reference samples of size n from the uniform distribution on the box with those minima and maxima as corners, in the same spirit as a bootstrap from a featureless distribution on that box. For each reference sample and each k, run K-means and store W^k_b. The gap and its simulation scale are

$$\text{Gap}(k) = \frac{1}{B}\sum_{b=1}^{B} \log W^k_b - \log W^k_{\text{data}}, \tag{4.6}$$

$$s'(k) = \sqrt{1 + 1/B}\ \text{s. d.}\left(\log W^k_1, \dots, \log W^k_B\right), \tag{4.7}$$

Choose the smallest k at which a further cluster does not raise the gap by more than that scale,

$$k^* = \min\left\{k :\ \text{Gap}(k) \geq \text{Gap}(k+1) - s'(k+1)\right\}, \tag{4.8}$$

Equivalently, k^* is the smallest k with $\text{Gap}(k+1) - \text{Gap}(k) \leq s'(k+1)$: the next increase in the gap is no larger than the variability of the reference scores at $k+1$ clusters. Take the smallest k at which adding a cluster no longer improves the gap beyond that variability.

The rule is a comparison with a chosen reference distribution, not a test proving that a particular number of populations exists. Its result depends on the reference geometry and on how well each clustering optimisation is solved. The original paper considers both an axis-aligned box and a box aligned with principal components; these need not give the same answer (Tibshirani et al., 2001, Section 2).

An outlier can stretch the reference box and change the benchmark substantially. Rotating to principal components addresses orientation but does not make the method insensitive to skewness or outliers. Inspect the reference samples as well as the gap curve. A featureless reference is a modelling decision, and its consequences should be visible.

Remark 4.4 (Why not minimise W over k?). W (in either version) is non-increasing in the number of clusters and reaches 0 at $k = n$, so minimising it over k always selects the largest k. The gap statistic corrects for this by comparing $\log W^k_{\text{data}}$ with what is expected for data without cluster structure (the uniform reference), which also falls as k grows.

4.6 Voronoi geometry

The assignment step gives every point to the nearest current mean, so the clusters are the cells of the Voronoi tessellation with those means as seeds. The boundary between the cells of $\boldsymbol{m}_a$ and $\boldsymbol{m}_b$ is the set of points at equal squared distance from both. Expand the squared norms. The quadratic terms $\|\boldsymbol{x}\|^2$ cancel, which is special to squared Euclidean distance, and the boundary is linear:

$$\|\boldsymbol{x} - \boldsymbol{m}_a\|^2 = \|\boldsymbol{x} - \boldsymbol{m}_b\|^2 \iff 2(\boldsymbol{m}_b - \boldsymbol{m}_a)^\top \boldsymbol{x} = \|\boldsymbol{m}_b\|^2 - \|\boldsymbol{m}_a\|^2.$$

This is a hyperplane of dimension $p - 1$, perpendicular to the segment joining the means and passing through the midpoint of that segment. The cluster boundaries are pieces of hyperplanes in every dimension.

In the plane, three non-collinear centres have perpendicular bisectors meeting at their circumcentre. In $\mathbb{R}^p$ with $p > 2$, the points equidistant from all three instead form a $(p-2)$-dimensional affine set. Its intersection with the plane of the three centres is the circumcentre. This distinction prevents a two-dimensional picture from quietly becoming a false higher-dimensional claim.

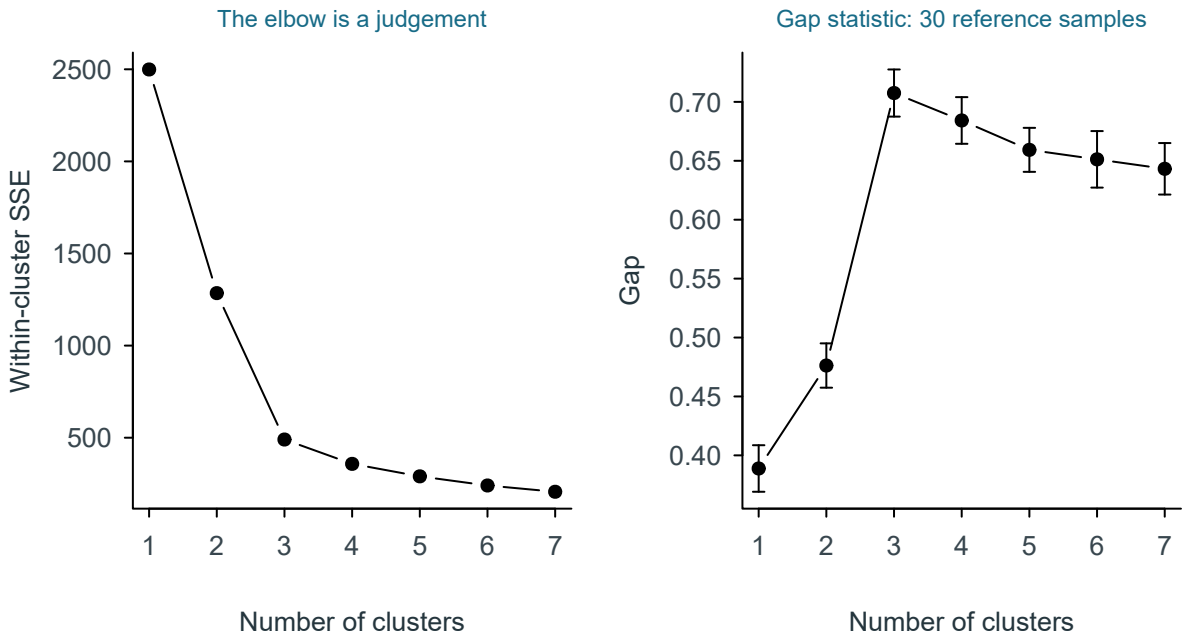


Figure 4.2: Within-cluster SSE and the gap statistic from the same R simulation. Bars show the simulation uncertainty used in the gap selection rule.

The criterion does not specify a density. There is nothing to sample from the fitted partition, and this chapter gives no confidence interval or test for the centres. The cells just derived are convex and cut by hyperplanes, so the method is aimed at separated spherical clusters. Overlapping groups are separated by force, because each point is given wholly to one centre. A ring of points is a poor target for the same geometric reason: the cells cannot follow a hole.

Key idea.
K-means turns a geometric preference into a computable objective. Lloyd's two steps decrease that objective, subject to the conventions just stated, but do not certify the best partition. The metric, number of centres and starting values each affect the result. Check all three before interpreting a cluster label.

4.7 R laboratory: do three clusters recover three species?

We now hide the iris species labels while fitting. Standardisation chooses a distance measured in standard deviations. Setting $K = 3$ uses the known number of species: the exercise is whether three geometric groups resemble those species, not whether the algorithm discovers the number three. The cross-tabulation answers that question without pretending cluster numbers are species names.

```
set.seed(4006)
z <- scale(iris_x)
km <- kmeans(z, centers = 3, nstart = 50)
table(Cluster = km$cluster, Species = iris_y)
#>         Species
#> Cluster setosa versicolor virginica
#>       1      0         11        36
#>       2     50          0         0
#>       3      0         39        14
```

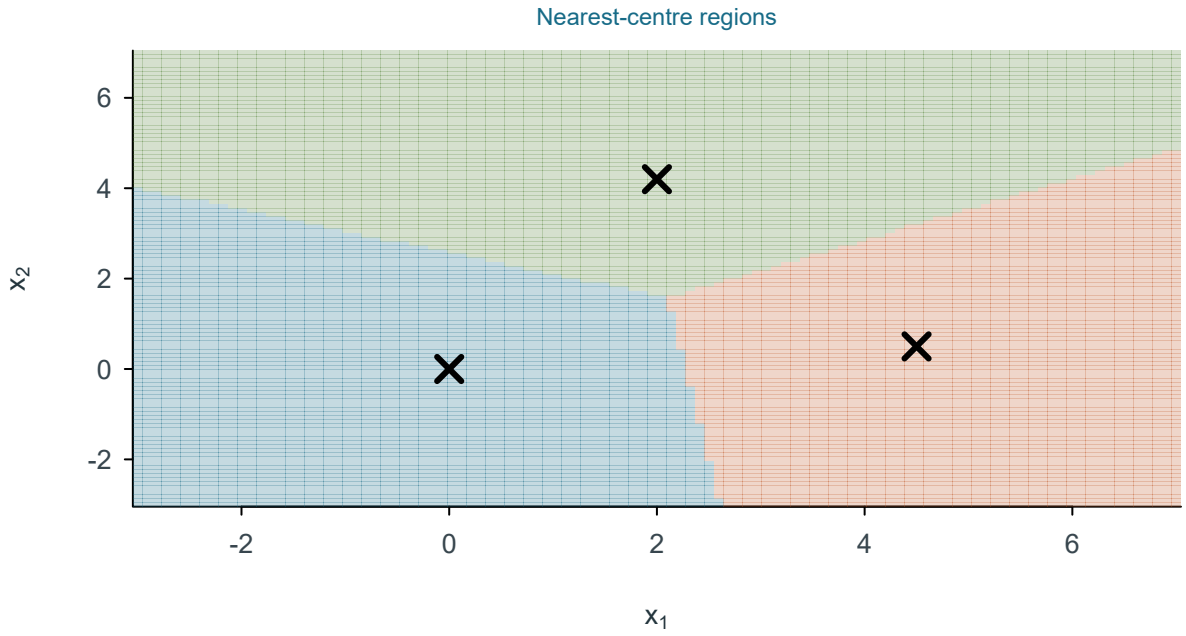


Figure 4.3: Voronoi regions induced by three fixed centres under squared Euclidean distance.

```
c(within_SSE = km$tot.withinss,
  adjusted_Rand_index = mclust::adjustedRandIndex(km$cluster, iris_y))
#>          within_SSE adjusted_Rand_index
#>            138.8884              0.6201
direct_SSE <- sum((z - km$centers[km$cluster, ])^2)
stopifnot(abs(direct_SSE - km$tot.withinss) < 1e-8)
```

The adjusted Rand index compares the clustering with the species partition without requiring cluster 1 to mean species 1. On its chance-corrected scale, identical partitions score 1, and the index can be negative. K-means favours compact groups in the standardised Euclidean metric. A species can have a shape, or an overlap, that this geometry does not represent.

4.8 Further exercises

Exercise 4.1 (Pairs and means)**.** Prove that for any set $\{\boldsymbol{x}_i\}_{i=1}^N$ with mean $\overline{\boldsymbol{x}}$, $\sum_{i,l}\|\boldsymbol{x}_i - \boldsymbol{x}_l\|^2 = 2N\sum_i\|\boldsymbol{x}_i - \overline{\boldsymbol{x}}\|^2$. Use it to show that (4.2) equals (4.3).

Solution

Exercise 4.2 (Counting partitions)**.** List all partitions of $\{1,2,3,4\}$ into two non-empty clusters and compare with $S(4,2)$ from (4.5). Compute $S(10,3)$ from the formula.

Solution

Exercise 4.3 (K-means by hand)**.** Apply Algorithm 4.1 to the one-dimensional data $0, 1, 3, 8, 9, 11$ with $K = 2$ and initial means 0 and 1. Report the partition after each pass and the final value of $\widetilde{W}$.

Solution

Exercise 4.4 (A Voronoi boundary)**.** For means $(0,0)$, $(4,2)$, and $(0,4)$ find the three pairwise boundaries and verify that they meet at the circumcentre. What is the circumradius?

Solution

Exercise 4.5 (Reading the gap statistic)**.** For $k = 1, \ldots, 5$ the gap values are 0.30, 0.70, 1.10,

1.15, 1.00 and $s'(k) = 0.05, 0.05, 0.06, 0.06, 0.07$. Which k^* does (4.8) select?

Solution

Exercise 4.6 (A tempting wrong turn)**.** A student reports that $K = 10$ is best because it gives the smallest total within-cluster sum of squares among $K = 1, \ldots, 10$. Explain what is wrong with the argument and what to compute instead.

Solution

Exercise 4.7 (What a k-means diagram must show)**.** Consider Euclidean K-means in two dimensions with $K = 3$ and distinct fitted centroids.

(a) Describe the possible cluster boundaries and draw a labelled example containing observations, centroids and the resulting regions.

(b) Explain what the algorithm is minimising and how the centroids are updated.

(c) Explain why different starting centroids can produce different answers and how you would respond in practice.

Solution

Exercise 4.8 (Euclidean clusters)**.** Consider K-means with $K = 3$ in two dimensions.

(a) Describe the cluster boundaries and draw an example showing sample observations, the three fitted centroids and the assignment regions.

(b) Explain two differences between this method and a Gaussian finite mixture with component-specific covariance matrices, including the consequences for interpreting a fitted clustering.

Solution

Chapter 5

Mixture models and the EM algorithm

A point near a cluster boundary presents a useful objection to hard assignment: why must it belong entirely to one group? A mixture model gives a probabilistic answer. Imagine first drawing an unobserved component label, then drawing measurements from that component. We observe the measurements but lose the label. Summing over its possible values gives the observed density; estimating those missing memberships leads to EM.

An increase in the observed likelihood at every iteration is not evidence that the limit is a global maximum. For an unconstrained Gaussian mixture the likelihood can diverge, because a component covariance can collapse onto one or more sample points.

Source. Lecture notes, Sections 2.2–2.4 (pp. 39–50).

5.1 Finite mixture models

Definition 5.1 (Finite mixture model)**.** A *finite mixture model* assumes that the data have been generated by a distribution whose density is a finite sum of weighted component densities,

$$f(\boldsymbol{x}) = \sum_{h=1}^{H} \pi_h f_h(\boldsymbol{x}), \qquad 0 \leq \pi_h \leq 1, \quad \sum_{h=1}^{H} \pi_h = 1, \tag{5.1}$$

where H is the number of components, π_h is the probability that an observation is drawn from component h, and f_h is the hth component density. Equivalently, each observation comes from one of a finite set of parametric distributions.

The weights are the probabilities of the latent component, so they lie in $[0, 1]$ and sum to one. The component densities in this chapter are multivariate normal. That choice keeps the complete-data step inside the normal maximum likelihood theory of Chapter 2. Other families are possible. Mixtures of t distributions, of beta distributions, and of generalised linear models are surveyed in (McLachlan and Peel, 2000, Chapter 5).

A binary coordinate is outside the Gaussian density used here. The model in this chapter is for continuous measurements. One joint extension, the location model, fits a separate mixture at each value of a binary variable and multiplies by the probability of that value. Comparing the continuous distributions in the two binary groups shows whether the binary split corresponds to different clouds. The split multiplies the parameter count, and it shares no parameter across the two values. Several binary variables multiply the number of cells.

There are two interpretations worth keeping separate. Components may represent populations that we hope to identify, or they may simply be convenient pieces of a flexible density. A good fit supports the density approximation; it does not, by itself, establish the population story. In iris, agreement between components and species is something to check against the labels, not something built into a Gaussian mixture.

For fixed H and a specified component family, a finite mixture is a parametric model. Allowing H to increase with sample size gives an increasingly flexible family. The distinction depends on the model sequence, not on whether the density is a sum. Here we hold H fixed while deriving the fit, then consider its selection in the next chapter.

Three tasks follow. This chapter fits every parameter for a fixed H. Chapter 6 chooses H. Section 5.8 describes uncertainty in the fitted parameter.

5.2 The likelihood of a normal mixture

Fix H. The observations are continuous and p-dimensional, each component is multivariate normal, and the fitting principle is maximum likelihood, the same principle that produced $\overline{\boldsymbol{x}}$ and $\boldsymbol{A}/n$ for a single normal sample. Let $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$ be an iid sample from the mixture (5.1), stored as an $n \times p$ matrix $\boldsymbol{X}_{\text{all}}$. Collect the component parameters and the weights in one vector $\boldsymbol{\Psi}$. The constraint $\sum_h \pi_h = 1$ leaves only $\pi_1, \ldots, \pi_{H-1}$ free. For normal components,

$$\boldsymbol{\Psi} = (\boldsymbol{\mu}_1^\top, \ldots, \boldsymbol{\mu}_H^\top,\ \boldsymbol{\xi}^\top,\ \pi_1, \ldots, \pi_{H-1})^\top,$$

where $\boldsymbol{\xi}$ holds the distinct entries of $\boldsymbol{\Sigma}_1, \ldots, \boldsymbol{\Sigma}_H$. The number of free parameters is therefore

$$d = Hp + H\,\frac{p(p+1)}{2} + (H-1). \tag{5.2}$$

For $p = 2$ and $H = 3$ the count is $6 + 9 + 2 = 17$: three means of length 2, three covariance matrices with three distinct entries each, and two free weights. The observed likelihood multiplies the mixture densities. The component label was summed out inside each factor, so every factor is a sum,

$$L(\boldsymbol{\Psi}; \boldsymbol{X}_{\text{all}}) = \prod_{i=1}^{n} \sum_{h=1}^{H} \pi_h f_h(\boldsymbol{x}_i), \tag{5.3}$$

and the log-likelihood is the sum of the logarithms of those sums,

$$\begin{aligned} \ell(\boldsymbol{\Psi}; \boldsymbol{X}_{\text{all}}) &= \sum_{i=1}^{n} \ln\Big(\sum_{h=1}^{H} \pi_h f_h(\boldsymbol{x}_i) \Big) \\ &= \sum_{i=1}^{n} \ln\Big(\sum_{h=1}^{H} \frac{\pi_h}{(2\pi)^{p/2} |\boldsymbol{\Sigma}_h|^{1/2}} \mathrm{e}^{-\frac{1}{2}(\boldsymbol{x}_i - \boldsymbol{\mu}_h)^\top \boldsymbol{\Sigma}_h^{-1} (\boldsymbol{x}_i - \boldsymbol{\mu}_h)} \Big). \end{aligned} \tag{5.4}$$

The logarithm of a sum does not become a sum of logarithms, so the observed log-likelihood does not separate across components. Newton–Raphson on ℓ is a different algorithm: every derivative has to pass through the sum. EM replaces ℓ by an expected complete-data log-likelihood, for which the normal formulas of Section 2.1 apply with weights.

5.3 The idea behind EM

If the component of each observation were known, the fit would be the grouped normal problem already solved in Chapter 2. The observations in component h would contribute their sample mean, their sample covariance, and their share of the sample. The labels are missing. The *expectation–maximisation (EM) algorithm* is built on that complete-data problem. It alternates an expectation of the missing labels, computed at the current parameter, with a maximisation that treats the expectations as weights.

In general, EM maximises a log-likelihood by adding latent variables that make the complete-data maximisation direct. The E step takes the conditional expectation of the complete log-likelihood given the observations and the current parameter. The M step maximises that expectation in the parameter. The original sample is called incomplete, and its likelihood is the incomplete likelihood. The sample together with the latent variables is the complete sample.

5.3.1 What the stationarity equations suggest

At an interior maximum the derivative of the observed log-likelihood is zero. For mixtures, the solutions of interest are not on the boundary of the parameter space. Differentiating ℓ and rearranging produces a fixed-point equation for the weights: the maximum likelihood estimate satisfies

$$\widehat{\pi}_h = \frac{1}{n}\sum_{i=1}^{n} \tau_h(\boldsymbol{x}_i; \widehat{\boldsymbol{\Psi}}), \qquad h = 1, \ldots, H, \tag{5.5}$$

where $\tau_h(\boldsymbol{x}_i; \widehat{\boldsymbol{\Psi}})$ is the posterior probability that observation i came from component h,

$$\tau_h(\boldsymbol{x}_i; \boldsymbol{\Psi}) = \frac{\pi_h f_h(\boldsymbol{x}_i; \boldsymbol{\theta}_h)}{\sum_{m=1}^{H} \pi_m f_m(\boldsymbol{x}_i; \boldsymbol{\theta}_m)}, \tag{5.6}$$

and $\boldsymbol{\theta}_h$ holds the mean and covariance parameters of component h. The display is not a closed form. The weights $\widehat{\pi}_h$ appear on the right-hand side through τ_h, which depends on the whole of $\widehat{\boldsymbol{\Psi}}$. Earlier iterative attacks on this equation had properties that were difficult to describe. The equation does identify the missing data. The weights it asks for are the conditional probabilities of the component indicators.

5.4 The complete-data likelihood

Give observation i a latent vector $\boldsymbol{z}_i$ of length H, containing one 1 and otherwise zeros. The indicator z_{ih} answers whether component h generated that observation. For continuous $\boldsymbol{X}_i$, the following expression is a joint density in $\boldsymbol{x}_i$ and a probability mass in $\boldsymbol{z}_i$; it is not the probability of observing one exact real-valued vector. The product notation selects the active component:

$$f(\boldsymbol{x}_i, \boldsymbol{z}_i; \boldsymbol{\Psi}) = \mathbb{P}(\boldsymbol{Z}_i = \boldsymbol{z}_i; \boldsymbol{\Psi})\, f(\boldsymbol{x}_i \mid \boldsymbol{Z}_i = \boldsymbol{z}_i; \boldsymbol{\Psi}) \tag{5.7}$$

$$= \prod_{h=1}^{H} \pi_h^{z_{ih}} f_h(\boldsymbol{x}_i)^{z_{ih}}. \tag{5.8}$$

Every coordinate of $\boldsymbol{z}_i$ but one is zero, so every factor raised to the power 0 equals 1. The product keeps the component that generated $\boldsymbol{x}_i$. The complete likelihood and its logarithm are therefore

$$L_C(\boldsymbol{\Psi}; \boldsymbol{X}_{\text{all}}, \boldsymbol{Z}) = \prod_{i=1}^{n} \prod_{h=1}^{H} \pi_h^{z_{ih}} f_h(\boldsymbol{x}_i)^{z_{ih}}, \tag{5.9}$$

$$\ell_C(\boldsymbol{\Psi}; \boldsymbol{X}_{\text{all}}, \boldsymbol{Z}) = \sum_{i=1}^{n} \sum_{h=1}^{H} z_{ih} \big[\ln \pi_h + \ln f_h(\boldsymbol{x}_i; \boldsymbol{\theta}_h) \big]. \tag{5.10}$$

For normal components, substitute the normal log-density from (2.3), one term for each observation and each component it might belong to:

$$\ell_C = \sum_{i=1}^{n} \sum_{h=1}^{H} z_{ih} \Big[\ln \pi_h - \frac{p}{2} \ln(2\pi) - \frac{1}{2} \ln |\boldsymbol{\Sigma}_h| - \frac{1}{2} (\boldsymbol{x}_i - \boldsymbol{\mu}_h)^\top \boldsymbol{\Sigma}_h^{-1} (\boldsymbol{x}_i - \boldsymbol{\mu}_h) \Big]. \tag{5.11}$$

Compare this with the observed log-likelihood (5.4). There is no logarithm of a sum, and the expression is linear in the indicators z_{ih}. Linearity is what lets the E step replace each unobserved indicator by its conditional expectation.

5.5 The E and M steps

Let $\boldsymbol{\Psi}^{(k)}$ be the parameter at iteration k. The update maximises the conditional expected complete log-likelihood. The expectation is over the latent indicators, drawn from their conditional distribution at $\boldsymbol{\Psi}^{(k)}$, with the observations held fixed. That expectation is the Q function,

$$Q(\boldsymbol{\Psi}; \boldsymbol{\Psi}^{(k)}) = \mathbb{E}_{\boldsymbol{Z}; \boldsymbol{\Psi}^{(k)}} \{ \ell_C(\boldsymbol{\Psi}) \mid \boldsymbol{X}_{\text{all}} \}. \tag{5.12}$$

The two arguments of Q have different jobs. The current value $\boldsymbol{\Psi}^{(k)}$ determines the distribution of the missing labels. The candidate $\boldsymbol{\Psi}$ is what we vary during maximisation. Compute the memberships once in the E step, hold them fixed in the M step, then recompute them at the new parameter. Updating the memberships inside the M-step derivatives would be a different calculation.

5.5.1 The E step

The complete log-likelihood (5.10) is linear in the indicators, so the conditional expectation passes inside the sum. The expectation of a binary indicator is the probability that it equals one. Bayes' rule for densities, (1.4), identifies that probability with the posterior membership, evaluated at the current parameter:

$$\begin{aligned} Q(\boldsymbol{\Psi}; \boldsymbol{\Psi}^{(k)}) &= \mathbb{E}\Big\{ \sum_{i=1}^{n} \sum_{h=1}^{H} Z_{ih} \big[\ln \pi_h + \ln f_h(\boldsymbol{x}_i; \boldsymbol{\theta}_h) \big] \;\Big|\; \boldsymbol{X}_{\text{all}} \Big\} \\ &= \sum_{i=1}^{n} \sum_{h=1}^{H} \mathbb{E}(Z_{ih} \mid \boldsymbol{x}_i) \big[\ln \pi_h + \ln f_h(\boldsymbol{x}_i; \boldsymbol{\theta}_h) \big] \\ &= \sum_{i=1}^{n} \sum_{h=1}^{H} \tau_h(\boldsymbol{x}_i; \boldsymbol{\Psi}^{(k)}) \big[\ln \pi_h + \ln f_h(\boldsymbol{x}_i; \boldsymbol{\theta}_h) \big]. \end{aligned} \tag{5.13}$$

The last step of the display is that identification. The current parameter $\boldsymbol{\Psi}^{(k)}$ is held fixed inside τ_h. The free parameter $\boldsymbol{\Psi}$ remains in $\ln \pi_h + \ln f_h$, which is what the M step maximises. Explicitly,

$$\tau_h(\boldsymbol{x}_i;\boldsymbol{\Psi}^{(k)}) = \frac{\pi_h^{(k)} f_h(\boldsymbol{x}_i;\boldsymbol{\theta}_h^{(k)})}{\sum_{m=1}^{H} \pi_m^{(k)} f_m(\boldsymbol{x}_i;\boldsymbol{\theta}_m^{(k)})}. \tag{5.14}$$

5.5.2 The M step

Maximise (5.13) in $\boldsymbol{\Psi}$, with the weights $\tau_{ih} := \tau_h(\boldsymbol{x}_i;\boldsymbol{\Psi}^{(k)})$ held at the values just computed. The objective separates into a term in the weights π_h and a weighted normal log-likelihood for each component. Write $N_h = \sum_i \tau_{ih}$ for the effective sample size of component h.

Weights.

Maximise $\sum_h N_h \ln \pi_h$ subject to $\sum_h \pi_h = 1$, the same constraint that made only $H-1$ weights free in the observed model. The Lagrangian $\sum_h N_h \ln \pi_h - \kappa(\sum_h \pi_h - 1)$ has derivative $N_h/\pi_h - \kappa$. Set the derivative to zero, so $\pi_h = N_h/\kappa$. Sum over h. The identity $\sum_h N_h = \sum_i \sum_h \tau_{ih} = n$ forces $\kappa = n$, and therefore

$$\pi_h^{(k+1)} = \frac{1}{n}\sum_{i=1}^{n} \tau_h(\boldsymbol{x}_i;\boldsymbol{\Psi}^{(k)}) = \frac{N_h}{n}. \tag{5.15}$$

Means.

The terms of (5.13) that involve $\boldsymbol{\mu}_h$ are $-\frac{1}{2}\sum_i \tau_{ih}(\boldsymbol{x}_i - \boldsymbol{\mu}_h)^\top \boldsymbol{\Sigma}_h^{-1}(\boldsymbol{x}_i - \boldsymbol{\mu}_h)$. The gradient in $\boldsymbol{\mu}_h$ is $\boldsymbol{\Sigma}_h^{-1}\sum_i \tau_{ih}(\boldsymbol{x}_i - \boldsymbol{\mu}_h)$. Set it to zero and multiply by $\boldsymbol{\Sigma}_h$, which is positive definite at an interior point. The solution is the weighted mean, the same critical point as (2.6) with weights τ_{ih} in place of equal weights:

$$\boldsymbol{\mu}_h^{(k+1)} = \frac{\sum_{i=1}^{n} \tau_{ih}\, \boldsymbol{x}_i}{\sum_{i=1}^{n} \tau_{ih}}. \tag{5.16}$$

Covariance matrices.

Pass to the precision $\boldsymbol{\Theta}_h = \boldsymbol{\Sigma}_h^{-1}$, as in Section 2.1. The terms that involve $\boldsymbol{\Theta}_h$ are $\frac{1}{2}N_h \ln|\boldsymbol{\Theta}_h| - \frac{1}{2}\sum_i \tau_{ih}(\boldsymbol{x}_i - \boldsymbol{\mu}_h)^\top \boldsymbol{\Theta}_h(\boldsymbol{x}_i - \boldsymbol{\mu}_h)$. The same two matrix derivatives used there, $\partial \ln|\boldsymbol{\Theta}|/\partial\boldsymbol{\Theta} = \boldsymbol{\Theta}^{-1}$ and $\partial(\boldsymbol{v}^\top\boldsymbol{\Theta}\boldsymbol{v})/\partial\boldsymbol{\Theta} = \boldsymbol{v}\boldsymbol{v}^\top$, produce $\frac{1}{2}N_h\boldsymbol{\Theta}_h^{-1} - \frac{1}{2}\sum_i \tau_{ih}(\boldsymbol{x}_i - \boldsymbol{\mu}_h)(\boldsymbol{x}_i - \boldsymbol{\mu}_h)^\top$. Set the derivative to zero and insert the updated mean, which plays the role $\overline{\boldsymbol{x}}$ played for a single normal sample:

$$\boldsymbol{\Sigma}_h^{(k+1)} = \frac{\sum_{i=1}^{n} \tau_{ih}\,(\boldsymbol{x}_i - \boldsymbol{\mu}_h^{(k+1)})(\boldsymbol{x}_i - \boldsymbol{\mu}_h^{(k+1)})^\top}{\sum_{i=1}^{n} \tau_{ih}}. \tag{5.17}$$

The formulas now have a familiar shape: proportions, means and covariance matrices, with fractional membership replacing a hard group label. The divisor is N_h, not $N_h - 1$, because this is likelihood maximisation. These updates require $N_h > 0$ and positive definite updated covariance matrices. A collapsed component is a failure of those conditions, not an invitation to divide by zero.

Algorithm 5.2 (EM for a normal mixture)**.** Choose initial values $\boldsymbol{\Psi}^{(0)}$. For $k = 0, 1, 2, \ldots$:

1. *E step:* compute $\tau_{ih} = \tau_h(\boldsymbol{x}_i;\boldsymbol{\Psi}^{(k)})$ by (5.14) for all i, h.
2. *M step:* compute $\pi_h^{(k+1)}$, $\boldsymbol{\mu}_h^{(k+1)}$, and $\boldsymbol{\Sigma}_h^{(k+1)}$ by (5.15), (5.16), (5.17).
3. Stop when the change in $\ell(\boldsymbol{\Psi}^{(k)}; \boldsymbol{X}_{\text{all}})$ is smaller than a threshold.

EM also fits the minorisation–maximisation viewpoint (Lange, 2013). More precisely, $Q(\boldsymbol{\Psi}; \boldsymbol{\Psi}^{(k)})$ plus a constant independent of $\boldsymbol{\Psi}$ is a lower bound on the observed log-likelihood, touching it at the current parameter. The constant matters for the touching statement, although it does not change the maximiser.

5.5.3 Why the likelihood never decreases

The useful guarantee is monotonicity: an exact EM step cannot decrease the observed log-likelihood (Dempster et al., 1977). Convergence of the likelihood values requires an upper bound; convergence of the parameters requires further conditions. Under the regularity conditions studied by Wu (1983), limit points are stationary, which does not make them global maxima. Keep those three claims separate while reading the proof.

Write the observed log-density by Bayes' rule as $\ln f(\boldsymbol{X}_{\text{all}}; \boldsymbol{\Psi}) = \ell_C(\boldsymbol{\Psi}; \boldsymbol{X}_{\text{all}}, \boldsymbol{Z}) - \ln \mathbb{P}(\boldsymbol{Z} \mid \boldsymbol{X}_{\text{all}}; \boldsymbol{\Psi})$. Take the expectation over $\boldsymbol{Z}$ drawn from the conditional distribution at the current parameter $\boldsymbol{\Psi}^{(k)}$. The first term becomes Q, and the second becomes a function G:

$$\ell(\boldsymbol{\Psi}) = Q(\boldsymbol{\Psi}; \boldsymbol{\Psi}^{(k)}) - G(\boldsymbol{\Psi}; \boldsymbol{\Psi}^{(k)}), \qquad G(\boldsymbol{\Psi}; \boldsymbol{\Psi}^{(k)}) = \mathbb{E}_{\boldsymbol{\Psi}^{(k)}}\big[\ln \mathbb{P}(\boldsymbol{Z} \mid \boldsymbol{X}_{\text{all}}; \boldsymbol{\Psi}) \mid \boldsymbol{X}_{\text{all}}\big].$$

Gibbs' inequality is nonnegativity of Kullback–Leibler divergence, so $G(\boldsymbol{\Psi}; \boldsymbol{\Psi}^{(k)}) \le G(\boldsymbol{\Psi}^{(k)}; \boldsymbol{\Psi}^{(k)})$ for every $\boldsymbol{\Psi}$. Split the change in the observed log-likelihood:

$$\begin{aligned} \ell(\boldsymbol{\Psi}^{(k+1)}) - \ell(\boldsymbol{\Psi}^{(k)}) = {} & \underbrace{Q(\boldsymbol{\Psi}^{(k+1)}; \boldsymbol{\Psi}^{(k)}) - Q(\boldsymbol{\Psi}^{(k)}; \boldsymbol{\Psi}^{(k)})}_{\ge 0 \ \text{(M step)}} \\ & + \underbrace{G(\boldsymbol{\Psi}^{(k)}; \boldsymbol{\Psi}^{(k)}) - G(\boldsymbol{\Psi}^{(k+1)}; \boldsymbol{\Psi}^{(k)})}_{\ge 0 \ \text{(Gibbs)}} \ \ge \ 0. \end{aligned}$$

Both terms in the display are nonnegative. In fact, increasing Q is enough; finding its exact maximum is sufficient but stronger than necessary. This gives generalised EM. At a differentiable interior fixed point, with differentiation under the expectation justified, the score of G vanishes and the score of ℓ equals the score of Q. Hence such a fixed point is stationary. The interior and smoothness qualifications are essential near zero weights or singular covariances.

Key idea.
EM replaces an awkward log-of-a-sum by the expected complete-data log-likelihood. The E step computes the soft memberships τ_{ih}; the M step is the weighted version of the ordinary sample mean, covariance, and proportion; and the observed-data log-likelihood is non-decreasing at every iteration.

The six points below are small enough that both updates can be written out. The log-likelihood in the solution is the observed mixture log-likelihood, and the increase from one iteration to the next is the monotonicity just proved.

Example 5.3 (Two iterations on six points)**. Question.** Take the one-dimensional data $x = (-1.2, -0.4, 0.3, 2.1, 2.8, 3.5)$ and a two-component normal mixture with starting values $\pi = (0.5, 0.5)$, $\mu = (0, 3)$, $\sigma^2 = (1, 1)$. Perform the E and M steps.

Solution. *E step.* The component densities are

x_i	-1.2	-0.4	0.3	2.1	2.8	3.5
$f_1(x_i) = \phi(x_i; 0, 1)$	0.1942	0.3683	0.3814	0.0440	0.0079	0.0009
$f_2(x_i) = \phi(x_i; 3, 1)$	0.0001	0.0012	0.0104	0.2661	0.3910	0.3521
τ_{i1}	0.9997	0.9967	0.9734	0.1419	0.0198	0.0025
$\tau_{i2} = 1 - \tau_{i1}$	0.0003	0.0033	0.0266	0.8581	0.9802	0.9975

For example, $\tau_{41} = \dfrac{0.5 \times 0.0440}{0.5 \times 0.0440 + 0.5 \times 0.2661} = 0.1419$. The current log-likelihood is $\ell = \sum_i \ln\left(0.5 f_1(x_i) + 0.5 f_2(x_i)\right) = -10.861$.

M step. $N_1 = \sum_i \tau_{i1} = 3.1339$ and $N_2 = 2.8661$, so $\pi^{(1)} = (0.5223, 0.4777)$. The weighted means are $\mu_1^{(1)} = \sum_i \tau_{i1} x_i / N_1 = -0.3013$ and $\mu_2^{(1)} = 2.8067$. The weighted variances, about the *new* means, are $\sigma_1^{2\,(1)} = 0.7063$ and $\sigma_2^{2\,(1)} = 0.3888$.

The next iteration has $\ell = -9.808$, then -9.539, with $\pi \to (0.5002, 0.4998)$, $\mu \to (-0.4323, 2.8002)$, $\sigma^2 \to (0.378, 0.327)$ after three iterations. The log-likelihood is non-decreasing, as the argument above guarantees.

The fragment implements the normal-mixture iteration just derived. It normalises log densities after subtracting their row maximum, which avoids many underflow problems. The returned responsibilities and final likelihood are recomputed at the returned parameters. This is an educational unconstrained fit: it stops if a covariance loses positive definiteness or a component has zero numerical weight. It does not silently replace the model with a regularised one.

```
em_mixture <- function(x, mu, Sigma, pi, iters = 100L) {
  x <- as.matrix(x); n <- nrow(x); H <- length(pi)
  stopifnot(iters >= 0, iters == as.integer(iters),
            H == length(mu), H == length(Sigma),
            all(pi > 0), abs(sum(pi) - 1) < 1e-8)
  expectation <- function() {
    # Cholesky fails explicitly if a covariance is not positive definite.
    invisible(lapply(Sigma, chol))
    ld <- vapply(seq_len(H), function(h)
      log(pi[h]) + mvtnorm::dmvnorm(x, mu[[h]], Sigma[[h]], log = TRUE),
      numeric(n))
    a <- apply(ld, 1, max)
    stopifnot(all(is.finite(a)))
    e <- exp(ld - a)  # subtract before exponentiating
    z <- rowSums(e)
    list(tau = e / z, ll = sum(a + log(z)))
  }
  state <- expectation()
  loglik <- c(state$ll, numeric(iters))
  for (it in seq_len(iters)) {
    tau <- state$tau
    Nh <- colSums(tau)
    stopifnot(all(Nh > 0))
    pi <- Nh / n
    for (h in seq_len(H)) {
      mu[[h]] <- colSums(tau[, h] * x) / Nh[h]
      xc <- sweep(x, 2, mu[[h]])
      Sigma[[h]] <- crossprod(xc * tau[, h], xc) / Nh[h]
    }
    state <- expectation()  # evaluate the updated parameters
```

```
    loglik[it + 1L] <- state$ll
  }
  list(pi = pi, mu = mu, Sigma = Sigma,
       tau = state$tau, loglik = loglik)
}
```

5.6 Pathologies and practical advice

Unbounded likelihood.

For $H \geq 2$, keep one component broad with positive density at every observation. Place another mean exactly on one observation and let its covariance be $\varepsilon^2 I$. Its density at that point grows as ε^{-p}, while the broad component keeps every other likelihood factor positive. The likelihood therefore diverges as $\varepsilon \downarrow 0$. This construction proves unboundedness without pretending that the boundary contains ordinary finite maxima.

The same divergence occurs when a component is supported on too few points to determine a positive definite covariance. Two points can pin a component in the plane, three points in three dimensions, and so on. The remaining components are left to describe the rest of the sample, while the collapsed component is made narrower. Starting a component at one point with a very small variance is enough to begin the collapse.

In one dimension, stop and restart if a component variance falls below a threshold. In two dimensions the analogous failure is a component collapsing toward a line. A singular covariance is the algebraic form of that failure: the determinant is 0, and the precision in the normal density does not exist. The eigenvalues of a component covariance are the variances along its elliptical axes. Positive definiteness requires every one of them to be positive. If an eigenvalue becomes small, stop. A variance on its way to 0 is a failure of the unconstrained fit.

Non-identifiability.

Renaming components changes the parameter vector but leaves the density unchanged. Finite Gaussian mixtures with distinct components and positive weights are identifiable up to this permutation; coincident components or zero weights introduce additional non-identifiability (McLachlan and Peel, 2000). Before comparing component parameters across fits, decide how to match labels. Ordering means can help in one dimension, but becomes unstable when means are close.

Label switching is that permutation. Nothing in the model says that the component on the left is component 1. A start with μ_1 to the right of μ_2 returns the swapped labelling, and the weights and covariances move with the labels. Averaging parameters across restarts, as if the subscript 1 named the same component every time, mixes distinct components.

Initialisation.

The observed likelihood on one sample can have two local modes. An exact fixed point in the regular interior satisfies the stationary likelihood equations; a small numerical update alone is weaker evidence. It is not evidence that the mode is global. One start can finish with components of different shapes and little overlap, and another start can finish at a different stationary point. Further iteration from a stopped update stays at that point. Given the current parameter, the map is deterministic.

Different valid starts can enter different regions of the likelihood surface. Monotonicity alone does not guarantee that any one run reaches a finite stationary point: a run can approach a singular boundary. Inspect the likelihood, component weights and smallest covariance eigenvalues together.

One possible start assigns observations at random and fits each resulting group. Its means lie in the sample convex hull, but its covariances need not be well conditioned: a small group can already be singular. Random allocation also tends to place the means near the overall mean, giving weak initial separation. Check group sizes and covariance eigenvalues before calling EM.

A common alternative runs K-means with $K = H$, then uses the group proportions, means and covariances as initial values. This often gives more separated means, but the K-means criterion does not make every realised group spherical. A small or nearly collinear group can still produce a singular covariance. Initialisation supplies a feasible starting point; it does not settle whether the mixture is a good description of the data.

Use several starts to explore sensitivity to initialisation, and report how degeneracy is handled. An eigenvalue constraint or covariance penalty changes the fitted optimisation problem and should be stated explicitly. Keeping the largest finite value encountered is a computational choice, not a proof that an unconstrained maximum exists.

Detecting spurious solutions.

A component covariance on its way to zero makes the density, and therefore the log-likelihood, numerically unstable. Discard those runs. A run can also place a few points in a component whose covariance is still positive definite. The likelihood is then finite, and the component may still fail to describe a subgroup one is willing to interpret. Penalising small variances would move every component, including those that have not collapsed. A hard floor on the variances fails in both directions: a floor that is too low lets the iteration reach the undesirable mode and remain there, and a floor that is too high blocks legitimate components and blocks intermediate values the iteration may need. Requiring a minimum total weight $\sum_i \tau_{ih}$, with the floor set above 1, changes the algorithm, so the monotonicity argument no longer applies as stated. The eigenvalues of the component covariances, rather than a single variance, detect a collapse toward a lower-dimensional subspace. They are the variances along the elliptical axes, and a small eigenvalue is the signal to stop. In practice the best finite solutions often have to be compared by hand. If the only aim is density estimation, a small component that improves the approximation may be kept, and that inspection is less urgent.

5.7 Clustering with a fitted mixture

With a fitted parameter $\widehat{\boldsymbol{\Psi}}$ in hand, assign a point $\boldsymbol{x}$ to the component with the largest posterior probability. Under the fitted model that is the same as the component with the largest weighted density, which is the Bayes rule for 0–1 loss when the parameter is treated as known:

$$C(\boldsymbol{x}; \widehat{\boldsymbol{\Psi}}) = \arg\max_{h \in 1,\dots,H} \widehat{\tau}_h(\boldsymbol{x}) = \arg\max_{h \in 1,\dots,H} \widehat{\pi}_h f_h(\boldsymbol{x}; \widehat{\boldsymbol{\theta}}_h). \tag{5.18}$$

The components may differ in weight, orientation, and shape, which K-means, with one spherical covariance, cannot represent. The number $1 - \max_h \widehat{\tau}_h(\boldsymbol{x})$ measures the uncertainty of the assigned label. It is computed at $\widehat{\boldsymbol{\Psi}}$, so it is uncertainty about membership given the fitted parameter, not uncertainty about the parameter itself.

5.7.1 K-means as a limiting case

Restrict every component to a common spherical covariance $\boldsymbol{\Sigma}_h = \sigma^2 \boldsymbol{I}$, and give the components equal weights. Substitute those restrictions into the E step (5.14). The posterior membership then depends on $\boldsymbol{x}_i$ only through squared Euclidean distance to the component means,

$$\tau_{ih} = \frac{\exp\{-\|\boldsymbol{x}_i - \boldsymbol{\mu}_h\|^2/(2\sigma^2)\}}{\sum_m \exp\{-\|\boldsymbol{x}_i - \boldsymbol{\mu}_m\|^2/(2\sigma^2)\}}.$$

Send $\sigma^2 \to 0$. For each i the sum is dominated by the smallest distance, so $\tau_{ih} \to 1$ for a nearest mean and $\tau_{ih} \to 0$ otherwise, ties aside. The E step becomes the hard assignment of Algorithm 4.1, and the weighted mean (5.16) becomes the ordinary cluster mean. K-means is this small-variance, equal-weight, spherical limit of EM. The three restrictions are what replace ellipsoidal components by Voronoi cells.

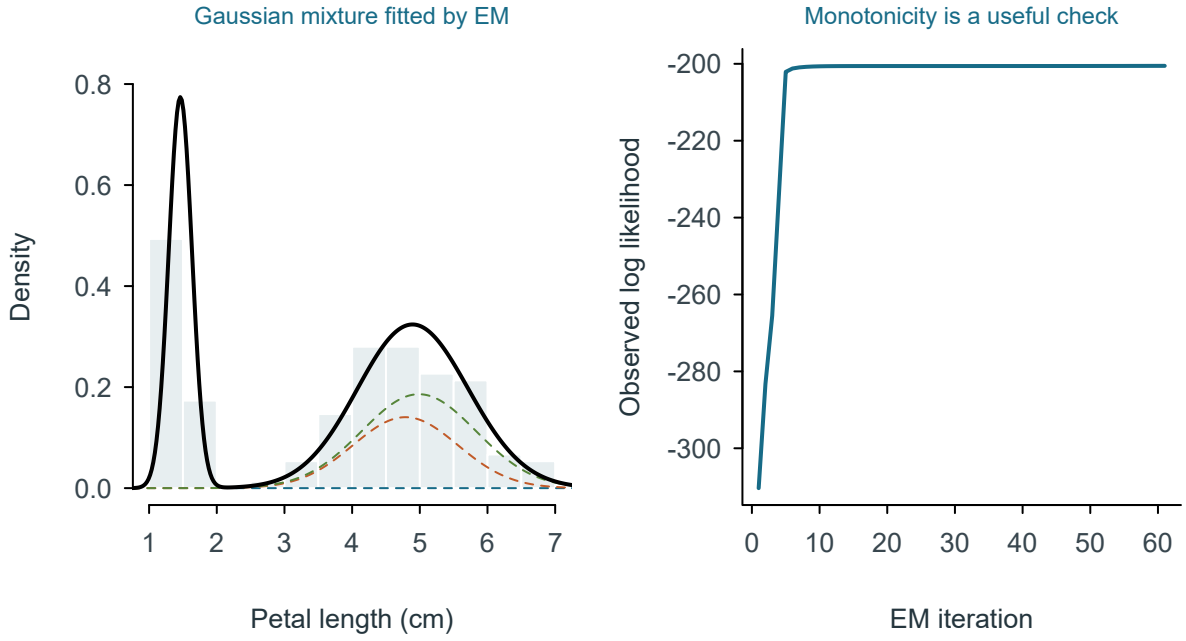


Figure 5.1: A three-component univariate Gaussian mixture fitted to iris petal length by explicit R EM code. The observed log likelihood is checked for monotonicity.

5.8 Uncertainty about the parameters

Standard errors can be built from the observed information $I(\widehat{\boldsymbol{\Psi}}; \boldsymbol{X}_{\text{all}})$, expressed through the complete-data information and the complete and incomplete scores (McLachlan and Peel, 2000, Sections 2.15–2.16). The algebra is heavy, and it still replaces the expected information by an observed-information approximation.

Bayesian analysis assigns a distribution to the unknown parameters and updates it using the likelihood. Its influence depends on the prior, the amount of information in the data and the model geometry; there is no general rule that a prior equals one extra observation. Mixtures require particular care around weakly identified components. The next chapter uses integration against a prior to motivate a model-comparison criterion.

The parametric bootstrap asks what would happen if the fitted mixture generated a new sample of the same size. Refit to each simulated sample and examine the variation in the estimates. This conditions on the chosen family and H; it cannot detect a mistaken component model simply by resimulating it. Near singular fits, ordinary bootstrap uncertainty summaries also need care.

Draw B bootstrap samples $\boldsymbol{X}^{(1)}, \ldots, \boldsymbol{X}^{(B)}$, each of size n, from the fitted mixture. For each observation, draw a component label with probabilities $\widehat{\pi}_h$, then draw the observation from that component's normal distribution.

Fit EM to each bootstrap sample. The fits are B parameter vectors $\widehat{\boldsymbol{\Psi}}^{(1)}, \ldots, \widehat{\boldsymbol{\Psi}}^{(B)}$.

The bootstrap covariance is the sample covariance of those vectors,

$$\widehat{\mathrm{Cov}}_B(\widehat{\boldsymbol{\Psi}}) = \frac{1}{B-1}\sum_{b=1}^{B}(\widehat{\boldsymbol{\Psi}}^{(b)} - \overline{\widehat{\boldsymbol{\Psi}}})(\widehat{\boldsymbol{\Psi}}^{(b)} - \overline{\widehat{\boldsymbol{\Psi}}})^\top, \qquad \overline{\widehat{\boldsymbol{\Psi}}} = \frac{1}{B}\sum_{b=1}^{B}\widehat{\boldsymbol{\Psi}}^{(b)}. \tag{5.19}$$

The labels are defined only up to the permutation in Section 5.6. Before the bootstrap vectors enter (5.19), match the components of each bootstrap fit to the components of $\widehat{\boldsymbol{\Psi}}$, for example by closeness of the means. Without that match, label switching inflates the covariance: the average mixes different components.

5.9 R laboratory: uncertainty about component membership

The fit is a three-component Gaussian mixture on the two petal measurements, with H held fixed. Fixing H separates the membership probabilities from the choice of how many components to use. The next chapter takes up that choice.

```
set.seed(5006)
mix <- mclust::Mclust(iris_x[, 3:4], G = 3,
                      modelNames = "VVV", verbose = FALSE)
table(Component = mix$classification, Species = iris_y)
#>           Species
#> Component setosa versicolor virginica
#>         1     50          0         0
#>         2      0         49         3
#>         3      0          1        47
round(head(mix$z), 3)
#>      [,1] [,2] [,3]
#> [1,]    1    0    0
#> [2,]    1    0    0
#> [3,]    1    0    0
#> [4,]    1    0    0
#> [5,]    1    0    0
#> [6,]    1    0    0
summary(1 - apply(mix$z, 1, max))
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>  0.0000  0.0000  0.0003  0.0310  0.0175  0.4596
stopifnot(max(abs(rowSums(mix$z) - 1)) < 1e-10)
```

Each row of `mix$z` is the posterior membership (5.6) at the fitted parameter. The summary one minus the largest membership probability is the uncertainty of the assigned component given that parameter and given $H = 3$. It does not include uncertainty about $\boldsymbol{\Psi}$ or about H. Permuting the component labels leaves the fitted density unchanged, which is why bootstrap components have to be matched before their parameters are averaged.

5.10 Further exercises

Exercise 5.1 (Mean and variance of a mixture)**.** For the mixture (5.1) with component means μ_h and variances σ_h^2 in one dimension, show $\mathbb{E}X = \sum_h \pi_h\mu_h =: \bar{\mu}$ and $\mathrm{Var}\,X = \sum_h \pi_h\sigma_h^2 + \sum_h \pi_h(\mu_h - \bar{\mu})^2$. Evaluate for $\pi = (0.3, 0.7)$, $\mu = (0, 4)$, $\sigma^2 = (1, 4)$.

Solution

Exercise 5.2 (An E step)**.** For the mixture of Exercise 5.1, compute $\tau_1(x)$ at $x = 1$ and $x = 3$.

Solution

Exercise 5.3 (An M step)**.** Using the two τ vectors from Exercise 5.2 as the memberships of the two data points $x = 1$ and $x = 3$, carry out the M step (5.15)–(5.17) for π, μ, and σ^2.

Solution

Exercise 5.4 (An unbounded likelihood)**.** Take $H = 2$, $p = 1$, with $\mu_1 = x_1$ and $\sigma_1^2 \to 0$, and the other component fixed at some $\mathrm{N}(\mu_2, \sigma_2^2)$ with $\pi_1 = \pi_2 = \frac{1}{2}$. Show that $\ell \to \infty$.

Solution

Exercise 5.5 (Counting parameters)**.** Using (5.2), find the number of free parameters for $H = 3$ components in $p = 2$ and $p = 5$ dimensions. How many if the covariance matrices are constrained to be diagonal, and how many if they are all equal to a common full matrix?

Solution

Exercise 5.6 (Label switching)**.** Show that the two-component parameter vectors $(\pi, \mu_1, \sigma_1^2, \mu_2, \sigma_2^2)$ and $(1-\pi, \mu_2, \sigma_2^2, \mu_1, \sigma_1^2)$ give the same density. Why does this complicate averaging the parameters across bootstrap fits?

Solution

Exercise 5.7 (Mixtures versus k-means)**.** Two analysts cluster the same two-dimensional data. One fits K-means; the other fits a three-component Gaussian mixture with unrestricted component covariance matrices.

(a) Explain two important differences between the fitted models. For each difference, explain its practical consequence.

(b) Write the posterior probability that an observation belongs to mixture component k, defining all quantities.

(c) If the fitted mixture has three components, must there be three biological populations? Explain.

Solution

Chapter 6

Model selection for stochastic models

We can fit a mixture for each proposed component count. Which fit should we keep? Training likelihood alone rewards flexibility, so it does not answer the question. First choose the target: predicting new observations and identifying a population's component count are different tasks. This chapter develops two ways of comparing models, marginal likelihood and held-out prediction, and explains why their answers can differ.

BIC comes from approximating an integral over parameters. We will derive its penalty in a regular model before asking whether the derivation applies to mixtures. That order matters: an attractive formula does not carry its assumptions with it automatically. Cross-validation and AIC then provide comparisons aimed at prediction.

Source. Lecture notes, Section 2.5 (pp. 50–57).

6.1 Bayes factors and posterior odds

Consider two candidate models, $\mathcal{M}_1$ and $\mathcal{M}_2$. Within each, specify a proper prior on its parameters; also specify prior probabilities for the models. Bayes' theorem then gives probabilities conditional on this candidate list and these priors. Neither candidate has to be literally true for us to compute the comparison, but the resulting probabilities do not include models omitted from the list (Kass and Raftery, 1995).

$$\mathbb{P}(\mathcal{M}_k \mid \boldsymbol{X}_{\text{all}}) = \frac{\mathbb{P}(\boldsymbol{X}_{\text{all}} \mid \mathcal{M}_k)\mathbb{P}(\mathcal{M}_k)}{\mathbb{P}(\boldsymbol{X}_{\text{all}} \mid \mathcal{M}_1)\mathbb{P}(\mathcal{M}_1) + \mathbb{P}(\boldsymbol{X}_{\text{all}} \mid \mathcal{M}_2)\mathbb{P}(\mathcal{M}_2)}, \qquad k = 1, 2. \tag{6.1}$$

The denominator does not depend on which model is being scored, so the comparison lives in the numerator. The *posterior odds* of $\mathcal{M}_1$ against $\mathcal{M}_2$ are

$$\frac{\mathbb{P}(\mathcal{M}_1 \mid \boldsymbol{X}_{\text{all}})}{\mathbb{P}(\mathcal{M}_2 \mid \boldsymbol{X}_{\text{all}})} = \frac{\mathbb{P}(\boldsymbol{X}_{\text{all}} \mid \mathcal{M}_1)\mathbb{P}(\mathcal{M}_1)}{\mathbb{P}(\boldsymbol{X}_{\text{all}} \mid \mathcal{M}_2)\mathbb{P}(\mathcal{M}_2)}, \tag{6.2}$$

and the *Bayes factor* for model 1 against model 2 is the ratio of marginal likelihoods,

$$B_{12} = \frac{\mathbb{P}(\boldsymbol{X}_{\text{all}} \mid \mathcal{M}_1)}{\mathbb{P}(\boldsymbol{X}_{\text{all}} \mid \mathcal{M}_2)}. \tag{6.3}$$

For either model, the posterior odds equal the Bayes factor times the prior odds. The same relation holds for any finite list of models. The marginal likelihood sometimes has a closed form. More often it is approximated, by Markov chain Monte Carlo or by the Laplace expansion of the next section.

The main difficulty is that models with different component counts have different parameter spaces. We do not compare the heights of two likelihood peaks and stop. We integrate within each model's own parameter space, then compare the resulting marginal likelihoods. That integration accounts for how much parameter space fits the observations well.

That integral is the obstacle. The marginal likelihood of model k is

$$\mathbb{P}(\boldsymbol{X}_{\text{all}} \mid \mathcal{M}_k) = \int_{\Theta_k} \mathbb{P}(\boldsymbol{X}_{\text{all}} \mid \boldsymbol{\Psi}_k, \mathcal{M}_k)\mathbb{P}(\boldsymbol{\Psi}_k \mid \mathcal{M}_k)\,\mathrm{d}\boldsymbol{\Psi}_k, \tag{6.4}$$

Think of the maximised likelihood as a peak height and the marginal likelihood as a prior-weighted area or volume. A tall but very narrow peak need not win against a lower, broader one. For continuous data the probability notation in these displays denotes densities in the observed measurements; the probabilities of the discrete model labels remain ordinary probabilities.

6.2 The Bayesian information criterion

Schwarz (1978) derived the large-sample penalty now called BIC. A clean way to organise its ingredients is a joint prior on the model index and that model's parameter:

$$p(k, \boldsymbol{\Psi}_k) = \mathbb{P}(\mathcal{M}_k)\, p(\boldsymbol{\Psi}_k \mid \mathcal{M}_k), \qquad \boldsymbol{\Psi}_k \in \Theta_k, \tag{6.5}$$

Each conditional prior is normalised on its own Θ_k. We do not add densities of vectors of different dimensions as though they shared one coordinate system. Under zero–one loss for choosing a model, the posterior-optimal decision is the model with largest posterior probability.

The posterior over models is the Bayesian summary. If one model must be reported, take the *maximum a posteriori* model, the model with the largest posterior probability. In (6.1) the denominator is common to every k, so the maximiser k^* satisfies

$$\begin{aligned} k^* &= \arg\max_k \mathbb{P}(\boldsymbol{X}_{\text{all}} \mid \mathcal{M}_k)\mathbb{P}(\mathcal{M}_k) && (6.6)\\ &= \arg\max_k \mathbb{P}(\mathcal{M}_k) \int_{\Theta_k} \mathbb{P}(\boldsymbol{X}_{\text{all}}, \boldsymbol{\Psi}_k \mid \mathcal{M}_k)\,\mathrm{d}\boldsymbol{\Psi}_k && (6.7)\\ &= \arg\max_k \mathbb{P}(\mathcal{M}_k) \int_{\Theta_k} \exp\left\{ \log \mathbb{P}(\boldsymbol{X}_{\text{all}}, \boldsymbol{\Psi}_k \mid \mathcal{M}_k)\right\} \mathrm{d}\boldsymbol{\Psi}_k. && (6.8) \end{aligned}$$

6.2.1 Laplace approximation of the integral

The integral in (6.8) is what has to be approximated. Expand $\log \mathbb{P}(\boldsymbol{X}_{\text{all}}, \boldsymbol{\Psi}_k \mid \mathcal{M}_k)$ to second order about a mode $\widetilde{\boldsymbol{\Psi}}_k$. The gradient vanishes at a mode, so the linear term is absent and the quadratic term is the negative Hessian:

$$\log \mathbb{P}(\boldsymbol{X}_{\text{all}}, \boldsymbol{\Psi}_k \mid \mathcal{M}_k) \approx \log \mathbb{P}(\boldsymbol{X}_{\text{all}}, \widetilde{\boldsymbol{\Psi}}_k \mid \mathcal{M}_k) - \tfrac{1}{2}(\boldsymbol{\Psi}_k - \widetilde{\boldsymbol{\Psi}}_k)^\top \boldsymbol{H}(\widetilde{\boldsymbol{\Psi}}_k)(\boldsymbol{\Psi}_k - \widetilde{\boldsymbol{\Psi}}_k), \tag{6.9}$$

Here $\boldsymbol{H}$ is the negative Hessian of the log joint density. An interior mode only guarantees a positive semidefinite negative Hessian; strict positive definiteness is an additional assumption. For example, $-x^4$ has a maximum at zero and zero second derivative there. Laplace's Gaussian approximation needs nonzero curvature in every parameter direction. A singular direction breaks this particular argument.

Substitute the quadratic approximation into (6.8):

$$k^* \approx \arg\max_k \mathbb{P}(\mathcal{M}_k)\mathbb{P}(\boldsymbol{X}_{\text{all}}, \widetilde{\boldsymbol{\Psi}}_k \mid \mathcal{M}_k) \int_{\Theta_k} \exp\Big\{ -\tfrac{1}{2}(\boldsymbol{\Psi}_k - \widetilde{\boldsymbol{\Psi}}_k)^\top \boldsymbol{H}(\widetilde{\boldsymbol{\Psi}}_k)(\boldsymbol{\Psi}_k - \widetilde{\boldsymbol{\Psi}}_k)\Big\} \mathrm{d}\boldsymbol{\Psi}_k. \quad (6.10)$$

Up to the normalising constant, the integrand is the density of $\mathrm{N}_{d_k}(\widetilde{\boldsymbol{\Psi}}_k, \boldsymbol{H}(\widetilde{\boldsymbol{\Psi}}_k)^{-1})$. The domain Θ_k is not $\mathbb{R}^{d_k}$. Weights lie in $(0, 1)$, variances are positive, and covariance matrices are constrained, so the integral of that normal kernel over the legal region is not exactly 1.

The normalising factor is $(2\pi)^{d_k/2}|\boldsymbol{H}|^{-1/2}$. Extending the local integral to all of $\mathbb{R}^{d_k}$ is justified asymptotically when the mass concentrates near an interior, identifiable parameter and the information per observation approaches a positive definite matrix. For a smooth positive prior, posterior mode and maximum likelihood estimator are then asymptotically close, not exactly equal. These assumptions keep the boundary away from the shrinking region that contributes most of the integral (Drton and Plummer, 2017, Section 2).

Use $|\boldsymbol{A}^{-1}| = |\boldsymbol{A}|^{-1}$ to write the approximate integral as

$$\int_{\Theta_k} \exp\Big\{ -\tfrac{1}{2}(\boldsymbol{\Psi}_k - \widetilde{\boldsymbol{\Psi}}_k)^\top \boldsymbol{H}(\boldsymbol{\Psi}_k - \widetilde{\boldsymbol{\Psi}}_k)\Big\} \mathrm{d}\boldsymbol{\Psi}_k \approx (2\pi)^{d_k/2}\, |\boldsymbol{H}(\widehat{\boldsymbol{\Psi}}_k)|^{-1/2}. \quad (6.11)$$

The approximate MAP index is therefore

$$k^* \approx \arg\max_k \; \mathbb{P}(\mathcal{M}_k)\, \mathbb{P}(\boldsymbol{X}_{\text{all}}, \widehat{\boldsymbol{\Psi}}_k \mid \mathcal{M}_k)\, (2\pi)^{d_k/2}\, |\boldsymbol{H}(\widehat{\boldsymbol{\Psi}}_k)|^{-1/2}. \quad (6.12)$$

The logarithm is strictly increasing, so it preserves the maximiser. Factor the joint probability at $\widehat{\boldsymbol{\Psi}}_k$ into the likelihood and the prior density:

$$\begin{aligned} k^* &\approx \arg\max_k \; \log \mathbb{P}(\mathcal{M}_k) + \log \mathbb{P}(\boldsymbol{X}_{\text{all}} \mid \widehat{\boldsymbol{\Psi}}_k, \mathcal{M}_k) + \log \mathbb{P}(\widehat{\boldsymbol{\Psi}}_k \mid \mathcal{M}_k) \\ &\qquad + \frac{d_k}{2}\log(2\pi) - \frac{1}{2}\log|\boldsymbol{H}(\widehat{\boldsymbol{\Psi}}_k)| \\ &\approx \arg\max_k \; \log \mathbb{P}(\mathcal{M}_k) + \ell(\widehat{\boldsymbol{\Psi}}_k; \boldsymbol{X}_{\text{all}}, \mathcal{M}_k) + \log \mathbb{P}(\widehat{\boldsymbol{\Psi}}_k \mid \mathcal{M}_k) \\ &\qquad + \frac{d_k}{2}\log(2\pi) - \frac{1}{2}\log|I(\widehat{\boldsymbol{\Psi}}_k, \boldsymbol{X}_{\text{all}})|, \end{aligned} \quad (6.13)$$

The observed information is the negative Hessian of the log-likelihood alone. The negative Hessian of the log joint density also contains prior curvature. In this regular large-sample calculation the likelihood contribution is of order n, while a fixed smooth prior contributes order one. Replacing the joint curvature by observed information is therefore another approximation, as the second line indicates. An EM output must also be checked before treating it as the required maximiser.

6.2.2 Dropping terms that do not scale with n

The next cut keeps only the terms that grow with n. The maximised log-likelihood is a sum of one log-density per observation, and the sum is not divided by n, so it grows as the sample grows. The model prior was chosen before the data arrived, so $\log \mathbb{P}(\mathcal{M}_k)$ does not grow with n. The prior density evaluated at the maximum likelihood estimator stays $O(1)$ under Schwarz's conditions. The term $\frac{d_k}{2}\log(2\pi)$ contains no n.

Drop every term that remains $O(1)$. That removes $\log \mathbb{P}(\mathcal{M}_k)$, which equal prior probabilities on the models would have removed on their own, the log prior density $\log \mathbb{P}(\widehat{\boldsymbol{\Psi}}_k \mid \mathcal{M}_k)$, and $\frac{d_k}{2}\log(2\pi)$. For n observations the observed information is of order n, provided the average information per

observation stays of order 1. That proviso is the regular case, and it is the step that fails near a vanishing component. Under it, $|I(\widehat{\boldsymbol{\Psi}}_k, \boldsymbol{X}_{\text{all}})| = n^{d_k}\left|\frac{1}{n} I(\widehat{\boldsymbol{\Psi}}_k, \boldsymbol{X}_{\text{all}})\right| = n^{d_k}\, O(1)$, so

$$-\frac{1}{2}\log\left|I(\widehat{\boldsymbol{\Psi}}_k, \boldsymbol{X}_{\text{all}})\right| = -\frac{d_k}{2}\log n + O(1). \tag{6.14}$$

Drop the remaining $O(1)$ term. The approximate MAP rule is

$$k^* \approx \arg\max_k \; \ell(\widehat{\boldsymbol{\Psi}}_k; \boldsymbol{X}_{\text{all}}, \mathcal{M}_k) - \frac{d_k}{2}\log n, \tag{6.15}$$

Multiply by -2, which reverses the direction of the optimisation, and the same rule is

$$\boxed{k^* \approx \arg\min_k \; \text{BIC}_k, \qquad \text{BIC}_k = -2\,\ell(\widehat{\boldsymbol{\Psi}}_k; \boldsymbol{X}_{\text{all}}, \mathcal{M}_k) + d_k \log n.} \tag{6.16}$$

6.2.3 Reading the criterion

BIC retains the terms that grow with sample size and discards bounded terms. It does not require exactly equal model priors or a flat parameter prior: many fixed, smooth, positive priors give the same leading expression. Their differences survive in the omitted terms, which can matter when BIC values are close. Retaining only one omitted constant would not generally recover an accurate marginal likelihood.

The penalty comes from integration, specifically the shrinking volume around a regular likelihood peak. With information of order n in each of d_k directions, that volume is of order $n^{-d_k/2}$. Taking logs gives $-(d_k/2)\log n$. Thus one extra parameter must improve the log-likelihood by more than $\frac{1}{2}\log n$ to improve BIC. This is a comparison rule, not a claim that a prior mechanically spreads itself more thinly whenever we add a parameter.

Key idea.
BIC is minus twice the maximised log-likelihood plus $d \log n$. It approximates (minus twice) the log of the marginal likelihood of a model, which is why it carries a Bayesian penalty: each additional parameter must buy at least $\frac{1}{2}\log n$ units of log-likelihood. To use it, fit each candidate, for instance $H = 1, \ldots, 10$ components, and choose the smallest BIC.

Mixtures need a separate qualification. Coincident components and zero weights make model-order comparison singular, so the ordinary Laplace argument is generally invalid there. BIC can nevertheless be consistent under particular mixture-model assumptions; consistency and accuracy as a marginal-likelihood approximation are different properties (Drton and Plummer, 2017, Sections 1–2). The unconstrained Gaussian family with collapsing covariances does not acquire a finite global MLE merely because we attach a BIC penalty. State the fitted covariance restrictions and candidate component counts when reporting the comparison.

6.3 Cross-validation of the likelihood

BIC approximates a marginal probability of the sample already observed. Cross-validation estimates a different functional: the expected log-likelihood of a new draw, with the parameter fixed at a fit from the current sample. Smyth (2000) fits on training observations and scores the log-likelihood on test observations, and selects the model with the highest test score. The target is

$$k^* = \arg\max_k \mathbb{E}_{\boldsymbol{X}} \log \mathbb{P}(\boldsymbol{X} \mid \boldsymbol{\Psi} = \widehat{\boldsymbol{\Psi}}_k, \mathcal{M}_k), \tag{6.17}$$

The expectation is over a new observation $\boldsymbol{X}$. For each candidate, $\widehat{\boldsymbol{\Psi}}_k$ is the maximum likelihood fit on the data already in hand, so the parameter is not recomputed inside the expectation. A test set that was excluded from the fit approximates the expectation, as an average or as a sum. Multiplying the sum by a positive constant does not change the maximiser, and the two forms select the same model:

$$k^* \approx \arg\max_k \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \log \mathbb{P}(\boldsymbol{x}_{\text{test},i} \mid \widehat{\boldsymbol{\Psi}}_k, \mathcal{M}_k) \;\approx\; \arg\max_k \sum_{i=1}^{n_{\text{test}}} \log \mathbb{P}(\boldsymbol{x}_{\text{test},i} \mid \widehat{\boldsymbol{\Psi}}_k, \mathcal{M}_k). \tag{6.18}$$

In v-fold cross-validation, refit every candidate on each training fold and sum its held-out log densities. Any preprocessing belongs inside those fits. The winning specification may then be refitted on all development data. If we also want an honest estimate of the selected procedure's predictive performance, use an untouched test sample or an outer cross-validation loop. The folds used to choose the model are validation folds, even if older notation calls them test folds (Cawley and Talbot, 2010, Section 5).

6.3.1 The Kullback–Leibler connection

The expected log-likelihood of a new observation differs from a Kullback–Leibler divergence by a constant that does not depend on the candidate. Let f be the density of a new observation and $\widehat{f}$ the fitted density. Then

$$D(f \,\|\, \widehat{f}) = \int \ln\Big(\frac{f(\boldsymbol{x})}{\widehat{f}(\boldsymbol{x})}\Big) f(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x} = \underbrace{\int f \ln f}_{\text{constant}} - \mathbb{E}_f \ln \widehat{f}(\boldsymbol{X}). \tag{6.19}$$

The integral $\int f \ln f$ does not depend on which candidate produced $\widehat{f}$. Maximising $\mathbb{E}_f \ln \widehat{f}(\boldsymbol{X})$ is therefore the same comparison as minimising $D(f \,\|\, \widehat{f})$. The divergence is not symmetric. The convention puts the density of the expectation, here the true f, in the first argument.

6.4 The Akaike information criterion

Akaike's criterion estimates the same Kullback–Leibler target from the training log-likelihood, with an asymptotic penalty in place of a test set (Akaike, 1974). The penalty is $2d_k$:

$$k^* \approx \arg\min_k \mathrm{AIC}_k, \qquad \mathrm{AIC}_k = -2\,\ell(\widehat{\boldsymbol{\Psi}}_k; \boldsymbol{X}_{\text{all}}, \mathcal{M}_k) + 2d_k. \tag{6.20}$$

For $n > e^2$, BIC charges more per parameter than AIC. This explains why AIC often favours more flexible candidates, but it does not make every larger choice an error: predictive density estimation and recovery of a component count have different goals. The usual $2d_k$ correction is itself an asymptotic result for regular likelihood fitting. Singular mixtures and severe misspecification require care with that justification as well (Akaike, 1974; Drton and Plummer, 2017).

Table 6.1 supplies numbers for comparing the rules. Its original simulation and restart history are unavailable, so treat it as an arithmetic exercise, not verified empirical evidence. The iris analysis below is the reproducible example.

Table 6.1: A supplied arithmetic illustration of choosing components with $n = 300$ ($p = 2$, full covariance matrices, so $d = 5, 11, 17, 23, 29, 35$ by (5.2)). The log-likelihood and CV values are inherited inputs; the original simulation is not reproducible from the supplied record. The preferred value of each criterion is in bold.

H	d	$\widehat{\ell}$	BIC	AIC	5-fold CV log-lik.
1	5	−1274.1	2576.7	2558.2	−1278.1
2	11	−1076.9	2216.5	2175.8	−1088.4
3	17	−1012.3	**2121.6**	2058.6	**−1033.9**
4	23	−999.7	2130.6	2045.4	−1040.8
5	29	−995.5	2156.4	2049.0	−1048.7
6	35	−986.7	2173.0	**2043.4**	−1058.7

Example 6.1 (Choosing H for the simulated mixture)**.** Use the displayed log-likelihoods as inputs. For $H = 3$, $d = 17$ and $n = 300$, giving BIC $= 2024.6 + 17 \log 300 = 2121.6$ after rounding. BIC and the supplied cross-validation scores prefer $H = 3$; AIC prefers $H = 6$. This difference illustrates the penalties, without establishing that either rule recovers a known generating model. In `mclust`, BIC has the opposite sign, $2\ell - d \log n$, so larger values are preferred (mclust developers, 2026).

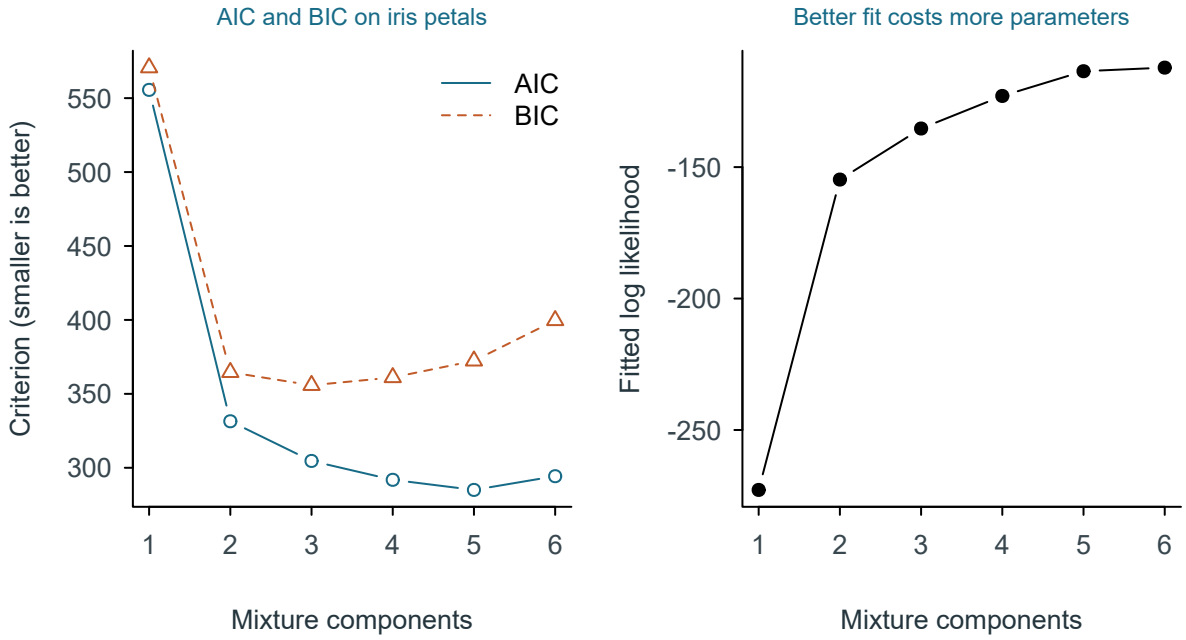


Figure 6.1: Full-covariance Gaussian mixtures fitted to iris petal measurements. AIC and BIC use the smaller-is-better convention.

The fragment asks `mclust` for $H = 1, \ldots, 6$ and several covariance structures. The matrix it stores is $2\ell - d \log n$, so the preferred entry is the largest. Equation (6.16) is the sign reversal of that quantity, and the preferred entry is then the smallest. The laboratory computes both and checks that they differ only by a sign.

```
library(mclust)
fit <- Mclust(x, G = 1:6)        # EM fits for H = 1,...,6 and several covariance
↪  structures
fit$BIC                           # mclust convention: 2*loglik - d*log(n),
↪  larger is better
```

```
summary(fit)                          # chosen H, mixing proportions, means
```

6.5 R laboratory: make the BIC convention explicit

One to six full-covariance mixtures are fitted to the same two petal measurements. The package reports $2\ell - d\log n$, larger meaning better. The table also computes the book's $-2\ell + d\log n$ from (6.16), smaller meaning better. The check is that the two columns sum to zero.

```
set.seed(6006)
fits <- lapply(1:6, function(g)
  mclust::Mclust(iris_x[, 3:4], G = g,
                 modelNames = "VVV", verbose = FALSE))
bic_table <- data.frame(
  Components = 1:6,
  Parameters = vapply(fits, function(f) f$df, numeric(1)),
  BIC = vapply(fits, function(f)
    -2 * f$loglik + f$df * log(nrow(iris)), numeric(1)))
print(bic_table, row.names = FALSE)
#>  Components Parameters   BIC
#>           1          5 570.6
#>           2         11 364.6
#>           3         17 355.8
#>           4         23 361.1
#>           5         29 372.3
#>           6         35 399.7
stopifnot(max(abs(bic_table$BIC +
                  vapply(fits, function(f) f$bic, numeric(1)))) < 1e-7)
```

On this list of full-covariance Gaussian mixtures, the BIC of (6.16) selects 3 components. That number describes these two measurements under this candidate family. It is not a count of biological species. A Gaussian component can approximate one piece of a non-Gaussian species distribution, and a different constraint on the covariances defines a different list of models.

6.6 Further exercises

Exercise 6.1 (Posterior odds)**.** Two models have prior probabilities $\mathbb{P}(\mathcal{M}_1) = 0.25$ and $\mathbb{P}(\mathcal{M}_2) = 0.75$, and the data give Bayes factor $B_{12} = 6$. Find the posterior odds and the posterior probability of $\mathcal{M}_1$.

Solution

Exercise 6.2 (BIC and AIC by hand)**.** With $n = 300$, the two-component fit has $\widehat{\ell} = -1100$ and $d = 11$, and the three-component fit has $\widehat{\ell} = -1050$ and $d = 17$. Compute BIC and AIC for each and state which model each criterion selects.

Solution

Exercise 6.3 (The Laplace integral in one dimension)**.** For scalar θ show $\int_{-\infty}^{\infty} \exp\{-\tfrac{1}{2}h(\theta - \tilde{\theta})^2\}\,\mathrm{d}\theta = \sqrt{2\pi/h}$, and explain how this is the $d_k = 1$ case of (6.11). If $h = nI_1$ for a constant I_1, show that $-\tfrac{1}{2}\log h = -\tfrac{1}{2}\log n + O(1)$.

Solution

Exercise 6.4 (KL divergence)**.** Show that $D(f\|\widehat{f}) \geq 0$ with equality only when $\widehat{f} = f$ (use $\ln u \leq u - 1$). Show that $D(f\|\widehat{f}) = \int f \ln f - \mathbb{E}_f \ln \widehat{f}(\boldsymbol{X})$, and explain why selecting the model with the largest expected log-likelihood on new data selects the model closest to f in this sense.

Solution

Exercise 6.5 (A tempting wrong turn)**.** A student selects the number of mixture components by maximising the log-likelihood on the training data, for $H = 1, \dots, 10$. Why will this almost always select $H = 10$ (or the largest value tried), and which two changes to the procedure in this chapter fix it?

Solution

Exercise 6.6 (Choosing model complexity)**.** For $n = 100$, three fitted density models have the following maximised log-likelihoods ℓ and parameter counts d:

Model	ℓ	d
A	-180	5
B	-170	9
C	-166	14

Use BIC $= -2\ell + d \log n$.

(a) Compute BIC for each model and choose the preferred one under this convention.

(b) Explain why choosing the largest training likelihood alone would not provide the same protection against overfitting.

(c) Explain why a program reporting $2\ell - d \log n$ should select its largest value instead.

Solution

Exercise 6.7 (Mixture selection and uncertainty)**.** Two two-dimensional Gaussian mixtures fitted to $n = 80$ observations have $(\ell, d) = (-140, 11)$ and $(-133, 17)$. Use BIC $= -2\ell + d \log n$.

(a) Explain how posterior component probabilities are calculated and why they are more informative than a hard label for an observation in an overlap region.

(b) Calculate BIC and choose between the two fitted candidates.

(c) Would a larger fitted training likelihood prove that a model predicts new data better? Explain.

Solution

Chapter 7

Hierarchical clustering and kernel density estimation

Sometimes choosing one partition is premature. We may want to see which groups merge first and which remain separate until the end. Hierarchical clustering records that sequence. Sometimes the question is instead where observations tend to occur. Kernel density estimation smooths the sample into a density. These are different outputs, but both make us confront the same issue: how much detail should we retain?

Both outputs depend on scale and on a tuning choice. A height in a dendrogram is the linkage dissimilarity at which two clusters merged. It is not a posterior probability that those clusters are the groups which generated the sample. A peak in a density estimate is a feature of the smoothed distribution. It need not be a separate population.

Source. Lecture notes, Sections 2.6–2.7 (pp. 57–62).

7.1 Hierarchical clustering

A hierarchy gives nested partitions, from single observations to the whole sample. Start by choosing a meaningful dissimilarity and its scale; standardising variables is one possible choice, not an automatic prerequisite. Then choose how to compare groups using their pointwise dissimilarities. Once ties are resolved, the merge rule determines the tree.

The original vectors are not required. Edit distance between words, or between languages, counts insertions, deletions, and substitutions. The same counting compares genetic sequences. People can be clustered from the strength of a relationship, and sites from an ecological similarity, with no coordinates in sight. Single linkage and complete linkage, defined below, use only those pairwise numbers.

7.1.1 Agglomerative and divisive clustering

Agglomerative clustering starts with each observation alone and merges neighbouring clusters until one cluster remains. Divisive clustering starts with one cluster and splits until every observation is alone. Either way the hierarchy has n levels, counting the single cluster at the top, and n leaves. The picture of the hierarchy is a *dendrogram*. Agglomerative clustering is the usual choice because the first merges are between points, where the dissimilarity is already defined. A divisive step has to invent a split of a set.

The pointwise dissimilarity may be Euclidean distance, or any other dissimilarity $d(\boldsymbol{x}, \boldsymbol{y})$. The arguments may be observation vectors or summaries such as cluster means, when the linkage uses means.

Algorithm 7.1 (Agglomerative clustering)**.** Start with n singleton clusters. At each step, find the two closest clusters under the chosen inter-cluster distance (a *linkage*) and merge them. Record the distance at which each merge occurs. Stop when one cluster remains.

The first merge compares points, so d applies directly. Every later merge compares sets. A linkage is a rule that extends d from points to pairs of clusters $\mathcal{D}_1$ and $\mathcal{D}_2$. Three rules are used here. They answer different questions about what makes two sets close.

Single linkage, or nearest neighbour, takes the distance between the closest pair with one point in each cluster, $d_{\mathrm{S}}(\mathcal{D}_1, \mathcal{D}_2) = \min\{d(\boldsymbol{x}, \boldsymbol{y}) : \boldsymbol{x} \in \mathcal{D}_1,\ \boldsymbol{y} \in \mathcal{D}_2\}$. The clusters are close as soon as they touch.

Complete linkage, or furthest neighbour, takes the distance between the farthest pair, $d_{\mathrm{C}}(\mathcal{D}_1, \mathcal{D}_2) = \max\{d(\boldsymbol{x}, \boldsymbol{y}) : \boldsymbol{x} \in \mathcal{D}_1,\ \boldsymbol{y} \in \mathcal{D}_2\}$. A merge is cheap only when every cross pair is close.

Average linkage takes the mean of all cross pairs. That average is group-average linkage. Distance between cluster means is a different rule, centroid linkage, and the note under the formula records where the two disagree.

Average linkage The group-average distance is

$$d_{\mathrm{A}}(\mathcal{D}_1, \mathcal{D}_2) = \frac{1}{|\mathcal{D}_1||\mathcal{D}_2|} \sum_{\boldsymbol{x} \in \mathcal{D}_1} \sum_{\boldsymbol{y} \in \mathcal{D}_2} d(\boldsymbol{x}, \boldsymbol{y}), \tag{7.1}$$

where $|\cdot|$ is the cardinality of the multiset (clusters must be allowed to be multisets to allow duplicates).

Terminology check.
Averaging distances and taking the distance between averages are different operations. Formula (7.1) is group-average linkage; distance between centroids defines centroid linkage. R's `hclust(method = "average")` implements the former. Centroid linkage can produce inversions, in which a later merge has a smaller height (R Core Team, 2026c, Details).

7.1.2 Properties of the linkages

Read each linkage as a different question. Single linkage asks whether any cross-pair is close, so a chain of nearby points can connect distant ends. Complete linkage asks whether every cross-pair is close, discouraging large diameters. Average linkage asks about the typical cross-pair. These criteria suggest tendencies, not guarantees about spherical shape or equal cluster sizes.

The partition can change substantially when a new sample is drawn from the same distribution, or when n changes. Single linkage shows this as *chaining*. A gap that separates two groups at one sample size can contain points when n is larger, and the linkage then joins the groups at the height of the closest new pair across the gap. Complete linkage is sensitive to distant outliers: one far point raises the maximum pairwise distance and can block a merge of clusters that are otherwise close. Average linkage lies between those two behaviours. No linkage is the most stable under every contamination pattern and every cluster shape. Single and complete linkage need only the pairwise dissimilarities. A linkage that uses means also needs an average of points, which may not exist for the dissimilarities of the previous section.

The six points below have increasing gaps between neighbours. Single linkage follows those gaps one at a time and builds a chain. Complete linkage can join interior pairs before it attaches the far point. The example records both sequences of merge heights, and average linkage is included in the figure.

Example 7.2 (Chaining on six points)**. Question.** Cluster the one-dimensional points 0, 1, 2.1, 3.3, 4.6, 6.9 by single and complete linkage.

Solution. The gaps between neighbours are 1, 1.1, 1.2, 1.3, 2.3. *Single linkage*: merge $\{0\}, \{1\}$ at 1.0; then $\{0, 1\}$ with $\{2.1\}$ at 1.1 (the nearest points are 1 and 2.1); then with $\{3.3\}$ at 1.2; with $\{4.6\}$ at 1.3; finally with $\{6.9\}$ at 2.3. The tree is a chain: each point is attached to the growing cluster through its nearest neighbour, and the merge heights are exactly the neighbour gaps. *Complete linkage*: the first merge is again $\{0, 1\}$ at 1.0, but then $\{2.1, 3.3\}$ forms at 1.2 (the next smallest distance), $\{4.6, 6.9\}$ at 2.3, $\{0, 1\} \cup \{2.1, 3.3\}$ at 3.3 (the maximum distance, from 0 to 3.3), and the two halves merge at 6.9. Complete linkage finds three pairs of compact groups; single linkage finds only a chain with a possible split of 6.9 from the rest. Figure 7.1 compares the dendrograms, including average linkage, whose merge heights are 1.0, 1.2, 1.9, 2.83, 4.7.

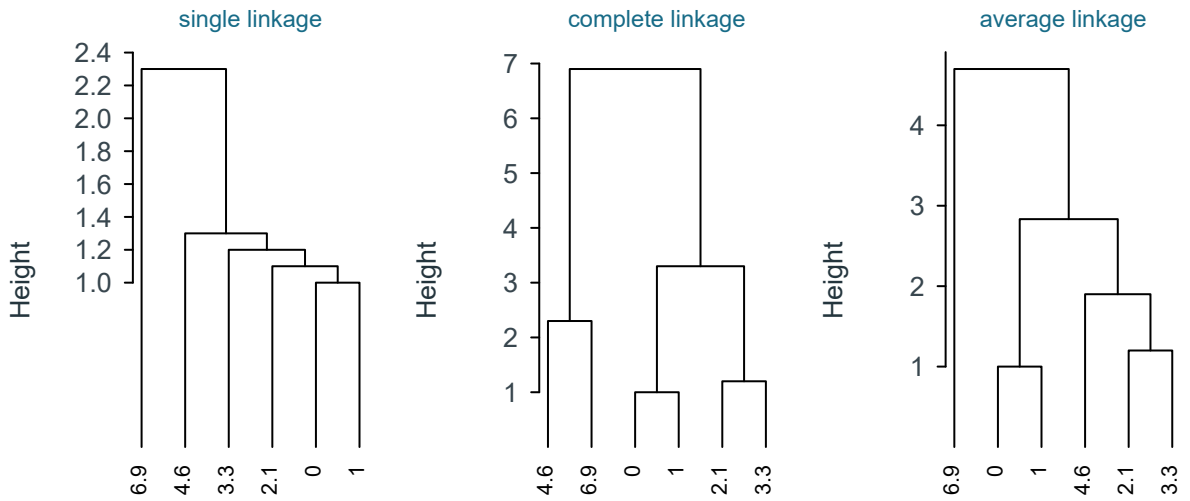


Figure 7.1: Single, complete and average linkage for the six one-dimensional points in the chaining example. Merge heights reproduce the stated distances.

7.1.3 Divisive clustering

Agglomerative clustering fixes the small merges first, and every later merge inherits those decisions. Divisive clustering starts from the whole sample, so the first split is a decision about the large-scale partition. One standard rule selects the cluster of largest diameter, the maximum distance between two of its members, and moves aside the observation whose average distance to the other members is largest. The move continues until transferring another observation no longer increases the separation of the two sides. Categorical variables enter mainly through the choice of dissimilarity, for example Gower's distance. If one partition is required, the gap statistic of Section 4.5 can cut the dendrogram at a single level. The dendrogram itself remains the object the method was built to show: every nested partition at once.

A dendrogram is also a record of irreversible decisions: later merges cannot undo earlier ones. A horizontal cut chooses one partition from that record. Compare nearby cuts and inspect the observations around the large jumps before attaching a scientific interpretation to one level.

The fragment standardises columns, computes distances between rows and cuts an average-linkage tree into three groups. Keeping the distances fixed while changing the linkage isolates the effect

of the merge rule.

```
d  <- dist(scale(x))                          # Euclidean distances between
↪  standardised rows
hc <- hclust(d, method = "average")           # or "single", "complete"
plot(hc)                                      # the dendrogram
cutree(hc, k = 3)                             # cut the tree into 3 clusters
```

7.2 Kernel density estimation

Place probability mass $1/n$ at each observation. That empirical distribution remembers the sample exactly, but has no ordinary density with respect to volume. A kernel estimate replaces each point mass by a small density bump. Its width controls how far the influence of an observation extends. With Gaussian bumps this is literally a mixture with one equally weighted component per observation; we choose the bandwidth rather than a component count.

For a sample $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$ in p dimensions the *kernel density estimate* is

$$\widehat{f}(\boldsymbol{x}; \boldsymbol{H}) = \frac{1}{n} \sum_{i=1}^{n} K_{\boldsymbol{H}}(\boldsymbol{x} - \boldsymbol{x}_i). \tag{7.2}$$

Here we take K to be a symmetric probability density and $\boldsymbol{H}$ a symmetric positive definite bandwidth matrix. These assumptions give a nonnegative estimate that integrates to one. More general kernel constructions exist; the formulas below use this conventional density-kernel setting. Define

$$K_{\boldsymbol{H}}(\boldsymbol{x}) = |\boldsymbol{H}|^{-1/2} K(\boldsymbol{H}^{-1/2}\boldsymbol{x}), \tag{7.3}$$

with $\boldsymbol{H}^{-1/2}$ obtained from the spectral decomposition. The factor $|\boldsymbol{H}|^{-1/2}$ is the Jacobian that keeps $K_{\boldsymbol{H}}$ a density. One kernel is the standard multivariate normal density,

$$K(\boldsymbol{x}) = \frac{1}{(2\pi)^{p/2}} \mathrm{e}^{-\frac{1}{2}\boldsymbol{x}^\top \boldsymbol{x}}, \qquad K_{\boldsymbol{H}}(\boldsymbol{x}) = \frac{1}{(2\pi)^{p/2}|\boldsymbol{H}|^{1/2}} \mathrm{e}^{-\frac{1}{2}\boldsymbol{x}^\top \boldsymbol{H}^{-1}\boldsymbol{x}}, \tag{7.4}$$

so $K_{\boldsymbol{H}}$ is the $\mathrm{N}_p(\boldsymbol{0}, \boldsymbol{H})$ density. Each sample point is the centre of one such normal. The mixture reading is then exact: n components, weights $1/n$, means $\boldsymbol{x}_i$, and one common covariance $\boldsymbol{H}$. The components, the weights, and the means are fixed by the sample. The choice that remains is $\boldsymbol{H}$. The factor $|\boldsymbol{H}|^{-1/2}$ in (7.3) is the Jacobian of $\boldsymbol{u} = \boldsymbol{H}^{-1/2}\boldsymbol{x}$. It is there so that $K_{\boldsymbol{H}}$ integrates to one whenever K does. Exercise 7.4 is that change of variables.

The Epanechnikov kernel has bounded support, so a sample point contributes nothing beyond a finite radius. The normal kernel is the usual multivariate default, and its support is the whole space. A full matrix $\boldsymbol{H}$ can follow an elliptical cloud. A diagonal $\boldsymbol{H}$, or a product of separate univariate smoothers, has axis-aligned contours. On a tilted cloud those contours place part of each kernel's mass off the data. One $\boldsymbol{H}$ everywhere is a further restriction: a cloud whose shape changes from one region to another is smoothed with the same orientation in every region.

7.2.1 Choosing the bandwidth

The matrix $\boldsymbol{H}$ may be constrained to be diagonal, or left unconstrained. Either way the target is approximation of the density f that generated the sample. There is no unique choice, and the multivariate problem is not a coordinate-wise copy of the univariate one. Likelihood cross-validation, as in Chapter 6, fits the estimate on one part of the sample and scores the log-likelihood

on the left-out part. The bandwidth with the highest held-out score is preferred. With a short sample the split is repeated over folds. A standard distance between f and $\widehat{f}$, different from Kullback–Leibler divergence, is the *mean integrated squared error*,

$$\mathrm{MISE}(\boldsymbol{H}) = \mathbb{E}\int_{\mathbb{R}^p} \left[\widehat{f}(\boldsymbol{x}; \boldsymbol{H}) - f(\boldsymbol{x})\right]^2 \mathrm{d}\boldsymbol{x}, \tag{7.5}$$

regarded as a function of $\boldsymbol{H}$. The optimal bandwidth minimises MISE over symmetric positive definite $p \times p$ matrices. The expectation is over the sample that builds $\widehat{f}$. MISE can be estimated in several ways, including cross-validation. The derivations are long; Wand and Jones (1995), Chacón and Duong (2011) and Chacón and Duong (2018) are the references, and the choice of bandwidth is still an active subject. Most methods optimise $\boldsymbol{H}$ numerically. A full matrix has $p(p+1)/2$ free entries, which is already a difficult optimisation at moderate p. Large n adds a computational cost, although approximations exist. Likelihood cross-validation, or a single validation set, is one practical selector. The `ks` package (Duong, 2007) implements multivariate kernel density estimation and classification up to six dimensions. In higher dimension one usually reduces the coordinates first, and that reduction discards information.

Reducing dimension before density estimation can make the smoothing problem manageable, but changes its target to the projected distribution. Discarded directions cannot be reconstructed from the density of the retained scores alone. There is no universal dimension, such as six, at which estimation becomes impossible: difficulty depends on sample size, smoothness and effective dimension. The software's supported dimension is a separate constraint.

Without a smoothness restriction, many irregular densities are compatible with the same finite sample, and the sample does not choose among them. Smoothness is the assumption that lets the estimate take values between the observed points. Remark 7.3 specialises the tension to one bandwidth h. A very small h tracks each observation. A very large h erases modes. The likelihood computed on the training sample prefers $h \to 0$, for the same reason an unconstrained mixture covariance can collapse in Section 5.6. The bandwidth is chosen by a held-out score or by an estimate of MISE, not by the training likelihood.

Remark 7.3 (Bias–variance trade-off in h). In one dimension, $\boldsymbol{H} = h^2$. A very small bandwidth gives a spiky estimate with one bump per observation (low bias, high variance); a very large one gives an over-smoothed estimate that hides structure such as two modes (high bias, low variance). MISE balances the two. Figure 7.2 shows the three regimes. Note that the likelihood *on the training data* always favours $h \to 0$ (each component then sits on its observation and the density at that point diverges, as with the singularities of Section 5.6), which is why the bandwidth must be chosen by cross-validated or held-out likelihood.

Key idea.
For a Gaussian kernel, the estimate is a mixture of bumps centred at the data. Small bandwidths retain fine detail and sampling noise; large bandwidths erase both. Choose the scale using a criterion matched to the goal, then inspect how nearby choices change the picture. A visible mode is a feature of this smoothed estimate, not automatically evidence of a separate population.

7.3 R laboratory: linkage and bandwidth are part of the analysis

The same scaled observations are clustered by four linkages, so the only change from one result to the next is the merge rule. Each tree is then cut into three groups and compared with the species labels by the adjusted Rand index. Those labels are not an input to the linkage. Two univariate bandwidth selectors are also applied to petal length. They are different approximations

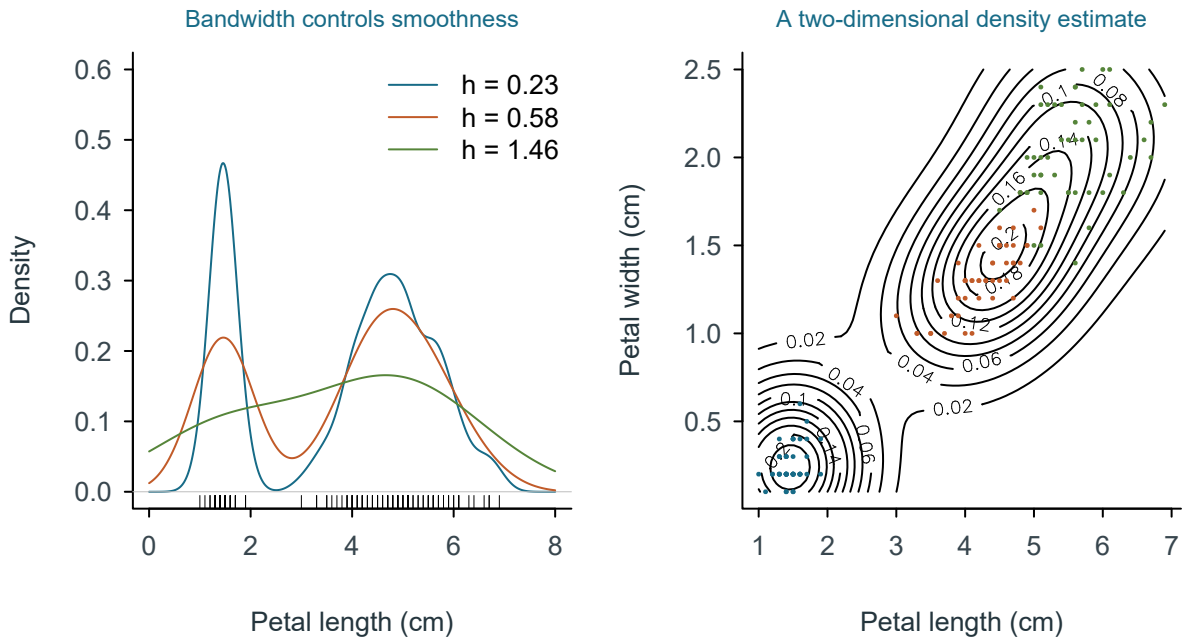


Figure 7.2: Iris kernel density estimates. Smaller bandwidth reveals local variation; larger bandwidth smooths it. The right panel uses both petal measurements.

to a bandwidth that balances bias and variance, as in Remark 7.3.

```
z <- scale(iris_x)
ari <- sapply(c("single", "complete", "average", "ward.D2"),
  function(method) {
    groups <- cutree(hclust(dist(z), method = method), k = 3)
    mclust::adjustedRandIndex(groups, iris_y)
  })
round(ari, 3)
#>   single complete  average  ward.D2
#>    0.558    0.573    0.562    0.615
c(rule_of_thumb = bw.nrd0(iris$Petal.Length),
  Sheather_Jones = bw.SJ(iris$Petal.Length))
#>  rule_of_thumb Sheather_Jones
#>         0.5832         0.1920
```

The adjusted Rand indices tell us how each three-group partition agrees with the iris species labels on this sample. Choosing a linkage because this agreement is high would make the labels part of model selection. In the same way, a bandwidth comparison needs its units: a width of 0.3 means something only relative to the variable being smoothed.

7.4 Further exercises

Exercise 7.1 (Linkage by hand)**.** Cluster the points $1, 2, 6, 8, 15$ on a line by single, complete, and average linkage. List the merge heights for each. Do the three dendrograms have the same topology?

Solution

Exercise 7.2 (Number of merges)**.** How many merges does agglomerative clustering of n observations require? How many distinct clusterings (levels) does the hierarchy contain, including the

singletons and the single cluster?

Solution

Exercise 7.3 (Kernel density by hand)**.** With observations $-1, 0, 2$ and a Gaussian kernel of bandwidth $h = 1$, evaluate $\widehat{f}(0)$ using (7.2) and (7.4) (here $\boldsymbol{H} = h^2 = 1$).

Solution

Exercise 7.4 (A kernel is a density)**.** Show that $\widehat{f}(\boldsymbol{x}; \boldsymbol{H})$ in (7.2) integrates to 1 whenever K does, using the change of variables $\boldsymbol{u} = \boldsymbol{H}^{-1/2}\boldsymbol{x}$ in each term.

Solution

Exercise 7.5 (A tempting wrong turn)**.** A student selects h by maximising $\sum_i \log \widehat{f}(x_i; h)$ over h, where $\widehat{f}$ is computed from all the data including x_i. What happens, and how would you modify the criterion?

Solution

Exercise 7.6 (Clustering and smoothing choices)**.** A two-dimensional dataset contains two dense groups joined by a thin chain of observations. You consider agglomerative clustering and a kernel density estimate.

(a) Explain why single linkage can merge the two groups early. Contrast it with complete linkage.

(b) Explain the role of bandwidth in kernel density estimation and describe the consequences of a very small and a very large bandwidth.

(c) Would a peak in an estimated density necessarily identify a distinct generating population? Explain.

Solution

Part III

Classification

Chapter 8

Statistical classifiers and discriminant analysis

Suppose a new iris flower arrives with four measurements and no species label. We have labelled flowers to learn from, so the task is to predict an existing class. Even with good measurements, species can overlap. Classification therefore needs two ingredients: information about which labels are plausible and a rule for deciding what to do with that information.

The route in this chapter is to model the distribution of the predictors inside each class and then apply Bayes' theorem, the same density form as (1.4). If f_g is the density of class g and π_g is the probability of that class before the measurements are seen, the probability of class g given the measurements is proportional to $\pi_g f_g(\boldsymbol{x})$. Linear discriminant analysis takes each class to be multivariate normal and forces those normals to share one covariance. Quadratic discriminant analysis estimates a separate covariance in each class. The extra matrices can follow shapes that one shared matrix cannot. They are also estimated from smaller samples, so a more flexible covariance buys a reduction in structural bias and pays for it in estimation noise, especially when a class is small.

The same mathematics can be used for documents, manufactured parts or diagnostic measurements, but the meaning of a mistake changes with the application. Choosing a class label and assessing its consequences are separate steps. We will first treat all wrong labels as equally costly, then relax that choice.

The outcome must also be defined before modelling. A current condition and an event over a specified future period are different labels. A dataset whose follow-up varies between observations cannot simply be relabelled as a common fixed-horizon outcome. The probability model answers the question encoded in its response variable.

The Bayes rule below is the classifier we would use if the class densities and the class probabilities were known. The rest of the chapter estimates those densities: linear and quadratic discriminant analysis, Fisher's discriminant, and the mixture and kernel-density versions. Estimating the error of the resulting rule is Chapter 9. Classifiers that model the posterior $\tau_g(\boldsymbol{x})$ directly, or that avoid a probability model for the predictors, follow in Chapters 10–13.

Source. Lecture notes, Sections 3.1–3.8 (pp. 63–75).

8.1 The classification problem

The object we want is a rule that reads a new measurement vector and returns a class. A density estimator from the earlier chapters does not return a class, and a clustering method invents groups rather than using the labels we already have. The rule has to be a function of the new vector and

of the labelled training set.

Definition 8.1 (Training data and classifier)**.** The training data consist of multivariate observations $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ of p predictor variables, stored as the rows of a data matrix $\boldsymbol{X}_{\text{all}}$, together with class labels $\boldsymbol{y}_{\text{all}} = (y_1, \ldots, y_n)^\top$, $y_i \in \{1, \ldots, G\}$, where G is the number of classes. A *classifier* is a function $\eta_{(\boldsymbol{X}_{\text{all}}, \boldsymbol{y}_{\text{all}})}(\boldsymbol{x}) \in \{1, \ldots, G\}$ of a new observation $\boldsymbol{X} = \boldsymbol{x}$ and the training set.

Assume initially that the labelled pairs are independent draws from a target population. Its class probabilities are $\pi_1, \ldots, \pi_G$, summing to one. These are called prior class probabilities because they describe a flower before its measurements are observed; they are not priors on unknown parameters. Representative training proportions estimate them as follows. External prevalence estimates are useful only when they refer to the population in which the rule will be used.

$$\widehat{\pi}_g = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}_g(y_i), \tag{8.1}$$

where $\mathbb{I}_g(y) = 1$ if $y = g$ and 0 otherwise. Those proportions estimate the population priors only when each case was drawn from F without fixing the class sizes in advance. A case–control sample that enrols equal numbers of cases and controls has class counts chosen by the design. Its proportions describe the sample, not the population, and the priors have to come from outside the training counts.

8.2 Posterior probabilities and the Bayes rule

The class density $f_g(\boldsymbol{x})$ answers a different question from the one we are asking. It says how the measurements are distributed once the class is known. We observe the measurements and want the class. A rare class can have a high density at $\boldsymbol{x}$ and still be unlikely. Bayes' theorem is the conversion, and it is the same identity as (1.4), with the class in the role of the discrete variable and the measurement vector in the role of the continuous one. If the joint distribution $F_{\boldsymbol{X},Y}$ were known, the conversion would be exact:

$$\mathbb{P}(Y = g \mid \boldsymbol{X} = \boldsymbol{x}) = \frac{h(\boldsymbol{X} = \boldsymbol{x} \mid Y = g)\mathbb{P}(Y = g)}{f(\boldsymbol{X} = \boldsymbol{x})}, \tag{8.2}$$

where $\mathbb{P}(Y = g) = \pi_g$, $h(\boldsymbol{X} = \boldsymbol{x} \mid Y = g)$ is the density of the gth class (abbreviated $f_g(\boldsymbol{x})$), and $f(\boldsymbol{X} = \boldsymbol{x}) =: f_{\boldsymbol{X}}(\boldsymbol{x})$ is the marginal density of the measurements. The marginal is not a new object. By the law of total probability it is the mixture of the class densities, with weights $\pi_1, \ldots, \pi_G$. That is the mixture of Chapter 5, with one difference that matters for estimation: here the component label is observed on every training case, so we do not have to infer which component produced which point.

$$f_{\boldsymbol{X}}(\boldsymbol{x}) = \sum_{g=1}^{G} \pi_g f_g(\boldsymbol{x}). \tag{8.3}$$

The probability $\mathbb{P}(Y = g \mid \boldsymbol{X} = \boldsymbol{x})$ is the *posterior probability* of class g given $\boldsymbol{x}$. It is abbreviated $\tau_g(\boldsymbol{x})$, in the notation of (McLachlan, 1992). Substituting the mixture for the marginal density gives

$$\tau_g(\boldsymbol{x}) = \frac{\pi_g f_g(\boldsymbol{x})}{\sum_{g'=1}^{G} \pi_{g'} f_{g'}(\boldsymbol{x})}. \tag{8.4}$$

We want the class that this posterior says is most probable. The rule that chooses it is

$$B(\boldsymbol{x}) = \underset{g\in 1,\ldots,G}{\arg\max}\, \tau_g(\boldsymbol{x}) = \underset{g\in 1,\ldots,G}{\arg\max}\, \pi_g f_g(\boldsymbol{x}), \tag{8.5}$$

The second equality holds because the marginal density $f_{\boldsymbol{X}}(\boldsymbol{x})$ is the same for every class, so it cannot change which class wins. When the densities and the priors are the true ones, this rule has the lowest expected zero–one risk (Theorem 8.3). We do not have those densities. The *plug-in* rule keeps the same comparison and replaces each unknown quantity by an estimate from the training sample:

$$\eta_{(\boldsymbol{X}_{\text{all}},\boldsymbol{y}_{\text{all}})}(\boldsymbol{x}) = \underset{g\in 1,\ldots,G}{\arg\max}\, \widehat{\tau}_g(\boldsymbol{x}) = \underset{g\in 1,\ldots,G}{\arg\max}\, \widehat{\pi}_g \widehat{f}_g(\boldsymbol{x}). \tag{8.6}$$

8.3 Error rates and optimal classifiers

Imagine two normal class distributions with overlapping support, both having positive prior probability. At a measurement where their weighted densities agree, the posterior probabilities are both 1/2. Every rule using only that measurement has conditional error 1/2 there. More training observations can improve our estimates of the distributions, but cannot remove their overlap. Additional informative measurements can change the problem. Bayes error is the floor for the information currently available.

A rule $\eta(\cdot)$ always returns a class, and that class may be the wrong one. For G classes there are $G^2 - G$ ordered pairs of distinct classes, and each pair is a different kind of mistake. Let $R_h \subset \mathbb{R}^p$ be the region that the rule assigns to class h. The regions are disjoint and cover the space. The rate at which the rule predicts class h when the true class is g is

$$e_{gh}(\eta) = \mathbb{P}(\eta(\boldsymbol{X}) = h \mid Y = g) = \int_{R_h} f_g(\boldsymbol{x})\, \mathrm{d}\nu, \qquad g, h \in 1, \ldots, G, \tag{8.7}$$

where ν is the appropriate measure on $\mathbb{R}^p$. Adding the mistakes that leave class g gives the class-specific error rate,

$$\mathrm{Err}_g(\eta) = \mathbb{P}(\eta(\boldsymbol{X}) \neq g \mid Y = g) = \int_{\cup_{h\neq g} R_h} f_g(\boldsymbol{x})\, \mathrm{d}\nu = 1 - \int_{R_g} f_g(\boldsymbol{x})\, \mathrm{d}\nu. \tag{8.8}$$

To average class-specific errors, weight by the target prevalences: $\mathrm{Err}(\eta) = \sum_g \pi_g \mathrm{Err}_g(\eta)$. For a fixed rule and fixed class-conditional distributions, changing only the prevalences leaves the class-specific rates unchanged. If we also change the rule to reflect the new prevalences, its decision regions move and those rates can change. Keep the rule fixed when making the first comparison.

To decide which rule is best we need a loss. Write c_{gh} for the cost of predicting class h when the true class is g, with $c_{gg} = 0$, so a correct label is costless. The first choice is the simplest one that treats every mistake as equally unwanted: $c_{gh} = 1$ whenever $g \neq h$. Under that choice only the overall error rate matters. Multiplying every cost by the same positive constant multiplies every risk by that constant and does not change which class is chosen, so the unit cost is a normalisation. It is the wrong loss when a missed weapon is several times as costly as an empty bag searched; that case is taken up once the equal-cost rule is in hand. The true class is coded as a binary vector $\boldsymbol{y}$ of length G, with a 1 in the position of the true class, so the second of three classes is $\boldsymbol{y} = (0, 1, 0)^\top$. Write y_g for its gth element.

Definition 8.2 (Zero–one loss and risk)**.** Let $(\boldsymbol{X}, Y)$ be a new point drawn from F. The *zero–one loss* of the classifier is

$$Q[Y, \eta(\boldsymbol{X})] = \begin{cases} 0, & \eta(\boldsymbol{X}) = Y, \\ 1, & \eta(\boldsymbol{X}) \neq Y, \end{cases} \qquad l(Y, \eta(\boldsymbol{X})) = \sum_{g=1}^{G} Y_g \, Q[g, \eta(\boldsymbol{X})]. \tag{8.9}$$

The *(conditional) risk* at $\boldsymbol{X} = \boldsymbol{x}$ is the expected loss over Y drawn from its conditional distribution $F_{Y|\boldsymbol{X}=\boldsymbol{x}}$, which is $\text{Mult}(1, \tau_1(\boldsymbol{x}), \ldots, \tau_G(\boldsymbol{x}))$.

The conditional risk is the expected loss when $\boldsymbol{X}$ is held at $\boldsymbol{x}$ and Y is drawn from its conditional distribution. The strategy is short. Under zero–one loss the sum over classes charges 0 to the predicted class and 1 to every other class, and $\mathbb{E}(Y_g \mid \boldsymbol{x}) = \tau_g(\boldsymbol{x})$ because Y_g is the indicator of class g. The expectation therefore collapses to one minus the posterior of the class we chose:

$$\begin{aligned} \mathbb{E}_{F_{Y|\boldsymbol{X}=\boldsymbol{x}}} \, l(Y, \eta(\boldsymbol{x})) &= \sum_{g=1}^{G} \mathbb{E}(Y_g \mid \boldsymbol{x}) \, Q[g, \eta(\boldsymbol{x})] && (8.10) \\ &= \sum_{g=1}^{G} \tau_g(\boldsymbol{x}) \, Q[g, \eta(\boldsymbol{x})] && (8.11) \\ &= \sum_{g \neq \eta(\boldsymbol{x})} \tau_g(\boldsymbol{x}) = 1 - \tau_{\eta(\boldsymbol{x})}(\boldsymbol{x}). && (8.12) \end{aligned}$$

What this calculation establishes is an identity, not yet an optimality claim. The conditional risk equals the posterior probability that the predicted class is wrong. Zero–one loss did the work: every wrong class contributes its posterior, and the correct class contributes nothing. No normality was used. In most cases every class density is positive at $\boldsymbol{x}$, so every posterior is positive and every rule has some conditional risk there. The rule itself is deterministic, so at each $\boldsymbol{x}$ it names one class.

Theorem 8.3 (Bayes-optimal rule)**.** A rule that minimises the conditional risk (8.12) for every $\boldsymbol{x}$ is called a *Bayes (decision) rule*, Bayes-optimal rule, or minimum-risk rule. Under zero–one loss it is

$$\eta(\boldsymbol{x}) = B(\boldsymbol{x}) = \arg\max_{g \in 1, \ldots, G} \tau_g(\boldsymbol{x}). \tag{8.13}$$

Proof. At a fixed $\boldsymbol{x}$, predicting h incurs conditional error $1 - \tau_h(\boldsymbol{x})$. Choose a largest posterior to make this as small as possible. Integrating that pointwise inequality over $\boldsymbol{X}$ proves optimality for overall error as well. Ties can be resolved by any fixed convention; the risk is the same. □

This result concerns the true joint distribution and the specified loss. A fitted classifier uses estimated probabilities and need not attain the bound. Its training error can even fall below the Bayes risk, because training error reuses the observations that built the rule. There is no contradiction: the bound concerns a fresh observation.

Unequal costs.

Zero–one loss treats a missed weapon and a searched empty bag as the same kind of event. When they are not, the loss has to say so. A correct classification is still taken to cost nothing. What moves the rule is the ratio of the costs of the two mistakes. Scaling every cost by the same positive constant scales the risk and leaves the minimiser unchanged, so a false negative several times as costly as a false positive is fully described by that ratio.

For general costs c_{gh} and a new observation $(\boldsymbol{x}, y)$ the loss is $l(y, \eta(\boldsymbol{x})) = \sum_g y_g \, c_{g\eta(\boldsymbol{x})} Q[g, \eta(\boldsymbol{x})]$. The same expectation as before, now with cost c_{gh} in place of 1, gives the conditional risk

$$\mathbb{E}_{F_{Y|\boldsymbol{X}=\boldsymbol{x}}} l(Y, \eta(\boldsymbol{x})) = \sum_{g \neq \eta(\boldsymbol{x})} \tau_g(\boldsymbol{x}) \, c_{g\eta(\boldsymbol{x})}. \tag{8.14}$$

The strategy is the same as for zero–one loss: at each $\boldsymbol{x}$, choose the predicted class that makes this sum smallest. The minimiser is

$$\eta(\boldsymbol{x}) = B(\boldsymbol{x}) = \underset{h \in 1,\ldots,G}{\arg\min} \sum_{g \neq h} \tau_g(\boldsymbol{x}) \, c_{gh}. \tag{8.15}$$

For two classes the comparison simplifies. The rule predicts class 1 when $\tau_2(\boldsymbol{x}) \, c_{21} < \tau_1(\boldsymbol{x}) \, c_{12}$, which is $\tau_1(\boldsymbol{x})/\tau_2(\boldsymbol{x}) > c_{21}/c_{12}$. The threshold on the posterior odds moves toward the class whose misclassification costs more. Equal costs put the threshold at 1 and recover (8.13). The posteriors themselves still come from Bayes' theorem; only the cutoff has changed.

Key idea.
Statistical classifiers essentially estimate π_g and $f_g(\boldsymbol{x})$ for $g = 1, \ldots, G$, use them to estimate the posterior probabilities $\tau_g(\boldsymbol{x})$, and plug these into the Bayes rule (8.13) (equal costs) or (8.15) (unequal costs). The Bayes risk is a floor: no classifier, however flexible, can beat it.

8.4 Linear discriminant analysis

LDA supplies the missing class densities by assuming $\boldsymbol{X} \mid Y = g \sim \mathcal{N}_p(\boldsymbol{\mu}_g, \Sigma)$, with one covariance shared across classes. This is a distribution conditional on a class label, not the Gaussian conditioning calculation for splitting coordinates in Chapter 1. Different means move the class centres; the common covariance gives them the same shape and orientation. The reward for that restriction is a linear log-odds comparison (Hastie et al., 2009, Section 4.3).

Estimation.

The labels are known, so the training sample splits into classes and each piece can be estimated on its own. The priors are (8.1) unless the design fixed the class sizes or a better external estimate exists. The mean of each class is the ordinary sample mean of its own observations. The common covariance centres every observation by its own class mean, pools the cross-products, and divides by $n - G$. That divisor is the unbiased one: one degree of freedom is spent on each of the G estimated means. The matrix is the pooled estimator (2.10) with G classes,

$$\widehat{\boldsymbol{\Sigma}} = \boldsymbol{S} = \frac{1}{n - G} \sum_{g=1}^{G} \sum_{i:\, y_i = g} (\boldsymbol{x}_i - \widehat{\boldsymbol{\mu}}_g)(\boldsymbol{x}_i - \widehat{\boldsymbol{\mu}}_g)^\top. \tag{8.16}$$

8.4.1 The discriminant boundary

Assume equal costs, so the plug-in rule ties two classes where their weighted densities are equal. That tie is the decision boundary. The strategy is to substitute the two normal densities, cancel every factor that does not depend on the class, take logarithms, and expand the quadratic forms. The shared inverse covariance is the assumption that will make the terms in $\boldsymbol{x}^\top \widehat{\boldsymbol{\Sigma}}^{-1} \boldsymbol{x}$ cancel.

$$\frac{\widehat{\pi}_j}{(2\pi)^{p/2}|\widehat{\boldsymbol{\Sigma}}|^{1/2}}\,\mathrm{e}^{-\frac{1}{2}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_j)^\top\widehat{\boldsymbol{\Sigma}}^{-1}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_j)} = \frac{\widehat{\pi}_l}{(2\pi)^{p/2}|\widehat{\boldsymbol{\Sigma}}|^{1/2}}\,\mathrm{e}^{-\frac{1}{2}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_l)^\top\widehat{\boldsymbol{\Sigma}}^{-1}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_l)}. \tag{8.17}$$

The factor $(2\pi)^{p/2}|\widehat{\boldsymbol{\Sigma}}|^{1/2}$ is the same on both sides because the covariance was forced to be common, so it cancels. The logarithms turn the remaining ratio of exponentials into a difference of quadratic forms:

$$\log\frac{\widehat{\pi}_j}{\widehat{\pi}_l} = \frac{1}{2}\Big[(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_j)^\top\widehat{\boldsymbol{\Sigma}}^{-1}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_j) - (\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_l)^\top\widehat{\boldsymbol{\Sigma}}^{-1}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_l)\Big]. \tag{8.18}$$

Expand each quadratic form. Symmetry of $\widehat{\boldsymbol{\Sigma}}^{-1}$ gives $\boldsymbol{a}^\top\widehat{\boldsymbol{\Sigma}}^{-1}\boldsymbol{b} = \boldsymbol{b}^\top\widehat{\boldsymbol{\Sigma}}^{-1}\boldsymbol{a}$, so the cross terms can be written as $\boldsymbol{x}^\top\widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_g$. Grouping then produces

$$\begin{aligned}\log\frac{\widehat{\pi}_j}{\widehat{\pi}_l} &= \frac{1}{2}\Big[\boldsymbol{x}^\top\widehat{\boldsymbol{\Sigma}}^{-1}\boldsymbol{x} - 2\boldsymbol{x}^\top\widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_j + \widehat{\boldsymbol{\mu}}_j^\top\widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_j - \boldsymbol{x}^\top\widehat{\boldsymbol{\Sigma}}^{-1}\boldsymbol{x} + 2\boldsymbol{x}^\top\widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_l - \widehat{\boldsymbol{\mu}}_l^\top\widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_l\Big] \\ &= \boldsymbol{x}^\top\widehat{\boldsymbol{\Sigma}}^{-1}(\widehat{\boldsymbol{\mu}}_l - \widehat{\boldsymbol{\mu}}_j) + \frac{1}{2}\Big[\widehat{\boldsymbol{\mu}}_j^\top\widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_j - \widehat{\boldsymbol{\mu}}_l^\top\widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_l\Big].\end{aligned} \tag{8.19}$$

The two copies of $\boldsymbol{x}^\top\widehat{\boldsymbol{\Sigma}}^{-1}\boldsymbol{x}$ are identical, so they cancel, and the equation that remains is linear in $\boldsymbol{x}$. The shared covariance did that work. Had the two classes used different covariances, the quadratic terms would have carried different matrices and would have survived; that is the calculation in the next section. Rearrangement puts the linear equation in a form that is easy to name:

$$\boldsymbol{x}^\top\widehat{\boldsymbol{\Sigma}}^{-1}(\widehat{\boldsymbol{\mu}}_l - \widehat{\boldsymbol{\mu}}_j) = \log\frac{\widehat{\pi}_j}{\widehat{\pi}_l} + \frac{1}{2}\Big[\widehat{\boldsymbol{\mu}}_l^\top\widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_l - \widehat{\boldsymbol{\mu}}_j^\top\widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_j\Big]. \tag{8.20}$$

Define

$$\boldsymbol{\beta} = \widehat{\boldsymbol{\Sigma}}^{-1}(\widehat{\boldsymbol{\mu}}_j - \widehat{\boldsymbol{\mu}}_l), \qquad \alpha = \log\frac{\widehat{\pi}_j}{\widehat{\pi}_l} + \frac{1}{2}(\widehat{\boldsymbol{\mu}}_l + \widehat{\boldsymbol{\mu}}_j)^\top\widehat{\boldsymbol{\Sigma}}^{-1}(\widehat{\boldsymbol{\mu}}_l - \widehat{\boldsymbol{\mu}}_j), \tag{8.21}$$

The second form of α rewrites the difference of quadratic mean terms by the identity $\widehat{\boldsymbol{\mu}}_l^\top\widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_l - \widehat{\boldsymbol{\mu}}_j^\top\widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_j = (\widehat{\boldsymbol{\mu}}_l + \widehat{\boldsymbol{\mu}}_j)^\top\widehat{\boldsymbol{\Sigma}}^{-1}(\widehat{\boldsymbol{\mu}}_l - \widehat{\boldsymbol{\mu}}_j)$. The left side of (8.20) is then $-\boldsymbol{x}^\top\boldsymbol{\beta}$ and the right side is α, so the boundary is the hyperplane

$$\alpha + \boldsymbol{x}^\top\boldsymbol{\beta} = 0, \tag{8.22}$$

The same algebra shows that the linear score is the plug-in log posterior odds, $\alpha + \boldsymbol{x}^\top\boldsymbol{\beta} = \log\big[\widehat{\pi}_j\widehat{f}_j(\boldsymbol{x})/\widehat{\pi}_l\widehat{f}_l(\boldsymbol{x})\big]$. Predicting the class with the larger weighted density is therefore the sign rule

$$\eta(\boldsymbol{x}) = \begin{cases} j, & \alpha + \boldsymbol{x}^\top\boldsymbol{\beta} \geq 0, \\ l, & \alpha + \boldsymbol{x}^\top\boldsymbol{\beta} < 0. \end{cases} \tag{8.23}$$

One check of the sign: $\log\frac{\widehat{\pi}_j\widehat{f}_j}{\widehat{\pi}_l\widehat{f}_l} = \log\frac{\widehat{\pi}_j}{\widehat{\pi}_l} - \frac{1}{2}(q_j - q_l)$ with $q_g = (\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_g)^\top\widehat{\boldsymbol{\Sigma}}^{-1}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_g)$, and (8.19) says that this difference equals $\boldsymbol{x}^\top\boldsymbol{\beta} + \alpha$.

What the cancellation establishes is the shape of the boundary, not a claim that the classes really share a covariance. Under that assumption the log posterior odds are linear in $\boldsymbol{x}$, so the

comparison of two classes is a single number, the projection $\alpha + \boldsymbol{x}^\top \boldsymbol{\beta}$, and the boundary in that one-dimensional projection is a point. With more than two classes there is one such score for each pair, and the predicted class is the one that wins the comparisons it enters. The regions of the partition are bounded by pieces of these hyperplanes, so the boundary is *piecewise linear*. Two equivalent forms are recorded in the remark: a score δ_g for each class, which avoids writing every pair when $G > 2$, and, when the priors are equal, assignment to the class mean nearest in squared Mahalanobis distance (Appendix A.3). Unequal priors shift each boundary parallel to itself through the term $\log(\widehat{\pi}_j/\widehat{\pi}_l)$ in α.

Remark 8.4 (Equivalent forms). Two equivalent ways of writing the same rule are useful. First, define the *linear discriminant score* $\delta_g(\boldsymbol{x}) = \boldsymbol{x}^\top \widehat{\boldsymbol{\Sigma}}^{-1} \widehat{\boldsymbol{\mu}}_g - \frac{1}{2} \widehat{\boldsymbol{\mu}}_g^\top \widehat{\boldsymbol{\Sigma}}^{-1} \widehat{\boldsymbol{\mu}}_g + \log \widehat{\pi}_g$; then the rule is $\arg\max_g \delta_g(\boldsymbol{x})$, which avoids pairwise comparisons for $G > 2$. Second, with equal priors the rule assigns $\boldsymbol{x}$ to the class whose mean is nearest in squared Mahalanobis distance $(\boldsymbol{x} - \widehat{\boldsymbol{\mu}}_g)^\top \widehat{\boldsymbol{\Sigma}}^{-1} (\boldsymbol{x} - \widehat{\boldsymbol{\mu}}_g)$ (Appendix A.3). Unequal priors shift each boundary parallel to itself by $\log(\widehat{\pi}_j/\widehat{\pi}_l)$ in α.

Example 8.5 (A two-class LDA by hand)**. Question.** Class 1 has observations $(3,3), (1,1), (3,2), (1,2)$ and class 2 has $(6,5), (4,3), (6,4), (4,4)$. Find the LDA rule with equal priors and classify the point $(5, 1.4)$.

Solution. The class means are $\widehat{\boldsymbol{\mu}}_1 = (2,2)^\top$ and $\widehat{\boldsymbol{\mu}}_2 = (5,4)^\top$, and $\widehat{\pi}_1 = \widehat{\pi}_2 = \frac{1}{2}$. The centred observations of each class are $(\pm 1, \pm 1)$-type: class 1 deviates from its mean by $(1,1), (-1,-1), (1,0), (-1,0)$, and class 2 has exactly the same deviations. Hence each class has $\boldsymbol{A}_g = \begin{pmatrix} 4 & 2 \\ 2 & 2 \end{pmatrix}$ and the pooled covariance matrix is

$$\widehat{\boldsymbol{\Sigma}} = \frac{1}{8-2}\Big[\begin{pmatrix} 4 & 2 \\ 2 & 2 \end{pmatrix} + \begin{pmatrix} 4 & 2 \\ 2 & 2 \end{pmatrix} \Big] = \begin{pmatrix} 4/3 & 2/3 \\ 2/3 & 2/3 \end{pmatrix}, \qquad \widehat{\boldsymbol{\Sigma}}^{-1} = \begin{pmatrix} 1.5 & -1.5 \\ -1.5 & 3 \end{pmatrix}$$

(using $|\widehat{\boldsymbol{\Sigma}}| = \frac{8}{9} - \frac{4}{9} = \frac{4}{9}$). With $j = 1$ and $l = 2$,

$$\boldsymbol{\beta} = \widehat{\boldsymbol{\Sigma}}^{-1}(\widehat{\boldsymbol{\mu}}_1 - \widehat{\boldsymbol{\mu}}_2) = \widehat{\boldsymbol{\Sigma}}^{-1} \begin{pmatrix} -3 \\ -2 \end{pmatrix} = \begin{pmatrix} -1.5 \\ -1.5 \end{pmatrix},$$
$$\alpha = 0 + \tfrac{1}{2} \begin{pmatrix} 7 \\ 6 \end{pmatrix}^\top \widehat{\boldsymbol{\Sigma}}^{-1} \begin{pmatrix} 3 \\ 2 \end{pmatrix} = \tfrac{1}{2} \begin{pmatrix} 7 \\ 6 \end{pmatrix}^\top \begin{pmatrix} 1.5 \\ 1.5 \end{pmatrix} = 9.75.$$

The boundary is $9.75 - 1.5x_1 - 1.5x_2 = 0$, i.e. $x_1 + x_2 = 6.5$, which passes through the midpoint $(3.5, 3)$ of the means. The rule predicts class 1 when $x_1 + x_2 \le 6.5$. For $(5, 1.4)$: $x_1 + x_2 = 6.4 < 6.5$, so class 1. In contrast, the *Euclidean* bisector of the means is $3x_1 + 2x_2 = 16.5$, and $3(5) + 2(1.4) = 17.8 > 16.5$ puts the point in class 2. The squared Mahalanobis distances agree with LDA: 19.98 to $\widehat{\boldsymbol{\mu}}_1$ and 20.28 to $\widehat{\boldsymbol{\mu}}_2$. The covariance structure tilts the boundary (normal $(1,1)^\top$ rather than $(3,2)^\top$): the within-class scatter is largest along the direction $(1, 0.62)^\top$, which is close to the direction $(3,2)^\top$ joining the means, so a displacement along that direction is less informative than its Euclidean length suggests (Figure 8.1).

8.5 Quadratic discriminant analysis

To allow different class shapes, estimate a separate covariance for each class. QDA makes that change while retaining Gaussian class densities. The pairwise score difference becomes a polynomial of degree at most two. It can describe curved boundaries, though special parameter choices give degenerate or empty boundaries. The next display shows exactly which cancellation LDA loses.

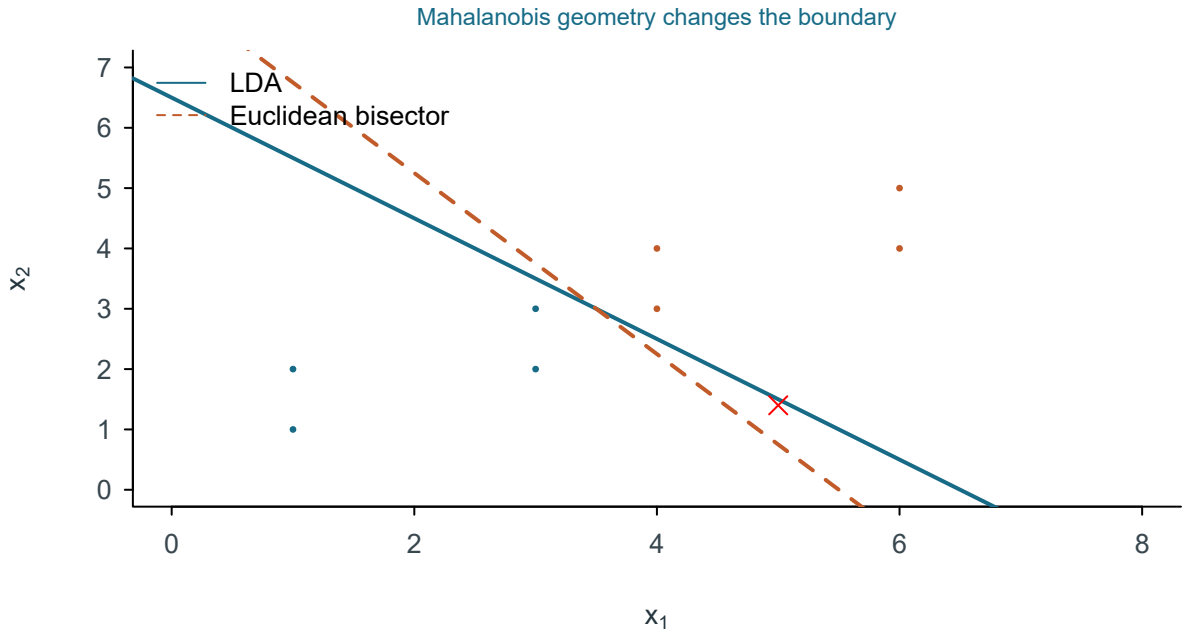


Figure 8.1: The original eight-point LDA example. The covariance-adjusted boundary differs from the Euclidean bisector.

$$\frac{\widehat{\pi}_j}{(2\pi)^{p/2}|\widehat{\boldsymbol{\Sigma}}_j|^{1/2}}\,\mathrm{e}^{-\frac{1}{2}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_j)^\top \widehat{\boldsymbol{\Sigma}}_j^{-1}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_j)} = \frac{\widehat{\pi}_l}{(2\pi)^{p/2}|\widehat{\boldsymbol{\Sigma}}_l|^{1/2}}\,\mathrm{e}^{-\frac{1}{2}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_l)^\top \widehat{\boldsymbol{\Sigma}}_l^{-1}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_l)}. \tag{8.24}$$

The common factor $(2\pi)^{p/2}$ still cancels. The determinants do not, because each class now has its own covariance. Taking logarithms leaves them on the boundary:

$$\log\frac{\widehat{\pi}_j\,|\widehat{\boldsymbol{\Sigma}}_l|^{1/2}}{\widehat{\pi}_l\,|\widehat{\boldsymbol{\Sigma}}_j|^{1/2}} = \frac{1}{2}\Big[(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_j)^\top\widehat{\boldsymbol{\Sigma}}_j^{-1}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_j) - (\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_l)^\top\widehat{\boldsymbol{\Sigma}}_l^{-1}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_l)\Big]. \tag{8.25}$$

The quadratic terms are now $\boldsymbol{x}^\top\widehat{\boldsymbol{\Sigma}}_j^{-1}\boldsymbol{x}$ and $\boldsymbol{x}^\top\widehat{\boldsymbol{\Sigma}}_l^{-1}\boldsymbol{x}$. They cancel only if the two inverses agree. Once the covariances differ, the difference $\boldsymbol{x}^\top(\widehat{\boldsymbol{\Sigma}}_j^{-1}-\widehat{\boldsymbol{\Sigma}}_l^{-1})\boldsymbol{x}$ survives, and the boundary is quadratic in $\boldsymbol{x}$. The class regions therefore have piecewise quadratic edges. The same comparison can be written as a score for each class, $\delta_g(\boldsymbol{x}) = -\frac{1}{2}\log|\widehat{\boldsymbol{\Sigma}}_g| - \frac{1}{2}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_g)^\top\widehat{\boldsymbol{\Sigma}}_g^{-1}(\boldsymbol{x}-\widehat{\boldsymbol{\mu}}_g) + \log\widehat{\pi}_g$, and the rule predicts the class with the largest score. What changed relative to LDA is the covariance assumption. Normality and equal costs are still in force.

QDA can represent class shapes that a single covariance cannot, and it estimates more parameters from the same sample. LDA has $Gp + p(p+1)/2 + (G-1)$ free parameters; QDA has $Gp + Gp(p+1)/2 + (G-1)$. Each class covariance is estimated from that class alone, so its sampling variability is larger, especially when a class is small or p is large. If a common covariance is a reasonable approximation and the sample is limited, LDA can have the smaller error on new observations. The extra flexibility has then been spent on variation in the training sample that does not repeat. That is *over-fitting*. How large a sample is enough depends on p: the parameter counts grow like p^2, and faster for QDA. Bivariate or trivariate marginal plots are the practical check on whether one covariance is plausible for the classes at hand.

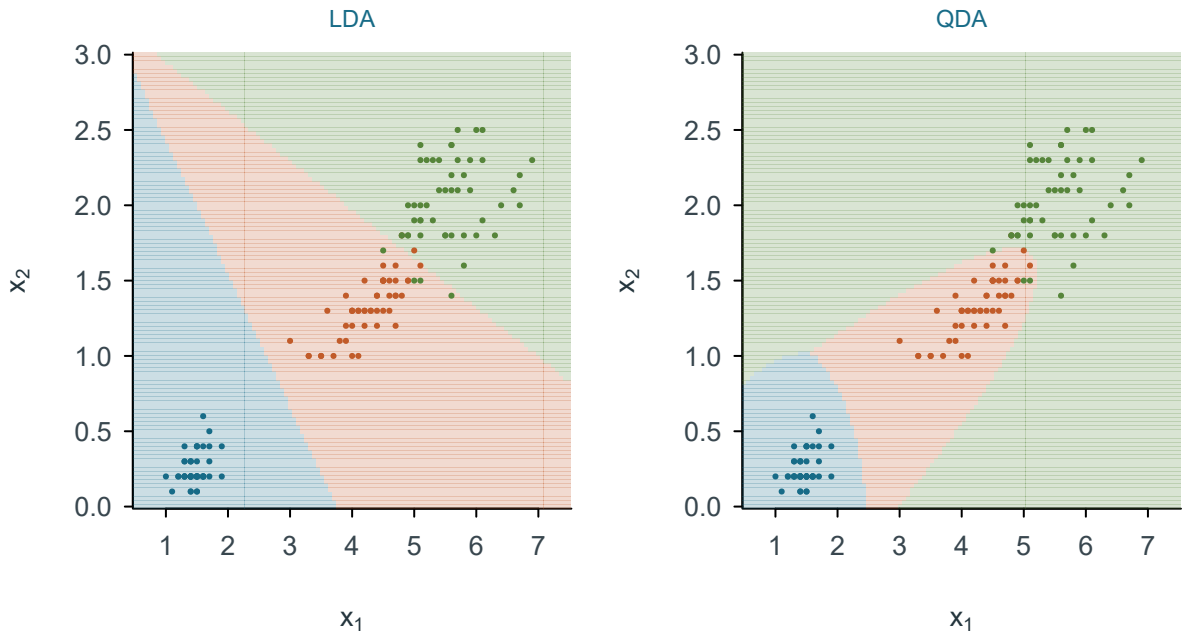


Figure 8.2: Descriptive LDA and QDA decision regions fitted to the two iris petal measurements. These are training-data illustrations, not test assessments.

8.6 Fisher's linear discriminant analysis

LDA obtained a linear boundary by assuming normal classes with a shared covariance. Many labelled samples are not normal, and writing down a density may be more than the problem requires. In 1936 Fisher asked a different question for two classes (Fisher, 1936). Among all one-dimensional projections, which direction separates the two class means the most, relative to the scatter within the classes? *Fisher's linear discriminant analysis* (FLDA) maximises that ratio. A hyperplane perpendicular to the maximising direction is then the boundary. The criterion uses the class means and a pooled covariance. It does not assume normality, and it does not require the population covariances to be equal. The pooled matrix $\boldsymbol{S}$ is still a constraint: the denominator treats the two classes as having a common scatter. That constraint is why the direction will match the vector $\boldsymbol{\beta}$ of normal LDA, even though the two derivations start from different places.

Let the sample means be $\widehat{\boldsymbol{\mu}}_1$ and $\widehat{\boldsymbol{\mu}}_2$, and let $\boldsymbol{S}$ be their pooled sample covariance. A projection direction is a vector $\boldsymbol{a} \in \mathbb{R}^p$. Its length will not matter, because the ratio below is unchanged if $\boldsymbol{a}$ is rescaled. In the projected space the class means are the scalars $\boldsymbol{a}^\top \widehat{\boldsymbol{\mu}}_1$ and $\boldsymbol{a}^\top \widehat{\boldsymbol{\mu}}_2$, and the *between-class variance* is

$$\left[\boldsymbol{a}^\top\left(\widehat{\boldsymbol{\mu}}_1 - \tfrac{1}{2}(\widehat{\boldsymbol{\mu}}_1 + \widehat{\boldsymbol{\mu}}_2)\right)\right]^2 + \left[\boldsymbol{a}^\top\left(\widehat{\boldsymbol{\mu}}_2 - \tfrac{1}{2}(\widehat{\boldsymbol{\mu}}_1 + \widehat{\boldsymbol{\mu}}_2)\right)\right]^2 = \tfrac{1}{2}\left(\boldsymbol{a}^\top \widehat{\boldsymbol{\mu}}_1 - \boldsymbol{a}^\top \widehat{\boldsymbol{\mu}}_2\right)^2. \tag{8.26}$$

Each bracket equals $\pm\frac{1}{2}\boldsymbol{a}^\top(\widehat{\boldsymbol{\mu}}_1 - \widehat{\boldsymbol{\mu}}_2)$, so the displayed sum is half the squared distance between the projected means. The *within-class variance* in the projected space is $\boldsymbol{a}^\top \boldsymbol{S} \boldsymbol{a}$. Using a pooled $\boldsymbol{S}$ here is a summary of scatter, not a belief that the two population covariances are equal: it adds squared distances from each observation to its own class mean and multiplies by a constant. Without that constant the same sum is the *within-class scatter*. The ratio to be maximised is

$$\frac{(\boldsymbol{a}^\top \widehat{\boldsymbol{\mu}}_1 - \boldsymbol{a}^\top \widehat{\boldsymbol{\mu}}_2)^2}{2\,\boldsymbol{a}^\top \boldsymbol{S} \boldsymbol{a}}. \tag{8.27}$$

The ratio depends on the direction of $\boldsymbol{a}$ and not on its length. The strategy is to apply the Cauchy–

Schwarz inequality after a change of inner product that absorbs $\boldsymbol{S}$, rather than differentiating and solving an eigenvalue problem. Positive-definiteness of $\boldsymbol{S}$ is what lets the square-root factors exist.

Proposition 8.6 (Fisher's direction)**.** Let $\boldsymbol{d} = \widehat{\boldsymbol{\mu}}_1 - \widehat{\boldsymbol{\mu}}_2 \neq 0$ and let S be positive definite. Among nonzero directions, the Fisher ratio is maximised by $\boldsymbol{a} \propto S^{-1}\boldsymbol{d}$. If $\boldsymbol{d} = 0$, the ratio is zero for every nonzero direction, so it selects no preferred direction.

Proof. The notes obtain the maximiser by differentiating and solving an eigenvalue problem. A direct argument uses the Cauchy–Schwarz inequality, and it makes the assumption easy to see. Positive-definiteness of $\boldsymbol{S}$ supplies the square-root factors. Write $\boldsymbol{a}^\top \boldsymbol{d} = (\boldsymbol{S}^{1/2}\boldsymbol{a})^\top (\boldsymbol{S}^{-1/2}\boldsymbol{d})$. Then $(\boldsymbol{a}^\top \boldsymbol{d})^2 \leq (\boldsymbol{a}^\top \boldsymbol{S}\boldsymbol{a})(\boldsymbol{d}^\top \boldsymbol{S}^{-1}\boldsymbol{d})$, so the ratio is at most $\boldsymbol{d}^\top \boldsymbol{S}^{-1}\boldsymbol{d}/2$, with equality if and only if $\boldsymbol{S}^{1/2}\boldsymbol{a}$ is proportional to $\boldsymbol{S}^{-1/2}\boldsymbol{d}$, that is $\boldsymbol{a} \propto \boldsymbol{S}^{-1}\boldsymbol{d}$.

□

The maximiser is $\boldsymbol{a} \propto \boldsymbol{S}^{-1}\boldsymbol{d}$. Cauchy–Schwarz did the work, and it needed $\boldsymbol{S}$ positive definite so that $\boldsymbol{S}^{1/2}$ exists. The maximal value of the ratio is $\boldsymbol{d}^\top \boldsymbol{S}^{-1}\boldsymbol{d}/2$. The discriminant function is the projection $\boldsymbol{a}^\top \boldsymbol{x} = \boldsymbol{x}^\top \boldsymbol{S}^{-1}(\widehat{\boldsymbol{\mu}}_1 - \widehat{\boldsymbol{\mu}}_2)$. The ratio fixes the direction and does not fix the threshold. The boundary is a hyperplane $\boldsymbol{a}^\top \boldsymbol{x} - a_0 = 0$ orthogonal to $\boldsymbol{a}$, and a_0 is a separate choice. A common choice is the midpoint of the two projected class means, $a_0 = \boldsymbol{a}^\top (\widehat{\boldsymbol{\mu}}_1 + \widehat{\boldsymbol{\mu}}_2)/2$. If a normal model is then placed on each class, the threshold can instead sit where the posterior densities are equal, which returns the LDA intercept.

Key idea.
Fisher's direction $\boldsymbol{a} \propto \boldsymbol{S}^{-1}(\widehat{\boldsymbol{\mu}}_1 - \widehat{\boldsymbol{\mu}}_2)$ is exactly the vector $\boldsymbol{\beta}$ of normal LDA (equation (8.21)). Fisher arrived at it without a normality assumption by maximising separation; the normal model supplies it, and the intercept, as a Bayes rule. With equal priors and the midpoint threshold the two rules coincide.

8.7 Mixture discriminant analysis

A single Gaussian density cannot represent a ring-shaped concentration of observations. That does not mean QDA cannot classify a ring around a central group: a quadratic boundary can be circular. Density fit and classification boundary are different questions. Mixture discriminant analysis gives each class a more flexible density by combining Gaussian components (Hastie and Tibshirani, 1996; Fraley and Raftery, 2002). Its benefit should be assessed on the actual prediction task, since estimating more components also costs data.

The number of components is not fixed by the geometry. More data can support more of them, and BIC or AIC (Chapter 6) is the criterion already used for that choice, typically applied separately in each class. If one class is too thin for those components, a kernel density follows the same curve, and it is the estimator that fails first as the dimension grows. When the geometry does not decide between the methods, the comparison is an error rate on observations the fit did not see, which is the subject of the next chapter.

The plug-in step is the Bayes rule with each true class density replaced by its fitted mixture. For each of the G classes, fit a normal mixture to the observations from that class. For a new $\boldsymbol{x}$, evaluate the mixture density of each class, weight by the class probability, known or estimated, and assign $\boldsymbol{x}$ to the class with the largest weighted density:

$$g^* = \arg\max_g \pi_g \widehat{f}_g(\boldsymbol{x}), \qquad \widehat{f}_g(\boldsymbol{x}) = \sum_{h=1}^{H_g} \lambda_{gh} \widehat{f}_{gh}(\boldsymbol{x}), \tag{8.28}$$

where π_g is the prior probability of class g, λ_{gh} is the probability of component h within the mixture for class g, $\widehat{f}_{gh}$ is the fitted density of that component, and H_g is the number of components fitted to class g, chosen separately or jointly. Ordinary LDA and QDA are the special cases $H_g = 1$, with a common covariance or a covariance of their own.

8.8 Kernel density discriminant analysis

Kernel discriminant analysis replaces each class density by a kernel estimate. This avoids choosing a finite component count, but introduces bandwidths and still needs enough observations locally. Its difficulty grows with dimension; six is a software limit in some implementations, not a theorem about when the method stops working. Choose bandwidths inside each training split, then apply the same weighted-density rule:

$$g^* = \arg\max_g \pi_g \widehat{f}_g(\boldsymbol{x}), \tag{8.29}$$

where π_g is the prior probability of class g and $\widehat{f}_g(\boldsymbol{x}) = \frac{1}{n_g}\sum_{i:\, y_i = g} K_{\boldsymbol{H}_g}(\boldsymbol{x} - \boldsymbol{x}_i)$ is the kernel density estimate of class g at $\boldsymbol{x}$. The bandwidth matrices are $\boldsymbol{H}_g$. As in Chapter 7, the method is practical only in a few dimensions.

8.9 R laboratory: LDA and QDA on held-out iris flowers

The common split holds out 15 flowers of each species and trains on the remaining 35, so both fits below see the same four predictors and the same 105 labelled flowers. LDA forces one covariance; QDA estimates one per species. That is the only difference between the two rules being compared.

```
lda_fit <- MASS::lda(Species ~ ., data = train)
qda_fit <- MASS::qda(Species ~ ., data = train)
lda_pred <- predict(lda_fit, test)$class
qda_pred <- predict(qda_fit, test)$class
table(Observed = test$Species, Predicted = lda_pred)
#>             Predicted
#> Observed     setosa versicolor virginica
#>   setosa         15          0         0
#>   versicolor      0         13         2
#>   virginica       0          1        14
c(LDA_test_error = mean(lda_pred != test$Species),
  QDA_test_error = mean(qda_pred != test$Species))
#> LDA_test_error QDA_test_error
#>        0.06667        0.06667
```

One test flower changes the error by 1/45, about 2.2 percentage points, so a sample of 45 held-out flowers makes a coarse comparison. Equal class counts describe one assessment population. A different prevalence, or a different cost of mistakes, changes which accuracy matters, as in the unequal-cost rule (8.15).

8.10 Further exercises

Exercise 8.1 (A Bayes rule in one dimension)**.** Class 1 is $\mathrm{N}(0, 1)$ with prior 0.7 and class 2 is $\mathrm{N}(2, 1)$ with prior 0.3. Find the Bayes decision boundary and the Bayes error rate $\sum_g \pi_g \mathrm{Err}_g$.
Solution

Exercise 8.2 (Unequal costs)**.** A test has posterior probability $\tau_1(\boldsymbol{x}) = 0.3$ of disease. Missing a case costs $c_{12} = 5$ and a false alarm costs $c_{21} = 1$. Which class does (8.15) choose? At which value of τ_1 does the decision switch?

Solution

Exercise 8.3 (LDA with unequal priors)**.** Redo Example 8.5 with priors $\widehat{\pi}_1 = \frac{2}{3}$, $\widehat{\pi}_2 = \frac{1}{3}$. Give the new boundary and classify $(3.4, 3.2)$.

Solution

Exercise 8.4 (A quadratic boundary)**.** Class 1 is $\mathrm{N}(0, 1)$ and class 2 is $\mathrm{N}(0, 4)$ with equal priors. Find the QDA decision boundary and explain why LDA cannot separate these classes at all.

Solution

Exercise 8.5 (Parameter counts)**.** For $p = 10$ and $G = 3$ count the free parameters of LDA and QDA. How large must the training set be, at least, for each of the three QDA class covariance matrices to be nonsingular?

Solution

Exercise 8.6 (Fisher's direction)**.** For the data of Example 8.5, compute Fisher's direction $\boldsymbol{S}^{-1}(\widehat{\boldsymbol{\mu}}_1 - \widehat{\boldsymbol{\mu}}_2)$ and the value of the ratio (8.27) for it and for the naive direction $\widehat{\boldsymbol{\mu}}_1 - \widehat{\boldsymbol{\mu}}_2$. Which is larger?

Solution

Exercise 8.7 (Explain the fitted LDA model)**.** Iris setosa and versicolor are to be classified using sepal length and sepal width.

(a) State the two-class LDA model, its unknown parameters and how they are ordinarily estimated from labelled training data.

(b) Explain why the equal-cost LDA decision boundary is linear.

(c) Explain how unequal class frequencies or unequal error costs can affect the rule.

Solution

Exercise 8.8 (LDA and PCA)**.** Give mathematical explanations, defining the quantities you use.

(a) Explain the two-class LDA model and how its parameters and classification rule are obtained from labelled observations.

(b) Explain the main concepts and mathematical construction of PCA, including principal directions, scores and variance explained.

Solution

Chapter 9

Estimating error rates

The classifier has finished fitting. The question now is how often it will fail on a new observation from the population we care about. Scoring the training sample is easy, but those observations helped choose the rule. We need an assessment in which each scored observation was excluded from every decision that built its predictor, including preprocessing and tuning (Hastie et al., 2009, Chapter 7).

This chapter defines that target and compares the estimators: the apparent error rate, the test-set method, cross-validation, nested cross-validation when a model is being selected, and the bootstrap and the jackknife. The same split between fitting and assessment supplies the likelihood cross-validation of Chapter 6, the choice of k for nearest neighbours (Chapter 11), and the out-of-bag estimate for a random forest (Chapter 13).

Source. Lecture notes, Section 3.9 (pp. 75–82).

9.1 What is being estimated

There are two possible targets. We can hold this particular fitted rule fixed and ask about its error on fresh observations, or repeat the entire training procedure on new datasets and average its error. A test set estimates the first target conditionally on the trained rule. Cross-validation averages scores from several different fitted rules and is commonly used to assess the procedure at the corresponding training size. Start with the fixed-rule quantity:

$$\mathrm{Err} = \mathbb{E}_F \, Q(Y, \eta(\boldsymbol{X})), \tag{9.1}$$

with Q the zero–one loss (8.9). That expectation is over a new draw, with the trained rule held fixed. If mistakes have different costs, or if a single overall rate hides which class is being missed, the class-specific rates Err_g are the right companion. They are the same conditional error probabilities as (8.8), now written so that an estimator can be built from indicators:

$$\begin{aligned} \mathrm{Err}_g &= \mathbb{E}_{F \mid Y=g} \, Q(Y, \eta(\boldsymbol{X})) = \mathbb{P}(\eta(\boldsymbol{X}) \neq Y \mid Y = g) \\ &= \mathbb{P}(\eta(\boldsymbol{X}) \neq Y, \; Y = g) / \mathbb{P}(Y = g) = \mathbb{E}_F \left[Q(Y, \eta(\boldsymbol{X})) \, \mathbb{I}(Y, g) \right] / \pi_g, \end{aligned} \tag{9.2}$$

They recombine by $\mathrm{Err} = \sum_g \pi_g \mathrm{Err}_g$, the same weighted sum as in the previous chapter. Estimating every pairwise rate Err_{gh} for $g \neq h$ is possible in principle and usually unnecessary; without a substantial sample from every class it is also unstable. An estimator $\widehat{\mathrm{Err}}$ is biased by $\mathbb{E}(\widehat{\mathrm{Err}} - \mathrm{Err})$. The expectation runs over whatever data the estimator consumes, training cases, test cases, or resamples, and the sign of the bias is the first thing that distinguishes the methods below.

Definition 9.1 (Confusion matrix)**.** Some authors report a *confusion matrix*, whose (i, j) element is the number of observations of true class i that the classifier predicted to be class j. Dividing it by the total number of observations in the test set(s), the sum of the off-diagonal elements is $\widehat{\text{Err}}$. Dividing the gth row by its own total, the sum of that row excluding the gth element is $\widehat{\text{Err}}_g$.

Example 9.2 (Reading a confusion matrix)**.** A three-class classifier is evaluated on 150 test observations, 50 from each class:

	predicted 1	predicted 2	predicted 3	row total
true 1	**40**	5	5	50
true 2	4	**38**	8	50
true 3	2	6	**42**	50

The off-diagonal total is $5 + 5 + 4 + 8 + 2 + 6 = 30$, so $\widehat{\text{Err}} = 30/150 = 0.20$. By rows, $\widehat{\text{Err}}_1 = 10/50 = 0.20$, $\widehat{\text{Err}}_2 = 12/50 = 0.24$, and $\widehat{\text{Err}}_3 = 8/50 = 0.16$. Because the class sizes are equal, $\widehat{\text{Err}} = \frac{1}{3}(0.20 + 0.24 + 0.16)$. The largest confusion is class 2 predicted as class 3 (8 cases).

9.2 The apparent error rate

The apparent error scores a rule on its own training cases. It describes fit to those cases and is usually optimistic as an estimate of future error. This is a repeated-sampling tendency, not a promise that every observed training error is below every test error. For 1-nearest neighbour with distinct inputs, each training observation supplies its own correct label, giving zero apparent error regardless of how useful its neighbours will be for a new point.

Parameter count alone cannot tell us the size of that optimism. Stability, regularisation, signal strength and the geometry of the predictors all matter. Even a low-dimensional fit can be unstable when the information is weak. A zero training error therefore needs an independent assessment, not a story about how many parameters the algorithm contains.

9.3 The test-set method

The apparent rate fails because the cases used to estimate the error helped to build the rule. Holding some labelled cases out of the fit removes that dependence. The *training* set builds the classifier and the *test* set estimates the error. This was first called the *holdout* method and is now usually called the test-set method. It raises four questions, and the first of them is the one most often skipped:

1. Which classifier are we trying to estimate the error rate for?
2. What proportion of the data will be allocated to the test set?
3. How will this proportion be implemented?
4. How can confidence intervals for the error rate be provided?

Which classifier.

An independent test sample evaluates the particular fit trained without it. If we subsequently refit on training plus test data, we have changed the rule; the old score is not an independent assessment of that new fit. It may be a useful guide to a procedure trained on slightly more data, but that requires a judgement about how its performance changes with training size.

What proportion.

Choosing the split involves a trade-off. More training observations may improve the rule, while more test observations make its estimated error more precise. Improvement with training size is not a universal monotonicity theorem. Choose a split large enough to assess the errors that matter, especially within rare classes, and report the training and test sizes alongside the score.

Confidence intervals.

If the test cases are independent draws and the trained rule is held fixed, the number of errors on the test set is a binomial count. A confidence interval for a binomial proportion then covers the error of that particular trained rule. Its width is proportional to $1/\sqrt{n_{\text{test}}}$, which is why a small test set, however carefully held out, still gives a coarse estimate. The interval does not cover the extra uncertainty that comes from having trained the rule on one sample rather than another.

Implementation.

For iid observations, random splitting prevents a file's sorting order from determining the assessment. It does not repair dependence. Repeated measurements from the same person require group-based splits when the target is a new person; future prediction may require a time-ordered split. The sampling unit and deployment target determine what must be held out together. Under the iid setup, the empirical test error is

$$\widehat{\text{Err}}_{\text{test}} = \frac{\sum_{i=1}^{n_{\text{test}}} Q\left[y_i, \eta_{(\boldsymbol{X}_{\text{train}}, \boldsymbol{y}_{\text{train}})}(\boldsymbol{x}_i)\right]}{n_{\text{test}}}, \tag{9.3}$$

and the class-specific rates divide the errors inside class g by the number of test cases from that class,

$$\widehat{\text{Err}}_{\text{test},g} = \frac{\sum_{i=1}^{n_{\text{test}}} Q\left[y_i, \eta_{(\boldsymbol{X}_{\text{train}}, \boldsymbol{y}_{\text{train}})}(\boldsymbol{x}_i)\right] \mathbb{I}(y_i, g)}{\sum_{l=1}^{n_{\text{test}}} \mathbb{I}(y_l, g)}. \tag{9.4}$$

Example 9.3 (How many test observations?)**.** Suppose a classifier makes 18 errors on $n_{\text{test}} = 120$ test observations, so $\widehat{\text{Err}}_{\text{test}} = 0.15$. Treating the count as binomial, the standard error is $\sqrt{0.15 \times 0.85/120} = 0.0326$ and the approximate 95% interval is $0.15 \pm 1.96(0.0326) = (0.086, 0.214)$: far from precise. To halve the half-width to 0.03 with $\widehat{p} \approx 0.15$ one would need $n_{\text{test}} \approx 1.96^2(0.15)(0.85)/0.03^2 \approx 545$ observations.

9.4 Cross-validation

A single holdout spends some cases on testing and never uses them for training, and it spends the rest on training and never uses them for testing. When the labelled sample is small, that waste is the difficulty. Cross-validation, proposed around 1968 as an improvement on the holdout (Lachenbruch and Mickey, 1968) and developed by Stone (1974) and Geisser (1975), still partitions the data into a training piece and a test piece and counts errors. It then repeats the partition so that every case is held out once. The rule scored in each repetition is the rule trained without that case.

9.4.1 Leave-one-out cross-validation

The smallest holdout that still keeps the test case out of the fit leaves out one observation and trains on the other $n - 1$. Repeating this once for each observation uses every case as a test case and almost every case, each time, as a training case. The *leave-one-out* estimator averages those n errors:

$$\widehat{\mathrm{Err}}_{\mathrm{LOOCV}} = \frac{\sum_{i=1}^{n} Q\left[y_i, \eta_{(\boldsymbol{X}_{\mathrm{all}}^{-i}, \boldsymbol{y}_{\mathrm{all}}^{-i})}(\boldsymbol{x}_i)\right]}{n}, \tag{9.5}$$

where $-i$ means that the ith observation is left out of the fit that is then scored on $\boldsymbol{x}_i$. The class-specific estimators restrict the average to class g,

$$\widehat{\mathrm{Err}}_{\mathrm{LOOCV},g} = \frac{\sum_{i=1}^{n} Q\left[y_i, \eta_{(\boldsymbol{X}_{\mathrm{all}}^{-i}, \boldsymbol{y}_{\mathrm{all}}^{-i})}(\boldsymbol{x}_i)\right] \mathbb{I}(y_i, g)}{\sum_{l=1}^{n} \mathbb{I}(y_l, g)}. \tag{9.6}$$

Leave-one-out trains on almost as many observations as the final fit. That reduces the difference in training size, but does not make the error indicators independent. Two training sets share $n-2$ observations, and both depend on the same original sample. A binomial standard error applied to the n leave-one-out errors would ignore this dependence.

A useful check is to ask what changes between repetitions. In leave-one-out, the held-out case changes, but almost the entire training sample stays the same. Repeating the same leave-one-out calculation cannot reveal the variability between genuinely new datasets. More fitting is not automatically more independent evidence.

9.4.2 K-fold cross-validation

Leave-one-out refits the model n times, and its training sets are almost copies of one another. Geisser's K-fold cross-validation spends more cases on each test and refits only K times. About n/K observations, a *fold*, are held out at each iteration, the folds do not overlap, and after K iterations every observation has been tested once (Figure 9.1):

$$\widehat{\mathrm{Err}}_{K\mathrm{CV}} = \frac{\sum_{k=1}^{K} \sum_{i=1}^{n^k} Q\left[y_i^k, \eta_{(\boldsymbol{X}_{\mathrm{all}}'^k, \boldsymbol{y}_{\mathrm{all}}'^k)}(\boldsymbol{x}_i^k)\right]}{n}, \tag{9.7}$$

Here n^k is the size of fold k, and $'^k$ denotes the fit excluding that fold. Pooling individual errors gives each observation equal weight even if fold sizes differ. Increasing K enlarges each training set and increases computation. Its effect on variance is problem-dependent; overlap alone does not prove a universal ordering of cross-validation variances.

Five or ten folds often provide a manageable compromise between training size and fitting cost (Hastie et al., 2009, Section 7.10). Repeating the random partition can show sensitivity to the split and reduce that component of variation. It does not create new observations. Once the data are fixed, leave-one-out has no partition randomness to average away.

9.4.3 Variations

Stratification keeps class proportions approximately balanced across folds when counts permit it. This can prevent a rare class from disappearing from a training set, although one observation in a class cannot appear in every training set while also being held out honestly. Check the actual fold counts rather than assuming that a named resampling method solves the problem.

Repeated K-fold validation and repeated random holdout reuse observations in different ways. Their relative performance depends on the estimator and data-generating setting. The comparisons in Burman (1989) do not establish a universal rule that one repeated scheme is always inferior. Choose the design for the assessment target and available computation.

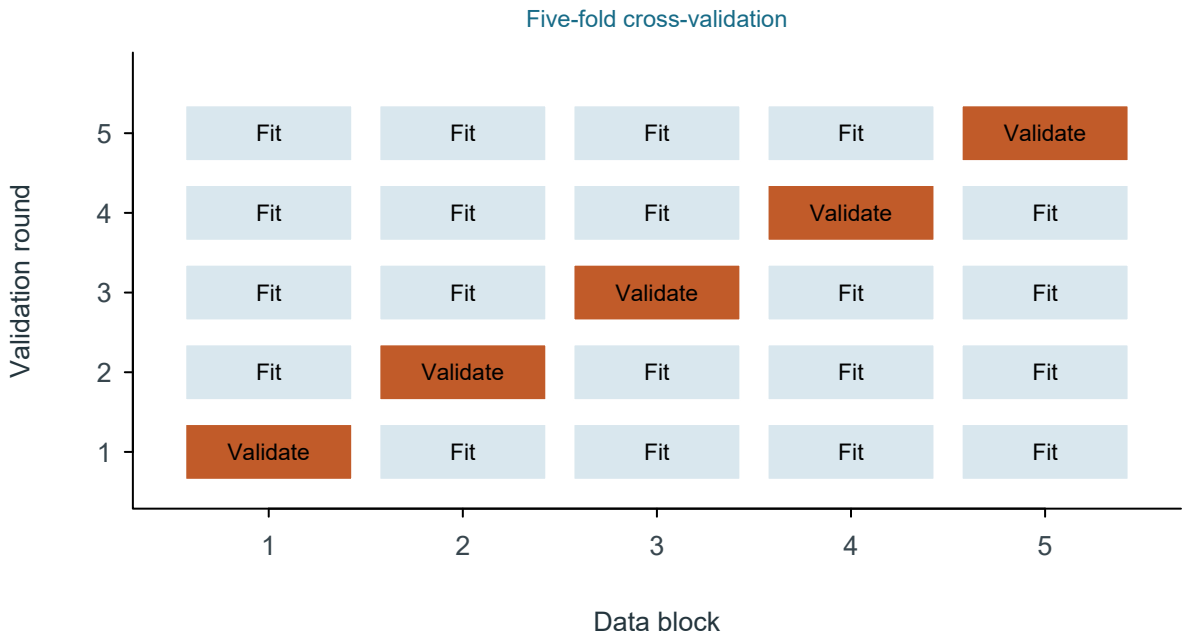


Figure 9.1: Five-fold cross-validation. Every observation is used once for validation and four times for fitting; preprocessing belongs inside each fit.

9.4.4 Nested cross-validation

Cross-validation as written above estimates the error of a rule whose form is already fixed. Choosing a tuning constant λ by the same errors, and then reporting the smallest of those errors as the error of the chosen rule, uses each case twice: once to pick λ, and again to advertise the pick. The advertised number is optimistically biased. As a numerical sketch, the test errors across $\lambda = 1, \ldots, 10$ may have a best value of 0.18. That 0.18 was the criterion that selected λ, so among noisy estimates it is the one that fell furthest below the truth. A validation set may choose λ. A test set that did not enter the choice estimates the error. The optimism is a tendency over repeated samples, not an inequality that every split obeys: one test score can still come out smaller than the selected validation score. Nested cross-validation is this separation when the sample is too small for three permanent blocks.

Nested cross-validation places the complete selection procedure inside an outer assessment loop. This is the separation studied by Cawley and Talbot (2010); the inner winner can differ between outer folds. In the construction below, the remaining $m - 1$ outer folds also serve as inner folds.

1. Split the data into m folds; $m - 2$ of them are for training, one for validation, and one for testing.
2. Using the training and validation data, perform $(m - 1)$-fold CV for each candidate $k = 1, \ldots, K$. Estimate the overall error for each k from the CV results and choose the k that gives the lowest error rate.
3. With this $k = k_1^*$, train the classifier on the training and validation data and test it on the test fold.
4. Repeat for the other choices of test fold (m times in all), getting possibly different $k_2^*, \ldots, k_m^*$ each time. Average the results to estimate the error rate of the final classifier on future observations.
5. For the final classifier, choose a new $k = k_{\text{final}}^*$ by a similar procedure but without a test set

(only training and validation sets).

6. Train the final classifier on all the data using $k = k^*_{\text{final}}$.

The separation is the whole point. The cases that *select* k are not the cases that *assess* the selected model. If one cross-validation error both picks the best of many candidates and is then reported as that candidate's accuracy, the report is optimistic, because the minimum of many noisy estimates is systematically one of the estimates that happened to be low. Step 5 rebuilds the choice of k without a test fold, and step 6 trains the final classifier on every observation at that chosen k. The nested average estimates the error of this whole procedure, including the selection, not the error of whichever k looked best inside one split.



Figure 9.2: Training, ten-fold CV and independent simulated test errors for nearest neighbours; bootstrap inclusion probability approaches 1 - exp(-1).

9.5 The bootstrap

A holdout and a cross-validation fold both train on a subset and test on the complement. The bootstrap builds the training set differently (Efron, 1981, 1983). A non-parametric bootstrap sample of size n is drawn with replacement from the observed n cases, that is, from the empirical distribution. Some observations are missing from the draw and others appear more than once. The probability that a given observation enters a bootstrap sample is one minus the probability that it is left out of all n draws, $1 - (1 - 1/n)^n$, which tends to $1 - 1/\mathrm{e} \approx 0.632$ as $n \to \infty$. For $n = 10$ the probability is 0.651; for $n = 100$ it is 0.634. The expected number of distinct observations in a bootstrap sample is therefore about $0.632n$. The cases left out are a test set that the draw itself created.

Efron's bootstrap estimator scores each observation only on the bootstrap samples that excluded it:

$$\widehat{\mathrm{Err}}_{(\mathrm{BOOT})} = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{|C^{-i}|}\sum_{b\in C^{-i}} Q\left[y_i, \eta_b(\boldsymbol{x}_i)\right], \tag{9.8}$$

where C^{-i} is the set of bootstrap samples that do not contain observation i, $|C^{-i}|$ is the number of such samples, and η_b is the classifier trained on bootstrap sample b. Each case is tested only by rules that did not see it, which is the same discipline as cross-validation. Because each bootstrap training set contains only about $0.632n$ distinct observations, the trained rules are weaker than a rule trained on all n distinct cases, and $\widehat{\mathrm{Err}}_{(\mathrm{BOOT})}$ is generally biased upward. Efron proposed pulling it back toward the apparent error $\overline{\mathrm{err}}$, with weights taken from the inclusion probability and its complement:

$$\widehat{\mathrm{Err}}_{(.632)} = 0.368\,\overline{\mathrm{err}} + 0.632\,\widehat{\mathrm{Err}}_{(\mathrm{BOOT})}. \tag{9.9}$$

The .632 formula combines apparent and out-of-bag errors to address their different biases (Efron, 1983). It is a correction with assumptions, not an identity recovering the unknown generalisation error. Severe overfitting can still make it optimistic. Compare it with a resampling design suited to the full fitting procedure instead of treating its numerical weights as a guarantee.

9.6 The jackknife

Leave-one-out cross-validation scores a classifier. The *jackknife* is the older leave-one-out idea, built to estimate the bias (Quenouille, 1949, 1956) and the standard error (Tukey, 1958) of a parameter estimator rather than an error rate. The difficulty it addresses is that the bias of $\widehat{\theta}$ is $\mathbb{E}(\widehat{\theta}) - \theta$, and θ is unknown, so the bias cannot be computed by subtracting the truth. The jackknife manufactures a comparison inside the sample. The ith jackknife sample leaves out the ith observation. Estimate θ on each of these samples, giving $\widehat{\theta}_{(i)}$ for $i = 1, \ldots, n$, and estimate it on the full sample, giving $\widehat{\theta}$. The average of the leave-one-out estimates is

$$\widehat{\theta}_{(\cdot)} = \frac{1}{n}\sum_{i=1}^{n} \widehat{\theta}_{(i)}. \tag{9.10}$$

If $\widehat{\theta}$ is biased, the leave-one-out estimates, each based on $n-1$ observations, are biased by a slightly different amount. The factor $n-1$ rescales that gap into an estimate of the bias of the full-sample estimator, and the spread of the $\widehat{\theta}_{(i)}$ about their average estimates the standard error. The two formulas are

$$\widehat{\mathrm{bias}}_{\mathrm{jack}} = (n-1)\big(\widehat{\theta}_{(\cdot)} - \widehat{\theta}\big), \qquad \widehat{\mathrm{se}}_{\mathrm{jack}} = \Big[\frac{n-1}{n}\sum_{i=1}^{n}\big(\widehat{\theta}_{(i)} - \widehat{\theta}_{(\cdot)}\big)^2\Big]^{1/2}. \tag{9.11}$$

The factor $n-1$ can be understood from a leading bias term a/n: deleting one observation changes that term to $a/(n-1)$, and multiplying the difference by $n-1$ gives a/n. This explains the first-order correction. It does not mean every realised jackknife bias estimate equals the true bias. Smooth estimators are the natural setting; nonsmooth statistics can behave differently (Efron and Tibshirani, 1993, pp. 141–143).

Example 9.4 (Jackknife bias of the maximum likelihood variance)**.** Let the data be $2, 4, 4, 6, 9$ ($n = 5$) and $\widehat{\theta} = \frac{1}{n}\sum(x_i - \bar{x})^2$, the maximum likelihood variance estimator, which is biased downward by the factor $(n-1)/n$ (equation (2.8)). Here $\bar{x} = 5$ and $\widehat{\theta} = (9 + 1 + 1 + 1 + 16)/5 = 5.6$. Deleting each observation in turn and recomputing the same estimator gives $\widehat{\theta}_{(i)} = 4.1875,\ 6.6875,\ 6.6875,\ 6.6875,\ 2.0$ (deleting 2, 4, 4, 6, 9 respectively), with average $\widehat{\theta}_{(\cdot)} = 26.25/5 = 5.25$. Then $\widehat{\mathrm{bias}}_{\mathrm{jack}} = 4(5.25 - 5.6) = -1.4$, a negative bias as expected, and the bias-corrected estimate is $\widehat{\theta} - \widehat{\mathrm{bias}}_{\mathrm{jack}} = 5.6 + 1.4 = 7.0$, which equals the unbiased sample variance $\frac{1}{n-1}\sum(x_i - \bar{x})^2 = 28/4 = 7$. The jackknife standard error of the (uncorrected) estimate is 3.78.

9.7 Choosing between estimators

Choose an assessment design by naming its target and respecting the sampling structure. An untouched iid test set gives a direct estimate for a fixed fitted rule. Cross-validation uses the available observations more extensively, at the cost of dependent scores from several fits. If model selection is part of training, repeat it inside the assessment loop. No choice of five, ten or n folds compensates for leaking the assessed cases into that selection.

Key idea.
Estimate the error rate of the classifier you intend to *use*, on data that played no part in building or selecting it. When data are scarce, cross-validation lets every observation serve in both roles, but never at the same time, and any model selection must be repeated inside each training split.

9.8 R laboratory: tune neighbours without leaking the test set

The test flowers stay out of the choice of k. Inside the 105 training flowers, five stratified folds choose k, and the scaling is refit on each fold's training piece so that the held-out fold does not help to set the means and standard deviations. The final test evaluation uses the chosen k and is not allowed to change it.

```
fold <- stratified_folds(train$Species, k = 5, seed = 9006)
ks <- seq(1, 21, by = 2)
cv_error <- vapply(ks, function(k) {
  pred <- character(nrow(train))
  for (f in 1:5) {
    a <- fold != f; b <- !a
    zz <- scale_train(train[a, 1:4], train[b, 1:4])
    pred[b] <- as.character(class::knn(
      zz$train, zz$test, train$Species[a], k = k))
  }
  mean(pred != train$Species)
}, numeric(1))
chosen_k <- ks[which.min(cv_error)]  # first minimum is the fixed tie rule
data.frame(k = ks, CV_error = cv_error)
#>     k CV_error
#> 1   1  0.03810
#> 2   3  0.02857
#> 3   5  0.03810
#> 4   7  0.04762
#> 5   9  0.02857
#> 6  11  0.04762
#> 7  13  0.05714
#> 8  15  0.03810
#> 9  17  0.03810
#> 10 19  0.04762
#> 11 21  0.05714
final_pred <- class::knn(iris_scaled$train, iris_scaled$test,
                         train$Species, k = chosen_k)
c(chosen_k = chosen_k, test_error = mean(final_pred != test$Species))
#>   chosen_k test_error
#>    3.00000    0.08889
```

The smallest cross-validation error in the table is an optimistic estimate of the selected procedure, because the same folds chose k. That is the nested-cross-validation point in miniature: selection and assessment have to use different cases. The untouched test flowers supply the separate assessment. Across this book the same test set is used to illustrate one method after another. Reusing it that way does not make a competition in which the best observed test score can be declared the winner.

9.9 Further exercises

Exercise 9.1 (A confusion matrix)**.** A two-class classifier tested on 200 observations (120 from class 1, 80 from class 2) gives the confusion matrix $\begin{pmatrix} 108 & 12 \\ 16 & 64 \end{pmatrix}$ (rows true, columns predicted). Compute $\widehat{\mathrm{Err}}$, $\widehat{\mathrm{Err}}_1$, $\widehat{\mathrm{Err}}_2$, and check $\widehat{\mathrm{Err}} = \sum_g \widehat{\pi}_g \widehat{\mathrm{Err}}_g$ using the class proportions in the test set.

Solution

Exercise 9.2 (Test-set size)**.** For an error rate near 0.10, how many test observations are needed so that the approximate 95% binomial interval has half-width 0.02?

Solution

Exercise 9.3 (The 0.632 rule)**.** Compute $1 - (1 - 1/n)^n$ for $n = 5$ and $n = 20$, and show that it tends to $1 - 1/\mathrm{e}$. If $\overline{\mathrm{err}} = 0.05$ and $\widehat{\mathrm{Err}}_{(\mathrm{BOOT})} = 0.20$, compute the .632 estimate.

Solution

Exercise 9.4 (Jackknife of the mean)**.** For $\widehat{\theta} = \bar{x}$ show that $\widehat{\theta}_{(i)} = \bar{x} + (\bar{x} - x_i)/(n-1)$, that the jackknife bias is 0, and that $\widehat{\mathrm{se}}_{\mathrm{jack}} = s/\sqrt{n}$ with s^2 the unbiased sample variance.

Solution

Exercise 9.5 (Costs of cross-validation)**.** For $n = 1000$, how many times must the classifier be trained for LOO CV, for 10-fold CV, and for nested CV with $m = 10$ outer folds and 9-fold inner CV over $K = 5$ candidate models (count the final training steps too)?

Solution

Exercise 9.6 (A tempting wrong turn)**.** A student tries 50 values of a tuning parameter, picks the one with the lowest 10-fold CV error (8.0%), and reports 8.0% as the expected error on new data. Explain why this is biased and how nested CV corrects it.

Solution

Exercise 9.7 (Separate tuning from evaluation)**.** A student tries 20 classifiers on one validation set and reports the highest validation accuracy as the accuracy expected on new observations.

(a) Explain the problem with this reported value.

(b) Describe a training/validation/test workflow that selects a classifier and estimates its performance. Include preprocessing and any final refitting.

(c) Explain what changes if cross-validation replaces the single validation split.

Solution

Chapter 10

Logistic regression

For classification we need probabilities of labels given measurements. LDA obtains them by modelling the measurements within each class. Logistic regression takes a shorter route: specify the conditional class probability directly. This avoids a distributional model for the predictors, but it still imposes a structure on how they predict the response. The object we want is

$$\mathbb{P}(Y = g \mid \boldsymbol{X} = \boldsymbol{x}), \qquad g = 1, \ldots, G. \tag{10.1}$$

were available directly, the class densities would not be needed, and neither would a second application of Bayes' theorem.

For two classes, logistic regression makes the log odds linear in the predictors. That gives probabilities between zero and one without making probability itself a straight line. We will derive its likelihood, then examine what a coefficient means and when the fit can fail. A coefficient describes a conditional association under the model; causal interpretation requires additional assumptions.

The most familiar conditional model is the linear model for a quantitative response. A linear equation for a probability will not do, because a probability has to stay between 0 and 1, and a line does not. The modification below keeps the linear piece and changes the scale on which it is linear.

Source. Lecture notes, Section 3.10 (pp. 82–89).

10.1 From a linear model to class probabilities

Take two classes, coded 0 and 1. The probability we want to predict is the conditional probability of class 1, which is also the conditional mean of a binary response:

$$\tau_1(\boldsymbol{x}) = \mathbb{P}(Y = 1 \mid \boldsymbol{X} = \boldsymbol{x}), \qquad \tau_0(\boldsymbol{x}) = 1 - \tau_1(\boldsymbol{x}). \tag{10.2}$$

A line fitted to $y_i \in \{0, 1\}$ can predict values below 0 or above 1, and those values are not probabilities. A *generalised linear model* keeps a linear predictor and passes it through an inverse link h^{-1} to produce a mean. For a binary response the mean is the probability $\widehat{\tau}_1(\boldsymbol{x})$, so the distribution around that mean is Bernoulli: each y_i is a single trial with success probability p_i. The link is the function that is assumed linear,

$$h\big(\widehat{\tau}_1(\boldsymbol{x})\big) = \boldsymbol{\beta}^\top \boldsymbol{x} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p, \tag{10.3}$$

where h is the *link function*. The vector $\boldsymbol{x}$ has been augmented with a leading 1, so it has length $p+1$ and β_0 is an intercept.

The inverse link has to return values in $(0,1)$, and it has to be non-decreasing, so that a larger linear predictor means a larger probability of class 1. The standard normal distribution function Φ meets both requirements and gives *probit regression*. The logistic function is the usual choice, because its inverse is the log odds and the derivatives needed for fitting stay simple:

$$\widehat{\tau}_1(\boldsymbol{x}) = h^{-1}(\boldsymbol{\beta}^\top \boldsymbol{x}) = \frac{1}{1+\mathrm{e}^{-\boldsymbol{\beta}^\top \boldsymbol{x}}}. \tag{10.4}$$

Solving $\widehat{\tau}_1 = 1/(1+\mathrm{e}^{-\eta})$ for the linear predictor η gives the link. The algebra is $\eta = \log(\widehat{\tau}_1/(1-\widehat{\tau}_1))$, so the link is the log odds of class 1:

$$h(\widehat{\tau}_1) = \log\Big(\frac{\widehat{\tau}_1}{1-\widehat{\tau}_1}\Big) = \log\Big(\frac{\widehat{\tau}_1}{\widehat{\tau}_0}\Big), \tag{10.5}$$

the estimated *log odds* of class 1, also called the *logit*. A Bernoulli response with this link is *logistic regression*, and the model statement is $h(\widehat{\tau}_1) = \boldsymbol{\beta}^\top \boldsymbol{x}$. The probability is nonlinear in $\boldsymbol{x}$. The log odds are linear in $\boldsymbol{x}$. That split is the reason a one-unit change in a predictor multiplies the odds by a constant and does not add a constant to the probability.

Logistic versus probit.

Both links are smooth curves from 0 to 1, so the choice is not forced by the range of a probability. The scale of the linear predictor can be absorbed into $\boldsymbol{\beta}$, which means the two curves should be compared after they are given the same slope at 0. The logistic function $\sigma(\eta) = 1/(1+\mathrm{e}^{-\eta})$ has derivative $\sigma'(\eta) = \sigma(\eta)(1-\sigma(\eta))$, hence $\sigma'(0) = \frac{1}{4}$. The rescaled normal distribution function $\Phi(c\eta)$ has slope $c\,\phi(0) = c/\sqrt{2\pi}$ at 0. Equating the slopes gives $c = \sqrt{2\pi}/4 = \sqrt{\pi/8}$. With that scaling the curves nearly agree (Figure 10.1). The logistic function approaches 0 and 1 a little more slowly, so it is the heavier-tailed of the two links. The later algebra uses the logistic derivative $\sigma(1-\sigma)$, which is why the logistic link is the one carried forward.

10.2 Likelihood, score, and fitting

During fitting, $\boldsymbol{\beta}$ is the candidate parameter and $p_i = \sigma(\boldsymbol{\beta}^T \boldsymbol{x}_i)$ is its model probability, not yet an estimated probability. Treat the design vectors, including their intercept entries, as fixed. Conditional independence of the responses then gives one Bernoulli factor per observation: p_i if the label is one and $1-p_i$ if it is zero. Combining the cases gives

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{1-y_i}, \tag{10.6}$$

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} \{y_i \log p_i + (1-y_i)\log(1-p_i)\}. \tag{10.7}$$

The log likelihood is the object to maximise. The logit link says $\log \frac{p_i}{1-p_i} = \boldsymbol{\beta}^\top \boldsymbol{x}_i =: \eta_i$, and solving for the two log probabilities gives $\log p_i = \eta_i - \log(1+\mathrm{e}^{\eta_i})$ and $\log(1-p_i) = -\log(1+\mathrm{e}^{\eta_i})$. Substituting those expressions removes p_i from the criterion and leaves a function of $\boldsymbol{\beta}$ alone:

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} \{y_i\eta_i - \log(1+\mathrm{e}^{\eta_i})\}. \tag{10.8}$$

10.2.1 Score and Hessian

There is no closed form for the maximiser, so the next step is the derivative. Differentiate (10.8) with $\partial\eta_i/\partial\boldsymbol{\beta} = \boldsymbol{x}_i$ and $\partial \log(1+\mathrm{e}^{\eta_i})/\partial\eta_i = p_i$. The first derivative is the score; the second is the Hessian. The logistic identity $p_i(1-p_i) = \sigma'(\eta_i)$ is what puts the weights on the diagonal of $\boldsymbol{W}$:

$$\frac{\partial \ell}{\partial \boldsymbol{\beta}} = \sum_{i=1}^{n} (y_i - p_i)\boldsymbol{x}_i = \boldsymbol{X}^\top(\boldsymbol{y}-\boldsymbol{p}), \qquad \frac{\partial^2 \ell}{\partial \boldsymbol{\beta}\,\partial \boldsymbol{\beta}^\top} = -\sum_{i=1}^{n} p_i(1-p_i)\,\boldsymbol{x}_i\boldsymbol{x}_i^\top = -\boldsymbol{X}^\top \boldsymbol{W}\boldsymbol{X}, \quad (10.9)$$

The design matrix is $n \times (p+1)$, the score is $(p+1)\times 1$, and the Hessian is $(p+1)\times(p+1)$. At every finite coefficient vector the weights $p_i(1-p_i)$ are positive. Thus the log-likelihood is concave, and strictly concave when the design has full column rank. This makes any finite stationary point a global maximiser, but does not prove such a point exists or that an unguarded numerical iteration will find it.

Complete separation gives a concrete failure. Move the coefficient vector farther along a separating direction: probabilities approach the observed zeroes and ones, and the log-likelihood approaches its upper bound zero without attaining it at a finite vector. The likelihood is bounded above by one; the coefficients, not the likelihood value, diverge. Quasi-complete separation can also prevent a finite MLE. Check convergence and separation before interpreting standard errors (Hastie et al., 2009, Section 4.4).

10.2.2 Iteratively reweighted least squares

Newton–Raphson updates $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + (\boldsymbol{X}^\top\boldsymbol{W}\boldsymbol{X})^{-1}\boldsymbol{X}^\top(\boldsymbol{y}-\boldsymbol{p})$. The strategy is to rewrite that update as the normal equation of a weighted least-squares problem, so that each iteration can be computed by a regression routine. Define the *working response* $\boldsymbol{z} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{W}^{-1}(\boldsymbol{y}-\boldsymbol{p})$. Then

$$(\boldsymbol{X}^\top\boldsymbol{W}\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{W}\boldsymbol{z} = \boldsymbol{\beta} + (\boldsymbol{X}^\top\boldsymbol{W}\boldsymbol{X})^{-1}\boldsymbol{X}^\top(\boldsymbol{y}-\boldsymbol{p}),$$

which is exactly the Newton step. So each step is a *weighted least-squares* fit of $\boldsymbol{z}$ on $\boldsymbol{X}$, with weights $p_i(1-p_i)$. The weights are functions of the current $\boldsymbol{\beta}$, through p_i, and they are recomputed every iteration. This is iteratively reweighted least squares (IRLS). It typically converges in a handful of iterations. R's `glm` reports the number of Fisher scoring iterations; for logistic regression, Fisher scoring and Newton–Raphson coincide, because the expected Hessian equals the observed Hessian once the variance of a Bernoulli trial is taken to be $p_i(1-p_i)$. The working response is a device that makes one Newton step look like weighted least squares. It is recomputed from the current fit, and it is not itself the binary observation.

Example 10.1 (Newton's method on four points)**. Question.** Fit $\log\frac{p}{1-p} = \beta_0 + \beta_1 x$ to $x = (0,1,2,3)$, $y = (0,1,0,1)$.

Solution. Start at $\boldsymbol{\beta} = \mathbf{0}$: all $p_i = \frac12$, $\boldsymbol{W} = \frac14\boldsymbol{I}$, and $\boldsymbol{y}-\boldsymbol{p} = (-\frac12, \frac12, -\frac12, \frac12)$. The score is $\boldsymbol{X}^\top(\boldsymbol{y}-\boldsymbol{p}) = (0,\ 0\cdot(-\frac12) + 1\cdot\frac12 + 2\cdot(-\frac12) + 3\cdot\frac12)^\top = (0,1)^\top$, and

$$\boldsymbol{X}^\top\boldsymbol{W}\boldsymbol{X} = \frac14\begin{pmatrix}4 & 6\\ 6 & 14\end{pmatrix} = \begin{pmatrix}1 & 1.5\\ 1.5 & 3.5\end{pmatrix}, \qquad \Delta\boldsymbol{\beta} = \frac{1}{1.25}\begin{pmatrix}3.5 & -1.5\\ -1.5 & 1\end{pmatrix}\begin{pmatrix}0\\1\end{pmatrix} = \begin{pmatrix}-1.2\\ 0.8\end{pmatrix}.$$

After this first step $\boldsymbol{\beta} = (-1.2, 0.8)^\top$ and ℓ has risen from $4\ln\frac12 = -2.7726$ to -2.3526. Two more steps give $(-1.356, 0.904)^\top$ and $(-1.3623, 0.9082)^\top$ with $\ell = -2.3475$, after which the score is zero to four decimal places. The asymptotic standard errors, the square roots of the diagonal of $(\boldsymbol{X}^\top\boldsymbol{W}\boldsymbol{X})^{-1}$ at the solution, are 1.97 and 1.09: with four observations, nothing is significant.

10.3 Interpretation and decision boundary

The fitted model is linear on the log-odds scale, which is the scale the link was chosen to linearise. The estimated odds of class 1 are therefore an exponential of the linear predictor:

$$\frac{\widehat{\tau}_1(\boldsymbol{x})}{\widehat{\tau}_0(\boldsymbol{x})} = \frac{\frac{1}{1+\mathrm{e}^{-\boldsymbol{\beta}^\top \boldsymbol{x}}}}{1-\frac{1}{1+\mathrm{e}^{-\boldsymbol{\beta}^\top \boldsymbol{x}}}} = \mathrm{e}^{\boldsymbol{\beta}^\top \boldsymbol{x}}, \qquad \text{so} \qquad \log \frac{\widehat{\tau}_1(\boldsymbol{x})}{\widehat{\tau}_0(\boldsymbol{x})} = \boldsymbol{\beta}^\top \boldsymbol{x}. \tag{10.10}$$

Hold every predictor other than x_j fixed and increase x_j by one unit. The linear predictor increases by β_j, so the log odds of class 1 increase by β_j and the odds are multiplied by e^{β_j}. The probability does not increase by a constant, because the logistic curve flattens near 0 and near 1. The same one-unit reading can be derived for the LDA log-odds score, since under a shared covariance that score is also linear in $\boldsymbol{x}$. A coefficient remains a conditional association inside the fitted model. It does not, by itself, measure the effect of intervening on x_j.

The equal-cost Bayes rule predicts class 1 when $\tau_1(\boldsymbol{x}) \geq 1/2$. Under the logistic model that event is the event that the odds are at least 1, which is the event that the log odds are at least 0:

$$\boldsymbol{\beta}^\top \boldsymbol{x} = 0. \tag{10.11}$$

The boundary is a hyperplane, one dimension below the predictor space, and it has the same shape as the LDA boundary. The agreement in shape is not an agreement in estimates. Under two normal classes with a common covariance, the true posterior log odds are exactly linear, by (8.19), so the LDA model implies a logistic model. LDA estimates the coefficients from the joint distribution of $(\boldsymbol{x}, y)$, using the normality assumption. Logistic regression estimates them from the conditional distribution of y given $\boldsymbol{x}$ and does not model the marginal of $\boldsymbol{x}$. The remark below records the consequences, including what happens when only the class prevalence changes. The classification rule at the probability threshold $1/2$ is the sign of the linear predictor:

$$\eta(\boldsymbol{\beta}) = \begin{cases} 1, & \boldsymbol{\beta}^\top \boldsymbol{x} \geq 0, \\ 0, & \boldsymbol{\beta}^\top \boldsymbol{x} < 0. \end{cases} \tag{10.12}$$

Changing the decision threshold changes the trade-off between false positives and false negatives while leaving the fitted scores fixed. An ROC curve plots these two class-conditional operating characteristics over thresholds. Under pure prior shift, with the class-conditional score distributions unchanged, the ROC curve remains the same although predictive values and the cost-optimal threshold can change.

Products and powers such as x_1, x_1^2, and $x_1 x_2$ can be added to the linear compound. The log odds stay linear in the coefficients, and the boundary in the original predictors becomes curved. The one-unit odds interpretation is then an interpretation in the expanded predictors, and it is harder to say in the original units.

Remark 10.2 (LDA and logistic regression). Two Gaussian classes with common covariance imply linear posterior log odds, so the population LDA model implies a logistic model. Their estimates differ: LDA uses a joint model for measurements and labels, whereas logistic regression uses the conditional likelihood of labels. Correct extra assumptions can improve statistical efficiency; incorrect ones can hurt. Logistic regression avoids Gaussian predictor assumptions, but can still be misspecified through its linear log-odds form (Hastie et al., 2009, Section 4.4.5).

Under pure prior shift, logistic probabilities can also be adjusted without refitting slopes. If $f(\boldsymbol{x} \mid Y)$ stays fixed and the source and target class-one prevalences are π_s and π_t, Bayes' rule gives

$$\operatorname{logit} p_t(\boldsymbol{x}) = \operatorname{logit} p_s(\boldsymbol{x}) + \operatorname{logit} \pi_t - \operatorname{logit} \pi_s.$$

Thus a correctly specified logistic model needs an intercept adjustment. This conclusion depends on stable class-conditional distributions and valid source probabilities; arbitrary distribution shift does not satisfy it.

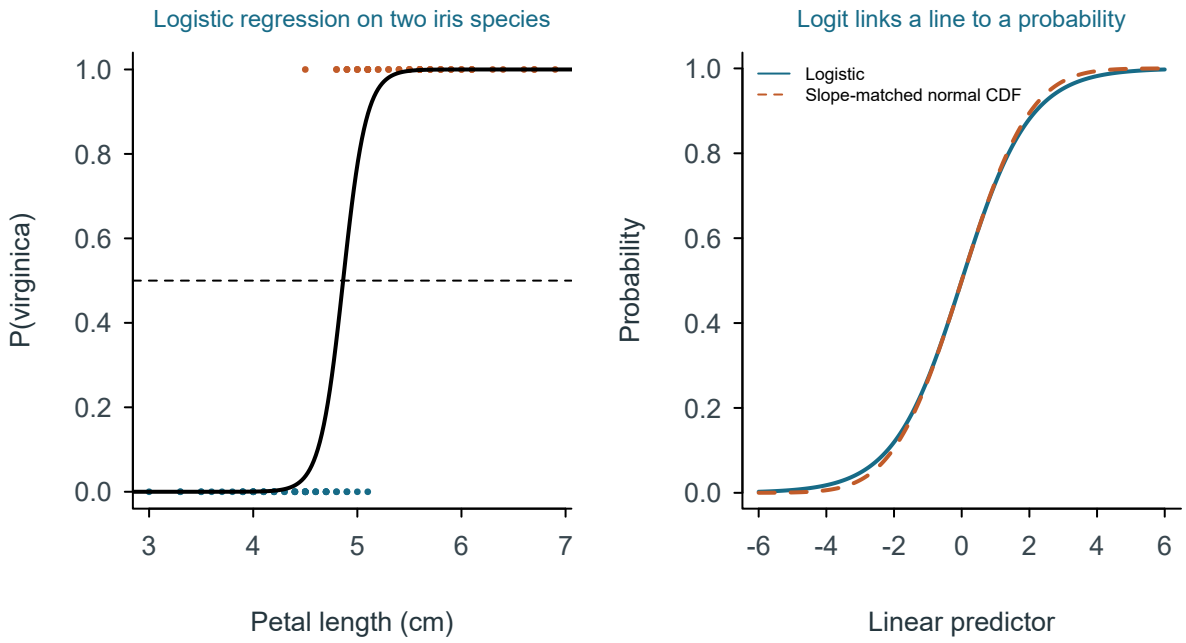


Figure 10.1: A descriptive iris binary logistic fit (left), and the logistic function compared with a slope-matched normal CDF (right).

10.4 More than two classes

Two classes were enough to fix the link. With $G > 2$ nominal classes, a single log odds no longer describes the conditional distribution. Multinomial, or polytomous, logistic regression chooses one class as the *reference*; take it to be class 1. Each remaining class gets its own coefficient vector $\boldsymbol{\beta}_g$, $g \in \{2, \ldots, G\}$, and the model is linear in the log odds against the reference, the same comparison LDA makes one pair at a time:

$$\log\left(\frac{\widehat{\tau}_g(\boldsymbol{x})}{\widehat{\tau}_1(\boldsymbol{x})}\right) = \boldsymbol{\beta}_g^\top \boldsymbol{x}. \tag{10.13}$$

The strategy is to exponentiate those log odds and then use $\sum_{g=1}^{G} \widehat{\tau}_g(\boldsymbol{x}) = 1$, which the binary model obtained for free from $\tau_0 = 1 - \tau_1$. Exponentiating gives $\widehat{\tau}_g(\boldsymbol{x}) = \widehat{\tau}_1(\boldsymbol{x})\, \mathrm{e}^{\boldsymbol{\beta}_g^\top \boldsymbol{x}}$ for $g \geq 2$. Substituting into the sum-to-one constraint,

$$\widehat{\tau}_1(\boldsymbol{x}) = 1 - \sum_{g=2}^{G} \widehat{\tau}_g(\boldsymbol{x}) = 1 - \widehat{\tau}_1(\boldsymbol{x}) \sum_{g=2}^{G} \mathrm{e}^{\boldsymbol{\beta}_g^\top \boldsymbol{x}} \quad \Longrightarrow \quad \widehat{\tau}_1(\boldsymbol{x}) = \frac{1}{1 + \sum_{g=2}^{G} \mathrm{e}^{\boldsymbol{\beta}_g^\top \boldsymbol{x}}},$$

and the remaining probabilities are

$$\widehat{\tau}_g(\boldsymbol{x}) = \frac{\mathrm{e}^{\boldsymbol{\beta}_g^\top \boldsymbol{x}}}{1 + \sum_{g'=2}^{G} \mathrm{e}^{\boldsymbol{\beta}_{g'}^\top \boldsymbol{x}}}. \tag{10.14}$$

What the algebra uses is the linear log-odds specification together with the requirement that the fitted probabilities sum to 1. The reference class is a labelling choice: a different reference

reparameterises the same family of conditional distributions. The coefficient vectors are estimated jointly by maximum likelihood, or by an extension of the binary IRLS iteration. The map from linear predictors to probabilities is the softmax used as the output of a neural network in Chapter 16. An ordered response can use a multinomial model, at some loss of efficiency, because that model ignores the order. Models that use the order, the several forms of *ordinal* logistic regression, make further assumptions and are not developed here.

10.5 Inference, deviance, and model comparison

Likelihood curvature also supplies an approximate uncertainty calculation. At a regular finite logistic MLE, invert X^TWX and take square roots of its diagonal entries to obtain asymptotic standard errors. The Wald statistic divides an estimated coefficient by its standard error. This is an approximation justified by sampling behaviour under the model, not a guarantee supplied by the numerical inverse. Coefficients of discriminant rules can also have uncertainty estimates; their usual software summaries simply answer a different question.

A comparison of two nested models uses the likelihood ratio rather than a single coefficient. The *deviance* is minus twice a log likelihood. For binary data the saturated model, with a free probability at each case, has log likelihood 0, so the deviance of a fitted model is simply -2ℓ. The null deviance is $D_0 = -2\ell$ for the model with only an intercept, and the residual deviance is $D = -2\ell$ for the model with the full $\boldsymbol{\beta}$. A higher likelihood is a smaller deviance. The drop $D_0 - D$ measures the improvement from the explanatory variables, and under the null that those variables have coefficients zero it is approximately χ^2 with degrees of freedom equal to the number of coefficients added. Which model to keep can also be decided by BIC or AIC (Chapter 6), which penalise the likelihood by the number of parameters rather than by a significance cutoff. A categorical explanatory variable with m levels enters as $m - 1$ binary indicators, the dummy-variable coding, with the all-zero pattern as the reference level. Continuous, binary, and categorical predictors are therefore all admissible in the same linear predictor. The Wald interval and the deviance comparison both assume that a finite maximum was attained. Under complete separation the coefficients diverge and those standard errors are not meaningful.

Example 10.3 (Survival after heart failure)**.** The following output is retained as a supplied interpretation exercise using age and high-blood-pressure status. The associated UCI dataset contains 299 patients and records death during each patient's follow-up, with follow-up time stored separately. It does not define a common one-year outcome (Chicco and Jurman, 2020). The printed fit below has not been reconstructed from its original preprocessing, so its coefficients are inputs to the exercise, not newly verified clinical estimates.

R template. This inherited fragment illustrates the interface; the executed laboratory below supplies complete data and checks.

```
> fit <- glm(death ~ age + hbp, data = heart, family = "binomial")
> summary(fit)
Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -3.70568    0.71072  -5.214 1.85e-07 ***
age          0.04618    0.01110   4.159 3.20e-05 ***
hbp          0.26920    0.26487   1.016    0.309
    Null deviance: 375.35  on 298  degrees of freedom
Residual deviance: 354.97  on 296  degrees of freedom
AIC: 360.97
Number of Fisher Scoring iterations: 4
> confint(fit)
                  2.5 %      97.5 %
```

```
(Intercept) -5.14257577 -2.34844333
age          0.02483923  0.06851426
hbp         -0.25344324  0.78698218
```

Interpretation. In the supplied fit, increasing age by one year at fixed blood-pressure status multiplies the modelled odds by $e^{0.04618} = 1.0473$. Ten years multiplies them by $e^{0.4618} = 1.59$. These are odds ratios, not risk ratios: if the baseline probability is p and the odds multiplier is a, the new probability is $ap/(1 - p + ap)$. The printed age interval gives an annual odds-ratio interval of approximately $(1.025, 1.071)$. For high blood pressure, the interval $(0.78, 2.20)$ includes one; it does not establish that the association is exactly zero. The deviance reduction is 20.38 on two degrees of freedom and the printed AIC is $354.97 + 2(3) = 360.97$. These calculations check the interpretation of the supplied numbers, without converting them into causal effects or fixed-horizon predictions.

10.6 R laboratory: odds and probabilities for two iris species

Setosa is easy to separate on these measurements, so a three-class fit would hide the binary difficulty. Restrict training to versicolor and virginica and use sepal length alone. Set $Y = 1$ for virginica, so a positive coefficient means larger modelled odds of virginica. The probability threshold $1/2$ is the equal-cost rule; the Brier score below scores the probabilities themselves.

```
btrain <- droplevels(subset(train, Species != "setosa"))
btest <- droplevels(subset(test, Species != "setosa"))
btrain$y <- as.integer(btrain$Species == "virginica")
logit_fit <- glm(y ~ Sepal.Length, data = btrain, family = binomial())
round(coef(summary(logit_fit)), 4)
#>              Estimate Std. Error z value Pr(>|z|)
#> (Intercept)   -12.446     3.3824  -3.680    2e-04
#> Sepal.Length    1.995     0.5439   3.669    2e-04
exp(coef(logit_fit)["Sepal.Length"])
#> Sepal.Length
#>        7.355
prob <- predict(logit_fit, btest, type = "response")
truth <- as.integer(btest$Species == "virginica")
c(test_error = mean((prob >= 0.5) != truth),
  Brier_score = mean((prob - truth)^2))
#>  test_error Brier_score
#>      0.2000      0.1844
```

The estimated odds multiplier for one additional centimetre of sepal length is 7.35. That number multiplies the odds. It does not add a constant to the probability, because the logistic curve is nonlinear. The Brier score averages the squared difference between the fitted probability and the binary label, so two classifiers can make the same hard decisions at the threshold $1/2$ and still receive different Brier scores when their probabilities differ.

10.7 Further exercises

Exercise 10.1 (Derivative of the logistic function)**.** Show $\sigma'(\eta) = \sigma(\eta)(1 - \sigma(\eta))$ for $\sigma(\eta) = 1/(1 + \mathrm{e}^{-\eta})$, and that $\sigma(-\eta) = 1 - \sigma(\eta)$. Use the first fact to confirm the Hessian (10.9).

Solution

Exercise 10.2 (Odds ratios)**.** Using the heart-failure output, find the odds ratio for a 10-year

increase in age and its 95% interval (from the profile-likelihood interval for the age coefficient). By how much does the predicted probability of death change for a 60-year-old without high blood pressure compared with a 70-year-old, i.e. compute both probabilities?

Solution

Exercise 10.3 (LDA implies a logistic model)**.** For two normal classes with common covariance matrix $\boldsymbol{\Sigma}$, means $\boldsymbol{\mu}_1, \boldsymbol{\mu}_0$ and priors π_1, π_0, show that $\log \frac{\tau_1(\boldsymbol{x})}{\tau_0(\boldsymbol{x})} = \alpha_0 + \boldsymbol{\beta}^\top \boldsymbol{x}$ and identify α_0 and $\boldsymbol{\beta}$.

Solution

Exercise 10.4 (Multinomial probabilities)**.** With three classes and class 1 as the reference, suppose at some $\boldsymbol{x}$ that $\boldsymbol{\beta}_2^\top \boldsymbol{x} = 0.5$ and $\boldsymbol{\beta}_3^\top \boldsymbol{x} = -0.2$. Find $\widehat{\tau}_1, \widehat{\tau}_2, \widehat{\tau}_3$ and the log odds of class 3 against class 2.

Solution

Exercise 10.5 (Complete separation)**.** For $x = (1, 2, 3, 4)$ and $y = (0, 0, 1, 1)$, show that $\ell(\beta_0, \beta_1)$ in (10.8) increases to 0 along $\beta_1 = s$, $\beta_0 = -2.5s$ as $s \to \infty$, so no maximum exists. What does this say about the claim that the log-likelihood has a single maximum?

Solution

Exercise 10.6 (Reading the deviance)**.** In the heart-failure output, the Wald statistic for `hbp` is $z = 1.016$. For a single coefficient the likelihood-ratio statistic for dropping it is approximately z^2. What residual deviance would you therefore expect for the age-only model, and with how many degrees of freedom? Compare with the χ^2_1 critical value 3.84 at the 5% level and state the conclusion. Separately, test the full model against the null model using the two deviances printed.

Solution

Exercise 10.7 (A logistic rule in an unfamiliar setting)**.** A fitted binary logistic model is

$$\log \frac{P(Y = 1 \mid x)}{P(Y = 0 \mid x)} = -3 + 0.8x_1 - 0.4x_2.$$

Assume $Y = 1$ is the positive class.

(a) Find the probability at $x = (4, 2)^T$ and give its classification under a probability threshold of 0.5.

(b) Give the decision-boundary equation and interpret the coefficient of x_1 with x_2 fixed.

(c) Explain two modelling differences between logistic regression and LDA.

Solution

Chapter 11

Nearest-neighbour classifiers

Instead of fitting a global boundary, look near the flower we want to classify. Which species occur among nearby training flowers? The nearest-neighbour rule uses the closest label; k-nearest neighbours uses a vote among the closest k. There is no fitted coefficient vector, but there are still assumptions: the distance must make nearby measurements informative about nearby class probabilities.

Two choices carry the assumptions that a density model would have carried explicitly. The metric decides which differences count as large, so a change of units can change the neighbours unless the predictors are put on a common scale first. That scaling has to be learned from the training cases alone; a scale computed from the whole sample, test cases included, leaks the test set into the fit. In high dimension an irrelevant predictor adds a coordinate that moves the distances without moving the class, and it can overwhelm the coordinates that were actually close. Ties, the choice of k, and the sense in which the rule approaches the Bayes rule are the remaining questions. k is chosen by cross-validation, Chapter 9, because the apparent error always prefers $k = 1$.

Source. Lecture notes, Section 3.11 (pp. 89–93) and Appendix F (R code, pp. 163–165).

11.1 The distance metric and scale

The rule is only as good as its notion of closeness. Any metric that matches the closeness we care about can be used. The usual default is Euclidean distance, the L_2 norm, because it is the ordinary geometric distance once the coordinates are on comparable scales:

$$\|\boldsymbol{x} - \boldsymbol{x}_i\|_2 = \sqrt{(\boldsymbol{x} - \boldsymbol{x}_i) \cdot (\boldsymbol{x} - \boldsymbol{x}_i)} = \sqrt{(x_1 - x_{i1})^2 + \cdots + (x_p - x_{ip})^2}. \tag{11.1}$$

Euclidean distance depends on scale. Changing metres to centimetres multiplies a coordinate's squared contribution by 10^4, while subtracting a common mean leaves every distance exactly unchanged. Standardisation divides by a training-sample measure of spread and therefore changes the metric. Use the same fitted transformation on new observations; recomputing it with held-out data changes the procedure being assessed.

Example 11.1 (Rescaling one coordinate)**.** Take two classes of ten points each, class 1 at (x, x) and class 2 at $(x + 3.5, x)$ for $x = 1, \ldots, 10$ (the data of Appendix B). In original units a 1-nearest-neighbour classifier separates them along a diagonal band and works well: leaving each point out in turn, all 20 are classified correctly. Multiply the first coordinate by 10. Distances are now dominated by differences in x_1, so a point's nearest neighbours are points with similar x_1 regardless of class; the leave-one-out accuracy falls to 0.30 (Figure 11.1). Neither choice of units is more "correct" than the other; the point is that the classifier depends on that choice.

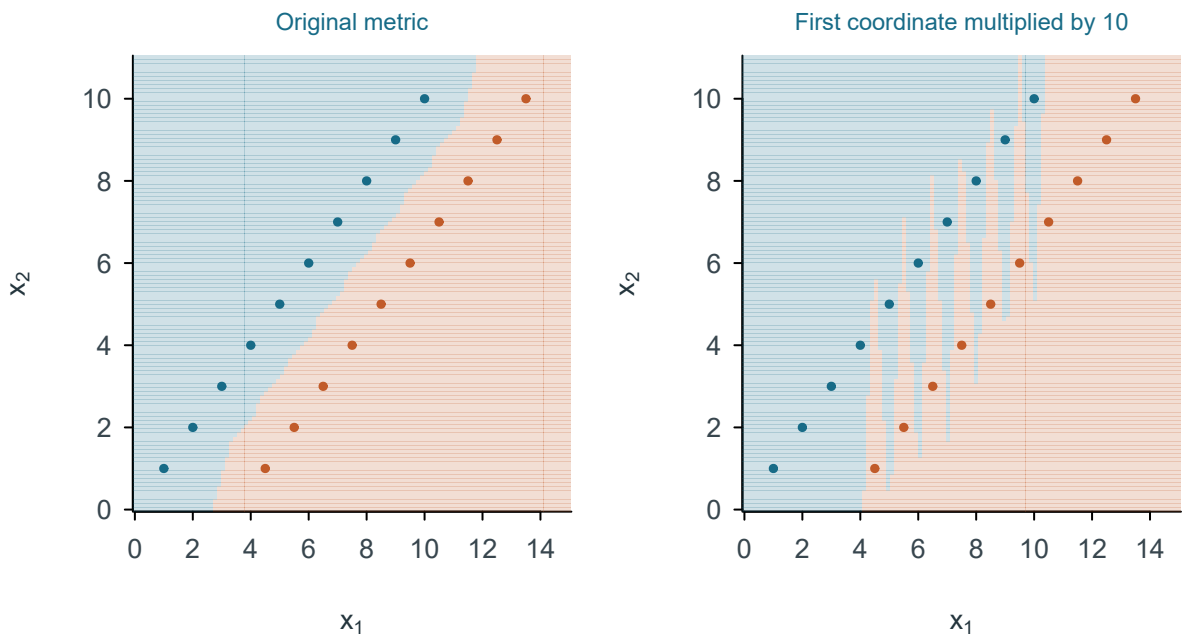


Figure 11.1: The twenty-point scaling example under two distance metrics. Both plots use the original coordinates for comparison; the right classifier multiplies the first coordinate by ten before computing distance.

Cross-reference check.
The notes refer to "R code is in Appendix B" for this example. Appendix B of the notes is the spectral decomposition; the R code for the nearest-neighbour example is in Appendix F (the code is reproduced in Appendix B of this book).

The example makes a useful distinction: describing the same observations in different units need not leave the classifier unchanged. Scaling is one way to specify what comparable differences mean. Standard deviation uses second moments and can be unstable for heavy tails; an interquartile range uses a different notion of spread. Neither makes an irrelevant predictor useful. The scientific question still has to determine which differences should count.

11.1.1 A rank-based distance

Ranks offer a different trade-off. They preserve order while discarding spacing, so a very large value is simply the largest rank rather than an arbitrarily distant point. A strictly increasing coordinate transformation leaves the ranks unchanged. This is useful when order is meaningful and spacing is unreliable, but loses information when actual distances matter.

The rank distance of Olshen (1977) and Devroye (1978) does this one coordinate at a time. For each predictor, the training values and the new observation's value are ranked from 1 to $n+1$, from smallest to largest. Let $r_i^{(j)}$ be the rank of the jth component of the ith training observation and $r^{(j)}$ the rank of the jth component of the new observation. The empirical distance uses the worst coordinate, the L_∞ distance on the ranks:

$$\rho(\boldsymbol{x}, \boldsymbol{x}_i) = \max_{1 \le j \le p} |r_i^{(j)} - r^{(j)}|. \tag{11.2}$$

Any strictly increasing transformation of a single coordinate, including a change of units, leaves the ranks unchanged, so it leaves ρ unchanged. The distances are integers, so ties are common.

They are usually broken by a uniform random ordering among the tied neighbours. The metric has bought invariance by throwing away spacing. If the spacing is real information, for instance if two units of x_1 are known to be equivalent to one unit of x_2, the ranks are the wrong summary. Use the information: replace x_1 by $x_1' = x_1/2$ so that one unit means the same thing on both axes, and then compute an ordinary distance (see Devroye et al., 1996). Scaling, ranks, and a subject-matter equivalence are three answers to the same difficulty, and they are answers about the metric, not about k.

11.2 Ties and the choice of k

There are two kinds of ties: several observations can be equally distant, and several classes can receive the same vote. With exactly k voters and two classes, odd k rules out a vote tie. Software that includes every observation tied at the kth distance can use more than k voters, so that shortcut may no longer apply. Record the tie convention and seed when random tie breaking is used (Venables and Ripley, 2026).

The neighbourhood size is a bias–variance choice, and the apparent error cannot make it. We would like the neighbours of a new $\boldsymbol{x}$ to carry the same class as $\boldsymbol{x}$ itself. If the regions where one class has the highest true posterior are large and roughly spherical in the scaled predictor space, a k with k/n not tiny can be used: many neighbours still lie inside the region. If those regions are small or winding, a large neighbourhood mixes in the other class, and a smaller k is required. The smallest choice is $k = 1$. Nothing guarantees that the single nearest training point is close enough to $\boldsymbol{x}$ to say anything useful, particularly in high dimension, where every point has most of its neighbours far away. Small k produces a jagged boundary, flexible and variable from sample to sample. Large k produces a smooth boundary, stable and biased toward the majority. At $k = n$ every query has the same neighbours, the whole sample, and the rule always predicts the majority class (Figure 11.2). In practice k is chosen by cross-validation (Chapter 9). The apparent error always prefers $k = 1$, for the same reason the 1-nearest-neighbour apparent error is zero: each training point is its own nearest neighbour.

The simulated examples in Hastie et al. (2009, Section 2.5) make the trade-off visible because their true class distributions are known. Small neighbourhoods follow local noise; large ones can smooth away a real minority-class pocket. The iris figure illustrates the same geometric issue without claiming that the true iris Bayes boundary is known. Only a held-out comparison can tell us whether added detail improves prediction.

11.3 Theoretical properties

In fixed-dimensional Euclidean space, the standard iid k-nearest-neighbour rule is weakly universally consistent when $k = k_n \to \infty$ and $k_n/n \to 0$, with appropriate tie handling: its expected risk approaches Bayes risk (Devroye et al., 1996). More neighbours reduce voting noise; a vanishing sample fraction keeps the neighbourhood local. Fixed-k bounds, including the familiar factor-two bound for one neighbour, concern asymptotic risk under their stated conditions. They are not finite-sample upper bounds for an arbitrary metric or iris split.

The factor of two has a short heuristic for two classes, and the strategy is worth seeing before the inequalities. As $n \to \infty$ the nearest neighbour of a fixed $\boldsymbol{x}$ converges to $\boldsymbol{x}$, so its label is an independent draw from the same conditional distribution that produced the label Y of $\boldsymbol{x}$. The two labels differ with probability $2\tau(\boldsymbol{x})(1 - \tau(\boldsymbol{x}))$, where $\tau = \tau_1(\boldsymbol{x})$: either the new point is class 1 and the neighbour is class 0, or the reverse. The Bayes rule, which knows $\tau(\boldsymbol{x})$, errs with conditional probability $r(\boldsymbol{x}) = \min(\tau, 1 - \tau)$. The identity $\tau(1 - \tau) = r(1 - r)$ rewrites the nearest-neighbour conditional error as $2r(1 - r)$. Taking the expectation, the asymptotic error is $\mathbb{E}[2r(1 - r)] \leq 2\mathbb{E}[r] = 2R^*$, where R^* is the Bayes risk, because $1 - r \leq 1$. Jensen's

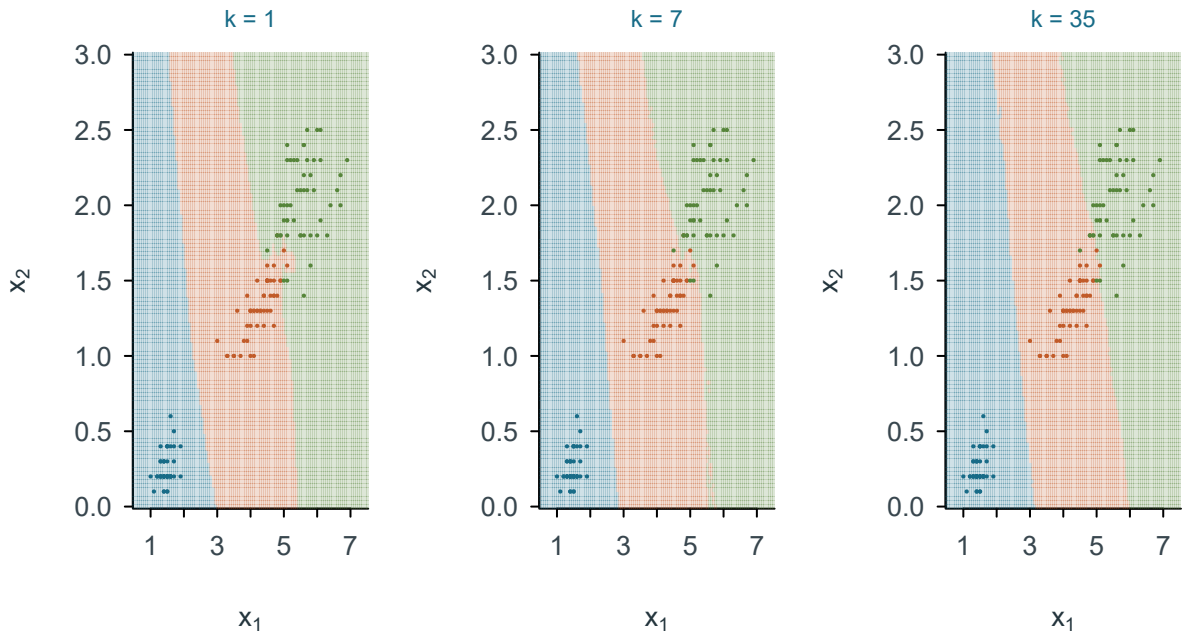


Figure 11.2: Iris nearest-neighbour decision regions for three neighbourhood sizes, using petal measurements.

inequality, using the concavity of $r(1-r)$ on $[0,1]$ together with $\mathbb{E}[r] = R^*$, gives the sharper bound $2R^*(1-R^*)$. The assumption that did the work is the convergence of the neighbour to $\boldsymbol{x}$, which makes the neighbour's label a fresh draw from $\tau(\boldsymbol{x})$ rather than a draw from some other region. No parametric model was used. The conclusion is that even $k = 1$, which stores the sample and estimates nothing else, pays at most twice the minimum possible error in the limit. The universal-consistency conditions ask for more: k must grow, so that the vote estimates $\tau(\boldsymbol{x})$ rather than merely sampling it once, and k/n must shrink, so that the estimate is local. Finite-sample error can still be far from the Bayes error when the dimension is high or the sample is small. Consistency is a limit, not a promise about the iris sample.

Key idea.
The rule replaces a coefficient fit by stored observations, a metric and a neighbourhood size. That simplicity does not remove model selection. Choose preprocessing and k within training data, and inspect whether the resulting neighbours are meaningful. Consistency describes an infinite-data limit, not the adequacy of a particular sample.

R template. This inherited fragment illustrates the interface; the executed laboratory below supplies complete data and checks.

```
library(class)
scaled <- scale_train(x[tr, ], x[te, ])
pred <- knn(train = scaled$train, test = scaled$test, cl = y[tr], k = 7)
mean(pred != y[te])  # assess a pre-specified k
# For CV with estimated scaling, refit scale_train() inside every fold;
# see the executed laboratory in the error-rate chapter.
```

11.4 R laboratory: see what a distance metric changes

The comparison holds k fixed at 7, chosen in advance, so the test set is not used to retune the neighbourhood. One classifier uses the raw predictors. The other standardises from the training flowers only and applies that same shift and scale to the test flowers. Any change in the predictions is then a change in the metric.

```
raw_pred <- class::knn(train[, 1:4], test[, 1:4], train$Species, k = 7)
std_pred <- class::knn(iris_scaled$train, iris_scaled$test,
                       train$Species, k = 7)
c(raw_test_error = mean(raw_pred != test$Species),
  scaled_test_error = mean(std_pred != test$Species),
  changed_predictions = sum(raw_pred != std_pred))
#>      raw_test_error   scaled_test_error changed_predictions
#>             0.06667             0.06667             2.00000
# Distances from the first held-out flower to the training flowers:
d <- sqrt(rowSums(sweep(iris_scaled$train, 2,
                        iris_scaled$test[1, ])^2))
data.frame(Species = train$Species[order(d)[1:7]],
           Distance = round(sort(d)[1:7], 3))
#>    Species Distance
#> 17  setosa    0.225
#> 19  setosa    0.444
#> 47  setosa    0.503
#> 11  setosa    0.554
#> 22  setosa    0.600
#> 15  setosa    0.652
#> 5   setosa    0.906
```

The two versions can have similar accuracy and still use different neighbours, which is why the laboratory also prints the seven nearest training flowers to the first held-out flower under the standardised metric. Standardisation gives the four measurements a common marginal scale. It does not make them equally relevant, it does not remove correlation, and it does not produce an optimal metric. An irrelevant predictor, scaled to unit variance, still moves the distances.

11.5 Further exercises

Exercise 11.1 (Neighbours by hand)**.** Training data on a line: $x = 1$ (class A), 2 (A), 3 (B), 6 (B), 7 (A). Classify $x = 4$ with $k = 1$, 3, and 5. How is the tie at the third neighbour handled for $k = 3$?

Solution

Exercise 11.2 (Changing units)**.** The query is $(0, 0)$ and the training points are $a = (1, 10)$ and $b = (3, 1)$. Which is nearer in Euclidean distance? Divide the second coordinate by 10 and repeat. What does this say about scale invariance?

Solution

Exercise 11.3 (Rank distance)**.** Three training points are $(1, 10)$, $(3, 200)$, $(2.5, 40)$ and the new point is $(2, 50)$. Compute the ranks (from 1 to $n + 1 = 4$) in each coordinate and the rank distances (11.2). Which training point is the nearest neighbour? Show that the answer is unchanged if the second coordinate is replaced by its logarithm.

Solution

Exercise 11.4 (The factor of two)**.** If $\tau_1(\boldsymbol{x}) = 0.9$ at a point, find the conditional Bayes error and the asymptotic nearest-neighbour error. Compare the bound $1 + \sqrt{2/k}$ for $k = 1, 7, 50$.

Solution

Exercise 11.5 (A tempting wrong turn)**.** A student chooses k by minimising the training-set error of the k-nearest-neighbour classifier over $k = 1, \ldots, 50$, counting each training point as its own neighbour. What happens, and what should be done instead?

Solution

Exercise 11.6 (Normalisation and nearest neighbours)**.** A nearest-neighbour classifier uses height in millimetres and a proportion between 0 and 1. An analyst proposes standardising the predictors before selecting k.

(a) Explain why their relative scales matter and state one suitable transformation.

(b) Describe how to choose k using a validation set without contaminating a final test set.

(c) Explain why standardisation does not guarantee that the two predictors become equally informative.

Solution

Exercise 11.7 (Tuning a nearest-neighbour classifier)**.** A dataset contains labelled measurements of two plant species. Predictor variables use different units. There are enough observations for separate training, validation and test sets. You want to select k and estimate the accuracy of the resulting nearest-neighbour procedure.

(a) Explain what normalising the predictors means here and why it is desirable. Specify which data determine the transformation.

(b) Describe how all three sets should be used, including candidate choices, selection, optional refitting and final evaluation.

(c) Across many independent repetitions, selected validation accuracy tends to exceed test accuracy. Explain the mechanism.

(d) Would a validation set also be useful for a decision tree? Give a reason and a concrete example.

Solution

Exercise 11.8 (What an accuracy table does and does not say)**.** An untouched test set gives the following confusion matrix for a tuned classifier. Rows are true classes; columns are predicted classes.

True / predicted	Class 1	Class 2
Class 1	18	2
Class 2	8	32

The chosen model had validation accuracy 0.93.

(a) Calculate the overall and both class-specific test error rates.

(b) Explain why the validation accuracy may exceed test accuracy, without assuming that the data-generating distribution changed.

(c) Describe a valid cross-validation design for choosing both scaling/preprocessing and k in a nearest-neighbour classifier.

(d) A colleague compares 50 further settings using this same test set. Explain how the status of that set changes.

(e) Would classifying everyone as class 2 be a useful baseline? Calculate its overall accuracy on this set and explain the limitation of that metric.

Solution

Chapter 12

Classification trees

A classification tree predicts by asking a sequence of questions: is petal length below this threshold, and, if not, is petal width below that one? Following the answers leads to a leaf and its prediction. For numerical predictors, single-coordinate thresholds carve the space into rectangles. CART and C4.5 are influential constructions of this kind (Breiman et al., 1984; Quinlan, 1993). We will examine how the questions are chosen and why a readable rule can still be unstable.

The price of the readable partition is the way it is grown. At each node the algorithm takes the split that most improves a local impurity score. That greedy step does not look ahead, so an early split that is only slightly better than its rival can commit the rest of the tree. Pruning, or a penalty on the number of leaves, is what stops the partition from cutting until every training case sits in a pure leaf. Even a pruned tree need not be stable. A small change in the sample can change the first split, and every later branch is then a branch of a different tree. That instability is the reason for the ensembles in Chapter 13. Regression trees assign a number rather than a class to each leaf; the splitting idea is the same, and this chapter stays with classification.

Source. Lecture notes, Section 3.12 (pp. 93–101).

12.1 Building a tree

At a node, compare possible splits and choose one that makes the children purer according to a stated criterion. Repeat inside each child. This is a greedy search: a split is judged by its immediate improvement, not by every tree that could eventually grow beneath it. Stopping rules and pruning control the size. Contradictory labels at identical predictor values cannot be separated by further predictor-based splits.

After the tree is built, each training case is passed from the root along the branch its measurements select, until it reaches a leaf. The class proportions in that leaf are recorded, and the leaf is labelled with its majority class. A new $\boldsymbol{x}$ follows the same questions and receives the label of the leaf it reaches. The proportions in the leaf are a crude local estimate of the posterior $\tau_g(\boldsymbol{x})$, constant on the rectangle the leaf owns. The Bayes rule would use the true posterior. The tree uses the majority in the rectangle that contains $\boldsymbol{x}$.

The geometry is a partition of the predictor space by thresholds on single coordinates. The same partition reads as a nested list of rules. Four questions organise the rest of the chapter: which impurity a split should reduce, how the best split of a given variable is found, when splitting stops, and what a fitted tree can and cannot be trusted to mean.

12.2 Node impurity

The split has to optimise something. The training misclassification rate is the natural first choice, because the Bayes rule itself minimises the probability of a mistake under zero–one loss. It is a poor guide for growing a tree. A node that is 80% class 1 and a node that is 51% class 1 have different mixtures, and a split that moves probability mass without yet changing the majority can leave the misclassification rate unchanged. Smoother impurities notice that move. The three usual measures of node *impurity* are cross-entropy, the Gini index, and the misclassification rate. At node m let $\widehat{p}_{mg}$ be the proportion of the training cases at that node which belong to class g. These proportions are the empirical posterior at the node, constant for every $\boldsymbol{x}$ the node still contains.

Definition 12.1 (Node impurity functions)**.** For G classes, the impurity of node m is

$$\text{Cross-entropy:} \quad I_C(m) = -\sum_{g=1}^{G} \widehat{p}_{mg} \log \widehat{p}_{mg}, \tag{12.1}$$

$$\text{Gini index:} \quad I_G(m) = \sum_{g=1}^{G} \widehat{p}_{mg}(1 - \widehat{p}_{mg}) = 1 - \sum_{g=1}^{G} \widehat{p}_{mg}^2, \tag{12.2}$$

$$\text{Misclassification rate:} \quad I_M(m) = 1 - \widehat{p}_{mg^*}, \qquad g^* = \arg\max_g \widehat{p}_{mg}, \tag{12.3}$$

where in (12.1) a term with $\widehat{p}_{mg'} = 0$ is taken to be 0, the limiting value of $p \log p$ as $p \to 0$.

For two classes, Gini and entropy are strictly concave functions of the class proportion. Misclassification impurity is piecewise linear, not flat: $\min(p, 1-p)$. If both children retain the parent's majority class, their weighted misclassification impurity equals the parent's even when their proportions differ. Gini and entropy can reward that change before a majority flips. Entropy is differentiable in the interior; its derivative is not finite at the endpoints (Hastie et al., 2009, Section 9.2.3).

A node whose cases are all one class has impurity zero under every one of these functions, and splitting it further cannot help. Every one of the functions is largest when the class proportions at the node are equal. The local aim is therefore to reduce impurity as much as the current node allows. A split replaces node m by a left child and a right child. The reduction weights the children by the fraction of the node's cases they receive, so a tiny pure child does not count as much as a large one:

$$\Delta I(m) = I(m) - \widehat{P}_{Lm}\, I(m_L) - (1 - \widehat{P}_{Lm})\, I(m_R), \tag{12.4}$$

where m_L and m_R are the children of node m, and $\widehat{P}_{Lm}$ is the proportion of the cases at node m that take the left branch. For the Gini index and for entropy, concavity of I as a function of the class proportions gives $\Delta I(m) \geq 0$: a split cannot increase the weighted impurity. The misclassification rate is not strictly concave, and a split can leave it unchanged. The search described next maximises $\Delta I(m)$ over variables and cut-points. It is greedy. It does not score the tree that the children will eventually become.

12.3 Choosing a split

The value of $\Delta I(m)$ depends on which cases go left. The split is a variable j_m and a threshold on that variable. If predictor j is continuous, the threshold is a number λ_m, and $\boldsymbol{x}$ goes left when $x_{j_m} < \lambda_m$. A discrete or ordinal predictor uses the same rule with λ_m restricted to the observed

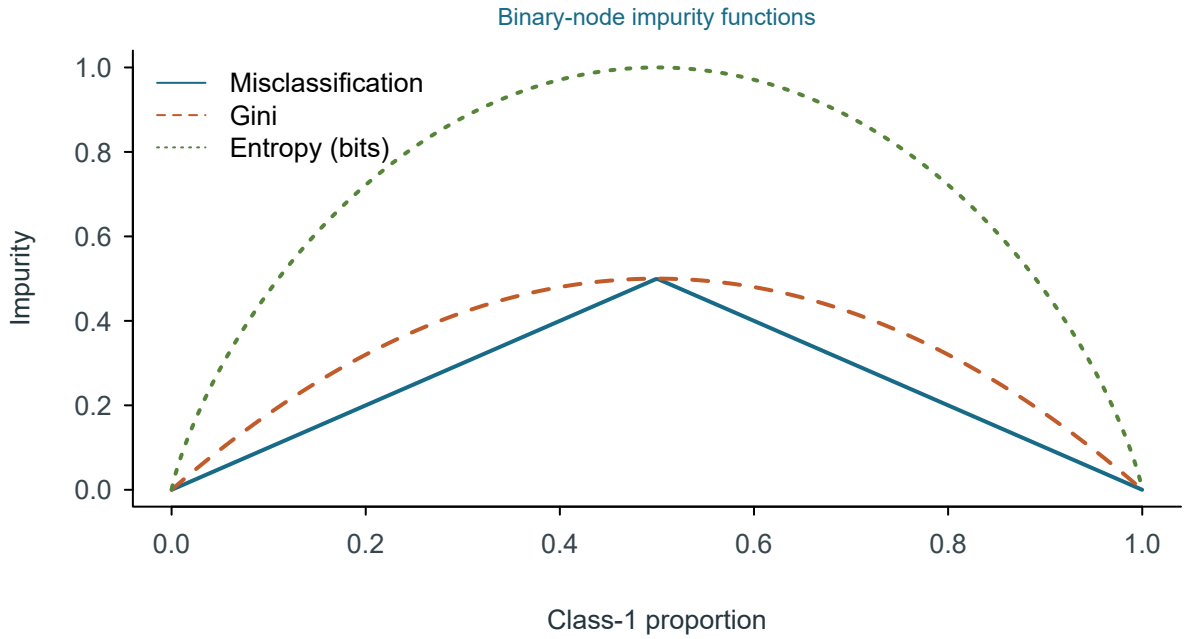


Figure 12.1: Binary misclassification, Gini impurity and entropy. Entropy here uses base-2 logarithms.

values, from the second through the last. A binary predictor sends $x_{j_m} = 0$ left and $x_{j_m} = 1$ right. A nominal predictor has no order to cut. One candidate is to peel off a single category and leave the rest together. Another is to search subsets of categories, which is feasible when the number of subsets is moderate, say 100 or fewer. Both are still binary questions. The tree's readability comes from refusing to cut on a linear combination of coordinates, which is exactly the cut LDA and logistic regression are willing to make. A diagonal boundary has to be approximated by a staircase of axis-aligned cuts.

When the variable is discrete or categorical, every legal split can be scored and the best kept. For a continuous variable there is no analytic maximiser of $\Delta I(m)$, because the class distributions are not known; the criterion is an average over the cases at the node, not an integral against f_g. The search runs through the ordered observed values, from the second smallest to the largest, and places the cut between a value and the one before it. Any cut strictly between two consecutive training values $a < b$ sends the same training cases left and right, so it has the same estimated impurity. The training criterion does not say where in (a, b) the cut should sit.

Where in (a, b)?

Any threshold strictly between two consecutive distinct observations gives the same training partition. The midpoint is a convention for extending that partition to future observations. The training impurity does not identify a special scientific threshold within the gap. Report the fitted cutoff as part of a predictive rule, with this finite resolution in mind.

Invariance.

Nearest neighbours changed their minds when a coordinate was rescaled, because Euclidean distance mixes the units. A tree never mixes units inside a split. Replacing predictor j by $ax_j + b$ with $a \neq 0$ transforms every candidate cut on that variable and leaves ΔI unchanged, so the sequence of variables and the order of splits stay the same (Exercise 12.4). The predictors do not need to be standardised before the fit. After a transformation the published cut-points transform with the data. A rotation is a different matter: the tree can cut only on coordinate axes, and a

rotation moves those axes. The invariance is to monotone affine changes of single coordinates, not to linear combinations.

Multi-way splits.

Allowing more children can reduce weighted impurity simply by refining the partition; the improvement need not be strict. A comparison across different numbers of children therefore needs a complexity adjustment. The gain ratio divides the impurity reduction by the entropy of the split allocation:

$$\Delta I(m) = \frac{I(m) - \sum_{k=1}^{K} \widehat{P}_{km} I_{mk}}{-\sum_{k=1}^{K} \widehat{P}_{km} \log \widehat{P}_{km}}, \tag{12.5}$$

Here $\widehat{P}_{km}$ is the proportion sent to child k. The denominator is positive for a nontrivial split. Dividing by it adjusts for the split's allocation pattern, but can also favour highly unbalanced splits with a small denominator. It is not a universal correction for selection bias or a guarantee of balanced children.

Missing data.

Missing-value handling is part of the algorithm and differs between implementations. CART can use surrogate splits: questions on other observed variables chosen to mimic the primary split's left/right allocation. Sending a case to the majority branch is a fallback, not the definition of a surrogate. R's `rpart` controls these choices through `usesurrogate` and `maxsurrogate`; state the settings when they affect predictions (Therneau and Atkinson, 2023).

Example 12.2 (Building a tree in one dimension)**. Question.** Two classes, five observations of a single variable: class 1 has $x = 1$ and 6; class 2 has $x = 2$, 5, and 11. What tree is built using cross-entropy?

Solution. The root node has $\widehat{p} = (2/5,\ 3/5)$, so it would be labelled class 2 if the tree stopped, and

$$I_C(\text{root}) = -\left(\tfrac{2}{5}\ln\tfrac{2}{5} + \tfrac{3}{5}\ln\tfrac{3}{5}\right) = 0.3665 + 0.3065 = 0.673.$$

The ordered observations are $1, 2, 5, 6, 11$ (classes $1, 2, 2, 1, 2$), so there are $n - 1 = 4$ candidate splits; the child's class is determined automatically by the majority at the child. The cut-point is taken at the midpoint and $\widehat{P}_L$ is the fraction of the five observations on the left.

Split (cut-point)	$\widehat{P}_L$	Left observations	$I_C(m_L)$	Right observations	$I_C(m_R)$	ΔI
$1 \mid 2, 5, 6, 11$ (1.5)	0.2	$\{1\}$ pure	0	$\{2, 5, 6, 11\}$: 1:3	0.562	**0.223**
$1, 2 \mid 5, 6, 11$ (3.5)	0.4	$\{1, 2\}$: 1:1	0.693	$\{5, 6, 11\}$: 1:2	0.637	0.014
$1, 2, 5 \mid 6, 11$ (5.5)	0.6	$\{1, 2, 5\}$: 1:2	0.637	$\{6, 11\}$: 1:1	0.693	0.014
$1, 2, 5, 6 \mid 11$ (8.5)	0.8	$\{1, 2, 5, 6\}$: 2:2	0.693	$\{11\}$ pure	0	0.118

(The notation a:b gives the number of class-1 and class-2 observations.) For the first split, $\Delta I = 0.673 - 0.2\times 0 - 0.8\times 0.562 = 0.223$, the largest. The left child $\{1\}$ is pure, hence a leaf of class 1. Now repeat on the right node $\{2, 5, 6, 11\}$ with classes $2, 2, 1, 2$ (impurity 0.562): cutting at 3.5 gives $\Delta I = 0.562 - 0.25(0) - 0.75(0.637) = 0.085$; at 5.5, $\Delta I = 0.562 - 0.5(0) - 0.5(0.693) = \mathbf{0.216}$; at 8.5, 0.085. So the next split is at 5.5, producing the pure leaf $\{2, 5\}$ (class 2) and the node $\{6, 11\}$, which contains one observation of each class and is split at 8.5 into two pure leaves. The final tree is shown in Figure 12.2: it classifies all five training observations correctly but with four leaves for five points. With the Gini index the same first split wins ($\Delta I = 0.180$ versus $0.013, 0.013, 0.080$).

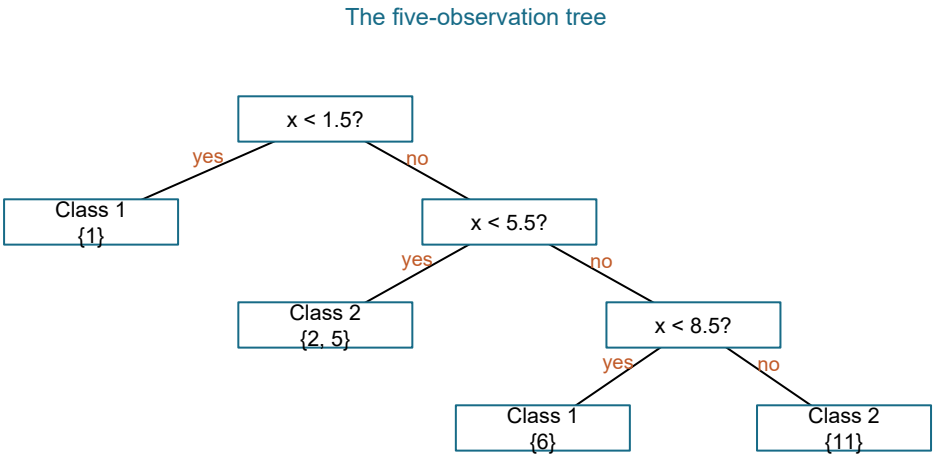


Figure 12.2: The exact five-observation tree from the worked example, redrawn in R. Leaves list the observations they contain.

12.4 Controlling the size of a tree

Grown until every leaf is pure, a tree can cut a noise island around a single training case. Its apparent error is then zero, and Chapter 9 says that number is not an estimate of the error on a new draw. One control is to score the whole tree by its leaf impurities and to charge for the number of leaves:

$$I_{\text{penalised}} = \sum_{\text{leaf nodes}} I(m) + \alpha \,|\text{set of leaf nodes}|, \tag{12.6}$$

where $|\cdot|$ is the number of leaves and $\alpha \geq 0$ is the price of an extra leaf. The tree we want minimises $I_{\text{penalised}}$. One algorithm refuses a split that does not reduce the penalised criterion. Another grows a large tree and then prunes it back to the subtree with the best penalised score. Either way α is still free. There is little theory that names a universal value. Cross-validation can choose it. The same cross-validation can instead watch the classification error directly and stop adding leaves when that error stops falling. Both uses of cross-validation are part of building the tree, so the cases that chose α, or that chose where to stop, cannot also be the cases that estimate the error of the chosen tree. A separate test set can do the estimation. When the sample is too small for a permanent test set, the two-layer scheme of Chapter 9 puts the choice of α inside each training split. The source check below records a further point about the formula as printed: the leaf impurities in (12.6) are unweighted, whereas the CART cost-complexity criterion weights each leaf by its share of the training sample. The weighting is what makes $\Delta I(m) \geq 0$ imply that a split never increases the impurity term, leaving α as the only reason to refuse it.

Weighting in the penalised impurity.
The displayed unweighted sum is retained to distinguish it from CART's standard cost-complexity criterion. CART uses sample-weighted terminal-node risk plus a leaf penalty. Split impurity and pruning risk also need not be the same quantity:

Gini may grow a classification tree while misclassification risk governs pruning. In `rpart`, `cp` is relative to root-node risk, so its numerical value is not automatically the unnormalised α in every textbook formula (Therneau and Atkinson, 2023).

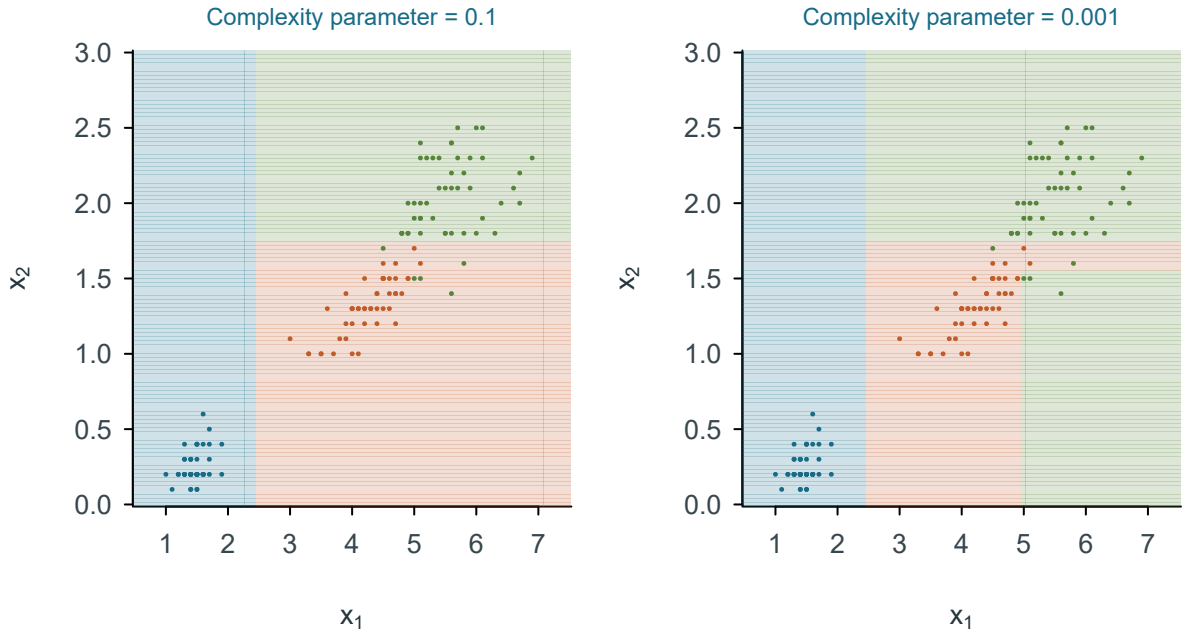


Figure 12.3: Axis-aligned iris classification regions at two complexity penalties, using the same observations and predictors.

12.5 Interpretation and instability

Read a fitted tree from root to leaf to understand its decisions. Then ask whether those questions persist when the training sample changes. A small difference in the best first split can change the data reaching every later node. Pruning reduces complexity but does not prove stability. The next chapter averages trees to reduce sensitivity to individual fits.

Key idea.
A tree is a sequence of locally chosen partitions. Axis-aligned splits make the decisions readable, while limiting the shapes represented efficiently. Pure leaves give zero training error only when purity is attainable and the growth rules allow it. Assess the pruned procedure on held-out observations, rather than treating training purity as the objective of the analysis.

R template. This inherited fragment illustrates the interface; the executed laboratory below supplies complete data and checks.

```
library(rpart)
fit <- rpart(class ~ ., data = train, method = "class",
             parms = list(split = "gini"),        # or "information" for
              ↪  cross-entropy
             control = rpart.control(cp = 0.01))  # complexity parameter alpha
printcp(fit)                                      # cross-validated error by
↪  tree size
```

```
pruned <- prune(fit, cp = fit$cptable[which.min(fit$cptable[, "xerror"]), "CP"])
predict(pruned, newdata = test, type = "class")
```

12.6 R laboratory: prune using training cross-validation

The laboratory grows a large tree on the training flowers and then uses `rpart`'s cross-validation table, computed from those training flowers alone, to prune it. The one-standard-error rule takes the smallest tree whose cross-validated error is within one reported standard error of the minimum. It is a heuristic aimed at stability. It is not a confidence interval for the risk, and the test flowers are not used to choose the penalty.

```
set.seed(12006)
tree <- rpart::rpart(Species ~ ., data = train, method = "class",
                     control = rpart::rpart.control(cp = 0.001, xval = 10))
cp <- tree$cptable
j <- which.min(cp[, "xerror"])
cutoff <- cp[j, "xerror"] + cp[j, "xstd"]
chosen <- which(cp[, "xerror"] <= cutoff)[1]
small_tree <- rpart::prune(tree, cp = cp[chosen, "CP"])
print(cp)
#>       CP nsplit rel error  xerror    xstd
#> 1 0.5000      0   1.00000 1.17143 0.06054
#> 2 0.4714      1   0.50000 0.75714 0.07319
#> 3 0.0010      2   0.02857 0.07143 0.03117
pred <- predict(small_tree, test, type = "class")
c(number_of_splits = sum(small_tree$frame$var != "<leaf>"),
  test_error = mean(pred != test$Species))
#> number_of_splits       test_error
#>           2.0000           0.1111
```

The complexity table is ordered from larger penalties to smaller ones, so the first row that meets the cutoff is the simpler tree. The split thresholds are functions of this training sample. They move if the sample moves, which is the instability above, and they are not botanical constants.

12.7 Further exercises

Exercise 12.1 (Three impurities)**.** A node contains 6, 2, and 2 observations of classes 1, 2, 3. Compute its cross-entropy, Gini index, and misclassification rate. Show that $\sum_g \widehat{p}_g(1-\widehat{p}_g) = 1-\sum_g \widehat{p}_g^2$.

Solution

Exercise 12.2 (Which split is better?)**.** A node has 400 observations of each of two classes. Split A gives children with class counts $(300, 100)$ and $(100, 300)$; split B gives $(200, 400)$ and $(200, 0)$. Show that the misclassification rate cannot distinguish them, and that the Gini index and the cross-entropy both prefer split B. Why is the pure child valuable?

Solution

Exercise 12.3 (A tree by hand)**.** Class 1 has observations $x = 2, 3, 10$ and class 2 has $x = 4, 5, 6, 12$. Find the first split of a tree using the Gini index (compute ΔI for all six cut-points).

Solution

Exercise 12.4 (Affine invariance)**.** Show that if the jth predictor is replaced by $ax_j + b$ with $a \neq 0$, every candidate split on that variable has the same ΔI (with the cut-point transformed to

$a\lambda + b$). What changes if $a < 0$? Does the same hold for a rotation of the predictors?

Solution

Exercise 12.5 (Penalised impurity)**.** Tree 1 has two leaves with (weighted) impurities 0.30 and 0.20; tree 2 has five leaves with impurities $0.05, 0.05, 0.10, 0.00, 0.10$. Compute (12.6) for $\alpha = 0.15$ and for $\alpha = 0.05$ and say which tree is preferred in each case.

Solution

Exercise 12.6 (A tempting wrong turn)**.** A fully grown tree has apparent error rate 0. A colleague concludes that the classifier is perfect. Explain what is wrong and what you would report instead.

Solution

Exercise 12.7 (Decision trees and validation)**.** A binary classification tree is grown until every terminal node contains observations of only one class.

(a) Explain why its training error may be misleading as an estimate of future error.

(b) Is a validation set useful? Explain with a concrete choice of tuning parameter and an explicit selection procedure.

(c) Describe the geometry of the decision regions for a tree that splits one numerical predictor at a time.

Solution

Chapter 13

Ensembles, random forests, and boosting

If one tree changes substantially when we perturb the training sample, try fitting many perturbed versions and combining their predictions. Bagging creates those versions by bootstrap resampling. A random forest also restricts which predictors compete at each split. Boosting takes a different route: each new learner responds to the current ensemble's errors. We will derive what averaging guarantees, then examine the additional choices these algorithms make.

The classifiers of the previous chapters differ in the boundaries they can draw, in how hard they are to compute, and in how much they move when the sample moves. Some regions of predictor space are easy for a given rule and some are not. A method with a random start can return a different fit from the same data; every method returns a different fit from different data, by an amount that depends on the method. Ensemble methods treat that variation as something to average or to correct, rather than as a nuisance to be stabilised away before prediction. The device is a collection of models, combined to predict a new $\boldsymbol{x}$. Averaging reduces variance when the members are not copies of one another. The Bayes rule of Chapter 8 is still a floor: no ensemble undercuts the Bayes risk of the true distribution. The floor may be low. The methods in this chapter spend interpretability and computing time to get closer to it. Useful accounts are Kuncheva (2014), Hastie et al. (2009, Chapters 10, 15, 16), and Rokach (2019).

Source. Lecture notes, Sections 3.13–3.15 (pp. 101–117).

13.1 Why averaging helps

There are two reasons to combine learners. Averaging can cancel variation among reasonably useful fits; sequential correction can target structure left unexplained by an earlier fit. Tukey's twicing applies a smoother to its own residuals (Tukey, 1977), an early example of the second idea. Neither principle says that collecting arbitrary bad models makes a good one.

13.1.1 An error decomposition for regression

Classification error is an indicator, and an average of indicators is not itself the indicator of a majority vote, so the clean algebra starts with a quantitative response. Suppose T models predict a number, with functions $f_1, \ldots, f_T$, and the ensemble predicts their average:

$$f(\boldsymbol{x}) = \frac{1}{T} \sum_{t=1}^{T} f_t(\boldsymbol{x}). \tag{13.1}$$

If the response at this $\boldsymbol{x}$ is y, the squared error of the average splits into the average squared error of the members minus a term that measures how much the members disagree (Perales-González et al., 2022). The strategy is to write each member's error as its deviation from the average plus the average's error, square, and average over the T members. The cross term vanishes because the deviations from the average sum to zero:

$$\big(f(\boldsymbol{x}) - y\big)^2 = \frac{1}{T}\sum_{t=1}^{T}\big(f_t(\boldsymbol{x}) - y\big)^2 - \frac{1}{T}\sum_{t=1}^{T}\big(f_t(\boldsymbol{x}) - f(\boldsymbol{x})\big)^2. \tag{13.2}$$

To see the identity, write $f_t - y = (f_t - f) + (f - y)$ and square: $(f_t - y)^2 = (f_t - f)^2 + 2(f_t - f)(f - y) + (f - y)^2$. Average over t. The cross term vanishes because $\frac{1}{T}\sum_t (f_t - f) = 0$ by definition of the average, and what remains is $\frac{1}{T}\sum_t (f_t - y)^2 = \frac{1}{T}\sum_t (f_t - f)^2 + (f - y)^2$, which is (13.2). All three terms are non-negative. The squared error of the ensemble is therefore at most the average squared error of its members, and the gap is the *diversity*, the sample variance of the members about their average. The algebra used only the definition of the average and the expansion of a square. No model for y was required. The conclusion is a design requirement: the members must be reasonably accurate, so that the first term is tolerable, and they must disagree, so that the diversity term is large. If every member were already the Bayes regression function, the diversity term would be zero and averaging would change nothing.

The identity also applies componentwise to averaged class probabilities under squared error. It does not give the same guarantee for the zero–one loss after taking a majority vote, and it compares with the average member, not the best member. Diversity and individual accuracy cannot be varied independently without changing the first term. Adding noise merely to make predictors disagree need not help.

13.1.2 Stability of base classifiers

Separate variation caused by the dataset from variation caused by the fitting algorithm. A deterministic tree can be sensitive to a small data change; a neural network can also change because of its initial weights. Convexity addresses the optimisation problem, not sensitivity to data: a convex fit can still be unstable when the design is poorly conditioned. Evaluate stability in the setting at hand rather than assigning each method a permanent label.

Two kinds of base learner do supply disagreement. One has many parameters relative to the sample and is hard to optimise, so different runs stop in different places. The other is constrained in size or structure and cannot fit the sample in only one way, so a small change in the data produces a different constrained fit. Trees are in the second group. Networks are in the first.

An average is already an ensemble in a setting that has nothing to do with classifiers. A signal from a distant spacecraft is received together with noise from the Sun, the atmosphere, and other sources. Sending the same signal repeatedly and averaging the receptions lets independent noise cancel at the rate the central limit theorem describes, and leaves the signal. Each noisy reception is the analogue of one flawed classifier. The next two sections build the average in different ways: random forests, which average trees that were made to disagree, and boosting, which adds a learner aimed at the residual of the ones already fitted.

13.2 Decision forests, bagging, and random forests

Trees are useful base learners because they can represent nonlinear interactions and can change under resampling. Random forests commonly grow deep trees. They need not be weak in the precise boosting-theory sense of having only a small advantage over chance. The ensemble's benefit depends on how member errors combine (Breiman, 1996, 2001).

Ho's random-subspace work explored ensembles built from different predictor subsets (Ho, 1995, 1998). Breiman's random forest uses random predictor subsets at individual splits, together with randomised tree construction (Breiman, 2001). These choices create different trees without requiring different measured datasets.

A new $\boldsymbol{x}$ lands in one leaf of each tree. A hard vote takes the majority of the leaf labels. A soft vote uses the class proportions in the leaf as $\widehat{\mathbb{P}}(\text{class} = g \mid \boldsymbol{x}, \text{tree } t)$, a piecewise-constant estimate of the posterior, and averages those probabilities. The soft aggregate predicts

$$g^* = \underset{g \in \{1,\dots,G\}}{\arg\max} \frac{1}{T} \sum_{t=1}^{T} \widehat{\mathbb{P}}(\text{class} = g \mid \boldsymbol{x}, \text{tree } t). \tag{13.3}$$

Ho varied the columns. *Bagging* (bootstrap aggregation), proposed by Breiman (1996), varies the rows. Each model is fit on a non-parametric bootstrap resample, and the fitted models are averaged or voted. The bootstrap is the same resample used for the error-rate estimator in Chapter 9: about $0.632n$ distinct cases appear in each draw. Bagging improves unstable fits, trees and neural networks among them, and does little for LDA, as the diversity calculation already showed. Breiman (2001) combined the bootstrap sample with Ho's random variables in the *random forest*.

At each split of a classification tree the forest offers only about $\sqrt{p}$ randomly chosen coordinates. The restriction forces trees to split on different variables even when their bootstrap samples are similar. The $\sqrt{p}$ choice is a heuristic, not a consequence of the diversity identity. The algorithm is the combination of the two ideas.

Algorithm 13.1 (Random forest)**.** For $t = 1, \dots, T$ build a forest of T trees as follows:

1. Generate a bootstrap sample $(\boldsymbol{X}^{(t)}, \boldsymbol{y}^{(t)})$ from the training data.
2. Fit a tree f_t to $(\boldsymbol{X}^{(t)}, \boldsymbol{y}^{(t)})$, and at each node: (a) randomly select k explanatory variables; (b) determine the best explanatory variable and split among them; (c) add this split to the tree. Repeat (a)–(c) until a stopping criterion is met.

For a new observation $\boldsymbol{x}$ the forest predicts, for regression, $f(\boldsymbol{x}) = \frac{1}{T}\sum_t f_t(\boldsymbol{x})$, and for classification $f(\boldsymbol{x}) = \arg\max_g \sum_{t=1}^{T} \mathbb{I}(f_t(\boldsymbol{x}) = g)$ (or (13.3) if each tree produces class probabilities).

Each node sees only k candidate variables, but a tree has many nodes, so one tree still has chances to split on a substantial fraction of the predictors. The restriction bites locally, at the choice of each split, which is enough to make neighbouring trees disagree.

Why decorrelating the trees helps.

The diversity term said that disagreement helps. The variance of an average says the same thing with a correlation. Suppose the T tree predictions at a fixed $\boldsymbol{x}$ each have variance σ^2 and every pair has correlation ρ. Expand the variance of the sum, using $\operatorname{Var}(f_t) = \sigma^2$ and $\operatorname{Cov}(f_t, f_s) = \rho\sigma^2$ for $t \neq s$:

$$\operatorname{Var}\Big(\frac{1}{T}\sum_t f_t\Big) = \frac{1}{T^2}\Big[T\sigma^2 + T(T-1)\rho\sigma^2\Big] = \rho\sigma^2 + \frac{1-\rho}{T}\sigma^2.$$

Adding trees shrinks only the second term. The first term, $\rho\sigma^2$, is a floor set by the correlation between trees, and no number of additional copies of a highly correlated tree will remove it. The calculation assumes a common variance and a common pairwise correlation. It does not assume normality. Bootstrap samples reduce ρ somewhat, because each tree misses a different third of the cases. Restricting each split to k random variables reduces ρ much more, because the trees are then forbidden from always cutting on the same strongest predictor. That reduction in ρ is the quantitative form of the diversity term in (13.2). The reason to randomise the candidate predictors is to lower the floor, not to improve any one tree.

Hyper-parameters.

Three quantities are still free: k, the number of predictors offered at each split; a control on tree size, such as a maximum depth or a minimum leaf size; and T, the number of trees. Rules of thumb often suffice. A search has to stay off the test set. Nested cross-validation (Chapter 9) puts the search inside the assessment. A simpler design uses one cross-validation to choose the triple and a separate test set to estimate the error of the chosen forest.

- *Number of trees.* The variance formula says the second term falls as T grows and the correlation floor does not, so extra trees help until that floor dominates, and they do not start to over-fit in the way an extra leaf of a single tree does. Computation grows linearly in T, both to train and to predict. The default in R's `randomForest` package is 500. Searching over T is rarely worth the fits; computing time is the practical limit.
- *Variables per split k.* The variance formula wants ρ small, and a single tree wants enough variables to be accurate. k has to be large enough for each tree to have a tolerable error and small enough for the trees to disagree. The usual suggestions are $k = \sqrt{p}$ for classification and $k = p/3$ for regression, the defaults in `randomForest`.
- *Tree size.* A maximum depth is one control, and trees are often asymmetric, so a depth limit cuts some branches that still have cases to split and spares others that do not. The minimum number of observations in a leaf is the control in R's implementation, with default 1 for classification and 5 for regression. The classification default places no limit on size: a leaf may be a single case. A maximum number of leaves is a related control and has no default.

The defaults tend to produce large trees, each fitted carefully on its random variable sets, diverse because of the bootstrap and the restriction on k, and individually accurate enough that the first term of (13.2) stays tolerable. Whether those defaults are enough depends on the problem: the complexity of the target, including the Bayes rule, the number of predictors, their associations with the response and with each other, and the sample size. A search over k and the leaf size is sometimes needed. The number of trees can stay at the default while that search is done.

13.2.1 Out-of-bag estimation and variable importance

An observation is absent from a size-n bootstrap sample with probability $(1 - 1/n)^n \approx 0.368$. Its out-of-bag prediction therefore uses about 36.8% of the trees, not most of them. Aggregate only those trees, then compare the prediction with its label. This estimates performance of the specified fitting recipe, with finite-forest and resampling effects. If out-of-bag scores select among recipes, the selected minimum also contains selection optimism; use a separate assessment of that selection procedure (Breiman, 2001; Cawley and Talbot, 2010).

The average that lowers the variance also removes the single readable tree. Individual trees can still be inspected, and they were built to disagree, so reading one of them does not describe the forest. A partial substitute ranks the predictors. Count how often each variable is chosen for a split, or how much total impurity reduction it contributes, or how much the error rises when its values are permuted and the forest is applied again. The rank is a summary of predictive contribution inside this fit. It is not a ranking of causal effects, and correlated predictors can share or mask one another's rank. The same two devices, random subsets of predictors and random subsets of cases, can be wrapped around base learners other than trees. Trees remain the usual choice because they are fast, unstable in the way the diversity term requires, and individually limited enough that no one tree dominates the average.

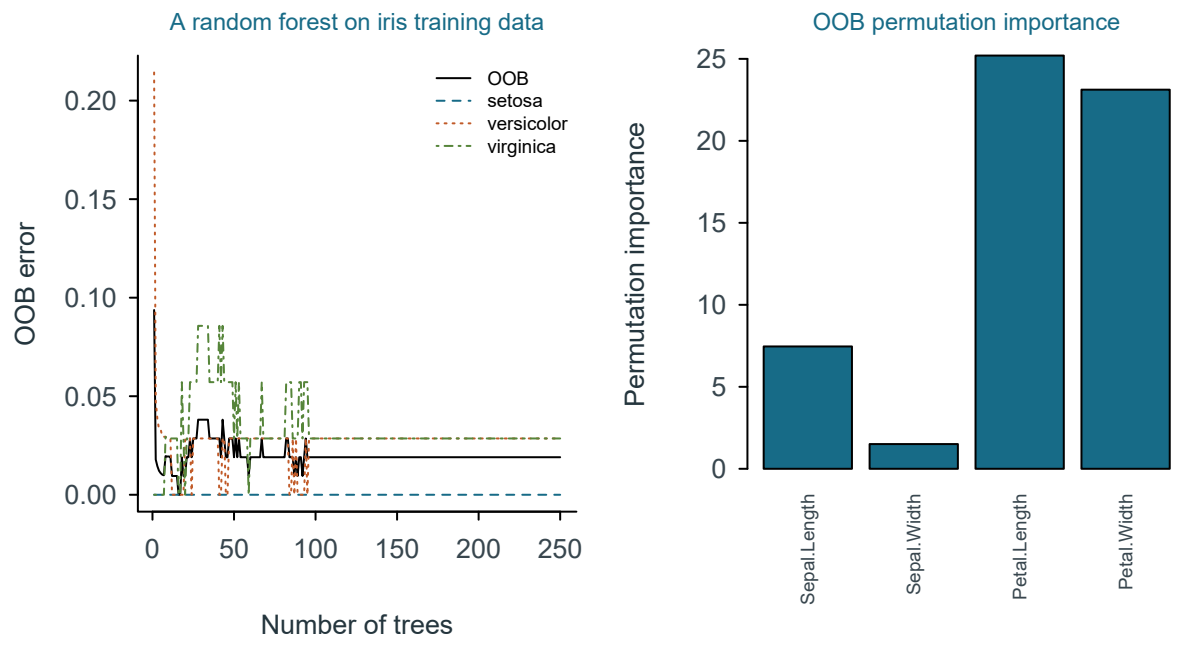


Figure 13.1: Out-of-bag error by forest size and permutation importance on the fixed iris training set. Curves are one seeded fit.

13.3 Boosting

A random forest draws its diversity from different samples and different candidate predictors. The members are fit separately, and none of them is told where the others failed. Equation (13.2) suggests a sharper use of the same idea: the members should be reasonably accurate and should be accurate in different regions, so that each compensates for the others. *Boosting* builds that compensation on purpose. Fit one learner. Find the training cases it gets wrong, or the residuals it leaves. Fit the next learner to those mistakes, without abandoning the cases the first one already got right. Continue, and combine the sequence. The learners are not exchangeable draws. Later ones exist because earlier ones failed somewhere. Schapire (1990) showed that a weak learner can be improved by an ensemble built this way. The concrete algorithm is *Adaptive Boosting* (AdaBoost) (Freund and Schapire, 1995); later methods and theory are in Schapire and Freund (2012). Bagging reduces variance by averaging unstable fits of the same problem. Boosting changes the problem at each round.

13.3.1 The AdaBoost algorithm

AdaBoost needs a coding on which a correct label and a predicted label multiply to $+1$ when they agree and to -1 when they disagree. Record the two classes as $y_i \in \{-1, +1\}$, the same coding used for support vector machines. At iteration t the learner f_t is fit to the training cases with weights $w_{t,i}$. The weights are a distribution over the training sample: they are non-negative and, after the normalisation in the algorithm, they sum to 1. A case the previous learner missed has its weight increased. A case it got right has its weight decreased. The next learner is therefore aimed at the current mistakes and is not released from the rest of the sample. The size of the increase is chosen so that, under the new weights, the learner just fitted has weighted error exactly $1/2$. That property is Proposition 13.3, and it is the reason the particular factor α_t appears in the algorithm rather than an arbitrary step length.

Algorithm 13.2 (AdaBoost)**.** Training data $\{\boldsymbol{x}_i, y_i\}_{i=1}^{n_{\text{train}}}$, $\boldsymbol{x}_i \in \mathbb{R}^p$, $y_i \in \{-1, +1\}$.

1. Set $w_{1,i} = 1/n_{\text{train}}$ for all i.
2. For $t = 1, \ldots, T$:
    1. Fit a learner f_t to minimise the weighted loss, for example the weighted training error rate $\epsilon_t = \sum_i w_{t,i} Q[y_i, f_t(\boldsymbol{x}_i)] / \sum_i w_{t,i}$, where $Q[a, b] = 0$ if $a = b$ and 1 otherwise.
    2. Let $\alpha_t = \frac{1}{2} \log\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$.
    3. Update the weights:

$$w_{t+1,i} = \frac{w_{t,i}}{Z_t} \times \begin{cases} \mathrm{e}^{-\alpha_t}, & f_t(\boldsymbol{x}_i) = y_i, \\ \mathrm{e}^{\alpha_t}, & f_t(\boldsymbol{x}_i) \neq y_i, \end{cases} = \frac{w_{t,i}\, \mathrm{e}^{-\alpha_t y_i f_t(\boldsymbol{x}_i)}}{Z_t},$$

    where $Z_t = \sum_i w_{t,i} \mathrm{e}^{-\alpha_t y_i f_t(\boldsymbol{x}_i)}$ is a normalisation term.

The AdaBoost ensemble function is

$$f(\boldsymbol{x}) = \operatorname{sign}\Big(\sum_{t=1}^{T} \alpha_t f_t(\boldsymbol{x})\Big). \tag{13.4}$$

Dimension check.
The notes write the label vector as $\boldsymbol{y}_{\text{train}} \in \{-1, 1\}^p$. The label vector has one entry per training observation, so it lies in $\{-1, 1\}^{n_{\text{train}}}$; p is the number of predictors.

The weight update is easier to read after simplifying the factor. Because $\alpha_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$, one has $\mathrm{e}^{-\alpha_t} = \sqrt{\epsilon_t/(1-\epsilon_t)}$, the square root of the weighted odds of an error. A correct case is multiplied by $\mathrm{e}^{-\alpha_t}$ and an incorrect case by e^{α_t}, so the ratio of the new un-normalised weight of a correct case to that of an incorrect case, relative to their old weights, is $\epsilon_t/(1-\epsilon_t)$. Freund and Schapire's original weights put $\epsilon_t/(1-\epsilon_t)$ on the correct cases and 1 on the incorrect cases. After normalisation that is the same distribution. If $\epsilon_t < 1/2$, then $\epsilon_t/(1-\epsilon_t) < 1$, and the incorrect cases gain weight. A two-class learner that cannot beat $\epsilon_t = 1/2$ is no better than a coin toss on the current weighted sample, so a usable round satisfies $\epsilon_t < 1/2$ and the update does emphasise the mistakes. If a round has $\epsilon_t > 1/2$, its vote α_t is negative, which is the algorithm voting the opposite of that learner.

Proposition 13.3 (The reweighted error is one half)**.** If $0 < \epsilon_t < 1$, the updated weights give f_t weighted error exactly $1/2$. For the usual positive-vote boosting round we require $0 < \epsilon_t < 1/2$. At zero error, stop or handle the perfect learner explicitly; the logarithmic formula has no finite value.

Proof. The strategy is to add up the new weights of the two groups before normalisation. Assume the current weights sum to 1, so the incorrect cases carry total weight ϵ_t and the correct cases carry $1-\epsilon_t$. With $\mathrm{e}^{2\alpha_t} = (1-\epsilon_t)/\epsilon_t$, the incorrect cases contribute $\epsilon_t \mathrm{e}^{\alpha_t} = \epsilon_t \sqrt{(1-\epsilon_t)/\epsilon_t} = \sqrt{\epsilon_t(1-\epsilon_t)}$ and the correct cases contribute $(1-\epsilon_t)\mathrm{e}^{-\alpha_t} = (1-\epsilon_t)\sqrt{\epsilon_t/(1-\epsilon_t)} = \sqrt{\epsilon_t(1-\epsilon_t)}$. The two totals agree, so after dividing by their sum each group carries weight $1/2$, and the normaliser is $Z_t = 2\sqrt{\epsilon_t(1-\epsilon_t)}$.

□

What the equality establishes is a property of this particular α_t, not a property of every reweighting. Under the new weights the learner just fitted has weighted error $1/2$, so it is no better than guessing on the problem the next learner will see. The next learner has to use some other structure. The assumption that did the work is $\alpha_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$ together with a weighted error ϵ_t computed

from weights that sum to 1. Other weight updates can also produce a useful ensemble. This one has the further merit, shown below, of being the stagewise step for exponential loss.

Example 13.4 (Three rounds of AdaBoost with stumps)**. Question.** Run AdaBoost on $x = 1, \dots, 10$ with labels $y = (+,+,+,-,-,-,+,+,+,-)$, using decision stumps (a threshold on x and a sign) as learners.

Solution. Initially every $w_{1,i} = 0.1$. *Round 1.* The best stump predicts $+1$ for $x < 3.5$ and -1 otherwise; it errs on $x = 7, 8, 9$, so $\epsilon_1 = 0.3$ and $\alpha_1 = \frac{1}{2}\ln(0.7/0.3) = 0.4236$. Correct points have their weight multiplied by $\mathrm{e}^{-\alpha_1} = 0.655$ and wrong points by $\mathrm{e}^{\alpha_1} = 1.528$; with $Z_1 = 2\sqrt{0.3 \times 0.7} = 0.9165$ the new weights are $1/14 = 0.0714$ and $1/6 = 0.1667$. *Round 2.* The best stump on the reweighted data predicts $+1$ for $x < 9.5$ and -1 otherwise; it errs on $x = 4, 5, 6$ (weight $3/14$), so $\epsilon_2 = 0.2143$, $\alpha_2 = 0.6496$. *Round 3.* The stump that predicts -1 for $x < 6.5$ and $+1$ otherwise errs on $x = 1, 2, 3$ and $x = 10$, so $\epsilon_3 = 0.1818$, $\alpha_3 = 0.7520$.

x_i	y_i	$w_{1,i}$	f_1	$w_{2,i}$	f_2	$w_{3,i}$	f_3	$\sum_t \alpha_t f_t$	sign
$1, 2, 3$	$+$	0.100	$+$	0.0714	$+$	0.0455	$-$	0.321	$+$
$4, 5, 6$	$-$	0.100	$-$	0.0714	$+$	0.1667	$-$	-0.526	$-$
$7, 8, 9$	$+$	0.100	$-$	0.1667	$+$	0.1061	$+$	0.978	$+$
10	$-$	0.100	$-$	0.0714	$-$	0.0455	$+$	-0.321	$-$

(Weights are listed per observation in each group and are the weights *used* in that round.) After three rounds the ensemble $f = \mathrm{sign}(0.4236 f_1 + 0.6496 f_2 + 0.7520 f_3)$ classifies all ten training points correctly, although no single stump gets more than seven right. The first learner is wrong on $7, 8, 9$; the second corrects those but is wrong on $4, 5, 6$, which the third corrects, and so on.

13.3.2 AdaBoost as exponential-loss minimisation

The factor $\epsilon_t/(1-\epsilon_t)$ was a choice in the original algorithm, not a consequence that the first papers derived from a loss. Friedman et al. (2000) supplied the loss. The population criterion is the *exponential loss* $J(f) = \mathbb{E}_{Y|\boldsymbol{x}}(\mathrm{e}^{-Yf(\boldsymbol{x})})$, where $(\boldsymbol{X}, Y)$ is a fresh draw and $f(\boldsymbol{x})$ is a real-valued score, positive when the classifier leans toward class $+1$. Exponential loss charges e^{-yf}. A negative margin is a mistake, and a large negative margin is punished exponentially, while a large positive margin is almost free. That emphasis on the worst mistakes is why the loss matches a procedure that keeps raising the weights of the cases it gets wrong. With $\tau_1(\boldsymbol{x}) = \mathbb{P}(Y = 1 \mid \boldsymbol{x})$, the conditional expected loss is an average of two exponentials:

$$\mathbb{E}_{Y|\boldsymbol{x}}(\mathrm{e}^{-Yf(\boldsymbol{x})}) = \tau_1(\boldsymbol{x})\,\mathrm{e}^{-f(\boldsymbol{x})} + (1-\tau_1(\boldsymbol{x}))\mathrm{e}^{f(\boldsymbol{x})}. \tag{13.5}$$

The score we want minimises this conditional loss at every $\boldsymbol{x}$. Differentiate in the scalar $f = f(\boldsymbol{x})$:

$$\frac{\partial}{\partial f}\mathbb{E}(\mathrm{e}^{-Yf}) = -\tau_1 \mathrm{e}^{-f} + (1-\tau_1)\mathrm{e}^{f}.$$

The second derivative is $\tau_1 \mathrm{e}^{-f} + (1-\tau_1)\mathrm{e}^{f} > 0$, so the single stationary point is a minimum. Set the first derivative to zero: $(1-\tau_1)\mathrm{e}^{f^*} = \tau_1 \mathrm{e}^{-f^*}$, hence $\mathrm{e}^{2f^*} = \tau_1/(1-\tau_1)$ and

$$f^*(\boldsymbol{x}) = \tfrac{1}{2}\log\Big(\frac{\tau_1(\boldsymbol{x})}{1-\tau_1(\boldsymbol{x})}\Big). \tag{13.6}$$

The minimiser is half the conditional log odds of class 1. The posterior $\tau_1(\boldsymbol{x})$ did the work: it is the only feature of the conditional distribution of Y that enters the expected exponential loss, because Y takes only two values. When the log odds are 0, one has $\tau_1(\boldsymbol{x}) = 1/2$ and $f^*(\boldsymbol{x}) = 0$. The sign of f^* is therefore the equal-cost Bayes rule, which is why (13.4) classifies by $\mathrm{sign}(f)$.

Exponential loss does not estimate the class densities. It estimates the same posterior comparison that logistic regression writes as a logit, up to the factor 1/2 in (13.6).

Stagewise derivation.

The population minimiser is not a finite combination of weak learners. AdaBoost builds an approximation one learner at a time. Replace the expectation by the training average $\widehat{J}(f) = \frac{1}{n_{\text{train}}} \sum_i \mathrm{e}^{-y_i f(\boldsymbol{x}_i)}$, drop the outer sign, and let the ensemble score be $g(\boldsymbol{x}) = \sum_{t=1}^{T} \alpha_t f_t(\boldsymbol{x})$, with each f_t taking values in $\{-1, +1\}$. After $m-1$ rounds the score is g_{m-1}, and round m adds one term, $g_m = \alpha_m f_m + g_{m-1}$. The strategy is to minimise the training exponential loss in that one new term, holding g_{m-1} fixed. Factor the loss as

$$\widehat{J}(g_m) = \frac{1}{n_{\text{train}}} \sum_i \mathrm{e}^{-y_i g_{m-1}(\boldsymbol{x}_i)} \mathrm{e}^{-y_i \alpha_m f_m(\boldsymbol{x}_i)}.$$

The first factor in each summand, $c_{m,i} = \mathrm{e}^{-y_i g_{m-1}(\boldsymbol{x}_i)}$, does not depend on the new learner. These weights are proportional to the AdaBoost weights $w_{m,i}$: the cases with negative margin under the ensemble so far carry larger $c_{m,i}$. The constant $1/n_{\text{train}}$ does not change the minimiser, so the criterion to be minimised in (f_m, α_m) is

$$\begin{aligned} \sum_i c_{m,i} \mathrm{e}^{-y_i \alpha_m f_m(\boldsymbol{x}_i)} &= \mathrm{e}^{-\alpha_m} \sum_{y_i = f_m(\boldsymbol{x}_i)} c_{m,i} + \mathrm{e}^{\alpha_m} \sum_{y_i \neq f_m(\boldsymbol{x}_i)} c_{m,i} \\ &= \mathrm{e}^{-\alpha_m} \sum_i c_{m,i} + \left(\mathrm{e}^{\alpha_m} - \mathrm{e}^{-\alpha_m}\right) \sum_i c_{m,i} Q[y_i, f_m(\boldsymbol{x}_i)], \end{aligned} \tag{13.7}$$

The rewriting uses $y_i f_m(\boldsymbol{x}_i) = 1$ when f_m is correct and -1 when it is wrong, so $\mathrm{e}^{-y_i \alpha_m f_m}$ equals $\mathrm{e}^{-\alpha_m}$ on the correct cases and e^{α_m} on the incorrect ones. The first sum on the first line equals $\sum_i c_{m,i}$ minus the weight of the incorrect cases. For $\alpha_m > 0$ the coefficient $\mathrm{e}^{\alpha_m} - \mathrm{e}^{-\alpha_m}$ is positive, so the sum is smallest, for that fixed α_m, when the weighted error is smallest. The learner can therefore be chosen first:

$$f_m^* = \arg\min_{f_m} \sum_i c_{m,i} Q[y_i, f_m(\boldsymbol{x}_i)], \tag{13.8}$$

which is step 2(a) of the algorithm: the next learner minimises the weighted training error. With that learner held fixed, differentiate (13.7) in α_m and set the derivative to zero. Write ϵ_m for the minimised weighted error, $\sum_i c_{m,i} Q[y_i, f_m^*(\boldsymbol{x}_i)] / \sum_i c_{m,i}$. The derivative condition is

$$-\mathrm{e}^{-\alpha_m} + \left(\mathrm{e}^{\alpha_m} + \mathrm{e}^{-\alpha_m}\right)\epsilon_m = 0 \implies \mathrm{e}^{\alpha_m}\epsilon_m = \mathrm{e}^{-\alpha_m}(1-\epsilon_m) \implies \mathrm{e}^{2\alpha_m} = \frac{1-\epsilon_m}{\epsilon_m},$$

so the optimal step length is

$$\alpha_m^* = \tfrac{1}{2} \log\Big(\frac{1-\epsilon_m}{\epsilon_m}\Big), \tag{13.9}$$

exactly step 2(b). The ensemble is updated by $g_m = \alpha_m^* f_m^* + g_{m-1}$. The next round's un-normalised weights are the old ones times the new exponential factor, $c_{m+1,i} = c_{m,i} \mathrm{e}^{-\alpha_m^* y_i f_m^*(\boldsymbol{x}_i)}$. Because $-y_i f_m^*(\boldsymbol{x}_i) = 2Q[y_i, f_m^*(\boldsymbol{x}_i)] - 1$,

$$w_{m+1,i} = w_{m,i}\, \mathrm{e}^{\alpha_m^*(2Q[y_i, f_m^*(\boldsymbol{x}_i)]-1)} = w_{m,i}\, \mathrm{e}^{2\alpha_m^* Q[y_i, f_m^*(\boldsymbol{x}_i)]}\, \mathrm{e}^{-\alpha_m^*} = w_{m,i} \Big(\frac{1-\epsilon_m}{\epsilon_m}\Big)^{Q[y_i, f_m^*(\boldsymbol{x}_i)]} \mathrm{e}^{-\alpha_m^*}.$$

The factor $\mathrm{e}^{-\alpha_m^*}$ multiplies every weight and cancels when the weights are normalised. Every misclassified case is therefore multiplied by $(1-\epsilon_m)/\epsilon_m$, and every correctly classified case is left unchanged. After normalisation this is the original AdaBoost update. What the stagewise argument establishes is that the algorithm is greedy minimisation of the training exponential loss, one weak learner at a time. The exponential form of the loss did the work: it factors across rounds into a weight that depends on the margin so far and a factor that depends on the new learner. A different loss would produce a different weight update. The fit is greedy. It does not go back and revise $\alpha_1, \ldots, \alpha_{m-1}$ when round m is added.

Key idea.
AdaBoost adds a weighted learner to a real-valued score. Exponential loss explains the reweighting and the coefficient α_t. Its unrestricted population minimiser is half the conditional log odds; the final sign is a class label and cannot estimate that real number. A finite, restricted, stagewise fit need not attain the population minimiser or produce calibrated probabilities.

13.3.3 Real AdaBoost and extensions

AdaBoost as written asks each learner for a label in $\{-1, +1\}$. A probability $p_t(\boldsymbol{x})$ carries more information than that label, and throwing it away is the step that fails when the learner is already able to say how sure it is. *Real AdaBoost* (Friedman et al., 2000) keeps the probability and converts it to the same half-logit that (13.6) says is the exponential-loss minimiser: $f_t(\boldsymbol{x}) = \frac{1}{2}\log\left(\frac{p_t(\boldsymbol{x})}{1-p_t(\boldsymbol{x})}\right)$. There is no separate vote α_t, because the magnitude of $f_t(\boldsymbol{x})$ already records the confidence at that $\boldsymbol{x}$. The weight update is the exponential-loss factor itself, $w_{t+1,i} = w_{t,i}\mathrm{e}^{-y_i f_t(\boldsymbol{x}_i)}/Z_t$ with $Z_t = \sum_i w_{t,i}\mathrm{e}^{-y_i f_t(\boldsymbol{x}_i)}$. The ensemble adds the scores and then takes the sign:

$$f(\boldsymbol{x}) = \operatorname{sign}\Big(\sum_{t=1}^{T} f_t(\boldsymbol{x})\Big). \tag{13.10}$$

Other loss functions.

A loss-based view lets us change the loss while keeping sequential fitting. With the score convention $p = \sigma(2f)$, logistic negative log-likelihood and exponential loss have the same unrestricted population minimiser (Friedman et al., 2000; Friedman, 2001). Binomial deviance is twice the negative log-likelihood relative to a saturated model; the constant factor changes neither minimiser. The derivation below uses negative log-likelihood, without that factor two.

$$\tau_1(\boldsymbol{x}) = \frac{1}{1+\mathrm{e}^{-2f^*(\boldsymbol{x})}}, \qquad 1-\tau_1(\boldsymbol{x}) = \frac{1}{1+\mathrm{e}^{2f^*(\boldsymbol{x})}}. \tag{13.11}$$

The strategy is to write the Bernoulli log likelihood in the ± 1 coding and substitute $p = 1/(1+\mathrm{e}^{-2f})$. Let $p(\boldsymbol{x}) = \tau_1(\boldsymbol{x})$ and $Y' = (Y+1)/2 \in \{0,1\}$, so Y' is the 0/1 label used in logistic regression. The log likelihood of one case is $Y'\log p + (1-Y')\log(1-p)$. Minus that log likelihood, the deviance contribution, becomes

$$-\ell(Y,p) = Y'\log\left(1+\mathrm{e}^{-2f}\right) + (1-Y')\log\left(1+\mathrm{e}^{2f}\right)$$

after the substitution. If $Y'=1$, then $Y=1$ and the expression is $\log(1+\mathrm{e}^{-2Yf})$. If $Y'=0$, then $Y=-1$ and it is $\log(1+\mathrm{e}^{2f}) = \log(1+\mathrm{e}^{-2Yf})$. Both cases are therefore

$$-\ell(Y,p(\boldsymbol{x})) = \log\left(1+\mathrm{e}^{-2Yf(\boldsymbol{x})}\right). \tag{13.12}$$

The population minimisers agree. Both losses are minimised by half the conditional log odds, so with enough data a procedure that truly minimises either loss estimates the same f^*. The finite-sample paths differ, because the losses grow differently as the margin becomes more negative, and the stagewise weights follow the loss that is being minimised.

AdaBoost reweights the training cases at every round. A weak round, one with ϵ_t near $1/2$, receives a small vote $\alpha_t = \frac{1}{2}\log\frac{1-\epsilon_t}{\epsilon_t}$, and the next round faces a weighted problem on which the previous learner scores exactly $1/2$. Friedman and colleagues identified exponential loss as the criterion this procedure minimises. Figure 13.2 compares exponential, logistic and hinge losses at a common margin, using the normalisations in its R code.

Figure 13.2 plots the losses against the *margin* yf, where $y \in \{-1, 1\}$ is the correct label and f is the score before the threshold at 0 converts it to a label. A positive margin is a correct classification. A negative margin is a mistake, and a loss used for training should charge it. Squared error $(y - f)^2$ equals $(1 - yf)^2$ when $y \in \{-1, +1\}$. It matches a prediction that is itself a label in $\{-1, +1\}$. As a function of a real-valued score it is a parabola in the margin, minimised at $yf = 1$, so a more confident correct call, with $yf > 1$, is penalised more rather than less. Exponential loss grows exponentially as the margin falls below zero, so the fit concentrates on the worst mistakes. When the Bayes error is not near zero, some of those mistakes are unavoidable under any classifier, and concentrating on them fits noise. Binomial deviance grows only linearly in the negative margin, so it is the milder penalty in that regime. Both losses extend to more than two classes. The algorithms that minimise them by adding a weak learner, usually a tree, along the gradient of the training loss are collected under the name *gradient boosting*. Implementations include `xgboost` and `lightgbm`. In those programs the penalty on tree size and the limit on depth are tuning choices, selected by the cross-validation of Chapter 9 or by a validation set. They are not consequences of the population minimiser.

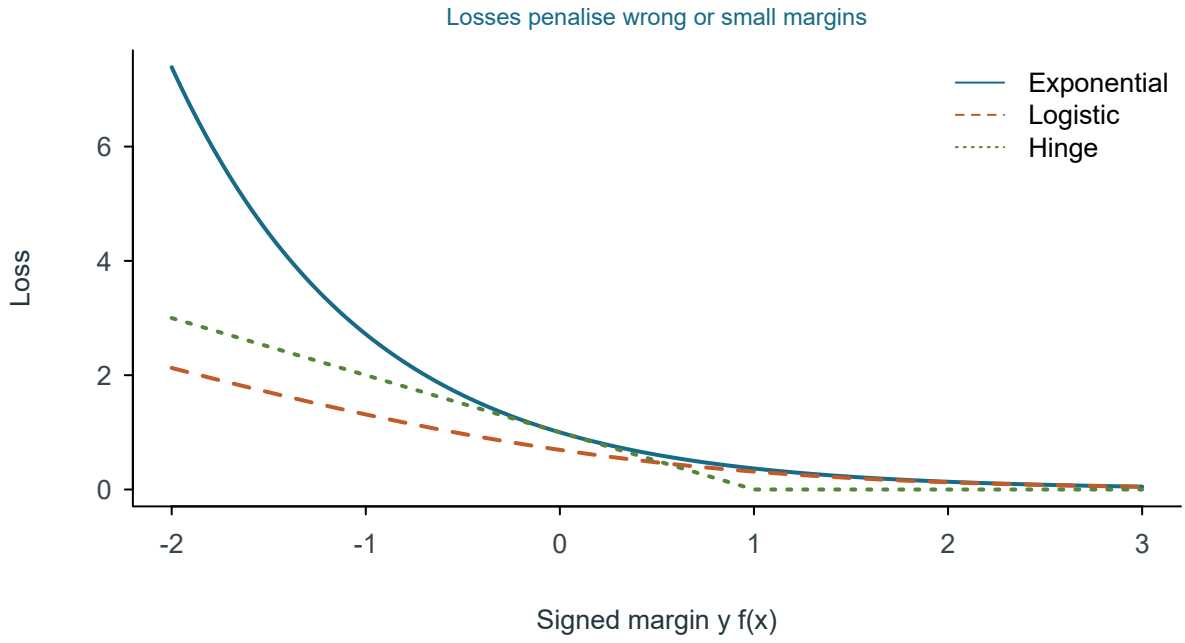


Figure 13.2: Exponential, logistic and hinge loss as functions of the signed margin. The normalisations are those plotted in the R code.

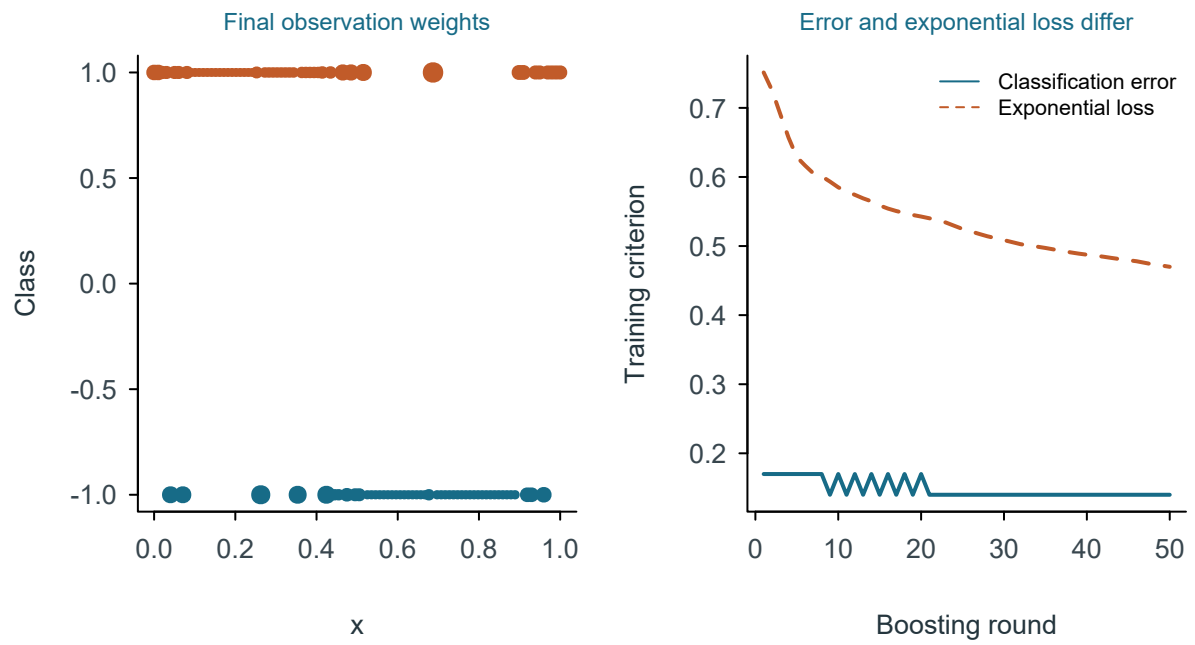


Figure 13.3: A one-dimensional AdaBoost simulation. Point sizes reflect final weights; training classification error and exponential loss are different criteria.

13.4 R laboratory: a forest and its out-of-bag observations

The forest below is one recipe: 500 trees, two predictors offered at each split, grown on the training flowers. Each bootstrap tree omits some of those flowers. The out-of-bag prediction for a flower averages only the trees that did not train on it. The implementation draws the candidate predictors at each split. The test flowers are not used to choose T or k.

```
set.seed(13006)
forest <- randomForest::randomForest(
  Species ~ ., data = train, ntree = 500, mtry = 2, importance = TRUE)
pred <- predict(forest, test)
c(final_OOB_error = tail(forest$err.rate[, "OOB"], 1),
  test_error = mean(pred != test$Species))
#> final_OOB_error      test_error
#>        0.009524        0.088889
round(randomForest::importance(forest, type = 1), 3)
#>              MeanDecreaseAccuracy
#> Sepal.Length                9.695
#> Sepal.Width                 3.914
#> Petal.Length               34.745
#> Petal.Width                32.222
```

Out-of-bag and test errors score different observations using different collections of trees, so they need not agree. The displayed `randomForest::importance(type = 1)` values also use the package's default standardisation of permutation importance; they are not raw percentage-point increases in error. The perturbation measures predictive dependence within this fit, and correlated variables can mask or share importance. It does not identify causal effects (Liaw and Wiener, 2026).

13.5 Further exercises

Exercise 13.1 (The error decomposition)**.** Verify (13.2) numerically for member predictions $f_t(\boldsymbol{x}) = (1, 2, 6)$ and truth $y = 4$. Then derive it in general.

Solution

Exercise 13.2 (Variance of an average)**.** For T predictors with common variance σ^2 and common pairwise correlation ρ, derive $\operatorname{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{T}\sigma^2$. Evaluate it for $\rho = 0.3$, $T = 100$ and for $\rho = 0.8$, $T = 100$. Why do random feature subsets help?

Solution

Exercise 13.3 (Out-of-bag counts)**.** Each observation is excluded from a given bootstrap sample with probability $(1 - 1/n)^n$. For $n = 200$ and $T = 500$ trees, how many trees are out-of-bag for an observation on average, and what is the probability that an observation is in *every* bootstrap sample?

Solution

Exercise 13.4 (One round of AdaBoost)**.** Five observations $x = 1, \dots, 5$ with labels $y = (+1, +1, -1, -1, +1)$ and weights 0.2 each. The stump predicts $+1$ for $x < 2.5$ and -1 otherwise. Compute ϵ_1, α_1, the updated weights, and verify that the new weighted error is 0.5.

Solution

Exercise 13.5 (The exponential-loss minimiser)**.** If $\tau_1(\boldsymbol{x}) = 0.8$, find $f^*(\boldsymbol{x})$ from (13.6) and check that (13.11) returns 0.8. What would the minimiser be if $\tau_1(\boldsymbol{x}) = 0.5$?

Solution

Exercise 13.6 (A tempting wrong turn)**.** A practitioner averages 500 copies of the *same* deterministic classifier trained on the *same* data, hoping for a variance reduction. What does (13.2) (or the variance formula of Exercise 13.2) predict? What are the ways a random forest and AdaBoost avoid this?

Solution

Exercise 13.7 (Why an ensemble might improve a tree)**.** You compare one deep classification tree with a random forest built from many bootstrap samples.

(a) Explain how a random forest produces a prediction and why predictor subsampling can help.

(b) Explain what out-of-bag error estimates and one limitation if you use it repeatedly to select settings.

(c) Contrast the main training idea of boosting with bagging.

Solution

Part IV

Neural networks and support vector machines

Chapter 14

Artificial neurons and networks

Begin with four points, not a large network. They will tell us exactly why another layer can be useful.

The four corners of the unit square already show what one linear score cannot do. Label $(0, 0)$ and $(1, 1)$ as one class and $(0, 1)$ and $(1, 0)$ as the other. A straight cut that places both positive corners on the positive side also keeps at least one negative corner there. AND, OR, and NAND ask only that a single corner be separated, and a hyperplane can do that. XOR asks for a diagonal, and a hyperplane cannot.

An artificial neuron is that hyperplane followed by an activation. It combines a linear score with a fixed function of the score, and it is the right object for the gates that are linearly separable. XOR forces the next object: a hidden layer of such neurons, whose outputs are new coordinates. In those coordinates the same four points can sit so that one further linear score separates them. The construction is an existence claim about features. Some choice of weights represents the target. Finding those weights efficiently, keeping the network small, and generalising from a finite noisy sample are further questions, taken up when the weights are learned in Chapter 15 and when the trained network is used in Chapter 16.

The biological analogy motivates the word neuron, but the statistical object here is a composition of weighted sums and activation functions. We will build that object one layer at a time. Representation, optimisation and generalisation remain separate questions throughout: a network can express a function without our being able to learn it reliably from the available observations.

Source. Neural-network lecture slides (slides 1–36 and 51–56), the two XOR solutions on the supplementary slide, and the references the slides cite: Bishop (2006), Duda et al. (2001), Goodfellow et al. (2016), Hastie et al. (2009), Haykin (1999), and Ripley (1996).

14.1 Biological background and a little history

The calculation we are about to write down was first a picture of a cell. A *neuron* has a cell body (the soma) containing the nucleus, branching input fibres (dendrites), and a long output fibre (the axon) that branches into an axonal arborisation ending in synapses, which connect it to the dendrites of other cells.

Table 14.1 preserves the historical comparison printed on slide 6. Its hardware specifications and biological conversion factors were not sourced sufficiently to verify them. Treat the entries as an illustration of the lecture's analogy, not as a current benchmark or a measured equivalence between neurons, transistors and bits.

McCulloch and Pitts's 1943 paper developed a logical model of neural activity (McCulloch and Pitts, 1943). Later work introduced trainable units and multilayer learning procedures. The

Table 14.1: Historical slide illustration, with unverified numerical entries. Neurons and transistors are different physical objects; these counts do not establish equivalent computing capacity.

	Human brain	2007 PC	2018 PC	Units
Computational units	10^{11}	3×10^{8}	3×10^{9}	neurons / transistors
Storage units	10^{11}	10^{10}	3×10^{10}	neurons / bits RAM
		10^{12}	2×10^{12}	bits disk
Cycle time	10^{-3}	3×10^{-10}	3×10^{-10}	seconds
Bandwidth	10^{14}	10^{10}	5×10^{11}	bits / second

dates below identify selected publications, rather than assigning a single invention date to a broad research field:

- 1943: McCulloch and Pitts publish a logical neuron model with binary output.
- 1958: Rosenblatt (1958) creates the perceptron, a single binary neuron, with a learning algorithm. He also considered multi-layer perceptrons, but without a learning algorithm.
- 1960: Widrow and Hoff (1960) introduce the ADALINE neuron, with a real-valued response, trained by a learning algorithm using stochastic gradient descent.
- 1969: Minsky and Papert (1969) publish *Perceptrons*, which analyses the perceptron and points out its failure to learn non-linearly separable problems such as XOR.
- 1986: Rumelhart et al. (1986) publish the backpropagation algorithm for training multi-layer perceptrons with differentiable activation functions, previously discovered by Werbos (1974). It is largely the chain rule.
- 1998: LeCun and colleagues publish work on gradient-based document recognition, building on earlier convolutional-network research (LeCun et al., 1998).
- 2006: Hinton, Osindero and Teh publish a fast learning algorithm for deep belief networks (Hinton et al., 2006). This is one development in deep learning, not the first successful training of every kind of deep network. Many other networks, algorithms, theoretical contributions, and applications sit between these dates.

The sequence matters because it separates two obstacles. Adding hidden units changes what the model can represent. Developing a learning algorithm changes how we search for its weights. The XOR construction below addresses the first obstacle; backpropagation addresses the second.

14.2 The unit

To turn the picture into a classifier, write the score explicitly. An artificial neuron ("unit") has p inputs $x_1, \ldots, x_p$, weights $w_1, \ldots, w_p$, and a *bias* w_0 (also written β or b). It forms its output in two steps. The split matters: the first step is linear, and every limitation of a single unit is a limitation of that linear step. The second step only shapes the number the linear step has already produced.

Weighting the input.

The *net input* (the weighted input, or activation) is

$$net = \sum_{j=0}^{p} w_j x_j = w_0 + \boldsymbol{w}^\top \boldsymbol{x}, \qquad x_0 \equiv 1, \tag{14.1}$$

where the bias is treated as a weight on an extra input x_0 that is permanently 1. Folding the constant into the inner product keeps every later derivative in the same form: the bias is the weight on a coordinate that happens to be 1.

Forming the output.

The output is $o = f(net)$ for an *activation function* f. Two are used in the course:

$$f(net) = \begin{cases} 0, & net < 0 \\ 1, & net \geq 0 \end{cases} \quad \text{(threshold)}, \qquad f(net) = \frac{1}{1 + \mathrm{e}^{-net}} \quad \text{(logistic sigmoid)}. \tag{14.2}$$

Both are shown in Figure 14.1. The threshold is the McCulloch–Pitts decision. It is the right picture of a hard cut, and it is awkward to train, because a derivative that is zero on each side of the cut gives a gradient method nothing to follow. The sigmoid is a smooth stand-in for the same cut, with $g'(z) = g(z)(1 - g(z))$ (Exercise 14.1). That identity is what the chain rule will multiply by in Chapter 15. The derivative is largest at $z = 0$ and becomes small once $|z|$ is large, which is why a unit that has already saturated is slow to correct.

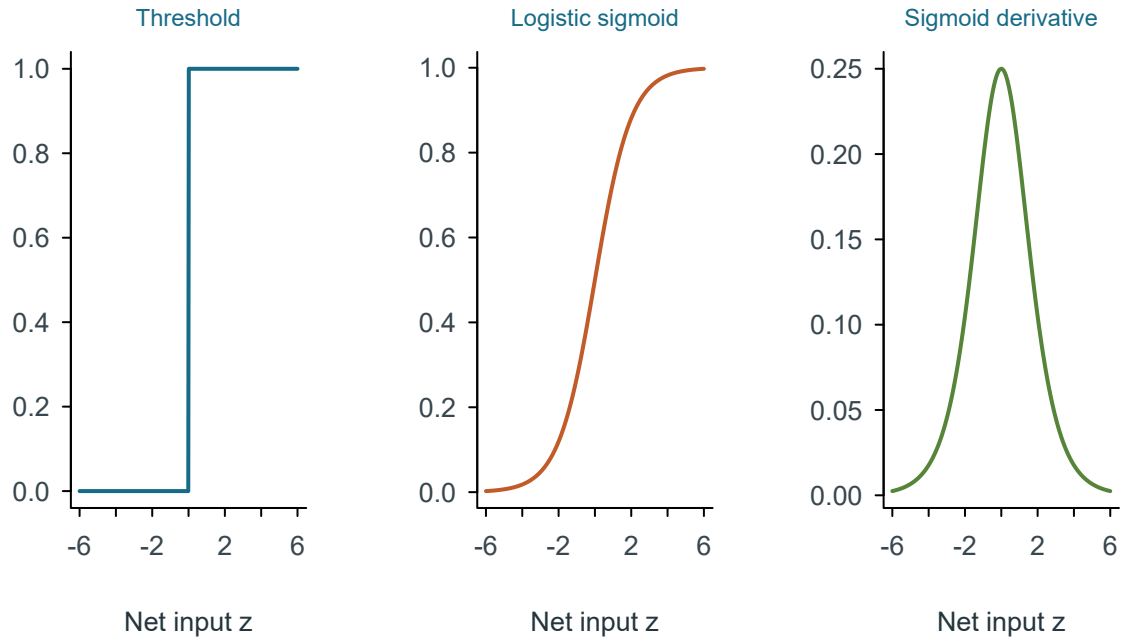


Figure 14.1: Threshold activation, logistic sigmoid and its derivative. Saturation makes the sigmoid derivative small.

Layers of units.

Place m units side by side with the same inputs and different weights. This computes m scalar features, each a function of a linear score. A later layer can combine those features. If they are used only as separate threshold outputs, each output still defines just a half-space in the original coordinates.

Connection with earlier chapters.

A sigmoid unit with output $o = 1/(1 + \mathrm{e}^{-\boldsymbol{w}^\top \boldsymbol{x} - w_0})$ is exactly the logistic regression model (10.4): $o = \widehat{\tau}_1(\boldsymbol{x})$. A threshold unit is the classification rule (10.12). A single unit is a linear classifier of the same kind as LDA and logistic regression. Stacking units, so that one unit receives the outputs of others, is the step those models do not take.

14.3 What can a single unit do?

Consider classification with two inputs x_1, x_2 and one threshold output, 0 or 1. The output changes where the net input changes sign, so the *decision boundary* is the zero set of $w_0 + \boldsymbol{w}^\top \boldsymbol{x}$. The task of any classifier is to partition the input space into classes by such boundaries. For one threshold unit the partition is always a half-space, and the only geometric question left is where the bounding hyperplane sits.

14.3.1 Without a bias

With bias $w_0 = 0$, the boundary is the set where $\sum_j x_j w_j = 0$, which must pass through the origin. That is a genuine restriction: a separator that ought to miss the origin cannot be represented. For $\boldsymbol{w} = (1, -2)^\top$ the boundary is $x_1 - 2x_2 = 0$, the line $x_2 = 0.5x_1$. In Figure 14.2 (left), weights $(0.5, 1)$, $(1, 1)$ and $(2, 1)$ instead give the three lines $x_2 = -x_1/2$, $x_2 = -x_1$ and $x_2 = -2x_1$.

14.3.2 With a bias

The bias is the constant term that frees the decision boundary from the origin. With it, a threshold unit can realise any linear separator. For $\boldsymbol{w} = (1, 1)^\top$ the boundary is $x_1 + x_2 + w_0 = 0$, which is shifted by changing w_0 (Figure 14.2, right). In general the boundary is the hyperplane $\{\boldsymbol{x} : w_0 + \boldsymbol{w}^\top \boldsymbol{x} = 0\}$, which has normal vector $\boldsymbol{w}$, and the unit outputs 1 on the side the normal points to. Section 17.1 derives the distance from a point to this hyperplane; the same normal vector is the object that derivation starts from.

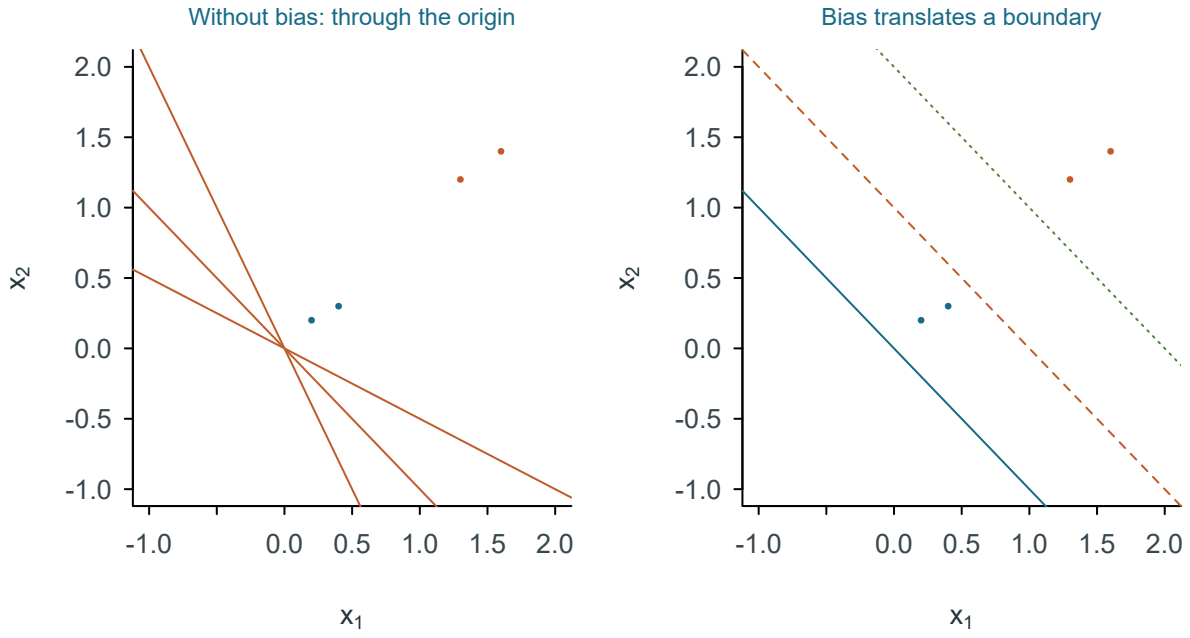


Figure 14.2: Bias allows a threshold boundary to move away from the origin. The panels illustrate geometry, not an optimisation trajectory.

14.4 Learning for a single threshold unit: the perceptron

A family of hyperplanes can separate the same labelled sample, and a threshold unit does not come with normal equations. The threshold is constant on each open half-space, so there is no linear system whose solution is "the" weight vector. Suitable weights have to be built by correction. Watching the decision boundary in the input space makes the corrections visible. The perceptron

rule automates them: it changes the weights only on a misclassified pattern, and it moves them along that pattern so the score on the pattern increases when the label was 1 and decreases when the label was 0.

Algorithm 14.1 (Perceptron learning rule)**.** Let the training data be $(\boldsymbol{x}_i, y_i)$ with $y_i \in \{0, 1\}$ and $\boldsymbol{x}_i$ augmented by $x_{i0} = 1$. Start from any weights $\boldsymbol{w}$ (including the bias). Repeat until no pattern is misclassified: for each pattern i compute $o_i = f(\boldsymbol{w}^\top \boldsymbol{x}_i)$ and set

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + (y_i - o_i)\, \boldsymbol{x}_i. \tag{14.3}$$

If the unit is right, $y_i - o_i = 0$ and nothing changes. If it outputs 0 for a pattern whose label is 1, the pattern is *added* to the weights, which increases $\boldsymbol{w}^\top \boldsymbol{x}_i$ by $\|\boldsymbol{x}_i\|^2$; if it outputs 1 for a label 0, the pattern is *subtracted*. The squared length is the amount of the correction because the added term is $(y_i - o_i)\boldsymbol{x}_i$ and the score changes by $(y_i - o_i)\|\boldsymbol{x}_i\|^2$. Rosenblatt's perceptron convergence theorem says that if the classes are linearly separable the rule reaches a separating set of weights in a finite number of corrections; if they are not, it never stops. Linear separability is the assumption that does the work. The rule returns a separator, when one exists, and it has no further criterion: any separating line ends the algorithm. The support vector machine of Chapter 17 adds a margin, which is exactly the preference this rule lacks.

Example 14.2 (Watching a perceptron learn)**.** A unit with two inputs, started at $\boldsymbol{w} = (2, 1)^\top$ and bias -1, is trained on five points: class 1 ("+") at $(1, 1)$, $(0, 0.5)$, $(-0.5, -0.5)$ and class 0 ("$\circ$") at $(1, 0)$, $(0.5, -1)$. At each step one misclassified point is corrected by (14.3):

Step	Misclassified point	y	o	Update	New (w_1, w_2), w_0
0					$(2,\ 1)$, -1
1	$(0, 0.5)$	1	0	add $(0, 0.5)$, w_0+1	$(2,\ 1.5)$, 0
2	$(1, 0)$	0	1	subtract $(1, 0)$, w_0−1	$(1,\ 1.5)$, -1
3	$(0, 0.5)$	1	0	add $(0, 0.5)$, w_0+1	$(1,\ 2)$, 0
4	$(1, 0)$	0	1	subtract $(1, 0)$, w_0−1	$(0,\ 2)$, -1
5	$(-0.5, -0.5)$	1	0	add $(-0.5, -0.5)$, w_0+1	$(-0.5,\ 1.5)$, 0
6	$(-0.5, -0.5)$	1	0	add $(-0.5, -0.5)$, w_0+1	$(-1,\ 1)$, 1

Each row can be checked by hand: for step 1, $net = -1 + 0 \cdot 2 + 0.5 \cdot 1 = -0.5 < 0$, so $o = 0 \neq 1$. After step 6, $(1, 0)$ has net input exactly 0 and so is still (just) misclassified as class 1 under the convention $f(0) = 1$. Continuing the same rule, cycling through the five points in the order listed, converges after three further corrections to $\boldsymbol{w} = (-2, 2)^\top$, $w_0 = 0$, i.e. the boundary $x_1 = x_2$, with $(1, 1)$ and $(-0.5, -0.5)$ lying on it (net input 0, classified as 1 by the convention). Because the rule stops as soon as every point is on the correct side, it returns *a* separating line and not the best one: it has no notion of a margin. The support vector machine of Chapter 17 adds exactly that. Figure 14.3 shows the boundary after each step.

14.5 The XOR problem

Return to the four corners. A single threshold unit separates two classes by a hyperplane, so the problems it can solve are the linearly separable ones. AND, OR, and NAND are linearly separable: one hyperplane does each of them. OR is class 1 except where both inputs are off, and a hyperplane can put that single point on the other side. AND is the opposite corner. XOR is the diagonal. The positive class occupies $(0, 1)$ and $(1, 0)$, and the other class occupies $(0, 0)$ and $(1, 1)$.

The standard example is XOR, the function $g : \{0, 1\} \times \{0, 1\} \to \{0, 1\}$ with desired (x_1, x_2, y) values $(0, 0, 0)$, $(0, 1, 1)$, $(1, 0, 1)$, $(1, 1, 0)$. The proof that no threshold unit computes it is a comparison of four linear inequalities. The strategy is to add the two inequalities that come from

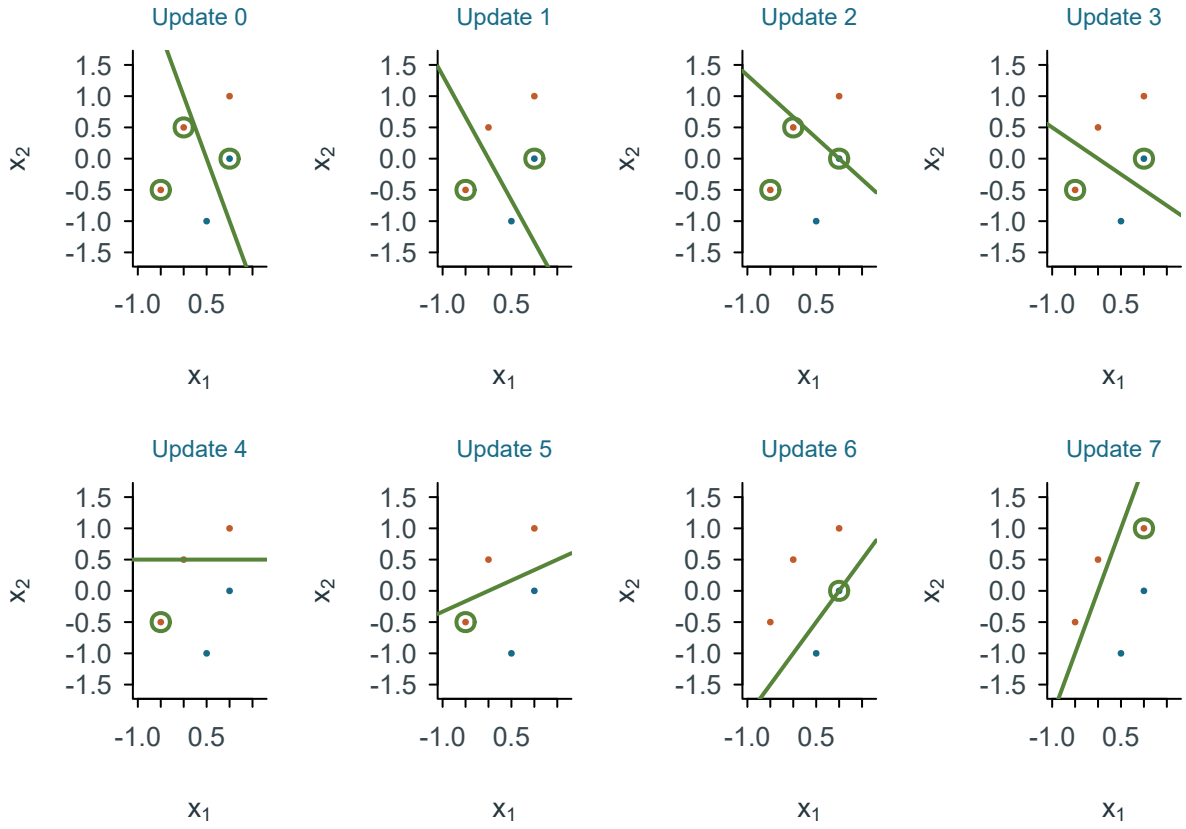


Figure 14.3: The eight specified perceptron states from the worked example. Rings identify currently misclassified inputs.

the positive corners and compare the sum with the two inequalities that come from the negative corners.

Proposition 14.3 (A single threshold unit cannot compute XOR)**.** No choice of w_0, w_1, w_2 makes $f(w_0 + w_1x_1 + w_2x_2)$ equal to XOR on $\{0,1\}^2$.

Proof. The four patterns require $w_0 < 0$; $w_0 + w_2 \geq 0$; $w_0 + w_1 \geq 0$; and $w_0 + w_1 + w_2 < 0$. Adding the middle two gives $2w_0 + w_1 + w_2 \geq 0$. But the last requirement and $w_0 < 0$ give $2w_0 + w_1 + w_2 = w_0 + (w_0 + w_1 + w_2) < 0$, a contradiction.

□

Adding the two middle inequalities produces a quantity that the outer two inequalities force to be negative. The four requirements cannot hold together, so no choice of w_0, w_1, w_2 reproduces XOR. This is the failure that Minsky and Papert (1969) pointed out for the perceptron. These four points, with this labelling, are one configuration a linear classifier leaves unshattered. The VC dimension of half-spaces in the plane is 3. Chapter 17 records that count: it uses a general-position argument, and a single failed labelling is the witness that these four points are not a shattered set.

14.6 Multi-layer networks

The proposition forces a change of coordinates. If no hyperplane separates the classes in the input space, the useful next step is to build new coordinates in which a hyperplane does. A *hidden layer* is that change of coordinates. In a multi-layer network the activation (node output) flows from input to output through a set of intermediate *hidden* (or *latent*) nodes $h_1, \ldots, h_l$, whose activations are decided internally by the network and depend on the weights between the input and hidden nodes. With l hidden units and m outputs:

$$u_q = w_{0q} + \sum_{j=1}^{p} w_{jq}x_j, \qquad h_q = g(u_q), \qquad q = 1, \ldots, l, \tag{14.4}$$

$$z_k = w_{0k} + \sum_{q=1}^{l} w_{qk}h_q, \qquad o_k = g(z_k), \qquad k = 1, \ldots, m. \tag{14.5}$$

(Chapter 15 adds the pattern index i and fixes the notation.)

How does this work?

The input–hidden weights map $\boldsymbol{x}$ to the vector of hidden activations. The output unit then solves a linear problem in that new vector. The hope is to choose the weights so that the problem at the hidden–output layer is linearly separable. Seen back in the original input space, the set where the output unit fires is bounded by curves, because each hidden unit folds the space before the output unit cuts it. One example is the predicted presence and absence of tsetse flies in Zimbabwe from climate and vegetation data, using a network with 24 hidden neurons (Williams et al., 1994): the boundary between "present" and "absent" regions on the map is an irregular curve that no linear classifier could draw.

One construction of XOR, written out below, places an OR gate and an AND gate in the hidden layer and combines them so that the output fires for OR and stays off for AND. A wider construction copies each input through a hidden unit and computes AND in a third unit. Copying an input through a hidden unit does no new work: that unit can be removed and the connection taken directly from the input, skipping the copy. The same reason leads one to standardise before k-means (Chapter 4): the representation has to match the method. A coordinate that repeats an input does not enlarge the set of functions the output unit can compute.

14.6.1 Two solutions for XOR

Two constructions with threshold units and integer weights show that a hidden layer is enough for this particular function. Weights are labelled on connections, the bias is the number inside a unit, and units 3–6 are numbered above them (h_3, h_4, h_5 hidden, $o =$ unit 6).

First solution: copy and subtract AND.

Nodes 3 and 5 copy x_1 and x_2 through, and node 4 computes $x_1 \wedge x_2$:

$$\begin{aligned} h_3 &= f(1 \cdot x_1 - 0.5) = x_1, \\ h_4 &= f(1 \cdot x_1 + 1 \cdot x_2 - 1.5) = x_1 \wedge x_2, \\ h_5 &= f(1 \cdot x_2 - 0.5) = x_2, \\ o &= f(1 \cdot h_3 - 2\,h_4 + 1 \cdot h_5 - 0.5). \end{aligned}$$

The output is 1 if x_1 OR x_2, and it is 0 if x_1 AND x_2. The four patterns confirm the arithmetic:

x_1	x_2	h_3	h_4	h_5	net input of o	o
0	0	0	0	0	-0.5	0
0	1	0	0	1	$+0.5$	1
1	0	1	0	0	$+0.5$	1
1	1	1	1	1	$1 - 2 + 1 - 0.5 = -0.5$	0

Second construction.

Node 3 does AND and node 4 does OR, and node 6 calculates OR minus AND. The threshold lets only XOR through:

$$\begin{aligned} h_3 &= f(2x_1 + 2x_2 - 3) = x_1 \wedge x_2, \\ h_4 &= f(2x_1 + 2x_2 - 1) = x_1 \vee x_2, \\ o &= f(-2h_3 + 2h_4 - 1). \end{aligned}$$

x_1	x_2	net of h_3	net of h_4	(h_3, h_4)	net of o	o
0	0	-3	-1	$(0,0)$	-1	0
0	1	-1	$+1$	$(0,1)$	$+1$	1
1	0	-1	$+1$	$(0,1)$	$+1$	1
1	1	$+1$	$+3$	$(1,1)$	-1	0

The representation at the hidden layer.

In the second solution the hidden layer maps the four input points to three points of the (h_3, h_4) plane: $(0,0)$ for the input $(0,0)$, $(1,1)$ for $(1,1)$, and $(0,1)$ for both $(0,1)$ and $(1,0)$. In the hidden space the problem is linearly separable: the output unit's boundary $h_4 = h_3 + 0.5$ puts $(0,1)$ above it and the other two points below (Figure 14.4). The hidden units have re-described the inputs by two logical features, "at least one is on" and "both are on", and XOR means "at least one is on, but not both". The first solution does the same with a three-dimensional representation in which two of the coordinates are the raw inputs. Neither solution is unique: weights can be rescaled by any common positive factor, and the network can be permuted by relabelling hidden units. What the constructions establish is existence of a representation. A learning rule is a separate question. The threshold units used here have a derivative that is zero on each open half-space, so a gradient method receives no signal from them.

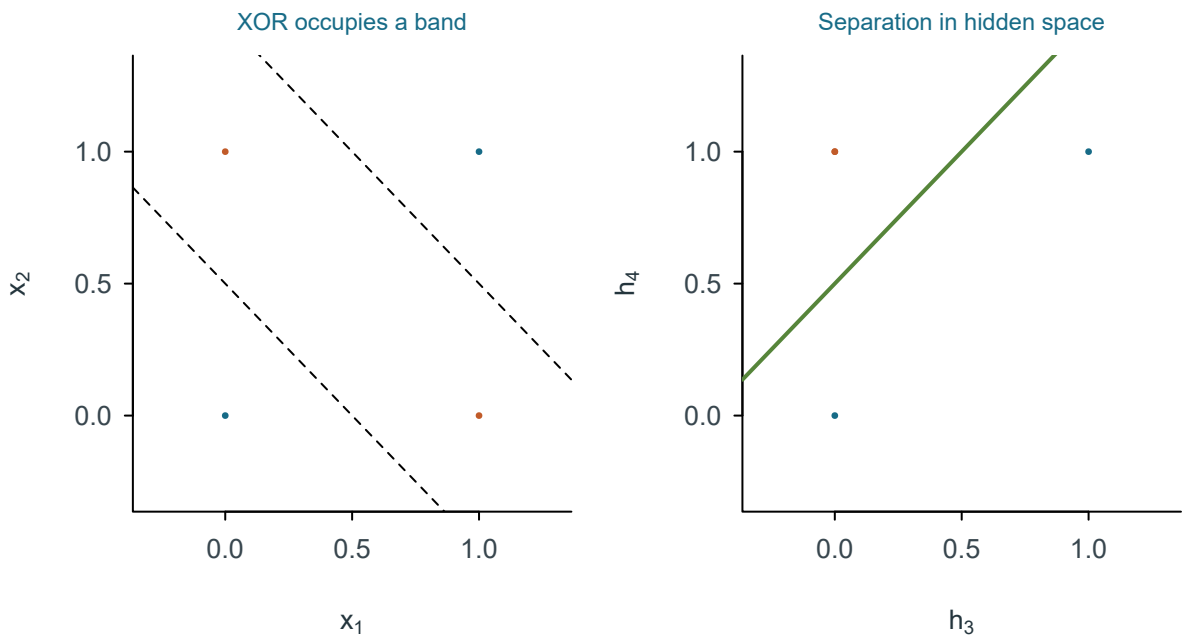


Figure 14.4: XOR is not linearly separable in input space; OR and AND hidden features make a linear output separator possible.

14.7 Universal approximation

XOR is a finite construction. A precise approximation theorem asks about an entire function class and states how error is measured. For a continuous target f on $[0,1]^p$ and any $\varepsilon > 0$, there is a finite sum

$$F(\boldsymbol{x}) = \sum_{q=1}^{l} a_q \, \sigma(\boldsymbol{w}_q^T \boldsymbol{x} + b_q)$$

with a continuous sigmoidal activation such that $\sup_{\boldsymbol{x}\in[0,1]^p} |F(\boldsymbol{x}) - f(\boldsymbol{x})| < \varepsilon$ (Cybenko, 1989, Theorem 2). The output here is a linear combination, not a bounded sigmoid forced to represent arbitrary real values. Results for measurable targets use appropriate weaker notions of approximation; they do not assert uniform approximation of every discontinuous function by continuous networks.

Remark 14.4 (What the theorem does and does not say). The theorem says that suitable weights and a finite width exist for each continuous target and accuracy. It supplies neither a training algorithm nor a sample-size guarantee. Approximation error concerns the best function available in the model class; optimisation error concerns finding it; statistical error concerns learning from finite observations. These can behave differently.

The remark records the limits of that existence result. One hidden layer can approximate a wide class of functions arbitrarily closely. The theorem is silent on the number of units and on whether one hidden layer is an efficient architecture. A deeper network can be smaller for the same approximation. The tradeoff among target complexity, sample size, and width is the over-fitting problem of Chapter 16.

14.8 What needs to be chosen

A representation theorem leaves a practitioner with a list. To use a neural network one must choose:

- whether to have a hidden layer at all (a network is generally more capable with one);
- how many hidden layers;
- any other structural choices;
- how many hidden units in each layer;
- values for each of the weights and biases to solve the problem of interest.

Choose an architecture by the function it must represent and the evidence available for fitting it. More hidden units enlarge a nested model class, so its ideal minimum training loss cannot increase. An actual optimisation run can still finish worse. Compare architecture and regularisation choices using validation within the training process, then assess the selected procedure separately.

14.9 Notation for the rest of Part IV

The next two chapters differentiate these units and then train them, so the names of the weights need to stay fixed. The following conventions are used here and in Chapters 15 and 16.

Symbol	Meaning
$i = 1, \ldots, n$	observations (patterns); n_{tr} training patterns, n_{te} test patterns ($n_{\text{tr}} + n_{\text{te}} = n$)
$j = 1, \ldots, p$	inputs (attributes)
$k = 1, \ldots, m$	outputs (responses)
$q = 1, \ldots, l$	hidden units (a lowercase L)
x_{ij}, y_{ik}	input j and correct output k of pattern i
h_{iq}, o_{ik}	hidden unit q and output unit k for pattern i; $o_{ik} = f_{\text{network},k}(\boldsymbol{x}_i)$
w_{jq}, w_{qk}	weight from input j to hidden unit q; from hidden unit q to output k
$u_{iq} = \sum_{j=0}^{p} w_{jq} x_{ij}$	activation (weighted input) of hidden unit q for pattern i
$z_{ik} = \sum_{q=0}^{l} w_{qk} h_{iq}$	activation of output unit k for pattern i
$g(\cdot)$	unit activation function, e.g. the sigmoid $g(x) = 1/(1 + \mathrm{e}^{-x})$
w_{0b}, w_{0c}	bias weight of hidden unit b, of output unit c (extra input x_{i0} and extra hidden unit h_{i0} fixed at $+1$)

Key idea.
XOR shows why changing the representation can matter: two hidden logical features turn an inseparable input pattern into a separable one. The approximation theorem extends representational capacity to continuous functions on a compact domain. Neither argument tells us that a particular training run will find useful features or generalise to new data.

14.10 R laboratory: verify XOR before training anything

A hand-built network is a specification one can check before trusting an optimiser. The first hidden unit detects OR, the second detects AND, and the output keeps the cases where OR is on and AND is off. That is the second construction of this chapter, written as thresholds on row sums.

```
X <- rbind(c(0, 0), c(0, 1), c(1, 0), c(1, 1))
h_or <- as.integer(rowSums(X) >= 0.5)
h_and <- as.integer(rowSums(X) >= 1.5)
out <- as.integer(h_or - 2 * h_and >= 0.5)
data.frame(X, OR = h_or, AND = h_and, XOR = out)
#>   X1 X2 OR AND XOR
#> 1  0  0  0   0   0
#> 2  0  1  1   0   1
#> 3  1  0  1   0   1
#> 4  1  1  1   1   0
stopifnot(identical(out, c(0L, 1L, 1L, 0L)))
```

The four rows match XOR, so a hidden layer of two nonlinear units is sufficient for this function. The inequality argument earlier in the chapter is the matching necessity result for a single affine threshold. Nothing here was optimised: the weights were chosen so that the hidden units would be the OR and AND features, and the check only confirms that those features do what the arithmetic said they would.

14.11 Further exercises

Exercise 14.1 (Derivative of the sigmoid)**.** Show that $g(z) = 1/(1 + \mathrm{e}^{-z})$ satisfies $g'(z) = g(z)(1 - g(z)) = \mathrm{e}^{-z} g(z)^2$. Find its maximum value and where it occurs.

Solution

Exercise 14.2 (Logic gates)**.** Find weights and a bias for a single threshold unit that computes (a) AND, (b) OR, (c) NAND, of two binary inputs. Why can a single unit not compute XNOR (the complement of XOR)?

Solution

Exercise 14.3 (A decision boundary)**.** A threshold unit has $\boldsymbol{w} = (2, -1)^\top$ and bias $w_0 = -1$. Sketch or describe the boundary, state which side gives output 1, and classify the points $(1, 0)$, $(0, 0)$, $(1, 2)$. What is the distance from the origin to the boundary?

Solution

Exercise 14.4 (Perceptron by hand)**.** Apply the perceptron rule to the three points $(0, 0)$ (class 0), $(1, 0)$ (class 1), $(0, 1)$ (class 1), starting from $\boldsymbol{w} = (0, 0)$, bias 0, with the order of presentation $(0, 0)$, $(1, 0)$, $(0, 1)$ repeated. Give the weights after each correction until convergence.

Solution

Exercise 14.5 (An XOR network of your own)**.** Design a network with two hidden threshold units that computes XOR using different weights from the two solutions in the text, for example by using NAND and OR in the hidden layer. Give the truth table.

Solution

Exercise 14.6 (Linear hidden units)**.** Suppose every hidden unit has the *identity* activation $g(u) = u$. Show that the network is equivalent to a single-layer network, and that therefore it cannot compute XOR with a threshold output.

Solution

Exercise 14.7 (Why XOR needs an extra layer)**.** The four inputs $(0, 0), (1, 0), (0, 1), (1, 1)$ have labels $0, 1, 1, 0$, respectively. A threshold unit outputs 1 when its net input is nonnegative.

(a) Explain geometrically why a single linear threshold unit cannot classify all four inputs correctly.

(b) Construct a two-hidden-unit threshold network for these labels, specifying all weights and biases.

(c) Explain what would be lost by replacing all the activations in a multilayer network by linear functions.

Solution

Chapter 15

Learning in neural networks: gradient descent and backpropagation

We know how to compute a network output once its weights are given. Learning reverses the practical task: change the weights to reduce a loss. With nonlinear hidden units, the gradient equations generally do not reduce to a solvable linear system. We therefore need a way to compute derivatives efficiently and a separate rule for using them to update the weights.

Backpropagation computes derivatives by applying the chain rule through the network in reverse order. Gradient descent uses those derivatives to take a step. Keeping the two ideas separate helps when we change optimisers: the derivative calculation can stay the same. Store the forward values and use the same weights throughout one backward pass (Goodfellow et al., 2016, Sections 6.5 and 8.3).

At a point with nonzero gradient, a sufficiently small step in its negative direction decreases a differentiable objective. This is a local statement. It does not say that a chosen fixed step size always works, or that the eventual limit is a global minimum. Saturation can make the gradient tiny; a large step can overshoot or increase the loss.

This chapter defines the error, derives gradient descent for a single sigmoid layer and then for a network with a hidden layer, works a complete numerical example including a numerical check of the gradient, and examines how learning behaves on XOR. By the end you should be able to say what the learning error is measuring, how backpropagation builds each weight's step, what a learning rate does to that step, where the procedure can stall, and how the hidden activations move while XOR is being learned.

Source. Neural-network lecture slides, "Learning in neural networks" through "Backpropagation implementation" (slides 37–61).

15.1 The error function

For classification the aim is a low error rate, or a related loss; for regression, a small squared error. The performance measure used with the networks in this chapter is the sum of squared errors on the training data. Squared error is differentiable, it treats a numeric target the same way whether the target is a measurement or a class code, and any constant multiplier is absorbed into the learning rate below, so the fitted direction does not depend on writing $\frac{1}{2}$ in front of the sum. This chapter omits that factor. The omission is why a 2 appears in every gradient.

$$E = \sum_{i=1}^{n_{\text{tr}}} \sum_{k=1}^{m} E_{ik} = \sum_{i=1}^{n_{\text{tr}}} \sum_{k=1}^{m} \left[y_{ik} - f_{\text{network},k}(\boldsymbol{x}_i) \right]^2 = \sum_{i=1}^{n_{\text{tr}}} \sum_{k=1}^{m} \left[y_{ik} - o_{ik} \right]^2, \tag{15.1}$$

where y_{ik} is the kth correct output for pattern i and o_{ik} the kth network output. For comparisons it can be useful to divide by mn_{tr} to give the mean squared error, which does not affect the optimisation. The loss treats an error of a given size on each output as equally important. If that is the wrong weighting, the terms can be scaled by output.

For targets in $\{0, 1\}$, squared error on a single output equals absolute error when the prediction is also in $\{0, 1\}$, because the nonzero difference is 1. Equation (15.1) is wider than that special case. With a sigmoid the output lies strictly between 0 and 1, and $(y_{ik} - o_{ik})^2$ is not a count of mistakes.

The free parameters are the weights and biases, and possibly the number of hidden units. The bias weights are connected to every hidden and output unit; each is equivalent to a weight from an extra input x_0 that is always 1.

Fix the architecture while differentiating; its size is a separate choice. The weights form a vector in parameter space, and ∇E collects one partial derivative per coordinate. In the Euclidean geometry of that space, $-\nabla E$ is the direction of steepest local decrease for a fixed step length. If only one coordinate is changed, move opposite the sign of that coordinate's derivative.

15.2 Gradient descent

Algorithm 15.1 (Basic gradient-descent learning)**.**

1. Choose a random set of initial weights, for example from $U[-0.5, 0.5]$.
2. Repeat:
   - Run every input training pattern through the network and record the output values of the hidden and output units.
   - Calculate E for the current weights.
   - Update every weight by

$$w' = w - \varepsilon \frac{\partial E}{\partial w} = w + \Delta w, \tag{15.2}$$

   until a stopping condition holds: the performance is acceptable, or the weights are barely changing.

The *learning rate* $\varepsilon > 0$ is the step size along minus the gradient. The value 0.1 is an example of a step size. It is not a distinguished constant.

In one dimension the error is a curve in the single weight. The update subtracts ε times the derivative, so the step length is ε times the slope. A steep region takes a longer step, and a large ε can jump over a valley. A flat region, where the derivative is near zero, barely moves. A start in one basin stays in that basin under a small enough step. Momentum adds a fraction of the previous step and can cross a flat ridge. A mini-batch replaces the full gradient by an average over a random subset and so adds noise to the gradient. A line search spends several evaluations of the loss on one step. A second-order step uses the Hessian and costs more per iteration.

A randomly sampled mini-batch can give an unbiased estimate of the gradient of the mean loss. That expectation, with the current weights held fixed, is the relevant property; the angle of one noisy draw is not a convergence argument. Its variance and the learning-rate schedule still affect the trajectory (Goodfellow et al., 2016, Section 8.1.3).

One pass through all the training patterns is an *epoch* (NETtalk is the example in Section 16.6.4).

Numerical derivative.

For checking a gradient, perturb one weight at a time and compare the analytical derivative with $[E(w+\delta) - E(w-\delta)]/(2\delta)$. A very large δ causes approximation error; a very small one causes cancellation in floating-point arithmetic. Try a small range of steps and compare absolute as well as relative discrepancies, especially near zero derivatives. This check is expensive for training but useful for finding a missing sign or factor.

Why the sign of the step matters.

The strategy is a first-order expansion. To first order, $E(w+\Delta w) \approx E(w) + \frac{\partial E}{\partial w}\Delta w$, which is decreased most for a given step length by taking Δw proportional to $-\partial E/\partial w$. What this establishes is a descent direction, and the assumption that does the work is that the step is small enough for the remainder to stay smaller than the linear term. Section 15.6 shows what the same update does when that assumption fails.

15.3 A single layer of sigmoid units

Consider a network with only one layer of weights and no hidden units, with sigmoid outputs $o_{ik} = g(z_{ik})$, $z_{ik} = \sum_{j=0}^{p} w_{jk}x_{ij}$. The derivative below is the chain rule applied to one weight w_{ab}, from input a to output b. Differentiate the sum of squares, keep the single output that depends on this weight, and multiply by the sigmoid slope and by $\partial z_{ib}/\partial w_{ab} = x_{ia}$.

$$\frac{\partial E}{\partial w_{ab}} = \sum_{i=1}^{n_{\text{tr}}} \sum_{k=1}^{m} -2[y_{ik} - o_{ik}] \frac{\partial o_{ik}}{\partial w_{ab}}. \tag{15.3}$$

Only o_{ib} depends on w_{ab}, which is why the inner sum over outputs collapses to the term $k = b$. The sigmoid derivative $g'(z) = \mathrm{e}^{-z} g(z)^2$ is Exercise 14.1, and $\partial z_{ib}/\partial w_{ab} = x_{ia}$, so

$$\frac{\partial o_{ib}}{\partial w_{ab}} = g'(z_{ib})\, x_{ia} = \mathrm{e}^{-z_{ib}} o_{ib}^2\, x_{ia}, \tag{15.4}$$

and the gradient step is minus ε times that derivative:

$$\frac{\partial E}{\partial w_{ab}} = -2 \sum_{i=1}^{n_{\text{tr}}} [y_{ib} - o_{ib}]\, \mathrm{e}^{-z_{ib}}\, o_{ib}^2\, x_{ia}, \qquad \Delta w_{ab} = 2\varepsilon \sum_{i=1}^{n_{\text{tr}}} [y_{ib} - o_{ib}]\, \mathrm{e}^{-z_{ib}}\, o_{ib}^2\, x_{ia}. \tag{15.5}$$

Including the bias weights as w_{0k} ($j = 0$) means $x_{i0} = 1$ for all patterns. What the three displays establish is $\partial E/\partial w_{ab}$ for squared error without a leading $\frac{1}{2}$, under a logistic activation. The factor 2 is the derivative of a square. The factor $\mathrm{e}^{-z_{ib}}$ is the numerator of g'. The source note that follows records a version of this formula in which that numerator was dropped, and the numerical check later in the chapter is how that error shows up.

Dropped factor in the single-layer derivative.
The slides' single-layer derivation (slides 47–48) writes $\partial o/\partial w_{ab} = x_{ia}/(1 + \mathrm{e}^{-\sum_j w_{jk}x_{ij}})^2$ and concludes $\Delta w_{ab} = 2\varepsilon \sum_i [y_{ib} - o_{ib}]\, o_{ib}^2\, x_{ia}$, with no factor $\mathrm{e}^{-z_{ib}}$. The derivative of the logistic function is $\mathrm{e}^{-z}/(1+\mathrm{e}^{-z})^2$, so the numerator e^{-z} has been dropped; the correct result is (15.5). The hidden–output update on slide 59 correctly contains $\mathrm{e}^{-z_{ic}}$, and the two agree once it is restored. The error matters numerically: in Example 15.4 below the formula without the factor overstates a gradient component by $\mathrm{e}^{z} \approx 1.35$ and fails the numerical gradient check.

Remark 15.2 (Link with logistic regression). For one sigmoid output, squared-error and Bernoulli likelihood fitting are different optimisation problems. The squared-error gradient includes $o_i(1-o_i)$, which becomes small at saturation. The gradient of negative log-likelihood is $\sum_i (o_i - y_i)x_{ia}$, without that extra factor. The score of the positive log-likelihood has the opposite sign. Distinguish the gradient from the descent update.

15.4 Backpropagation

For a network with one hidden layer, learning has three phases: a *feed-forward* phase, an *error calculation*, and a *weight modification*. The algebra that follows is the chain rule written along those phases. The object to keep is the update for one weight.

1. Feed forward.

For each pattern i, with $g(u) = 1/(1+\mathrm{e}^{-u})$:

$$u_{iq} = \sum_{j=0}^{p} w_{jq} x_{ij}, \qquad h_{iq} = g(u_{iq}), \qquad z_{ik} = \sum_{q=0}^{l} w_{qk} h_{iq}, \qquad o_{ik} = g(z_{ik}), \tag{15.6}$$

with $x_{i0} = h_{i0} = 1$ (bias).

2. Error calculation.

$E_{ik} = [y_{ik} - o_{ik}]^2$ and $E = \sum_i \sum_k E_{ik}$.

3. Weight modification.

$w = w_{\text{old}} + \Delta w$ with $\Delta w = -\varepsilon\, \partial E/\partial w$.

Start at the loss and work backwards through each dependency. A hidden unit influences several outputs, so its derivative sums the contributions along all outgoing paths. This is sensitivity accounting, not a literal allocation of causal blame. Once those sensitivities have been computed, gradient descent or another optimiser can use them.

15.4.1 Hidden–output weights

Let w_{bc} connect hidden unit $b \in \{0,\dots,l\}$ to output $c \in \{1,\dots,m\}$ (so $b=0$ is the output bias). The strategy is the same chain as in the single-layer case. Only the term E_{ic} depends on w_{bc}, the sigmoid slope contributes $\mathrm{e}^{-z_{ic}} o_{ic}^2$, and the signal on this connection is the hidden activation h_{ib}.

$$\Delta w_{bc} = -\varepsilon \frac{\partial E}{\partial w_{bc}} = -\varepsilon \sum_{i=1}^{n_{\text{tr}}} \frac{\partial E_{ic}}{\partial w_{bc}}, \qquad \frac{\partial E_{ic}}{\partial w_{bc}} = -2[y_{ic} - o_{ic}] \frac{\partial o_{ic}}{\partial w_{bc}},$$
$$\frac{\partial o_{ic}}{\partial w_{bc}} = g'(z_{ic}) \frac{\partial z_{ic}}{\partial w_{bc}} = \mathrm{e}^{-z_{ic}} o_{ic}^2\, h_{ib},$$

so that

$$\boxed{\Delta w_{bc} = 2\varepsilon \sum_{i=1}^{n_{\text{tr}}} [y_{ic} - o_{ic}]\, \mathrm{e}^{-z_{ic}}\, o_{ic}^2\, h_{ib}} \tag{15.7}$$

where $h_{i0} = 1$ for the bias. The display is the gradient step for a weight that sees the hidden activation directly. The factor $\mathrm{e}^{-z} o^2$ is again $g'(z)$.

15.4.2 Input–hidden weights

Let w_{ab} connect input $a \in \{0, \dots, p\}$ to hidden unit $b \in \{1, \dots, l\}$. This weight affects *every* output, because every output depends on h_{ib}. The chain rule therefore sums over outputs: each contribution is a residual, times the output slope, times the outgoing weight w_{bk} that carried this hidden unit into that output, times the hidden slope, times the input x_{ia}.

$$\frac{\partial E_{ik}}{\partial w_{ab}} = -2[y_{ik} - o_{ik}]\frac{\partial o_{ik}}{\partial w_{ab}}, \qquad \frac{\partial o_{ik}}{\partial w_{ab}} = \mathrm{e}^{-z_{ik}} o_{ik}^2 \frac{\partial z_{ik}}{\partial w_{ab}} = \mathrm{e}^{-z_{ik}} o_{ik}^2 \, w_{bk} \frac{\partial h_{ib}}{\partial w_{ab}},$$
$$\frac{\partial h_{ib}}{\partial w_{ab}} = g'(u_{ib})\frac{\partial u_{ib}}{\partial w_{ab}} = \mathrm{e}^{-u_{ib}} h_{ib}^2 \, x_{ia},$$

and summing over k and i,

$$\boxed{\Delta w_{ab} = 2\varepsilon \sum_{i=1}^{n_{\mathrm{tr}}} \sum_{k=1}^{m} [y_{ik} - o_{ik}] \, \mathrm{e}^{-z_{ik}} o_{ik}^2 \, w_{bk} \, \mathrm{e}^{-u_{ib}} h_{ib}^2 \, x_{ia}} \tag{15.8}$$

The boxed expression is the update $-\varepsilon \, \partial E/\partial w_{ab}$, not the derivative itself. Its factors trace every path from input weight to loss. All are evaluated at the weights used in the forward pass; mixing old activations with newly updated outgoing weights would not evaluate one gradient.

Pulling out the factors that do not depend on k,

$$\Delta w_{ab} = 2\varepsilon \sum_{i=1}^{n_{\mathrm{tr}}} \mathrm{e}^{-u_{ib}} h_{ib}^2 \, x_{ia} \Big(\sum_{k=1}^{m} [y_{ik} - o_{ik}] \mathrm{e}^{-z_{ik}} o_{ik}^2 \, w_{bk} \Big). \tag{15.9}$$

The inner sum is the share of the output error that belongs to hidden unit b. Factoring it out does not change the value. It shows which pieces depend on the output index and which pieces are local to the hidden unit.

15.4.3 The delta form

Both updates have the structure "input signal × local derivative × error signal". Using $g'(z) = g(z)(1 - g(z))$, which is the same identity as $\mathrm{e}^{-z} g(z)^2$, define for pattern i the *error signals* (sometimes called deltas)

$$\delta_{ik}^{(o)} = -2 \, (y_{ik} - o_{ik}) \, o_{ik}(1 - o_{ik}), \qquad \delta_{iq}^{(h)} = h_{iq}(1 - h_{iq}) \sum_{k=1}^{m} w_{qk} \, \delta_{ik}^{(o)}, \tag{15.10}$$

so that $\partial E/\partial w_{qk} = \sum_i \delta_{ik}^{(o)} h_{iq}$ and $\partial E/\partial w_{jq} = \sum_i \delta_{iq}^{(h)} x_{ij}$, and the update is $\Delta w = -\varepsilon \, \partial E/\partial w$. Compare with the slide forms: $\mathrm{e}^{-z} o^2 = o(1 - o)$, so $\delta_{ik}^{(o)} = -2[y_{ik} - o_{ik}]\mathrm{e}^{-z_{ik}} o_{ik}^2$ and $\delta_{iq}^{(h)}$ equals the bracketed sum in (15.9) times -2 times $\mathrm{e}^{-u} h^2$. The output error is propagated *backwards* through the weights w_{qk}, and each hidden unit receives its share of the blame. That backward pass is where the name comes from.

Algorithm 15.3 (Backpropagation, batch form)**.** For one pass through the data with current weights:

1. *Forward:* for every pattern compute and *store* $u_{iq}, h_{iq}, z_{ik}, o_{ik}$.
2. *Output deltas:* $\delta_{ik}^{(o)}$ from (15.10).
3. *Backward:* hidden deltas $\delta_{iq}^{(h)}$ from (15.10).

4. *Gradients:* $G_{qk} = \sum_i \delta_{ik}^{(o)} h_{iq}$, $G_{jq} = \sum_i \delta_{iq}^{(h)} x_{ij}$ (with $h_{i0} = x_{i0} = 1$).
5. *Update:* $w \leftarrow w - \varepsilon G$ for every weight.

The forward phase has to store, for each training input, the hidden and output values and the weighted inputs that produced them. The backward phase multiplies those stored values by the deltas. Nothing in the algorithm minimises E in closed form; each pass is one gradient step.

Example 15.4 (One backpropagation step, checked numerically)**. Question.** A network has $p = 2$ inputs, $l = 2$ hidden sigmoid units, and $m = 1$ sigmoid output. For the single training pattern $\boldsymbol{x} = (1, 0)$, $y = 1$ the weights are, with the first index 0 for the bias,

$$w_{01} = 0.1,\ w_{11} = 0.4,\ w_{21} = -0.3; \qquad w_{02} = -0.2,\ w_{12} = 0.3,\ w_{22} = 0.2;$$
$$w_{01}^{\text{out}} = 0.2,\ w_{11}^{\text{out}} = 0.5,\ w_{21}^{\text{out}} = -0.4,$$

where w_{jq} is the weight from input j to hidden unit q and w_{q1}^{out} from hidden unit q (or the bias, $q = 0$) to the output. Perform one gradient step with $\varepsilon = 0.5$.

Solution. *Forward.* $u_1 = 0.1 + 0.4(1) + (-0.3)(0) = 0.5$ and $u_2 = -0.2 + 0.3(1) + 0.2(0) = 0.1$, so $h_1 = g(0.5) = 0.6225$ and $h_2 = g(0.1) = 0.5250$. Then $z = 0.2 + 0.5(0.6225) - 0.4(0.5250) = 0.3012$ and $o = g(0.3012) = 0.5748$. The error is $E = (1 - 0.5748)^2 = 0.1808$.

Output delta. $\delta^{(o)} = -2(1 - 0.5748)(0.5748)(0.4252) = -0.2079$. Gradients of the output weights are $\delta^{(o)} \times (1, h_1, h_2) = (-0.2079,\ -0.1294,\ -0.1091)$.

Hidden deltas. $\delta_1^{(h)} = h_1(1 - h_1)\, w_{11}^{\text{out}}\, \delta^{(o)} = 0.2350 \times 0.5 \times (-0.2079) = -0.02443$ and $\delta_2^{(h)} = 0.2494 \times (-0.4) \times (-0.2079) = +0.02074$. The gradients of the input–hidden weights are $\delta_q^{(h)} \times (1, x_1, x_2) = \delta_q^{(h)} \times (1, 1, 0)$: $(-0.02443, -0.02443, 0)$ for unit 1 and $(0.02074, 0.02074, 0)$ for unit 2. The weights from the second input have zero gradient because $x_2 = 0$.

Check. Central differences with $\delta = 10^{-6}$ for all nine weights agree with these analytical gradients to 2×10^{-11}.

Update ($\varepsilon = 0.5$): $w_{01} = 0.1 + 0.5(0.02443) = 0.1122$, $w_{11} = 0.4122$, $w_{02} = -0.2104$, $w_{12} = 0.2896$, output weights $0.2 + 0.5(0.2079) = 0.3039$, $0.5 + 0.5(0.1294) = 0.5647$, $-0.4 + 0.5(0.1091) = -0.3454$. The new output is $o = 0.6175$ and the error has fallen from 0.1808 to 0.1463.

Automatic differentiation applies the chain rule to the elementary operations of the computation. It evaluates the same derivative the algebra above organises by hand. The central-difference check in Example 15.4 is a separate numerical test of that derivative.

15.5 How many weights?

A hidden layer that sees every input, and an output layer that sees every hidden unit, produces one weight per connection, plus a bias on every unit. The count is

$$(p + 1)\, l + (l + 1)\, m \tag{15.11}$$

For example, NETtalk (Section 16.6.4) had $p = 7 \times 29 = 203$ inputs, $l = 80$ hidden units and $m = 26$ output units: $204 \times 80 + 81 \times 26 = 16\,320 + 2106 = 18\,426$ weights. The search for a global minimum of E is a search in that dimension.

A large parameter count raises estimation and optimisation questions, but does not prove that useful prediction is impossible. Symmetries can make individual weights non-identifiable even when the fitted function is well determined. A smaller network reduces one source of flexibility,

yet can underfit or be harder to optimise. Compare functions and validation performance, rather than assuming smaller always generalises better.

Remark 15.5 (Local minima, plateaus, symmetry). The loss is generally nonconvex. A run can approach a local minimum, a saddle or a nearly flat region, or fail to settle. Permuting hidden units and their incident weights preserves the function; there are up to $l!$ distinct permutations, fewer if some units coincide. Fully symmetric hidden-unit initialisation preserves that symmetry under deterministic gradient updates. Random initialisation breaks it. For sigmoids, replacing $g(u)$ by $g(-u) = 1 - g(u)$ also requires compensating outgoing weights and biases to preserve the function.

The loss need not be convex, so a run can stop at a local minimum. The remark records the symmetry that repeats each minimum, and the reason a constant initialisation never breaks that symmetry. Local minima are not asserted to be absent, and the start matters. The practical obstruction is often a flat region of the loss surface, where the gradient is near zero without the point being a minimiser. Saturation of a sigmoid is one way to land in such a region: g' is then close to zero, and the chain rule multiplies every upstream weight by that small factor.

15.6 Behaviour on XOR

Training a 2–2–1 sigmoid network on XOR is where the learning rate and the random start stop being abstract. The curves below are recomputed in R at render time. Some runs can remain on a plateau; large steps can cause saturation or unstable updates. The finite trajectory is evidence about this implementation and starting point. It is not a general convergence guarantee.

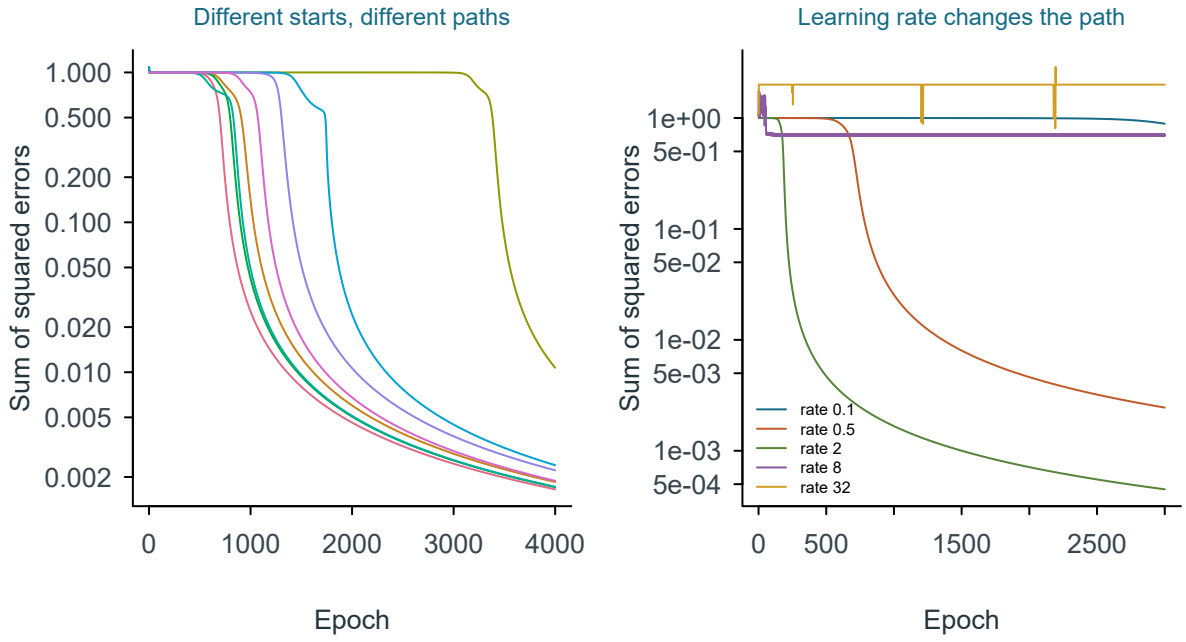


Figure 15.1: XOR training in R for eight random starts and five learning rates. Total squared error is plotted on a logarithmic scale.

The hidden units learn a nonlinear representation in which the output can separate the classes. The next plot is generated from the R fit itself. Hidden units may swap, change sign with compensating weights, or find different valid representations. Their particular weights are not unique scientific parameters. The existence of several representations is the same symmetry recorded in Remark 15.5.

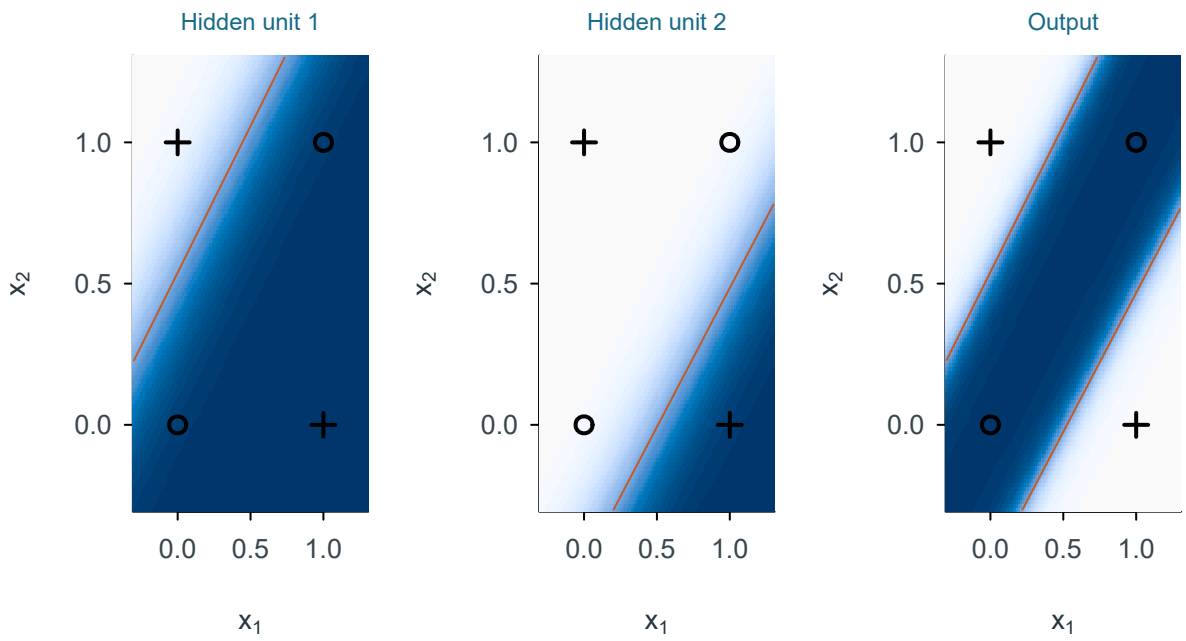


Figure 15.2: Two learned hidden activations and the output of the seeded R XOR network. Contours mark activation 0.5.

Key idea.
Backpropagation evaluates the gradient by reusing local derivatives. The forward pass supplies activations; the backward pass carries sensitivities through outgoing weights; the optimiser then updates parameters. Numerical gradient checks verify that calculation. They do not certify the model, the optimiser's global success or its performance on new observations.

R template. The fragment below is the batch update written as code: logistic units, squared error with no factor $\frac{1}{2}$, and one gradient step per epoch. The executed laboratory underneath is the numerical check.

```
# Batch backpropagation for one hidden layer, logistic units, squared error (no
↪  1/2)
g <- function(z) 1 / (1 + exp(-z))
backprop <- function(X, Y, l = 2, eps = 0.5, epochs = 5000, seed = 1) {
  set.seed(seed)
  X1 <- cbind(1, X)                                  # add bias input x0 = 1
  W1 <- matrix(runif((ncol(X) + 1) * l, -0.5, 0.5), ncol(X) + 1, l)
  W2 <- matrix(runif((l + 1) * ncol(Y), -0.5, 0.5), l + 1, ncol(Y))
  sse <- numeric(epochs)
  for (e in 1:epochs) {
    H  <- g(X1 %*% W1); H1 <- cbind(1, H)            # feed forward
    O  <- g(H1 %*% W2)
    sse[e] <- sum((Y - O)^2)                         # error
    dO <- -2 * (Y - O) * O * (1 - O)                 # output deltas
    dH <- (dO %*% t(W2[-1, , drop = FALSE])) * H * (1 - H)   # hidden deltas
    W2 <- W2 - eps * crossprod(H1, dO)               # weight updates
    W1 <- W1 - eps * crossprod(X1, dH)
  }
```

```
  list(W1 = W1, W2 = W2, sse = sse)
}
X <- rbind(c(0,0), c(0,1), c(1,0), c(1,1)); Y <- matrix(c(0,1,1,0), ncol = 1)
fit <- backprop(X, Y)
tail(fit$sse, 1)                                    # near 0 for most seeds
```

15.7 R laboratory: a numerical gradient check

Before trusting a training run, compare the analytic gradient with central finite differences at a fixed set of weights that are not saturated. Both gradients refer to the same total squared-error objective, with no factor of $\frac{1}{2}$. Agreement at one point checks the implemented derivative. It does not prove that training will converge, and it does not speak to generalisation.

```
X <- cbind(1, xor_x); Y <- matrix(xor_y, ncol = 1)
w <- seq(-0.4, 0.4, length.out = 9)
loss_grad <- function(w) {
  W1 <- matrix(w[1:6], 3, 2); W2 <- matrix(w[7:9], 3, 1)
  H <- plogis(X %*% W1); Hb <- cbind(1, H)
  O <- plogis(Hb %*% W2)
  dO <- 2 * (O - Y) * O * (1 - O)
  dH <- (dO %*% t(W2[-1, , drop = FALSE])) * H * (1 - H)
  list(value = sum((O - Y)^2),
       gradient = c(crossprod(X, dH), crossprod(Hb, dO)))
}
h <- 1e-5
numeric_grad <- sapply(seq_along(w), function(j) {
  plus <- minus <- w; plus[j] <- plus[j] + h; minus[j] <- minus[j] - h
  (loss_grad(plus)$value - loss_grad(minus)$value) / (2 * h)
})
max_error <- max(abs(numeric_grad - loss_grad(w)$gradient))
max_error
#> [1] 1.885e-11
stopifnot(max_error < 1e-7)
trained <- backprop_r(seed = 1, epochs = 6000)
tail(trained$loss, 1)
#> [1] 0.0009966
```

The maximum absolute discrepancy is 1.89e-11. This checks the implemented derivative at this point. The regenerated learning curves use R's random-number generator, so their trajectories need not match the earlier Python edition.

15.8 Further exercises

Exercise 15.1 (Forward pass)**.** For the network of Example 15.4, compute the output for the pattern $\boldsymbol{x} = (0, 1)$.

Solution

Exercise 15.2 (A gradient by hand)**.** A single sigmoid unit has $z = w_0 + w_1 x$, $w_0 = 0$, $w_1 = 1$, and a single pattern $x = 2$, $y = 0$. Compute $E = (y - o)^2$, $\partial E / \partial w_1$ using (15.5), and the same derivative by a central difference with $\delta = 0.001$. Then show the formula without the factor e^{-z} would give a different answer.

Solution

Exercise 15.3 (Counting weights)**.** Use (15.11) to count the weights of a network with $p = 10$ inputs, $l = 5$ hidden units, and $m = 3$ outputs. Compare with the number of parameters of a multinomial logistic regression on the same inputs and outputs.

Solution

Exercise 15.4 (Symmetric start)**.** Show that if all hidden units of a network start with identical incoming weights and identical outgoing weights, then batch gradient descent keeps them identical for all time. What does this say about initialising all weights at 0?

Solution

Exercise 15.5 (Delta form)**.** Verify that (15.8) equals $-\varepsilon \sum_i \delta_{ib}^{(h)} x_{ia}$ with $\delta^{(h)}$ given in (15.10).

Solution

Exercise 15.6 (Saturation)**.** For a sigmoid output unit with squared-error loss, a pattern with target $y = 1$ has net input $z = -8$. Compute the output and the gradient $\partial E/\partial z$, and compare with the gradient at $z = -1$. What does this say about recovering from a confidently wrong prediction, and how does cross-entropy loss change the picture?

Solution

Exercise 15.7 (Explain one backpropagation update)**.** A one-input network has $h = g(ax + c)$, $o = g(vh + b)$ and loss $L = \frac{1}{2}(o - y)^2$, where $g(z) = 1/(1 + e^{-z})$.

(a) Derive $\partial L/\partial v$ and $\partial L/\partial b$.

(b) Derive $\partial L/\partial a$, identifying the factors contributed by the chain rule.

(c) Explain the sign and the role of the learning rate in one gradient-descent update.

Solution

Chapter 16

Neural networks in practice

We now know how to differentiate a network. That settles one problem, but leaves the practical one: which fitted network should we trust on new observations? Backpropagation computes a gradient; the optimiser decides how to use it. Even a step that reduces training loss can make prediction worse by fitting accidental features of the sample.

Initialisation and the scale of the inputs decide whether the sigmoid starts on its steep part, where the chain rule of Chapter 15 has a slope to multiply. Validation, early stopping, and weight decay decide how far that slope is followed. Architecture, decay, and the stopping rule are chosen on training and validation data. The assessment data are reserved for the procedure once those choices are frozen.

The previous chapter derived the gradient. This chapter takes the choices in turn: how to initialise the weights, whether to update after every pattern or after every epoch, how to present data, how to encode classes and obtain probabilities, how to limit specialisation while training for generalisation, how to check a trained network, and where the same ideas have been used. It closes with the changes that lead to deep learning.

Source. Neural-network lecture slides 62–93.

16.1 Parameter values and other issues

Several quantities have to be settled before a gradient step is worth taking:

- weight initialisation (typically small random values with mean 0);
- batch or online updates;
- the learning rate, $\varepsilon > 0$, chosen for the scaling of the loss and inputs;
- the stopping criterion;
- the number of layers;
- the number of nodes in hidden layers;
- the number of input–output examples.

The first two are the subject of the next two sections. A learning rate of 0.1 is an example of a step size. It is not a distinguished constant. The step size is a tuning choice: if the error oscillates, take a smaller step; if the weights barely move, a larger step may be tried.

16.2 Weight initialisation

A sigmoid has its largest slope near zero. Far into either tail, its derivative is small, so the chain rule transmits little change back through that unit. We therefore want initial net inputs of a moderate size. We also want different hidden units to start differently: units with identical incoming and outgoing weights remain interchangeable under deterministic symmetric updates. Independent random initialisation breaks that symmetry. Standardising predictors using training-set means and standard deviations makes a common initialisation scale more meaningful; it does not guarantee that every net input stays near zero. See LeCun et al. (1998) for the scaling argument and its practical context.

16.2.1 A variance argument

Let each non-bias input have mean zero and variance one, and initialise the weights independently as $w_{jq} \sim \mathrm{N}(0, \sigma^2)$, independently of the inputs. Write $u_{iq} = \sum_{j=0}^{p} w_{jq} x_{ij}$, where $x_{i0} = 1$. Each summand has mean zero and second moment σ^2. Cross terms vanish because distinct weights are independent and centred: $\mathbb{E}(w_{jq} w_{kq} x_{ij} x_{ik}) = 0$ for $j \neq k$. Notice that the inputs themselves need not be uncorrelated. Consequently,

$$\mathrm{Var}(u_{iq}) = (p+1)\,\sigma^2. \tag{16.1}$$

Choosing $\sigma = (p+1)^{-1/2}$ makes this marginal variance one. That is a scale calculation, not yet a tail-probability calculation. Conditional on a particular input vector, Gaussian weights give

$$u_{iq} \mid \boldsymbol{x}_i \sim \mathrm{N}\Big(0, \sigma^2[1 + \|\boldsymbol{x}_i\|^2]\Big)\,.$$

Thus, for an input with $\|\boldsymbol{x}_i\|^2 = p$, the proposed scale gives $\mathbb{P}(|u_{iq}| \leq 2 \mid \boldsymbol{x}_i) = 2\Phi(2) - 1 \approx 0.9545$. Across random inputs, the distribution is generally a mixture of normals with different variances. Unit marginal variance alone does not justify that same 95% statement. Fan-in scaling is a useful starting point; activation choice and depth still matter (LeCun et al., 1998; Goodfellow et al., 2016).

16.3 Batch and online learning

The backpropagation update derived earlier is in *batch mode*: the sums over patterns in (15.7) and (15.8) require all n_{tr} training patterns to be fed in, and the hidden and output activations recorded, before any weight changes.

A full gradient requires work over all training examples. With a large sample, we may prefer more frequent, cheaper updates. Online stochastic gradient descent samples an index i uniformly and uses its loss gradient. If $E = \sum_i E_i$, then

$$\mathbb{E}_i\big[n_{\mathrm{tr}} \nabla E_i\big] = \nabla E.$$

This expectation is conditional on the current weights and uniform sampling of the index. It says that the gradient estimate is unbiased, not that each proposed step reduces the full loss. For the average loss E/n_{tr}, use ∇E_i without the factor n_{tr}. Confusing these two conventions changes the effective learning rate.

- *Cost per update.* One example requires less work than a full pass, so an online method can update weights sooner. A mini-batch shares this benefit while averaging some of the sampling noise.

- *Noise and convergence.* A noisy gradient can help leave some shallow regions, but it gives no general escape guarantee. With a fixed learning rate, fluctuations may persist near a solution; convergence results require assumptions on the objective, sampling and step sizes. Rare examples may be absent from short random runs. Shuffling each epoch ensures every training example is used during that epoch (Goodfellow et al., 2016, Chapter 8).

A compromise in common use is the *mini-batch*: the gradient is averaged over a small random subset of patterns. Batch mode uses the full sample gradient. Online mode uses one observation.

16.4 Preparing the data

Consider a table of patient attributes such as age, blood pressure, chest-pain category and a recorded diagnosis. This is the preprocessing example used in the course notes, attributed there to Thornton (1993); its original data file and reported 300-record sample are not available in this project. We use it to discuss encoding, without treating its performance as a reproducible clinical result. The statistical issue is simple: a numeric code must not invent a geometry that the variable does not have.

- *Nominal attributes* (such as chest pain type): use separate binary input nodes (dummy variables). Coding categories $1, 2, 3$ as a single numeric input would impose an ordering and distances between the categories that are not in the data.
- *Numeric attributes*: normalise (to a common scale, for example mean 0 and variance 1, which also helps initialisation).
- *Missing values*: choose an imputation rule fitted on the training data, and apply that fitted rule to validation and test data. A training mean is a simple baseline, not a universal answer. Zero has a meaningful interpretation only after the coding and scaling are specified. If missingness itself is informative, consider an indicator and assess the complete procedure by validation.

16.5 Output coding, softmax, and cross-entropy

Separate sigmoid outputs, one per class, need not sum to 1. They are scores, and a probability model for a single class label needs a distribution. For v classes, the simplest coding still uses v output units: for each pattern, output k is 1 if the pattern is in class k and 0 otherwise. This treats each class equally. On test data, and often on training data, every output lies in $(0, 1)$; the usual prediction is the class of the largest output.

When the outputs are meant to be class probabilities, they have to add to one. The *softmax* of Bridle (1990) exponentiates and then normalises:

$$s_k = \frac{\mathrm{e}^{z_k}}{\sum_{l=1}^{m} \mathrm{e}^{z_l}}, \qquad k = 1, \ldots, m. \tag{16.2}$$

Here z_k is the output of the last linear stage, often called a logit. Cross-entropy is the negative log-likelihood for the observed categorical label. Squared error is also a possible probability-scoring loss, but cross-entropy has a particularly convenient derivative with softmax: a confidently wrong output produces a substantial gradient with respect to its logits. This is a statement about the output layer, not a guarantee against vanishing gradients throughout the network (Goodfellow et al., 2016, Chapter 6).

Let the target be a one-hot vector $\boldsymbol{y}$ ($y_k = 1$ for the true class c) and the loss be

$$E = -\sum_{k=1}^{m} y_k \ln s_k = -\ln s_c. \tag{16.3}$$

The derivative is another chain rule. Differentiate $-\ln s_c$ through $s_k = \mathrm{e}^{z_k}/\sum_l \mathrm{e}^{z_l}$, using $\partial s_k/\partial z_j = s_k(\delta_{kj} - s_j)$. The factor $1/s_k$ from the logarithm cancels the s_k in the softmax Jacobian:

$$\frac{\partial E}{\partial z_j} = -\sum_k y_k \frac{1}{s_k} s_k(\delta_{kj} - s_j) = -\sum_k y_k(\delta_{kj} - s_j) = -y_j + s_j \sum_k y_k = s_j - y_j. \tag{16.4}$$

The result is $\partial E/\partial z_j = s_j - y_j$. The calculation only needs $\sum_k y_k = 1$, so it also works for probability-valued target vectors. There is no additional sigmoid-slope factor at this layer. A confidently correct prediction can, and should, have a small gradient; a confidently wrong prediction retains an error signal. Earlier hidden layers still contribute their own derivatives during backpropagation (Exercise 15.6).

Remark 16.1 (Softmax is multinomial logistic regression). With no hidden layer, $z_k = \boldsymbol{\beta}_k^\top \boldsymbol{x}$ and softmax gives $\widehat{\tau}_k(\boldsymbol{x}) = \mathrm{e}^{\boldsymbol{\beta}_k^\top \boldsymbol{x}}/\sum_l \mathrm{e}^{\boldsymbol{\beta}_l^\top \boldsymbol{x}}$, which is the multinomial logistic model (10.14) (with the reference class absorbed by setting $\boldsymbol{\beta}_1 = \mathbf{0}$; softmax is unchanged when the same vector is added to every $\boldsymbol{\beta}_k$), and minimising the cross-entropy over the training set is maximising the multinomial likelihood. For two classes, softmax reduces to the sigmoid: $\mathrm{e}^{z_1}/(\mathrm{e}^{z_1} + \mathrm{e}^{z_2}) = g(z_1 - z_2)$. A network with a softmax output layer is therefore multinomial logistic regression on learned features $\boldsymbol{h}$.

Example 16.2 (Softmax and its gradient)**.** Three output activations are $\boldsymbol{z} = (2, 1, 0.1)$ and the true class is 1. Then $\mathrm{e}^{\boldsymbol{z}} = (7.389, 2.718, 1.105)$ with sum 11.213, so $\boldsymbol{s} = (0.659, 0.242, 0.099)$ (these sum to 1). The cross-entropy is $-\ln 0.659 = 0.417$ and the error signals are $\boldsymbol{s} - \boldsymbol{y} = (-0.341,\ 0.242,\ 0.099)$: the correct class's activation is pushed up and the others down, each in proportion to its probability. Adding a constant to all of $\boldsymbol{z}$ leaves $\boldsymbol{s}$ unchanged, which is why software subtracts $\max_k z_k$ before exponentiating, to prevent overflow.

16.6 Constructing and training a network

To construct a neural network: choose a network topology; determine the number of hidden units, if any; create the network; learn the weight parameters; and evaluate the trained network through testing.

16.6.1 Generalisation and the learning phase

Minimising the error on the training set is a strategy for obtaining good performance on new data. It does not always work, for two reasons that show up as different shapes of the training loss.

Over-fitting. The model is a very good fit to the training data. It may have modelled some useful structure but has also modelled many irrelevant aspects. This is common with overly complex models.

Under-fitting. The model is not a good fit to the training data (the size of E is a good indication). The model may be too simple, there may not be enough data, or the data may be so noisy that the output can never be predicted very accurately.

A common pattern is that training loss keeps falling after validation loss begins to rise. Early stopping selects a point along the optimisation path before the training objective is fully minimised. It therefore restricts the fit indirectly, but is not generally equivalent to a specified L_2 penalty, nor does memorisation require every weight to be large. Its effect depends on the architecture,

initialisation and optimiser (Goodfellow et al., 2016, Section 7.8). More observations can help, although their usefulness depends on how they cover the prediction task.

The curve used for stopping is a validation curve. A test curve used to stop training has been spent as a tuning device, and it can no longer serve as an untouched assessment.

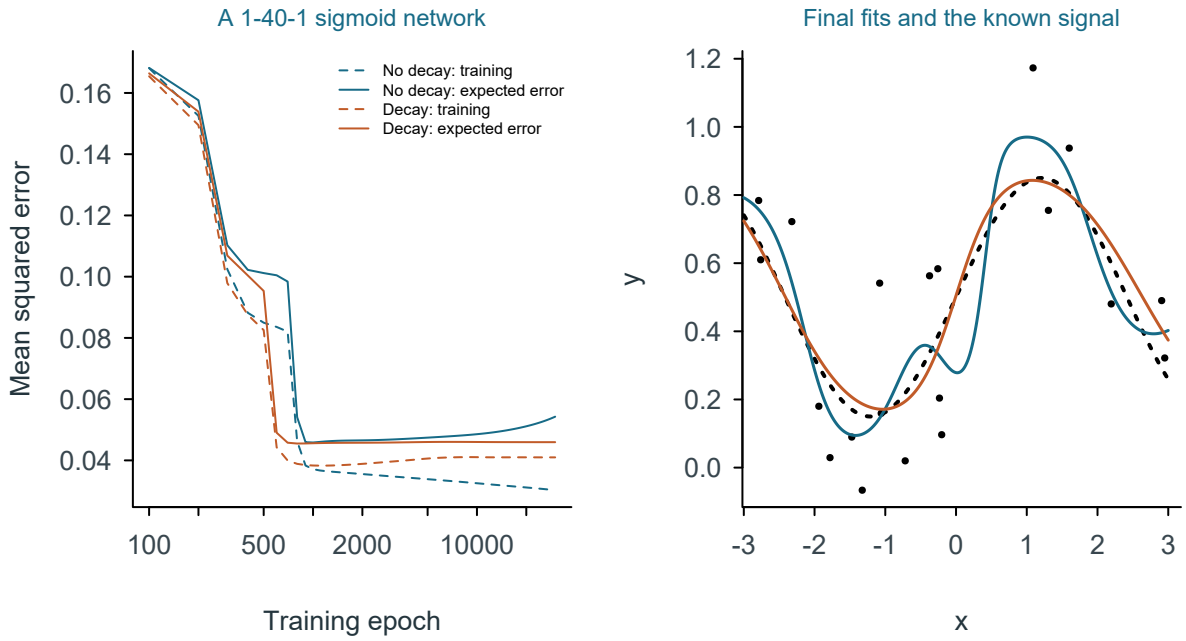


Figure 16.1: An explicit R backpropagation simulation with 20 observations and a 1-40-1 sigmoid network. Dashed curves are training MSE; solid curves are expected noisy-response MSE averaged over a fixed uniform predictor grid. Weight decay is 0 or 0.01; training runs for 30,000 epochs.

16.6.2 Training, validation, and test sets

To control generalisation the data set is divided into a training set, a validation set, and a test set:

- the *training set* is used to optimise the weights;
- the *validation set* is monitored during (or after) training to optimise model parameters: when to stop, the number of nodes or weights, the learning rate, and so on;
- the *test set* is used to assess how well the final model performs.

This is the same discipline as nested cross-validation (Chapter 9): anything chosen by looking at data is part of model building and must be assessed on data that played no part. *Early stopping* uses the validation error: stop at the epoch where it is smallest.

16.6.3 Regularisation and weight decay

A weight penalty changes the objective itself. With L_2 regularisation we charge for large squared weights, just as ridge regression does for linear coefficients. This is a modelling preference for smaller parameter values in the chosen representation; equivalent reparameterisations need not receive the same penalty. Under ordinary gradient descent, the resulting update contains a literal multiplicative decay. That equivalence should not be transferred automatically to an adaptive optimiser.

- L_2 *regularisation*, also known as *weight decay*, ridge regression (Hoerl and Kennard, 1970), or Tikhonov regularisation, adds a penalty proportional to the sum of squares of the weights

to the error function, i.e. optimise $E + \lambda\|\boldsymbol{w}\|^2$ with $\lambda > 0$ chosen by the user. After rescaling the tuning parameter, write the penalty as $E + \frac{\lambda}{2}\|\boldsymbol{w}\|^2$; then, the gradient step becomes

$$w \leftarrow w - \varepsilon\Big(\frac{\partial E}{\partial w} + \lambda w\Big) = (1 - \varepsilon\lambda)\, w - \varepsilon\frac{\partial E}{\partial w}, \tag{16.5}$$

When $0 \le \varepsilon\lambda \le 1$, the factor $1 - \varepsilon\lambda$ shrinks a weight towards zero without reversing its sign. With no data-gradient term, t steps multiply it by $(1 - \varepsilon\lambda)^t$; for $\varepsilon = 0.1$, $\lambda = 0.5$ and $t = 10$, this factor is $0.95^{10} \approx 0.599$. A step size is part of the statement: an excessive value can cause oscillation or divergence even in this simple recurrence. The penalty encourages small weights, but does not have the lasso's thresholding mechanism.

- L_1 *regularisation* uses $E + \lambda \sum |w|$. The derivative at zero is replaced by a subgradient or a proximal update. Such algorithms can produce exact zeros. The non-differentiability changes the optimisation method; it does not make the penalty unusable.

16.6.4 NETtalk

NETtalk is an early example of learning a mapping from written context to a pronunciation code (Sejnowski and Rosenberg, 1986). A window of letters supplies the input and the central letter supplies the prediction target. The course architecture, with 203 inputs, 80 hidden units and 26 outputs, has $(203 + 1)80 + (80 + 1)26 = 18\,426$ weights including biases. This count illustrates how quickly connections accumulate. Weight-decay experiments such as Krogh and Hertz (1992) motivate regularisation, but a successful setting in one experiment is not a transferable value of λ.

16.7 Evaluating the network

A trained network can be evaluated in three ways:

- *Record generalisation performance* on different test sets. This provides no model-specific explanation.
- *Inspect internal activations*, for example by visualising the spatial similarity between hidden-unit activations for different inputs. This provides knowledge on the effect of different inputs on the model.
- *Study the network weights*, for example with a *Hinton diagram* that shows (by the size and colour of boxes) how the weights are used and which units are relevant. This provides knowledge on the decisions made in the model.

Training loss, held-out classification error and stability across starts answer different questions. In particular, the smallest training SSE need not identify the smallest future classification error: the losses differ, and training data have already influenced the fit. Compare repeated starts within the training/validation procedure and report the variability relevant to the claim. If the test set is repeatedly used to pick a seed, it has become another validation set (Cawley and Talbot, 2010, Section 5.1). Examine errors by class and by relevant subgroups as well as reporting a single average.

16.8 Applications

ALVINN (NavLab).

Pomerleau (1989) used a network to steer a car. The input is a 30×32-pixel digitised picture of the driver's view and the 30 outputs represent the steering direction; the output is a steering signal and a confidence. The weights of the network are sometimes interpretable: shown at the

position of the corresponding image pixel, box size indicating magnitude and colour the sign, they reveal which parts of the road image influence the steering.

Reading handwritten ZIP codes.

Handwritten-digit recognition provides a concrete reason for weight sharing. A detector for a short stroke should be useful in more than one image location. Convolutional architectures apply shared filters to local patches, then combine the resulting features. LeCun et al. (1998) describe this approach and evaluate document-recognition systems. Any quoted accuracy must specify the test population and whether uncertain cases are rejected: accuracy among accepted cases and accuracy on every submitted digit are different quantities.

16.9 Recent innovations

Deep networks compose many learned transformations. ReLU, $g(z) = \max(0, z)$, has derivative one for $z > 0$ and zero for $z < 0$; that avoids sigmoid saturation on its positive branch, while allowing inactive units on the negative branch. Convolutional networks share local filters, recurrent networks reuse a state update across a sequence, and autoencoders learn to reconstruct their inputs. These are changes to the architecture and objective, not exemptions from validation (Goodfellow et al., 2016).

For a centred data matrix, a linear autoencoder with a bottleneck of dimension q and squared reconstruction loss has a global optimum spanning a leading q-dimensional principal subspace, provided no additional constraints change the problem. This follows from the same best-rank-q approximation argument as PCA. The weights need not be the individual principal directions, and tied eigenvalues make the optimal subspace non-unique. Nonlinear autoencoders solve a broader problem; calling their output "PCA'' would lose the distinction.

> **Key idea.**
> Initialise with small random weights scaled by the fan-in, normalise the inputs, and use cross-entropy with softmax for classification. Choose when to stop, how many hidden units, and how much weight decay by a validation set. Then report performance on a test set that was not used in any of those choices.

16.10 R laboratory: a small softmax network for iris

Use scaled predictors, one hidden layer, three output classes and weight decay. Architecture and decay are fixed for this illustration. A production comparison would tune these choices inside training data, by the validation discipline above.

```
set.seed(16006)
y_onehot <- nnet::class.ind(train$Species)
net <- nnet::nnet(iris_scaled$train, y_onehot,
                  size = 3, softmax = TRUE, decay = 0.1,
                  maxit = 1000, trace = FALSE)
prob <- predict(net, iris_scaled$test, type = "raw")
pred <- factor(colnames(y_onehot)[max.col(prob)],
               levels = levels(train$Species))
c(convergence_code = net$convergence,
  test_error = mean(pred != test$Species),
  max_probability_sum_error = max(abs(rowSums(prob) - 1)))
#>          convergence_code                test_error
```

```
#>                  0.000e+00                  8.889e-02
#> max_probability_sum_error
#>                  2.220e-16
stopifnot(net$convergence == 0,
          max(abs(rowSums(prob) - 1)) < 1e-8)
```

The convergence code reports the optimiser's stopping status. It is not a proof that the global optimum was found. Weight decay shrinks weights. The species probabilities must sum to one for each flower, which the explicit check verifies.

16.11 Further exercises

Exercise 16.1 (Initial weights)**.** A hidden layer has fan-in $p + 1 = 26$ (25 standardised inputs plus bias). What standard deviation should the initial weights have so that the net input has unit variance? What is the variance of the net input if the weights have standard deviation 0.5? For a fixed input with squared norm 25 and independent Gaussian initial weights, how likely is $|u| > 2$?

Solution

Exercise 16.2 (Softmax gradient)**.** Verify (16.4) for two classes and show that it reduces to $\partial E/\partial z = o - y$ for the sigmoid output with cross-entropy loss, where $o = g(z_1 - z_2)$.

Solution

Exercise 16.3 (Weight decay)**.** A weight $w = 2$ has $\partial E/\partial w = 0.4$, with $\varepsilon = 0.1$ and $\lambda = 0.5$ in (16.5). Compute the updated weight with and without decay. What value of w makes the net update zero?

Solution

Exercise 16.4 (Online versus batch)**.** For $n_{\text{tr}} = 5000$ patterns, each forward–backward pass costing one unit per pattern, compare the cost of 10 000 epochs of batch learning with 10 000 online iterations (one randomly chosen pattern each). What do the 10 000 online iterations amount to in epochs, and what does this suggest about when online learning is attractive?

Solution

Exercise 16.5 (SSE versus accuracy)**.** Explain how a network can have a smaller sum of squared errors than another but a lower classification accuracy. (Hint: think of outputs close to 0.5 and a threshold at 0.5.) Why is classification accuracy on the validation set a more direct criterion for early stopping in a classification problem, and what is its disadvantage?

Solution

Exercise 16.6 (Counting for NETtalk)**.** Verify the 18 426 weights of NETtalk. How many weights would a network with 50 hidden units have? If the training set has 5000 patterns and 26 outputs, how many numbers (targets) is it trying to fit, and what does the ratio of weights to targets suggest about over-fitting?

Solution

Exercise 16.7 (Neural-network complexity and early stopping)**.** A network uses four inputs, six sigmoid hidden units and three softmax outputs. Every non-input unit has a bias.

(a) Count the fitted weights and biases, showing how the total is obtained.

(b) Explain how a validation set can select the stopping epoch and why its best recorded score is not an independent performance estimate.

(c) Explain how an L_2 weight penalty changes training and why it does not usually produce exact zero weights.

Solution

Chapter 17

Support vector machines

Suppose several lines separate the two training classes perfectly. Their training error is identical, so that criterion cannot choose between them. A support vector machine adds a geometric preference: leave the widest possible slab between the classes. We will first derive that margin, then translate its maximisation into a convex optimisation problem. Convexity rules out nonglobal local optima; numerical accuracy and stopping still belong to the solver.

The same programme only ever sees the data through inner products. Replacing those inner products by a kernel is how a linear margin in a new feature space becomes a nonlinear boundary in the original one. The kernel, the scaling of the inputs, the cost C, and any kernel width are part of the model together. A large margin in one feature space need not be a large margin in another.

The kernel-based maximum-margin construction appears in Boser et al. (1992). Its appeal here is the connection between geometry, optimisation and prediction. Whether it outperforms another classifier on iris, text or gene-expression data is an empirical question about the complete fitting and assessment procedure.

Source. Support vector machine lecture slides (STAT3006/7305), based on slides by Prof. Raymond Mooney, UT Austin; the dual derivation, the XOR example, and the geometric derivations are filled in here.

17.1 Linear separators and the margin

Binary classification can be viewed as separating the classes in feature space. Assume p input variables drawn from two classes encoded as $+1$ and -1. The perceptron could stop at any hyperplane that puts the two classes on opposite sides. To prefer one of them we need a number that measures the gap, so the first object is the distance from a point to the hyperplane. The *unit function*, as for the threshold unit but with ± 1 coding, is

$$f(\boldsymbol{x}) = \operatorname{sign}(\boldsymbol{w}^\top \boldsymbol{x} + b), \tag{17.1}$$

where $\boldsymbol{w}$ is the weight vector, $\boldsymbol{x}$ the input vector, and b the bias term (called w_0 before). The hyperplane $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$ separates the region $\boldsymbol{w}^\top \boldsymbol{x} + b > 0$ from $\boldsymbol{w}^\top \boldsymbol{x} + b < 0$. Which of the linear separators in Figure 17.1 is optimal?

17.1.1 Distance to a hyperplane

Proposition 17.1 (Signed distance)**.** For $\boldsymbol{w} \neq \boldsymbol{0}$, the signed distance from a point $\boldsymbol{x}$ to the hyperplane $\{\boldsymbol{x} : \boldsymbol{w}^\top \boldsymbol{x} + b = 0\}$ is $(\boldsymbol{w}^\top \boldsymbol{x} + b)/\|\boldsymbol{w}\|$, where $\|\cdot\|$ is the Euclidean (2-)norm. In

particular the distance between the parallel hyperplanes $\boldsymbol{w}^\top \boldsymbol{x} + b = -1$ and $\boldsymbol{w}^\top \boldsymbol{x} + b = 1$ is $2/\|\boldsymbol{w}\|$.

Proof. The argument is as follows. The hyperplane $\boldsymbol{w} \cdot \boldsymbol{x} = 0$ goes through the origin and comprises all points with this property, so $\boldsymbol{w}$ is normal to it; the hyperplane $\boldsymbol{w} \cdot \boldsymbol{x} = c$, $c > 0$, is parallel to it (there is no intersection) and $\boldsymbol{w}$ is normal to this also. Let $\boldsymbol{x}_1$ be a point on $\boldsymbol{w} \cdot \boldsymbol{x} = 0$. Follow a normal vector until it meets the second hyperplane: find $\boldsymbol{x}_2 = \boldsymbol{x}_1 + a\boldsymbol{w}$ with $\boldsymbol{w} \cdot \boldsymbol{x}_2 = c$. Then $\boldsymbol{w} \cdot \boldsymbol{x}_1 + a\, \boldsymbol{w} \cdot \boldsymbol{w} = c$, so $a = c/\|\boldsymbol{w}\|^2$ and the distance between $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ is $\|a\boldsymbol{w}\| = c/\|\boldsymbol{w}\|$, which is the distance between the hyperplanes. Likewise the hyperplanes $\boldsymbol{w}^\top \boldsymbol{x} + b = \mp\rho/2$ are $\rho/\|\boldsymbol{w}\|$ apart (the offset b only translates them), and dividing $\boldsymbol{w}$ and b by $\rho/2$ turns this into $\boldsymbol{w}^\top \boldsymbol{x} + b = \mp 1$, a distance of $2/\|\boldsymbol{w}\|$ apart. For the signed distance of an arbitrary point, write $\boldsymbol{x} = \boldsymbol{x}_\perp + r\, \boldsymbol{w}/\|\boldsymbol{w}\|$ with $\boldsymbol{x}_\perp$ on the hyperplane; then $\boldsymbol{w}^\top \boldsymbol{x} + b = \boldsymbol{w}^\top \boldsymbol{x}_\perp + b + r\|\boldsymbol{w}\| = r\|\boldsymbol{w}\|$.

□

Rescaling $(\boldsymbol{w}, b)$ by a positive constant leaves the separating plane unchanged. It also rescales the numerator and denominator of the distance formula by the same factor. This is why a functional score such as $\boldsymbol{w}^\top \boldsymbol{x} + b$ is not itself a geometric distance until the norm of $\boldsymbol{w}$ has been accounted for.

The *margin* ρ of a separator is the distance between the closest points of the two classes (measured perpendicular to the separator), and those closest examples to the hyperplane are the *support vectors*. The *maximum margin* separator maximises ρ.

For hard-margin separation, only the points on the margin determine the solution. Moving a point that is strictly beyond the margin, without crossing it, does not change the optimum.

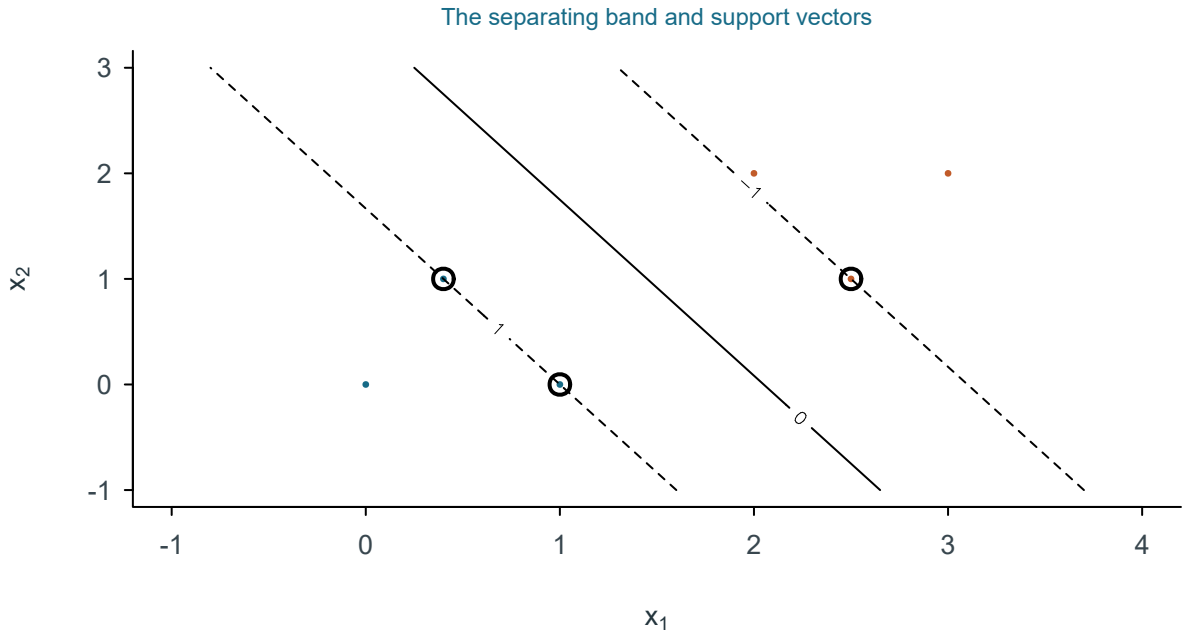


Figure 17.1: An R linear SVM with its decision boundary, margin contours and support vectors (rings).

17.1.2 Why maximise the margin?

A large margin gives a training point some tolerance to perturbation along the normal direction before its classification changes. Learning theory also relates margins to the richness of a class of decision rules. A set is *shattered* if every binary labelling of it can be realised by the class. The

VC dimension is the largest size of any shattered set; general position is not part of the definition. Affine half-spaces in $\mathbb{R}^p$ have VC dimension $p+1$. Some smaller degenerate configurations fail to be shattered, which is entirely compatible with that maximum.

The radius–margin bound uses a stronger requirement: each labelling of a set must be realisable with a prescribed geometric margin. State both the radius and the margin before applying it. The following classical form uses full slab width ρ and enclosing-ball diameter D (Burges, 1998, Section 7.1, Theorem 6).

Theorem 17.2 (Vapnik)**.** For points in a fixed ball of diameter D in $\mathbb{R}^p$, let h be the largest number that can be shattered by affine separators while every labelling has full geometric margin at least $\rho > 0$. The classical radius–margin bound is

$$h \le \min\left\{\left\lceil \frac{D^2}{\rho^2} \right\rceil,\ p\right\} + 1, \tag{17.2}$$

Here D/ρ is the same ratio as ball radius divided by distance from the boundary to the nearest class. This is a margin-restricted capacity statement. The ordinary class of all linear decision boundaries does not acquire a smaller VC dimension merely because one fitted boundary happens to have a large training margin.

For example, $D = 10$, $\rho = 2$ and $p = 1000$ give $h \le 26$. Compare this with the ordinary half-space VC dimension 1001. The restriction to labelings attainable with that margin is doing the work. It motivates a preference for large margins, but does not by itself supply a numerical test-error guarantee for one observed training set.

17.2 The hard-margin problem

For linearly separable data there are no slack variables. The construction chooses the orientation of the hyperplane so that the gap is as large as possible, with the support vectors on the boundary of that gap. The algebra that turns the largest gap into the smallest $\|\boldsymbol{w}\|$ is the derivation below. A factor $\frac{1}{2}$ in front of $\|\boldsymbol{w}\|^2$ does not move the hard-margin minimiser. It makes the derivative of the objective equal to $\boldsymbol{w}$, which is the form the Lagrange multiplier step wants.

Let the training set $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$, $\boldsymbol{x}_i \in \mathbb{R}^p$, $y_i \in \{-1, 1\}$, be separated by a hyperplane with margin ρ. Take a linear classifier with $\|\boldsymbol{w}\| = 1$. Then for each training example $(\boldsymbol{x}_i, y_i)$,

$$\boldsymbol{w}^\top \boldsymbol{x}_i + b \le -\rho/2 \ \text{ if } y_i = -1, \qquad \boldsymbol{w}^\top \boldsymbol{x}_i + b \ge \rho/2 \ \text{ if } y_i = 1 \qquad \Longleftrightarrow \qquad y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \ge \rho/2. \tag{17.3}$$

The constraint is a Euclidean width only together with a convention for $\|\boldsymbol{w}\|$. With $\|\boldsymbol{w}\| = 1$, ρ is the full distance between the parallel planes $y(\boldsymbol{w}^\top \boldsymbol{x} + b) = \pm\rho/2$. Without that normalisation the same algebra describes the width $\rho/\|\boldsymbol{w}\|$.

For every support vector $\boldsymbol{x}_s$ the inequality is an equality. After rescaling (dividing $\boldsymbol{w}$ and b by $\rho/2$, which does not alter the decision boundary), the distance between each $\boldsymbol{x}_s$ and the hyperplane is

$$r = \frac{y_s(\boldsymbol{w}^\top \boldsymbol{x}_s + b)}{\|\boldsymbol{w}\|} = \frac{1}{\|\boldsymbol{w}\|}, \tag{17.4}$$

and the new margin is $\rho' = 2r = 2/\|\boldsymbol{w}\|$ in terms of the rescaled $\boldsymbol{w}$ and b, with constraints $y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \ge 1$. Maximising $2/\|\boldsymbol{w}\|$ is equivalent to minimising $\|\boldsymbol{w}\|^2$:

Definition 17.3 (Primal problem, separable case)**.** Find $\boldsymbol{w}$ and b such that $\Phi(\boldsymbol{w}) = \frac{1}{2}\boldsymbol{w}^\top \boldsymbol{w}$ is minimised subject to $y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \ge 1$ for all $i = 1, \ldots, n$.

Factor $\frac{1}{2}$ in the objective.
The slides write the objective as $\Phi(\boldsymbol{w}) = \boldsymbol{w}^\top \boldsymbol{w}$ (and, in the soft-margin case, $\boldsymbol{w}^\top \boldsymbol{w} + C\sum_i \xi_i$) but then quote the dual $Q(\boldsymbol{\alpha}) = \sum_i \alpha_i - \frac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j \boldsymbol{x}_i^\top \boldsymbol{x}_j$ and the solution $\boldsymbol{w} = \sum_i \alpha_i y_i \boldsymbol{x}_i$. Those correspond to the objective $\frac{1}{2}\boldsymbol{w}^\top \boldsymbol{w}$ used above. In the separable case the factor does not change the minimiser, so the two forms are equivalent. In the soft-margin case it only rescales C: minimising $\boldsymbol{w}^\top \boldsymbol{w} + C\sum_i \xi_i$ gives $\boldsymbol{w} = \frac{1}{2}\sum_i \alpha_i y_i \boldsymbol{x}_i$ with $0 \le \alpha_i \le C$, i.e. it equals the $\frac{1}{2}\boldsymbol{w}^\top \boldsymbol{w} + C'\sum_i \xi_i$ problem with $C' = C/2$. This book uses the $\frac{1}{2}$ form so that the primal and dual match the slides' dual.

The feasible set is convex and the objective is strictly convex in $\boldsymbol{w}$. Thus the optimal weight vector is unique when the hard-margin problem is feasible, and every local optimum is global. Strict convexity in $\boldsymbol{w}$ does not establish uniqueness of all dual multipliers: several representations can give the same vector. The intercept is also unpenalised. These distinctions matter when comparing numerical output from different solvers (Boyd and Vandenberghe, 2004, Chapters 4–5).

17.2.1 The dual problem

The primal minimises a quadratic in $\boldsymbol{w}$ subject to one linear inequality per training point. The dual associates a Lagrange multiplier $\alpha_i \ge 0$ with each of those inequalities. The strategy is to form the Lagrangian, set the derivatives in the primal variables $\boldsymbol{w}$ and b to zero, and substitute those stationary conditions back. What remains is a function of the multipliers alone.

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{w}} = \boldsymbol{w} - \sum_i \alpha_i y_i \boldsymbol{x}_i = \boldsymbol{0} \;\Rightarrow\; \boldsymbol{w} = \sum_{i=1}^n \alpha_i y_i \boldsymbol{x}_i, \qquad \frac{\partial \mathcal{L}}{\partial b} = -\sum_i \alpha_i y_i = 0 \;\Rightarrow\; \sum_{i=1}^n \alpha_i y_i = 0. \tag{17.5}$$

Substituting back, $\frac{1}{2}\boldsymbol{w}^\top \boldsymbol{w} = \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \boldsymbol{x}_i^\top \boldsymbol{x}_j$, $\sum_i \alpha_i y_i \boldsymbol{w}^\top \boldsymbol{x}_i = \sum_{i,j} \alpha_i \alpha_j y_i y_j \boldsymbol{x}_i^\top \boldsymbol{x}_j$, and $b\sum_i \alpha_i y_i = 0$, so

$$\begin{aligned} \text{maximise} \quad & Q(\boldsymbol{\alpha}) = \sum_{i=1}^n \alpha_i - \frac{1}{2}\sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \, \boldsymbol{x}_i^\top \boldsymbol{x}_j \\ \text{subject to} \quad & \sum_{i=1}^n \alpha_i y_i = 0, \qquad \alpha_i \ge 0, \; i = 1, \dots, n. \end{aligned} \tag{17.6}$$

This is again a quadratic programme, now in $\boldsymbol{\alpha}$ (concave, since the matrix $(y_i y_j \boldsymbol{x}_i^\top \boldsymbol{x}_j)$ is positive semi-definite).

The Karush–Kuhn–Tucker conditions include *complementary slackness*, $\alpha_i[y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) - 1] = 0$ for every i. So either $\alpha_i = 0$, or $y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) = 1$, i.e. $\boldsymbol{x}_i$ lies exactly on the margin. Each non-zero α_i therefore identifies a support vector, and

$$\boldsymbol{w} = \sum_i \alpha_i y_i \boldsymbol{x}_i, \qquad b = y_k - \sum_i \alpha_i y_i \boldsymbol{x}_i^\top \boldsymbol{x}_k \;\text{ for any } k \text{ with } \alpha_k > 0. \tag{17.7}$$

(For a support vector $y_k(\boldsymbol{w}^\top \boldsymbol{x}_k + b) = 1$, and since $y_k^2 = 1$, $b = y_k - \boldsymbol{w}^\top \boldsymbol{x}_k$.) The classifying function is

$$f(\boldsymbol{x}) = \operatorname{sign}\Big(\sum_i \alpha_i y_i \, \boldsymbol{x}_i^\top \boldsymbol{x} + b\Big), \tag{17.8}$$

and one does not need $\boldsymbol{w}$ explicitly. Notice that it relies on an *inner product* between the test point $\boldsymbol{x}$ and the support vectors $\boldsymbol{x}_i$, and that solving the optimisation problem involved computing the inner products $\boldsymbol{x}_i^\top \boldsymbol{x}_j$ between all training points. Both facts are the key to non-linear SVMs. Strong duality holds: at the optimum $Q(\boldsymbol{\alpha}) = \frac{1}{2}\|\boldsymbol{w}\|^2$.

Strong duality is the convexity of the programme together with the existence of a strictly feasible point when the classes are linearly separable with a positive margin: the dual maximum equals the primal minimum. Complementary slackness, which is part of the Karush–Kuhn–Tucker conditions, is what identifies the support vectors. A multiplier that is zero drops its training point out of $\boldsymbol{w}$.

Example 17.4 (Three points)**. Question.** Find the maximum-margin separator of $\boldsymbol{x}_1 = (1, 0)$ with $y_1 = -1$ and $\boldsymbol{x}_2 = (3, 1)$, $\boldsymbol{x}_3 = (3, -1)$ with $y_2 = y_3 = +1$.

Solution. By symmetry, guess that all three are support vectors, so each constraint is an equality: $-(w_1 + b) = 1$, $3w_1 + w_2 + b = 1$, and $3w_1 - w_2 + b = 1$. The last two give $w_2 = 0$, and then $3w_1 + b = 1$ with $w_1 + b = -1$ gives $w_1 = 1$, $b = -2$. So $\boldsymbol{w} = (1, 0)^\top$, the boundary is $x_1 = 2$, and the margin is $\rho = 2/\|\boldsymbol{w}\| = 2$. To find $\boldsymbol{\alpha}$ use $\boldsymbol{w} = \sum_i \alpha_i y_i \boldsymbol{x}_i$: $-\alpha_1(1, 0) + \alpha_2(3, 1) + \alpha_3(3, -1) = (1, 0)$ gives $\alpha_2 = \alpha_3$ and $-\alpha_1 + 6\alpha_2 = 1$, and $\sum_i \alpha_i y_i = 0$ gives $-\alpha_1 + 2\alpha_2 = 0$. Hence $\alpha_1 = 2\alpha_2$ and $4\alpha_2 = 1$: $\boldsymbol{\alpha} = (\frac{1}{2}, \frac{1}{4}, \frac{1}{4}) > 0$, confirming that all three are support vectors. The dual objective is $Q = 1 - \frac{1}{2}\|\boldsymbol{w}\|^2 = \frac{1}{2}$, equal to the primal value $\frac{1}{2}\|\boldsymbol{w}\|^2 = \frac{1}{2}$. Checking (17.7): $b = y_1 - \sum_i \alpha_i y_i \boldsymbol{x}_i^\top \boldsymbol{x}_1 = -1 - (1, 0) \cdot (1, 0) = -2$. These primal feasibility, dual feasibility and objective-equality checks certify the proposed optimum; no numerical optimiser is needed for the argument.

17.3 Soft-margin classification

Overlapping classes can make the hard-margin constraints infeasible. Introduce a slack $\xi_i \geq 0$ to measure how much observation i falls short of functional margin one. The linear penalty $C \sum_i \xi_i$ prices those violations against $\frac{1}{2}\|\boldsymbol{w}\|^2$. It leads to a particularly simple change in the dual: each multiplier is bounded above by C. A squared-slack penalty gives a different quadratic term in the dual.

Definition 17.5 (Soft-margin primal problem)**.** Find $\boldsymbol{w}$ and b minimising $\Phi(\boldsymbol{w}, \boldsymbol{\xi}) = \frac{1}{2}\boldsymbol{w}^\top \boldsymbol{w} + C \sum_i \xi_i$ subject to $y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$ for all $i = 1, \ldots, n$.

Small C gives a relatively strong preference for a small weight norm, while large C places more emphasis on avoiding margin violations. For separable data, a sufficiently large C recovers a hard-margin solution. For inseparable data, increasing C cannot make a nonexistent hard-margin solution appear. Select the trade-off by validation, not by assuming that a larger value is always more accurate.

The dual is the same, with a box.

Introduce multipliers $\alpha_i \geq 0$ for the margin constraints and $\mu_i \geq 0$ for $\xi_i \geq 0$:

$$\mathcal{L} = \tfrac{1}{2}\boldsymbol{w}^\top \boldsymbol{w} + C \sum_i \xi_i - \sum_i \alpha_i \big[y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) - 1 + \xi_i \big] - \sum_i \mu_i \xi_i.$$

The derivatives with respect to $\boldsymbol{w}$ and b are as before, and $\partial \mathcal{L}/\partial \xi_i = C - \alpha_i - \mu_i = 0$. Since $\mu_i \geq 0$ this gives $\alpha_i \leq C$, and the ξ-terms cancel on substitution. Hence the dual problem is identical to the separable case except that the multipliers are bounded:

$$\text{maximise } Q(\boldsymbol{\alpha}) = \sum_i \alpha_i - \tfrac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \boldsymbol{x}_i^\top \boldsymbol{x}_j \quad \text{subject to} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C. \tag{17.9}$$

With squared slack, stationarity gives a relation involving ξ_i rather than the same constant upper bound on α_i. The issue is the changed objective, not a need for an additional species of multiplier. In the linear-slack problem, nonzero α_i identify support vectors, and

$$\boldsymbol{w} = \sum_i \alpha_i y_i \boldsymbol{x}_i, \qquad b = y_k(1-\xi_k) - \sum_i \alpha_i y_i \boldsymbol{x}_i^\top \boldsymbol{x}_k \;\text{ for any } k \text{ with } \alpha_k > 0, \tag{17.10}$$

and $f(\boldsymbol{x}) = \sum_i \alpha_i y_i \boldsymbol{x}_i^\top \boldsymbol{x} + b$, with no need to compute $\boldsymbol{w}$ explicitly. In (17.10) ξ_k is not known in advance. But complementary slackness gives $\mu_k \xi_k = 0$, and $\mu_k = C - \alpha_k > 0$ whenever $\alpha_k < C$, so $\xi_k = 0$ for any k with $0 < \alpha_k < C$. Such a margin support vector gives b as $b = y_k - \sum_i \alpha_i y_i \boldsymbol{x}_i^\top \boldsymbol{x}_k$, as in the separable case.

17.3.1 What the multipliers say

Complementary slackness gives three cases for a training point:

Multiplier	Position	Slack
$\alpha_i = 0$	$y_i f(\boldsymbol{x}_i) \geq 1$: outside the margin or on it, correctly classified	$\xi_i = 0$
$0 < \alpha_i < C$	$y_i f(\boldsymbol{x}_i) = 1$: exactly on the margin (a free support vector)	$\xi_i = 0$
$\alpha_i = C$	$y_i f(\boldsymbol{x}_i) \leq 1$: inside the margin or misclassified (a bound support vector)	$\xi_i = 1 - y_i f(\boldsymbol{x}_i) \geq 0$

So only points on or inside the margin influence the fit.

17.3.2 The hinge-loss view

At the optimum each ξ_i is as small as the constraints allow, so $\xi_i = \max\{0, 1 - y_i f(\boldsymbol{x}_i)\}$. Substituting that value removes the slacks and leaves a hinge loss plus a ridge penalty on $\boldsymbol{w}$. The hinge is the loss in use because it is zero once the margin exceeds 1: a point that is correctly classified and far from the boundary does not pull the fit. Squared error would keep pulling. The binomial deviance penalises errors roughly linearly, as the hinge does, but it never becomes exactly zero. The factor $1/(2C)$ plays the role of the regularisation parameter λ in weight decay (Chapter 16): small C is strong regularisation.

$$\sum_{i=1}^{n} \max\{0, 1 - y_i f(\boldsymbol{x}_i)\} + \frac{1}{2C}\|\boldsymbol{w}\|^2, \tag{17.11}$$

a *hinge loss* plus a ridge (L_2) penalty on $\boldsymbol{w}$ (compare weight decay, Chapter 16, and the loss functions of Figure 13.2). The hinge loss is zero for margins $yf \geq 1$ and grows linearly for smaller margins: like the binomial deviance it penalises errors linearly, and unlike squared error it does not penalise points that are correct with a large margin. The factor $1/(2C)$ plays the role of the regularisation parameter λ: small C is strong regularisation.

17.4 Non-linear SVMs and the kernel trick

A margin in the original coordinates cannot separate XOR, and it cannot separate a one-dimensional sample whose middle values belong to one class and whose two tails belong to the other. A map into a higher-dimensional feature space can turn either problem into a linear one. In one dimension, mapping $x \mapsto (x, x^2)$ separates the sandwiched classes with a line (Figure 17.3). For a finite training set in which no two identical inputs carry opposite labels, some feature map makes the training set linearly separable. The general construction is a map

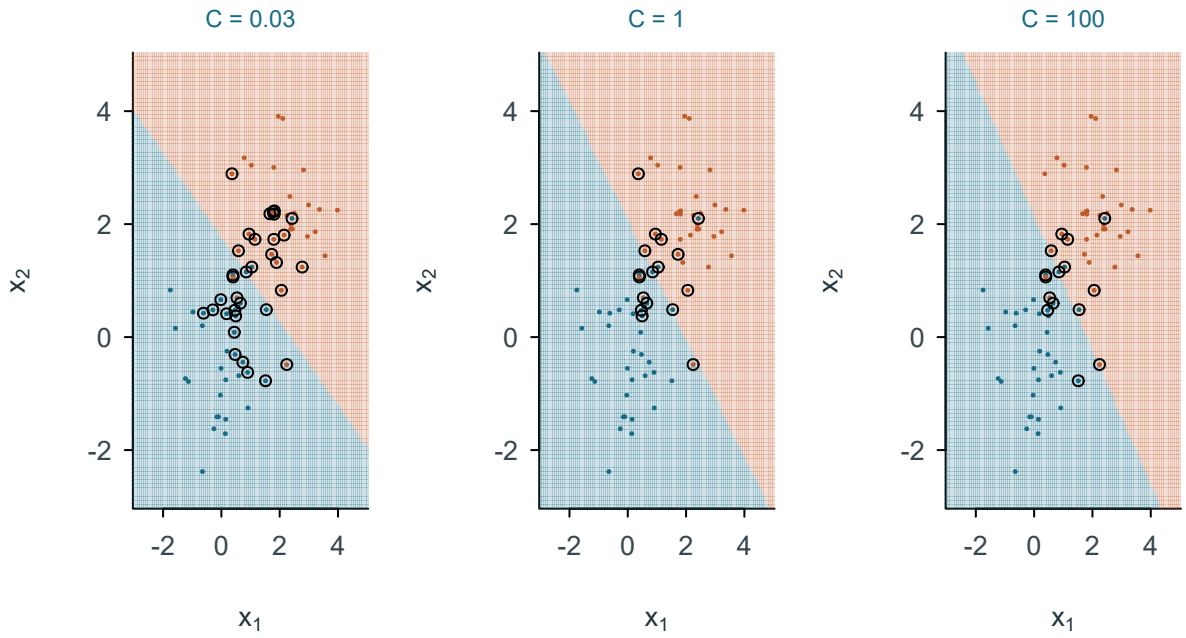


Figure 17.2: Linear SVM fits at three costs on the same overlapping Gaussian sample. Rings mark support vectors.

$$\varphi : \ \boldsymbol{x} \mapsto \varphi(\boldsymbol{x}). \tag{17.12}$$

17.4.1 The kernel trick

Both the dual problem (17.6) and the classifier (17.8) involve the data only through inner products. That is the opening the kernel uses. If every observation is mapped by φ, the inner product becomes $\varphi(\boldsymbol{x}_i)^\top \varphi(\boldsymbol{x}_j)$, and any function K that equals such an inner product can be written down without writing φ.

$$K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \varphi(\boldsymbol{x}_i)^\top \varphi(\boldsymbol{x}_j). \tag{17.13}$$

A *kernel function* is a function equivalent to an inner product in some feature space. The kernel trick replaces inner products in the feature space by a kernel on the original points. The feature map is never computed.

Example 17.6 (The quadratic kernel)**.** For 2-dimensional vectors $\boldsymbol{x} = (x_1, x_2)^\top$ let $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = (1 + \boldsymbol{x}_i^\top \boldsymbol{x}_j)^2$. Expanding,

$$\begin{aligned} K(\boldsymbol{x}_i, \boldsymbol{x}_j) &= 1 + x_{i1}^2 x_{j1}^2 + 2x_{i1}x_{j1}x_{i2}x_{j2} + x_{i2}^2 x_{j2}^2 + 2x_{i1}x_{j1} + 2x_{i2}x_{j2} \\ &= [1,\ x_{i1}^2,\ \sqrt{2}x_{i1}x_{i2},\ x_{i2}^2,\ \sqrt{2}x_{i1},\ \sqrt{2}x_{i2}] \\ &\quad \times [1,\ x_{j1}^2,\ \sqrt{2}x_{j1}x_{j2},\ x_{j2}^2,\ \sqrt{2}x_{j1},\ \sqrt{2}x_{j2}]^\top = \varphi(\boldsymbol{x}_i)^\top \varphi(\boldsymbol{x}_j), \end{aligned}$$

with $\varphi(\boldsymbol{x}) = [1,\ x_1^2,\ \sqrt{2}x_1x_2,\ x_2^2,\ \sqrt{2}x_1,\ \sqrt{2}x_2]^\top$. Numerically, for $\boldsymbol{x} = (1, 2)$ and $\boldsymbol{x}' = (3, 1)$: $K = (1 + 3 + 2)^2 = 36$, and $\varphi(\boldsymbol{x}) = (1, 1, 2.83, 4, 1.41, 2.83)$, $\varphi(\boldsymbol{x}') = (1, 9, 4.24, 1, 4.24, 1.41)$ have inner product $1 + 9 + 12 + 4 + 6 + 4 = 36$.

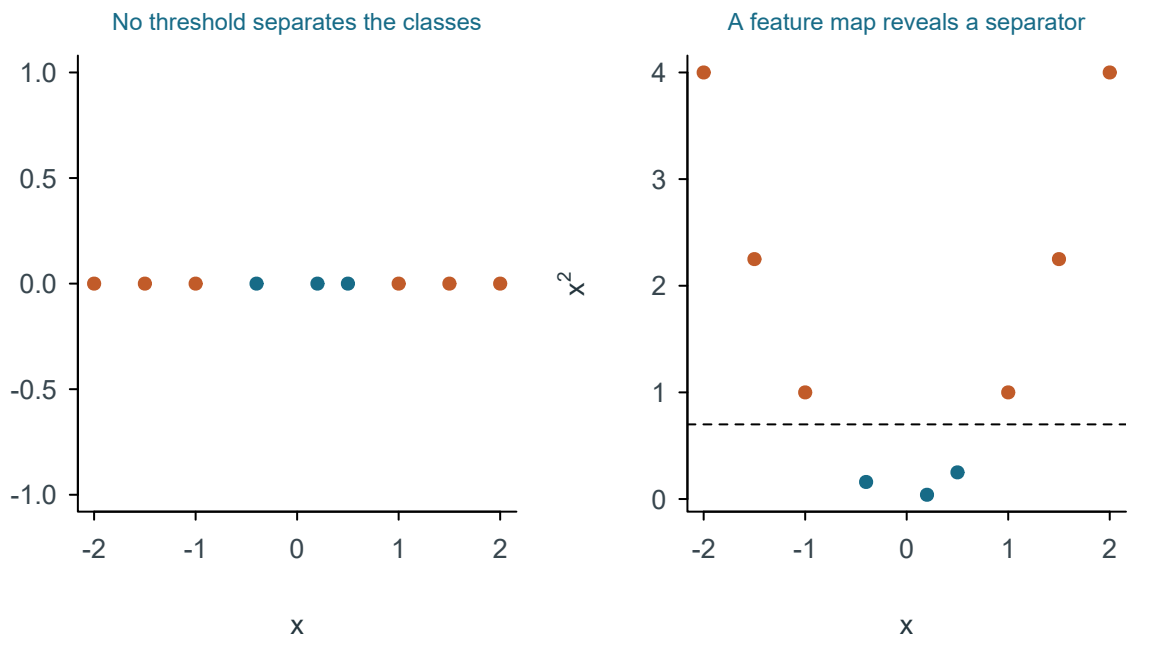


Figure 17.3: A quadratic feature map turns a nonseparable one-dimensional classification problem into a linearly separable one.

17.4.2 Which functions are kernels?

A proposed similarity function is not automatically an inner product. The necessary and sufficient kernel condition is symmetry and positive semidefiniteness of *every* finite Gram matrix. This yields a representation in a Hilbert feature space. Mercer's spectral theorem adds regularity and domain assumptions; the finite-Gram criterion below is the condition needed for the convex SVM dual (Burges, 1998).

$$\sum_{i=1}^{n}\sum_{j=1}^{n} c_i c_j K(\boldsymbol{x}_i, \boldsymbol{x}_j) \geq 0 \tag{17.14}$$

The inequality must hold for every finite choice of input points and every real vector $\boldsymbol{c}$. Equivalently, $\boldsymbol{K}_S = (K(\boldsymbol{x}_i, \boldsymbol{x}_j))_{ij}$ must be positive semidefinite. Checking one training Gram matrix checks that particular optimisation problem, not validity of the function on every possible dataset.

If K_1, K_2 are Mercer kernels, so are $\alpha K_1 + \beta K_2$ for $\alpha, \beta \geq 0$, the product $K_1 K_2$, and powers K^d for $d \in \mathbb{N}$ (the product statement is the Schur product theorem for entrywise products of positive semi-definite matrices). These closure properties let new kernels be built from old ones.

17.4.3 Common kernels

- *Linear:* $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \boldsymbol{x}_i^\top \boldsymbol{x}_j$, with $\varphi(\boldsymbol{x}) = \boldsymbol{x}$ itself.
- *Polynomial of power d:* $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = (1 + \boldsymbol{x}_i^\top \boldsymbol{x}_j)^d$, with $\varphi(\boldsymbol{x})$ of dimension $\binom{p+d}{d}$ (for $p = 2$, $d = 2$: $\binom{4}{2} = 6$, Example 17.6). The higher-dimensional space still has intrinsic dimensionality p (the map is not onto, so does not fill the higher-dimensional space), but linear separators in it correspond to non-linear separators in the original space.
- *Gaussian (radial basis function):* $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp(-\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2 / 2\sigma^2)$, with $\varphi(\boldsymbol{x})$ infinite-dimensional: every point is mapped to a function (a Gaussian), and a combination of such

functions for the support vectors is the separator. The Taylor expansion shows the infinite dimension:

$$\mathrm{e}^{-\|\boldsymbol{x}-\boldsymbol{x}'\|^2/2\sigma^2} = \mathrm{e}^{-\|\boldsymbol{x}\|^2/2\sigma^2}\,\mathrm{e}^{-\|\boldsymbol{x}'\|^2/2\sigma^2}\,\mathrm{e}^{\boldsymbol{x}^\top\boldsymbol{x}'/\sigma^2} = \mathrm{e}^{-\|\boldsymbol{x}\|^2/2\sigma^2}\mathrm{e}^{-\|\boldsymbol{x}'\|^2/2\sigma^2}\sum_{n=0}^{\infty}\frac{(\boldsymbol{x}^\top\boldsymbol{x}')^n}{\sigma^{2n}\,n!}, \tag{17.15}$$

a positive combination of the polynomial kernels $(\boldsymbol{x}^\top\boldsymbol{x}')^n$ of every degree n, times $a(\boldsymbol{x})a(\boldsymbol{x}')$, where $a(\boldsymbol{x}) = \exp[-\|\boldsymbol{x}\|^2/(2\sigma^2)]$. The paired product is a rank-one kernel; either one-variable factor by itself is not a two-input kernel.

17.4.4 The non-linear SVM

Replace each inner product by $K(\boldsymbol{x}_i, \boldsymbol{x}_j)$. The feature space may be infinite-dimensional, but the dual still has n coefficients. The computational cost depends on evaluating kernels, the sample size and the optimisation algorithm. We have avoided explicit feature coordinates, not made training cost independent of the kernel.

$$Q(\boldsymbol{\alpha}) = \sum_i \alpha_i - \tfrac{1}{2}\sum_{i,j}\alpha_i\alpha_j y_i y_j K(\boldsymbol{x}_i,\boldsymbol{x}_j) \quad \text{subject to} \quad \sum_i \alpha_i y_i = 0, \quad 0 \le \alpha_i \le C, \tag{17.16}$$

(the slides state the constraint $\alpha_i \ge 0$ for the hard-margin version and note that it "can be combined with the soft-margin approach if necessary", which adds the bound C). The solution is

$$f(\boldsymbol{x}) = \sum_i \alpha_i y_i K(\boldsymbol{x}_i,\boldsymbol{x}) + b, \qquad b = y_k - \sum_i \alpha_i y_i K(\boldsymbol{x}_i,\boldsymbol{x}_k) \ \text{ for any } k \text{ with } 0 < \alpha_k < C. \tag{17.17}$$

The optimisation techniques for finding the α_i remain the same.

Example 17.7 (Solving XOR with a quadratic kernel)**.** Return to XOR, now with the labels $y = \pm 1$ and points $\boldsymbol{x}_1 = (-1,-1)$ $(y=-1)$, $\boldsymbol{x}_2 = (-1,1)$ $(+1)$, $\boldsymbol{x}_3 = (1,-1)$ $(+1)$, $\boldsymbol{x}_4 = (1,1)$ (-1), the kernel $K(\boldsymbol{x},\boldsymbol{x}') = (1+\boldsymbol{x}^\top\boldsymbol{x}')^2$ of Example 17.6, and the hard-margin dual. The Gram matrix has $K_{ii} = (1+2)^2 = 9$ on the diagonal and, for $i \ne j$, $\boldsymbol{x}_i^\top\boldsymbol{x}_j \in \{0,-2\}$ so $K_{ij} = 1$:

$$\boldsymbol{K}_S = \begin{pmatrix} 9 & 1 & 1 & 1 \\ 1 & 9 & 1 & 1 \\ 1 & 1 & 9 & 1 \\ 1 & 1 & 1 & 9 \end{pmatrix}.$$

By symmetry take all $\alpha_i = \alpha$ (the constraint $\sum_i \alpha_i y_i = 0$ then holds since two labels are $+1$ and two -1). Then $\sum_{i,j} y_i y_j K_{ij} = 4\cdot 9 + \sum_{i\ne j} y_i y_j = 36 + \left[(\sum_i y_i)^2 - 4\right] = 36 + (0-4) = 32$, so $Q = 4\alpha - 16\alpha^2$, maximised at $\alpha = \tfrac{1}{8}$. The decision function is $f(\boldsymbol{x}) = \tfrac{1}{8}\sum_i y_i K(\boldsymbol{x}_i,\boldsymbol{x}) + b$. Using the feature map φ of Example 17.6,

$$\boldsymbol{w} = \tfrac{1}{8}\big[-\varphi(\boldsymbol{x}_1) + \varphi(\boldsymbol{x}_2) + \varphi(\boldsymbol{x}_3) - \varphi(\boldsymbol{x}_4)\big] = \tfrac{1}{8}\,(0,0,-4\sqrt{2},0,0,0)^\top = \big(0,0,-\tfrac{\sqrt{2}}{2},0,0,0\big)^\top,$$

because only the cross-product coordinate $\sqrt{2}x_1x_2$ differs in sign between the classes ($\sqrt{2}x_1x_2 = +\sqrt{2}$ for $\boldsymbol{x}_1,\boldsymbol{x}_4$ and $-\sqrt{2}$ for $\boldsymbol{x}_2,\boldsymbol{x}_3$). Then $f(\boldsymbol{x}) = -\tfrac{\sqrt{2}}{2}\cdot\sqrt{2}x_1x_2 + b = -x_1x_2 + b$, and the support-vector condition $y_1 f(\boldsymbol{x}_1) = 1$ with $\boldsymbol{x}_1 = (-1,-1)$ gives $-(-1+b) = 1$, so $b = 0$. Hence

$$f(\boldsymbol{x}) = -x_1x_2,$$

which classifies all four XOR points correctly with functional margin exactly 1 (all four are support vectors). Since $\|\boldsymbol{w}\| = 1/\sqrt{2}$, the geometric distance to the separating hyperplane in feature space is $\sqrt{2}$ and the full slab width is $2\sqrt{2}$. The rule classifies a new point $(0.5, -0.5)$ as $+1$ ($f = 0.25$) and $(2, 2)$ as -1 ($f = -4$). The hand-built hidden layers of Chapter 14 compute AND and OR; the kernel supplies, in the cross-product coordinate, exactly the feature that makes XOR linearly separable. The KKT equalities and the positive multipliers verify the solution directly. (Figure 17.4.)

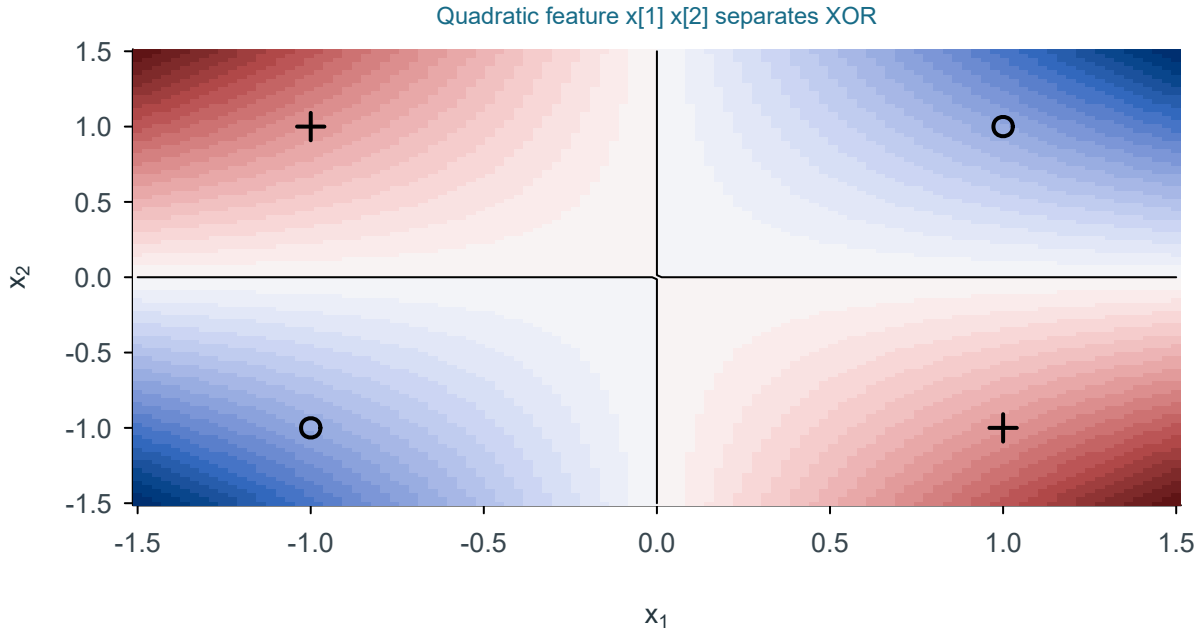


Figure 17.4: The product feature separates centred XOR inputs. The zero contours are the coordinate axes.

17.5 Practice, applications, and extensions

Computation.

Decomposition methods optimise subsets of the dual coefficients while preserving feasibility. Sequential Minimal Optimization updates a pair at a time, which allows the equality $\sum_i \alpha_i y_i = 0$ to be maintained (Platt, 1999). A fully stored Gram matrix uses n^2 entries, but implementations can cache or recompute selected entries instead. Thus quadratic storage is a property of full-matrix storage, not a requirement of every SVM implementation.

Tuning.

Tune the kernel, C and any kernel width inside the training data. Refit scaling within each validation fold because distances and inner products depend on units. For the Gaussian kernel, $\gamma = 1/(2\sigma^2)$: small σ makes similarity local. As $\sigma \to \infty$ at fixed inputs, however, $K(\boldsymbol{x}, \boldsymbol{x}') \to 1$, the constant kernel. A centred and rescaled leading-order expansion contains a linear inner product, but that is a different limiting argument and can require rescaling C. The unqualified statement that the RBF kernel tends to the linear kernel is incorrect.

Applications and extensions.

Kernels can encode similarities for sequences, graphs and other structured inputs, provided the resulting kernel meets the required positivity condition. Related constructions include support

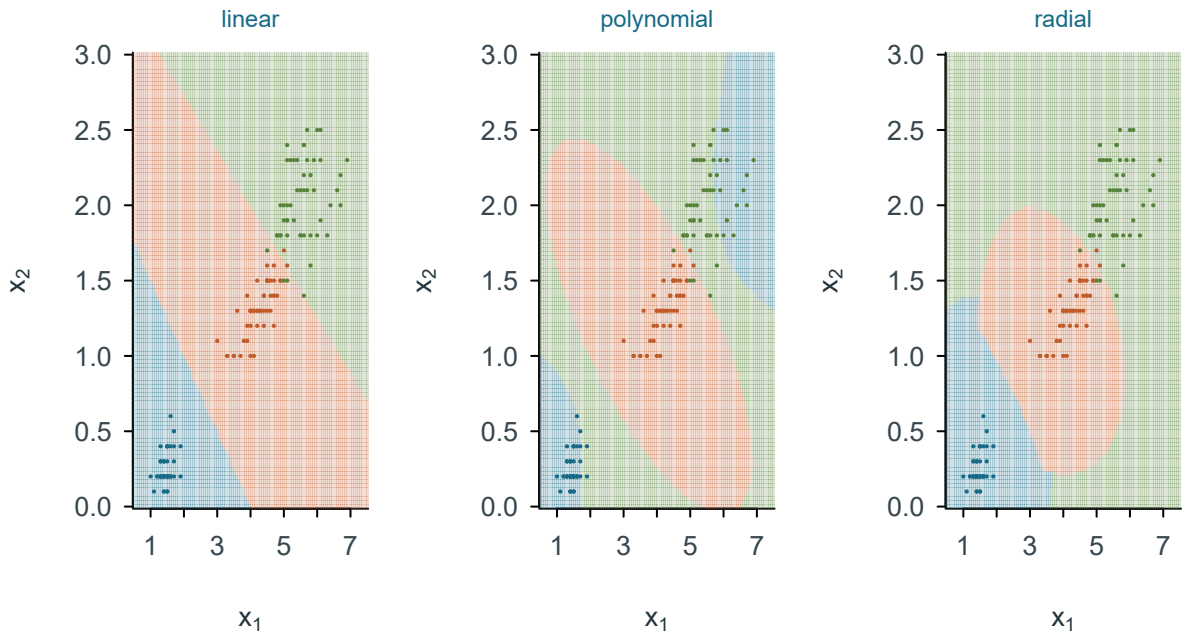


Figure 17.5: Linear, degree-two polynomial and radial SVM regions for iris petals. This is a descriptive comparison at fixed illustrative settings.

vector regression and kernel PCA (Schölkopf et al., 1999; Burges, 1998). Comparative accuracy must be assessed for the actual representation, tuning procedure and target population.

Comparison with other classifiers.

The convex SVM objective and a network's nonconvex objective have different optimisation geometry. Convexity does not make a fitted classifier insensitive to data or kernel choices, so SVM ensembles can still differ through resampling, features and tuning. An SVM produces a decision score; its sign gives a binary label. For a linear SVM with $\boldsymbol{w} \neq 0$, divide the score by $\|\boldsymbol{w}\|$ to obtain signed distance. A probability requires a separate calibration model assessed on data not used to fit that calibration. Multiclass fitting also requires a construction, such as one-versus-rest or pairwise classifiers.

Key idea.
The SVM chooses the separating hyperplane with the largest margin $2/\|\boldsymbol{w}\|$. The solution is a combination $\sum_i \alpha_i y_i \boldsymbol{x}_i$ of the training points whose multipliers are nonzero, found from a convex quadratic programme that involves the data only through inner products. Replacing those inner products by a kernel $K(\boldsymbol{x}, \boldsymbol{x}')$ gives nonlinear classifiers in the original coordinates. The constant C prices margin violations against the squared norm.

17.6 R laboratory: tune an RBF SVM inside training data

Cross a small grid of costs and kernel widths. Standardisation is recomputed in each fold and internal SVM scaling is disabled so the transformation is explicit. The held-out fold is used only to score a candidate (C, γ). That is the cross-validation discipline of Chapter 9: the labels that assess a setting are not the labels that train it.

```
fold <- stratified_folds(train$Species, 5, 17006)
grid <- expand.grid(cost = c(0.1, 1, 10), gamma = c(0.1, 0.5, 2))
grid$CV_error <- apply(grid, 1, function(pars) {
  pred <- character(nrow(train))
  for (f in 1:5) {
    a <- fold != f; zz <- scale_train(train[a, 1:4], train[!a, 1:4])
    fit <- e1071::svm(zz$train, train$Species[a], kernel = "radial",
                      cost = pars[1], gamma = pars[2], scale = FALSE)
    pred[!a] <- as.character(predict(fit, zz$test))
  }
  mean(pred != train$Species)
})
print(grid)
#>   cost gamma CV_error
#> 1  0.1   0.1  0.15238
#> 2  1.0   0.1  0.03810
#> 3 10.0   0.1  0.04762
#> 4  0.1   0.5  0.07619
#> 5  1.0   0.5  0.02857
#> 6 10.0   0.5  0.02857
#> 7  0.1   2.0  0.07619
#> 8  1.0   2.0  0.04762
#> 9 10.0   2.0  0.06667
best <- grid[which.min(grid$CV_error), ]
svm_fit <- e1071::svm(iris_scaled$train, train$Species,
  kernel = "radial", cost = best$cost, gamma = best$gamma, scale = FALSE)
mean(predict(svm_fit, iris_scaled$test) != test$Species)
#> [1] 0.06667
```

The grid is intentionally modest. The tuning score estimates a procedure trained on less data than the final fit, and minimising it adds selection optimism. A kernel decision score is not automatically a calibrated probability.

17.7 Further exercises

Exercise 17.1 (Distance to a hyperplane)**.** For $\boldsymbol{w} = (3, 4)^\top$ and $b = -5$, compute the signed distance of $(2, 1)$ and $(0, 0)$ from the hyperplane $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$, and the margin between $\boldsymbol{w}^\top \boldsymbol{x} + b = \pm 1$.

Solution

Exercise 17.2 (A two-point SVM)**.** Points $\boldsymbol{x}_1 = (3, 3)$ with $y_1 = +1$ and $\boldsymbol{x}_2 = (1, 1)$ with $y_2 = -1$. Solve the dual (17.6): find $\alpha_1 = \alpha_2$, $\boldsymbol{w}$, b, and the margin. Verify that the margin equals the distance between the points.

Solution

Exercise 17.3 (Which points matter?)**.** In Example 17.4 add a fourth point $\boldsymbol{x}_4 = (6, 0)$ with $y_4 = +1$. Does the separator change? What are the new support vectors and α_4?

Solution

Exercise 17.4 (Kernel as inner product)**.** Show that $K(x, x') = (1 + xx')^2$ for scalars equals $\varphi(x)^\top \varphi(x')$ with $\varphi(x) = (1, \sqrt{2}x, x^2)$. Using it, map the one-dimensional data of Figure 17.3 to (φ_2, φ_3) and explain why a linear classifier in φ-space can separate the classes.

Solution

Exercise 17.5 (Gram matrix)**.** For the Gaussian kernel with $\sigma = 1$ and the points $0, 1, 2$ in one dimension, compute the Gram matrix and show that it is positive semi-definite by checking that its leading principal minors are positive (or its eigenvalues are non-negative).

Solution

Exercise 17.6 (Interpreting multipliers)**.** In a soft-margin SVM with $C = 2$ three training points have $\alpha = 0$, $\alpha = 1.3$, and $\alpha = 2$. For each, state where the point is relative to the margin and its slack. Which can be used to compute b?

Solution

Exercise 17.7 (A tempting wrong turn)**.** A practitioner fits a Gaussian-kernel SVM with a very large γ and C and reports 100% training accuracy, concluding the problem is solved. Explain what has probably happened and how to check it.

Solution

Exercise 17.8 (Read the SVM geometry correctly)**.** A hard-margin linear SVM uses the canonical constraints $y_i(w^T x_i + b) \geq 1$. For a fitted model, $w = (3, 4)^T$ and $b = -10$.

(a) Explain w and b, and give the decision-boundary equation.

(b) Find the full distance between the two margin hyperplanes.

(c) Explain what distinguishes support vectors and determine whether a positive observation at $(2, 1)^T$ satisfies the hard-margin constraint.

Solution

Exercise 17.9 (Interpret an SVM constraint)**.** For two linearly separable classes, suppose $\|w\| = 1$, $y_i \in \{-1, +1\}$ and

$$y_i(w^T x_i + b) \geq \rho/2, \qquad \rho > 0.$$

The unit-norm convention is explicit in this question.

(a) Explain the geometric meaning of w, b and ρ.

(b) Give the equation of the decision boundary and the prediction rule away from it.

(c) Explain what distinguishes support vectors from the remaining training observations and how they relate to the maximum margin.

Solution

Exercise 17.10 (Nonlinear separation)**.** The four XOR observations $(0, 0), (1, 0), (0, 1), (1, 1)$ are labelled $-1, +1, +1, -1$. Consider the feature map $\phi(x) = (x_1, x_2, x_1 x_2)^T$.

(a) Explain why a linear separator in the original two predictors is impossible.

(b) Show that $f(x) = x_1 + x_2 - 2x_1 x_2 - 1/2$ correctly classifies the four observations by its sign.

(c) Explain how a linear separator in feature space can have a nonlinear boundary in the original space, and what a kernel supplies to an SVM.

Solution

Part V

High-dimensional analysis and graphical models

Chapter 18

High-dimensional analysis

With four measurements on each iris flower, estimating a covariance matrix is manageable. Now imagine measuring thousands of gene-expression features on a few dozen tissue samples. More columns do not create more independent observations. Some familiar formulas stop being defined, and searching many possible signals creates a second difficulty: even noise gets plenty of opportunities to look interesting.

Two questions organise this chapter. First, what remains identifiable when p is large relative to n? Second, what error does a selection procedure control? A singular covariance prevents ordinary inverse-covariance formulas from being used. Least squares still has a solution, but its coefficients may not be unique. Prediction error, recovery of nonzero coefficients and false-discovery rate are distinct targets; success at one does not establish success at the others.

We begin with the rank obstruction, then consider PCA, multiple testing and penalised regression. Each changes the problem in a different way: PCA chooses a lower-dimensional representation, multiple-testing procedures control specified error rates, and regularisation adds a preference among competing fits. The preference is part of the model, so it deserves an explanation.

Source. Lecture notes, Chapter 4 (pp. 118–138).

18.1 Why classical methods fail when $p > n$

Dempster (1958) pointed out that neither Hotelling's T^2 test to compare two groups (Chapter 2) nor Fisher's LDA (Chapter 8) can be used when $p > n_1 + n_2 - 2$. In both cases the difficulty is in fitting a multivariate normal distribution in this number of dimensions: the procedures need a covariance inverse, and the sample covariance does not have one.

Prediction from a regularised fit can still have small error. Attribution of that error to individual coefficients is not identified, because many coefficient vectors produce the same fitted values once p exceeds the rank of the design. The rank argument below is why the inverse fails.

18.1.1 The rank of the sample covariance matrix

For a single data set with $p > n - 1$ the sample covariance matrix is singular, hence not invertible. The $n \times p$ data matrix $\boldsymbol{X}_{\text{all}}$ has rank at most $\min(n, p)$. With $\boldsymbol{A} = \sum_{i=1}^{n} (\boldsymbol{x}_i - \overline{\boldsymbol{x}})(\boldsymbol{x}_i - \overline{\boldsymbol{x}})^\top$ and $\boldsymbol{S} = \frac{1}{n-1}\boldsymbol{A}$, the matrices $\boldsymbol{A}$ and $\boldsymbol{S}$ have the same rank. Each observation can contribute one dimension, and centring spends one of them: the rows of the centred matrix sum to zero. The proof writes that dependence down.

Proposition 18.1 (Rank bound)**.** $\operatorname{rank}(\boldsymbol{S}) \leq \min(n-1, p)$. Hence $\boldsymbol{S}$ is invertible only if $p \leq n-1$ (and $\boldsymbol{X}_{\text{all}}$ has rank p after centring).

Proof. Let $\boldsymbol{X}_c$ be the centred data matrix with rows $(\boldsymbol{x}_i - \overline{\boldsymbol{x}})^\top$, so that $\boldsymbol{A} = \boldsymbol{X}_c^\top \boldsymbol{X}_c$ and $\operatorname{rank}(\boldsymbol{A}) = \operatorname{rank}(\boldsymbol{X}_c)$. The rows of $\boldsymbol{X}_c$ sum to zero, $\sum_i (\boldsymbol{x}_i - \overline{\boldsymbol{x}}) = \boldsymbol{0}$, so they are linearly dependent and span a subspace of dimension at most $n-1$. Also $\operatorname{rank}(\boldsymbol{X}_c) \leq p$.

□

The key dependence is $\boldsymbol{1}^\top \boldsymbol{X}_c = \boldsymbol{0}^\top$: each *column* of the centred matrix lies in the $(n-1)$-dimensional subspace of $\mathbb{R}^n$ perpendicular to $\boldsymbol{1}$. Equivalently, its rows sum to zero. Centring each of g nonempty groups gives the pooled within-group rank bound $n - g$. For $n = 100$ and $g = 2$, the full sample can have at most 99 positive principal variances, while a pooled covariance has rank at most 98. Neither upper bound guarantees full rank. In cross-validation, calculate these bounds using the training fold.

Example 18.2 (A singular covariance matrix)**.** With three observations and five variables, $\boldsymbol{S}$ has rank at most two. At least three eigenvalues are zero. Substituting this matrix into ordinary Mahalanobis distance, Hotelling's T^2 or an unregularised Gaussian discriminant rule therefore fails. The Gaussian likelihood itself is still defined for a candidate positive-definite population covariance; the problem is that its unrestricted covariance MLE is not a positive-definite matrix. For QDA, $n_g - 1 \geq p$ is necessary in each class. For LDA with G nonempty classes, $n - G \geq p$ is necessary for an invertible pooled estimate. Collinearity can make either estimate singular even when those inequalities hold.

18.1.2 Regression

The same rank obstruction appears in the normal equations. The least-squares coefficient for a linear model with an intercept is

$$\widehat{\boldsymbol{\beta}} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}, \tag{18.1}$$

where $\boldsymbol{y}$ is the vector of continuous responses and $\boldsymbol{X}$ the $n \times (p+1)$ matrix of explanatory variables augmented by a column of ones.

Transposes in the normal equations.
The notes print the least-squares and ridge solutions as $(\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X} \boldsymbol{y}$ and $(\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I})^{-1} \boldsymbol{X} \boldsymbol{y}$ in Section 4.4, with the transpose missing on the last $\boldsymbol{X}$ (Section 4.1 has it correctly). The dimensions force $\boldsymbol{X}^\top \boldsymbol{y}$: $\boldsymbol{X}$ is $n \times (p+1)$ and $\boldsymbol{y}$ is $n \times 1$, so $\boldsymbol{X} \boldsymbol{y}$ is undefined. The lasso paper is cited in the notes as 1994; the published version is Tibshirani (1996).

The inverse formula requires full column rank, not merely a favourable count of rows and columns. When rank fails, least squares still projects $\boldsymbol{y}$ onto the column space of $\boldsymbol{X}$, giving a unique fitted vector. The coefficient representation is non-unique: adding any $\boldsymbol{v}$ with $\boldsymbol{X}\boldsymbol{v} = 0$ leaves the fitted values unchanged. Exact interpolation occurs when $\boldsymbol{y}$ lies in that column space; full row rank is sufficient. Having $p \geq n$ alone is not sufficient.

18.1.3 Where high-dimensional data come from

A large sample relative to the number of fitted coefficients makes classical estimation easier, but collecting another subject can cost much more than measuring another feature on an existing sample. High-dimensional methods address this imbalance by adding structure. They cannot manufacture information about directions that the observed design never distinguishes.

Microarray studies are a course-relevant example: one tissue sample produces measurements of many transcript-related features, while the number of sampled tissues can remain small. The

Alon study below analyses 2000 selected gene-expression features from 62 samples (Alon et al., 1999). These measurements are not direct measurements of protein production, and a biological gene is not interchangeable with one independent predictor. Correlation, measurement processing and tissue composition all affect the statistical question.

In that setting the scientific question is often which coordinates are associated with the response, not the size of the prediction error. Association is not causation.

18.1.4 What do we want to know?

The choice of methods depends greatly on the question to be answered. The main possibilities are:

1. Which of the predictor variables are associated with the response variable? (a) Considering each variable separately; (b) considering the set of explanatory variables all at once.
2. Can the set of explanatory variables be transformed into a space of smaller dimension without losing anything important in terms of the association with the response?
3. Can the set of explanatory variables be transformed into a space of smaller dimension when there is no response variable? (a) When the aim is to capture structure in the data; (b) when the aim is clustering.
4. How can we make the highest-quality predictions of the response variable based on the explanatory variables?

Selecting a subset of coordinates is one restricted form of dimension reduction, but finding scientific associations and finding a predictive representation need not give the same answer. We study unsupervised PCA first, then marginal testing and joint penalised regression. Keep the target question visible when comparing their outputs.

18.2 Principal component analysis for high-dimensional data

Standard PCA can be used when $p > n$, with the rank bound already in hand. Since $\boldsymbol{S}$ is symmetric, all its eigenvalues are real. Its rank is at most $n-1$ (Proposition 18.1), so the number of non-zero eigenvalues is at most $n-1$, and the number of principal components is limited to $\min(n-1, p)$, which equals $n-1$ when $p > n$. n vectors in a p-dimensional space span at most an n-dimensional subspace, and PCA loses one degree of freedom through the sample mean.

Later principal components can separate groups when the first does not. PC1 is the direction of greatest variance. It is not automatically the scientific direction: the leading components often follow the bulk of the variation, and a later component may be the one aligned with a group difference.

Computing PCA when $p \gg n$.

For $p \gg n$, forming a full $p \times p$ covariance can waste memory and arithmetic. Write the compact SVD $\boldsymbol{X}_c = \boldsymbol{U}_r \boldsymbol{E}_r \boldsymbol{V}_r^\top$, where $r = \operatorname{rank}(\boldsymbol{X}_c) \leq n-1$ and all diagonal entries e_i of $\boldsymbol{E}_r$ are positive. The nonzero eigenvalues of $\boldsymbol{X}_c \boldsymbol{X}_c^\top$ are $e_i^2 = (n-1)\lambda_i$. Recover the corresponding loading as $\boldsymbol{a}_i = \boldsymbol{X}_c^\top \boldsymbol{u}_i / e_i$. Division by a zero singular value is not a way to recover an additional component. An economy SVD avoids explicitly constructing either a large covariance or a zero-variance basis (R Core Team, 2026a).

The $(n-1)$-dimensional PCA basis can be an extreme summary of a small sample. The main purpose of PCA is to reduce dimensionality, often well below $\min(n-1, p)$, which reduces the room to over-fit. Other methods may require this before they can be applied. To fit the covariance matrices, QDA requires the dimension of the data to be less than the sample size of the smallest

class, while LDA estimates a pooled covariance and so uses all the data. There have been many extensions of principal components, including sparse methods for high-dimensional data.

18.3 Single-variable analysis

In many studies the scientific question is which coordinates are associated with the response, not the size of a prediction error. Association is not causation. With two groups, one with a disease and one healthy, the question may be which explanatory variables have a different distribution in each group. With a continuous response, it may be which variables are most strongly related to it. The variables that survive are candidates for further study.

The simplest approach is to work through the explanatory variables one by one. When the response is nominal and the explanatory variable is continuous, ANOVA is the usual test if its conditions are met: independence of observations, approximate normality within each group, and similar variances. A transformation sometimes achieves that. When it cannot, a rank test such as Kruskal–Wallis asks for less. With two groups, Welch's t-test allows unequal group variances. When both variables are nominal, a chi-squared or Fisher exact test is the usual choice. For a continuous response and a categorical explanatory variable, ANOVA again; when both are continuous, linear regression, perhaps after transformation. The result is a p-value for every explanatory variable. The difficulty is the threshold.

18.3.1 Why the usual threshold fails

If m_0 null hypotheses are true and each has an exactly uniform null p-value, testing each at 0.05 gives $\mathbb{E}(V) = 0.05m_0$. Independence is not needed for this expectation: use linearity on the rejection indicators. Valid conservative p-values instead satisfy $\mathbb{P}(P_i \leq t) \leq t$, giving an upper bound. Uniformity requires a correctly calibrated continuous null test; continuity of a reported statistic alone is not enough. With 20,000 true nulls, the expected false-rejection count is therefore 1000.

The family-wise error rate (FWER) is controlled at a level such as 0.05. Solutions such as the Bonferroni method exist, and they are often far too stringent when p is in the thousands. The next example is the arithmetic of that stringency. No particular value of a null p-value, including a very small one, is specially expected: under a continuous null the p-value is uniform.

Example 18.3 (The cost of Bonferroni)**.** If $n = 100$ and $p = 20\,000$, the Bonferroni method for a family-wise error rate of 0.05 requires an individual threshold of $0.05/20\,000 = 2.5 \times 10^{-6}$. For a two-sample t-test with equal variances and 98 degrees of freedom this requires $|t| \geq 5.00$. With 50 observations in each group, $t = \dfrac{\overline{x}_1 - \overline{x}_2}{s\sqrt{2/50}}$, so $|t| \geq 5.00$ requires a difference between group means of $5.00 \times s \times 0.2 = 1.00\,s$: at least as large as the standard deviation within a group. This arithmetic concerns the observed standardised difference, not the power for a specified population effect. Moderate effects may fail to clear the threshold. An exploratory plot does not account for the 20,000 opportunities to select a striking difference.

It is often difficult to know how the set of variables found will be used, but sometimes the intended use is known. A laboratory may want to investigate up to 30 genes and like a list of those that are most promising. In one sense the task is then easy: sort the variables by their p-values and take the smallest 30. (If p-values are not sufficiently accurate, one can simply sort by the absolute value of the test statistic, for two-sided tests of one type.) The problem is that we do not then know whether we are wasting people's time. Are any of the 30 relevant? Where should we draw the line in a principled fashion?

18.3.2 The false discovery rate

False discovery rate control offers a different error criterion (Benjamini and Hochberg, 1995). It concerns the expected fraction of false rejections among all rejections. We now write m for the number of tests, keeping p available for a p-value only when there is no ambiguity.

Table 18.1: Number of errors in the testing of m null hypotheses (after a threshold has been chosen).

	Declared non-significant	Declared significant	Total
H_0 true	U	V	M_0
H_0 false	T	S	M_1
Total	$m - R$	R	m

Table 18.1 separates the unknown truth from the random decisions. In this frequentist formulation the set of true nulls, and hence M_0 and M_1, is fixed but unknown. The counts U, V, T, S, R vary across repeated datasets. For $M_0 > 0$, the average null rejection probability is

$$\bar{\alpha} = \frac{\mathbb{E}(V)}{M_0}. \tag{18.2}$$

This average need not equal a common nominal level for every test. When $M_1 > 0$, the corresponding average type II error probability and power are

$$\bar{\beta} = \frac{\mathbb{E}(T)}{M_1}, \qquad \text{average power} = 1 - \bar{\beta} = \frac{\mathbb{E}(S)}{M_1}, \qquad M_1 > 0. \tag{18.3}$$

The family-wise error rate is $\mathrm{FWER} = \mathbb{P}(V \geq 1)$. The *false discovery proportion* is $Q = V/R$ when $R > 0$ and $Q = 0$ when $V = R = 0$: the proportion of the variables declared significant that are in fact null. (The notes describe Q as the proportion of "alternative hypotheses declared significant which are false", which is the same quantity: a declared-significant hypothesis is a false discovery when its null hypothesis is true.) The *false discovery rate* is its expected value,

$$\mathrm{FDR} = \mathbb{E}(Q) = \mathbb{E}\Big(\frac{V}{R} \,\Big|\, R > 0\Big)\mathbb{P}(R > 0). \tag{18.4}$$

Set the false discovery proportion to zero when there are no rejections. This convention makes FDR well-defined even under the complete null. In contrast, $\mathbb{E}(V/R \mid R > 0)$ conditions on making at least one discovery. The two quantities differ by the factor $\mathbb{P}(R > 0)$, which can be small. Neither quantity bounds the false fraction in every individual realised list.

Proposition 18.4 (FDR is controlled by FWER control)**.** When all null hypotheses are true ($m_0 = m$), FDR = FWER. In general FDR $\leq$ FWER.

Proof. If all nulls are true, $S = 0$ and so $V = R$. If $V = 0$ then $Q = 0$; if $V \geq 1$ then $Q = 1$. So $\mathbb{P}(V \geq 1) = \mathbb{E}(Q)$. In general, claim $\mathbb{I}(V \geq 1) \geq Q$: if $V = 0$ both sides are 0; if $V \geq 1$ the left side is 1 and $Q = V/R \leq 1$. Taking expectations, $\mathbb{P}(V \geq 1) \geq \mathbb{E}(Q)$.

□

Any procedure that controls (places an upper bound on) the FWER therefore controls the FDR. However, if one controls just the FDR, the FWER can be considerably higher. This may be of no concern, and the benefit can be a large increase in the power of the overall procedure.

18.3.3 The Benjamini–Hochberg procedure

Assume we are testing m null hypotheses $H_1, \dots, H_m$ whose p-values are $P_1, \dots, P_m$ (upper case, since before the data are collected they are random variables). Order them $P_{(1)} \le P_{(2)} \le \cdots \le P_{(m)}$ and let $H_{(i)}$ be the hypothesis whose p-value is $P_{(i)}$.

Algorithm 18.5 (Benjamini–Hochberg)**.** Let k be the largest i for which $P_{(i)} \le \dfrac{i}{m} q^*$. Then reject all $H_{(i)}$, $i = 1, \dots, k$ (reject none if no such i exists).

Theorem 18.6 (Benjamini and Hochberg, 1995)**.** Suppose the p-values are mutually independent and each true-null p-value is valid: $\mathbb{P}(P_i \le t) \le t$ for $0 \le t \le 1$. The BH procedure at $q^* \in (0, 1)$ satisfies FDR $\le (m_0/m)q^* \le q^*$. Equality holds in the first inequality for exactly uniform null p-values (Benjamini and Hochberg, 1995; Benjamini and Yekutieli, 2001).

The notes state the theorem without proof. Here is a short proof for independent p-values with exactly uniform null p-values.

The denominator R depends on all the tests, including the one whose false-rejection probability we want to compute. Remove that dependence by setting the selected null p-value to zero. This gives a rejection count determined only by the other p-values. The useful cancellation will then be between its appearance in the threshold and in the denominator.

Proof. Write $q = q^*$ and let $\mathcal{N}$ be the true-null set. For $i \in \mathcal{N}$, let R_i be the BH rejection count after replacing P_i by zero. Then $R_i \ge 1$. If H_i is rejected, lowering its already-rejected p-value cannot create a new crossing at an index greater than R, so $R_i = R$. Conversely, if $P_i \le qR_i/m$, restoring P_i retains at least R_i values below that threshold. It cannot increase the count above the count obtained with zero, so $R = R_i$ and H_i is rejected. Hence

$$\frac{V}{R \vee 1} = \sum_{i \in \mathcal{N}} \frac{\mathbb{I}(P_i \le qR_i/m)}{R_i}.$$

Condition on all p-values except P_i. Independence and exact null uniformity give $\mathbb{P}(P_i \le qR_i/m \mid R_i) = qR_i/m$, so each summand has expectation q/m. Summing gives FDR $= m_0 q/m$. For super-uniform null p-values, replace equality by an upper bound in the conditional probability calculation.

□

Independence is sufficient, not necessary. Benjamini and Yekutieli (2001, Theorem 1.2) establish BH control under a particular positive-dependence condition, PRDS on the true nulls; arbitrary correlation is not that condition. Their Theorem 1.3 gives an arbitrary-dependence correction for valid marginal p-values: replace q^* by $q^*/c(m)$, where $c(m) = \sum_{j=1}^{m} 1/j$. Thus reject through the largest i satisfying $P_{(i)} \le iq^*/[mc(m)]$. The price of the broader guarantee is a smaller threshold.

Example 18.7 (BH by hand)**.** Ten p-values, sorted: 0.0002, 0.0009, 0.0041, 0.0121, 0.0230, 0.0302, 0.0420, 0.2400, 0.5200, 0.8900. With $m = 10$ and $q^* = 0.05$ the BH thresholds iq^*/m are 0.005, 0.010, 0.015, …, 0.050. Comparing: $p_{(5)} = 0.0230 \le 0.025$ holds, $p_{(6)} = 0.0302 \le 0.030$ fails by a hair, and $p_{(7)} = 0.0420 > 0.035$, and so on. The largest i meeting the condition is $k = 5$: reject the five smallest. Bonferroni at $0.05/10 = 0.005$ would reject only 3 $(0.0002, 0.0009, 0.0041)$, and the unadjusted rule at 0.05 would reject 7. The Benjamini–Yekutieli thresholds are those divided by $c(10) = 2.929$ and give $k = 3$. Note that the rule takes the *largest* i for which $p_{(i)}$ lies on or below the line iq^*/m and rejects everything up to it; in this example every earlier p-value is also below its own threshold, but this need not happen (Exercise 18.4).

Marginal testing can identify several correlated coordinates because each has an association with the response. A lasso fit instead considers them jointly and may retain only some interchangeable

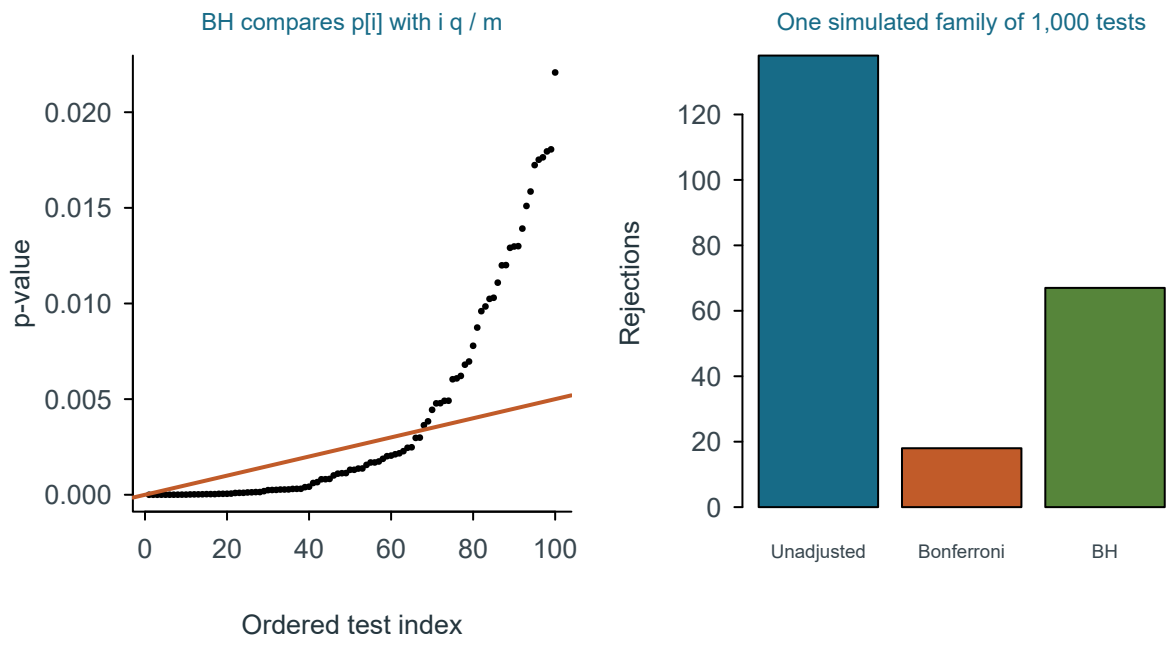


Figure 18.1: One simulated multiple-testing family: ordered p-values against the BH line, and rejection counts for three rules.

predictors. Neither procedure is guaranteed to select an entire correlated group, or exactly one representative. Their objectives differ before any particular selection occurs.

18.4 Variable selection and the lasso

Single-variable tests ignore the other explanatory variables. If the selected variables are also going to be used for prediction, the fit has to be joint: each coefficient is the association that remains after the others are in the model. Variable selection is a special case of model selection (Chapter 6). Only two types of model are considered here: multiple linear regression for a continuous response and logistic regression for a binary response. The hope is a small model that predicts well. With p large, exhaustive search over 2^p subsets can exceed a practical computation budget, and the normal equations may not have a unique solution. A penalty that can set coefficients to exact zeros is the device that does both shrinkage and selection in one convex problem.

18.4.1 Subset selection and shrinkage

As seen with neural networks, one way of limiting or avoiding over-fitting is to penalise the magnitude of the parameters, which are called *shrinkage* methods. As seen with BIC, another way is to penalise the number of parameters. Another way to choose a subset of explanatory variables for a model begins with a method of assessing predictive accuracy: the cross-validation-based PRESS statistic for a continuous response, or classical cross-validation for a nominal response (Chapter 9). We then search over all possible combinations of the available variables; with p explanatory variables the number of possible subsets is 2^p (for $p = 30$ this is more than 10^9, and for $p = 100$ about 10^{30}).

Greedy searches make a manageable sequence of comparisons: forward selection adds one variable at a time, while backward selection removes one from an initially fitted model. That reduces the search, but does not guarantee the best subset. Nor does a better training fit settle the prediction question. For nested least-squares models, training R^2 cannot decrease as variables are added.

Selection therefore needs a criterion that accounts for complexity or evaluates predictions on observations excluded from the relevant fitting and selection steps.

Best subset selection fixes the number of selected variables at some value $k < p$ and searches over this smaller domain. It can be viewed as a hard-constraint version of BIC, or as a method that uses an L_0 "norm" penalty, which counts the non-zero elements:

$$\|\boldsymbol{\beta}\|_{L_0} = \big|\{j : \beta_j \neq 0\}\big|. \tag{18.5}$$

This is not formally a norm since, for example, absolute homogeneity does not hold in general: $\|a\boldsymbol{\beta}\|_{L_0} \neq |a|\,\|\boldsymbol{\beta}\|_{L_0}$ for most scalars a and vectors $\boldsymbol{\beta}$.

18.4.2 Least squares, ridge regression, and the lasso

For standard linear regression use the model $Y = \beta_0 + \boldsymbol{X}^\top\boldsymbol{\beta} + \epsilon$ with $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_p)^\top$ the vector of regression slopes, β_0 the constant, and $\epsilon \sim \mathrm{N}(0, \sigma^2)$. We are not interested in shrinking or removing β_0 or σ^2: they must be present in all models of interest. The maximum likelihood solution is the least-squares solution,

$$(\widehat{\beta}_0, \widehat{\boldsymbol{\beta}}) = \underset{\beta_0,\boldsymbol{\beta}}{\arg\min} \Big\{ \sum_{i=1}^{n} \Big(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij}\Big)^2 \Big\}, \tag{18.6}$$

which has the inverse-based closed form (18.1) when the augmented design has full column rank.

Ridge regression.

A long-standing shrinkage method is *ridge regression* (also weight decay or Tikhonov regularisation) (Hoerl and Kennard, 1970). One typically standardises the predictors first (subtract each sample mean and divide by the sample standard deviation) and adds as a penalty the sum of squared regression weights, an L_2 penalty. Do not run one λ on variables that live on different scales. Divide a variable by 1000 and its coefficient has to grow by 1000 to give the same fit. The same λ is then a different penalty, because a large coefficient dominates the sum. Standardise first.

$$(\widehat{\beta}_0, \widehat{\boldsymbol{\beta}}) = \underset{\beta_0,\boldsymbol{\beta}}{\arg\min} \Big\{ \sum_{i=1}^{n} \Big(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij}\Big)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \Big\}. \tag{18.7}$$

Here λ is a tuning parameter controlling the amount of shrinkage, which can be chosen manually, by trial and error, or by optimising a measure of predictive accuracy such as cross-validation. Note that we would need something like two layers of cross-validation to both select such a parameter and estimate the accuracy of the resulting model (Chapter 9). Ridge regression was introduced primarily to improve regression for non-orthogonal predictors, i.e. those with substantial multicollinearity, where the slope estimates are unstable with large confidence intervals and high sensitivity to individual observations. This is also seen as near-singularity of $\boldsymbol{X}^\top\boldsymbol{X}$. The ridge estimator is

$$\widehat{\boldsymbol{\beta}}_R = (\boldsymbol{X}^\top\boldsymbol{X} + \lambda\boldsymbol{I})^{-1}\boldsymbol{X}^\top\boldsymbol{y}, \tag{18.8}$$

(with centred $\boldsymbol{y}$ and standardised $\boldsymbol{X}$ so that the intercept is $\bar{y}$), which reduces the condition number of the matrix to be inverted. If $p > n$ then $\boldsymbol{X}^\top\boldsymbol{X}$ is rank-deficient, so not invertible; but it is positive semi-definite, so at least one eigenvalue is zero and all the others are nonnegative. Adding $\lambda\boldsymbol{I}$ raises every eigenvalue by $\lambda > 0$, making the matrix positive definite and invertible, and hence allowing a unique solution $\widehat{\boldsymbol{\beta}}_R$ even when $p > n$.

The lasso.

Tibshirani (1996), building in part on Donoho and Johnstone (1994) and Breiman (1995), replaces the L_2 penalty by an L_1 penalty. The resulting estimator is the *lasso*, the "least absolute shrinkage and selection operator":

$$(\widehat{\beta}_0, \widehat{\boldsymbol{\beta}}) = \arg\min_{\beta_0, \boldsymbol{\beta}} \Big\{ \sum_{i=1}^{n} \Big(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \Big)^2 + \lambda \sum_{j=1}^{p} |\beta_j| \Big\}. \tag{18.9}$$

This is the Lagrangian equivalent of minimising the same sum of squares subject to $\sum_j |\beta_j| \le t$, with a corresponding constraint radius t for a penalised solution. This correspondence need not be one-to-one: flat portions of the solution path and inactive constraints can give multiple parameter values for the same fit. The method aims to shrink some parameter values and set others to exactly zero, hence achieving both shrinkage and variable selection. With standardised data the solution for $\widehat{\beta}_0$ is $\bar{y}$, and one then only needs to choose $\beta_1, \ldots, \beta_p$.

18.4.3 Why the L_1 penalty gives zeros

Figure 18.2 compares the lasso and ridge regression. The ellipses are contours of the least-squares cost, centred at the unpenalised estimate. The constraint sets are $|\beta_1| + |\beta_2| \le t$ and $\beta_1^2 + \beta_2^2 \le t^2$. The penalised estimate is the point where the lowest contour still touches the constraint. The L_1 set is a diamond. A smooth ellipse tends to touch it at a corner, and a corner sets one coefficient to zero. The L_2 set is a disk. The ellipse meets the disk at a smooth point, and both coefficients stay nonzero. Sparsity depends on the penalty, the loss, and the data. High dimension alone does not force the right variables to be the ones selected.

The diamond does not force the optimum to a corner on every dataset. It makes corners available over a range of loss contours, which is the mechanism for exact zeros. Ridge can have a zero coefficient through cancellation or symmetry, but its smooth penalty does not create a thresholding region around zero.

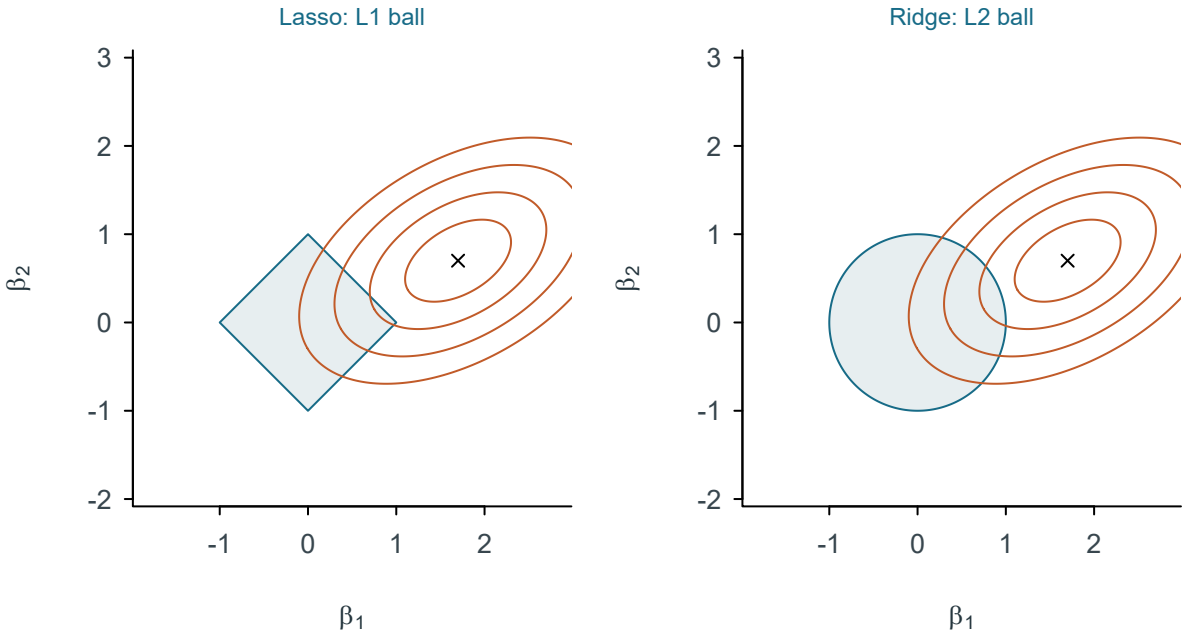


Figure 18.2: L1 and L2 constraint sets with quadratic loss contours. Corners of the L1 ball make exact zero coefficients possible.

Exact solutions with orthonormal predictors.

When $\boldsymbol{X}^\top \boldsymbol{X} = \boldsymbol{I}$, the penalty separates across coordinates. For ridge, the derivative of $(\beta_j - \widehat{\beta}_j^{\mathrm{LS}})^2 + \lambda\beta_j^2$ is zero at $\widehat{\beta}_j = \widehat{\beta}_j^{\mathrm{LS}}/(1+\lambda)$: every coefficient shrinks by the same factor. For the lasso, minimise $(\beta_j - b)^2 + \lambda|\beta_j|$ with $b = \widehat{\beta}_j^{\mathrm{LS}}$. For $\beta_j > 0$ the derivative is $2(\beta_j - b) + \lambda$, zero at $\beta_j = b - \lambda/2$, which is positive only if $b > \lambda/2$; similarly for $\beta_j < 0$. If $|b| \le \lambda/2$ the minimum is at 0. Hence

$$\widehat{\beta}_j^{\mathrm{lasso}} = \mathrm{sign}(b)\big(|b| - \tfrac{\lambda}{2}\big)_+, \tag{18.10}$$

the *soft-thresholding* operator: coefficients smaller than $\lambda/2$ in magnitude are set to zero, and the others are shrunk towards zero by $\lambda/2$ (Figure 18.3). Best subset selection with a penalty λ on $\|\boldsymbol{\beta}\|_{L_0}$ gives *hard* thresholding: keep b unchanged if $|b| > \sqrt{\lambda}$ and set it to 0 otherwise. At $|b| = \sqrt{\lambda}$, both 0 and b minimise the criterion; the displayed rule chooses zero. For example, with least-squares estimates $(3, -1.5, 0.8, -0.3, 0.1)$ and $\lambda = 1$: ridge gives $(1.5, -0.75, 0.4, -0.15, 0.05)$ (all non-zero), the lasso gives $(2.5, -1.0, 0.3, 0, 0)$, and hard thresholding gives $(3, -1.5, 0, 0, 0)$. The soft-threshold also shows the lasso's *bias*: even a coefficient as large as the 3 is shrunk by $\lambda/2$.

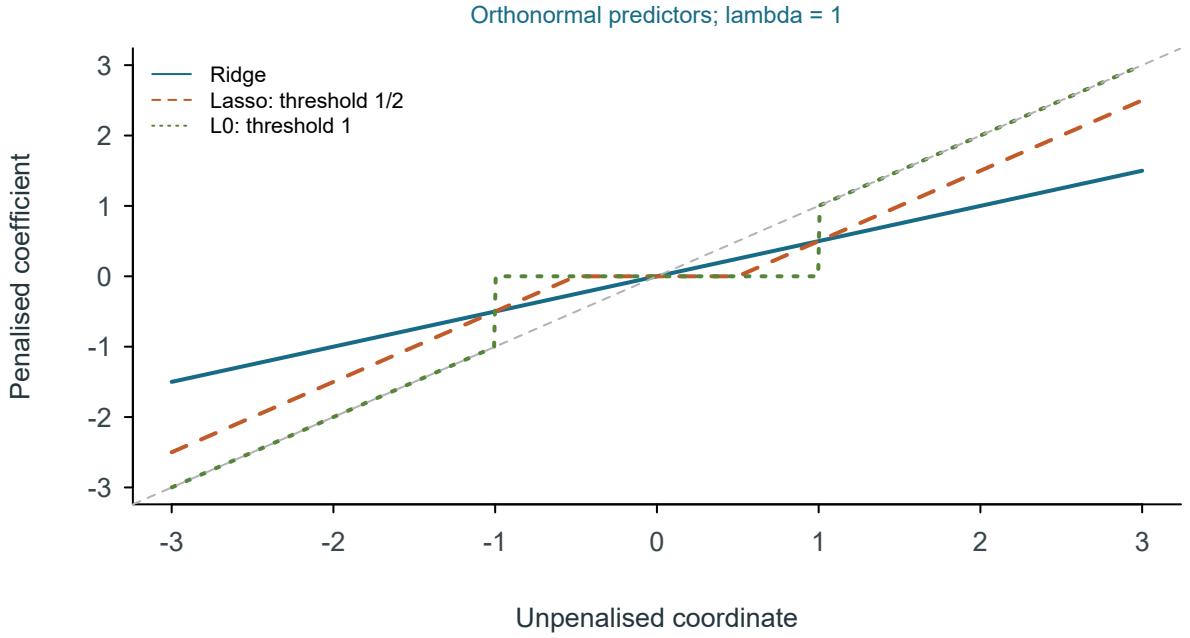


Figure 18.3: Orthonormal predictors under the chapter objective SSE plus lambda times the penalty, with lambda = 1. Lasso thresholds at 1/2, ridge shrinks by 1/2, and L0 selection thresholds at 1.

18.4.4 Choosing λ and the solution path

The penalty determines a trade-off, so it must be chosen for an objective. Prediction-oriented tuning can use cross-validation with the entire fitting procedure repeated within each fold. The broader family $\lambda \sum_j |\beta_j|^q$ connects several familiar penalties: $q = 2$ gives ridge, $q = 1$ gives lasso, and counting nonzero coefficients gives the L_0 formulation. The mathematical appeal of $q = 1$ is concrete: it retains convexity while introducing a corner at zero. For $0 < q < 1$, the penalty is nonconvex. These geometric facts do not by themselves select a penalty value (Tibshirani, 1996; Frank and Friedman, 1993).

A path algorithm computes fits over a range of penalties. Each fixed-λ least-squares lasso problem is convex. Under the usual nondegeneracy conditions, coefficients vary piecewise linearly with

λ, with changes when variables enter or leave the active set. Calling the *path itself* convex is a different, generally unjustified claim. Efficient path algorithms exploit this structure; their workload still depends on the design and the number of changes (Hastie et al., 2009, Section 3.4).

Read a coefficient path as a sequence of fitted models, not a ranking of biological importance. A coefficient can change sign, reach zero or later re-enter. The figure below uses a seeded simulation so that every plotted value is generated by the accompanying R code. Its cross-validation curve selects a prediction trade-off; it does not certify support recovery.

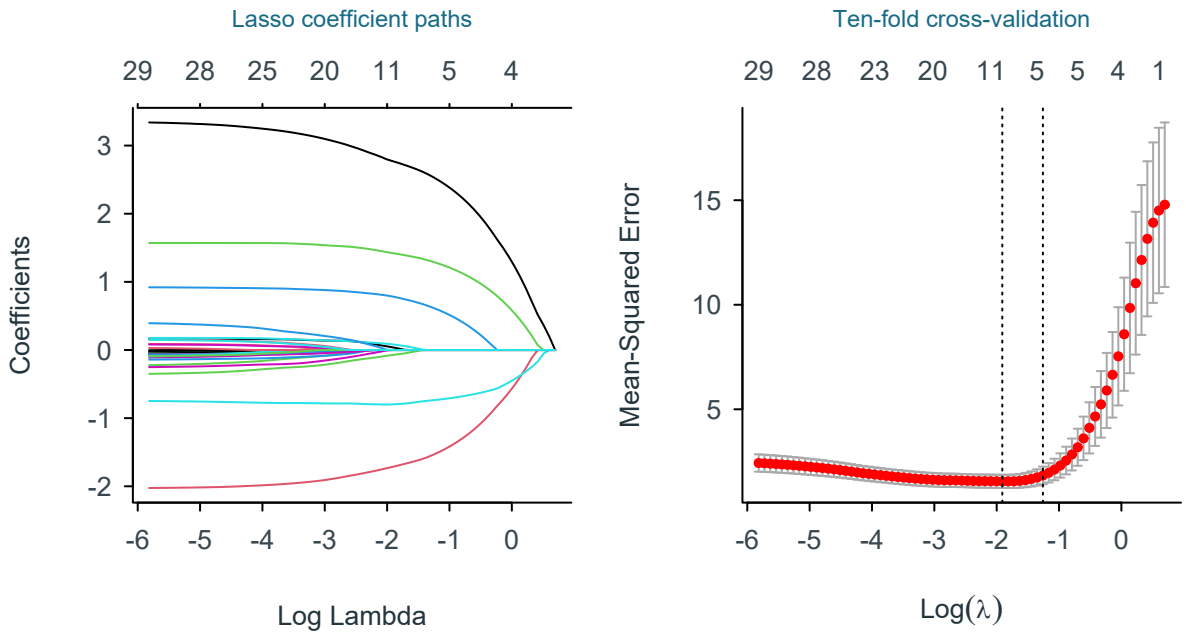


Figure 18.4: A seeded sparse regression simulation: lasso coefficient paths and ten-fold CV with its reported standard errors.

18.4.5 Extensions

The lasso can also be used with any generalised linear model, one with $g(\mu(\boldsymbol{x})) = \boldsymbol{x}^\top \boldsymbol{\beta}$ where $\mu(\boldsymbol{x}) = \mathbb{E}(Y \mid \boldsymbol{X} = \boldsymbol{x})$, g is monotone, and Y has a distribution from the exponential family. In particular it applies to logistic regression: minimise the negative log-likelihood (10.7) plus $\lambda \sum_j |\beta_j|$, with λ chosen by cross-validation of the binomial deviance (R package `glmnet`).

The lasso left a number of problems and possible extensions as a base for further research, which was quickly taken up by many mathematical statisticians. One issue is that the lasso is a shrinkage method: an L1 penalty constrains the overall coefficient size, but with correlated predictors an individual coefficient need not be smaller in magnitude than its least-squares counterpart, so the lasso estimator is biased. Is there a way to reduce or eliminate this bias, at least as $n \to \infty$? Assume a lasso is fit with λ chosen by cross-validation. This typically removes many variables. Refitting an unpenalised model on the same data the lasso selected is biased. A sample the lasso did not see is the clean check; only about half the data is then used for each of selection and fitting.

Another option, proposed by Zou (2006), is the *adaptive lasso*, which alters the penalty to

$$(\widehat{\beta}_0, \widehat{\boldsymbol{\beta}}) = \arg\min_{\beta_0, \boldsymbol{\beta}} \Big\{ \sum_{i=1}^{n} \Big(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \Big)^2 + \lambda \sum_{j=1}^{p} w_j |\beta_j| \Big\}, \tag{18.11}$$

The displayed version uses $w_j = 1/|\widetilde{\beta}_j|$ from a pilot fit; the general adaptive lasso permits powers $|\widetilde{\beta}_j|^{-\gamma}$ with $\gamma > 0$. Large pilot coefficients receive less shrinkage. A zero pilot requires an explicit rule, such as exclusion or a stated stabilisation. Zou (2006, Theorem 2) give oracle properties under assumptions on the design, a sufficiently accurate pilot and the rate of the penalty sequence. These are not guarantees for any cross-validated penalty or any marginal-regression pilot, particularly with correlated predictors and growing dimension. Selection consistency and coefficient consistency must be checked under the stated asymptotic regime.

Key idea.
High dimension makes rank and selection assumptions consequential. Least-squares fitted values still exist, while coefficient identification can fail. Ridge supplies a unique penalised coefficient vector; lasso can produce sparse fits. BH controls an expected false-discovery proportion under its calibration and dependence assumptions. None of these facts says that the variables selected in one dataset are exactly the true ones.

18.5 Example application: the Alon colon-cancer microarray data

Alon et al. (1999) analysed expression measurements for 2000 selected genes from 62 colon tissue samples, including 40 tumour and 22 normal samples. The processed file used for the course analysis is not supplied here. Consequently, the numerical PCA, testing and lasso results below are retained as reported course results, not independently reproduced results from the original paper. Centring and scaling a feature to mean zero and variance one does not make it normally distributed. Figures 18.1 and 18.4 provide executable synthetic illustrations of the methods.

PCA.

The first ten principal components were examined; there were three motives: one or more components might fortuitously pick up variation associated with the class difference; one can look for outliers, including possibly mislabelled observations; and the dimension reduction allows the full suite of classifiers to be constructed. A plot of PC1 against PC2 offered almost no separation between the classes and indicated no outliers. Welch t-tests of each component between the classes gave the following p-values:

Component	1	2	3	4	5	6	7	8	9	10
p-value	0.479	0.339	1.8×10^{-7}	0.0995	0.0958	0.0166	0.346	0.84	0.915	0.431

PC3 appears very aligned with the class differences, with PC6 also promising. The scatter plot of PC3 against PC6 separates the classes substantially, although with some overlap; a simple classifier such as QDA in these two dimensions might separate them fairly accurately. But the labels were used in choosing these dimensions, so the analysis would need to be redone with left-out data to estimate classification accuracy honestly (the selection-bias point of Chapter 9). Since the initial interest was in what differs between the classes, the genes with the largest magnitude in the PC3 and PC6 direction vectors were listed (top 20 of each). There were no genes in common between these two lists.

FDR.

A Welch t-test was run for each gene, comparing the tumour and normal groups, using BH at nominal level 0.01. Under the required null calibration and dependence assumptions, this targets an expected false-discovery proportion at most 1%; it does not cap the false fraction in this realised gene list. The Benjamini–Hochberg procedure declared 173 genes significant. Twelve of the 20 top

PC3 genes were among them (1635, 493, 625, 765, 1511, 1634, 1153, 992, 1293, 1002, 1959, 609), and one of the top PC6 genes (286).

Lasso.

The lasso was used to perform model selection at the same time as fitting a logistic regression of the class on all the genes. The binomial deviance was the cost function, with λ chosen as the value with the lowest binomial deviance on left-out data by cross-validation (package `glmnet`). This selected 14 gene indices: 14, 249, 377, 493, 576, 625, 792, 1360, 1473, 1582, 1679, 1772, 1843, 1924, with fitted coefficients (in the same order) -0.111, -0.081, -0.588, -0.275, 0.136, 0.118, -0.116, 0.089, 0.113, 0.085, 0.141, 0.302, -0.207, and -0.141: the largest in magnitude are gene 377 (-0.588) and gene 1772 ($+0.302$), and the smallest is gene 249 (-0.081). Eleven of these were on the FDR list (14, 249, 377, 493, 625, 1473, 1582, 1679, 1772, 1843, 1924) and two were on the PC3 list (493, 625).

Which genes to hand back?

The reported BH list contains 173 genes. A laboratory able to follow up only ten might prioritise the overlap with the lasso list and then apply an additional ranking. The course example retains genes $14, 377, 493, 625, 1473, 1582, 1679, 1772, 1843, 1924$ after removing gene 249 from the eleven-gene overlap. This is a prioritisation rule, not a new FDR theorem: an arbitrary data-dependent subset of BH discoveries need not inherit the original FDR bound. Replication, measurement quality and biological follow-up can test the candidates. The current data do not establish a causal role for them.

A principal-component analysis of the Alon matrix followed by a lasso on the component scores selects combinations of all the genes, since each component is already a combination of all 2000 genes. That does not answer which genes. A lasso on the genes themselves is the fit that matches the question.

18.6 R laboratory: a sparse simulation where the truth is known

Iris has only four predictors, so it is a poor example of a genuinely high-dimensional problem. Instead simulate 100 training observations and 150 predictors, of which five have nonzero coefficients. Independent test observations separate prediction error from support recovery. Cross-validation chooses λ inside the training sample.

`glmnet` normalises its loss differently from the chapter's raw SSE objective. Compare fitted procedures and the definition of the penalty, not bare numerical values of λ across conventions (Hastie et al., 2026).

```
set.seed(18006)
n <- 100; p <- 150
beta <- c(3, -2, 1.5, 1, -1, rep(0, p - 5))
X <- matrix(rnorm(n * p), n, p)
y <- drop(X %*% beta + rnorm(n))
Xnew <- matrix(rnorm(500 * p), 500, p)
ynew <- drop(Xnew %*% beta + rnorm(500))
fold <- sample(rep(1:10, length.out = n))
lasso <- glmnet::cv.glmnet(X, y, alpha = 1, foldid = fold)
pred <- drop(predict(lasso, Xnew, s = "lambda.1se"))
b <- as.numeric(coef(lasso, s = "lambda.1se"))[-1]
selected <- which(abs(b) > 1e-8)
c(test_MSE = mean((pred - ynew)^2), selected = length(selected),
```

```
  true_positives = sum(selected <= 5), false_positives = sum(selected > 5))
#>        test_MSE        selected  true_positives false_positives
#>           1.534           6.000           5.000           1.000
```

The coefficient pattern is known because the simulation specified it. Prediction error, false inclusions, and missed signals are different outcomes. A single simulated dataset illustrates the distinction. It does not establish a universal selection guarantee, and it is not a false-discovery calculation: the lasso was not asked to control an expected proportion of false inclusions.

18.7 Further exercises

Exercise 18.1 (Rank)**.** Let $n = 4$ observations in $p = 6$ dimensions. What is the maximum rank of the sample covariance matrix $\boldsymbol{S}$, and how many of its eigenvalues are necessarily zero? How many principal components with positive variance can there be? If the data are instead $n = 4$ in $p = 3$ dimensions, what is the maximum rank?

Solution

Exercise 18.2 (Ridge makes it invertible)**.** $\boldsymbol{X} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ ($n = 2$ observations, two perfectly collinear predictors). Show that $\boldsymbol{X}^\top \boldsymbol{X}$ is singular, find its eigenvalues, and compute the ridge estimate $(\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$ for $\boldsymbol{y} = (1, 3)^\top$ and $\lambda = 1$.

Solution

Exercise 18.3 (Bonferroni threshold)**.** With $m = 5000$ tests and $\alpha = 0.05$, find the Bonferroni threshold and the required $|z|$ for a two-sided normal test (use $\Phi^{-1}(1 - 10^{-5}/2) = 4.42$). What would the BH thresholds be for the 5th and the 100th smallest p-values at $q^* = 0.05$?

Solution

Exercise 18.4 (A step-up subtlety)**.** Five sorted p-values 0.001, 0.025, 0.026, 0.027, 0.800, $q^* = 0.05$, $m = 5$. Apply BH: list iq^*/m and find k. Is $H_{(2)}$ rejected even though $p_{(2)} = 0.025$ exceeds its own threshold $2(0.05)/5 = 0.02$? Why does the procedure reject all hypotheses up to k even if some earlier p-values exceed their own thresholds?

Solution

Exercise 18.5 (Soft threshold)**.** Orthonormal predictors, with least-squares estimates 2.4, -0.9, 0.5, and -0.1. Compute the ridge estimates for $\lambda = 2$, and the lasso estimates (18.10) for $\lambda = 2$. Which variables are selected by the lasso?

Solution

Exercise 18.6 (A tempting wrong turn)**.** A researcher screens 20 000 genes, picks the 20 with the smallest p-values, fits a logistic regression on these using all 62 samples, and reports the cross-validated error from this model (4%). Explain why this estimate is optimistic and how to correct the procedure.

Solution

Exercise 18.7 (When ordinary LDA is not defined)**.** There are $n = 90$ independent labelled observations, 30 in one class and 60 in another, with $p = 500$ numerical predictors. You propose PCA followed by ordinary, unregularised LDA.

(a) Explain a fitting difficulty and an evaluation difficulty for this dataset.

(b) Give upper bounds on the number of nonzero sample principal components and on the dimension in which the pooled within-class covariance can be invertible. Explain the difference.

(c) Describe how to choose and evaluate the PCA dimension, and explain why a feasible dimension need not be a good predictive dimension.

Solution

Exercise 18.8 (PCA before high-dimensional LDA)**.** There are 72 observations with 400 continuous predictors: 24 observations in class 1 and 48 in class 2. Ordinary LDA is proposed for future classification.

(a) Explain one difficulty in fitting LDA and one in estimating its predictive performance here.

(b) Explain how PCA could make fitting feasible. Give the maximum number of nonzero sample PCs and the necessary upper bound on the retained dimension for an invertible pooled within-class covariance. Explain how to choose that dimension.

Solution

Exercise 18.9 (Selection in many dimensions)**.** You have 200 observations and 1000 predictors. In a simulation, only the first five coefficients in a linear conditional-mean model are nonzero.

(a) Explain why cross-validated lasso need not select exactly those five variables, even if it predicts well.

(b) You also test 1000 separate marginal regressions. Explain the Benjamini-Hochberg procedure at target FDR 0.1, define the error criterion, and distinguish marginal association from a nonzero coefficient in the joint model.

Solution

Chapter 19

Graphical models

Two measurements can move together because both track a third variable. To ask whether any dependence remains, we must specify what is being conditioned on. An undirected graphical model records conditional-independence restrictions through missing edges. In a nonsingular Gaussian model, these restrictions have an especially useful form: a missing edge corresponds to a zero off-diagonal entry of the precision matrix $\boldsymbol{K} = \boldsymbol{\Sigma}^{-1}$ (Lauritzen, 1996, Chapter 5).

An undirected edge is not a causal arrow. Nor does an inverse covariance automatically describe conditional independence for non-Gaussian data. Pooling iris species, for example, mixes populations with different means and potentially different covariance matrices. A precision matrix computed from that pooled sample is a descriptive inverse-covariance summary; a Gaussian graphical interpretation needs an additional modelling assumption.

There is no response variable. The model is a joint distribution and, in that respect, closer to a mixture model (Chapter 5) than to a regression: it describes dependence among the variables. This chapter develops the Markov properties, the Gaussian graphical model, maximum likelihood when the graph is known, and the graphical lasso when it is not, building on the conditional-normal result of Chapter 1 and the lasso of Chapter 18.

Source. Lecture notes, Chapter 5 (pp. 139–157). Some of the material is covered in Chapter 17 of Hastie et al. (2009); a more thorough reference is Lauritzen (1996).

19.1 Markov graphs

To say that a correlation is carried by a third variable, one needs names for paths and for the set of variables that every path must cross. Those names are the graph.

19.1.1 Graph terminology

A *graph* $\mathcal{G} = (V, E)$ is a set of *vertices* or *nodes* V along with a set of *edges* $E \subseteq V \times V$ joining pairs of vertices. We exclude self-edges and consider only *undirected* edges: $(j, k) \in E$ implies $(k, j) \in E$, so edges are listed in any convenient order of the pair of vertices, with the reverse understood to be present.

- Two connected vertices are *adjacent*. A graph is *complete* if every node is adjacent to every other node.
- If $A \subseteq V$ is a subset of nodes, the *subgraph* $\mathcal{G}_A = (A, E_A)$ has $E_A = E \cap (A \times A)$: keep every edge for which both vertices are in A. A subset is *complete* if it induces a complete subgraph. A complete subset that is maximal with respect to $\subseteq$ is a *clique*: a complete subset that cannot be made into a larger complete subset by adding any vertex.

- In an undirected graph, the *boundary* $\mathrm{bd}(A)$ of a subset A is the set of vertices in $V \setminus A$ that are adjacent to vertices in A. The *closure* of A is $\mathrm{cl}(A) = A \cup \mathrm{bd}(A)$.
- A *path* $X_1, \dots, X_k$ is a sequence of vertices joined in a sequence: X_i is adjacent to X_{i+1} for $i = 1, \dots, k-1$. A subset $C \subseteq V$ is a (j,k)-*separator* if all paths from j to k intersect C. C *separates* A *from* B if it is a (j,k)-separator for every $j \in A$, $k \in B$.

In a graphical model each vertex represents a random variable and an edge permits conditional dependence between its endpoints. In a minimal Gaussian concentration graph with nonzero precision entries on edges, the edge represents conditional dependence given all remaining variables. The absence of an edge indicates that the two variables are *conditionally independent* given the values of all the other variables. For many models the conditioning set is smaller than the full remainder, and the graph says which set.

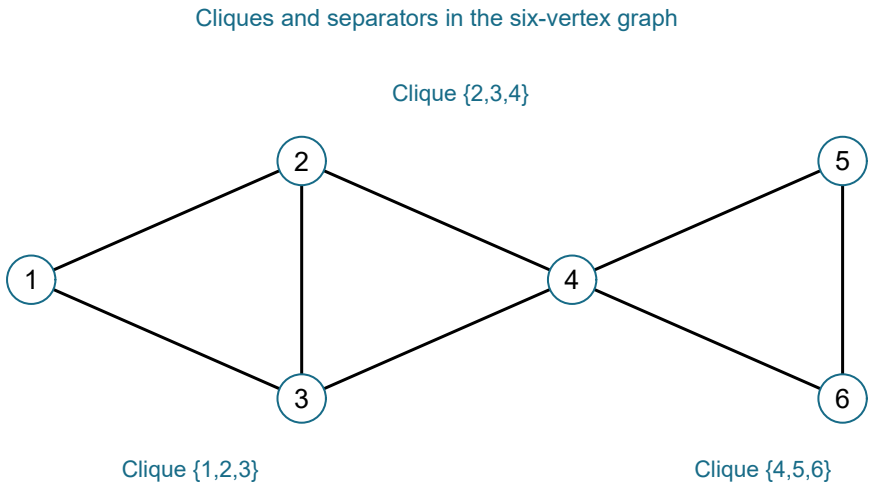


Figure 19.1: The six-vertex graph used in the text and exercises, with cliques {1,2,3}, {2,3,4} and {4,5,6}. Vertex 4 separates the left group from vertices 5 and 6.

19.1.2 Conditional independence

Conditional independence is a statement about the conditional distribution, not about the marginal correlation. If X, Y, Z are random variables with joint distribution P, X is *conditionally independent of* Y *given* Z under P, written $X \perp\!\!\!\perp Y \mid Z \ [P]$ (with P usually omitted), if for any measurable set A in the sample space of X there exists a conditional probability $P(A \mid Y, Z)$ that is a function of Z alone, i.e. $P(A \mid Y, Z) = P(A \mid Z)$. For discrete variables,

$$X \perp\!\!\!\perp Y \mid Z \iff P(X = x, Y = y \mid Z = z) = P(X = x \mid Z = z)\, P(Y = y \mid Z = z), \tag{19.1}$$

and when X, Y, Z have a joint density on a product measure,

$$\begin{aligned} X \perp\!\!\!\perp Y \mid Z &\iff f_{XY|Z}(x, y \mid z) = f_{X|Z}(x \mid z) f_{Y|Z}(y \mid z) \\ &\iff f_{XYZ}(x, y, z) f_Z(z) = f_{XZ}(x, z) f_{YZ}(y, z), \end{aligned} \tag{19.2}$$

almost surely with respect to P (for all z with $f_Z(z) > 0$ if the densities are continuous). The properties (Lauritzen, 1996)

- if $X \perp\!\!\!\perp Y \mid Z$ then $Y \perp\!\!\!\perp X \mid Z$;
- if $X \perp\!\!\!\perp Y \mid Z$ and $U = h(X)$ then $U \perp\!\!\!\perp Y \mid Z$;
- if $X \perp\!\!\!\perp Y \mid Z$ and $U = h(X)$ then $X \perp\!\!\!\perp Y \mid (Z, U)$;
- if $X \perp\!\!\!\perp Y \mid Z$ and $X \perp\!\!\!\perp W \mid (Y, Z)$ then $X \perp\!\!\!\perp (W, Y) \mid Z$

hold, and $X \perp\!\!\!\perp Y \mid Z$ is equivalent to each of the following (almost surely, with f the density of its arguments):

$$f(x, y, z) = f(x, z)f(y, z)/f(z), \tag{19.3}$$
$$f(x \mid y, z) = f(x \mid z), \tag{19.4}$$
$$f(x, z \mid y) = f(x \mid z)f(z \mid y), \tag{19.5}$$
$$f(x, y, z) = g(x, z)h(y, z) \text{ for some } g, h, \tag{19.6}$$
$$f(x, y, z) = f(x \mid z)f(y, z). \tag{19.7}$$

The factorisation (19.6) is the form that generalises to graphs. Also, if C separates A from B then $\boldsymbol{X}_A \perp\!\!\!\perp \boldsymbol{X}_B \mid \boldsymbol{X}_C$ for a graphical model with the global Markov property below.

Equation numbering.
The list in the notes ends with an equation number (5.8) attached to an empty line, which looks like a typesetting slip: only five statements (5.3)–(5.7) are displayed. Equation (19.5) does follow from (19.4): $f(x, z \mid y) = f(x \mid y, z)f(z \mid y) = f(x \mid z)f(z \mid y)$.

19.1.3 Markov properties and factorisation

Undirected graphical models are also known as *Markov random fields*. A probability measure on $\boldsymbol{X}$ may have any of the following properties with respect to a graph $\mathcal{G}$:

- **(P)** the *pairwise Markov property*: for any pair of non-adjacent vertices $j, k \in V$, $X_j \perp\!\!\!\perp X_k \mid \boldsymbol{X}_{V\setminus\{j,k\}}$;
- **(L)** the *local Markov property*: for any vertex $j \in V$, $X_j \perp\!\!\!\perp \boldsymbol{X}_{V\setminus \mathrm{cl}(j)} \mid \boldsymbol{X}_{\mathrm{bd}(j)}$;
- **(G)** the *global Markov property*: for any triple (A, B, S) of disjoint subsets of V such that S separates A from B, $\boldsymbol{X}_A \perp\!\!\!\perp \boldsymbol{X}_B \mid \boldsymbol{X}_S$.

The global property implies the local property, which implies the pairwise property. The reverse implications need additional conditions; they are not true for every distribution. To connect them, introduce factorisation. A distribution factorises over $\mathcal{G}$ when its density relative to a product measure can be written as a product of nonnegative functions, each involving only variables from a complete subset:

$$f(\boldsymbol{x}) = \prod_{a \text{ complete}} \psi_a(\boldsymbol{x}_a). \tag{19.8}$$

Without loss of generality the sets a can be limited to the cliques of $\mathcal{G}$:

$$f(\boldsymbol{x}) = \frac{1}{Z} \prod_{c \in \mathcal{C}} \psi_c(\boldsymbol{x}_c), \tag{19.9}$$

where $\mathcal{C}$ is the set of cliques, the ψ_c are the *clique potentials* (typically not densities), and Z is a normalising constant. The functions ψ_a are not unique: groups of them can be combined by multiplication in a variety of ways. If P factorises we say it has property **(F)**, and (F) $\Rightarrow$ (G).

Theorem 19.1 (Hammersley–Clifford)**.** A probability distribution P with positive and continuous density f with respect to a product measure μ satisfies the pairwise Markov property with respect to an undirected graph $\mathcal{G}$ if and only if it factorises according to $\mathcal{G}$.

Under the positivity assumptions of the theorem, factorisation, global, local and pairwise Markov properties coincide (Hastie et al., 2009, Section 17.2). Positivity matters: zeros in the support can break the implication from conditional-independence statements to factorisation. This is a statement about distributions, not an assertion that every fitting problem for every graphical model is convex. The Gaussian precision-matrix likelihood developed below is a specific convex optimisation problem after changing sign.

19.2 Gaussian graphical models

A marginal correlation can be nonzero for every pair while most conditional correlations, given the rest, are zero. The Gaussian case turns that distinction into an entry of a matrix. Consider a p-dimensional random vector $\boldsymbol{X} = (X_1, \dots, X_p)^\top$ with $V = \{1, \dots, p\}$. The inverse of the covariance matrix is the *concentration matrix* or *precision matrix*, $\boldsymbol{K} = \boldsymbol{\Sigma}^{-1}$, with entries k_{jk}, and $\boldsymbol{\Sigma}$ is assumed regular enough to be invertible.

The proof that a zero in $\boldsymbol{K}$ is conditional independence reorders the variables so that the pair of interest is the first block, then reads the conditional covariance from the block inverse. That block inverse is the same Schur complement already used for the conditional normal distribution (Theorem 1.6).

Proposition 19.2 (Zeros of the precision matrix)**.** Let $\boldsymbol{X} \sim \mathrm{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with $\boldsymbol{\Sigma}$ regular. For $j \neq k$,

$$X_j \perp\!\!\!\perp X_k \mid \boldsymbol{X}_{V \setminus \{j,k\}} \iff k_{jk} = 0.$$

That is, the zero entries of the precision matrix indicate conditional independence between the corresponding variables, with the conditioning on all the remaining variables.

Proof. The variables can be put in any order, and a corresponding mean vector and covariance matrix formed, so without loss of generality $j = 1$, $k = 2$, $A = \{1, 2\}$, $B = \{3, \dots, p\}$. From the block inverse (1.26), writing

$$\boldsymbol{\Sigma}^{-1} = \boldsymbol{K} = \begin{pmatrix} \boldsymbol{K}_{AA} & \boldsymbol{K}_{AB} \\ \boldsymbol{K}_{AB}^\top & \boldsymbol{K}_{BB} \end{pmatrix}, \qquad \text{we have} \qquad \boldsymbol{K}_{AA} = \boldsymbol{\Sigma}_{A|B}^{-1}.$$

By Theorem 1.6, $\boldsymbol{X}_A \mid \boldsymbol{X}_B \sim \mathrm{N}(\boldsymbol{\mu}_{A|B}, \boldsymbol{\Sigma}_{A|B}) = \mathrm{N}(\boldsymbol{\mu}_{A|B}, \boldsymbol{K}_{AA}^{-1})$. If

$$\boldsymbol{K}_{AA} = \begin{pmatrix} k_{11} & k_{12} \\ k_{12} & k_{22} \end{pmatrix} \qquad \text{then} \qquad \boldsymbol{\Sigma}_{A|B} = \boldsymbol{K}_{AA}^{-1} = \frac{1}{|\boldsymbol{K}_{AA}|} \begin{pmatrix} k_{22} & -k_{12} \\ -k_{12} & k_{11} \end{pmatrix}.$$

Hence when $k_{12} = 0$ the conditional covariance of X_1 and X_2 given $\boldsymbol{X}_B$ is 0; conditional on $\boldsymbol{X}_B$ the pair is bivariate normal with zero covariance, so X_1 and X_2 are conditionally independent. Conversely, conditional independence gives zero conditional covariance, so $k_{12} = 0$. The argument applies to any pair j, k.

□

There are two separate steps in the proof. Matrix inversion identifies the conditional covariance; Gaussianity turns a zero conditional covariance into conditional independence. Drop the second assumption and the last inference can fail. The negative sign also deserves attention: after normalisation, a negative precision entry represents a positive partial correlation.

Several consequences follow from the same block formula.

- The conditional covariance of (X_j, X_k) given all the others is proportional to $-k_{jk}$: $\sigma_{jk|V\setminus\{j,k\}} = -k_{jk}/|\boldsymbol{K}_{AA}|$. The *partial correlation* of X_j and X_k given the rest is therefore

$$\rho_{jk|V\setminus\{j,k\}} = \frac{-k_{12}}{\sqrt{k_{11}k_{22}}} = \frac{-k_{jk}}{\sqrt{k_{jj}k_{kk}}}, \tag{19.10}$$

  since $\boldsymbol{\Sigma}_{A|B}$ has diagonal $k_{22}/|\boldsymbol{K}_{AA}|$, $k_{11}/|\boldsymbol{K}_{AA}|$ and off-diagonal $-k_{12}/|\boldsymbol{K}_{AA}|$ (compare (1.34)).

- The diagonal elements of the precision matrix are the reciprocals of the conditional variances given the remaining variables: $k_{jj} = \left(\sigma_{j|V\setminus j,\, jj}\right)^{-1}$.

- A *normalised* precision matrix (with diagonal 1, as in the example below and in the plots) is $\boldsymbol{D}^{-1/2}\boldsymbol{K}\boldsymbol{D}^{-1/2}$ with $\boldsymbol{D} = \mathrm{diag}(\boldsymbol{K})$. Its off-diagonal entries are *minus* the partial correlations, which is why a red (negative) edge in a plot of that matrix means positive conditional dependence.

By (19.10), a negative off-diagonal of the precision corresponds to positive conditional dependence.

Gaussian graphical models have the pairwise, local, and global Markov properties with respect to the graph whose edges are the non-zero entries of $\boldsymbol{K}$.

A matrix of marginal correlations can be dense when the conditional dependence graph is sparse. Partial correlation is the dependence that remains after the other variables are held fixed. The chain below is that contrast in four variables.

Example 19.3 (Zeros in $\boldsymbol{K}$, not in $\boldsymbol{\Sigma}$)**.** Let $\boldsymbol{X}$ be a stationary **Gaussian** autoregressive process observed at four times with $\mathrm{corr}(X_j, X_k) = 0.6^{|j-k|}$:

$$\boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0.6 & 0.36 & 0.216 \\ 0.6 & 1 & 0.6 & 0.36 \\ 0.36 & 0.6 & 1 & 0.6 \\ 0.216 & 0.36 & 0.6 & 1 \end{pmatrix}, \quad \boldsymbol{K} = \boldsymbol{\Sigma}^{-1} = \begin{pmatrix} 1.5625 & -0.9375 & 0 & 0 \\ -0.9375 & 2.125 & -0.9375 & 0 \\ 0 & -0.9375 & 2.125 & -0.9375 \\ 0 & 0 & -0.9375 & 1.5625 \end{pmatrix}.$$

Every pair of the four variables is marginally correlated, so the covariance graph is complete, but $\boldsymbol{K}$ is *tridiagonal*: $X_1 \perp\!\!\!\perp X_3 \mid (X_2, X_4)$, $X_1 \perp\!\!\!\perp X_4 \mid (X_2, X_3)$, and $X_2 \perp\!\!\!\perp X_4 \mid (X_1, X_3)$. The graph is the chain $1 - 2 - 3 - 4$ (a Markov chain: each variable depends on the past only through the most recent value). The partial correlations from (19.10) are $\rho_{12|34} = 0.9375/\sqrt{1.5625 \times 2.125} = 0.515$, $\rho_{23|14} = 0.9375/2.125 = 0.441$ and $\rho_{34|12} = 0.515$. As a check of the diagonal formula, $k_{11} = 1.5625 = 1/0.64$ and the variance of X_1 given (X_2, X_3, X_4) is $1 - 0.6^2 = 0.64$ (since X_1 depends on the rest only through X_2); and $k_{22} = 2.125 = 1/0.4706$, the variance of X_2 given the others. Figure 19.2 shows the same pattern for six variables.

19.2.1 The log density

Write the multivariate normal density in terms of $\boldsymbol{K}$:

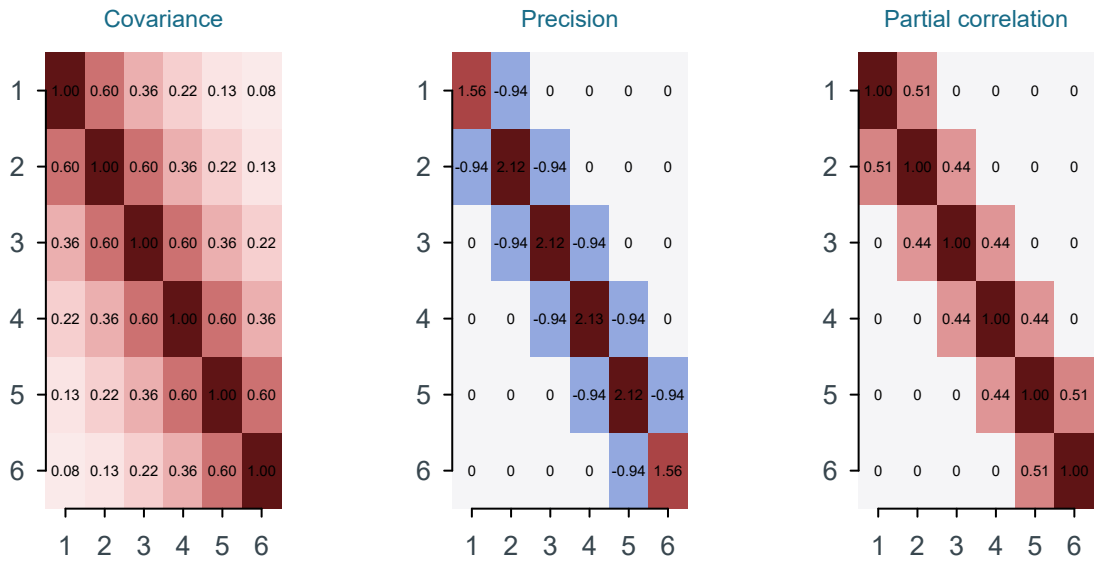


Figure 19.2: An AR(1) covariance is dense while its precision is tridiagonal. Partial correlations expose the Gaussian chain structure.

$$
\begin{aligned}
f(\boldsymbol{x}) &= \frac{1}{(2\pi)^{p/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\Big[-\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\Big] \\
&= \frac{|\boldsymbol{K}|^{1/2}}{(2\pi)^{p/2}} \exp\Big[-\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{K}(\boldsymbol{x}-\boldsymbol{\mu})\Big]. \qquad (19.11)
\end{aligned}
$$

Treating $\boldsymbol{\mu}$ and $\boldsymbol{K}$ as constants, the logarithm of the density, up to constants c, c_1, is

$$
\begin{aligned}
\log f(\boldsymbol{x}) &= c - \tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{K}(\boldsymbol{x}-\boldsymbol{\mu}) = c - \tfrac{1}{2}\boldsymbol{x}^\top \boldsymbol{K}\boldsymbol{x} + \boldsymbol{x}^\top \boldsymbol{K}\boldsymbol{\mu} - \tfrac{1}{2}\boldsymbol{\mu}^\top \boldsymbol{K}\boldsymbol{\mu} \\
&= c_1 - \tfrac{1}{2}\sum_{j\in V} k_{jj}x_j^2 - \sum_{j\in V}\sum_{k<j} k_{jk}x_j x_k + \sum_{j\in V} x_j\Big(\sum_{k\in V} k_{jk}\mu_k\Big) \\
&= c_1 - \tfrac{1}{2}\sum_{j\in V} k_{jj}x_j^2 + \sum_{j\in V} h_j x_j - \sum_{j\in V,\,k<j} k_{jk}x_j x_k, \qquad \boldsymbol{h} = \boldsymbol{K}\boldsymbol{\mu}. \qquad (19.12)
\end{aligned}
$$

So the log of the density is composed of a sum of *quadratic main effects* with coefficients $-k_{jj}/2$, *linear main effects* with coefficients h_j, and *quadratic interactions* with coefficients $-k_{jk}$. Taking the exponential, f is a product over single variables and over *pairs*, with a pair appearing only if $k_{jk} \neq 0$. Each pair with $k_{jk} \neq 0$ is an edge, so the factors involve complete subsets of size at most 2 (singletons or edges). Such subsets need not be maximal cliques; their factors can be absorbed into containing clique potentials, an explicit instance of the factorisation (19.8) (and of the Hammersley–Clifford theorem) with $\psi_{jk}(x_j, x_k) = \exp(-k_{jk}x_j x_k)$.

19.3 Model selection in Gaussian graphical models

Model selection here means choosing which edges to exclude, that is, which pairs can be treated as conditionally independent given the others. In the Gaussian model that is the search for symmetric zeros in $\boldsymbol{K}$. Some of the better-known methods:

- *Optimisation of an information criterion*, which balances model fit against complexity: the Akaike and Bayesian information criteria (Chapter 6).
- *Stepwise optimisation of the model by likelihood-ratio testing*: compare a model with a given link against one without it; if the test supports adding or removing the link at a specified significance level it is done, and this is repeated until no further change is supported.
- *Optimisation of predictive accuracy*, which needs some balance between model fitting and model complexity.

There are $2^{p(p-1)/2}$ undirected simple graphs, since each unordered pair can be present or absent. Even $p = 10$ gives about 3.5×10^{13} candidates. Exhaustive search quickly becomes impractical. When the sample covariance is singular, the unrestricted Gaussian precision MLE also fails to exist. A sufficiently restrictive known graph can still have an MLE, so singularity of $\boldsymbol{S}$ does not rule out every constrained fit. Regularisation provides another route, with assumptions of its own.

19.4 Regression-linked estimation for a known graph

A number of methods fit a Gaussian graphical model with a known partial covariance structure, for example the iterative proportional fitting procedure of Speed and Kiiveri (1986). Here we follow an alternative approach (Hastie et al., 2009) that can be extended to model selection using a form of the lasso.

19.4.1 The constrained likelihood

For iid observations $\boldsymbol{x}_1, \dots, \boldsymbol{x}_n$ from $\mathrm{N}_p(\boldsymbol{\mu}, \boldsymbol{K}^{-1})$ the likelihood is

$$L(\boldsymbol{X}_{\text{all}}; \boldsymbol{\mu}, \boldsymbol{K}) = \prod_{i=1}^{n} \frac{|\boldsymbol{K}|^{1/2}}{(2\pi)^{p/2}} \mathrm{e}^{-\frac{1}{2}(\boldsymbol{x}_i-\boldsymbol{\mu})^\top \boldsymbol{K}(\boldsymbol{x}_i-\boldsymbol{\mu})} = \frac{|\boldsymbol{K}|^{n/2}}{(2\pi)^{np/2}} \exp\Big[-\tfrac{1}{2} \sum_{i=1}^{n} (\boldsymbol{x}_i-\boldsymbol{\mu})^\top \boldsymbol{K}(\boldsymbol{x}_i-\boldsymbol{\mu}) \Big],$$

and the log-likelihood is

$$\ell(\boldsymbol{X}_{\text{all}}; \boldsymbol{\mu}, \boldsymbol{K}) = \frac{n}{2} \log|\boldsymbol{K}| - \frac{np}{2} \log(2\pi) - \frac{1}{2} \sum_{i=1}^{n} (\boldsymbol{x}_i-\boldsymbol{\mu})^\top \boldsymbol{K}(\boldsymbol{x}_i-\boldsymbol{\mu}). \tag{19.13}$$

We maximise it subject to constraints on $\boldsymbol{K}$: edges missing from the graph correspond to zeros in $\boldsymbol{K}$, i.e. if E is the edge set then $k_{jk} = 0$ for $j \neq k$ with $(j,k) \notin E$. The domain is $\boldsymbol{K} \succ 0$; diagonal entries are not constrained to zero. The second term is not a function of the parameters, and the likelihood is maximised for $\boldsymbol{\mu}$ at $\widehat{\boldsymbol{\mu}} = \overline{\boldsymbol{x}}$ regardless of $\boldsymbol{K}$ (Section 2.1). Divide by $n/2$ without affecting the maximisation, so that we maximise $\log|\boldsymbol{K}| - \frac{1}{n}\sum_i (\boldsymbol{x}_i - \overline{\boldsymbol{x}})^\top \boldsymbol{K}(\boldsymbol{x}_i - \overline{\boldsymbol{x}})$. Following Hastie et al. (2009) and to simplify later notation, let

$$\boldsymbol{S} = \frac{1}{n} \sum_{i=1}^{n} (\boldsymbol{x}_i - \overline{\boldsymbol{x}})(\boldsymbol{x}_i - \overline{\boldsymbol{x}})^\top, \tag{19.14}$$

the *biased* sample covariance matrix. (Usually, e.g. in (Anderson, 2003), this matrix is denoted $\widehat{\boldsymbol{\Sigma}}$ and $\boldsymbol{S}$ is reserved for the unbiased matrix with divisor $n-1$; in this chapter the $1/n$ version is called $\boldsymbol{S}$.) Since a scalar equals its trace, and by the cyclic property of the trace,

$$\frac{1}{n} \sum_i (\boldsymbol{x}_i - \overline{\boldsymbol{x}})^\top \boldsymbol{K}(\boldsymbol{x}_i - \overline{\boldsymbol{x}}) = \operatorname{tr}\Big(\frac{1}{n} \sum_i (\boldsymbol{x}_i - \overline{\boldsymbol{x}})(\boldsymbol{x}_i - \overline{\boldsymbol{x}})^\top \boldsymbol{K} \Big) = \operatorname{tr}(\boldsymbol{S}\boldsymbol{K}).$$

So the problem is to maximise

$$g(\boldsymbol{K}) = \log|\boldsymbol{K}| - \operatorname{tr}(\boldsymbol{S}\boldsymbol{K}) \tag{19.15}$$

subject to the fixed zeros of $\boldsymbol{K}$. With Lagrange multipliers γ_{jk} for the constraints,

$$g_C(\boldsymbol{K}, \boldsymbol{\Gamma}) = \log|\boldsymbol{K}| - \operatorname{tr}(\boldsymbol{S}\boldsymbol{K}) - \sum_{j \neq k,\, (j,k) \notin E} \gamma_{jk} k_{jk}, \tag{19.16}$$

Here $\boldsymbol{\Gamma}$ is symmetric, zero on the diagonal and on allowed edges, and potentially nonzero on missing edges. We use a full-matrix trace inner product, so each off-diagonal pair appears twice; the symmetric multiplier convention absorbs that factor consistently. The derivatives with respect to symmetric $\boldsymbol{K}$ are $\boldsymbol{K}^{-1} = \boldsymbol{\Sigma}$ for the log determinant and $\boldsymbol{S}$ for the trace term, giving

$$\nabla g_C(\boldsymbol{K}, \boldsymbol{\Gamma}) = \boldsymbol{\Sigma} - \boldsymbol{S} - \boldsymbol{\Gamma}. \tag{19.17}$$

Using $\boldsymbol{W}$ for the estimate of $\boldsymbol{\Sigma}$ (as in (Hastie et al., 2009)) and $\boldsymbol{\Theta}$ for the estimate of $\boldsymbol{K}$ (so $\boldsymbol{W}\boldsymbol{\Theta} = \boldsymbol{I}$), the estimating equations are

$$\boldsymbol{W} - \boldsymbol{S} - \boldsymbol{\Gamma} = \boldsymbol{0}. \tag{19.18}$$

Reading (19.18) entrywise: for every edge $(j,k) \in E$ and for the diagonal, $\gamma_{jk} = 0$ and so $w_{jk} = s_{jk}$: *the maximum likelihood estimate of the covariance matrix reproduces the sample covariances on the edges and diagonal exactly.* For non-edges, w_{jk} is whatever value makes $(\boldsymbol{W}^{-1})_{jk} = 0$.

19.4.2 Solving by regression

Isolate the last row and column of $\boldsymbol{W}$ and of its inverse $\boldsymbol{\Theta}$ (any variable can be moved to the last position): write A for the first $p-1$ indices and

$$\begin{pmatrix} \boldsymbol{W}_{AA} & \boldsymbol{w}_{Ap} \\ \boldsymbol{w}_{Ap}^{\top} & w_{pp} \end{pmatrix} \begin{pmatrix} \boldsymbol{\Theta}_{AA} & \boldsymbol{\theta}_{Ap} \\ \boldsymbol{\theta}_{Ap}^{\top} & \theta_{pp} \end{pmatrix} = \begin{pmatrix} \boldsymbol{I} & \boldsymbol{0} \\ \boldsymbol{0}^{\top} & 1 \end{pmatrix}. \tag{19.19}$$

The top $p-1$ entries of the last column of the identity matrix give $\boldsymbol{W}_{AA}\boldsymbol{\theta}_{Ap} + \boldsymbol{w}_{Ap}\theta_{pp} = \boldsymbol{0}$. Noting that θ_{pp} is a scalar, solve for $\boldsymbol{w}_{Ap}$:

$$\boldsymbol{w}_{Ap} = -\boldsymbol{W}_{AA}\boldsymbol{\theta}_{Ap}/\theta_{pp} = \boldsymbol{W}_{AA}\boldsymbol{\beta}, \qquad \boldsymbol{\beta} = -\boldsymbol{\theta}_{Ap}/\theta_{pp}. \tag{19.20}$$

The top $p-1$ entries of the last column of (19.18) are $\boldsymbol{w}_{Ap} - \boldsymbol{s}_{Ap} - \boldsymbol{\gamma}_{Ap} = \boldsymbol{0}$, which with (19.20) becomes

$$\boldsymbol{W}_{AA}\boldsymbol{\beta} - \boldsymbol{s}_{Ap} - \boldsymbol{\gamma}_{Ap} = \boldsymbol{0}. \tag{19.21}$$

These $p-1$ equations are the estimating equations for a *constrained regression* of X_p on the other variables, with $\boldsymbol{W}_{AA}$ (the current estimate of $\boldsymbol{\Sigma}_{AA}$) used in place of $\boldsymbol{S}_{AA}$. The unconstrained normal equations of the regression of X_p on $\boldsymbol{X}_A$ are $\boldsymbol{S}_{AA}\boldsymbol{\beta} = \boldsymbol{s}_{Ap}$.

On a complete sample those coefficients are exactly the ordinary least-squares coefficients, whenever every entry of $\boldsymbol{S}_{AA}$ and of $\boldsymbol{s}_{Ap}$ is computed with the same divisor. The factor n or $n-1$ cancels in the product $\boldsymbol{S}_{AA}^{-1}\boldsymbol{s}_{Ap}$. The adjective *biased*, applied above to the $1/n$ covariance, is a statement about the covariance estimator. It does not move the fitted conditional-mean line relative to least squares.

The link with regression is the conditional-normal result of Chapter 1. For a multivariate normal vector the conditional mean of X_p given $\boldsymbol{X}_A = \boldsymbol{x}_A$ is $\mu_p + \boldsymbol{\Sigma}_{pA}\boldsymbol{\Sigma}_{AA}^{-1}(\boldsymbol{x}_A - \boldsymbol{\mu}_A)$ by (1.30), the form of the population multiple regression of X_p on $\boldsymbol{X}_A$ with $\boldsymbol{\beta} = \boldsymbol{\Sigma}_{AA}^{-1}\boldsymbol{\Sigma}_{Ap}$. The block-inverse formula (1.26) with the roles of the blocks swapped (the last variable first) gives $\boldsymbol{K}_{Ap} = -\boldsymbol{K}_{pp}\,\boldsymbol{\Sigma}_{AA}^{-1}\boldsymbol{\Sigma}_{Ap}$ (the off-diagonal block equals minus the Schur-complement inverse times the regression coefficients), that is

$$\boldsymbol{\Sigma}_{AA}^{-1}\boldsymbol{\Sigma}_{Ap} = -\boldsymbol{K}_{Ap}/k_{pp}, \tag{19.22}$$

the same as $\boldsymbol{\beta}$ in (19.20) except that the latter is an estimate. So estimating $\boldsymbol{\Sigma}$ or $\boldsymbol{K}$ via $\boldsymbol{W}$ or $\boldsymbol{\Theta}$ can be viewed within the linear regression framework, constrained by the known zeros of $\boldsymbol{\Theta}$.

Using the zeros.

Suppose X_p has q neighbours. Its $p-1-q$ missing edges force the same coordinates of $\boldsymbol{\theta}_{Ap}$, and hence $\boldsymbol{\beta}$, to zero. On the q allowed edges the multipliers are zero, so those equations identify the q free regression coefficients. The remaining equations determine the missing-edge multipliers. Retain the neighbour rows and columns to obtain

$$\boldsymbol{W}_{AA}^{*}\boldsymbol{\beta}^{*} - \boldsymbol{s}_{Ap}^{*} = \boldsymbol{0}, \qquad \widehat{\boldsymbol{\beta}}^{*} = \left(\boldsymbol{W}_{AA}^{*}\right)^{-1}\boldsymbol{s}_{Ap}^{*}, \tag{19.23}$$

where $\boldsymbol{W}_{AA}^{*}$ and $\boldsymbol{s}_{Ap}^{*}$ keep only the rows and columns of the neighbours of p. Pad $\widehat{\boldsymbol{\beta}}^{*}$ with zeros to length $p-1$ to get $\widehat{\boldsymbol{\beta}}$. We can also solve for $\boldsymbol{\theta}_{Ap}$: from the bottom-right entry of (19.19), $\boldsymbol{w}_{Ap}^{\top}\boldsymbol{\theta}_{Ap} + w_{pp}\theta_{pp} = 1$, and with $\boldsymbol{\theta}_{Ap} = -\widehat{\boldsymbol{\beta}}\theta_{pp}$ we get $1/\theta_{pp} = w_{pp} - \boldsymbol{w}_{Ap}^{\top}\widehat{\boldsymbol{\beta}}$, where $w_{pp} = s_{pp}$ because the diagonal is an "edge" set.

Algorithm 19.4 (ML fitting, known graph)**.** 1. Initialise $\boldsymbol{W} = \boldsymbol{S}$.

2. Repeat for $j = 1, \ldots, p$ until convergence:
   1. Partition $\boldsymbol{W}$ into part A (all but the jth row and column) and part B (the jth row and column).
   2. Solve $\boldsymbol{W}_{AA}^{*}\boldsymbol{\beta}^{*} - \boldsymbol{s}_{AB}^{*} = \boldsymbol{0}$ for the unconstrained edge parameters $\boldsymbol{\beta}^{*}$. Obtain $\widehat{\boldsymbol{\beta}}$ by padding $\widehat{\boldsymbol{\beta}}^{*}$ with zeros in the appropriate positions.
   3. Update $\boldsymbol{w}_{AB} = \boldsymbol{W}_{AA}\widehat{\boldsymbol{\beta}}$ and the corresponding row $\boldsymbol{w}_{BA} = \boldsymbol{w}_{AB}^{\top}$.
3. In the final cycle for each j, solve for the estimated precision matrix elements: $\widehat{\boldsymbol{\theta}}_{AB} = -\widehat{\boldsymbol{\beta}}\,\widehat{\theta}_{BB}$ with $1/\widehat{\theta}_{BB} = s_{BB} - \boldsymbol{w}_{AB}^{\top}\widehat{\boldsymbol{\beta}}$.

The displayed initialisation assumes $\boldsymbol{S} \succ 0$, a convenient sufficient condition for this presentation. Every update must maintain symmetry and positive definiteness, and convergence should be assessed through both parameter changes and likelihood equations. A known graph can permit an MLE with singular $\boldsymbol{S}$, but that case requires an appropriate feasible start and an existence check; the inverse operations above cannot simply be applied to singular blocks (Hastie et al., 2009, Section 17.3).

Example 19.5 (Fitting a chain graph)**.** Forty observations from the four-variable autoregressive model of Example 19.3 give the biased sample covariance matrix

$$\boldsymbol{S} = \begin{pmatrix} 0.8088 & 0.3166 & 0.2911 & 0.1228 \\ 0.3166 & 0.5490 & 0.4638 & 0.2318 \\ 0.2911 & 0.4638 & 0.9569 & 0.5472 \\ 0.1228 & 0.2318 & 0.5472 & 0.8223 \end{pmatrix}.$$

Fit the chain graph $1-2-3-4$ with Algorithm 19.4. In the sweep for $j=1$ the only neighbour of variable 1 is 2, so the reduced regression of X_1 on X_2 has $\widehat{\beta}^* = s_{12}/w_{22} = 0.3166/0.5490 = 0.577$ and the update $\boldsymbol{w}_{A1} = \boldsymbol{W}_{AA}\widehat{\boldsymbol{\beta}}$ sets $w_{12} = \widehat{\beta}^* w_{22} = s_{12}$ (the edge is reproduced exactly) while $w_{13} = \widehat{\beta}^* w_{23}$ and $w_{14} = \widehat{\beta}^* w_{24}$: each non-edge covariance is filled in from the covariances along the path through the neighbour. The sweeps for the other variables do the same, and the constrained covariance, rounded to four decimals, is

$$\boldsymbol{W} = \begin{pmatrix} 0.8088 & 0.3166 & 0.2675 & 0.1530 \\ 0.3166 & 0.5490 & 0.4638 & 0.2652 \\ 0.2675 & 0.4638 & 0.9569 & 0.5472 \\ 0.1530 & 0.2652 & 0.5472 & 0.8223 \end{pmatrix}, \quad \boldsymbol{W}^{-1} = \begin{pmatrix} 1.597 & -0.921 & 0 & 0 \\ -0.921 & 3.616 & -1.495 & 0 \\ 0 & -1.495 & 2.412 & -1.123 \\ 0 & 0 & -1.123 & 1.963 \end{pmatrix}.$$

The diagonal and the three edge entries of $\boldsymbol{W}$ equal those of $\boldsymbol{S}$ (for example $w_{23} = s_{23} = 0.4638$). The non-edge entries are *filled in* so that the inverse has exact zeros where the graph has no edges: $w_{13} = 0.2675$ (the sample value is $s_{13} = 0.2911$), $w_{14} = 0.1530$ ($s_{14} = 0.1228$), and $w_{24} = 0.2652$ ($s_{24} = 0.2318$). This is the Gaussian MLE for the constrained model. For example $w_{13} = w_{12}w_{23}/w_{22} = 0.3166 \times 0.4638/0.5490 = 0.2675$, the relation implied by $X_1 \perp\!\!\!\perp X_3 \mid X_2$ in a chain (the regression of X_3 on X_2 is unaffected by X_1), and similarly $w_{14} = w_{12}w_{24}/w_{22} = 0.1530$ and $w_{24} = w_{23}w_{34}/w_{33} = 0.2652$.

19.5 The graphical lasso

When the graph is unknown, put a cost on nonzero precision entries. The graphical lasso combines the Gaussian log-likelihood with an entrywise L_1 penalty. Its corners allow exact zeros, while the negative log-likelihood plus penalty remains convex on $\boldsymbol{K} \succ 0$. An L_2 penalty lacks this thresholding mechanism, although special data can still produce zero entries (Friedman et al., 2008).

We usually do not know of any constraints on the graph structure and would like to estimate it from the data together with the elements of $\boldsymbol{K}$. The penalised objective is

$$g(\boldsymbol{K}) = \log|\boldsymbol{K}| - \operatorname{tr}(\boldsymbol{S}\boldsymbol{K}) - \lambda\|\boldsymbol{K}\|_1, \tag{19.24}$$

where $\|\boldsymbol{K}\|_1$ is the L_1 norm, the sum of absolute values of the elements of $\boldsymbol{K}$, and $\lambda \geq 0$ is a penalty hyper-parameter. This is the *graphical lasso*. It can be solved by an algorithm very similar to Algorithm 19.4: the vector $\boldsymbol{\beta}$ in step 2 is treated as the parameter vector of a multiple linear regression and estimated by a lasso regression, including suitable zeros. The subgradient of (19.24) gives the estimating equations $\boldsymbol{W} - \boldsymbol{S} - \lambda\boldsymbol{\Gamma} = \boldsymbol{0}$ with $\boldsymbol{\Gamma} \in \partial\|\boldsymbol{K}\|_1$ ($\Gamma_{jk} = \operatorname{sign}(k_{jk})$ if $k_{jk} \neq 0$, $\Gamma_{jk} \in [-1,1]$ otherwise), and the regression step becomes the lasso estimating equation of step 2(b) below.

Algorithm 19.6 (Graphical lasso).

1. Initialise $\boldsymbol{W} = \boldsymbol{S} + \lambda\boldsymbol{I}$.
2. Repeat for $j = 1, \ldots, p$ until convergence:
   1. Partition $\boldsymbol{W}$ into part A (all but the jth row and column) and part B (the jth row and column).
   2. Solve $\boldsymbol{W}_{AA}\boldsymbol{\beta} - \boldsymbol{s}_{AB} + \lambda\boldsymbol{z} = \boldsymbol{0}$, where $z_k = \operatorname{sign}(\beta_k)$ for $\beta_k \neq 0$ and $z_k \in [-1,1]$ for $\beta_k = 0$, using cyclic coordinate descent.
   3. Update $\boldsymbol{w}_{AB} = \boldsymbol{W}_{AA}\widehat{\boldsymbol{\beta}}$ and the corresponding row $\boldsymbol{w}_{BA} = \boldsymbol{w}_{AB}^\top$.
3. In the final cycle for each j, solve for the estimated precision matrix elements by $\widehat{\boldsymbol{\theta}}_{AB} = -\widehat{\boldsymbol{\beta}}\,\widehat{\theta}_{BB}$ with $1/\widehat{\theta}_{BB} = w_{BB} - \boldsymbol{w}_{AB}^\top\widehat{\boldsymbol{\beta}}$.

For the all-entry penalty written here, the diagonal KKT equations give $w_{jj} = s_{jj} + \lambda$. Some software instead penalises only off-diagonal entries, in which case $w_{jj} = s_{jj}$. Specify the convention before comparing estimates or tuning parameters. The algorithm treats λ as fixed; choosing it by cross-validation or a criterion is a separate outer calculation. A fitted zero is an estimate of a missing population edge, not a proof of one (Friedman et al., 2008, 2026).

Extended BIC.

One option for choosing λ is the extended Bayesian information criterion (Foygel and Drton, 2010). For a Gaussian graphical model with edge set E and control parameter $\gamma \in [0, 1]$,

$$\mathrm{BIC}_\gamma(E) = -2\,\ell\big(\widehat{\theta}(E)\big) + |E| \log n + 4|E|\gamma \log p. \tag{19.25}$$

Here $|E|$ counts unordered edges once. Diagonal precision parameters and any freely estimated means have the same dimension in every candidate graph, so their common BIC terms are omitted. $\widehat{\theta}(E)$ is the unpenalised constrained MLE when it exists. At $\gamma = 0$, the criterion agrees with BIC up to those common terms. The extra $4|E|\gamma \log p$ charge accounts for the growing collection of graph candidates; consistency results impose assumptions on sample size, dimension, signal and the candidate graphs (Foygel and Drton, 2010).

Example 19.7 (Graphical lasso on simulated autoregressive data)**.** Generate $n = 120$ observations from the $p = 8$ Gaussian AR(1) model $\mathrm{Corr}(X_j, X_k) = 0.6^{|j-k|}$. Its true graph is a chain with seven edges out of 28 possible pairs. The R figure fits graphical lasso over 18 penalties from 0.02 to 0.65. For each candidate edge set it refits the unpenalised Gaussian likelihood subject to that graph, then evaluates BIC and EBIC. The final panel compares each criterion after subtracting its own minimum. The first three panels show a small penalty, an intermediate penalty and the EBIC choice; their numerical labels are generated from the R fits.

The refit matters: a penalised precision estimate is shrunk, so its likelihood is not the constrained maximum-likelihood value required by this version of BIC. The simulation has a known population graph, but neither the penalty nor the criterion guarantees its exact recovery in a finite sample. The laboratory below separately checks the population zero pattern and illustrates sampling error.

19.6 An application: the top fifteen genes from the colon-cancer FDR analysis

The course notes also report a graphical-lasso analysis of the fifteen genes with the smallest marginal p-values in the Alon analysis. The processed file is absent, so the following figures are reported results, not fits verified here. Approximate normality of each marginal variable would not establish joint multivariate normality. Pooling tumour and normal tissue, and selecting variables using the same data, add further qualifications to a population interpretation.

The notes report average absolute marginal correlation near 0.46. They describe normalised precision entries of -0.52 for genes 1843 and 1494, and $+0.47$ for genes 625 and 1635. Remember the sign reversal: a negative precision entry corresponds to positive partial correlation. After graphical-lasso selection using EBIC with $\gamma = 0.5$, the first pair reportedly has normalised precision -0.42. The gene indices are 1494 and 1843, rather than the truncated labels 149 and 184.

An estimated zero, such as the reported entry for genes 1042 and 377, is a restriction in the *fitted* Gaussian model. It is not independent evidence that the biological variables are conditionally independent in the population. Shrinkage, selection, sampling variation and misspecification can all affect the graph. The reproducible Gaussian simulation above lets us study these distinctions with a known population structure.

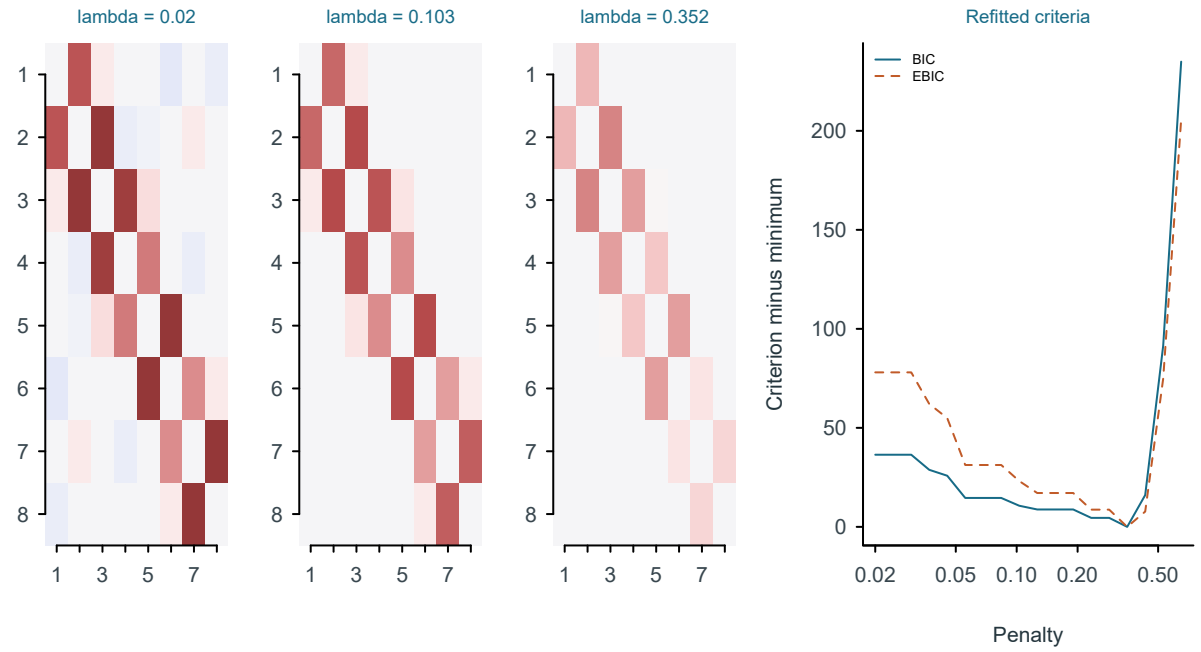


Figure 19.3: Graphical-lasso candidate graphs and BIC/EBIC scores after unpenalised refitting within each graph. The R simulation has eight variables and 120 observations.

Key idea.
In a Gaussian graphical model the graph is the sparsity pattern of the precision matrix $\boldsymbol{K} = \boldsymbol{\Sigma}^{-1}$. A zero k_{jk} means $X_j \perp\!\!\!\perp X_k$ given all the other variables, and $-k_{jk}/\sqrt{k_{jj}k_{kk}}$ is their partial correlation. For a known graph, the maximum likelihood estimate matches the sample covariance on the diagonal and on the edges. For an unknown graph, an L_1 penalty on $\boldsymbol{K}$ estimates the edges and the parameters together. λ is chosen by cross-validation or by an information criterion, ordinary or extended.

19.7 R laboratory: a chain with dense marginal correlations

Specify an AR(1) correlation matrix with $\Sigma_{ij} = 0.6^{|i-j|}$. Its inverse has zero entries except on the diagonal and the neighbouring off-diagonals: the marginal correlations are dense, and the conditional independences are the missing edges of a chain. Verify that algebra before estimating a graph from simulated data.

```
p <- 6
Sigma <- 0.6^abs(outer(1:p, 1:p, "-"))
K <- solve(Sigma)
partial <- -cov2cor(K); diag(partial) <- 1
round(partial, 3)
#>       [,1]  [,2]  [,3]  [,4]  [,5]  [,6]
#> [1,] 1.000 0.514 0.000 0.000 0.000 0.000
#> [2,] 0.514 1.000 0.441 0.000 0.000 0.000
#> [3,] 0.000 0.441 1.000 0.441 0.000 0.000
#> [4,] 0.000 0.000 0.441 1.000 0.441 0.000
#> [5,] 0.000 0.000 0.000 0.441 1.000 0.514
#> [6,] 0.000 0.000 0.000 0.000 0.514 1.000
```

```r
stopifnot(max(abs(K[abs(outer(1:p, 1:p, "-")) > 1])) < 1e-10)
set.seed(19006)
X <- MASS::mvrnorm(200, rep(0, p), Sigma)
S <- cov(X) * (nrow(X) - 1) / nrow(X)
fit <- glasso::glasso(S, rho = 0.1, penalize.diagonal = FALSE)
edges <- abs(fit$wi) > 1e-6; diag(edges) <- FALSE
round(-cov2cor(fit$wi) + diag(2, p), 2)
#>      [,1] [,2] [,3] [,4] [,5] [,6]
#> [1,] 1.00 0.44 0.07 0.00 0.00 0.00
#> [2,] 0.44 1.00 0.36 0.06 0.00 0.00
#> [3,] 0.07 0.36 1.00 0.28 0.00 0.05
#> [4,] 0.00 0.06 0.28 1.00 0.41 0.08
#> [5,] 0.00 0.00 0.00 0.41 1.00 0.36
#> [6,] 0.00 0.00 0.05 0.08 0.36 1.00
c(estimated_edges = sum(edges) / 2, true_edges = p - 1)
#> estimated_edges      true_edges
#>              10               5
```

This laboratory explicitly sets `penalize.diagonal = FALSE`. It therefore uses the off-diagonal convention, unlike the all-entry penalty in the displayed algorithm; its fitted covariance diagonal should match $\boldsymbol{S}$ to numerical tolerance.

The penalty was fixed for demonstration. An estimated missing edge may reflect shrinkage or sampling noise. In the population model the zero pattern has an exact conditional-independence meaning because Gaussianity and positive definiteness were assumed. An analogous inverse-covariance calculation on pooled iris flowers does not automatically inherit that interpretation: pooling distinct species mixes several conditional distributions.

19.8 Further exercises

Exercise 19.1 (Cliques and separators)**.** For the graph of Figure 19.1, list $\mathrm{bd}(\{1,2\})$ and $\mathrm{cl}(\{5\})$, and state three conditional-independence statements implied by the global Markov property. Is $\{2\}$ a $(1,4)$-separator?

Solution

Exercise 19.2 (Precision to graph)**.** A normal vector (X_1, X_2, X_3) has precision matrix $\boldsymbol{K} = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 2 \end{pmatrix}$. Draw the graph, find the covariance matrix $\boldsymbol{\Sigma} = \boldsymbol{K}^{-1}$, and show that X_1 and X_3 are marginally correlated but conditionally independent given X_2. Compute the partial correlations.

Solution

Exercise 19.3 (A three-variable chain)**.** $X_1 \sim \mathrm{N}(0,1)$, $X_2 = 0.8X_1 + \epsilon_2$, $X_3 = 0.8X_2 + \epsilon_3$ with $\epsilon_2, \epsilon_3 \sim \mathrm{N}(0, 0.36)$ independent of everything else. Find $\boldsymbol{\Sigma}$ and $\boldsymbol{K} = \boldsymbol{\Sigma}^{-1}$, and check $k_{13} = 0$.

Solution

Exercise 19.4 (Conditional independence from partitioned $\boldsymbol{\Sigma}$)**.** Using Theorem 1.6, verify directly for the covariance matrix of Exercise 19.3 that $\mathrm{Cov}(X_1, X_3 \mid X_2) = 0$ but that $\mathrm{Cov}(X_1, X_3) \neq 0$.

Solution

Exercise 19.5 (Number of graphs)**.** How many undirected graphs are there on $p = 5$ vertices, and on $p = 20$? Why does this make search impossible without regularisation?

Solution

Exercise 19.6 (A tempting wrong turn)**.** A student draws a graph for gene-expression data by connecting every pair of genes whose sample correlation exceeds 0.5 in absolute value, and interprets the missing edges as conditional independencies. What is wrong with this, and what should be used instead?

Solution

Exercise 19.7 (Read a Gaussian graphical model)**.** A mean-zero Gaussian vector has positive-definite precision matrix

$$K = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix}.$$

You may use $\rho_{ij\cdot\text{rest}} = -K_{ij}/\sqrt{K_{ii}K_{jj}}$.

(a) Draw the undirected graph and state the conditional independence it encodes.

(b) Calculate the nonzero partial correlations and explain why the entries of K are not themselves partial correlations.

(c) Are X_1 and X_3 marginally independent? Justify your answer, using $\Sigma_{13} = 1/4$ if useful.

Solution

Exercise 19.8 (Conditional association and sparsity)**.** A Gaussian vector has precision matrix

$$K = \begin{pmatrix} 1 & -0.2 & 0 \\ -0.2 & 1 & 0.3 \\ 0 & 0.3 & 1 \end{pmatrix}.$$

(a) Draw its graph, state the conditional independence, calculate the two nonzero partial correlations, and explain why a missing edge need not mean zero marginal correlation.

(b) Explain the purpose of the penalty in the graphical lasso, how a tuning parameter may be chosen, and why selected edges should not automatically be described as causal effects.

Solution

Source map and editorial notes

Source coverage

The four principal lecture PDFs establish the chapter sequence below. Tutorial sheets and the already integrated exercises supplement that sequence. Printed page numbers refer to the lecture notes; slide numbers begin with the first slide. Academic references support the claims identified in the separate research audit.

Source	Coverage in this book
STAT3006 Statistical Learning: Lecture Notes (I. A. Wood, 1 Nov. 2025), Chapter 1, pp. 4–33	Part I: Chapters 1 (Sections 1.1–1.5), 2 (1.6–1.7), 3 (1.8)
Lecture notes, Chapter 2, pp. 34–62	Part II: Chapters 4 (2.1), 5 (2.2–2.4), 6 (2.5), 7 (2.6–2.7)
Lecture notes, Chapter 3, pp. 63–117	Part III: Chapters 8 (3.1–3.8), 9 (3.9), 10 (3.10), 11 (3.11), 12 (3.12), 13 (3.13–3.15)
Neural Networks lectures 2025b, slides 1–36 and 51–56; *XOR solutions* slide	Chapter 14: neurons, history, decision boundaries, XOR and its two hand-built solutions, universal approximation
Neural-network slides 37–61	Chapter 15: error, gradient descent, single-layer and multi-layer derivations
Neural-network slides 49–50, 62–93	Chapter 16: initialisation, batch/online learning, data preparation, softmax, generalisation, NETtalk, ALVINN, ZIP codes
STAT3006/7305 SVM 2025b, slides 1–26	Chapter 17
Lecture notes, Chapter 4, pp. 118–138	Part V: Chapter 18
Lecture notes, Chapter 5, pp. 139–157	Part V: Chapter 19
Lecture notes, Appendices A–G, pp. 158–178	Appendix A (A–E) and Appendix B (F, G)

The notes' bibliography is merged into a single list at the end of the book, together with the references that the slides cite.

Lecture transcripts

The spoken lectures supplied motivation, counterexamples, and a small number of corrections. Those contributions are absorbed into the chapter arguments. The main text does not attribute them to a classroom, and it does not record who asked a question.

Three corrections change a result that is easy to mis-state.

1. **The Hessian in the Laplace approximation** (Chapter 6). Inversion preserves positive definiteness: the eigenvalues of the inverse are the reciprocals of the eigenvalues. The matrix $\boldsymbol{H}$ in the second-order expansion is the negative Hessian of the log posterior at a mode, and that matrix must itself be positive definite. Positive semi-definiteness is not enough, because the determinant can vanish.

2. **The divisor in the normal likelihood** (Chapter 2). The maximum likelihood estimator of $\boldsymbol{\Sigma}$ divides by n. The unbiased sample covariance divides by $n - 1$. They are different estimators. The likelihood derivation is the one used for the multivariate normal.

3. **The sign of BIC in `mclust`** (Chapter 6). The package reports a quantity whose sign is opposite to the penalty form minimised in the text. A larger reported value is the preferred model. The chapter states the convention next to the criterion.

Classroom logistics in the recordings — room checks, weather, attendance, illness, assignment deadlines — were not used. The formal development remains the lecture notes, as mapped above. Expanded proofs, exercises, figures, and R laboratories listed in the next section are independent of the transcripts.

The further-problems chapter is taken from the course tutorial sheets. The sheet solutions were not rewritten here, so those problems have no solution in this book. A few numerical corrections to the sheets are noted with the problems themselves.

What was added, and what was left out

Added.

The aim of the reconstruction is a self-contained account, so each chapter adds material that the source assumes or leaves to the reader. Typical additions are: expanded algebra at every step the source skips; short proofs of results the source only states (Propositions 18.1, 8.6, 18.4, and 13.3, Theorem 18.6, the perceptron and EM monotonicity arguments, the SVM dual and its KKT conditions); one fully worked numerical example in most sections; figures produced from simulations or from the sources' own small data sets; and exercises with solutions. Material from outside the sources is flagged in the text with a phrase such as "standard, but not on the slides".

Not reproduced.

Photographs and scanned figures from the slides (brain scans, neurons, the tsetse-fly map, NETtalk and ALVINN diagrams, the weight-decay plot of Krogh and Hertz) are described in words and, where a diagram carries mathematical content, redrawn (decision boundaries, the XOR networks, the error-versus-epoch curve, the lasso–ridge constraint picture, the loss functions). The Alon colon-cancer data set, the R packages' outputs other than those printed in the notes, and the Hinton diagram are not reproduced.

Editorial checks

The following places in the sources were ambiguous, inconsistent, or wrong. In each case the book states the source's version, the correct or standard alternative, and which one it follows.

1. **Weighted within-cluster scatter** (notes 2.1; Chapter 4). The pairwise form of the within-cluster scatter equals the sum of squared distances to the cluster means weighted by the cluster size, but the K-means algorithm decreases the *unweighted* sum. A numerical

example in which the weighted criterion increases at a reassignment is stated; software reports the unweighted sum.

2. **SVD and orthogonality** (notes 1.8; Chapter 3). The statement $\boldsymbol{U}\boldsymbol{U}^\top = \boldsymbol{I}$ holds for the full SVD; for the thin SVD used in practice $\boldsymbol{U}^\top\boldsymbol{U} = \boldsymbol{I}$.

3. **"Average linkage"** (notes 2.6; Chapter 7). The notes first describe it as the distance between cluster means (centroid linkage), then as the average of all pairwise distances (group-average linkage). The two differ; the book defines the latter as average linkage and notes that centroid linkage can produce inversions.

4. **Appendix reference** (notes 3.11; Chapter 11). The notes point to Appendix B for the nearest-neighbour R code; it is in Appendix F.

5. **Penalised impurity** (notes 3.12; Chapter 12). The cost-complexity criterion adds leaf impurities without weights; the standard (CART) version weights each leaf by its share of the training data.

6. **Dimension of the label vector** (notes 3.15; Chapter 13). $\boldsymbol{y}_{\text{train}} \in \{-1, 1\}^p$ should be $\{-1, 1\}^{n_{\text{train}}}$.

7. **Dropped exponential factor** (NN slides 47–48; Chapter 15). The single-layer gradient omits the factor e^{-z} from the derivative of the sigmoid. The corrected result agrees with the hidden–output update on slide 59 and passes a numerical gradient check.

8. **Objective of the SVM** (SVM slides; Chapter 17). The slides minimise $\boldsymbol{w}^\top\boldsymbol{w}$ but quote the dual and solution that correspond to $\frac{1}{2}\boldsymbol{w}^\top\boldsymbol{w}$; in the soft-margin case the factor only rescales C.

9. **Missing transposes and the lasso reference** (notes 4.4; Chapter 18). The least-squares and ridge solutions are printed with $\boldsymbol{X}\boldsymbol{y}$ for $\boldsymbol{X}^\top\boldsymbol{y}$ in Section 4.4; the lasso paper is cited as 1994 and was published in 1996. The notes' description of the false discovery proportion in Section 4.3 is reworded to say that it is the proportion of declared-significant hypotheses that are true nulls.

10. **Equation label and gene indices** (notes 5.1, 5.6; Chapter 19). The list of equations (5.3)–(5.7) ends with the label (5.8) on a blank line. The genes with strongest positive conditional dependence are named in the notes as "149 and 184", which are not among the fifteen genes analysed; the book reads them as 1494 and 1843 (the heaviest edge in the notes' plot) and says that this is an inference.

11. **Notation of the known-graph algorithm** (notes 5.4; Chapter 19). The derivation isolates the last variable p and writes A for the other indices, while the algorithm is stated for the jth variable with parts A (all but j) and B (the jth row and column). The book keeps both forms and explains that the algorithm is the derivation applied to each variable in turn after moving it to the last position.

12. **Heart-failure output** (notes 3.10; Chapter 10). The R output is reproduced as printed (with the duplicate `summary` call of the notes omitted); every interpretive number in the example (odds ratios, intervals, the deviance comparison, the sample size $n = 299$ implied by the 298 degrees of freedom) was recomputed from the printed coefficients.

Corrections in the guided revision

The exposition was rewritten so that each chapter argues from a concrete difficulty to the construction that resolves it. Exercise labels, solution environments, R chunk labels and the 24

further problems were counted before and after that rewrite; the counts match the snapshot of 12 October 2026 in `snapshots/2026-10-12`.

1. **Gaussian conditional mean and least squares** (Chapter 1, and the regression form in Chapter 19). An earlier sentence said that fitting a bivariate normal and fitting least squares to the same cloud produce different lines. That is false for the conditional-mean line. On a complete sample, with an unconstrained mean and a covariance that uses one divisor in every entry,
$$\widehat{\beta} = \frac{s_{YX}}{s_{XX}}, \qquad \widehat{\alpha} = \widehat{\mu}_Y - \widehat{\beta}\widehat{\mu}_X$$
are exactly the ordinary-least-squares intercept and slope. The divisor n or $n-1$ cancels in the ratio. What does not agree with least squares is the major axis of the ellipse, and the line obtained by regressing X on Y and solving for Y. The Chapter 1 laboratory checks the identity on the setosa sepal measurements. The population residual $X_A - \mu_{A|B}(X_B)$ is independent of X_B under joint normality. A residual from a line fitted to the same sample is orthogonal to the regression columns by the normal equations; it is not independent of those columns as a random variable, because the fitted coefficients are functions of the sample.

2. **The one-sample t statistic** (Chapter 2). The statistic used as the template for Hotelling's test is a pivot under the null: its null distribution does not depend on (μ, σ^2). It is computed at the null value, so that property is not ancillarity for the unrestricted model. The rejection rule is unchanged.

Verification record of the earlier edition

The previous edition reported independent numerical and mathematical checks, and retained its checking scripts. That historical report is not used as proof that the present release has been verified. This revision executes the R laboratories and current checking scripts afresh, audits labels and coverage against its own dated snapshot, and records rendered-page inspection separately. Consult the release verification report for the actual results and their scope.

Research and exposition revision, 13 October 2026

This revision develops methods from their purpose, supplies missing intermediate reasoning and separates model consequences from empirical claims. All 19 chapters and all 144 existing solutions receive an exposition pass. The 24 tutorial problems remain without newly added solutions. Stable labels and citation keys are retained.

Substantive corrections include the conditions for EM convergence and Gaussian likelihood existence; the limitations of ordinary BIC for singular mixtures; selection bias in validation; the assumptions of neural-network approximation; gradient signs and loss normalisations; FDR versus a realised false-discovery proportion; SVM margins and kernel limits; and Gaussian graphical-model assumptions. Historical results without the processed data are explicitly marked as reported results. Detailed claim-to-source mappings, correction records and verification results accompany the editable project.

Limitations

This book reconstructs lecture materials; it does not replace the references the materials cite. Several results are quoted without proof from those references (the multivariate central limit theorem, the distribution of the Wishart matrix, the properties of the Hammersley–Clifford theorem, the universal approximation theorem, the consistency of nearest neighbours), and the

text says so where it happens. The simulation studies are illustrations: they use a single data set or a few hundred replications and carry no claims about general performance.

Changes in the executable R edition

All 43 figures in the reconstructed book, including its three embedded diagrams, are recreated by R functions. Their dataset, simulation seed, fitted model or layout can differ from the earlier edition; the new captions specify what is actually plotted. Earlier hand-worked examples and exercise solutions are retained. An inherited simulation result describes its stated original experiment, and should not be read as a numerical description of a different R figure. The new laboratories use R-computed output rather than transcribed values.

The original source PDFs include photographs and historical application images which the earlier book only described. This edition preserves those descriptions; the figure-reproduction inventory concerns the figures in the reconstructed book, not a pixel-for-pixel copy of every slide.

Further problems

These 24 tutorial problems extend the chapter exercises. Some ask for a proof, others for a numerical argument or an R analysis. Identify which kind of answer is required before computing: simulated agreement is not a proof, and an unevaluated code fragment is not a numerical result. The code blocks are starting points. Solutions to these tutorial problems have deliberately not been added in this revision.

Multivariate statistics

T-MV1: Marginal and conditional distributions

Assume $(X_1, X_2)^T$ is bivariate normal with

$$\mu = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} 1 & 2 \\ 2 & 5 \end{pmatrix}.$$

(a) Derive an expression for the marginal distribution of X_1.

(b) Derive an expression for the conditional distribution of X_1 given $X_2 = 0$.

(c) What conditions must be met for a real symmetric $p \times p$ matrix A to be positive definite?

(d) Check, using mathematical properties rather than code, whether the covariance matrix above is positive definite. Explain the method.

T-MV2: Two independent normal vectors

The vectors $X = (X_1, X_2, X_3)^T$ and $Y = (Y_1, Y_2, Y_3)^T$ are independent, with trivariate normal distributions and parameters

$$\mu_X = \begin{pmatrix} 2 \\ 2 \\ 2 \end{pmatrix}, \quad \Sigma_X = \begin{pmatrix} 3 & 2 & 1 \\ 2 & 4 & 1 \\ 1 & 1 & 2 \end{pmatrix}, \qquad \mu_Y = \begin{pmatrix} 3 \\ 4 \\ 2 \end{pmatrix}, \quad \Sigma_Y = \begin{pmatrix} 4 & 2 & 0 \\ 2 & 4 & 2 \\ 0 & 2 & 4 \end{pmatrix}.$$

(a) Determine the distribution of $X - Y$.

(b) Determine the correlation matrix of X.

(c) Use R to determine the correlation matrix of Y.

(d) What is the joint distribution of Y_1 and Y_2 conditional on $Y_3 = 4$?

(e) What is the partial correlation of Y_1 and Y_2 given Y_3?

Editorial note: The sheet permits R or Python for computational parts. This edition uses R consistently, as requested. The conditional mean depends on the value 4; the conditional covariance and partial correlation do not depend on that value in this Gaussian model.

T-MV3: Linear combinations without independence

For scalar variables, recall

$$E(aX_1 + bX_2) = aE(X_1) + bE(X_2)$$

and

$$\mathrm{Var}(aX_1 + bX_2) = a^2 \,\mathrm{Var}(X_1) + b^2 \,\mathrm{Var}(X_2) + 2ab \,\mathrm{Cov}(X_1, X_2).$$

Derive corresponding expressions for the mean and covariance of $AX + BY$, where X and Y are p-dimensional random vectors and A, B are fixed matrices with compatible dimensions. **Do not assume independence.**

You may take $Z = (X^T, Y^T)^T$, choose a suitable matrix C, and use the properties of CZ. Let $E(X) = \mu_X$, $E(Y) = \mu_Y$, $\mathrm{Cov}(X) = \Sigma_{XX}$, $\mathrm{Cov}(Y) = \Sigma_{YY}$ and $\mathrm{Cov}(X, Y) = \Sigma_{XY}$.

T-MV4: Simulation, estimation, accuracy and plotting

Use the distribution of X **in T-MV2**, with its stated numerical mean and covariance.

(a) Generate 100 pseudorandom vectors from this distribution using R's `mvtnorm` package or an equivalent function such as `MASS::mvrnorm`.

(b) Calculate the sums-of-squares-and-cross-products matrix A and the maximum likelihood estimates of all parameters of the corresponding normal model, using the simulated data.

(c) Calculate the Euclidean error between the estimated and true mean vectors.

(d) Calculate the Frobenius norm of the difference between the estimated and true correlation matrices.

(e) Plot two or three contours of the first-two-coordinate marginal density, for both the true and estimated distributions, in different colours. Add the observations.

(f) If you had instead generated 100,000 observations, how could you reasonably make a similar plot? Implement the method and produce a plot. Does it also work well for the original sample size of 100?

Editorial correction: The sheet says "X given in the previous question", although its numerical normal distribution is in Q2. The reference is resolved here. Remember that the normal covariance MLE uses divisor n, whereas the usual unbiased sample covariance uses $n - 1$.

Clustering

T-CL1: Random mixture distributions

You will generate a randomly chosen three-component normal mixture. A covariance matrix must be symmetric and positive semidefinite; use positive-definite matrices here. Filling a symmetric matrix with off-diagonal numbers between -1 and 1, even with unit diagonal, does not guarantee a correlation matrix.

The sheet proposes this construction. Form a real symmetric matrix B, with diagonal entries 1 and sampled off-diagonal entries. If its smallest eigenvalue $\lambda_{\min}$ is not positive, replace it by

$$B + (-\lambda_{\min} + \epsilon)I, \qquad \epsilon > 0.$$

Adding aI shifts each eigenvalue by a without changing the eigenvectors. Convert the resulting positive-definite covariance matrix to a correlation matrix using diagonal rescaling, for example `cov2cor()`. Off-diagonal zero locations are preserved. This constructs valid matrices but does not claim a particular standard distribution over correlation matrices. The sheet also mentions

normal priors for means, Dirichlet priors for weights, and Wishart/inverse-Wishart alternatives for covariance matrices.

(a) Randomly generate three valid 3×3 correlation matrices. Use these as the covariance matrices of three mixture components.

(b) Generate three random three-dimensional mean vectors, drawing every coordinate from a standard normal distribution.

(c) Draw one vector from a Dirichlet distribution with parameters $(1, 1, 1)$ for the component proportions. The following R starter is equivalent to the sheet's `MCMCpack::rdirichlet(1,c(1,1,1))` call:

```
weights <- rgamma(3, shape = 1, rate = 1)
props <- weights / sum(weights)
```

(d) Draw 100 observations from this mixture distribution.

(e) For each pair of coordinates, plot two contours of each component's marginal distribution. Include the component proportions: use the same **weighted density levels** for each component.

(f) Choose the two clusters that appear closest. Conduct a two-sample Hotelling T^2 comparison of their means under a common-covariance assumption. Use R to obtain the sample means and pooled covariance, calculate the F statistic, and show all working.

(g) Apply K-means with $K = 1, \ldots, 5$.

(h) Use the Gap statistic to choose a number of clusters.

(i) Fit mixture models with one to five components.

(j) Use BIC to choose a mixture model.

Interpretation note for (f): This reproduces the source task, but the simulated component covariances generally differ, and choosing the "closest" pair from the data is selection. A textbook equal-covariance F reference does not automatically give a valid unselected confirmatory p-value here. State how you identify the two groups, examine the common-covariance assumption, and distinguish a worked calculation from a calibrated test of a prespecified comparison. Use 5% if you need a decision level; that is an editorial convention because the sheet does not specify one.

Classification 1: the Bayes classifier

T-C11: A known two-class distribution

There are two classes. Class 1 has probability 0.2 and class 2 probability 0.8. Given class 1, the univariate predictor is normal with mean 2 and variance 4; given class 2 it is normal with mean 4 and variance 4.

(a) Write a formal description of the data distribution, denoting the class by Y and predictor by X.

(b) In R, plot the marginal density of X together with the two weighted component densities.

(c) Determine the Bayes-optimal classification rule for equal misclassification costs.

(d) Determine its expected overall and class-specific error rates. Round to three significant figures only at the end.

(e) Describe the role of this classifier when comparing classifiers fitted to data generated from this distribution.

(f) Fit a classifier of your choice using 1000 observations generated from the distribution. Estimate its overall and class-specific errors, compare them with the Bayes errors, and comment.

Evaluation note: State whether the estimate in (f) uses an independent simulated test sample or an appropriate cross-validation procedure. An apparent error on the same 1000 fitting observations is a training error, not an honest estimate of future error. A new random draw need not match an earlier simulation.

Classification 2: iris and discriminant analysis

Use R's `iris` data: 150 flowers, four measurements in centimetres and three species. Start with **sepal length and sepal width**, so the fitted boundaries can be displayed in two dimensions. Then examine the effect of using all four measurements. This is the same iris dataset used in the chapter laboratories.

T-C21: Plot the data

Plot the observations in the sepal-length/sepal-width plane, distinguishing the species. The following starter preserves the sheet's task:

```
colours <- c("red", "green", "blue")
shapes <- c(0, 2, 5)
groups <- as.integer(iris$Species)
plot(iris$Sepal.Length, iris$Sepal.Width,
     pch = shapes[groups], col = colours[groups],
     main = "Iris data", xlab = "Sepal length (cm)",
     ylab = "Sepal width (cm)")
legend("topright", legend = levels(iris$Species),
       col = colours, pch = shapes)
```

T-C22: Fit LDA

Fit an LDA classifier to these two predictors, predict the labels of the observed flowers, and construct a confusion matrix.

```
modlda <- MASS::lda(Species ~ Sepal.Length + Sepal.Width,
                    data = iris)
ldapred <- predict(modlda, newdata = iris)$class
table(True = iris$Species, Predicted = ldapred)
```

T-C23: Estimate error rates

Use that confusion matrix to calculate overall and class-specific error rates. As an optional extension, repeat the error estimate by leave-one-out cross-validation.

Interpretation note: The first matrix is resubstitution on the fitting data. Call its rates apparent/-training errors. Report validation/CV errors separately, and state which dimension of the table represents the true class.

T-C24: Display classification regions

Pass a grid of points through the fitted classifier to display its decision regions and boundaries. You may start with sepal lengths from 4 to 8 and widths from 2 to 5, at increments of 0.1. Add the original observations and a species legend. The sheet permits changing colours, using a finer grid, and adding class-model density contours.

The following vectorised starter replaces the sheet's nested prediction loop without changing the task:

```
lengths <- seq(4, 8, by = 0.1)
widths <- seq(2, 5, by = 0.1)
grid <- expand.grid(Sepal.Length = lengths, Sepal.Width = widths)
pred <- predict(modlda, newdata = grid)$class
regions <- matrix(as.integer(pred), nrow = length(lengths))
image(lengths, widths, regions, col = colours,
      xlab = "Sepal length (cm)", ylab = "Sepal width (cm)")
points(iris$Sepal.Length, iris$Sepal.Width,
       pch = shapes[groups], col = colours[groups])
```

The original measurements are recorded to 0.1 cm; a finer display grid is a drawing choice, not finer measurement precision.

T-C25: Use all four measurements

Repeat the fitting and error assessment using all four measurements. Compare the results with the two-predictor analysis. If showing two-dimensional plots from the four-dimensional rule, state which other coordinates are held fixed: these are **slices**, not the entire decision boundary.

T-C26: Try other classifiers

If time permits, repeat the analysis for other classifiers: QDA, mixture discriminant analysis, nearest neighbours, classification trees, SVM kernels, random forests, boosting, multinomial logistic regression, or a neural network.

Use a consistent assessment design when comparing models. If that comparison selects a model, its selected validation score must not be reused as an independent final performance estimate.

Classification 3: nearest neighbours and validation

T-C31: Training, validation and test data

The k-nearest-neighbour method classifies unlabelled observations. You want to select k to minimise error on new observations and estimate the resulting classifier's accuracy. There are enough labelled observations to form separate training, validation and test sets.

(a) Explain what normalising the predictors means and why it is desirable for nearest neighbours.

(b) Explain how the three sets should be used to achieve these aims.

(c) Across repeated applications to different datasets, the chosen k tends to have higher estimated validation accuracy than test accuracy. Explain why.

These are the direct tutorial counterparts of 2025 exam Q3(a–c).

T-C32: Tune nearest neighbours on iris

(a) Apply nearest-neighbour classification with $k = 1, \ldots, 10$ using iris sepal length and sepal width.

(b) Select k using, for example, cross-validation. Explain why the same optimised CV score cannot also serve as an unbiased assessment of the effect of that selection.

(c) Construct the final classifier using the selected k.

(d) Visualise its decision regions using the grid method from Classification 2. Compare the result with other k choices.

In `datasets::iris` the sepal measurements are columns 1 and 2, `Sepal.Length` and `Sepal.Width`. Using all four measurements is a different predictor set and should be labelled as such. Fit scaling within each training fold and document tie handling and random seeds where ties are randomised.

T-C33: Validation for a tree

Is a validation set useful with a decision-tree classifier? Explain why or why not.

Graphical models and high-dimensional analysis

T-G1: Precision matrix to graph

Consider a mean-zero Gaussian model for $X_1, \ldots, X_5$ with precision matrix defined by

```
prectut <- matrix(c(
   1,   0.1, -0.3, -0.4,  0,
   0.1, 1,    0,    0,    0,
  -0.3, 0,    1,   -0.5,  0,
  -0.4, 0,   -0.5,  1,   -0.2,
   0,   0,    0,   -0.2,  1
), nrow = 5, byrow = TRUE)
```

Draw the corresponding undirected graph. The sheet suggests `qgraph`; a correctly labelled base-R graph is also suitable.

Editorial correction: The sheet and its solution say unit diagonal makes this precision matrix the partial-correlation matrix. For distinct indices,

$$\rho_{ij\cdot\text{rest}} = -K_{ij}/\sqrt{K_{ii}K_{jj}},$$

so unit diagonal makes the off-diagonal partial correlations **the negatives** of the precision entries. The zero pattern, hence the unweighted graph, is unchanged. Label any signed edges by the correct quantity.

T-G2: Recover the covariance

Give the covariance matrix of $X_1, \ldots, X_5$.

T-G3: Simulate the model

Generate 200 observations from a multivariate normal distribution with mean zero and the stated precision matrix. The sheet suggests `mvtnorm`; `MASS::mvrnorm` is an equivalent option. Its covariance argument is a covariance matrix, not a precision matrix.

T-G4: Estimate the graph

Attempt to recover the precision matrix and graph from the sample. The source proposes graphical lasso with the `huge` package and EBIC selection:

```
glasso.out <- huge::huge(datat, method = "glasso")
glasso.sel <- huge::huge.select(glasso.out, criterion = "ebic")
bestind <- glasso.sel$opt.index
precm <- glasso.sel$icov[[bestind]]
```

Compare the estimated zero pattern with the true graph. If drawing partial-correlation edge weights, convert the estimated precision matrix using the formula above before plotting. Rounding can hide small nonzero estimates; it should not silently replace the estimator's actual zero pattern.

T-G5: Add many noise predictors

Generate 200 observations for another 995 variables $X_6, \ldots, X_{1000}$, independently standard normal and independent of the first five variables. Combine them with the data in T-G3 into a 200×1000 matrix, for example using `cbind()`.

T-G6: Generate the response

For observation $i = 1, \ldots, 200$, generate

$$Y_i = 2 + \sum_{j=1}^{1000} \beta_j X_{ij} + \epsilon_i, \qquad \epsilon_i \overset{\text{iid}}{\sim} N(0, 4), \qquad \beta_j = \begin{cases} 1, & j = 1, \ldots, 5, \\ 0, & j = 6, \ldots, 1000. \end{cases}$$

Assume the errors are independent of the predictors. Only the first five predictors enter the conditional mean.

Editorial correction: The sheet uses an inconsistent predictor index for the noise term. Here there is one independent error per observation, and the stated $\sigma = 2$ means variance $\sigma^2 = 4$.

T-G7: Fit the lasso

Use standard lasso to try to recover the relevant predictors and their coefficients. Suggested R functions are `glmnet::cv.glmnet`, `glmnet::glmnet`, `coef` and `which`. State the tuning rule and whether you include an unpenalised intercept.

T-G8: Interpret the selection

Comment on the results. Compare selected variables and estimated coefficients with the known simulation truth. Distinguish prediction quality, support recovery, coefficient shrinkage and random variation; one successful or unsuccessful simulation does not establish a general guarantee.

T-G9: Multiple marginal regressions

Fit a separate regression of Y on each of the 1000 predictors and apply an FDR procedure at target level 0.1 to the individual slope tests. Report the predictors identified as associated with the response and comment.

There are **1000 tests**. State the procedure and its assumptions: ordinary Benjamini-Hochberg control requires suitable dependence conditions; Benjamini-Yekutieli is an option under arbitrary dependence with valid marginal p-values. The target is an expectation over repeated datasets, not a guarantee that each selected list contains at most 10% false discoveries. Marginal association and a nonzero coefficient in a joint conditional-mean model are different hypotheses.

Part VI

Worked solutions

Solutions for Chapter 1

Exercise 1.1.

Turn the matrix expression into the variance of a scalar. That gives a nonnegative quantity without needing to compute any eigenvalues. The proposed correlation matrix can then be checked separately.

For any fixed $\boldsymbol{z}$, $\boldsymbol{z}^\top \boldsymbol{\Sigma} \boldsymbol{z} = \mathbb{E}[\boldsymbol{z}^\top (\boldsymbol{X} - \boldsymbol{\mu})(\boldsymbol{X} - \boldsymbol{\mu})^\top \boldsymbol{z}] = \mathbb{E}[(\boldsymbol{z}^\top (\boldsymbol{X} - \boldsymbol{\mu}))^2] = \mathrm{Var}(\boldsymbol{z}^\top \boldsymbol{X}) \geq 0$.

Taking $\boldsymbol{z}$ with only entries j and k nonzero shows that the 2×2 matrix $\begin{pmatrix} \sigma_{jj} & \sigma_{jk} \\ \sigma_{jk} & \sigma_{kk} \end{pmatrix}$ is positive semi-definite, so its determinant $\sigma_{jj}\sigma_{kk} - \sigma_{jk}^2 \geq 0$, which is $|\sigma_{jk}| \leq \sqrt{\sigma_{jj}\sigma_{kk}}$.

For the matrix given, the leading minors are 1, $1 - 0.81 = 0.19$, and

$$\begin{aligned} \det &= 1\,(1 - 0.81) - 0.9\,(0.9 - (-0.9)(0.9)) + 0.9\,(0.9(-0.9) - 0.9) \\ &= 0.19 - 0.9(1.71) + 0.9(-1.71) = 0.19 - 1.539 - 1.539 = -2.888 < 0. \end{aligned}$$

The determinant is negative, so the matrix is not positive semi-definite and cannot be a correlation matrix. Intuitively, X_1 cannot be strongly positively correlated with both X_2 and X_3 while X_2 and X_3 are strongly negatively correlated.

Exercise 1.2.

Write the coefficients of each linear combination as a vector. The covariance matrix then handles both the diagonal variance terms and the cross terms; the latter must be counted twice in a variance.

With $\boldsymbol{a} = (1, -2, 1)^\top$,

$$\begin{aligned} \mathrm{Var}(\boldsymbol{a}^\top \boldsymbol{X}) &= \sum_{j,k} a_j a_k \sigma_{jk} = 1 \cdot 4 + 4 \cdot 3 + 1 \cdot 2 + 2\big[(1)(-2)(2) + (1)(1)(1) + (-2)(1)(0.5)\big] \\ &= 18 + 2(-4 + 1 - 1) = 10. \end{aligned}$$

With $\boldsymbol{a} = (1, 1, 0)^\top$ and $\boldsymbol{b} = (0, 1, -1)^\top$, $\mathrm{Cov}(\boldsymbol{a}^\top \boldsymbol{X}, \boldsymbol{b}^\top \boldsymbol{X}) = \boldsymbol{a}^\top \boldsymbol{\Sigma} \boldsymbol{b}$, and $\boldsymbol{\Sigma}\boldsymbol{b} = (2-1,\ 3-0.5,\ 0.5-2)^\top = (1,\ 2.5,\ -1.5)^\top$, so the covariance is $1 + 2.5 = 3.5$.

Exercise 1.3.

The proposed example must pass two different tests: its covariance must vanish, and its joint distribution must still contain dependence. Symmetry settles the first; the deterministic relation settles the second.

$\mathrm{Cov}(X, X^2) = \mathbb{E}(X^3) - \mathbb{E}(X)\mathbb{E}(X^2) = 0 - 0 = 0$, because the third moment of a symmetric distribution about zero vanishes. Yet $Y = X^2$ is completely determined by X (and $\mathbb{P}(|X| > 1\,|$

$Y \leq 1) = 0$ while $\mathbb{P}(|X| > 1) > 0)$, so they are not independent. Zero covariance rules out a nonzero linear covariance; it does not rule out nonlinear dependence.

Exercise 1.4.

Diagonalisation identifies the directions, while the contour level determines how far to travel along them. Only after those two geometric steps do we choose a probability content.

The characteristic equation is $(2-\lambda)^2 - 1 = 0$, so $\lambda_1 = 3$ and $\lambda_2 = 1$. Solving $(\boldsymbol{\Sigma} - \lambda \boldsymbol{I})\boldsymbol{v} = 0$ gives $\boldsymbol{\gamma}_1 = (1,1)^\top/\sqrt{2}$ and $\boldsymbol{\gamma}_2 = (1,-1)^\top/\sqrt{2}$. The contour $\boldsymbol{x}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{x} = C$ has semi-axes $\sqrt{C\lambda_j}$ along $\boldsymbol{\gamma}_j$.

By Remark 1.4, $\boldsymbol{X}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{X} \sim \chi^2_2$, so probability 0.95 requires $C = \chi^2_{2;0.05} = -2\ln 0.05 = 5.991$. The semi-axes are then $\sqrt{5.991 \times 3} = 4.24$ along $(1,1)$ and $\sqrt{5.991 \times 1} = 2.45$ along $(1,-1)$.

Exercise 1.5.

Checking the two marginal distributions cannot settle joint normality. Find a linear combination that a jointly Gaussian vector could not produce.

For any y, $\mathbb{P}(Y \leq y) = \frac{1}{2}\mathbb{P}(X \leq y) + \frac{1}{2}\mathbb{P}(-X \leq y) = \Phi(y)$ because $-X \sim \mathrm{N}(0,1)$ as well. So $Y \sim \mathrm{N}(0,1)$. The covariance is $\mathbb{E}(XY) = \mathbb{E}(WX^2) = \mathbb{E}(W)\mathbb{E}(X^2) = 0$.

Now $X + Y = (1+W)X$ equals $2X$ with probability $\frac{1}{2}$ and 0 with probability $\frac{1}{2}$. It therefore has an atom at 0 of mass at least $\frac{1}{2}$. A linear combination of a bivariate normal vector is normal (possibly degenerate, but then with no atom unless the combination is identically zero), so (X,Y) cannot be bivariate normal. The pair has normal marginals and zero covariance but is clearly dependent ($|Y| = |X|$).

Exercise 1.6.

Keep the prediction direction fixed while comparing lines. Predicting the first variable from the second and solving the reverse prediction equation are different operations.

With $A = \{1\}$, $B = \{2\}$: $\boldsymbol{\Sigma}_{AB} = \rho\sigma_1\sigma_2$, $\boldsymbol{\Sigma}_{BB} = \sigma_2^2$, $\boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1} = \rho\sigma_1/\sigma_2$. So the mean is $\mu_1 + \rho(\sigma_1/\sigma_2)(x_2 - \mu_2)$ and the variance is $\sigma_1^2 - (\rho\sigma_1\sigma_2)^2/\sigma_2^2 = \sigma_1^2(1-\rho^2)$.

The regression of X_1 on X_2 predicts x_1 from x_2 and has slope $\rho\sigma_1/\sigma_2$ in the (x_2, x_1) plane. The regression of X_2 on X_1 is $x_2 = \mu_2 + \rho(\sigma_2/\sigma_1)(x_1 - \mu_1)$; solving it for x_1 gives $x_1 = \mu_1 + \dfrac{\sigma_1}{\rho\sigma_2}(x_2 - \mu_2)$, with slope $\sigma_1/(\rho\sigma_2)$ in the same plane. For $\rho \neq 0$, the ratio of the two slopes is ρ^2, so the regression lines agree only in the degenerate limiting case $\rho^2 = 1$. When $\rho = 0$, they are the horizontal and vertical lines through the mean. A principal axis may coincide with one of these lines when the covariance is diagonal, and there is no unique principal axis when the covariance is proportional to the identity. Each line answers a different conditional question. This is not a disagreement between least squares and the conditional mean of an unconstrained bivariate normal fitted to the same pairs: those two lines coincide. The second line here is the inverse regression, solved for the other coordinate.

Exercise 1.7.

The determinant factorisation is easiest to check by calculating its two sides independently. This also checks the conditional-covariance arithmetic from the chapter.

Expanding along the first row,

$$|\boldsymbol{\Sigma}| = 4(3 \cdot 2 - 0.25) - 2(2 \cdot 2 - 0.5) + 1(1 - 3) = 4(5.75) - 2(3.5) - 2 = 23 - 7 - 2 = 14.$$

Also $|\boldsymbol{\Sigma}_{BB}| = 2$ and $|\boldsymbol{\Sigma}_{A|B}| = 3.5 \times 2.875 - 1.75^2 = 10.0625 - 3.0625 = 7$, so $|\boldsymbol{\Sigma}_{BB}||\boldsymbol{\Sigma}_{A|B}| = 14$ as required.

Exercise 1.8.

Use the Gaussian block formula for the conditional distribution, then define the residual by subtracting that conditional mean. Independence requires one more argument than a zero covariance.

(a) The conditional mean is $2 + (3/4)(3 + 1) = 5$ and variance is $9 - 3(1/4)3 = 27/4$. Thus $X_1 \mid X_2 = 3 \sim N(5, 27/4)$. A complete answer includes the selected blocks, mean, variance and complete distribution.

(b) $R = X_1 - 2 - \frac{3}{4}(X_2 + 1)$ is a linear transformation of a jointly normal vector, so $R \sim N(0, 27/4)$. Moreover $\mathrm{Cov}(R, X_2) = 3 - \frac{3}{4}(4) = 0$. Joint normality turns this zero covariance into independence.

(c) No. With only means and covariances, the analogous linear residual is uncorrelated with X_2, but conditional expectations need not be linear and uncorrelated variables need not be independent.

Solutions for Chapter 2

Exercise 2.1.

The sample mean is random, so centring at it changes the expected scatter. Isolate that change before taking expectations.

Write $\boldsymbol{X}_i - \boldsymbol{\mu} = (\boldsymbol{X}_i - \overline{\boldsymbol{X}}) + (\overline{\boldsymbol{X}} - \boldsymbol{\mu})$ and expand the outer product. The cross terms are $\sum_i (\boldsymbol{X}_i - \overline{\boldsymbol{X}})(\overline{\boldsymbol{X}} - \boldsymbol{\mu})^\top = \mathbf{0}(\overline{\boldsymbol{X}} - \boldsymbol{\mu})^\top = \mathbf{0}$ and its transpose, so

$$\sum_i (\boldsymbol{X}_i - \boldsymbol{\mu})(\boldsymbol{X}_i - \boldsymbol{\mu})^\top = \boldsymbol{A} + n(\overline{\boldsymbol{X}} - \boldsymbol{\mu})(\overline{\boldsymbol{X}} - \boldsymbol{\mu})^\top.$$

Take expectations. The left side has expectation $n\boldsymbol{\Sigma}$. For the last term, $\mathbb{E}[(\overline{\boldsymbol{X}} - \boldsymbol{\mu})(\overline{\boldsymbol{X}} - \boldsymbol{\mu})^\top] = \operatorname{Var}(\overline{\boldsymbol{X}}) = \boldsymbol{\Sigma}/n$, which uses *uncorrelatedness* of the $\boldsymbol{X}_i$ (independence): $\operatorname{Var}(\overline{\boldsymbol{X}}) = n^{-2} \sum_{i,l} \operatorname{Cov}(\boldsymbol{X}_i, \boldsymbol{X}_l) = n^{-2} \cdot n\boldsymbol{\Sigma}$. Hence $n\boldsymbol{\Sigma} = \mathbb{E}(\boldsymbol{A}) + n \cdot \boldsymbol{\Sigma}/n$, i.e. $\mathbb{E}(\boldsymbol{A}) = (n-1)\boldsymbol{\Sigma}$. Normality is not used; only a common mean and covariance and uncorrelated observations.

Exercise 2.2.

First decide which variation the common covariance is meant to describe. Pooling within groups and ignoring the group labels answer different questions.

With $n = 25$, $G = 2$, $n - G = 23$:

$$\boldsymbol{S} = \frac{1}{23}\left(\begin{pmatrix} 18 & 6 \\ 6 & 12 \end{pmatrix} + \begin{pmatrix} 30 & -9 \\ -9 & 24 \end{pmatrix} \right) = \frac{1}{23} \begin{pmatrix} 48 & -3 \\ -3 & 36 \end{pmatrix} = \begin{pmatrix} 2.087 & -0.130 \\ -0.130 & 1.565 \end{pmatrix}.$$

For the combined sample, $\overline{\boldsymbol{x}}_1 - \overline{\boldsymbol{x}}_2 = (2, 1)^\top$ and $n_1 n_2 / n = 150/25 = 6$, so

$$\boldsymbol{A}_{\text{total}} = \begin{pmatrix} 48 & -3 \\ -3 & 36 \end{pmatrix} + 6 \begin{pmatrix} 4 & 2 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} 72 & 9 \\ 9 & 42 \end{pmatrix}, \qquad \frac{\boldsymbol{A}_{\text{total}}}{24} = \begin{pmatrix} 3 & 0.375 \\ 0.375 & 1.75 \end{pmatrix}.$$

The combined-sample estimate is larger on the diagonal and even has the opposite sign for the covariance. It mixes the within-group scatter, which is what $\boldsymbol{\Sigma}$ describes, with the between-group scatter caused by the different means, which $\boldsymbol{\Sigma}$ does not describe. Pooling centres each group at its own mean first.

Exercise 2.3.

A simulation factor need only satisfy the required matrix product. Different valid factors can map the same normal draw to different points without changing the resulting distribution.

Cholesky: $\boldsymbol{L} = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix}$ with $l_{11}^2 = 4$, $l_{11} l_{21} = 2$, $l_{21}^2 + l_{22}^2 = 3$, so $l_{11} = 2$, $l_{21} = 1$, $l_{22} = \sqrt{2}$:

$$\boldsymbol{L} = \begin{pmatrix} 2 & 0 \\ 1 & 1.414 \end{pmatrix}, \qquad \boldsymbol{L}\boldsymbol{L}^\top = \begin{pmatrix} 4 & 2 \\ 2 & 1+2 \end{pmatrix} = \boldsymbol{\Sigma}.$$

Spectral: the eigenvalues solve $\lambda^2-7\lambda+8=0$, so $\lambda_1=(7+\sqrt{17})/2=5.562$ and $\lambda_2=(7-\sqrt{17})/2=1.438$, with unit eigenvectors $\boldsymbol{\gamma}_1=(0.788,0.615)^\top$, $\boldsymbol{\gamma}_2=(-0.615,0.788)^\top$. Then

$$\boldsymbol{B}=\boldsymbol{\Gamma}\boldsymbol{\Lambda}^{1/2}=\begin{pmatrix}1.859 & -0.738\\ 1.451 & 0.945\end{pmatrix}.$$

For either factor $\boldsymbol{M}\in\{\boldsymbol{L},\boldsymbol{B}\}$, $\mathrm{Var}(\boldsymbol{MZ})=\boldsymbol{MIM}^\top=\boldsymbol{MM}^\top=\boldsymbol{\Sigma}$. The factors differ because $\boldsymbol{MM}^\top=\boldsymbol{\Sigma}$ determines $\boldsymbol{M}$ only up to a right multiplication by an orthogonal matrix $\boldsymbol{Q}$: $(\boldsymbol{MQ})(\boldsymbol{MQ})^\top=\boldsymbol{MM}^\top$. The simulated vectors differ draw by draw but have the same distribution.

Exercise 2.4.

Compute the rank-one update directly, then use the inverse formula as an independent check. Agreement verifies this example of the determinant identity.

$|\boldsymbol{A}|=12-1=11$. Here $\boldsymbol{dd}^\top=\begin{pmatrix}1 & 2\\ 2 & 4\end{pmatrix}$, so $\boldsymbol{A}+\boldsymbol{dd}^\top=\begin{pmatrix}5 & 3\\ 3 & 7\end{pmatrix}$ with determinant $35-9=26$. Also $\boldsymbol{A}^{-1}=\frac{1}{11}\begin{pmatrix}3 & -1\\ -1 & 4\end{pmatrix}$, so $\boldsymbol{d}^\top\boldsymbol{A}^{-1}\boldsymbol{d}=\frac{1}{11}(3-4+16)=\frac{15}{11}$ and $|\boldsymbol{A}|(1+\boldsymbol{d}^\top\boldsymbol{A}^{-1}\boldsymbol{d})=11\,(1+15/11)=11+15=26$. The identities agree.

Exercise 2.5.

The quadratic form measures the observed displacement. Its reference distribution belongs to the random statistic under the null, rather than to the numerical value obtained after observing the sample.

Here $n=6$, $p=2$. $|\boldsymbol{S}|=4-1=3$ and $\boldsymbol{S}^{-1}=\frac{1}{3}\begin{pmatrix}1 & -1\\ -1 & 4\end{pmatrix}$. Then $\overline{\boldsymbol{x}}^\top\boldsymbol{S}^{-1}\overline{\boldsymbol{x}}=\frac{1}{3}(1\cdot4-2\cdot2\cdot1+4\cdot1)=\frac{4}{3}$, so $T^2=6\cdot\frac{4}{3}=8$. Under H_0 and the iid Gaussian assumptions, the random scaled statistic has an $F_{2,4}$ distribution. Its observed value is $F_{\text{obs}}=\frac{n-p}{p(n-1)}T^2=\frac{4}{10}\cdot8=3.2$. Since $3.2<6.944$, H_0 is not rejected at the 5% level (the p-value is about 0.15). This non-rejection does not establish the null. Power would require specifying an alternative mean and covariance.

Exercise 2.6.

Pool the within-group information, form the mean difference and apply the correct two-sample scaling. The reference distribution also requires independent Gaussian samples with a common covariance.

$\boldsymbol{S}^{-1}=\frac{1}{3}\begin{pmatrix}2 & -1\\ -1 & 2\end{pmatrix}$, so with $\boldsymbol{d}=(1.5,1)^\top$, $\boldsymbol{d}^\top\boldsymbol{S}^{-1}\boldsymbol{d}=\frac{1}{3}(2\cdot2.25-2\cdot1.5\cdot1+2\cdot1)=\frac{3.5}{3}=1.1667$. The factor is $n_1n_2/(n_1+n_2)=100/20=5$, so $T^2=5.8333$. Then

$$F=\frac{n_1+n_2-p-1}{(n_1+n_2-2)p}T^2=\frac{17}{36}\times5.8333=2.755.$$

The null reference distribution is $F_{2,17}$. Since $2.755<3.592$, $H_0:\boldsymbol{\mu}_1=\boldsymbol{\mu}_2$ is not rejected at the 5% level (the p-value is about 0.09).

Exercise 2.7.

Whitening is an exact statement about the covariance used in the transformation. Work through the matrix identity before deciding what it says about a population distribution.

(a) $S^{-1/2}=U\,\mathrm{diag}(\lambda_1^{-1/2},\ldots,\lambda_4^{-1/2})U^T$, using positive eigenvalues and an orthonormal eigenbasis.

(b) $\bar{z} = S^{-1/2}(\bar{x} - \bar{x}) = 0$. With the same $n-1$ covariance divisor,

$$S_z = \frac{1}{n-1}\sum_i S^{-1/2}(x_i - \bar{x})(x_i - \bar{x})^T S^{-1/2} = S^{-1/2} S S^{-1/2} = I_4.$$

(c) These are empirical first- and second-moment statements. A linear transform does not generally make a nonnormal distribution normal, and zero sample correlations do not establish population independence. When $p = 100$, $\operatorname{rank}(S) \leq 79$, so an ordinary inverse square root in all 100 dimensions does not exist; reduction to the positive-eigenvalue subspace or regularisation changes the construction.

Exercise 2.8.

Identify the response block and the conditioning block first. The subtraction in the conditional variance is the reduction in uncertainty supplied by the observed coordinate.

(a) The conditional mean is $1 + \frac{2}{9}(5-2) = 5/3$ and variance is $4 - \frac{2^2}{9} = 32/9$. Hence $X_1 \mid X_2 = 5 \sim N(5/3, 32/9)$. The displayed substitutions identify the blocks that were used.

(b) The residual is a linear combination of jointly normal variables with mean zero and variance $32/9$. Its covariance with X_2 is $2 - (2/9)9 = 0$, so it is independent of X_2. The conditional residual variance therefore does not depend on the observed x_2; this uses the specified Gaussian model, not a universal property of residuals from fitted models.

(c) The empirical mean is zero and empirical covariance is I_p, using the same covariance divisor before and after transformation.

Exercise 2.9.

A linear image of a jointly normal vector is normal. Its mean and variance identify the resulting univariate distribution, while whitening asks a separate question about a whole coordinate system.

(a) $E(Y) = 0 - 4 = -4$ and $\operatorname{Var}(Y) = 2 + 4(3) - 4(1) = 10$. Linear combinations of a jointly normal vector are normal, so $Y \sim N(-4, 10)$.

(b) Orthogonal eigenvectors give principal axes; an ellipsoid $(x-\mu)^T\Sigma^{-1}(x-\mu) = c$ has semiaxis lengths $\sqrt{c\lambda_j}$. Larger eigenvalues mean more spread. The mean translates the centre, without changing orientation.

(c) No. Whitening controls empirical first and second moments. It does not determine the shape of the population distribution, establish independence between coordinates, or make observations independent across rows. Even whitened jointly normal population coordinates would need the population transformation for that exact population claim.

Solutions for Chapter 3

Exercise 3.1.

The symmetric covariance treats the two coordinates equally, suggesting the sum and difference directions. The eigenvalue calculation confirms which direction has greater variance.

The characteristic equation is $(3-\lambda)^2 - 1 = 0$, so $\lambda_1 = 4$ and $\lambda_2 = 2$. Solving $(\boldsymbol{S} - 4\boldsymbol{I})\boldsymbol{a} = \boldsymbol{0}$ gives $a_1 = a_2$, so $\boldsymbol{a}_1 = (1,1)^\top/\sqrt{2}$; the orthogonal direction is $\boldsymbol{a}_2 = (1,-1)^\top/\sqrt{2}$. The component variances are 4 and 2, and the first explains $4/(4+2) = 2/3 \approx 66.7\%$ of the total variance.

Exercise 3.2.

Express the scores as an orthogonal change of coordinates. The covariance then becomes diagonal because the chosen directions diagonalise the same matrix used to define PCA.

$\mathrm{Cov}(\boldsymbol{Y}) = \boldsymbol{\Gamma}^\top \boldsymbol{S} \boldsymbol{\Gamma} = \boldsymbol{\Gamma}^\top \boldsymbol{\Gamma} \boldsymbol{\Lambda} \boldsymbol{\Gamma}^\top \boldsymbol{\Gamma} = \boldsymbol{\Lambda}$ by (1.12) with $\boldsymbol{A} = \boldsymbol{\Gamma}^\top$ and orthogonality of $\boldsymbol{\Gamma}$, so the components are uncorrelated with variances λ_j. For the determinant, $|\boldsymbol{S}| = |\boldsymbol{\Gamma}|\,|\boldsymbol{\Lambda}|\,|\boldsymbol{\Gamma}^\top| = |\boldsymbol{\Gamma}|^2 \prod_j \lambda_j = \prod_j \lambda_j$ since $|\boldsymbol{\Gamma}|^2 = |\boldsymbol{\Gamma}^\top \boldsymbol{\Gamma}| = 1$.

Exercise 3.3.

For positive correlation, the sum direction collects shared variation. The limiting cases explain what happens when that shared variation disappears or becomes perfect.

$|\boldsymbol{R} - \lambda \boldsymbol{I}| = (1-\lambda)^2 - r^2 = 0$ gives $\lambda = 1 \pm r$. Since $\boldsymbol{R}(1,1)^\top = (1+r)(1,1)^\top$ and $\boldsymbol{R}(1,-1)^\top = (1-r)(1,-1)^\top$, the eigenvectors are $(1,1)^\top/\sqrt{2}$ and $(1,-1)^\top/\sqrt{2}$ for every $r > 0$. The proportion explained by the first component is $(1+r)/2$. As $r \to 0$ it tends to $1/2$: with $\boldsymbol{R} \to \boldsymbol{I}$ no direction is preferred (every direction is an eigenvector, and the first component is determined only by tie-breaking). As $r \to 1$ it tends to 1: the two standardised variables become the same variable and one component carries all the variance.

Exercise 3.4.

The squared singular values belong to the cross-product matrix. Divide by the covariance divisor before reporting principal-component variances.

The eigenvalues of $\begin{pmatrix} 10 & 2 \\ 2 & 4 \end{pmatrix}$ solve $\lambda^2 - 14\lambda + 36 = 0$, so $\lambda = 7 \pm \sqrt{13} = 10.606,\ 3.394$. The singular values of $\boldsymbol{X}$ are $e_1 = \sqrt{10.606} = 3.257$ and $e_2 = \sqrt{3.394} = 1.842$. With $n = 4$, the sample covariance eigenvalues are $e_i^2/3 = 3.535$ and 1.131. Their sum is 4.667, and $\boldsymbol{S} = \frac{1}{3}\begin{pmatrix} 10 & 2 \\ 2 & 4 \end{pmatrix}$ has trace $(10+4)/3 = 4.667$.

Exercise 3.5.

Use cumulative explained variance for the requested threshold. The individual percentages alone do not say how many components must be retained.

The total is $5.87 + 1.81 + 1.32 = 9.00$. The proportions are 0.652, 0.201, 0.147 and the cumulative proportions are 65.2%, 85.3%, 100%. Two components are needed to reach 80%.

Exercise 3.6.

Construct a case where predictor variance and predictive relevance point in different directions. That is enough to disprove a universal claim that PCA preserves the useful information.

$\mathrm{Var}(X_1) = 100$ and $\mathrm{Var}(X_2) = 1$ with zero covariance, so $\boldsymbol{\Sigma} = \mathrm{diag}(100, 1)$ and the first principal direction is $\boldsymbol{a}_1 = (1, 0)^\top$: the first component is X_1 itself. But $Y = X_2$ is uncorrelated with X_1, so a regression of Y on the first component explains $R^2 = 0$ of the variance of Y, even though Y is perfectly predictable from X_2. This establishes zero population linear-regression R^2; independence would need the additional independent-coordinate assumption. PCA chooses directions by the variance of the explanatory variables alone, so a low-variance direction can carry all of the predictive information. Selecting components should be checked against the response (for example by cross-validation) rather than assumed to preserve predictive power.

Exercise 3.7.

Separate directions, scores and variance fractions: they are related objects, but they have different dimensions and roles. Then ask where the transformation was fitted.

(a) Centre the variables and, here, divide by training standard deviations. Diagonalise their covariance matrix $S = U\Lambda U^T$, sorting eigenvalues from largest to smallest. A direction u_j is a unit vector in predictor space; a score for observation x_i is $t_{ij} = u_j^T z_i$. The first direction maximises score variance subject to unit norm; subsequent directions impose orthogonality. The retained variance fraction is $(\lambda_1 + \lambda_2)/\sum_j \lambda_j$.

(b) Multiplying one variable by 10 changes its covariance contributions, so covariance-PCA directions can change. Standardisation cancels a positive change of measurement unit and makes correlation-based PCA invariant to it.

(c) PCA ignores the labels; a low-variance direction can separate classes. Fit centring, scaling and PCA on each training fold and apply that fitted transform to its validation fold. Choose component count and classifier settings within training data, leaving the final test set untouched.

Solutions for Chapter 4

Exercise 4.1.

Subtract the common mean inside each pairwise difference. The cross term vanishes only after summing, which is the step that converts pairwise scatter into scatter around a centre.

Add and subtract $\overline{\boldsymbol{x}}$: $\|\boldsymbol{x}_i - \boldsymbol{x}_l\|^2 = \|\boldsymbol{x}_i - \overline{\boldsymbol{x}}\|^2 + \|\boldsymbol{x}_l - \overline{\boldsymbol{x}}\|^2 - 2(\boldsymbol{x}_i - \overline{\boldsymbol{x}})^\top(\boldsymbol{x}_l - \overline{\boldsymbol{x}})$. Sum over all N^2 pairs. The first term sums to $N\sum_i\|\boldsymbol{x}_i - \overline{\boldsymbol{x}}\|^2$ (each i appears with N values of l), the second to the same, and the cross-term to $-2\big[\sum_i(\boldsymbol{x}_i - \overline{\boldsymbol{x}})\big]^\top\big[\sum_l(\boldsymbol{x}_l - \overline{\boldsymbol{x}})\big] = 0$. So the total is $2N\sum_i\|\boldsymbol{x}_i - \overline{\boldsymbol{x}}\|^2$. Applying this inside each cluster k (with $N = N_k$) and multiplying by $\frac{1}{2}$ gives $W(C) = \sum_k N_k \sum_{C(i)=k}\|\boldsymbol{x}_i - \overline{\boldsymbol{x}}_k\|^2$.

Exercise 4.2.

Cluster labels are arbitrary, so swapping the two sides does not create a new partition. Enumerating this small case makes the division by label permutations concrete.

The partitions into two non-empty clusters are $\{1\}|\{2,3,4\}$, $\{2\}|\{1,3,4\}$, $\{3\}|\{1,2,4\}$, $\{4\}|\{1,2,3\}$, $\{1,2\}|\{3,4\}$, $\{1,3\}|\{2,4\}$, $\{1,4\}|\{2,3\}$: seven. The formula gives $S(4,2) = \frac{1}{2}\big[(-1)^1\binom{2}{1}1^4 + (-1)^0\binom{2}{2}2^4\big] = \frac{1}{2}(-2+16) = 7$. For $S(10,3) = \frac{1}{6}\big[\binom{3}{1}1^{10} - \binom{3}{2}2^{10} + \binom{3}{3}3^{10}\big] = \frac{1}{6}(3 - 3072 + 59049) = \frac{55980}{6} = 9330$.

Exercise 4.3.

Alternate two operations: assign using the current centres, then recompute means using the new assignments. Evaluate the objective after the mean update to compare completed passes.

Pass 1 (means 0 and 1): $0 \to$ cluster 1; $1, 3, \underset{\sim}{8}, 9, 11 \to$ cluster 2 (for $x = 3$: 9 versus 4). Partition $\{0\}, \{1,3,8,9,11\}$ with means 0 and 6.4; its $\widetilde{W} = 0 + (29.16 + 11.56 + 2.56 + 6.76 + 21.16) = 71.2$.

Pass 2 (means 0, 6.4): $x = 1$: 1 vs 29.16 $\to$ 1; $x = 3$: 9 vs 11.56 $\to$ 1; $x = 8$: 64 vs 2.56 $\to$ 2. Partition $\{0,1,3\}, \{8,9,11\}$ with means 4/3 and 28/3.

Pass 3: $x = 3$: $(3 - 4/3)^2 = 2.78$ vs $(3 - 28/3)^2 = 40.1 \to 1$; $x = 8$: 44.4 vs 1.78 $\to$ 2. No change, so stop. Final $\widetilde{W} = (\frac{16}{9} + \frac{1}{9} + \frac{25}{9}) \times 2 = \frac{84}{9} = 9.33$. The criterion fell from 71.2 to 9.33.

Exercise 4.4.

Equal squared distances cancel the quadratic terms in the observation. What remains is linear, so each pair of centres supplies a perpendicular bisector.

For $(0,0)$ and $(4,2)$: $2(4,2)^\top\boldsymbol{x} = \|(4,2)\|^2 - 0$, i.e. $8x_1 + 4x_2 = 20$ or $2x_1 + x_2 = 5$. For $(0,0)$ and $(0,4)$: $8x_2 = 16$, i.e. $x_2 = 2$. For $(4,2)$ and $(0,4)$: $2(-4,2)^\top\boldsymbol{x} = 16 - 20$, i.e. $-8x_1 + 4x_2 = -4$ or $2x_1 - x_2 = 1$. The first two meet where $x_2 = 2$ and $2x_1 = 3$, so $(1.5, 2)$; the third gives $2(1.5) - 2 = 1$, which holds. The circumradius is the distance to any centre: $\sqrt{1.5^2 + 2^2} = 2.5$, and the distances to $(4,2)$ and $(0,4)$ are $\sqrt{2.5^2 + 0} = 2.5$ and $\sqrt{1.5^2 + 2^2} = 2.5$.

Exercise 4.5.

Apply the stated stopping rule in increasing order of the number of clusters. The selected value need not maximise the numerical gap.

Check $\text{Gap}(k) \geq \text{Gap}(k+1) - s'(k+1)$ in order. $k = 1$: $0.30 \geq 0.70 - 0.05 = 0.65$? No. $k = 2$: $0.70 \geq 1.10 - 0.06 = 1.04$? No. $k = 3$: $1.10 \geq 1.15 - 0.06 = 1.09$? Yes. So $k^* = 3$, even though the gap is numerically largest at $k = 4$: the increase from 3 to 4 is only $0.05 < s'(4) = 0.06$.

Exercise 4.6.

Ask what the unpenalised objective would prefer if every observation were allowed its own cluster. That limiting case exposes why raw training fit cannot select complexity.

The globally minimised within-cluster sum of squares never increases as clusters are added, because any partition into K clusters can be refined into $K + 1$ clusters without raising the criterion, and it equals 0 when every point is its own cluster. Finite runs from poor starts need not follow this ordering, but the global objective favours the largest K tried; the comparison tells us nothing about structure. One should instead use a criterion that accounts for how much improvement is expected by chance or that penalises complexity, such as the gap statistic (compare $\log W$ with its value under a structureless reference), a look at where the decrease of W levels off, or, for mixture models, BIC or cross-validated likelihood (Chapter 6).

Exercise 4.7.

A bisector is only a candidate boundary. Keep the part where neither of the other centres is closer, then connect this geometry to the least-squares objective.

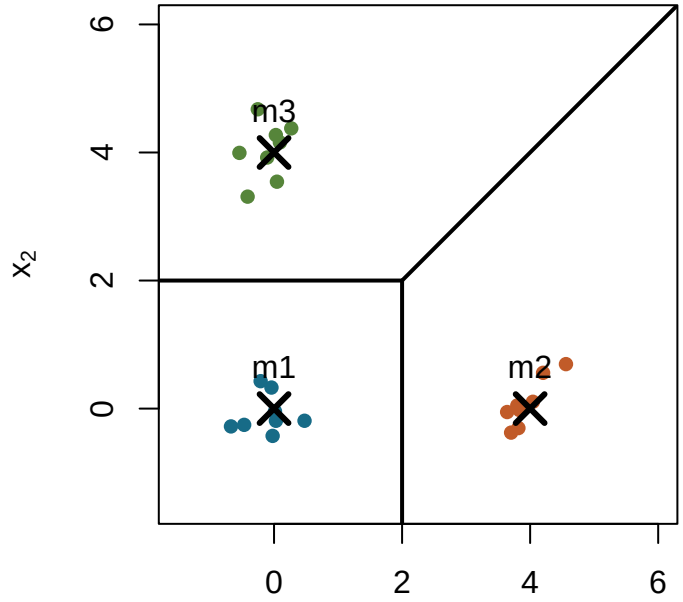
(a) Points are assigned to their nearest centroid. Equality of squared distances to m_j, m_k gives

$$2(m_k - m_j)^T x = \|m_k\|^2 - \|m_j\|^2,$$

a straight perpendicular bisector. The active portions form convex Voronoi cells; a full bisector need not be an actual boundary where a third centroid is closer. A valid sketch has three labelled centres, data near them, and the appropriate clipped bisectors. For example use centres $(0,0), (4,0), (0,4)$: active boundaries are $x_1 = 2$ for $x_2 \leq 2$, $x_2 = 2$ for $x_1 \leq 2$, and $x_1 = x_2 \geq 2$.

(b) Minimise $\sum_{k=1}^{3} \sum_{i \in C_k} \|x_i - m_k\|^2$. For fixed nonempty assignments, each updated centroid is the arithmetic mean of its assigned observations.

(c) The joint optimisation is nonconvex, so alternating assignment and updating can stop at different local solutions. Use multiple sensible starts, compare objective values, and inspect stability; multiple starts do not prove global optimality.



A valid three-centroid diagram: crosses are centres; only active nearest-centre boundaries are shown.

Exercise 4.8.

Compare both the assignment rule and the object being fitted. Hard clusters and mixture components differ even when a picture makes their groups look similar.

(a) Nearest-centroid assignment gives convex Voronoi cells. Pairwise boundaries are active segments or rays of perpendicular bisectors, with equation $2(m_k - m_j)^T x = \|m_k\|^2 - \|m_j\|^2$. The sketch must include three centres, data and valid nearest-centre regions. The three-centroid exercise supplies one exact construction; collinear centres, producing parallel boundaries, are also valid.

(b) Two differences: hard assignments versus posterior membership probabilities, and one Euclidean distance versus component densities that may have different covariances and weights. Consequences include representing uncertainty, ellipsoidal or differently oriented components, and curved MAP boundaries. Both are fitted summaries; neither automatically discovers biological classes.

Solutions for Chapter 5

Exercise 5.1.

Condition on the component label. There is variation inside each component and variation between its mean and the overall mean; both contribute.

$\mathbb{E}X = \int x \sum_h \pi_h f_h = \sum_h \pi_h \mu_h$. For the variance use $\mathbb{E}X^2 = \sum_h \pi_h(\sigma_h^2 + \mu_h^2)$, so $\operatorname{Var} X = \sum_h \pi_h \sigma_h^2 + \sum_h \pi_h \mu_h^2 - \bar{\mu}^2 = \sum_h \pi_h \sigma_h^2 + \sum_h \pi_h(\mu_h - \bar{\mu})^2$ (the last step expands $\sum_h \pi_h(\mu_h - \bar{\mu})^2 = \sum_h \pi_h \mu_h^2 - \bar{\mu}^2$). This is the law of total variance with the component label as the conditioning variable: within-component plus between-component variance.

Numerically $\bar{\mu} = 0.3 \cdot 0 + 0.7 \cdot 4 = 2.8$ and

$$\operatorname{Var} X = (0.3 \cdot 1 + 0.7 \cdot 4) + (0.3 \cdot 2.8^2 + 0.7 \cdot 1.2^2) = 3.1 + (2.352 + 1.008) = 6.46.$$

Exercise 5.2.

A responsibility is a normalised weighted density. Be careful to pass the standard deviation, rather than the variance, when evaluating a normal density.

Component 1 is $\mathrm{N}(0, 1)$ and component 2 is $\mathrm{N}(4, 4)$ (standard deviation 2). At $x = 1$: $f_1 = \phi(1) = 0.2420$, $f_2 = \phi(-1.5)/2 = 0.1295/2 = 0.0648$. The weighted values are $0.3 \times 0.2420 = 0.0726$ and $0.7 \times 0.0648 = 0.0453$, so $\tau_1(1) = 0.0726/0.1179 = 0.6156$. At $x = 3$: $f_1 = \phi(3) = 0.00443$ and $f_2 = \phi(-0.5)/2 = 0.3521/2 = 0.1760$; weighted 0.00133 and 0.1232, so $\tau_1(3) = 0.00133/0.1246 = 0.0107$.

Exercise 5.3.

Hold the E-step responsibilities fixed throughout this M-step. They act like fractional counts when updating weights, means and variances.

The memberships are $\tau_{\cdot 1} = (0.6156, 0.0107)$ and $\tau_{\cdot 2} = (0.3844, 0.9893)$. Then $N_1 = 0.6263$, $N_2 = 1.3737$ and $\pi = (N_1, N_2)/2 = (0.3131, 0.6869)$. Means: $\mu_1 = (0.6156 \cdot 1 + 0.0107 \cdot 3)/0.6263 = 1.034$ and $\mu_2 = (0.3844 \cdot 1 + 0.9893 \cdot 3)/1.3737 = 2.440$. Variances: $\sigma_1^2 = [0.6156(1 - 1.034)^2 + 0.0107(3 - 1.034)^2]/0.6263 = 0.067$ and $\sigma_2^2 = [0.3844(1 - 2.440)^2 + 0.9893(3 - 2.440)^2]/1.3737 = 0.806$. (With only two points the first component is essentially pinned on $x = 1$, its variance shrinking toward zero: a miniature of the singularity problem.)

Exercise 5.4.

Make one component concentrate on a single observation while another retains positive density at every remaining observation. A lower bound is enough to show divergence.

The log-likelihood contains the term for observation 1,

$$\ln\left(\tfrac{1}{2}\frac{1}{\sqrt{2\pi\sigma_1^2}} + \tfrac{1}{2} f_2(x_1)\right) \;\geq\; -\ln 2 - \tfrac{1}{2}\ln(2\pi\sigma_1^2),$$

using $\phi(x_1; x_1, \sigma_1^2) = (2\pi\sigma_1^2)^{-1/2}$. For $i \geq 2$, $\ln\left(\frac{1}{2}f_1(x_i) + \frac{1}{2}f_2(x_i)\right) \geq \ln\left(\frac{1}{2}f_2(x_i)\right)$, a finite constant not depending on σ_1. So $\ell \geq C - \frac{1}{2}\ln(2\pi\sigma_1^2) \to \infty$ as $\sigma_1^2 \to 0$.

Exercise 5.5.

Count means, covariance entries and independent mixing weights separately. Symmetry reduces a full covariance count, and the weights lose one degree of freedom because they sum to one.

By (5.2): $p = 2$, $H = 3$: $6 + 3 \cdot 3 + 2 = 17$. $p = 5$, $H = 3$: $15 + 3 \cdot 15 + 2 = 62$. Diagonal covariances replace $p(p+1)/2$ by p per component: 14 for $p = 2$ $(6 + 6 + 2)$ and 32 for $p = 5$ $(15 + 15 + 2)$. A common full matrix has $p(p+1)/2$ covariance parameters in total: $6 + 3 + 2 = 11$ for $p = 2$ and $15 + 15 + 2 = 32$ for $p = 5$.

Exercise 5.6.

Write the density before interpreting the component names. A permutation changes the parameter labels but leaves the sum, and therefore the likelihood, unchanged.

The density is $\pi\,\phi(x; \mu_1, \sigma_1^2) + (1 - \pi)\,\phi(x; \mu_2, \sigma_2^2)$; swapping the labels and the weights $\pi \leftrightarrow 1 - \pi$ merely reorders the two summands. So the likelihood is the same for both parameter vectors and the data cannot distinguish them. If two bootstrap fits have swapped labels, an average of their parameters, e.g. of μ_1, mixes the means of different components and gives a value in between, and the sample covariance (5.19) is inflated by the artificial spread. Components must be matched across fits first.

Exercise 5.7.

Membership uncertainty and covariance geometry are substantive differences. Explain what each permits the fitted model to do, rather than merely naming the algorithms.

(a) One distinction is hard nearest-centre assignment versus probabilistic membership from a density model, allowing uncertainty for overlapping components. Another is Euclidean nearest-centre geometry versus component-specific covariances and mixing weights, allowing different orientations, spreads and prevalences; MAP mixture boundaries may be curved. Award two well-explained differences, not four disconnected labels.

(b)

$$\tau_k(x) = \frac{\pi_k \phi_p(x; \mu_k, \Sigma_k)}{\sum_h \pi_h \phi_p(x; \mu_h, \Sigma_h)}, \qquad \pi_k \geq 0, \quad \sum_k \pi_k = 1.$$

Here π_k is a component probability, μ_k a mean and Σ_k a covariance; the denominator is the mixture density. The probabilities sum to one.

(c) No. Components are terms in an approximation to the observed density. Several components can represent one nonnormal population, and overlap or misspecification can obscure distinct populations. Biological interpretation requires additional evidence.

Solutions for Chapter 6

Exercise 6.1.

Bayes factors multiply odds, not probabilities. Convert the prior to odds first and convert back only after applying the evidence factor.

Prior odds are $0.25/0.75 = 1/3$, so posterior odds $= B_{12} \times$ prior odds $= 6 \times \frac{1}{3} = 2$. The posterior probability is $2/(1+2) = 2/3 \approx 0.667$.

Exercise 6.2.

Put every candidate on the same criterion convention: here smaller is better. Then compare the improvement in fit with the extra parameter charge.

$\ln 300 = 5.7038$. For $H = 2$: BIC $= 2200 + 11(5.7038) = 2262.7$ and AIC $= 2200 + 22 = 2222$. For $H = 3$: BIC $= 2100 + 17(5.7038) = 2197.0$ and AIC $= 2100 + 34 = 2134$. Both criteria select $H = 3$: the gain of 100 in -2ℓ more than pays for the six extra parameters under either penalty ($6 \times 5.70 = 34.2$ for BIC, 12 for AIC).

Exercise 6.3.

Recognise the local Gaussian shape and restore its normalising constant. The curvature must be positive for this integral to be finite.

The integrand is $\sqrt{2\pi/h}$ times the $\mathrm{N}(\tilde{\theta}, 1/h)$ density, which integrates to 1; hence the integral is $\sqrt{2\pi/h}$. In (6.11) with $d_k = 1$ and $\boldsymbol{H} = h$, $(2\pi)^{1/2} h^{-1/2}$ is the same quantity. If $h = nI_1$, then $-\frac{1}{2}\log h = -\frac{1}{2}\log n - \frac{1}{2}\log I_1$ and the second term is a constant, i.e. $O(1)$.

Exercise 6.4.

The logarithm inequality gives the sign of KL divergence. Splitting the log ratio then shows exactly which part changes when a candidate predictive density changes.

$D(f\|\widehat{f}) = -\int f \ln(\widehat{f}/f) \geq -\int f(\widehat{f}/f - 1) = -\int \widehat{f} + \int f = 0$ using $\ln u \leq u - 1$ (with $\int \widehat{f} \leq 1$ and $\int f = 1$); equality in $\ln u \leq u - 1$ holds only at $u = 1$, i.e. $\widehat{f} = f$ almost everywhere. Splitting the logarithm, $\ln(f/\widehat{f}) = \ln f - \ln \widehat{f}$, gives $D = \int f \ln f - \mathbb{E}_f \ln \widehat{f}(\boldsymbol{X})$. The first term does not depend on the fitted model, so maximising $\mathbb{E}_f \ln \widehat{f}(\boldsymbol{X})$ over models is the same as minimising D: it selects the fitted density closest to the truth in KL divergence.

Exercise 6.5.

Nested model classes can always reuse a simpler fit. That observation explains monotone optimal training fit, but it does not establish a predictive improvement.

If the candidate parameter spaces are nested, the supremum of training log-likelihood cannot decrease as H increases. It need not strictly increase. For unrestricted Gaussian mixtures with at least two components, the supremum is already infinite by the collapse construction in Chapter 5; a numerical finite local fit does not remove that pathology. Meaningful comparisons require a stated fitting or constraint convention. Complexity criteria or held-out log-likelihood then address

different model-selection targets. Ordinary BIC's regular Laplace justification needs qualification for singular mixtures (Drton and Plummer, 2017).

Exercise 6.6.

Calculate fit and penalty together for each candidate. The sign convention matters only if the direction of optimisation is changed inconsistently.

(a) The values are 383.026, 381.447, 396.472 for A, B, C. Select B, with the smallest BIC. One mark for correct calculations across the candidates and two for the penalty/sign and selection, or equivalent working.

(b) Extra flexibility can fit random sample variation. In nested models the global maximised likelihood cannot decrease as the model class expands, so it contains no explicit cost for extra parameters. BIC trades fit against a $d \log n$ penalty. A finite optimisation run or singular mixture fit can complicate this ideal comparison.

(c) The alternative is the negative of this BIC; negation reverses the ordering. The preferred statistical model is unchanged when the optimisation direction changes consistently.

Exercise 6.7.

The first part asks for posterior component probabilities; the second asks for a model comparison. A hard label discards uncertainty, while BIC introduces a complexity charge.

(a) Compute $\pi_k \phi(x; \mu_k, \Sigma_k)$ for each component and divide by their sum. Define the weight, density parameters and normalisation [2]. A hard label retains only the maximum, whereas the probability vector distinguishes a near tie from a confident assignment [2].

(b) BIC values are $280 + 11 \log 80 = 328.202$ and $266 + 17 \log 80 = 340.494$; select the first model.

(c) No. Training fit benefits from flexibility and can fit noise. Out-of-sample evidence or an explicit complexity criterion is needed; a numerical likelihood value alone also does not prove successful global optimisation.

Solutions for Chapter 7

Exercise 7.1.

After every merge, recompute the distance between clusters using the chosen linkage. The same observations can produce different merge heights even when the merge order agrees.

The pairwise gaps give: *single* linkage merges $\{1,2\}$ at 1, $\{6,8\}$ at 2, then $\{1,2\} \cup \{6,8\}$ at 4 (closest pair $2,6$), then 15 at 7 (closest pair $8,15$). *Complete* linkage: 1, 2, then $\{1,2\} \cup \{6,8\}$ at $\max = 8 - 1 = 7$, then 15 at $15 - 1 = 14$. *Average* linkage: 1, 2, then $\{1,2\}$ with $\{6,8\}$ at the mean of $(5,7,4,6)$, i.e. 5.5, then 15 at the mean of $(14,13,9,7)$, i.e. 10.75. The merge order and groupings are the same for all three: $\{1,2\}, \{6,8\}$, then their union, then 15. Only the heights differ, with single $\leq$ average $\leq$ complete at every merge.

Exercise 7.2.

One binary merge reduces the current cluster count by exactly one. Counting that change is simpler than trying to read the answer from a drawing.

Each merge reduces the number of clusters by one, so going from n clusters to 1 takes $n - 1$ merges. The hierarchy contains the clusterings with $n, n-1, \ldots, 1$ clusters: n levels (the notes' count, including the single cluster at the top). The dendrogram has n leaves and $n - 1$ internal nodes.

Exercise 7.3.

Evaluate the three kernels at the requested location, then average. Each observation contributes a translated density rather than a probability mass at the point.

With $h = 1$, $\widehat{f}(0) = \frac{1}{3}\big[\phi(0+1) + \phi(0-0) + \phi(0-2)\big] = \frac{1}{3}\big[\phi(1) + \phi(0) + \phi(2)\big] = \frac{1}{3}(0.24197 + 0.39894 + 0.05399) = \frac{0.69490}{3} = 0.2316$, where ϕ is the standard normal density.

Exercise 7.4.

The bandwidth stretches the kernel's support, so its normalising factor must cancel the Jacobian. This is why the determinant appears.

$\int K_{\boldsymbol{H}}(\boldsymbol{x} - \boldsymbol{x}_i)\,\mathrm{d}\boldsymbol{x} = \int |\boldsymbol{H}|^{-1/2} K(\boldsymbol{H}^{-1/2}(\boldsymbol{x} - \boldsymbol{x}_i))\mathrm{d}\boldsymbol{x}$. Put $\boldsymbol{u} = \boldsymbol{H}^{-1/2}(\boldsymbol{x} - \boldsymbol{x}_i)$, so $\mathrm{d}\boldsymbol{x} = |\boldsymbol{H}|^{1/2}\,\mathrm{d}\boldsymbol{u}$ (the Jacobian of $\boldsymbol{x} = \boldsymbol{H}^{1/2}\boldsymbol{u} + \boldsymbol{x}_i$ is $|\boldsymbol{H}^{1/2}| = |\boldsymbol{H}|^{1/2}$). The integral becomes $\int K(\boldsymbol{u})\,\mathrm{d}\boldsymbol{u} = 1$. Each of the n terms integrates to 1, and the factor $1/n$ gives total mass 1.

Exercise 7.5.

At a training observation, the density includes a kernel centred exactly on that observation. Examine that self-contribution as the bandwidth tends to zero.

Each $\widehat{f}(x_i; h)$ includes the kernel centred on x_i itself, which contributes $\frac{1}{n}K(0)/h$ and diverges as $h \to 0$. So the criterion increases without bound as $h \to 0$, selecting a spiky estimate that reproduces the sample, the same singularity as a mixture component collapsing on a data point. The criterion should evaluate each point under a density that does not use it: the leave-one-out

likelihood $\sum_i \log \widehat{f}_{-i}(x_i; h)$ with $\widehat{f}_{-i}(x_i; h) = \frac{1}{n-1} \sum_{l \neq i} K_h(x_i - x_l)$, or the likelihood on a held-out validation set.

Exercise 7.6.

Linkage chooses how clusters are compared; bandwidth chooses how much a density is smoothed. Neither tuning choice makes visual groups into known populations.

(a) Single linkage uses the smallest intercluster distance, so a chain of short links can join distant dense regions. Complete linkage uses the largest intercluster distance and tends to favour smaller-diameter clusters. Neither method is universally robust or guaranteed to recover the generating groups.

(b) Bandwidth sets the smoothing scale in $\hat{f}_h(x) = n^{-1} \sum_i h^{-p} K((x - x_i)/h)$. A very small h produces noisy peaks around observations; a very large h can merge features and hide modes. This is a bias-variance trade-off.

(c) No. Peaks depend on the smoothing choice and sample variability; a multimodal population or overlapping populations break a one-peak/one-population interpretation.

Solutions for Chapter 8

Exercise 8.1.

At the boundary the two posterior numerators tie. Solve that equality, check which class wins on each side, and then integrate the two error regions.

The boundary solves $\pi_1 f_1(x) = \pi_2 f_2(x)$: $0.7\,\mathrm{e}^{-x^2/2} = 0.3\,\mathrm{e}^{-(x-2)^2/2}$. Taking logs, $\ln(0.7/0.3) = \frac{1}{2}[x^2 - (x-2)^2] = \frac{1}{2}(4x-4) = 2x-2$, so $x^* = 1 + \frac{1}{2}\ln\frac{7}{3} = 1.4236$. Class 1 is predicted for $x < x^*$. The error rate is

$$\begin{aligned} 0.7\,\mathbb{P}(X > x^* \mid 1) + 0.3\,\mathbb{P}(X < x^* \mid 2) &= 0.7\,[1 - \Phi(1.4236)] + 0.3\,\Phi(1.4236 - 2) \\ &= 0.7(0.0773) + 0.3(0.2822) = 0.0541 + 0.0847 = 0.139. \end{aligned}$$

The unequal priors move the boundary away from the midpoint 1 toward the less probable class.

Exercise 8.2.

Compare posterior expected losses for the available decisions. The more likely class need not be the cheaper decision when errors have unequal consequences.

Predicting class 1 (with $\tau_1 = 0.3$, $\tau_2 = 0.7$) has expected cost $\tau_2 c_{21} = 0.7 \times 1 = 0.7$; predicting class 2 has expected cost $\tau_1 c_{12} = 0.3 \times 5 = 1.5$. So class 1 is chosen, even though it is the less probable class. The decision switches when $\tau_2 c_{21} = \tau_1 c_{12}$, i.e. $(1 - \tau_1) = 5\tau_1$, $\tau_1 = 1/6 \approx 0.167$: class 1 is predicted whenever $\tau_1 > 1/6$.

Exercise 8.3.

A change in priors shifts the intercept while leaving the shared-covariance slope unchanged. Re-evaluate the point against the shifted boundary.

Now $\alpha = \log 2 + 9.75 = 0.693 + 9.75 = 10.443$ and $\boldsymbol{\beta}$ is unchanged, so the boundary is $10.443 - 1.5(x_1 + x_2) = 0$, i.e. $x_1 + x_2 = 6.962$, which has moved away from class 1 (the more probable class) by 0.462. For $(3.4, 3.2)$: $x_1 + x_2 = 6.6 < 6.962$, so class 1. Under equal priors, $6.6 > 6.5$ gave class 2: the prior changes the decision at this point.

Exercise 8.4.

Equal means do not imply equal distributions. Here the variance difference makes the central region favour one class and both tails favour the other.

Equal priors give $\frac{1}{\sqrt{1}}\mathrm{e}^{-x^2/2} = \frac{1}{\sqrt{4}}\mathrm{e}^{-x^2/8}$, so $-\frac{x^2}{2} + \frac{x^2}{8} = -\ln 2$, i.e. $\frac{3x^2}{8} = \ln 2$ and $x^2 = \frac{8}{3}\ln 2 = 1.848$, $|x| = 1.360$. QDA predicts class 1 for $|x| < 1.36$ and class 2 otherwise: two boundary points. LDA assumes a common variance and the class means are equal ($\mu_1 = \mu_2 = 0$), so $\boldsymbol{\beta} = \boldsymbol{S}^{-1}(\mu_1 - \mu_2) = 0$: the linear discriminant function is constant and cannot separate the classes. All the information is in the variance.

Exercise 8.5.

Distinguish a covariance shared across classes from one estimated separately in each class. Parameter counts and rank requirements both change.

LDA: $Gp + p(p+1)/2 + (G-1) = 30 + 55 + 2 = 87$. QDA: $30 + 3 \times 55 + 2 = 197$. A class sample covariance matrix has rank at most $n_g - 1$ (Section 2.1; Chapter 18), so nonsingularity needs $n_g - 1 \geq p$, i.e. $n_g \geq 11$ per class, at least 33 observations in total (and more in practice, since a nearly singular matrix is already unreliable).

Exercise 8.6.

The optimal direction uses covariance to reweight the mean difference. Compare the same scale-invariant separation ratio for each proposed direction.

$\boldsymbol{d} = \widehat{\boldsymbol{\mu}}_1 - \widehat{\boldsymbol{\mu}}_2 = (-3, -2)^\top$ and $\boldsymbol{S}^{-1}\boldsymbol{d} = (-1.5, -1.5)^\top \propto (1, 1)^\top$. For $\boldsymbol{a} = (1, 1)^\top$: $\boldsymbol{a}^\top \boldsymbol{d} = -5$, $\boldsymbol{a}^\top \boldsymbol{S}\boldsymbol{a} = \frac{4}{3} + 2 \cdot \frac{2}{3} + \frac{2}{3} = \frac{10}{3}$, ratio $= 25/(2 \cdot 10/3) = 3.75$. For $\boldsymbol{a} = \boldsymbol{d}$: $\boldsymbol{a}^\top \boldsymbol{d} = 13$, $\boldsymbol{a}^\top \boldsymbol{S}\boldsymbol{a} = 9(\frac{4}{3}) + 2 \cdot 6(\frac{2}{3}) + 4(\frac{2}{3}) = 22.67$, ratio $= 169/45.33 = 3.728$. Fisher's direction attains the maximum $\boldsymbol{d}^\top \boldsymbol{S}^{-1}\boldsymbol{d}/2 = 7.5/2 = 3.75$; the naive direction is slightly lower because the mean difference is already nearly along the principal axis of $\boldsymbol{S}$.

Exercise 8.7.

Begin with a generative model: class probabilities and predictor distributions within classes. The linear decision rule follows after the shared quadratic term cancels.

(a) Write $P(Y = k) = \pi_k$ and $X \mid Y = k \sim N_2(\mu_k, \Sigma)$ for $k = 1, 2$, with shared positive-definite covariance. Estimate class means by within-class averages, covariance by pooled within-class variation, and priors by training proportions when those represent deployment prevalence. For the usual unbiased pool, $S_p = \sum_k \sum_{i:y_i=k} (x_i - \bar{x}_k)(x_i - \bar{x}_k)^T/(n-2)$; a likelihood convention uses divisor n.

(b) The common $x^T\Sigma^{-1}x$ term cancels from the log-density difference. The remaining discriminant is affine in x:

$$\delta_k(x) = x^T\Sigma^{-1}\mu_k - \tfrac{1}{2}\mu_k^T\Sigma^{-1}\mu_k + \log \pi_k.$$

(c) Priors alter the intercept of the log-posterior odds. Unequal misclassification losses change the posterior-odds threshold; choose the class with the smaller expected loss. Sampling proportions should not automatically replace known target-population priors.

Exercise 8.8.

LDA uses class labels to model conditional distributions. PCA uses predictor variation alone. Writing their assumptions first keeps those two constructions distinct.

(a) State $P(Y = k) = \pi_k$ and $X \mid Y = k \sim N_p(\mu_k, \Sigma)$ with a shared covariance [2]. Estimate means from class samples, common covariance from pooled within-class scatter, and priors from suitable class proportions or external prevalence [2]. Apply Bayes' rule and choose the largest posterior under equal losses; equivalent discriminants are $\delta_k(x) = x^T\hat{\Sigma}^{-1}\hat{\mu}_k - \frac{1}{2}\hat{\mu}_k^T\hat{\Sigma}^{-1}\hat{\mu}_k + \log \hat{\pi}_k$. Explain that common quadratic terms cancel, giving a linear boundary [2].

(b) Centre (and standardise when justified); compute the covariance/correlation eigendecomposition [1]. Directions are orthonormal eigenvectors ordered by decreasing eigenvalue [1], arising by maximising projected variance subject to norm and orthogonality constraints [1]. Scores are projections of centred observations onto these directions [1]. Eigenvalues are score variances, so their ratios to the trace quantify variance explained; PCA uses no response labels [1].

Solutions for Chapter 9

Exercise 9.1.

Read the row and column definitions before calculating rates. Overall error uses the full sample size; a class-specific error uses its own class denominator.

The off-diagonal total is $12+16=28$, so $\widehat{\mathrm{Err}} = 28/200 = 0.14$. By rows, $\widehat{\mathrm{Err}}_1 = 12/120 = 0.10$ and $\widehat{\mathrm{Err}}_2 = 16/80 = 0.20$. The test-set class proportions are 0.6 and 0.4, and $0.6(0.10) + 0.4(0.20) = 0.06 + 0.08 = 0.14$, as required.

Exercise 9.2.

Solve the approximate interval-width equation for the sample size, then round upward. This calculation assumes independent test outcomes and an error probability near the planning value.

Half-width $= 1.96\sqrt{p(1-p)/n} = 0.02$ gives $n = 1.96^2(0.10)(0.90)/0.02^2 = 3.8416 \times 0.09/0.0004 = 864.4$, so about 865 observations. The requirement grows like the inverse square of the half-width.

Exercise 9.3.

A point is absent only if every bootstrap draw misses it. Take the complement to obtain the inclusion probability before applying the stated weighted estimator.

For $n=5$: $1-0.8^5 = 0.672$; for $n=20$: $1-0.95^{20} = 0.642$. Since $(1-1/n)^n \to \mathrm{e}^{-1}$, the probability tends to $1-\mathrm{e}^{-1} = 0.632$. The .632 estimate is $0.368(0.05) + 0.632(0.20) = 0.0184 + 0.1264 = 0.145$.

Exercise 9.4.

Write every leave-one-out mean in terms of the full-sample mean. Their average and spread then simplify without fitting anything numerically.

Leaving out x_i gives $\widehat{\theta}_{(i)} = (n\bar{x} - x_i)/(n-1) = \bar{x} + (\bar{x} - x_i)/(n-1)$. Averaging over i, $\sum_i (\bar{x} - x_i) = 0$, so $\widehat{\theta}_{(\cdot)} = \bar{x} = \widehat{\theta}$ and $\widehat{\mathrm{bias}}_{\mathrm{jack}} = 0$. Next, $\sum_i (\widehat{\theta}_{(i)} - \widehat{\theta}_{(\cdot)})^2 = \sum_i (\bar{x} - x_i)^2/(n-1)^2 = (n-1)s^2/(n-1)^2 = s^2/(n-1)$, hence $\widehat{\mathrm{se}}^2_{\mathrm{jack}} = \frac{n-1}{n} \cdot \frac{s^2}{n-1} = s^2/n$. The jackknife reproduces the usual standard error of the mean.

Exercise 9.5.

Count model fits at each level of the procedure, including the refit after tuning. An outer assessment loop does not itself supply the final all-data fit.

LOO CV trains $n = 1000$ classifiers. Ten-fold CV trains 10. For nested CV with $m = 10$: each outer iteration runs $9 \times 5 = 45$ inner fits to compare the candidates and one refit of the chosen model on the training and validation data, 46 in all, so $10 \times 46 = 460$. The final classifier needs its own selection (say 10-fold CV over 5 candidates, 50 fits) plus one fit on all the data: 51. Total $460 + 51 = 511$. Here the full procedure requires about fifty times as many fits as one ten-fold calculation. That cost is specific to these loop sizes; the assessment benefit comes from keeping selection inside each outer training set.

Exercise 9.6.

The same noise that helps a candidate win is present in its reported winning score. An outer assessment must therefore evaluate the whole selection procedure.

Selection favours candidates with favourable estimation noise. Consequently, the minimum inner-CV score is not an independent estimate of the chosen procedure's prediction error; the reported 8.0% can be optimistic. Its bias relative to a particular full-sample target also involves training-size effects, so there is no guaranteed numerical ordering on every split. Outer CV repeats selection using only each outer training set, then evaluates the selected fit on the omitted fold. It estimates performance at that outer training size (Cawley and Talbot, 2010, Section 5.1).

Exercise 9.7.

Follow the flow of information. Every operation influenced by outcomes, including preprocessing choices, belongs inside the training and selection stage.

(a) Validation results vary because of sampling noise. Selecting the largest value favours candidates with favourable noise as well as genuinely good performance; reporting that selected value creates optimistic selection bias. It need not exceed test accuracy in every realised split.

(b) Fit preprocessing and candidate models on training data; apply the fitted transforms to validation data and choose settings using a stated criterion. Freeze those choices. Optionally refit the chosen procedure, including preprocessing, on training plus validation data. Evaluate that fixed fitted procedure once on the independent test set and state the metric and sample size. Further choices based on the test results would use up its independence.

(c) Each fold has its own fitted preprocessing and model. CV can tune, but the best CV score is still selected. Use an external test set or an outer CV loop for evaluation; an outer loop must repeat all inner choices.

Solutions for Chapter 10

Exercise 10.1.

Differentiate the inverse-logit once, then reuse that derivative in the score. The Hessian reveals concavity, while existence of a finite optimum requires separate conditions.

$\sigma'(\eta) = \dfrac{\mathrm{e}^{-\eta}}{(1+\mathrm{e}^{-\eta})^2} = \dfrac{1}{1+\mathrm{e}^{-\eta}} \cdot \dfrac{\mathrm{e}^{-\eta}}{1+\mathrm{e}^{-\eta}} = \sigma(\eta)(1-\sigma(\eta))$, since $1-\sigma = \mathrm{e}^{-\eta}/(1+\mathrm{e}^{-\eta})$. Also $\sigma(-\eta) = \dfrac{1}{1+\mathrm{e}^{\eta}} = \dfrac{\mathrm{e}^{-\eta}}{\mathrm{e}^{-\eta}+1} = 1-\sigma(\eta)$. From (10.9) the score has ith term $(y_i - p_i)\boldsymbol{x}_i$ with $p_i = \sigma(\eta_i)$ and $\eta_i = \boldsymbol{\beta}^\top \boldsymbol{x}_i$; differentiating again, $\partial p_i/\partial \boldsymbol{\beta} = p_i(1-p_i)\boldsymbol{x}_i$, so the Hessian is $-\sum_i p_i(1-p_i)\boldsymbol{x}_i\boldsymbol{x}_i^\top$.

Exercise 10.2.

Exponentiating a coefficient change gives an odds multiplier. Calculate probabilities separately if the question asks about risk; the two ratios are not interchangeable.

For ten years, the log odds rise by $10 \times 0.04618 = 0.4618$ and the odds ratio is $\mathrm{e}^{0.4618} = 1.587$. Scaling the profile interval endpoints by 10, $\exp(0.2484) = 1.282$ and $\exp(0.6851) = 1.984$: the odds ratio for ten years is 1.59 with 95% interval (1.28, 1.98). For a 60-year-old with `hbp` $= 0$: $\eta = -3.70568 + 0.04618(60) = -0.9349$, so $\widehat{p} = 1/(1+\mathrm{e}^{0.9349}) = 0.282$. For age 70: $\eta = -0.4731$ and $\widehat{p} = 1/(1+\mathrm{e}^{0.4731}) = 0.384$. Under this reported fitted association, the predicted probability of recorded death during follow-up changes from about 28% to 38% (a ratio of 1.36, smaller than the odds ratio of 1.59 because the probabilities are not small).

Exercise 10.3.

Take posterior log odds and expand the two Gaussian quadratic forms. Their common covariance is precisely what cancels the quadratic term in the predictors.

$\log \frac{\tau_1}{\tau_0} = \log \frac{\pi_1}{\pi_0} + \log \frac{f_1(\boldsymbol{x})}{f_0(\boldsymbol{x})}$, and with the common covariance matrix the quadratic terms cancel:

$$\begin{aligned}\log \frac{f_1}{f_0} &= -\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_1)^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_1) + \tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_0)^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_0) \\ &= \boldsymbol{x}^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0) - \tfrac{1}{2}(\boldsymbol{\mu}_1+\boldsymbol{\mu}_0)^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0).\end{aligned}$$

So $\boldsymbol{\beta} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$ and $\alpha_0 = \log \frac{\pi_1}{\pi_0} - \frac{1}{2}(\boldsymbol{\mu}_1+\boldsymbol{\mu}_0)^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$. This is a logistic model with those parameters.

Exercise 10.4.

Use one common denominator for all classes. In a pairwise log-odds comparison that denominator cancels, leaving a difference of linear predictors.

The denominator is $1 + \mathrm{e}^{0.5} + \mathrm{e}^{-0.2} = 1 + 1.6487 + 0.8187 = 3.4674$. So $\widehat{\tau}_1 = 1/3.4674 = 0.288$, $\widehat{\tau}_2 = 1.6487/3.4674 = 0.476$, $\widehat{\tau}_3 = 0.8187/3.4674 = 0.236$ (sum 1.000). The log odds of class 3 against class 2 are $\log(\widehat{\tau}_3/\widehat{\tau}_2) = (\boldsymbol{\beta}_3 - \boldsymbol{\beta}_2)^\top \boldsymbol{x} = -0.2 - 0.5 = -0.7$.

Exercise 10.5.

Find a coefficient path that increasingly separates every observation. If its likelihood approaches a bound without attaining it at finite coefficients, a finite MLE does not exist.

With $\beta_1 = s$ and $\beta_0 = -2.5s$, $\eta_i = s(x_i - 2.5) = (-1.5s, -0.5s, 0.5s, 1.5s)$. For $y_i = 0$ the term is $-\log(1+\mathrm{e}^{\eta_i}) = -\log(1+\mathrm{e}^{-|\eta_i|})$; for $y_i = 1$ it is $\eta_i - \log(1+\mathrm{e}^{\eta_i}) = -\log(1+\mathrm{e}^{-\eta_i}) = -\log(1+\mathrm{e}^{-|\eta_i|})$. Each term is negative and tends to 0 as $s \to \infty$, so $\ell \uparrow 0$. Since $\ell < 0$ for every finite $\boldsymbol{\beta}$ (each probability is below 1), the supremum 0 is never attained: there is no maximum, and the estimates diverge. Concavity alone does not establish a finite MLE. Complete or quasi-complete separation and rank deficiency need separate checks.

Exercise 10.6.

A Wald statistic and a likelihood-ratio statistic are asymptotically related, but they are not interchangeable observed statistics. Keep the approximation explicit when the reduced fit is unavailable.

The supplied Wald value gives $z^2 = 1.016^2 \approx 1.03$. Under the usual large-sample regularity conditions, this can approximate the likelihood-ratio statistic for removing `hbp`, suggesting a reduced-model deviance near $354.97 + 1.03 = 356.0$. It is not a calculation of that model's actual likelihood ratio: refitting the reduced model is needed for the exact observed difference. The approximate one-degree-of-freedom comparison is not significant at 5%.

For the full model against the intercept-only model, both reported deviances are available. Their difference is $375.35 - 354.97 = 20.38$ on two degrees of freedom, giving the regular asymptotic χ^2_2 comparison and $p \approx 3.8 \times 10^{-5}$. This supports improved fit from the included predictors jointly; it is not a causal conclusion about age or blood pressure.

Exercise 10.7.

Calculate the score, transform it to a probability and only then apply the classification threshold. A coefficient interpretation holds the other predictors fixed.

(a) The linear predictor is $-3 + 3.2 - 0.8 = -0.6$. Thus $p = 1/(1 + e^{0.6}) = 0.35434$, and the prediction is class 0. A complete answer includes the predictor, inverse-logit and decision.

(b) The boundary is $-3 + 0.8x_1 - 0.4x_2 = 0$. Increasing x_1 by one unit multiplies the conditional odds by $e^{0.8} = 2.2255$, holding x_2 fixed; it does not add 0.8 to the probability.

(c) Logistic regression specifies $P(Y \mid X)$ directly and does not require Gaussian class-conditional predictor distributions. LDA specifies Gaussian $X \mid Y$ with a shared covariance and class priors, then derives posterior probabilities. Both give linear log-odds under these forms, but their assumptions and estimation procedures differ.

Solutions for Chapter 11

Exercise 11.1.

Sort by distance before counting class votes. Distance ties and vote ties are distinct and require a stated convention.

Distances from 4: $x = 3$ is 1; $x = 2$ and $x = 6$ are 2; $x = 1$ and $x = 7$ are 3. For $k = 1$: the nearest is $x = 3$ (B), so B. For $k = 3$: the nearest is 3, then 2 and 6 tie at distance 2 for the second and third places; here exactly two points are tied for the last two slots, so both are included (the three neighbours are $3, 2, 6$), votes B, A, B, so B. (If a tie straddles the cutoff, the result depends on the convention: `class::knn` includes all distance ties by default, while `use.all = FALSE` selects enough tied points at random.) For $k = 5$: all points, votes A, A, B, B, A, so A (3 votes to 2). The prediction changes from B to A as k grows.

Exercise 11.2.

Changing a measurement unit changes its contribution to Euclidean distance. Compare both candidate distances before and after the rescaling.

Original units: $d(\mathbf{0}, a) = \sqrt{1 + 100} = 10.05$ and $d(\mathbf{0}, b) = \sqrt{9 + 1} = 3.16$, so b is nearer. After dividing the second coordinate by 10: $a' = (1, 1)$, $b' = (3, 0.1)$, $d(\mathbf{0}, a') = \sqrt{2} = 1.41$ and $d(\mathbf{0}, b') = \sqrt{9 + 0.01} = 3.00$, so now a is nearer. The nearest neighbour, and hence the classification, depends on the units: Euclidean distance is not scale invariant.

Exercise 11.3.

The exercise specifies ranks after including the query point. Apply that definition coordinate by coordinate, then use the maximum rank difference as the distance.

Coordinate 1 values $\{1, 3, 2.5\}$ plus the new 2: sorted $1 < 2 < 2.5 < 3$, so ranks are $1 \to 1$, new $\to 2$, $2.5 \to 3$, $3 \to 4$. Coordinate 2 values $\{10, 200, 40\}$ plus new 50: sorted $10 < 40 < 50 < 200$, so $10 \to 1$, $40 \to 2$, new $\to 3$, $200 \to 4$. Rank pairs: $(1, 10)$: $(1, 1)$; $(3, 200)$: $(4, 4)$; $(2.5, 40)$: $(3, 2)$; new: $(2, 3)$. Distances: $\max(|1 - 2|, |1 - 3|) = 2$; $\max(|4 - 2|, |4 - 3|) = 2$; $\max(|3 - 2|, |2 - 3|) = 1$. The nearest neighbour is $(2.5, 40)$ with $\rho = 1$. Taking logs of coordinate 2 does not change the ordering, hence not the ranks, hence not the distances.

Exercise 11.4.

At a fixed predictor value, the nearest label remains random even when its predictor is very close. Compare its disagreement probability with the Bayes decision's error.

The conditional Bayes error is $r = \min(0.9, 0.1) = 0.1$. The asymptotic nearest-neighbour error is $2\tau(1 - \tau) = 2(0.9)(0.1) = 0.18$, which is 1.8 times the Bayes error (and below the factor-two bound). The bound $1 + \sqrt{2/k}$ is 2.41 for $k = 1$, 1.53 for $k = 7$, and 1.20 for $k = 50$; (for $k = 1$ the sharper Cover–Hart bound $2R^*(1 - R^*)$ is better than the stated formula). This is a large-sample comparison for the stated nearest-neighbour setting. Bayes consistency of a sequence k_n requires conditions such as $k_n \to \infty$ and $k_n/n \to 0$; increasing k to the full sample size is not that regime.

Exercise 11.5.

A point at distance zero from itself makes resubstitution unusually easy for one-nearest-neighbour classification. Remove that self-information when assessing prediction.

If each training point may be its own neighbour, then for $k = 1$ each point with a unique predictor vector is classified correctly (distance 0 to itself), so the training error is 0, and $k = 1$ is always chosen: the apparent error rate is minimised by the most flexible classifier and says nothing about performance on new data. Use leave-one-out or K-fold cross-validation (the ith point must be excluded from its own neighbour list), or a separate validation set, to choose k; for a final error estimate use a test set or nested cross-validation (Chapter 9).

Exercise 11.6.

Scaling changes the metric; validation chooses among fitted rules under that metric. Both operations must be learned without looking at the final test outcomes.

(a) Euclidean distances sum squared differences in the chosen units, so height can dominate solely through its scale. Fit $z_j = (x_j - \bar{x}_{j,\text{train}})/s_{j,\text{train}}$ on training data and reuse those constants. A constant predictor must be removed or handled separately.

(b) For a prespecified set of k values, predict validation labels using training neighbours and the training-fitted scaling. Choose k from validation errors, optionally refit on training plus validation, and assess the frozen rule on the untouched test set. No test observation is a neighbour during fitting or tuning.

(c) Equalising numerical variance does not create signal, remove correlation, or assign an optimal metric. A pure-noise predictor still affects distance after standardisation. Normalisation does not mean transforming every marginal into a normal distribution.

Exercise 11.8.

Check class-specific rates alongside the aggregate. Then separate a possible explanation for a validation/test gap from what one realised gap actually establishes.

(a) Overall error is $(2 + 8)/60 = 1/6 = 0.1667$. Class 1 error is $2/20 = 0.10$; class 2 error is $8/40 = 0.20$. Class denominators are row totals here.

(b) Validation-driven selection favours favourable noise; the independent test sample has different noise. Sampling variation alone can also cause a difference, so a single gap is not a proof of selection bias or distribution shift.

(c) Keep an outer test split or outer CV folds separate [1]. For every candidate pipeline in inner CV, fit its scaling and any feature selection on the inner training fold [1], predict that fold's validation observations from its training neighbours [1], and select the candidate using aggregated inner loss, then refit on the outer training data and evaluate externally [1].

(d) Once used to choose settings, the former test set is now part of model selection. The best score on it is selected and no longer an independent final assessment. Use genuinely fresh test data or an assessment design that repeats the selection within outer training folds; renaming the reused data does not repair the problem.

(e) Always predicting class 2 gives $40/60 = 2/3$ accuracy and is a useful no-feature baseline. It misses every class 1 observation. Overall accuracy hides that failure, so report class-specific errors and consider deployment losses/prevalence.

Exercise 11.7.

Treat preprocessing, neighbour selection and tuning as one procedure. Moving only the final classifier inside validation does not protect against information used earlier.

(a) Different units can dominate squared distances. For example subtract each training mean and divide by its training standard deviation, then apply the same transform to held-out points. This makes differences comparable in standard-deviation units; it does not guarantee equal predictive relevance or Gaussian marginals.

(b) (1) Prespecify plausible candidate k values and a metric. (2) Fit preprocessing on training data. (3) For each k, predict validation observations from training neighbours. (4) Select the best k by the validation criterion with a stated tie rule. (5) Optionally refit preprocessing and the neighbour reference set on training plus validation with k fixed. (6) Estimate accuracy on the untouched test set, without making new choices from its results. State whether reporting overall or class-specific accuracy. Refit-on-all-data deployment is possible afterwards, but the test score evaluated the preceding procedure and is not a new independent test of that final refit.

(c) The best observed validation result benefits from favourable validation noise as well as true performance. Selection therefore induces optimism. Independent test outcomes do not share that same selection event; this is a repeated-sampling tendency, not a guaranteed ordering on every split.

(d) Yes: choose a pruning penalty, depth or leaf-size restriction from training-fitted candidates using validation loss. A fully grown tree can overfit. Preserve a separate final evaluation.

Solutions for Chapter 12

Exercise 12.1.

Each impurity is a different function of the same class proportions. Use the convention $0 \log 0 = 0$ if a class is absent.

With $\widehat{p} = (0.6, 0.2, 0.2)$: cross-entropy $= -(0.6 \ln 0.6 + 2 \cdot 0.2 \ln 0.2) = 0.3065 + 0.6438 = 0.950$; Gini $= 0.6(0.4) + 2(0.2)(0.8) = 0.24 + 0.32 = 0.56$; misclassification rate $= 1 - 0.6 = 0.4$. For the identity, $\sum_g \widehat{p}_g(1 - \widehat{p}_g) = \sum_g \widehat{p}_g - \sum_g \widehat{p}_g^2 = 1 - \sum_g \widehat{p}_g^2$ since the proportions sum to 1. (Check: $1 - (0.36 + 0.04 + 0.04) = 0.56$.)

Exercise 12.2.

Weight each child impurity by its share of the parent observations. An unweighted average would evaluate a different split criterion.

Parent: $(400, 400)$, misclassification 0.5, Gini 0.5, cross-entropy $\ln 2 = 0.693$. *Misclassification.* Split A: both children have error $100/400 = 0.25$, weighted 0.25, so $\Delta I = 0.25$. Split B: child $(200, 400)$ has error $200/600 = 1/3$ and weight 0.75; child $(200, 0)$ has error 0; weighted 0.25, so $\Delta I = 0.25$. Identical. *Gini.* A: each child $2(0.75)(0.25) = 0.375$, so $\Delta I = 0.5 - 0.375 = 0.125$. B: $0.75 \cdot 2(\frac{1}{3})(\frac{2}{3}) = 0.333$, so $\Delta I = 0.1667$. *Cross-entropy.* A: each child $-(0.75 \ln 0.75 + 0.25 \ln 0.25) = 0.5623$, $\Delta I = 0.693 - 0.562 = 0.131$; B: $0.75 \times 0.6365 = 0.4774$, $\Delta I = 0.216$. Both smooth criteria prefer B. A pure child is a finished leaf that classifies perfectly, while the remaining child $(200, 400)$ can still be split further; split A leaves two moderately impure nodes and the misclassification rate does not register that B has made progress. The strict concavity of Gini and entropy rewards purity.

Exercise 12.3.

Only cuts between consecutive distinct observed values need checking for the training partition. Compare weighted child impurities at those cuts.

Ordered data (class): $2(1), 3(1), 4(2), 5(2), 6(2), 10(1), 12(2)$; parent has 3 of class 1 and 4 of class 2, Gini $= 2 \cdot \frac{3}{7} \cdot \frac{4}{7} = 0.490$. Candidate cuts and ΔI: 2.5: 0.109; **3.5**: left $\{2, 3\}$ pure, right has one class 1 and four class 2 ($I = 0.32$), $\Delta I = 0.490 - \frac{5}{7}(0.32) = \mathbf{0.261}$; 4.5: 0.085; 5.5: 0.014; 8: 0.004; 11: 0.061. The first split is $x < 3.5$: it isolates the pure class-1 group $\{2, 3\}$.

Exercise 12.4.

Track which observations go to each child. An affine rescaling preserves that partition, whereas a rotation changes what a single-coordinate cut can express.

For a monotone increasing map $x_j \mapsto ax_j + b$ $(a > 0)$, the set of observations with $x_j < \lambda$ is the set with $ax_j + b < a\lambda + b$, so every split partitions the data identically and has the same child compositions, hence the same ΔI; the best split is at the transformed cut-point. If $a < 0$ the left and right children swap, but ΔI is symmetric in the two children ($\widehat{P}_L \leftrightarrow 1 - \widehat{P}_L$), so it is unchanged. A rotation does not preserve this: a split on a single coordinate after rotation is a split

on a linear combination before, which an axis-aligned tree cannot reproduce, so the trees (and their accuracy) can change. Training partitions are preserved by strictly monotone coordinate transformations, up to ties and equivalent-cut choices. Predictions between observed values can differ when a nonlinear transformation changes the midpoint convention. Rotations need not preserve even the available training partitions.

Exercise 12.5.

The larger tree improves training fit but pays for more leaves. Equating the two penalised objectives identifies the value where the preference changes.

Tree 1: $0.30 + 0.20 = 0.50$ plus 2α. Tree 2: $0.05 + 0.05 + 0.10 + 0 + 0.10 = 0.30$ plus 5α. At $\alpha = 0.15$: tree 1 $= 0.80$, tree 2 $= 1.05$, so tree 1 is preferred. At $\alpha = 0.05$: tree 1 $= 0.60$, tree 2 $= 0.55$, so tree 2 is preferred. A larger penalty favours the smaller tree; the break-even is at $0.50 + 2\alpha = 0.30 + 5\alpha$, i.e. $\alpha = 0.0667$.

Exercise 12.6.

Perfect training classification is a property of the fitted partition, not a measurement on new cases. Ask how tree size was selected before trusting its reported error.

An unrestricted tree can isolate distinct training inputs, so perfect training accuracy is possible; conflicting labels at identical inputs prevent perfect purity. Practical depth, leaf-size and split restrictions can prevent it too. None of these training outcomes measures new-data accuracy. Select tree complexity within training/validation data, then report held-out or outer-CV performance. A binomial interval is appropriate for independent test outcomes conditional on a fixed fitted rule; it is not automatically valid for pooled CV errors. Out-of-bag assessment specifically requires a bootstrap construction such as a forest.

Exercise 12.7.

Distinguish fitting split locations from choosing tree complexity. Both can adapt to noise, so the assessment design must account for both.

(a) Repeated splitting can fit peculiarities of the training sample and even isolate observations, so low training error can coexist with high variance and poor performance on new data. Perfect purity is impossible when identical predictor vectors have conflicting labels unless the fitting rule does something beyond ordinary deterministic splits.

(b) Yes. For example, grow candidate subtrees along a cost-complexity pruning path on training data, predict validation labels and choose a pruning parameter using validation loss. Refit as appropriate with that setting fixed, then evaluate on an untouched test set. Depth, leaf size or split thresholds are other acceptable tuning choices. A tree's greedy split fitting does not itself choose predictive complexity without overfitting.

(c) Each split is an axis-aligned threshold. A leaf is an intersection of half-spaces, hence an axis-aligned rectangular region (possibly unbounded). A class region can be a union of several such leaves.

Solutions for Chapter 13

Exercise 13.1.

Expand every member's error around the ensemble mean. The average centred deviation vanishes, leaving the exact squared-error decomposition.

Here $f = \frac{1}{3}(1+2+6) = 3$. Left side: $(f-y)^2 = (3-4)^2 = 1$. Average member loss: $\frac{1}{3}[(1-4)^2+(2-4)^2+(6-4)^2] = \frac{17}{3} = 5.667$. Diversity: $\frac{1}{3}[(1-3)^2+(2-3)^2+(6-3)^2] = \frac{14}{3} = 4.667$. Then $5.667 - 4.667 = 1$, matching the left side. In general, $\frac{1}{T}\sum_t (f_t - y)^2 = \frac{1}{T}\sum_t \big[(f_t - f) + (f - y)\big]^2 = \frac{1}{T}\sum_t (f_t - f)^2 + 2(f-y)\frac{1}{T}\sum_t (f_t - f) + (f-y)^2$, and $\frac{1}{T}\sum_t (f_t - f) = 0$, so $(f-y)^2 = \frac{1}{T}\sum_t (f_t - y)^2 - \frac{1}{T}\sum_t (f_t - f)^2$.

Exercise 13.2.

There are T variance terms and $T(T-1)$ ordered covariance terms. Keeping that count visible prevents the common mistake of dividing all variance by T.

$\mathrm{Var}(\bar{f}) = \frac{1}{T^2}\big[\sum_t \mathrm{Var}\, f_t + \sum_{s\neq t} \mathrm{Cov}(f_s, f_t)\big] = \frac{1}{T^2}\big[T\sigma^2 + T(T-1)\rho\sigma^2\big] = \frac{\sigma^2}{T} + \frac{(T-1)\rho\sigma^2}{T} = \rho\sigma^2 + \frac{1-\rho}{T}\sigma^2$. For $\rho = 0.3$, $T = 100$: $0.3 + 0.007 = 0.307\,\sigma^2$. For $\rho = 0.8$: $0.8 + 0.002 = 0.802\,\sigma^2$. Adding trees only removes the $(1-\rho)\sigma^2/T$ term; the floor $\rho\sigma^2$ is reduced only by making the trees less correlated, which motivates randomising trees. Whether bootstrap samples or random feature subsets reduce correlation, and what they do to individual bias and variance, depends on the data and base learner.

Exercise 13.3.

Compute omission for one observation and one tree first. Independent bootstrap draws across trees then give the probability of having no out-of-bag prediction.

$(1 - 1/200)^{200} = 0.3669$, so an observation is out-of-bag for about $0.3669 \times 500 \approx 184$ of the 500 trees. The probability that it is in every bootstrap sample is $(1 - 0.3669)^{500} = 0.6331^{500} = \mathrm{e}^{500 \ln 0.6331} = \mathrm{e}^{-228.6} \approx 10^{-99}$: effectively zero, so a zero out-of-bag count is extraordinarily unlikely under independent bootstrap draws. The count is still random, with expectation about 184.

Exercise 13.4.

Only one point is misclassified, so calculate its weighted error, the learner coefficient and the normalising constant in that order.

The stump predicts $(+,+,-,-,-)$ and the truth is $(+,+,-,-,+)$, so only $x = 5$ is wrong: $\epsilon_1 = 0.2$. Then $\alpha_1 = \frac{1}{2}\ln(0.8/0.2) = \frac{1}{2}\ln 4 = 0.693$, $\mathrm{e}^{-\alpha_1} = 0.5$, $\mathrm{e}^{\alpha_1} = 2$. Unnormalised weights: $0.2 \times 0.5 = 0.1$ for each correct point, $0.2 \times 2 = 0.4$ for the wrong one; $Z_1 = 4(0.1) + 0.4 = 0.8 = 2\sqrt{0.2 \cdot 0.8}$. Normalised: 0.125 for $x = 1, 2, 3, 4$ and 0.5 for $x = 5$. The weighted error of the same stump under the new weights is 0.5 (all of it from $x = 5$), as Proposition 13.3 states.

Exercise 13.5.

The minimiser is a real-valued score, not a class label. Its sign supplies a decision; the logistic transformation recovers the corresponding probability.

$f^* = \frac{1}{2}\ln\frac{0.8}{0.2} = \frac{1}{2}\ln 4 = 0.693$. Then $\mathrm{e}^{-2f^*} = \mathrm{e}^{-1.386} = 0.25$ and $\tau_1 = 1/(1+0.25) = 0.8$. For $\tau_1 = 0.5$ the log odds are 0, so $f^* = 0$: the decision boundary of the classifier, where the sign changes.

Exercise 13.6.

Averaging identical predictions changes nothing. The relevant issue is the joint distribution of member errors, not simply how many members were fitted.

Identical members have $f_t = f$ for all t, so the diversity term in (13.2) is exactly 0 and the ensemble's squared error equals the average member loss: no gain. In the variance formula, $\rho = 1$ gives $\mathrm{Var}(\bar{f}) = \sigma^2$. A random forest breaks the correlation by training each tree on a different bootstrap sample and by restricting each split to a random subset of variables; AdaBoost makes each learner different by re-weighting the observations so that every new learner is trained on a problem on which the previous learner has weighted error exactly 0.5.

Exercise 13.7.

Describe the randomisation used to build members, the aggregation rule and the observations used for assessment. These are separate parts of a forest.

(a) Fit a tree to each bootstrap sample, considering a random subset of predictors at each split, and aggregate class votes or probabilities. Random predictor subsets reduce dependence between trees; averaging/voting can reduce variance when individual predictors are useful and imperfectly correlated. More trees do not remove systematic bias.

(b) For each observation, use predictions only from trees whose bootstrap sample omitted it, then aggregate the loss. This imitates held-out prediction for that forest construction. Repeatedly selecting settings by the same OOB score can introduce selection optimism, so an independent assessment is still valuable.

(c) Bagging fits base learners to perturbed datasets without sequentially targeting the preceding fit's errors. Boosting adds learners sequentially to reduce a specified loss, often by fitting reweighted errors or negative gradients; it is not simply another name for voting over bootstrap trees.

Solutions for Chapter 14

Exercise 14.1.

Rewrite the derivative in terms of the output itself. The expression $g(1-g)$ immediately reveals both the maximum slope and saturation.

$g'(z) = \dfrac{\mathrm{e}^{-z}}{(1+\mathrm{e}^{-z})^2} = \mathrm{e}^{-z}g(z)^2$, and since $1 - g(z) = \dfrac{\mathrm{e}^{-z}}{1+\mathrm{e}^{-z}}$, also $g(z)(1-g(z)) = \dfrac{\mathrm{e}^{-z}}{(1+\mathrm{e}^{-z})^2}$, the same expression. The derivative $g(1-g)$ is largest when $g = \frac{1}{2}$, i.e. $z = 0$, where it equals $\frac{1}{4}$.

Exercise 14.2.

Choose a threshold between the possible sums of the binary inputs. For XNOR, incompatible inequalities give a proof that no single threshold unit works.

(a) AND: $\boldsymbol{w} = (1,1)^\top$, $w_0 = -1.5$. Net input is $x_1 + x_2 - 1.5 \geq 0$ only for $(1,1)$. (b) OR: $\boldsymbol{w} = (1,1)^\top$, $w_0 = -0.5$: $x_1 + x_2 \geq 0.5$ unless both are 0. (c) NAND: negate AND: $\boldsymbol{w} = (-1,-1)^\top$, $w_0 = 1.5$: net $= 1.5 - x_1 - x_2 \geq 0$ unless both are 1. XNOR has the pattern $(0,0) \to 1$, $(0,1) \to 0$, $(1,0) \to 0$, $(1,1) \to 1$. It is the complement of XOR, and the proof of Proposition 14.3 applies with all inequalities reversed in sense (the same contradiction arises: $w_0 \geq 0$, $w_0 + w_1 < 0$, $w_0 + w_2 < 0$, $w_0 + w_1 + w_2 \geq 0$; adding the middle two gives $2w_0 + w_1 + w_2 < 0$ while the first and last give $2w_0 + w_1 + w_2 \geq 0$). So no single unit computes it.

Exercise 14.3.

Set the net input to zero to locate the boundary. Substitution tells you which side receives output one; the normal-vector norm converts scores to distances.

The boundary is $2x_1 - x_2 - 1 = 0$, i.e. $x_2 = 2x_1 - 1$. The unit outputs 1 where $2x_1 - x_2 - 1 \geq 0$, the side containing the direction $\boldsymbol{w} = (2,-1)$ (below the line). $(1,0)$: $2 - 0 - 1 = 1 \geq 0 \Rightarrow 1$. $(0,0)$: $-1 < 0 \Rightarrow 0$. $(1,2)$: $2 - 2 - 1 = -1 < 0 \Rightarrow 0$. The distance from the origin to the boundary is $|w_0|/\|\boldsymbol{w}\| = 1/\sqrt{5} = 0.447$.

Exercise 14.4.

Fix the convention at a zero net input before starting. Otherwise the very first update, and the subsequent sequence, are ambiguous.

Write the state as $(w_1, w_2; w_0)$. Start $(0,0;0)$. Present $(0,0)$ (class 0): net $= 0$, output $f(0) = 1 \neq 0$. Update: subtract $(0,0)$ and $w_0 \leftarrow -1$: $(0,0;-1)$. Present $(1,0)$ (class 1): net $= -1 < 0$, output $0 \neq 1$. Add: $(1,0;0)$. Present $(0,1)$ (class 1): net $= 0 \geq 0$, output 1, correct. Second pass: $(0,0)$: net $= 0$, output $1 \neq 0$: $w_0 \leftarrow -1$: $(1,0;-1)$. $(1,0)$: net $= 0$, output 1, correct. $(0,1)$: net $= -1 < 0$, output $0 \neq 1$: add: $(1,1;0)$. Third pass: $(0,0)$: net 0, output $1 \neq 0$: $(1,1;-1)$. $(1,0)$: net 0, output 1 correct. $(0,1)$: net 0, correct. Fourth pass: $(0,0)$: net -1, output 0 correct; $(1,0)$ and $(0,1)$ correct. Converged at $(1,1;-1)$: the boundary $x_1 + x_2 = 1$ (a point on the boundary counts as class 1).

Exercise 14.5.

Build XOR out of simpler gates and check all four possible inputs. A truth table is a complete verification for this finite input domain.

Use NAND and OR in the hidden layer and AND at the output. $h_1 = f(-2x_1 - 2x_2 + 3)$ (NAND: 0 only for $(1,1)$), $h_2 = f(2x_1 + 2x_2 - 1)$ (OR), $o = f(h_1 + h_2 - 1.5)$ (AND). Truth table: $(0,0)$: $h = (1,0)$, net -0.5, $o = 0$. $(0,1)$: $h = (1,1)$, net 0.5, $o = 1$. $(1,0)$: $h = (1,1)$, $o = 1$. $(1,1)$: $h = (0,1)$, net -0.5, $o = 0$. This is XOR $= (x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2)$.

Exercise 14.6.

Substitute one affine layer into the next and collect coefficients. Without a nonlinear hidden activation, depth cannot create the needed nonlinear feature.

With $u_q = w_{0q} + \sum_j w_{jq} x_j$ and $h_q = u_q$, the output net input is $z_k = w_{0k} + \sum_q w_{qk} u_q = (w_{0k} + \sum_q w_{qk} w_{0q}) + \sum_j (\sum_q w_{qk} w_{jq}) x_j$, which is again an affine function of $\boldsymbol{x}$. So the whole network is a single-layer network with weights $\tilde{w}_{jk} = \sum_q w_{qk} w_{jq}$ and a threshold (or sigmoid) output. By Proposition 14.3 no such unit computes XOR. Hidden layers are useful only if the hidden activation function is non-linear.

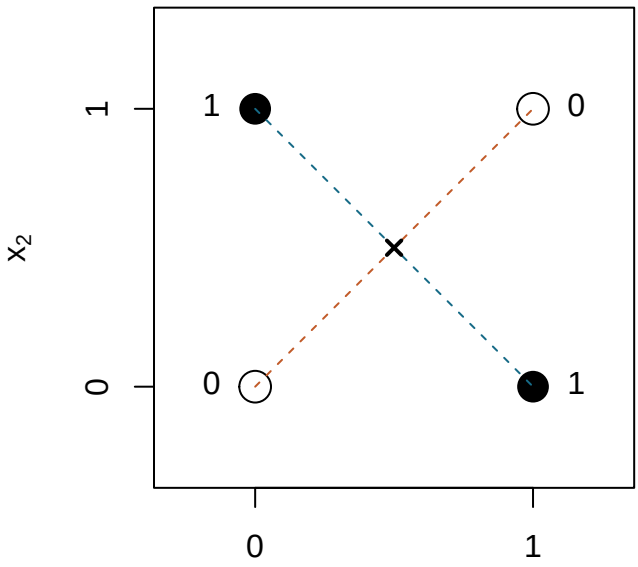
Exercise 14.7.

The geometric obstruction is intersecting class convex hulls. The hidden layer removes that obstruction by mapping the inputs to useful gate outputs.

(a) The two positive corners and the two negative corners lie on opposing diagonals of a square. Their convex hulls meet at $(1/2, 1/2)$, so no straight line strictly separates the two classes. Equivalently the four threshold inequalities are incompatible.

(b) Let $h_1 = \mathbf{1}\{x_1 + x_2 - 1/2 \geq 0\}$ (OR), $h_2 = \mathbf{1}\{-x_1 - x_2 + 3/2 \geq 0\}$ (NAND), and $y = \mathbf{1}\{h_1 + h_2 - 3/2 \geq 0\}$ (AND). Give weights and biases as specified by the three net inputs, and check the four inputs.

(c) Composing affine maps gives another affine map. Added depth alone would then create no nonlinear decision feature, so it would not solve this XOR classification problem.



The convex hulls of the two XOR classes intersect, preventing linear separation.

Solutions for Chapter 15

Exercise 15.1.

Carry out the forward pass in dependency order: hidden net inputs, hidden activations, output net input and final output. Keep those quantities distinct.

For $\boldsymbol{x} = (0, 1)$: $u_1 = 0.1 + 0.4(0) + (-0.3)(1) = -0.2$ and $u_2 = -0.2 + 0.3(0) + 0.2(1) = 0$. So $h_1 = g(-0.2) = 0.4502$ and $h_2 = g(0) = 0.5$. Then $z = 0.2 + 0.5(0.4502) - 0.4(0.5) = 0.2251$ and $o = g(0.2251) = 0.5560$.

Exercise 15.2.

Calculate the analytic derivative using the chain rule, then perturb the same weight in the original loss. The finite difference checks the complete path through the network.

$z = 1 \cdot 2 = 2$, $o = g(2) = 0.8808$, $E = (0 - 0.8808)^2 = 0.7758$. By (15.5): $\partial E/\partial w_1 = -2(y - o)\,\mathrm{e}^{-z} o^2 x = -2(-0.8808)(0.1353)(0.7758)(2) = 0.3699$. (Equivalently $2\,o \cdot o(1 - o)\,x = 2(0.8808)(0.1050)(2) = 0.3699$.) Central difference: $E(w_1 = 1.001) = (g(2.002))^2$ and $E(0.999) = (g(1.998))^2$; $g(2.002) = 0.88101$, $g(1.998) = 0.88059$, squares 0.77617 and 0.77543, difference $/0.002 \approx 0.370$, agreeing with 0.3699 to rounding. Without the e^{-z} factor the formula would give $0.3699/0.1353 = 2.734$, which is wrong by a factor $\mathrm{e}^2 = 7.39$.

Exercise 15.3.

Every receiving unit has a bias as well as an incoming weight per preceding unit. Count the two connections between layers separately.

$(p + 1)l + (l + 1)m = 11 \times 5 + 6 \times 3 = 55 + 18 = 73$. A multinomial logistic regression with $p = 10$ inputs and $G = 3$ classes has $(G - 1)(p + 1) = 2 \times 11 = 22$ parameters (Section 10.4); the network with five hidden units has more than three times as many.

Exercise 15.4.

Assume the two hidden units begin identically in both incoming and outgoing weights. Show that their gradients also agree, making equality persist under the stated update rule.

Suppose hidden units q and q' have $w_{jq} = w_{jq'}$ for all j and $w_{qk} = w_{q'k}$ for all k. Then for every pattern $u_{iq} = u_{iq'}$, so $h_{iq} = h_{iq'}$, and the error signals $\delta_{iq}^{(h)} = h_{iq}(1 - h_{iq}) \sum_k w_{qk} \delta_{ik}^{(o)}$ are identical for q and q'. Hence the gradients $\sum_i \delta_{iq}^{(h)} x_{ij}$ of their incoming weights are equal, and so are the gradients of their outgoing weights $\sum_i \delta_{ik}^{(o)} h_{iq}$. Both units receive the same update, so they remain identical forever: the network behaves as if it had one hidden unit (repeated). If all weights start at 0 then $h_{iq} = g(0) = 0.5$ for every hidden unit and every pattern, all hidden units are identical, and the output-layer gradients are also identical; the symmetry is never broken. A symmetry-breaking start is needed for these otherwise identical units to differentiate under this deterministic update. Random initialisation is one convenient way to obtain it.

Exercise 15.5.

The delta notation should reproduce the original gradient, including its sign and factor of two. Substitute the definitions rather than relying on remembered update formulas.

From (15.9), $\Delta w_{ab} = 2\varepsilon \sum_i \mathrm{e}^{-u_{ib}} h_{ib}^2 x_{ia} \sum_k [y_{ik} - o_{ik}] \mathrm{e}^{-z_{ik}} o_{ik}^2 w_{bk}$. Using $\mathrm{e}^{-u} h^2 = h(1-h)$ and $\mathrm{e}^{-z} o^2 = o(1-o)$, this is $2\varepsilon \sum_i h_{ib}(1-h_{ib}) x_{ia} \sum_k w_{bk} [y_{ik} - o_{ik}] o_{ik}(1-o_{ik})$. With $\delta_{ik}^{(o)} = -2(y_{ik} - o_{ik}) o_{ik}(1-o_{ik})$ we have $2[y_{ik} - o_{ik}] o_{ik}(1-o_{ik}) = -\delta_{ik}^{(o)}$, so $\Delta w_{ab} = -\varepsilon \sum_i x_{ia} h_{ib}(1-h_{ib}) \sum_k w_{bk} \delta_{ik}^{(o)} = -\varepsilon \sum_i x_{ia} \delta_{ib}^{(h)}$.

Exercise 15.6.

A large prediction error and a large parameter gradient are different things. The output activation derivative is the factor that separates them here.

$z = -8$: $o = g(-8) = 3.35 \times 10^{-4}$, error $= 1 - o \approx 0.9997$. $\partial E / \partial z = -2(y-o)\, o(1-o) = -2(0.9997)(3.35 \times 10^{-4})(0.9997) = -6.7 \times 10^{-4}$. At $z = -1$: $o = 0.2689$, $\partial E / \partial z = -2(0.7311)(0.2689)(0.7311) = -0.2874$. The gradient at the confidently wrong point is about 430 times smaller although the error is much larger: the unit is saturated and learns extremely slowly. For cross-entropy loss $-y \ln o - (1-y) \ln(1-o)$, $\partial E / \partial z = o - y$, which equals -0.9997 at $z = -8$ and -0.7311 at $z = -1$: the gradient grows with the error and is not damped by saturation.

Exercise 15.7.

Trace the single path from the selected input weight to the loss. For the half-squared-error convention in this exercise, there is no extra factor of two.

(a) With $\delta_o = (o-y)o(1-o)$, the output-weight derivative is $\delta_o h$ and the output-bias derivative is δ_o. These contain the loss derivative, sigmoid derivative and derivative of the net input.

(b)

$$\frac{\partial L}{\partial a} = (o-y)o(1-o)\, v\, h(1-h)\, x.$$

The factors traverse the path from loss to output, output net input, hidden activation, hidden net input and weight a.

(c) $a_{\text{new}} = a - \eta\, \partial L / \partial a$, $\eta > 0$: the minus sign moves locally against increasing loss. A step that is too large may increase loss or destabilise training; too small a step can be slow. The derivatives for a simultaneous update are evaluated at the same old parameters.

Solutions for Chapter 16

Exercise 16.1.

First use second moments to choose a variance scale. The tail probability requires the additional fixed-input Gaussian-initialisation assumption stated in the revised question.

$\mathrm{Var}(u) = (p+1)\sigma^2 = 1$ gives $\sigma = 26^{-1/2} = 0.196$. With $\sigma = 0.5$, $\mathrm{Var}(u) = 26 \times 0.25 = 6.5$, standard deviation 2.55, and, conditional on the fixed input with squared norm 25 and Gaussian initial weights, $\mathbb{P}(|u| > 2 \mid \boldsymbol{x}) = 2[1 - \Phi(2/2.55)] = 2[1 - \Phi(0.784)] = 2(0.2165) = 0.43$. Across repeated initialisations at that input, about 43% of net inputs fall outside $[-2, 2]$, where the sigmoid derivative is below 0.105, so many units begin partly saturated.

Exercise 16.2.

With two classes, only the difference of the logits affects probabilities. Re-express the loss in that difference before differentiating.

For $G = 2$: $E = -y_1 \ln s_1 - y_2 \ln s_2$ with $s_1 = \mathrm{e}^{z_1}/(\mathrm{e}^{z_1} + \mathrm{e}^{z_2}) = g(z_1 - z_2) =: o$ and $s_2 = 1 - o$. By (16.4), $\partial E/\partial z_1 = s_1 - y_1 = o - y_1$ and $\partial E/\partial z_2 = s_2 - y_2 = -(o - y_1)$ (since $y_2 = 1 - y_1$). So with the single logit $z = z_1 - z_2$, $\partial E/\partial z = o - y_1$, which is the cross-entropy gradient for a sigmoid output $o = g(z)$ with target $y = y_1$. Direct check: $E = -y \ln g(z) - (1 - y) \ln(1 - g(z))$ has $\partial E/\partial z = -y(1 - g) + (1 - y)g = g - y$.

Exercise 16.3.

Add the penalty derivative to the data derivative, then make one gradient step. A stationary balance is a condition involving both terms at the same weight.

Without decay: $w' = 2 - 0.1(0.4) = 1.96$. With decay: $w' = 2 - 0.1(0.4 + 0.5 \times 2) = 2 - 0.14 = 1.86$ (equivalently $(1 - 0.05) \times 2 - 0.04 = 1.86$). The net update is zero when $\partial E/\partial w + \lambda w = 0$, i.e. $w = -\frac{1}{\lambda}\frac{\partial E}{\partial w} = -0.8$. In general the penalised optimum balances the data gradient against the shrinkage force.

Exercise 16.4.

An epoch and an update are not the same unit of work. Count pattern evaluations before comparing the two computations.

Batch: each epoch passes all 5000 patterns, so 10 000 epochs cost 5×10^7 pattern-passes. Online: 10 000 iterations cost 10^4 pattern-passes, which is $10^4/5000 = 2$ epochs' worth of data, so the cost is 5000 times smaller. In terms of data seen, it is two epochs; whether two epochs of single-pattern updates are as good as thousands of batch epochs depends on the problem, and learning-rate schedules, noise and redundancy can change the comparison. Equal iteration counts do not imply equal computational effort or equal optimisation accuracy.

Exercise 16.5.

A counterexample only needs to reverse the ranking of two predictors under the two losses. Values close to the threshold can be correct yet contribute appreciable squared error.

SSE measures how close the outputs are to the targets 0/1, while accuracy only counts which side of the threshold each output lies on. Take 100 patterns. Network A outputs 0.6 for every target-1 pattern and 0.4 for every target-0 pattern: perfect accuracy but SSE $= 100 \times 0.16 = 16$. Network B outputs essentially exactly the target for 90 patterns and the opposite extreme for 10: accuracy 90%, but SSE $\approx 10 \times 1 = 10 < 16$. So B has the smaller SSE and the lower accuracy. If the target is zero-one classification loss, validation accuracy addresses that target directly; its disadvantages are that it is discrete and noisy (it changes in steps of $1/n_{\text{val}}$, a poor guide when the validation set is small) and it ignores how confident the predictions are. Cross-entropy or SSE on the validation set varies more smoothly.

Exercise 16.6.

Count connections and biases exactly. The ratio of target entries to weights is bookkeeping, not an effective-sample-size calculation.

$(p+1)l+(l+1)m = 204(80)+81(26) = 18\,426$. With $l = 50$, the count becomes $204(50)+51(26) = 11\,526$. A table with 5000 rows and 26 output entries has 130,000 target entries, about seven per weight in the larger network. Those entries are not necessarily independent information, and the count supplies no generalisation guarantee. The dependence structure of patterns and outputs, the architecture and the fitting procedure still matter.

Exercise 16.7.

Stored weights, identifiable parameters and effective model complexity are different counts. State the architecture convention, then describe how stopping and decay are selected.

(a) Input-to-hidden: $4 \times 6 + 6 = 30$; hidden-to-output: $6 \times 3 + 3 = 21$; total 51 stored trainable parameters. Softmax has a common-logit shift nonidentifiability, but the stated unconstrained implementation still stores these 51 coefficients; a baseline-constrained parametrisation has a different count.

(b) Track validation loss after epochs of training, retain the parameters at the selected minimum (possibly using a prespecified patience rule), then assess once on an independent test set. The epoch is chosen using validation outcomes, so that minimum is selected and can be optimistic.

(c) Adding $\lambda \sum w^2$ adds $2\lambda w$ to the gradient for each penalised weight and discourages large weights. This smooth penalty typically shrinks weights continuously, unlike the kink in L_1 penalisation that can yield exact zeros. State whether biases are included; either convention is valid when explicit.

Solutions for Chapter 17

Exercise 17.1.

The score changes if the weights are rescaled, while geometric distance should not. Divide by the norm before interpreting the numerical value.

$\|\boldsymbol{w}\| = 5$. For $(2,1)$: $\boldsymbol{w}^\top \boldsymbol{x} + b = 6 + 4 - 5 = 5$, so the signed distance is $5/5 = 1$. For $(0,0)$: $-5/5 = -1$ (the origin is on the negative side, one unit away). The margin between $\boldsymbol{w}^\top \boldsymbol{x} + b = \pm 1$ is $2/\|\boldsymbol{w}\| = 2/5 = 0.4$.

Exercise 17.2.

The dual equality reduces this example to one nonnegative variable. After maximising it, recover the primal separator and check both training constraints.

By (17.5) $\sum_i \alpha_i y_i = 0$ forces $\alpha_1 = \alpha_2 = \alpha$. Then $\boldsymbol{w} = \alpha[(3,3) - (1,1)] = \alpha(2,2)$ and $Q(\alpha) = 2\alpha - \frac{1}{2}\alpha^2 \|\boldsymbol{x}_1 - \boldsymbol{x}_2\|^2 = 2\alpha - 4\alpha^2$ (since $\|(2,2)\|^2 = 8$), maximised at $\alpha = \frac{1}{4}$. So $\boldsymbol{w} = (0.5, 0.5)^\top$, and from the support vector $\boldsymbol{x}_1$: $b = y_1 - \boldsymbol{w}^\top \boldsymbol{x}_1 = 1 - 3 = -2$; check with $\boldsymbol{x}_2$: $y_2(\boldsymbol{w}^\top \boldsymbol{x}_2 + b) = -(1-2) = 1$. The margin is $2/\|\boldsymbol{w}\| = 2/0.7071 = 2.828 = \sqrt{8} = \|\boldsymbol{x}_1 - \boldsymbol{x}_2\|$, as it must be: with only two points the separator is the perpendicular bisector of the segment joining them. The boundary is $0.5x_1 + 0.5x_2 = 2$, i.e. $x_1 + x_2 = 4$.

Exercise 17.3.

Test whether the old optimum remains feasible after adding the point. Extending its dual vector with a zero gives a direct optimality check.

The separator is $x_1 = 2$ with margin boundaries $x_1 = 1$ and $x_1 = 3$. For $\boldsymbol{x}_4 = (6,0)$: $\boldsymbol{w}^\top \boldsymbol{x}_4 + b = 6 - 2 = 4 > 1$, so $y_4 f(\boldsymbol{x}_4) = 4$, well outside the margin. By complementary slackness $\alpha_4 = 0$, and the solution of Example 17.4 (with the same $\boldsymbol{\alpha}$ and $\alpha_4 = 0$) still satisfies all the constraints and the KKT conditions, so it is still optimal and, by uniqueness, the solution. The support vectors remain $\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3$. Only support vectors matter.

Exercise 17.4.

An explicit feature map verifies the kernel identity. The transformed coordinate then makes the separation visible, but functional and geometric margins still need care.

$(1 + xx')^2 = 1 + 2xx' + x^2 x'^2 = (1)(1) + (\sqrt{2}x)(\sqrt{2}x') + (x^2)(x'^2) = \varphi(x)^\top \varphi(x')$. For the data of Figure 17.3, blue points have $|x| \le 0.9$ so $\varphi_3 = x^2 \le 0.81$, while orange points have $|x| \ge 1.5$ so $\varphi_3 \ge 2.25$. The horizontal line $\varphi_3 = 1.5$ (a hyperplane in φ-space) therefore separates them, The displayed boundary at 1.5 has guaranteed distances at least $1.5 - 0.81 = 0.69$ and $2.25 - 1.5 = 0.75$ on its two sides. Centring it at $(0.81 + 2.25)/2 = 1.53$ gives equal distances 0.72 and full slab width 1.44 in this coordinate direction. In the original space it is the pair of points $x = \pm\sqrt{1.5}$.

Exercise 17.5.

Positive diagonal entries are not enough for positive definiteness. For this symmetric matrix, check the leading principal minors or its eigenvalues.

$K_{ij} = \exp(-(i-j)^2/2)$ gives $K_{12} = K_{23} = \mathrm{e}^{-1/2} = 0.6065$, $K_{13} = \mathrm{e}^{-2} = 0.1353$, with ones on the diagonal. Leading principal minors: 1; $1 - 0.6065^2 = 0.632$; and $\det \boldsymbol{K} = 1(0.632) - 0.6065(0.6065 - 0.6065 \times 0.1353) + 0.1353(0.3679 - 0.1353) = 0.632 - 0.318 + 0.031 = 0.346$. All positive, so $\boldsymbol{K}$ is positive definite (Sylvester's criterion). Its eigenvalues are 0.207, 0.865, and 1.928, all positive and summing to $3 = \operatorname{tr} \boldsymbol{K}$.

Exercise 17.6.

Use complementary slackness together with the box constraints. A coefficient at the upper bound need not mean the point is misclassified; it can also lie on the margin.

With $C = 2$: $\alpha = 0$: the point is correctly classified and lies on or outside the margin ($yf \geq 1$), slack $\xi = 0$; it does not influence the solution. $\alpha = 1.3$ (strictly between 0 and C): the point lies exactly on the margin ($yf = 1$), slack $\xi = 0$; it is a free support vector and can be used to compute $b = y_k - \sum_i \alpha_i y_i \boldsymbol{x}_i^\top \boldsymbol{x}_k$. $\alpha = 2 = C$: a bound support vector, inside the margin or misclassified ($yf \leq 1$), with slack $\xi = 1 - yf \geq 0$ ($\xi > 1$ if misclassified); its slack is not known in advance, so it cannot be used for b without computing ξ.

Exercise 17.7.

Consider what a very narrow kernel does between distinct training points. It can make interpolation easy without supplying evidence about labels away from those points.

With distinct inputs and $\gamma = 1/(2\sigma^2)$ very large, off-diagonal Gaussian-kernel entries approach zero. The mapped observations become nearly orthogonal, making interpolation possible at a sufficiently large cost parameter. Finite C penalises slack; it does not literally forbid it. Away from the training inputs, the narrow kernel contributions fade and the prediction approaches the sign of the intercept. This construction explains why perfect training accuracy can coexist with poor generalisation, without proving that it must do so on every distribution. Tune (C, γ) inside training data and assess the selected procedure independently.

Exercise 17.8.

Keep the orientation, offset and normalisation of the hyperplane separate. Then use the given training point to test feasibility of the claimed hard-margin fit.

(a) w is a normal to the decision boundary and determines its orientation; b offsets it. The boundary is $3x_1 + 4x_2 - 10 = 0$ and the predicted sign is that of this expression (with a stated tie rule at zero).

(b) $\|w\| = 5$. The hyperplanes $w^T x + b = \pm 1$ are separated by $2/\|w\| = 0.4$, not by 2 unless w is a unit vector.

(c) Active margin observations satisfy $y_i(w^T x_i + b) = 1$; observations with nonzero dual coefficients contribute to $w = \sum_i \alpha_i y_i x_i$ (degeneracy can give active constraints with zero multiplier). At $(2, 1)$ the score is $6 + 4 - 10 = 0$, so the positive observation violates $y_i f_i \geq 1$ and could not be part of this hard-margin training fit.

Exercise 17.10.

Evaluate the proposed feature-space score at each input. A successful feature map explains representability, while fitting and generalisation remain separate questions.

(a) The positive and negative convex hulls intersect at the centre of the square; no affine line can strictly separate them. A labelled diagram and this explanation earn full credit.

(b) The scores at $(0,0), (1,0), (0,1), (1,1)$ are $-1/2, +1/2, +1/2, -1/2$ respectively, matching the labels.

(c) In feature space $f = w^T \phi(x) + b$ with $w = (1, 1, -2)^T$; composing with a nonlinear ϕ gives a nonlinear equation in x. A valid kernel evaluates feature-space inner products $K(x, z) =$

$\langle \phi(x), \phi(z) \rangle$ without necessarily constructing the features explicitly. This supports the SVM optimisation and prediction; it does not automatically choose a good kernel or tuning parameters.

Exercise 17.9.

The unit-normal convention turns the displayed slab width into a distance. Without it, the same algebraic constants would have a different geometric meaning.

(a) w is a unit normal, fixing the orientation; b fixes the signed offset of the hyperplane; ρ is the full geometric width between its two supporting parallel margin planes. The unit norm is essential to interpreting the given ρ as a distance.

(b) $w^T x + b = 0$, with positive scores predicted as $+1$ and negative scores as -1.

(c) In the hard-margin geometry, the closest observations lie on $w^T x + b = \pm\rho/2$. Their constraints limit enlargement of the separating margin. The optimal normal can be represented through observations with nonzero dual coefficients; in a degenerate configuration an active boundary observation may have a zero coefficient. Non-support observations strictly beyond the margin do not directly contribute to that representation.

Solutions for Chapter 18

Exercise 18.1.

Centring removes at least one row degree of freedom. Subtract the resulting rank upper bound from the matrix dimension to count unavoidable zero eigenvalues.

$\operatorname{rank}(\boldsymbol{S}) \leq \min(n-1, p) = \min(3, 6) = 3$. Since $\boldsymbol{S}$ is 6×6, at least $6 - 3 = 3$ eigenvalues are zero. At most 3 principal components have positive variance. For $n = 4$, $p = 3$: $\min(n-1, p) = \min(3, 3) = 3$, so $\boldsymbol{S}$ can have full rank 3 (if the centred observations are in general position), and is then invertible.

Exercise 18.2.

The null direction explains non-uniqueness of least squares. Adding a positive ridge penalty lifts that zero eigenvalue and selects a unique coefficient vector.

$\boldsymbol{X}^\top \boldsymbol{X} = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$ has determinant 0; its eigenvalues are 4 (eigenvector $(1, 1)$) and 0 (eigenvector $(1, -1)$). With $\lambda = 1$, $\boldsymbol{X}^\top \boldsymbol{X} + \boldsymbol{I} = \begin{pmatrix} 3 & 2 \\ 2 & 3 \end{pmatrix}$ has determinant 5 and inverse $\frac{1}{5} \begin{pmatrix} 3 & -2 \\ -2 & 3 \end{pmatrix}$. Also $\boldsymbol{X}^\top \boldsymbol{y} = (4, 4)^\top$. So $\widehat{\boldsymbol{\beta}}_R = \frac{1}{5}(12 - 8,\ -8 + 12)^\top = (0.8, 0.8)^\top$. The solution splits the effect equally between the two identical predictors, and is the unique ridge solution; least squares has infinitely many solutions $\beta_1 + \beta_2 = 2$.

Exercise 18.3.

Bonferroni uses the same cutoff for every test. BH compares ordered values with rank-dependent cutoffs and then applies a step-up rule.

The threshold is $0.05/5000 = 10^{-5}$. For a two-sided normal test this requires $|z| \geq \Phi^{-1}(1 - 5 \times 10^{-6}) = 4.42$. The BH thresholds at $q^* = 0.05$ are $5(0.05)/5000 = 5 \times 10^{-5}$ for the fifth smallest and $100(0.05)/5000 = 10^{-3}$ for the hundredth: the threshold relaxes linearly with rank, so the more discoveries are made, the easier it is to make the next.

Exercise 18.4.

Do not stop at the first failed comparison. Find the largest successful rank, then reject every hypothesis up to that rank.

Thresholds $iq^*/m = 0.01, 0.02, 0.03, 0.04, 0.05$. Compare: $p_{(1)} = 0.001 \leq 0.01$ yes; $p_{(2)} = 0.025 > 0.02$ no; $p_{(3)} = 0.026 \leq 0.03$ yes; $p_{(4)} = 0.027 \leq 0.04$ yes; $p_{(5)} = 0.800 > 0.05$ no. The largest i meeting the condition is $k = 4$, so $H_{(1)}, \ldots, H_{(4)}$ are all rejected, including $H_{(2)}$ even though its p-value exceeds its own threshold. It is a *step-up* procedure: working up from the largest p-value, the first one below its line triggers rejection of all smaller p-values. If the data support rejecting the hypothesis with the 4th smallest p-value at the FDR level, then the hypotheses with smaller p-values are even stronger evidence and cannot sensibly be retained. (Declaring $p_{(4)}$ significant but $p_{(2)}$ not would be incoherent, because this BH rule uses a common selected cutoff.)

Exercise 18.5.

Use the penalty normalisation in the question: raw SSE plus λ times the penalty. This is why the lasso threshold is $\lambda/2$ here.

Ridge ($\lambda = 2$): divide by 3: (0.80, −0.30, 0.167, −0.033), all non-zero. Lasso: threshold $\lambda/2 = 1$, so $\text{sign}(b)(|b| - 1)_+ = (1.4, 0, 0, 0)$. Only the first variable is selected; the coefficient is shrunk from 2.4 to 1.4.

Exercise 18.6.

Locate the first step that used the held-out labels. Feature selection already used them, so validating only the subsequent classifier leaves the leakage in place.

The 20 genes were chosen by looking at the labels of *all* 62 samples. The cross-validation is then done only on the modelling step, with the selection already informed by the held-out samples in each fold: the selected genes are, by construction, the ones that happen to separate these samples best, and with 20 000 candidates some genes will separate the classes well by chance alone. The held-out observations were therefore not independent of the model, and the 4% error is optimistic (it may be near 0 even for pure noise). The correction is to put the entire pipeline inside the cross-validation: in each fold, compute the p-values (or fit the lasso) using the training folds only, select the genes, fit the classifier, and evaluate on the held-out fold. Choosing the number of genes or λ also needs an inner loop (nested cross-validation, Chapter 9).

Exercise 18.7.

Total scatter and within-class scatter have different centring constraints. A PCA dimension allowed by the first can still be too large for ordinary LDA.

(a) The pooled covariance is singular because its rank is at most $n - g = 88 < 500$, so its ordinary inverse does not exist. With only 90 observations, tuning and test errors can be variable, and using the training or tuning data for final assessment creates optimism. Class imbalance also makes overall accuracy alone incomplete.

(b) Centred data have rank at most $\min(p, n-1) = 89$, so there can be at most 89 nonzero sample PCs. Within-class centring loses one degree of freedom per class, giving rank at most $n - g = 88$. Thus $q \leq 88$ is necessary for an invertible projected pooled covariance; it is not sufficient if further dependencies remain. Using all 89 PCs does not solve ordinary LDA's rank problem.

(c) Select q within training-only CV, refitting centring/scaling/PCA/LDA in every fold; use held-out testing or outer CV for assessment. Apply the rank restriction to the smaller training fold, not just to the full sample. High-variance directions may contain little class information, and near-singular covariance estimates can be unstable even below the rank limit.

Exercise 18.8.

The relevant bound is the rank of the pooled within-class covariance. Apply it again inside each training fold because that fold has fewer observations.

(a) The pooled within-class covariance has rank at most $72 - 2 = 70$, so a 400×400 inverse is impossible without changing the estimator. The small sample makes evaluation variable and makes resubstitution or tuning-set assessment particularly vulnerable to optimism. Accuracy must also be interpreted against the 2:1 class split.

(b) Fit PCA on training data and project into q dimensions [1]. Centred total data have rank at most 71 [1], whereas pooled within-class scatter has rank at most 70 [1]. Thus $q \leq 70$ is necessary for ordinary LDA, with actual full rank still required [1]. Choose q by training-only validation/CV, refitting the complete pipeline in each fold and obeying that fold's rank limit; high explained variance is not proof of predictive value [1].

Exercise 18.9.

The lasso asks a joint prediction question; marginal hypothesis tests ask about individual associations. Their selected lists need not target the same population property.

(a) CV commonly targets prediction error, not exact support recovery. Correlated predictors can substitute for one another and finite-sample noise can add or omit variables. Penalisation also biases coefficient estimates; good predictive loss is not proof of recovering the generating coefficients.

(b) Sort valid p-values $p_{(1)} \leq \cdots \leq p_{(1000)}$ and take $k = \max\{i : p_{(i)} \leq 0.1i/1000\}$, rejecting the first k (none if the set is empty) [2]. FDR is $E[V/\max(R, 1)]$, the expected fraction of false rejections among discoveries, not a guarantee on each realised sample [1]. Usual BH control requires appropriate dependence conditions, such as independence or suitable positive dependence; arbitrary dependence needs additional care [1]. A predictor can be marginally associated through correlation with other predictors even when its joint coefficient is zero, so the two procedures test different targets [1].

Solutions for Chapter 19

Exercise 19.1.

Find neighbours and paths directly from the edge set. A proposed separator fails as soon as one path avoids it.

The edges are $12, 13, 23, 24, 34, 45, 46, 56$. $\mathrm{bd}(\{1,2\})$ is the set of vertices outside $\{1,2\}$ adjacent to 1 or 2: vertex 3 (adjacent to both) and vertex 4 (adjacent to 2), so $\{3,4\}$. $\mathrm{cl}(\{5\}) = \{5\} \cup \mathrm{bd}(\{5\}) = \{4,5,6\}$. Global Markov statements: (i) $\{4\}$ separates $\{1,2,3\}$ from $\{5,6\}$, so $\boldsymbol{X}_{\{1,2,3\}} \perp\!\!\!\perp \boldsymbol{X}_{\{5,6\}} \mid X_4$; (ii) $\{2,3\}$ separates 1 from $\{4,5,6\}$: $X_1 \perp\!\!\!\perp \boldsymbol{X}_{\{4,5,6\}} \mid \boldsymbol{X}_{\{2,3\}}$; (iii) the local property at vertex 5: $X_5 \perp\!\!\!\perp \boldsymbol{X}_{\{1,2,3\}} \mid \boldsymbol{X}_{\{4,6\}}$ (and pairwise: $X_1 \perp\!\!\!\perp X_4 \mid \boldsymbol{X}_{\{2,3,5,6\}}$). $\{2\}$ is not a $(1,4)$-separator because the path $1-3-4$ avoids 2.

Exercise 19.2.

Read the graph from off-diagonal precision zeros, then invert to see why the covariance can tell a different marginal story.

Since $k_{13} = 0$ the graph is the chain $1-2-3$. $|\boldsymbol{K}| = 2(6-1) - (-1)(-2) = 10 - 2 = 8$, and the cofactors give $\boldsymbol{\Sigma} = \boldsymbol{K}^{-1} = \frac{1}{8}\begin{pmatrix} 5 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 5 \end{pmatrix}$. The marginal correlation of X_1 and X_3 is $\sigma_{13}/\sqrt{\sigma_{11}\sigma_{33}} = (1/8)/(5/8) = 0.2 \neq 0$, yet $k_{13} = 0$ so $X_1 \perp\!\!\!\perp X_3 \mid X_2$ by Proposition 19.2. Partial correlations (19.10): $\rho_{12|3} = 1/\sqrt{2 \times 3} = 0.408$, $\rho_{23|1} = 1/\sqrt{3 \times 2} = 0.408$, $\rho_{13|2} = 0$.

Exercise 19.3.

Propagate variances and covariances through the generating equations. This derives the covariance matrix before using its inverse to check the graph.

$\mathrm{Var}\, X_2 = 0.64 \cdot 1 + 0.36 = 1$, $\mathrm{Var}\, X_3 = 0.64 \cdot 1 + 0.36 = 1$; $\mathrm{Cov}(X_1, X_2) = 0.8$, $\mathrm{Cov}(X_2, X_3) = 0.8\,\mathrm{Var}\, X_2 = 0.8$, $\mathrm{Cov}(X_1, X_3) = 0.8\,\mathrm{Cov}(X_1, X_2) = 0.64$. So $\boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0.8 & 0.64 \\ 0.8 & 1 & 0.8 \\ 0.64 & 0.8 & 1 \end{pmatrix}$, the autoregressive form with $r = 0.8$, whose inverse is $\frac{1}{1-r^2}\begin{pmatrix} 1 & -r & 0 \\ -r & 1+r^2 & -r \\ 0 & -r & 1 \end{pmatrix} = \frac{1}{0.36}\begin{pmatrix} 1 & -0.8 & 0 \\ -0.8 & 1.64 & -0.8 \\ 0 & -0.8 & 1 \end{pmatrix} = \begin{pmatrix} 2.778 & -2.222 & 0 \\ -2.222 & 4.556 & -2.222 \\ 0 & -2.222 & 2.778 \end{pmatrix}$. Indeed $k_{13} = 0$. (Check: the first row of $\boldsymbol{\Sigma}$ times the third column of $\boldsymbol{K}$ is $0.8(-2.222) + 0.64(2.778) = -1.778 + 1.778 = 0$, as it should be for off-diagonal entries of $\boldsymbol{\Sigma}\boldsymbol{K} = \boldsymbol{I}$.)

Exercise 19.4.

Condition on the middle variable and calculate the Schur complement for the two endpoints. Their marginal covariance cancels against the part explained by the middle.

Take $A = \{1,3\}$, $B = \{2\}$. $\boldsymbol{\Sigma}_{AB} = (0.8, 0.8)^\top$, $\boldsymbol{\Sigma}_{BB} = 1$, so $\boldsymbol{\Sigma}_{A|B} = \begin{pmatrix} 1 & 0.64 \\ 0.64 & 1 \end{pmatrix} - \begin{pmatrix} 0.8 \\ 0.8 \end{pmatrix} \begin{pmatrix} 0.8 & 0.8 \end{pmatrix} = \begin{pmatrix} 1 - 0.64 & 0.64 - 0.64 \\ 0.64 - 0.64 & 1 - 0.64 \end{pmatrix} = \begin{pmatrix} 0.36 & 0 \\ 0 & 0.36 \end{pmatrix}$. The conditional covariance is 0 (hence conditional independence for a normal vector), and the conditional variances are 0.36 (the innovation variance), while the unconditional $\mathrm{Cov}(X_1, X_3) = 0.64 \neq 0$.

Exercise 19.5.

Each unordered pair contributes one binary choice. The resulting count explains the computational obstacle without implying that the likelihood at a candidate model is undefined.

Each of the $\binom{p}{2}$ possible edges is present or absent independently: $2^{\binom{p}{2}}$ graphs. For $p = 5$: $2^{10} = 1024$. For $p = 20$: $2^{190} \approx 1.6 \times 10^{57}$. Even allowing for symmetries (relabelling vertices) exhaustive search is impractical. Criteria can still compare a manageable candidate set. With singular sample covariance the unrestricted positive-definite Gaussian MLE fails to exist, although the likelihood at a candidate positive-definite matrix remains defined. Regularisation by an L_1 penalty turns the discrete search into a convex optimisation with a path of solutions indexed by one parameter λ.

Exercise 19.6.

A covariance graph and a Gaussian concentration graph encode different independence questions. A chain gives a concrete case where marginal thresholding answers the wrong one.

In the Gaussian chain $1 - 2 - 3$, the endpoints can have nonzero marginal covariance while being independent conditional on the middle variable. Thresholding marginal correlations therefore estimates a different object from a concentration graph; it is not an indirect way to estimate the same missing-edge restrictions. Under the nonsingular Gaussian model, use the negative normalised precision entries for partial correlations. In high dimension, graphical lasso or neighbourhood selection supplies an estimated sparse structure, whose tuning and sampling uncertainty still need assessment. The undirected graph itself identifies no causal direction.

Exercise 19.7.

The Gaussian assumption connects the precision calculation to conditional independence. Inverting the matrix separately checks that marginal independence does not follow.

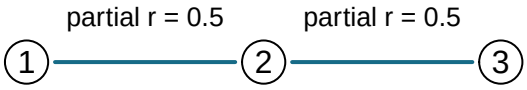
(a) The graph is the chain $1-2-3$, with edges 12 and 23 and no edge 13. It implies $X_1 \perp X_3 \mid X_2$ for the specified nonsingular Gaussian model.

(b) $\rho_{12\cdot 3} = \rho_{23\cdot 1} = 1/2$ and $\rho_{13\cdot 2} = 0$. The conversion uses a minus sign and normalises by precision diagonals, so raw precision entries are not the partial correlations.

(c) No. In fact

$$\Sigma = K^{-1} = \frac{1}{4} \begin{pmatrix} 3 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 3 \end{pmatrix}.$$

The nonzero covariance $1/4$ implies marginal dependence for this Gaussian vector (marginal correlation is $1/3$). Conditional separation does not remove the marginal association carried through X_2.



The Gaussian chain has positive partial correlations on both edges and conditional independence between nodes 1 and 3 given node 2.

Exercise 19.8.

Read signs only after normalising and negating the precision entries. The graphical-lasso objective then explains how an estimated edge can be removed by penalisation.

(a) Edges are 12 and 23 [1]; $X_1 \perp X_3 \mid X_2$ [1]. Partial correlations are $+0.2$ and -0.3, respectively, using the negative standardised precision entries [2]. The path through X_2 can give marginal association: in this example $(K^{-1})_{13} = -0.06/0.87 \neq 0$ [2].

(b) A typical objective is $\log\det K - \operatorname{tr}(SK) - \lambda \sum_{i \neq j} |K_{ij}|$ over $K \succ 0$ [1]. The penalty regularises and can create exact off-diagonal zeros, trading fit for a sparser graph [1]. Choose λ by a stated criterion such as EBIC or held-out likelihood, with selection performed within the intended assessment procedure [1]. An edge records conditional association under the fitted Gaussian model, not the direction or effect of intervention; omitted variables, sampling error and model misspecification remain possible [2].

References

Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19:716–723.

Alon, U., Barkai, N., Notterman, D., Gish, K., Ybarra, S., Mack, D., and Levine, A. (1999). Broad patterns of gene expression revealed by clustering analysis of tumor and normal colon tissues probed by oligonucleotide arrays. *Proceedings of the National Academy of Sciences of the USA*, 96(12):6745–6750.

Anderson, T. (2003). *An Introduction to Multivariate Statistical Analysis*. Wiley, 3rd edition.

Andrews, D. F. (1972). Plots of high-dimensional data. *Biometrics*, pages 125–136.

Barber, C. B., Dobkin, D. P., and Huhdanpaa, H. (1996). The quickhull algorithm for convex hulls. *ACM Transactions on Mathematical Software*, 22(4):469–483.

Benjamini, Y. and Hochberg, Y. (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B*, pages 289–300.

Benjamini, Y. and Yekutieli, D. (2001). The control of the false discovery rate in multiple testing under dependency. *The Annals of Statistics*, 29(4):1165–1188. https://doi.org/10.1214/aos/1013699998.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Boser, B. E., Guyon, I. M., and Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. In *Proceedings of the Fifth Annual Workshop on Computational Learning Theory*, pages 144–152. ACM.

Boyd, S. and Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

Breiman, L., Friedman, J., Olshen, R., and Stone, C. (1984). *Classification and Regression Trees*. Wadsworth, Belmont, CA.

Breiman, L. (1995). Better subset regression using the nonnegative garrote. *Technometrics*, 37(4):373–384.

Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2):123–140.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

Bridle, J. S. (1990). Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition. In *Neurocomputing*, pages 227–236. Springer.

Burges, C. J. C. (1998). A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2:121–167. https://doi.org/10.1023/A:1009715923555.

Burman, P. (1989). A comparative study of ordinary cross-validation, v-fold cross-validation and the repeated learning-testing methods. *Biometrika*, 76:503–514.

Casella, G. and Berger, R. (2002). *Statistical Inference*. Duxbury, Pacific Grove, CA, 2nd edition.

Cawley, G. C. and Talbot, N. L. C. (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107. https://www.jmlr.org/papers/v11/cawley10a.html.

Chacón, J. and Duong, T. (2011). Unconstrained pilot selectors for smoothed cross-validation. *Australian & New Zealand Journal of Statistics*, 53(3):331–351.

Chacón, J. and Duong, T. (2018). *Multivariate Kernel Smoothing and Its Applications*. CRC Press, Boca Raton, Florida.

Chicco, D. and Jurman, G. (2020). *Heart Failure Clinical Records*. UCI Machine Learning Repository. Variable definitions and dataset provenance. https://doi.org/10.24432/C5Z89R. Accessed 13 October 2026.

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314.

Dasarathy, B. and Sheela, B. (1979). A composite classifier system design: Concepts and methodology. *Proceedings of the IEEE*, 67(5):708–713.

Dempster, A., Laird, N., and Rubin, D. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society B*, 39(1):1–38.

Dempster, A. P. (1958). A high dimensional two sample significance test. *The Annals of Mathematical Statistics*, pages 995–1010.

Devroye, L., Györfi, L., and Lugosi, G. (1996). *A Probabilistic Theory of Pattern Recognition*. Springer-Verlag, New York.

Devroye, L. (1978). A universal k-nearest neighbor procedure in discrimination. In *Proceedings of the 1978 IEEE Computer Society Conference on Pattern Recognition and Image Processing*, pages 142–147, Long Beach, CA. IEEE Press.

Donoho, D. L. and Johnstone, I. M. (1994). Ideal spatial adaptation by wavelet shrinkage. *Biometrika*, 81(3):425–455.

Drton, M. and Plummer, M. (2017). A Bayesian information criterion for singular models. *Journal of the Royal Statistical Society, Series B*, 79(2):323–380. https://doi.org/10.1111/rssb.12187. Open manuscript: https://arxiv.org/abs/1309.0911.

Duda, R., Hart, P., and Stork, D. (2001). *Pattern Classification*. Wiley, New York.

Duong, T. (2007). ks: Kernel density estimation and kernel discriminant analysis for multivariate data in R. *Journal of Statistical Software*, 21(7):1–16.

Efron, B. and Hastie, T. (2016). *Computer Age Statistical Inference*. Cambridge University Press, Cambridge, UK.

Efron, B. and Tibshirani, R. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.

Efron, B. (1981). Nonparametric estimates of standard error: The jackknife, the bootstrap and other methods. *Biometrika*, 68:589–599.

Efron, B. (1983). Estimating the error rate of a prediction rule: Improvement on cross-validation. *Journal of the American Statistical Association*, 78(382):316–331.

Fan, J., Li, R., Zhang, C.-H., and Zou, H. (2020). *Statistical Foundations of Data Science*. Chapman and Hall/CRC.

Fisher, R. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188.

Foygel, R. and Drton, M. (2010). Extended Bayesian information criteria for Gaussian graphical models. In Lafferty, J., Williams, C., Shawe-Taylor, J., Zemel, R., and Culotta, A., editors, *Advances in Neural Information Processing Systems 23*, pages 604–612.

Fraley, C. and Raftery, A. (2002). Model-based clustering, discriminant analysis, and density estimation. *Journal of the American Statistical Association*, 97:611–631.

Frank, I. E. and Friedman, J. H. (1993). A statistical view of some chemometrics regression tools. *Technometrics*, 35(2):109–135.

Freund, Y. and Schapire, R. E. (1995). A decision-theoretic generalization of on-line learning and

an application to boosting. In Vitányi, P., editor, *Computational Learning Theory*, pages 23–37, Berlin, Heidelberg. Springer.

Friedman, J., Hastie, T., and Tibshirani, R. (2000). Additive logistic regression: a statistical view of boosting (with discussion and a rejoinder by the authors). *The Annals of Statistics*, 28(2):337–407.

Friedman, J., Hastie, T. and Tibshirani, R. (2008). Sparse inverse covariance estimation with the graphical lasso. *Biostatistics*, 9(3):432–441. https://doi.org/10.1093/biostatistics/kxm045.

Friedman, J., Hastie, T. and Tibshirani, R. (2026). *glasso: Graphical Lasso: Estimation of Gaussian Graphical Models*. R package reference manual. https://cran.r-project.org/web/packages/glasso/glasso.pdf. Accessed 13 October 2026; installed version recorded in the computational appendix.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232.

Fukunaga, K. (1990). *Introduction to Statistical Pattern Recognition*. Academic Press, San Diego.

Geisser, S. (1975). The predictive sample reuse method with applications. *Journal of the American Statistical Association*, 70:320–328.

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.

Graybill, F. (2001). *Matrices with Applications in Statistics*. Duxbury, Belmont, CA, 2nd edition.

Hastie, T. and Tibshirani, R. (1996). Discriminant analysis by Gaussian mixtures. *Journal of the Royal Statistical Society B*, 58:155–176.

Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning*. Springer, New York, 2nd edition.

Hastie, T., Tibshirani, R., and Wainwright, M. (2015). *Statistical Learning with Sparsity: The Lasso and Generalizations*. Chapman & Hall/CRC.

Hastie, T., Qian, J. and Tay, K. (2026). *An Introduction to glmnet*. Official package vignette. https://glmnet.stanford.edu/articles/glmnet.html. Accessed 13 October 2026.

Haykin, S. (1999). *Neural Networks: A Comprehensive Foundation*. Pearson, 2nd edition.

Hinton, G. E., Osindero, S. and Teh, Y.-W. (2006). A fast learning algorithm for deep belief nets. *Neural Computation*, 18(7):1527–1554. https://doi.org/10.1162/neco.2006.18.7.1527.

Ho, T. K. (1995). Random decision forests. In *Proceedings of the 3rd International Conference on Document Analysis and Recognition*, volume 1, pages 278–282.

Ho, T. K. (1998). The random subspace method for constructing decision forests. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(8):832–844.

Hoerl, A. and Kennard, R. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67.

Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366.

Härdle, W. and Simar, L. (2019). *Applied Multivariate Statistical Analysis*. Springer, Berlin, 5th edition.

Ito, K. and Schull, W. J. (1964). On the robustness of the t_0^2 test in multivariate analysis of variance when variance-covariance matrices are not equal. *Biometrika*, 51(1-2):71–82.

Jain, A. K. and Dubes, R. C. (1988). *Algorithms for Clustering Data*. Prentice-Hall, Englewood Cliffs, NJ.

Joachims, T. (1999). Making large-scale SVM learning practical. In Schölkopf, B., Burges, C., and Smola, A., editors, *Advances in Kernel Methods: Support Vector Learning*. MIT Press.

Kass, R. and Raftery, A. (1995). Bayes factors. *Journal of the American Statistical Association*, 90:773–795.

Keener, R. W. (2010). *Theoretical Statistics: Topics for a Core Course*. Springer, New York.

Krishnamoorthy, K. and Yu, J. (2004). Modified Nel and Van der Merwe test for the multivariate Behrens–Fisher problem. *Statistics & Probability Letters*, 66(2):161–169.

Krogh, A. and Hertz, J. (1992). A simple weight decay can improve generalization. In *Advances in Neural Information Processing Systems 4*, pages 950–957.

Kuncheva, L. (2014). *Combining Pattern Classifiers*. Wiley, Hoboken, NJ, 2nd edition.

L'Ecuyer, P. and Simard, R. (2007). TestU01: A C library for empirical testing of random number generators. *ACM Transactions on Mathematical Software*, 33:1–40.

Lachenbruch, P. and Mickey, M. (1968). Estimation of error rates in discriminant analysis. *Technometrics*, 10:1–11.

Lange, K. (2013). *Optimization*. Springer, New York, 2nd edition.

Lauritzen, S. (1996). *Graphical Models*. Clarendon Press.

LeCun, Y., Bottou, L., Orr, G. B., and Müller, K.-R. (1998). Efficient backprop. In *Neural Networks: Tricks of the Trade*, pages 9–50. Springer.

LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324. https://doi.org/10.1109/5.726791.

Lehmann, E. and Casella, G. (1998). *Theory of Point Estimation*. Springer, New York, 2nd edition.

Liaw, A. and Wiener, M. (2026). *randomForest: Breiman and Cutler's Random Forests for Classification and Regression*. R package reference manual, entries `importance` and `randomForest`. https://cran.r-project.org/web/packages/randomForest/randomForest.pdf. Accessed 13 October 2026.

Mason, L., Baxter, J., Bartlett, P., and Frean, M. (1999). Boosting algorithms as gradient descent. In Solla, S., Leen, T., and Müller, K., editors, *Advances in Neural Information Processing Systems*, volume 12, pages 512–518. MIT Press.

McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5:115–133.

McLachlan, G. and Gordon, R. (1989). Mixture models for partially unclassified data: a case study of renal venous renin levels in essential hypertension. *Statistics in Medicine*, 8:1291–1300.

McLachlan, G. and Peel, D. (2000). *Finite Mixture Models*. Wiley, New York.

McLachlan, G. J. (1992). *Discriminant Analysis and Statistical Pattern Recognition*. Wiley, New York.

mclust developers (2026). *bic: BIC for Parameterized Gaussian Mixture Models*. Official mclust reference. https://mclust-org.github.io/mclust/reference/bic.html. Accessed 13 October 2026; the installed function is checked in the release audit.

Millar, R. (2011). *Maximum Likelihood Estimation and Inference*. Wiley, Chichester, UK.

Minsky, M. and Papert, S. (1969). *Perceptrons*. MIT Press.

Mood, A., Graybill, F., and Boes, D. (1974). *Introduction to the Theory of Statistics*. McGraw-Hill, 3rd edition.

Morrison, D. F. (2005). *Multivariate Statistical Methods*. Duxbury, Belmont, CA, 4th edition.

Nguyen, H. and Wood, I. (2012). Variable selection in statistical models using population-based incremental learning with applications to genome-wide association studies. In *Proceedings of the IEEE Congress on Evolutionary Computation*, pages 1841–1848.

Olshen, R. (1977). Comments on a paper by C. J. Stone. *Annals of Statistics*, 5:632–633.

Pawlicki, T. F. et al. (1988). Neural network models and their application to handwritten digit recognition. In *IEEE 1988 International Conference on Neural Networks*. IEEE Press.

Perales-González, C., Fernández-Navarro, F., Carbonero-Ruz, M., and Pérez-Rodríguez, J. (2022). Global negative correlation learning: A unified framework for global optimization of ensemble models. *IEEE Transactions on Neural Networks and Learning Systems*, 33(8):4031–4042.

Petersen, K. and Pedersen, M. (2012). *The Matrix Cookbook*. Technical University of Denmark, https://www2.compute.dtu.dk/pubdb/pubs/3274-full.html.

Platt, J. (1999). Fast training of support vector machines using sequential minimal optimization. In Schölkopf, B., Burges, C., and Smola, A., editors, *Advances in Kernel Methods: Support Vector Learning*. MIT Press.

Pomerleau, D. A. (1989). ALVINN, an autonomous land vehicle in a neural network. Technical Report CMU-CS-89-107, Carnegie Mellon University.

Quenouille, M. (1949). Problems in plane sampling. *The Annals of Mathematical Statistics*, 20:355–375.

Quenouille, M. (1956). Notes on bias in estimation. *Biometrika*, 43:353–360.

Quinlan, J. (1993). *C4.5: Programs for Machine Learning*. Morgan Kaufmann, San Mateo, CA.

R Core Team (2026a). *prcomp: Principal Components Analysis*. Official R stats documentation. https://stat.ethz.ch/R-manual/R-devel/library/stats/html/prcomp.html. Accessed 13 October 2026.

R Core Team (2026b). *kmeans: K-Means Clustering*. Official R stats documentation. https://stat.ethz.ch/R-manual/R-devel/library/stats/html/kmeans.html. Accessed 13 October 2026.

R Core Team (2026c). *hclust: Hierarchical Clustering*. Official R stats documentation. https://stat.ethz.ch/R-manual/R-devel/library/stats/html/hclust.html. Accessed 13 October 2026.

R Core Team (2026d). *iris: Edgar Anderson's Iris Data*. Official R datasets documentation. https://stat.ethz.ch/R-manual/R-devel/library/datasets/html/iris.html. Accessed 13 October 2026.

Ripley, B. D. (1996). *Pattern Recognition and Neural Networks*. Cambridge University Press.

Roeder, K. and Wasserman, L. (1997). Practical Bayesian density estimation using mixtures of normals. *Journal of the American Statistical Association*, 92(439):894–902. https://doi.org/10.1080/01621459.1997.10474044.

Rokach, L. (2019). *Ensemble Learning*. World Scientific, Singapore, 2nd edition.

Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408.

Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323:533–536.

Schapire, R. E. and Freund, Y. (2012). *Boosting: Foundations and Algorithms*. The MIT Press.

Schapire, R. E. (1990). The strength of weak learnability. *Machine Learning*, 5(2):197–227.

Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464.

Schölkopf, B., Smola, A., and Müller, K.-R. (1999). Kernel principal component analysis. In Schölkopf, B., Burges, C., and Smola, A., editors, *Advances in Kernel Methods: Support Vector Learning*. MIT Press.

Sejnowski, T. J. and Rosenberg, C. R. (1986). NETtalk: A parallel network that learns to read aloud. Technical Report JHU/EECS-86/01, The Johns Hopkins University Electrical Engineering and Computer Science.

Smyth, P. (2000). Model selection for probabilistic clustering using cross-validated likelihood. *Statistics and Computing*, 10(1):63–72. https://doi.org/10.1023/A:1008940618127.

Speed, T. P. and Kiiveri, H. T. (1986). Gaussian Markov distributions over finite graphs. *The Annals of Statistics*, 14:138–150.

Stone, M. (1974). Cross-validatory choice and assessment of statistical predictions. *Journal of the*

Royal Statistical Society, B36:111–133.

Sun, W. and Cai, T. (2009). Large-scale multiple testing under dependence. *Journal of the Royal Statistical Society: Series B*, 71(2):393–424.

Therneau, T. M. and Atkinson, E. J. (2023). *An Introduction to Recursive Partitioning Using the RPART Routines*. Package technical report, Sections 3.1, 5.2 and 6.1. https://stat.ethz.ch/R-manual/R-devel/library/rpart/doc/longintro.pdf.

Thornton, C. (1993). Heart-disease example as cited on the lecture slide "Learning to diagnose heart disease"; the slides do not give a fuller reference.

Tibshirani, R., Walther, G., and Hastie, T. (2001). Estimating the number of clusters in a data set via the gap statistic. *Journal of the Royal Statistical Society B*, 63:411–423.

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58:267–288.

Tukey, J. (1958). Bias and confidence in not quite large samples. *The Annals of Mathematical Statistics*, 29:614.

Tukey, J. (1977). *Exploratory Data Analysis*. Addison-Wesley, Reading, Mass.

Vapnik, V. et al. (1997). Support vector regression, as cited on the lecture slides ("SVM techniques have been extended to regression"); the slides do not give a fuller reference.

Venables, W. N. and Ripley, B. D. (2026). *knn: k-Nearest Neighbour Classification*. Official R class package documentation. https://stat.ethz.ch/R-manual/R-devel/library/class/html/knn.html. Accessed 13 October 2026.

Wand, M. and Jones, M. (1995). *Kernel Smoothing*. Chapman and Hall, London.

Whittaker, J. (1990). *Graphical Models in Applied Multivariate Statistics*. Wiley.

Widrow, B. and Hoff, M. E. (1960). Adaptive switching circuits. In *1960 IRE WESCON Convention Record*, pages 96–104.

Williams, B. G., Rogers, D. J., Staton, G., Ripley, B., and Booth, T. H. (1994). Statistical modelling of georeferenced data: mapping tsetse distributions in Zimbabwe using climate and vegetation data. In Perry, B. D. and Hansen, J. W., editors, *Modelling Vector-borne and other Parasitic Diseases*. ILRAD, Nairobi.

Wishart, J. (1928). The generalised product moment distribution in samples from a normal multivariate population. *Biometrika*, 20:32–52.

Wu, C. F. J. (1983). On the convergence properties of the EM algorithm. *Annals of Statistics*, 11:95–103.

Zehna, P. (1966). Invariance of maximum likelihood estimation. *Annals of Mathematical Statistics*, 37:744.

Zhang, F. (2005). *The Schur Complement and its Applications*. Springer, New York.

Zou, H. (2006). The adaptive lasso and its oracle properties. *Journal of the American Statistical Association*, 101(476):1418–1429.

Appendix A

Mathematical background

These appendices supply the mathematical tools that the chapters repeatedly use. Return to them when a proof needs a matrix square root, a likelihood argument or a decomposition of variation. Each tool has a domain of validity; keeping that domain visible is more useful than memorising the displayed identity alone.

A.1 Maximum likelihood estimation

Let $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$ be observations from a proposed statistical model with density or mass function $f(\boldsymbol{x}; \boldsymbol{\theta})$. Estimation chooses a parameter value from the data. Before calling an estimate "best' ', we need a criterion: squared estimation error, predictive loss and likelihood are different criteria and need not select the same answer.

Maximum likelihood chooses the parameter value that maximises the joint density or mass at the observed data. The *estimator* is the rule applied to a random sample; an *estimate* is its realised value. The optimisation may have no finite maximiser or more than one maximiser, so the notation arg max does not establish existence or uniqueness.

Independence determines whether the joint density factorises. For iid observations it is $\prod_i f(\boldsymbol{x}_i; \boldsymbol{\theta})$; dependent observations require their joint density instead. After observing the sample, hold the data fixed and regard that expression as a function of $\boldsymbol{\theta}$: this is the likelihood. For continuous observations it is a density value, not the probability of observing those exact points, and it need not integrate to one over parameter space.

$$L(\boldsymbol{\theta}; \boldsymbol{X}_{\text{all}}) = \prod_{i=1}^{n} f(\boldsymbol{x}_i; \boldsymbol{\theta}), \qquad \widehat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} L(\boldsymbol{\theta}; \boldsymbol{X}_{\text{all}}). \tag{A.1}$$

Products of many numbers are computationally awkward, but maximisation is invariant to monotonic increasing transformations, so we maximise the log-likelihood (base e):

$$\widehat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \log L(\boldsymbol{\theta}; \boldsymbol{X}_{\text{all}}) = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{n} \log f(\boldsymbol{x}_i; \boldsymbol{\theta}). \tag{A.2}$$

What regular likelihood theory does and does not give

Under identifiability, correct specification, suitable smoothness and information conditions, and an interior true parameter, regular likelihood theory often gives consistency and

$$\sqrt{n}(\widehat{\boldsymbol{\theta}} - \boldsymbol{\theta}_0) \xrightarrow{d} \mathrm{N}\Big(\mathbf{0}, \boldsymbol{I}(\boldsymbol{\theta}_0)^{-1}\Big),$$

where $\boldsymbol{I}$ is the per-observation Fisher information. The inverse information is the asymptotic efficiency benchmark in the appropriate regular class of estimators. It is not a claim of minimum finite-sample MSE among every consistent estimator, nor a guarantee for singular mixtures, boundary parameters or increasing dimension (Lehmann and Casella, 1998; Millar, 2011; Drton and Plummer, 2017).

Invariance is a separate property: a function of an MLE maximises the induced profile likelihood for that transformed parameter, when the original maximum exists. It does not require the transformation to be one-to-one, but the transformed likelihood must be defined through the appropriate preimages (Lemma 2.1). Numerical optimisation adds another layer: a returned point must still be checked against constraints and stopping criteria.

A.2 Spectral decomposition

A real symmetric matrix can be understood direction by direction. Choose an orthonormal basis of eigenvectors, express a vector in that basis, and the matrix acts by multiplying coordinate j by its eigenvalue λ_j. In matrix form,

$$\boldsymbol{A} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^\top, \qquad \boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_p), \qquad \boldsymbol{\Gamma} = (\boldsymbol{\gamma}_1 \ \cdots \ \boldsymbol{\gamma}_p), \tag{A.3}$$

where λ_j is the jth eigenvalue of $\boldsymbol{A}$ and $\boldsymbol{\gamma}_j$ the corresponding eigenvector. The matrix $\boldsymbol{\Gamma}$ is orthogonal: its rows and columns are orthonormal vectors, so $\boldsymbol{\Gamma}\boldsymbol{\Gamma}^\top = \boldsymbol{I}_p = \boldsymbol{\Gamma}^\top\boldsymbol{\Gamma}$. The matrix is positive semi-definite if and only if all $\lambda_j \geq 0$, and positive definite if and only if all $\lambda_j > 0$ (so that $\boldsymbol{z}^\top\boldsymbol{A}\boldsymbol{z} = \sum_j \lambda_j(\boldsymbol{\gamma}_j^\top\boldsymbol{z})^2$).

For a positive-definite matrix, real powers are defined by taking the corresponding powers of its positive eigenvalues. For a positive-semidefinite matrix, positive powers are also defined with $0^\alpha = 0$ for $\alpha > 0$; negative powers require removing or regularising zero eigenvalues. Arbitrary real powers of an indefinite real matrix need not remain real. In the positive-definite case,

$$\boldsymbol{A}^\alpha = \boldsymbol{\Gamma}\boldsymbol{\Lambda}^\alpha\boldsymbol{\Gamma}^\top. \tag{A.4}$$

In particular $\boldsymbol{A}^{1/2} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}^{1/2}\boldsymbol{\Gamma}^\top$, and

$$\boldsymbol{A}^{1/2}\boldsymbol{A}^{1/2} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}^{1/2}\boldsymbol{\Gamma}^\top\boldsymbol{\Gamma}\boldsymbol{\Lambda}^{1/2}\boldsymbol{\Gamma}^\top = \boldsymbol{\Gamma}\boldsymbol{\Lambda}^{1/2}\boldsymbol{\Lambda}^{1/2}\boldsymbol{\Gamma}^\top = \boldsymbol{\Gamma}\boldsymbol{\Lambda}\boldsymbol{\Gamma}^\top = \boldsymbol{A}.$$

Also $\boldsymbol{A}^\alpha\boldsymbol{A}^{-\alpha} = \boldsymbol{\Gamma}\boldsymbol{\Lambda}^\alpha\boldsymbol{\Gamma}^\top\boldsymbol{\Gamma}\boldsymbol{\Lambda}^{-\alpha}\boldsymbol{\Gamma}^\top = \boldsymbol{\Gamma}\boldsymbol{\Lambda}^\alpha\boldsymbol{\Lambda}^{-\alpha}\boldsymbol{\Gamma}^\top = \boldsymbol{\Gamma}\boldsymbol{\Gamma}^\top = \boldsymbol{I}_p$.

A.3 Mahalanobis distances and transformation

Euclidean distance treats coordinate units as the metric. Mahalanobis distance instead measures displacement relative to a specified positive-definite covariance. A difference in a high-variance direction counts less than the same difference in a low-variance direction. It is a distance between vectors under that metric, not a general distance between probability distributions.

When random vectors $\boldsymbol{X}$ and $\boldsymbol{Y}$ have means $\boldsymbol{\mu}_X$ and $\boldsymbol{\mu}_Y$ and common covariance matrix $\boldsymbol{\Sigma}$, the Mahalanobis distance between the means is

$$\Delta(\boldsymbol{\mu}_X, \boldsymbol{\mu}_Y) = \sqrt{(\boldsymbol{\mu}_X - \boldsymbol{\mu}_Y)^\top\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_X - \boldsymbol{\mu}_Y)}. \tag{A.5}$$

With a sample of each and an estimated common covariance matrix $\boldsymbol{S}$, the Mahalanobis distance between $\overline{\boldsymbol{x}}$ and $\overline{\boldsymbol{y}}$ is $D(\overline{\boldsymbol{x}}, \overline{\boldsymbol{y}}) = \sqrt{(\overline{\boldsymbol{x}} - \overline{\boldsymbol{y}})^\top\boldsymbol{S}^{-1}(\overline{\boldsymbol{x}} - \overline{\boldsymbol{y}})}$. With one sample $\boldsymbol{X}_{\text{all}} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n)$ with sample covariance matrix $\boldsymbol{S}_{\boldsymbol{X}}$ and a single point $\boldsymbol{y}$, the distance between $\boldsymbol{y}$ and the sample is

$D(\overline{\boldsymbol{x}}, \boldsymbol{y}) = \sqrt{(\overline{\boldsymbol{x}} - \boldsymbol{y})^\top \boldsymbol{S}_{\boldsymbol{X}}^{-1} (\overline{\boldsymbol{x}} - \boldsymbol{y})}$, and the distance between a point $\boldsymbol{y}$ and an observation $\boldsymbol{x}_i$ using $\boldsymbol{S}_{\boldsymbol{X}}$ is $D(\boldsymbol{x}_i, \boldsymbol{y}) = \sqrt{(\boldsymbol{x}_i - \boldsymbol{y})^\top \boldsymbol{S}_{\boldsymbol{X}}^{-1} (\boldsymbol{x}_i - \boldsymbol{y})}$.

These distances suggest a corresponding transformation so that the transformed data set or distribution has identity covariance matrix. The *Mahalanobis transformation* of a data set $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n)$ is

$$\boldsymbol{z}_i = \boldsymbol{S}^{-1/2}(\boldsymbol{x}_i - \overline{\boldsymbol{x}}), \qquad i = 1, \ldots, n, \tag{A.6}$$

Here $\boldsymbol{S}$ must be positive definite and its square root is constructed spectrally. With the same covariance divisor before and after transformation,

$$\boldsymbol{S}_z = \boldsymbol{S}^{-1/2} \boldsymbol{S} \boldsymbol{S}^{-1/2} = \boldsymbol{I}_p.$$

This is empirical whitening. It produces zero sample means and identity sample covariance, not Gaussian observations, independence or spherical density contours. A population transformation using $\boldsymbol{\Sigma}^{-1/2}$ likewise standardises second moments, with stronger distributional conclusions only under extra assumptions such as joint Gaussianity. The distance identity is purely algebraic:

$$\|\boldsymbol{z}_i - \boldsymbol{z}_l\|^2 = (\boldsymbol{x}_i - \boldsymbol{x}_l)^\top \boldsymbol{S}^{-1} (\boldsymbol{x}_i - \boldsymbol{x}_l).$$

A.4 Convex hulls

A set is convex if it contains the line segment joining every pair of its points. A solid ball and a solid cube are convex; the surface of a sphere is not. The convex hull is the smallest convex set containing the original set, equivalently the set of all finite convex combinations of its points.

For a finite sample, the convex hull is a multivariate description of its observed geometric range. It lies inside the coordinatewise bounding box, often much more tightly. A new point outside it is an extrapolation relative to that sample geometry, but is not automatically a statistical outlier: in high dimension, many ordinary future observations can lie outside a small sample's hull. Convex-hull algorithms include Quickhull (Barber et al., 1996). A cluster mean with positive normalised weights is a convex combination, so it must lie inside the hull of the contributing observations.

A.5 The ANOVA identity

Add and subtract each group mean inside the deviation from the grand mean. The resulting cross term sums to zero within a group, leaving a within-group part and a between-group part. For a groups with r observations in each group, this gives

$$\begin{aligned}\sum_{i=1}^{a}\sum_{j=1}^{r}(y_{ij} - \bar{y}_{\cdot\cdot})^2 &= \sum_{i=1}^{a}\sum_{j=1}^{r}\left[(y_{ij} - \bar{y}_{i\cdot}) + (\bar{y}_{i\cdot} - \bar{y}_{\cdot\cdot})\right]^2 \\ &= \sum_{i=1}^{a}\sum_{j=1}^{r}(y_{ij} - \bar{y}_{i\cdot})^2 + 2\sum_{i=1}^{a}\sum_{j=1}^{r}(y_{ij} - \bar{y}_{i\cdot})(\bar{y}_{i\cdot} - \bar{y}_{\cdot\cdot}) + r\sum_{i=1}^{a}(\bar{y}_{i\cdot} - \bar{y}_{\cdot\cdot})^2.\end{aligned}$$

The cross-product term is zero, because $\sum_{j=1}^{r}(y_{ij} - \bar{y}_{i\cdot}) = y_{i\cdot} - r\bar{y}_{i\cdot} = 0$ for each group i. Therefore

$$\sum_{i=1}^{a}\sum_{j=1}^{r}(y_{ij}-\bar{y}_{\cdot\cdot})^2 = r\sum_{i=1}^{a}(\bar{y}_{i\cdot}-\bar{y}_{\cdot\cdot})^2 + \sum_{i=1}^{a}\sum_{j=1}^{r}(y_{ij}-\bar{y}_{i\cdot})^2, \tag{A.7}$$

that is, total sum of squares = between-group sum of squares + within-group sum of squares. This is the identity behind the pooled covariance matrix (Chapter 2), the within-cluster scatter of K-means (Chapter 4), and the between/within ratio of Fisher's discriminant (Chapter 8).

Appendix B

R code from the notes

The following templates retain the historical analyses from the course appendices, with explanatory comments and a corrected BH empty-rejection case. They are reference code, not executed outputs of this edition. The nearest-neighbour chapter provides a separate executable illustration; the Alon template requires the absent processed file `alon.rda`. Reproducing its historical numbers would also require the original preprocessing and software conventions.

B.1 Nearest-neighbour example (Chapter 11)

R template. Read the code as a proposed analysis requiring its stated inputs. It is not run during rendering.

```
library(MASS)
scale  = 10
offset = 3.5
x  <- seq(1, 10)
c1 <- cbind(x, x)                    # class 1: points (x, x)
c2 <- cbind(x + offset, x)           # class 2: points (x + 3.5, x)
call <- rbind(c1, c2)
eqscplot(call, pch = '', xlab = "x1", ylab = "x2")   # equal-scale axes
points(c1)
points(c2, pch = 'x')
cs1 <- cbind(x * scale, x)                    # multiply x1 by 10
cs2 <- cbind((x + offset) * scale, x)
calls <- rbind(cs1, cs2)
eqscplot(calls, pch = '', xlab = "x1", ylab = "x2")
points(cs1)
points(cs2, pch = 'x')

## Decision regions of the 1-nearest-neighbour classifier
library(class)
## scaled version
minx = min(cs1[, 1], cs2[, 1]);  maxx = max(cs1[, 1], cs2[, 1])
miny = min(cs1[, 2], cs2[, 2]) - 10;  maxy = max(cs1[, 2], cs2[, 2]) + 10
xs <- seq(minx, maxx);  ys <- seq(miny, maxy)
lenxs = length(xs);  lenys = length(ys)
classes = factor(c(rep(1, 10), rep(0, 10)))
z <- matrix(0, nrow = lenxs, ncol = lenys)
for (i in 1:lenxs) {
```

```
  for (j in 1:lenys) {
    z[i, j] = knn(calls, c(xs[i], ys[j]), cl = classes, k = 1)
  }
}
image(xs, ys, z, asp = 1)
points(cs1);  points(cs2, pch = 'x')

## unscaled version
minu = min(c1[, 1], c2[, 1]);  maxu = max(c1[, 1], c2[, 1])
minv = min(c1[, 2], c2[, 2]);  maxv = max(c1[, 2], c2[, 2])
us <- seq(minu, maxu, 0.02);  vs <- seq(minv, maxv, 0.02)
lenus = length(us);  lenvs = length(vs)
w <- matrix(0, nrow = lenus, ncol = lenvs)
for (i in 1:lenus) {
  for (j in 1:lenvs) {
    w[i, j] = knn(call, c(us[i], vs[j]), cl = classes, k = 1)
  }
}
image(us, vs, w, asp = 1)
points(c1);  points(c2, pch = 'x')
```

B.2 Analysis of the Alon colon-cancer data (Chapter 18)

R template. Read the code as a proposed analysis requiring its stated inputs. It is not run during rendering.

```
load("alon.rda")                    # x: 62 x 2000 expression matrix; y:
↪  tumour/normal labels
library(FactoMineR)
numpcs = 10                         # limit to 10 PC directions
res <- PCA(x, scale.unit = F, graph = F, ncp = numpcs)
plot(res$ind$coord[, 1], res$ind$coord[, 2], col = y, xlab = "PC1", ylab =
↪  "PC2")
legend("topleft", legend = c("Normal", "Tumour"), col = c("black", "red"), pch =
↪  c(1, 1))
# Welch's t-test comparing tumour and normal groups on each principal component
↪  (PC)
ttestres = vector("numeric", numpcs)
for (i in 1:10) {
  ttestres[i] = t.test(res$ind$coord[, i] ~ y)$p.value
  print(paste(i, signif(ttestres[i], digits = 3)))
}
plot(res$ind$coord[, 3], res$ind$coord[, 6], col = y, xlab = "PC3", ylab =
↪  "PC6")
legend("topleft", legend = c("Normal", "Tumour"), col = c("black", "red"), pch =
↪  c(1, 1))
# top 20 gene contributors, by absolute value, to the 3rd and 6th PC direction
↪  vectors
lst3 <- sort(abs(res$var$coord[, 3]), index.return = TRUE, decreasing = TRUE)
lst6 <- sort(abs(res$var$coord[, 6]), index.return = TRUE, decreasing = TRUE)
lst3ids = lapply(lst3, `[`, lst3$x %in% head(unique(lst3$x), 20))$ix
```

```r
lst6ids = lapply(lst6, `[`, lst6$x %in% head(unique(lst6$x), 20))$ix
intersect(lst3ids, lst6ids)       # any common indices (none in this case)

# original p-values, one t-test per gene (40 tumour, 22 normal)
orig_p <- rep(0, 2000)
for (i in c(1:2000)) {
  orig_p[i] <- t.test(x[1:40, i], x[41:62, i])$p.value
}
sorted <- sort(orig_p, index.return = TRUE)
sorted_p <- sorted$x
sorted_p_ix <- sorted$ix
# Benjamini-Hochberg (BH) method
thres <- 0.01
kk <- rep(0, 2000)
for (i in c(1:2000)) {
  if (sorted_p[i] <= thres * i / 2000) {
    kk[i] <- i
  }
}
cut_off_k <- max(kk)              # zero if no BH inequality succeeds
significant_genes <- sorted_p_ix[seq_len(cut_off_k)]
sg_ordered <- sort(significant_genes)
# log-log plot of gene order against unadjusted p-value
order_genes_relabel <- c(1:2000)
significant <- c(rep("red", cut_off_k), rep("blue", 2000 - cut_off_k))
plot(order_genes_relabel, sorted_p, log = "xy",
     xlab = "gene number ordered by p-value", ylab = "unadjusted p-value",
     main = "Log-log plot genes vs. p-values with BH threshold & 0.01 FDR
     ↪ control",
     col = significant)
abline(v = cut_off_k, lty = 2)    # cut-off for p-values
abline(a = 0, b = thres / 2000, untf = T)   # FDR control line
# genes in common with the PCA results
intersect(significant_genes, lst3ids)
intersect(significant_genes, lst6ids)
# Lasso
library(glmnet)
# cross-validation to select lambda (default cost: binomial deviance), lasso is
↪ alpha = 1
cv.lasso <- cv.glmnet(x, y, alpha = 1, family = "binomial")
plot(cv.lasso)                    # CV curve with lambda.min and lambda.1se
↪ (more conservative) marked
lambda <- cv.lasso$lambda.min
lasso.model <- glmnet(x, y, alpha = 1, family = "binomial", lambda = lambda)
beta_vals <- as.matrix(lasso.model$beta)      # matrix of coefficients
selected_genes <- which(beta_vals != 0)
selected_betas <- beta_vals[selected_genes]
selected_genes
selected_betas
beta_0 <- lasso.model$a0          # intercept (beta_0 term)
intersect(significant_genes, selected_genes)
```

Appendix C

Reproducibility and computational checks

Rendering executes the 19 chapter laboratories, the R figures and the computational checks included below. The edited Qmd files are the authoritative source. Migration and one-time editorial scripts record earlier transformations; rerunning them is neither necessary nor appropriate for an ordinary rebuild.

C.1 What is checked

`R/practice_checks.R` runs during rendering and verifies conditional-normal calculations, BIC values, a finite-difference gradient, XOR constructions, precision-matrix inverses, classification errors and the distinct bounds on the rank of a sample covariance and of a pooled within-class covariance. Three solution diagrams are drawn in R.

The numerical checks verify selected invariants that can fail in useful ways: PCA score covariance matches its eigenvalues; the k-means objective equals the direct sum of squared residuals; membership and softmax probabilities sum to one; the BIC conventions differ only by a sign; explicit EM log likelihood is monotone; neural-network gradients agree with finite differences; and the specified Gaussian chain has the expected precision zeros.

The checks test particular calculations and invariants. They do not prove every theorem, establish distributional assumptions or reproduce analyses whose data are absent. The worked solutions have been reviewed alongside the revised chapters, with substantive changes recorded in the correction log. The current release report distinguishes source checks, executed calculations and visual inspection of the compiled pages.

C.2 Rebuild instructions

Install R, Quarto and a LaTeX distribution. Run `Rscript R/install_packages.R` once from the project folder if packages are missing, then run `quarto render`. The installer uses the project-local `R-library` folder. `R/figures.R` contains all figure definitions; `R/render_figures.R` can regenerate an independent PDF gallery. The machine-readable `figure-manifest.json` maps each original figure to its new R function and chapter.

Random seeds are fixed in each experiment. Exact results can still change with R or package versions, numerical libraries and tie-breaking implementations; the following record identifies the environment used for this render.

Package	Version
knitr	1.43
rmarkdown	2.24
MASS	7.3.60
class	7.3.22
cluster	2.1.4
mclust	6.1.1
nnet	7.3.21
rpart	4.1.23
randomForest	4.7.1.2
e1071	1.7.16
glmnet	4.1.8
glasso	1.11

```
#>
#> R version: R version 4.3.1 (2023-06-16 ucrt)
```